

A Proposal for Functionality Classes for Random Number Generators

Version 3.0

Matthias Peter

Bundesamt für Sicherheit in der Informationstechnik (BSI)

Werner Schindler

Bundesamt für Sicherheit in der Informationstechnik (BSI)

September 10, 2024

Abstract

This document outlines evaluation criteria for deterministic and true random number generators and is an update to the mathematical-technical reference of both the AIS 20 (Funktionalitätsklassen und Evaluationsmethodologie für deterministische Zufallszahlengeneratoren) and the AIS 31 (Funktionalitätsklassen und Evaluationsmethodologie für physikalische Zufallszahlengeneratoren) which define the evaluation methodology for deterministic and true random number generators in the German Common Criteria certification scheme, respectively. The mathematical background is explained and the concepts are illustrated by numerous examples.

Acknowledgments

The authors would like to express their gratitude for the many comments, suggestions, and notes they received, many of which have been incorporated into this document.

Our special thanks to

Kyan Adib-Ghiassi (SRC Security Research & Consulting GmbH), Marco Bucci (Infineon Technologies AG), Markus Dichtl, Ignacio Dieguez (Entrust), Björn Fay (NXP Semiconductors Austria), Jonas Fiege (Bundesamt für Sicherheit in der Informationstechnik (BSI)), Viktor Fischer (Hubert Curien Laboratory, Jean Monnet University Saint-Étienne), Robert Freire-Stoegbuchner (Infineon Technologies AG), Adrian Gabriel (Bundesamt für Sicherheit in der Informationstechnik (BSI)), Berndt Gammel (Infineon Technologies AG), Aron Gohr, Gabriel Goller (Infineon Technologies AG), Rainer Göttfert (Infineon Technologies AG), Patrick Haddad (STMicroelectronics), Stefan Heiss (Infineon Technologies AG), Tobias Hemmert (Bundesamt für Sicherheit in der Informationstechnik (BSI)), John Kelsey (National Institute of Standards and Technology (NIST)), Wolfgang Krupp (SGS), Mario Lamberger (NXP Semiconductors Austria), Friederike Laus (Bundesamt für Sicherheit in der Informationstechnik (BSI)), Marc Le Guin (TÜVIT), Raimondo Luzzi (Infineon Technologies AG), Wolfgang Killmann, Kerry McKay (National Institute of Standards and Technology (NIST)), Florian Mendel (Infineon Technologies AG), Stephan Müller (atsec), Matthias Nagel (Rohde & Schwarz), Christoph Pacher (AIT Austrian Institute of Technology GmbH / fragmentiX Storage Solutions GmbH), Jean-René Reinhard (Agence nationale de la sécurité des systèmes d'information (ANSSI)), Gen'ya Sakurai, Ernst Schulte-Geers (Bundesamt für Sicherheit in der Informationstechnik (BSI)), Torsten Schütze (Rohde & Schwarz), Martin Steinbach (Infineon Technologies AG), Jan “Kuba” Tatarkiewicz (Randaemon Ltd.), Lars Tebelmann (Infineon Technologies AG), Yannick Tegli (Thales), Meltem Sönmez Turan (National Institute of Standards and Technology (NIST)), Aurélie Utralal (Giesecke & Devrient).

In particular, we would like to recognize the contributions of

Elaine Barker (National Institute of Standards and Technology (NIST)),
Johannes Mittmann (Bundesamt für Sicherheit in der Informationstechnik (BSI)).

Contents

1	Introduction	1
1.1	Foreword	1
1.2	Character of this Document	2
1.3	Structure of this Document	2
2	AIS 20 and AIS 31 — Scope, Limits, RNG Classes, and Concepts	4
2.1	Scope and Limitations of the AIS 20 and AIS 31	4
2.2	RNG Classification and Functionality Classes	5
2.3	Stochastic Models for PTRNGs	7
2.4	Other RNG Standards	9
3	Functionality Classes	11
3.1	Evaluation of an RNG: General Aspects	11
3.2	Overview of the Functionality Classes	12
3.3	DRNGs: Functionality Classes	15
3.3.1	DRG.[2,3,4], DRT.1: Definitions, Requirements, and Justification	16
3.3.2	Functionality Class DRG.2	34
3.3.3	Functionality Class DRG.3	39
3.3.4	Functionality Class DRG.4	42
3.3.5	Functionality Class DRT.1	46
3.4	PTRNGs: Functionality Classes	54
3.4.1	PTG.[2,3]: Definitions, Requirements, and Justification	56
3.4.2	Functionality Class PTG.2	61

3.4.3	Functionality Class PTG.3	72
3.5	NPTRNGs: Functionality Classes	90
3.5.1	NTG.1: Definitions, Requirements, and Justification	90
3.5.2	Functionality Class NTG.1	93
3.6	Cross-class Topics	97
4	Mathematical Background	100
4.1	Randomness and Random Experiments	100
4.2	Probability, Stochastics, and Random Variables	101
4.2.1	Definitions and Basic Concepts	101
4.2.2	Useful Theorems and Facts	112
4.3	Entropy and Guesswork	123
4.3.1	Entropy	124
4.3.2	Guesswork and Work Factor	136
4.4	Random Mappings	139
4.4.1	Iteration of Random Mappings: Statistical Properties	139
4.4.2	Impact on the Work Factor, on Shannon entropy, and min-entropy	140
4.5	Stochastic Model, Online Test, Total Failure Test, and Start-up Test	159
4.5.1	Stochastic Model: Motivation and Definition	160
4.5.2	Example: Stochastic Model for Coin Tossing	166
4.5.3	Online Test	168
4.5.4	Total Failure Test	174
4.5.5	Start-up Test	176

4.6	Statistical Tests and Predictors	176
4.6.1	General Considerations	177
4.6.2	Verification of the Stochastic Model	181
4.6.3	Statistical Tests and Predictors	182
4.6.4	Black Box Test Suite T_{irr}	187
5	Examples	189
5.1	Examples of Algorithmic Post-processing Algorithms	189
5.1.1	Fixed Compression Rate	190
5.1.2	Von Neumann Unbiasing	194
5.1.3	Thinning Out	194
5.1.4	Universal Families of Hash Functions	195
5.2	Evaluation of DRNGs: Miscellaneous Aspects	198
5.2.1	AES in OFB mode	198
5.2.2	Pure and Hybrid DRNGs and a (Too) Simple State Transition Function	200
5.2.3	One-way Functions Derived from the AES Block Cipher	204
5.3	NIST Approved Designs [SP800-90A]: Conformity Analysis with Regard to DRG.3 and DRG.4	204
5.3.1	Security Evaluation of the Hash_ DRBG [SP800-90A]	205
5.3.2	Security Evaluation of the HMAC_ DRBG [SP800-90A]	218
5.4	Noise Sources and Stochastic Models	224
5.4.1	Examples of Physical and Non-physical Noise Sources	225
5.4.2	PTRNG with Two Noisy Diodes	226
5.4.3	Sampling Events with iid Intermediate Time Intervals – Design A	236

5.4.4	Sampling Events with iid Intermediate Time Intervals – Design B	249
5.4.5	A PTRNG Design Exploiting Radioactive Decay	252
5.4.6	A PLL-based Physical Noise Source	255
5.5	Online Tests	260
5.5.1	A Look at Single Statistical Tests	261
5.5.2	A More Sophisticated Online Test Procedure	264
5.6	Linux /dev/random and /dev/urandom	272
Glossary		276
Acronyms		285
Abbreviations from Common Criteria		286
Symbols		287
References		289
6	Appendix	294
6.1	Main Differences from [AIS2031An_11]	294
6.1.1	DRNG: Main Differences from [AIS2031An_11]	294
6.1.2	PTRNG: Main Differences from [AIS2031An_11]	295
6.1.3	NPTRNG: Main Differences from [AIS2031An_11]	295
6.2	Further results on the work factor	296

1 Introduction

1.1 Foreword

- 1 Random numbers are required for most cryptographic applications. They are used to generate session keys, signature parameters, nonces, challenges, and blinding and masking values (in order to prevent implementation attacks) to name a few scenarios.
- 2 Weak **Random number generators (RNGs)** can significantly weaken cryptographic applications. This creates the need for reliable and trustworthy security evaluations of **RNGs**.
- 3 The **AIS 20** and the **AIS 31** have been used to specify how **RNGs** shall be evaluated in the German **Common Criteria (CC)** scheme for more than two decades. Above all, they define functionality classes for different types of **RNGs**. To be compliant with a particular functionality class, an **RNG** must fulfill all class-specific requirements. Furthermore, they outline an evaluation methodology for **Deterministic RNGs (DRNGs)** (**AIS 20**) and **True RNGs (TRNGs)** (**AIS 31**).
- 4 This document is the mathematical-technical reference for both the **AIS 20** as well as the **AIS 31** and is intended to be used by developers, evaluators, and certifiers.
Note: Depending on the context, this document itself is often loosely referred to as the **AIS 20**, **AIS 31**, or **AIS 20/31**. Henceforth, we follow this convention.
- 5 The first versions of this mathematical-technical reference were published in 1999 [**AIS20An_99**] and in 2001 [**AIS31An_01**] (mathematical-technical references for the **AIS 20** and the **AIS 31**, respectively) when **CC** was still new, and no guidelines for the evaluation of **RNGs** existed. Since then, their evaluation criteria have been field-tested and modernized accordingly. In 2011, the mathematical-technical references were updated and merged into [**AIS2031An_11**], which is the predecessor of this document. [**AIS2031An_11**] was then referenced in [**AIS20**] and [**AIS31**].
- 6 This document distinguishes between **DRNGs**, **Physical true RNGs (PTRNGs)**, and **Non-physical true RNGs (NPTRNGs)**. In Chapter 3, seven functionality classes are defined (DRG.2, DRG.3, DRG.4, DRT.1, PTG.2, PTG.3, and NTG.1). Each functionality class specifies the requirements that an **RNG** (in the case of functionality class DRT.1: a **DRNG tree**) must fulfill to be compliant with that class. Most of these functionality classes are hierarchically ordered according to the stringency of their requirements, and therefore, their security strength. The overall strongest class is PTG.3.
- 7 Compared to its predecessor [**AIS2031An_11**], two functionality classes have been removed, namely DRG.1 and PTG.1. The definitions of the remaining functionality classes, however, remain similar to those in [**AIS2031An_11**]. This justifies retaining their class names, even though they have been further developed and thus differ in detail. A new functionality class, DRT.1, has also been added. An in-depth explanation of the differences between the specifications given in [**AIS2031An_11**] and this document, can be found in the appendix, Subsections **6.1.1**, **6.1.2**, and **6.1.3**.
- 8 This document refers to Common Criteria CC:2022 Release 1, Part 1 to 5, [**CC2022R1**].

1.2 Character of this Document

The specifications of the functionality classes take into account that cryptographic applications that use **RNGs** have different security requirements and may run on devices with different resources and limitations. This document does not assign the functionality classes to particular cryptographic applications. The assignment of functionality classes is part of the security evaluation of the devices that consume random numbers. 9

Part 1 of the technical guideline TR 02102 [TR-02102] recommends cryptographic mechanisms, including appropriate functionality classes for the implemented **RNGs**. This and numerous other technical guidelines and **Protection Profiles (PPs)** issued by the **Bundesamt für Sicherheit in der Informationstechnik (BSI)** use the functionality classes defined in this document. 10

In addition to this document, the **BSI** also provides other documents on the security of **RNGs** (e.g., [Linux_RNG_2016], [Linux_RNG_2020], [Linux_RNG_2022], [Linux_RNG_2024], and [RNG_virtual_env]). They can be found [here](#). 11

This document does not contain a complete evaluation methodology (which would precisely describe the tasks of the developer, the evaluator, and the certifier). However, it specifies a list of deliverables (including security proofs) that an applicant for a certificate has to provide to the evaluator for each functionality class. The complete evaluation methodology is specified in several documents to which the **AIS 20** and the **AIS 31** refer. The advantage of this approach is that this document can easily be used within evaluation schemes other than the Common Criteria (**CC**). 12

This document makes no claims about the patent situation of mechanisms described in it. 13

1.3 Structure of this Document

This document consists of six chapters, including the appendix. 14

Chapter 1 describes the general context of this document. 15

Section 2.1 explains the scope and the limitations of this document. These explanations are relevant for the overall evaluation of the device in which the **RNG** is implemented. Sections 2.2 and 2.3 give a brief introduction and motivation for readers who are not yet familiar with the **AIS 20** and the **AIS 31**. Both sections may be skipped by readers familiar with these documents without loss of information. Finally, Section 2.4 briefly addresses other **RNG** standards. 16

Chapter 3 is the core of this document. The seven functionality classes which consist of **DRG.2**, **DRG.3**, **DRG.4 (DRNGs)**, **DRT.1 (DRNG trees)**, **PTG.2**, **PTG.3 (PTRNGs)** and **NTG.1 (NPTRNGs)** are defined in this chapter, and application notes on how to apply each particular requirement are also given in this chapter. Section 3.6 addresses cross-class aspects. Furthermore, in Chapter 3, the required background information is provided, definitions are introduced, and motivations for the specifications of the functionality classes are given. Chapter 3 refers to sections, subsec- 17

tions, paragraphs, and concrete examples of Chapter 4 and Chapter 5 at various places. These references may be normative or informative.

- 18 Chapter 4 provides central mathematical concepts that are important or at least helpful for evaluating RNGs according to the AIS 20 and the AIS 31. Chapter 4 also serves as a reference for different questions that commonly arise during the evaluation of an RNG. In Section 4.5, the concept of a stochastic model and the purpose of the online test, the total failure test, and the start-up test are explained in detail and illustrated by simple examples. To be compliant with functionality classes PTG.2 and PTG.3, PTRNGs need to apply online tests and total failure tests effectively, while the stochastic model is the core of a PTRNG evaluation. In Section 4.6, a statistical black box test suite that the evaluator has to apply to the internal random numbers of PTG.2-compliant PTRNGs is specified.
- 19 Chapter 5 illustrates the concepts of Chapter 4 with more complex examples. This may be useful for both the design and the evaluation of cryptographic post-processing and non-cryptographic post-processing algorithms, the evaluation of noise sources, and online tests. Exemplary verifications of the requirements of the functionality classes are provided to familiarize developers and evaluators with the subject matter. In Section 5.3, the conformance of two of the National Institute of Standards and Technology (NIST)-approved designs from [SP800-90A] with functionality classes DRG.3 and DRG.4 is analyzed. Developers (applicants for certificates) may refer to Section 5.3, which relieves them from having to develop the required security evidence themselves. Section 5.4 discusses several stochastic models exemplarily, and Section 5.5 considers online tests. Section 5.6 summarizes the results from a long-term study on the Linux RNGs, /dev/random and /dev/urandom, commissioned by the BSI. These results may be referenced by the developers.
- 20 A glossary, a list of acronyms, abbreviations from Common Criteria (CC), used symbols, the references, and the appendix (Chapter 6) conclude the document. The appendix summarizes the main differences of the functionality classes defined in this document with functionality classes in previous versions of AIS 20/31. Furthermore, it contains some results on random mappings that are not immediately required in this document.
- 21 **Important note:** It is NOT necessary to study the whole document in order to apply the AIS 20/31. The normative part, consisting of fundamental definitions, the specification of the functionality classes, and the application notes, is essentially contained in Chapter 3. Occasionally, references to Chapter 4 and Chapter 5 are given. The main purpose of Chapter 4 and Chapter 5 is to support the applicants for a certificate and the evaluators. Their contents are sketched in pars. 18 and 19. Section 2.1 explains the scope and the limits, and Section 2.4 briefly addresses other RNG standards. The other sections of Chapter 2 are intended to assist readers who are not yet familiar with the concepts of AIS 20/31.

2 AIS 20 and AIS 31 — Scope, Limits, RNG Classes, and Concepts

Chapter 2 is informative. Section 2.1 outlines the scope and the limitations of the AIS 20 and the AIS 31, while Sections 2.2 and 2.3 address the fundamental concepts of this document. In particular, the motivations for the classifications of RNGs are provided. Sections 2.2 and 2.3 are written for readers who are not yet or only slightly familiar with the AIS 20 and the AIS 31. These sections are intended to facilitate the introduction to the subject matter. We do not provide detailed definitions there, as they will be given and explained in the subsequent chapters. Sections 2.2 and 2.3 may be skipped by experienced readers.

2.1 Scope and Limitations of the AIS 20 and AIS 31

This document treats Deterministic RNGs (DRNGs), DRNG trees, Physical true RNGs (PTRNGs), and Non-physical true RNGs (NPTRNGs). In Chapter 3, seven functionality classes are defined. The generic requirements that an RNG (in the case of functionality class DRT.1: a DRNG tree) must satisfy to conform with a particular functionality class are set forth. The requirements are technology-agnostic, thereby allowing new designs to be incorporated and encourage research and new developments in this field. Whether a particular RNG actually fulfills these requirements has to be verified in a security evaluation.

In addition to the requirements of the functionality classes, further aspects and features exist, which are relevant to the security of random number generation. These are, for the most part, outside the scope of this document. An RNG is usually not the Target of Evaluation (TOE) of a CC evaluation but a component of a larger device (e.g., of a smart card or software product) that employs the RNG. Depending on the threats, assumptions, and security objectives formulated in the Security Target (ST), there are additional requirements that must be assessed during the overall security evaluation of the product. Below, we briefly address several aspects. We do not claim that this list is exhaustive.

As a rule, the vulnerability analysis of smart cards is performed according to the requirements of the highest class, AVA_VAN.5. In particular, if an RNG is a component of a smart card (which is usually the case for PTRNGs), the RNG implementation shall be secured against implementation attacks (par. 28) and attacks on the memory and data channels (par. 29), of high attack potential.

Although this document contains examples that are informative about the purposes for which different classes of functionality can be used, it does not assign functionality classes to cryptographic applications. Whether a particular functionality class is suitable for a cryptographic application is part of the security evaluation of implementation. The developer is responsible for assigning a functionality class to the implementation. Furthermore, considerations of resilience or redundancy to satisfy safety requirements or provide additional security are out of scope.

The output values of RNGs treated in this document behave (in a certain sense and to a particular degree) similarly to values assumed by independent random variables that are uniformly

distributed on $\{0, 1\}$ or on $\{0, 1\}^k$ for some integer $k > 1$. For many applications (e.g., generating a key for a block cipher), generating **bit strings** of appropriate length is usually sufficient. Other applications require random values with special properties (e.g., **uniformly distributed** values in $\{0, 1, \dots, q - 1\}$ for **Elliptic Curve Digital Signature Algorithm (ECDSA)** or prime numbers of a specific size for **Rivest–Shamir–Adleman cryptosystem (RSA)**). The transformation of random **bit strings** to application-specific random values is not part of a general-purpose **RNG**, and thus, not considered in this document. Instead, this is part of the evaluation of the respective cryptographic application. In the case of using a **DRNG**, this evaluation also needs to assess whether the **security strength (of cryptographic mechanisms)** provided by the random numbers is sufficient for the application. The technical guideline [TR-02102] proposes appropriate transformations for the applications mentioned above.

- 28 Devices intended for high-security applications are usually required to be resistant to implementation attacks. In particular, this comprises hardware attacks, side-channel attacks, and fault attacks. In this case, the **noise source** as well as the algorithmic components of the **RNG** need to be protected against attacks that might compromise or allow the manipulation of the outputs. Such attacks are not discussed in this document but must be covered by the overall security evaluation of the product. While fault attacks are outside the scope of this document, accidental failures and unintended weaknesses of the **noise source** are considered in the functionality classes for **PTRNGs** (cf. **online test** and **total failure test**).
- 29 Memory and data channels containing sensitive data should be protected against unauthorized access and manipulation during operation, and the used memory should be securely erased after use. This may include physical security measures, restrictions regarding logical access and attacks, or protection against cloning due to virtualization. Examples of sensitive information that may be stored in memory include the **internal state** of a **DRNG** instance, a ring buffer of a **PTRNG**, or registers used for **cryptographic post-processing**. Furthermore, it might be necessary to establish replay-protected secure channels to guarantee the authenticity, integrity, and confidentiality of messages sent between the components of the **RNG** and applications requesting random numbers. These considerations should be taken into account in assessing the overall system design and are outside the scope of this document.
- 30 The **online tests** and **total failure tests** introduced in this document focus on asserting proper working conditions for the **noise sources**. Other tests that are not directly used to assess the quality or the strength of the **RNG** are out of scope. For example, **Known-answer tests (KATs)** or other self-tests implemented to ensure the correct basic functioning of a device (e.g., algorithmic parts of an **RNG**) are not covered by this document. Whether the **RNG** implements such tests should be described in the **ST** of the **TOE**.

2.2 RNG Classification and Functionality Classes

- 31 The **random number generators (RNGs)** considered in this document output random **bit strings**, i.e., digital binary-valued random sequences. Note that this restriction does not preclude the use of constructions that use intermediate integer values or analog values. In this section, we illuminate and explain the fundamental concepts behind the **AIS 20** and the **AIS 31**. The following explanations are intended to facilitate comprehension of this document. They are informative

and neither replace nor supersede the requirements of the particular functionality classes defined in Chapter 3 and their corresponding application notes.

The crucial question in evaluating RNGs is: Which properties make up a secure RNG? A “natural” requirement would be the following: The RNG should output all of its admissible values with the same probability and independently from preceding and succeeding outputs. This characterizes an ideal RNG, which is easy to define in terms of random variables. However, it is a purely mathematical construct. In the real world, it is virtually impossible to build an ideal RNG, at least in the strict mathematical sense. Furthermore, even if an ideal RNG existed, it would be infeasible to prove or verify ideal randomness. 32

Instead, the best one can do is to aim for RNGs that are “close” to an ideal RNG in a certain sense. The rationale is that IT security applications usually demand “secure” random numbers, which can neither be predicted nor be determined at a later time. For TRNGs, this “security” can be measured in terms of entropy, while for DRNGs a computational equivalent is needed; cf. Section 3.3. This document divides RNGs into three main classes. 33

The first class comprises the Deterministic RNGs (DRNGs) a.k.a. Pseudorandom number generators (PRNGs). DRNGs “extend” a short random sequence (called a seed or seed material) to (possibly) very long output sequences of random bits in a deterministic way. These output sequences look random, but the unpredictability is based on the secrecy of the seed. The seed’s degree of unpredictability depends on the entropy available on the TRNG. Sometimes, additional input is provided to the DRNG during its lifetime. Well-known examples of DRNGs are the NIST-approved designs in [SP800-90A]. 34

The second class comprises the Physical true RNGs (PTRNGs). PTRNGs produce high-entropy random bits from a physical noise source based on a randomness-exhibiting physical phenomenon. This phenomenon may be realized by a physical experiment or by an electronic circuit. Usually, this allows precise entropy estimates. Examples of physical noise sources are discussed in Section 5.4. 35

Finally, the third class comprises Non-physical true RNGs (NPTRNGs). NPTRNGs also deliver true random bits but gather their entropy from non-physical noise sources for which a precise entropy estimate is usually impossible, because the NPTRNG might run on completely different platforms that are not under the control of the designer. A well-known example is the implementation of /dev/random in the early versions of the Linux kernel. 36

It is not always possible to sharply distinguish between these three classes because RNGs may have design features from both DRNGs and TRNGs. For instance, DRNGs might get additional input during their life cycle, and TRNGs might utilize cryptographic post-processing. 37

For the evaluation, we conceptually divide RNGs into a deterministic part and a non-deterministic part. 38

A hybrid RNG is a RNG that has security properties of both DRNGs and TRNGs. We distinguish between hybrid DRNGs and hybrid TRNGs. 39

40

This document aims to be as unprejudiced as possible regarding RNG designs and keep the requirements to a minimum. Apart from mild restrictions on the format of the random numbers and the minimum entropy per output bit (for TRNGs), there are almost no restrictions on what technology or constructions may be used to build an RNG that may be evaluated according to this document. Instead of approved designs, the functionality classes in Chapter 3 formulate technology-independent requirements and specify evidence that the developer needs to provide. *This does, however, exclude constructions for which the developer cannot provide sufficient evidence*, perhaps due to the lack of a clear design rationale or the lack of access to intermediate values (e.g., see par. 350).

- 41 Out of the seven functionality classes defined in this document (DRNGs: DRG.2, DRG.3, DRG.4, DRNG trees: DRT.1, PTRNGs: PTG.2, PTG.3, NPTRNGs: NTG.1), the functionality classes DRG.4, PTG.3, and NTG.1 define hybrid RNGs. Precise definitions of these functionality classes are given in Chapter 3.

2.3 Stochastic Models for PTRNGs

- 42 The key property of TRNGs is their ability to deliver random numbers derived from a source that contains a certain amount of entropy. This means that anyone observing the TRNG from the outside, irrespective of their knowledge about the design of the TRNG, its computational power, or cryptographic abilities, is uncertain about the value of the next random number to the degree specified by the entropy claim. This advantage over DRNGs comes at the price of being, in general, more difficult to evaluate. A DRNG's computational security can be evaluated independently of its implementation, and approved standard DRNG designs exist. This is not true for TRNGs, however, where the same design may behave entirely differently if using different hardware. Due to a lack of standards, TRNGs can take on many different forms and utilize various physical phenomena or properties of their underlying technology. A standardized evaluation approach for TRNGs must be able to handle this diversity and provide a method to assure that the output does indeed contain the claimed amount of entropy, with a very high certainty in each case.
- 43 A common approach to establish a baseline of assurance is subjecting a TRNG's noise source to a pre-defined series of statistical tests. The idea behind this approach is the following: Entropy is a property of the noise source itself and not the random data it produces. In mathematical terms, each random number is a realization of the sequence of random variables describing the output of the noise source. The entropy of the random numbers (prior to observing them) depends on the properties of the sequence of random variables. An empirical analysis of the random numbers allows for determining the properties of the random variables and thus, the entropy.
- 44 The statistical tests for TRNG evaluations, analyze the input data in search for specific properties or attempt to find regularities or patterns that allow a partial prediction of the input stream. Indeed, if a TRNG is consistently found to behave less random than expected, it can be safely assumed that the TRNG does not meet its claim. Unfortunately, the converse is not true. The property of a sequence of random variables having a certain amount of entropy means that it is not completely determined by patterns. However, there are infinitely many ways to completely or partially describe a stream of output values. Statistically verifying an entropy claim would

therefore require demonstrating the absence of any additional pattern (pattern type). Then again, any finite collection of statistical tests can only check for a finite number of pattern types.

Another problem with solely relying on statistical testing is that **entropy** depends on one's knowledge. **Black box** tests, meaning that they are not tailored to the design of the **noise source**, will perceive all information in their input as random. But while interference from a nearby data bus or power supply can provide additional **entropy**, this information may also be available to an outside observer. Furthermore, an **adversary** who is very acquainted with the design of the **noise source** may have a much higher chance of predicting the random number than a generic algorithm used by a test. Therefore, a **black box** statistical evaluation of a **noise source** without considering its nature is prone to overestimating the quality of a **TRNG**. 45

This document describes a practical approach that should, in principle, be applicable to any carefully designed **PTRNG**. (As will be explained later, this approach is not applicable to **NPTRNGs**, except perhaps for very special designs.) The relevant portions of the (possibly) extremely complex behavior of the **noise source** are considered and used to construct a **stochastic model** that approximates the true behavior. Taking the **stochastic model** as a postulate, suitable statistical tests can be chosen to empirically determine relevant parameters and finally calculate the **entropy**. This approach has been field-tested and successfully applied to many **PTRNGs**. A **stochastic model** is the core of evaluating **PTRNGs** according to the AIS 31. The working definition of a **stochastic model** is as follows: 46

A **stochastic model** provides a (partial) mathematical description (of the relevant properties) of a (physical) **noise source** using **random variables**. It allows for the verification of a lower **entropy** bound for the output data (**internal random numbers** or **intermediate random numbers**) during the lifetime of the **PTRNG**, even if the quality of the digitized data goes down. The **stochastic model** is based on and justified by an understanding of the **noise source**. Ideally, a **stochastic model** consists of a family of probability distributions that contains the true distribution of the **noise source** output (**raw random numbers**) or of suitably defined auxiliary **random variables** over the lifetime of the **PTRNG**. 47

Essentially, a **stochastic model** is a mathematical formulation of the idea from which the **PTRNG** was designed and of how it actually works. Much of the work necessary to construct a **stochastic model** should already have been done when the **PTRNG** was conceptually designed. Formulating the **stochastic model** requires an understanding of how the **PTRNG** functions and helps the evaluator to understand the idea behind it. A **stochastic model** of a **PTRNG** can be compared to annotated pseudocode for a piece of software and is, therefore, a very natural requirement for evaluating a **PTRNG**. It allows the evaluation of different **PTRNG** designs using different kinds of **noise sources** to have the same level of security and assurance so that each submission can be treated equally. 48

Once the relevant properties of the **raw random numbers** have been identified, they can be analyzed and estimated with customized statistical tests. While **black box** statistical tests have to consider all possible patterns, the **stochastic model** reveals which pattern a test needs to look for. This means that using a **stochastic model** is not in opposition to the use of statistical testing, but a catalyst for it to make statistical testing meaningful and practical. In most cases, a **stochastic model** states a class of mathematical distributions of an intermediate value, but not its precise parameters. Using the model, those parameters can be determined efficiently empirically 49

for a particular device in specific environmental conditions.

- 50 Knowing the range in which the true parameters for devices of a certain type of **PTRNG** lie, allows calculating the effect of **post-processing**, and finally, the determination of a lower **entropy** bound for the **internal random numbers**. Analyzing the relationship between parameters and the **entropy** of the output also allows a classification of a desired range, a tolerable range, and a non-tolerable range for the parameters. Then a lean **online test** can be chosen to monitor the parameters and thus determine whether the **entropy** claim still holds while the **RNG** is in operation.
- 51 Section 4.5 explains the concept of a **stochastic model** in detail.

2.4 Other RNG Standards

- 52 Below are some short remarks on other **RNG** standards. This list is not intended to be exhaustive.
- 53 The ISO standard [ISO_18031] specifies properties under which **RNGs** are compliant with this ISO standard.
- 54 The ISO standard [ISO_20543] considers the evaluation of **RNGs**. Like the **AIS 20** and **AIS 31**, this standard distinguishes between the evaluation of **PTRNGs** and **NPTRNGs**. The core of a **PTRNG** evaluation is a **stochastic model**. Furthermore, **PTRNGs** require efficient **online tests** (health tests) and **total failure tests**.
- 55 The **NIST** standard [SP800-22] provides a collection of statistical tests for **RNGs**. Currently, this document is under revision.
- 56 The **NIST** standard [SP800-90A] contains three **NIST**-approved designs (**DRNGs**). In Section 5.3 of the present document, the Hash_DRBG and the HMAC_DRBG are analyzed. It is shown that (for a set of specified hash algorithms) the Hash_DRBG and the HMAC_DRBG are compliant with functionality class DRG.3. If the requirements on the **seeding procedure**, the **reseeding procedure**, and/or the **high-entropy additional input** are satisfied, the Hash_DRBG and the HMAC_DRBG are compliant with functionality class DRG.4.
Note 1: This document does not contain a conformity analysis of the CTR_DRBG; cf. par. 1007.
Note 2: The document [SP800-90A] is under revision.
- 57 The **NIST** standard [SP800-90B] considers **entropy sources**. It requires that the developer justifies his **entropy** claim. Currently, a **stochastic model** is not mandatory but can be used to support the justification for the **entropy** claim.
- 58 The **NIST** standard [SP800-90C] specifies constructions for the implementations of random bit generators. These constructions include deterministic random bit generator mechanisms and use entropy sources as specified in [SP800-90A] and [SP800-90B], respectively.
- 59 [AIS 20/31, SP 800-90] **BSI** and **NIST** have been in the process of harmonizing the **AIS 20/31**

and the NIST SP 800-90 series. The document [BSI_NIST_joint_2024] illuminates the relation between AIS 20/31 and the NIST SP 800-90 series. Similarities and differences between the functionality classes in AIS 20/31 and constructions in NIST SP 800-90 are explained. The document [BSI_NIST_joint_2024] has been jointly edited by BSI and NIST. Its purpose is to support developers in designing RNGs that satisfy the requirements of both schemes.

3 Functionality Classes

- 60 In Sections 3.3 (DRNG), 3.4 (PTRNG), and 3.5 (NPTRNG), seven functionality classes are defined. Six of them refer to RNGs, while functionality class DRT.1 refers to DRNG trees. Within each subsection, the functionality classes are hierarchically ordered: the classes with greater numbers provide more security capabilities.
- 61 To keep redundancies low, general explanations are placed before the subsections that define the functionality classes. The reader is often referred to Chapter 4 and Chapter 5.
- 62 The definitions of the functionality classes use security functional requirements of the CC component FCS_RNG.1. The definition of the functionality classes is accompanied by application notes explaining their security capabilities and quality metrics.
- 63 The requirements of the functionality classes do not depend on the targeted assurance level (EAL level) of the CC certification process. This also applies to the depth of the evidence.
- 64 This document refers to Common Criteria CC:2022 Release 1, Part 1 to 5, [CC2022R1]. Note that selections in the functionality classes can be completed by choosing more than one item, unless otherwise stated („selection, choose one of“).
- 65 Section 3.6 considers cross-class aspects.

3.1 Evaluation of an RNG: General Aspects

- 66 Implementation attacks (e.g., side-channel or fault attacks) constitute severe threats against cryptographic implementations. This also applies to RNGs.
- 67 As explained in Subsection 2.1, in particular pars. 28 and 29, implementation attacks are not covered by AIS 20 and AIS 31. Consequently, implementation attacks must be part of the vulnerability analysis of the TOE to verify that successful implementation attacks on the RNG are impractical.
- 68 The most fundamental security requirements for RNGs are backward secrecy and forward secrecy. They formally describe the property of an RNG to be unpredictable, i.e., that knowledge of a subsequence of random numbers does not enable an adversary to determine or to guess the successor or predecessor of this subsequence with non-negligibly greater probability than without the knowledge of this subsequence.
- 69 More secure RNGs also provide enhanced backward secrecy and enhanced forward secrecy. These properties aim to lessen the impact of a compromise of the internal state. Enhanced backward secrecy means the following: Assurance that the knowledge of the current internal state of an RNG does not enable an adversary to derive information on previous output values. Analogously, enhanced forward secrecy assures that knowledge about subsequent output values cannot be

derived from the knowledge of the current **internal state** of an **RNG**.

In the case of **DRNGs**, the **backward secrecy**, **forward secrecy**, and **enhanced backward secrecy** requirements shall be ensured by algorithmic properties (i.e., aiming at **computational security**). In the case of **TRNGs**, these requirements shall be ensured using **fresh entropy** (i.e., aiming at **information-theoretic security**). The **enhanced forward secrecy** requirement can only be achieved using **fresh entropy**. 70

The **DRNG** functionality classes defined in Section 3.3 use the above security requirements. Classes DRG.2 through DRG.4 gradually increase in security by including more of them. Furthermore, Section 3.3 defines functionality class DRT.1 for **DRNG trees**. While this is done explicitly for **DRNGs**, the functionality classes for the **TRNGs** defined in Sections 3.4 and 3.5 shall satisfy the above requirements implicitly as a result of the **entropy** requirements. 71

3.2 Overview of the Functionality Classes

Figures 1, 2, 3, and 4 illustrate schematic **RNG** designs that are compliant with the particular functionality classes defined below. We point out that these designs are exemplary and that other technical realizations of the class requirements are possible. For example, the **total failure test** of a PTG.2 can be located behind the output of the **physical noise source** and inside it. The **physical noise source** of a PTG.3 can be a proper PTG.2 with its own **total failure test** such that an additional **total failure test** becomes unnecessary. 72

Figure 1 illustrates **pure DRNG** designs. For simplicity, in Figs. 1 and 2, we assume that **requests** are limited to a single random number (in particular, $S_{req} = S$, and ϕ_0 equals the identity mapping, see pars. 129 and 133). **Internal random numbers** denote the final stage of the random numbers of an **RNG** that are ready to be output. We use the following notation: 73

- ϕ = state transition function
- ψ = output function
- $A \rightarrow B$ = symbol for a **one-way function**

Figure 3 illustrates **PTRNG** designs. **Online tests** are indicated with a red background and **total failure tests** with a pink background. 74

Figure 4 illustrates a typical **NPTRNG** design. 75

Fig. 5 illustrates the hierarchy between the functionality classes. The requirements increase in the direction of the arrows. The dashed line is an exception. Algorithmically, the security level of functionality class DRT.1 and functionality class DRG.3 are similar. We have nevertheless positioned DRT.1 below DRG.3 because more potential security risks exist; in particular, the security of an individual **DRNG** cannot be guaranteed locally. 76

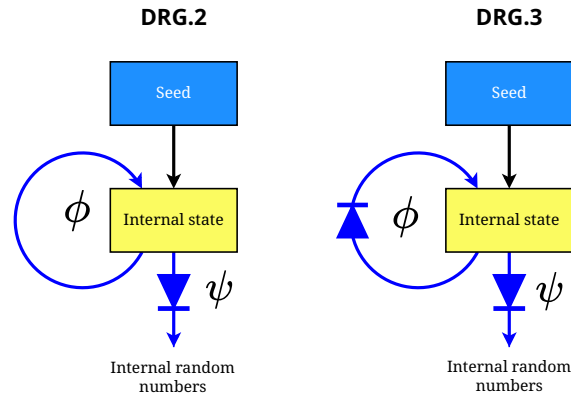


Figure 1: Functionality classes DRG.2 and DRG.3 (exemplary schematic designs)

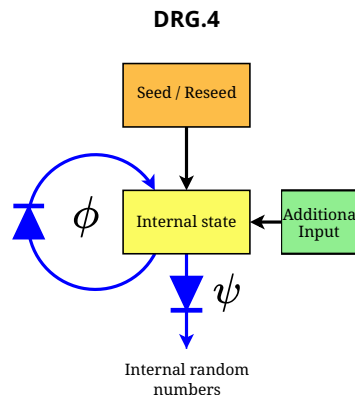


Figure 2: Functionality class DRG.4 (exemplary schematic design)

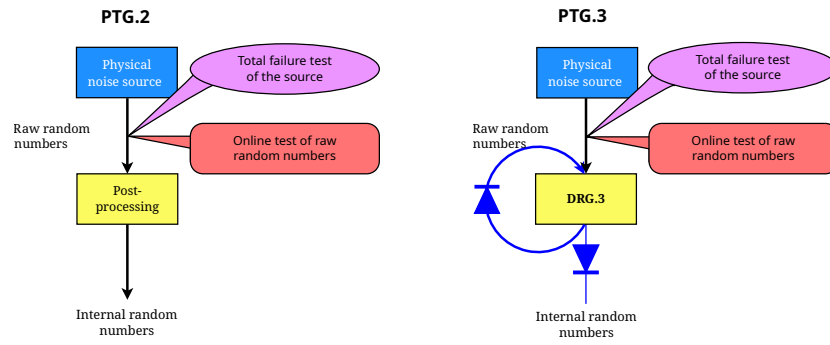


Figure 3: Functionality classes PTG.2 and PTG.3 (exemplary schematic designs)

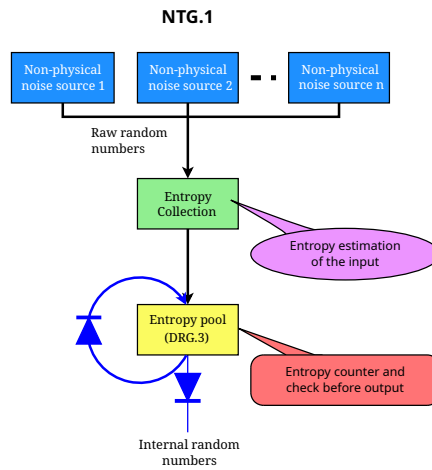


Figure 4: Functionality class NTG.1 (exemplary schematic design)

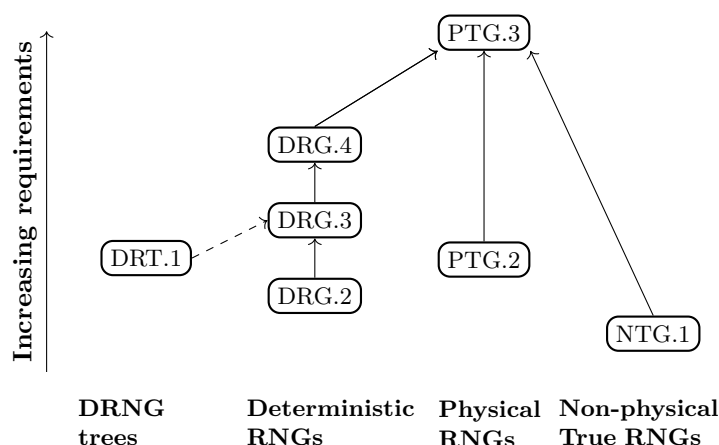


Figure 5: Hierarchy between the functionality classes

3.3 DRNGs: Functionality Classes

- 77 Subsections 3.3.2, 3.3.3, 3.3.4, and 3.3.5 define functionality classes DRG.2, DRG.3, DRG.4, and DRT.1. The differences between this document and its predecessor [AIS2031An_11] are explained in the appendix, Subsection 6.1.1. Subsection 3.3.1 contains explanations that are relevant for all DRNG classes. We begin with general remarks.
- 78 A DRNG is called a **pure DRNG** if it receives no external input data except during the **seeding procedure** or possibly during an (externally triggered) **reseeding procedure**. A DRNG is called a **hybrid DRNG** if it accepts **additional input** (regardless of any **entropy** that may be in the **additional input**) or if it is able to trigger **reseeding procedures**. The second condition requires that the DRNG has access to a randomness source, usually a **true RNG**.
- 79 Originally, functionality classes DRG.2 and DRG.3 were designed for **pure DRNGs**, but this document also explicitly covers **hybrid DRNG** designs.
- 80 A DRNG that is compliant with functionality class DRG.2 provides **backward secrecy** and **forward secrecy**.
- 81 If the DRNG is compliant with functionality class DRG.3, it additionally provides **enhanced backward secrecy**; cf. Requirements DRG.3.7 (and DRG.4.7).
- 82 **Enhanced backward secrecy** can be achieved by a state transition function ϕ that has the one-way property (i.e., a **one-way function**). For functionality classes DRG.3 and DRG.4, it is required that the state transition function is a **one-way function**.
- 83 For **hybrid DRNGs** the security requirements **backward secrecy**, **forward secrecy**, and **enhanced backward secrecy** presume that the **adversary** knows all **additional input** values. In other words: **backward secrecy**, **forward secrecy**, and **enhanced backward secrecy** shall be ensured by the algorithmic properties of the DRNG alone and without relying on any **entropy** in the **additional**

input data.

A **DRNG** compliant with DRG.4 is compliant with functionality class DRG.3 with the added benefit of **enhanced forward secrecy**. 84

The DRG.4-specific security requirement of being able to guarantee **enhanced forward secrecy** cannot be ensured by **pure DRNGs**, because without **fresh entropy** introduced as **additional input** or by a **reseeding procedure**, knowledge of the **internal state** (and further **additional input** values) reveals all future random numbers. **Enhanced forward secrecy** requires a **hybrid DRNG** that is able to trigger a **reseeding procedure**, or the input of **high-entropy additional input**. Providing **enhanced forward secrecy** can be due to a **request** of the **consuming application** or due to specified rules. 85

It should be noted that the definitions of functionality classes DRG.2, DRG.3, and DRG.4 as well as their objectives, have been reworked in this version of the document. The definitions are similar to those in [AIS2031An_11] (which justifies maintaining the class names), but they are different in detail. A new deterministic functionality class, DRT.1, is introduced. An in-depth explanation of the differences to the previous definitions in [AIS2031An_11] can be found in the appendix, Subsection 6.1.1. 86

3.3.1 DRG.[2,3,4], DRT.1: Definitions, Requirements, and Justification

The life cycle of a **DRNG** instance begins with the **seeding procedure** or the **reseeding procedure**. It ends with the following **reseeding procedure** or when the **DRNG** is uninstantiated. **Uninstantiation** results in the termination of a **DRNG** instance. In particular, the **internal state** and sensitive parameters are destroyed. (Compare with **seedlife**.) 87

Knowledge of the **internal state** of a **DRNG** allows the prediction of future outputs until sufficient **fresh entropy** is mixed into the **internal state** (from **additional input** or by applying the **reseeding procedure**). **Pure DRNGs** do not receive **fresh entropy** during their life cycle. 88

Note: An externally triggered **seeding procedure** or **reseeding procedure** starts a new life cycle.

Consequently, it is a minimal requirement for the security of a **DRNG** that it must not be possible to guess the entire **internal state** with non-negligible probability. 89

Depending on the **DRNG** design, it may be possible that an **adversary** knows or is able to learn parts of the **internal state**. This, in particular, refers to publicly known parameters and values. In Subsection 5.2.1, for example, a **DRNG** that applies **AES-256** in **Output feedback (OFB)** mode is discussed. A part of the **internal state** (128 bits) coincides with the next random number and may thus be known to an **adversary**. Those parts of the **internal state** do not contain **entropy** that defends against guessing attacks. Nevertheless, those parts may still have a positive impact on the security of the **DRNG**, e.g., against pre-computation attacks (unless these parts are always the same) and **multi-target attacks**; cf. par. 116. 90

[Definition of **effective internal state**] We refer to the security-critical part of the **internal state** of a **DRNG** that an **adversary** does not know and which he cannot determine or guess (with probability 91

that is significantly greater than indicated by its size; we always assume optimal encoding) from the knowledge of subsequences of **internal random numbers** as the **effective internal state**; cf. par. 93. Par. 101 provides an illustrating example.

- 92 The concept of the **effective internal state** applies to **backward secrecy** and **forward secrecy**. An essential requirement of functionality classes DRG.2, DRG.3, and DRG.4 is that enough **entropy** is inserted into the **DRNG** by the **seeding procedure**, the **reseeding procedure**, or by **high-entropy additional input**. For **DRNG trees**, the situation is more complex. There, **fresh entropy** is inserted only into the root **DRNG**. The other **DRNGs** in the **DRNG tree** are seeded (or reseeded) by **DRNGs** from the **DRNG tree**. This limits the overall **entropy** of the **DRNG tree**. For the **enhanced backward secrecy**, the size of the **effective internal state** is not relevant (neglecting the fact that enough **entropy** has been inserted) because the **adversary** is assumed to know the whole **internal state** anyway.
- 93 The definition of the **effective internal state** in par. 91, of course, does not take an **adversary** with unlimited computational power into account, because such an **adversary** could determine the entire **internal state** by just knowing a few output numbers. Instead, our definition aims at **computational security**. This is reasonable because an **adversary** with unlimited computational power would be able to break any **DRNG**.
- 94 The **effective internal state** and its size shall be determined under the assumption that the **adversary** knows a large number of **internal random numbers** (limited by the maximum number of random numbers between subsequent **seeding procedures**/**reseeding procedures**); cf. par. 189.
- 95 The uncertainty of the **effective internal state** from the view of an **adversary** shall be based on the **seed material**. The security of the **effective internal state** shall not be based on a **personalization string**, **secret parameters**, etc. (**Kerckhoffs's principle**) although, of course, these measures may support security. If it is possible to assign **entropy** to the **secret parameters**, e.g., because they were generated by a strong **TRNG**, they may be counted for the **effective internal state**.
- 96 The applicant for a certificate has to provide evidence that their **DRNG** design fulfills the class-specific requirements.
- 97 **AIS 20** does not prescribe approved **DRNG** designs. However, it is strongly recommended to use **widely recognized cryptographic primitives** and techniques. Otherwise, the evaluation and the verification of the algorithmic properties of the **DRNG** (such as **backward secrecy**, **forward secrecy**, and **enhanced backward secrecy**) may become impractical.
- 98 Cryptographic primitives are considered to be **widely recognized cryptographic primitives** if they have undergone diversified scientific review from many researchers, and if the cryptographic community has no serious doubts concerning their **security strength (of cryptographic mechanisms)** in relevant operational circumstances. Examples: the **AES** block cipher, and the **SHA-2** and **SHA-3** families of hash functions (**Secure Hash Algorithm (SHA)**).
Note: **AIS 20**, in particular, views cryptographic algorithms as widely recognized that are recommended in the technical guideline [TR-02102]. Further cryptographic primitives are possible *if accepted by the certification body*.

99

If **widely recognized cryptographic primitives** are used, the applicant may claim the generally accepted properties of these primitives in the security proofs of the class-specific requirements.

[Example] Examples of generally accepted security properties of cryptographic primitives: 100

- (i) The **AES** is not susceptible to chosen-plaintext attacks (cf. Subsection 5.2.1, pars. 967 and 969).
- (ii) The **one-way function** property, second pre-image resistance, and collision resistance of SHA-256.

Example: **AES-256** in **OFB** mode (cf. Subsection 5.2.1): The **internal state** comprises 384 bits 101
(a 128-bit vector plus the 256-bit long-term key). Since the first 128 bits of the **internal state** equal the next random number, the **effective internal state** only comprises 256 bits. In order to claim that an observer cannot practically determine bits of the long-term 256-bit key from the output (random numbers), the applicant may present security proofs relying on well-established properties of **AES**; cf. Subsection 5.2.1.

Example: CTR_DRBG with **AES-x**, $x \in \{128, 192, 256\}$; cf. [SP800-90A], Subsection 10.2.1. 102
The **internal state** is a pair (Key, V) and comprises $(x + 128)$ bits. Yet the **effective internal state** comprises only x bits, matching the security strength assigned to **AES-x**. This can be seen as follows: Assume that r_1, r_2 are consecutive 128-bit random numbers. For simplicity, we assume that the counter length `ctr_len` is 128 and that r_1 and r_2 belong to the same **request**. Decrypting r_1 with the key candidate Key' provides a candidate $V' = \text{AES-x}^{-1}(r_1, Key')$ for V . If $Key' = Key$, then $\text{AES-x}(V' + 1(\text{mod } 2^{128}), Key') = r_2$, providing a falsifier whether the key candidate Key' can be correct. For an incorrect key candidate Key' this equation is valid only with probability 2^{-128} . Testing all key candidates in this way reduces their number from 2^x to 2^{x-128} . If $x > 128$, using the following random number r_3 reduces the number of possible key candidates essentially to 1.

Note: This attack path works analogously if r_2 belongs to the next **request**. The only difference is that $V' + 1(\text{mod } 2^{128})$ has to be replaced by a candidate V'' that considers the CTR_DRBG_Update procedure.

Note: The size of the **effective internal state** is related to the security level of the **DRNG**. 103
Assessing the security level may require much deeper analysis unless the **DRNG** has already been analyzed in the literature. It should be noted, however, that the evaluation of a **DRNG** requires thorough security analysis to verify that the **DRNG** provides **backward secrecy**, **forward secrecy**, and (possibly) also **enhanced backward secrecy**. It can be the result of this analysis that the **effective internal state** is smaller than expected in the beginning.

We refer to state transition functions and output functions as **cryptographic** if they are composed 104
of cryptographic primitives (e.g., block ciphers or hash functions). Incrementation by 1, simple XOR-additions, additions and multiplications in small moduli, **Linear-feedback shift registers (LFSRs)**, and projections, for example, *are not viewed as cryptographic*. The composition of **cryptographic** primitives with a non-cryptographic operation usually remains **cryptographic**.
Example: $s \mapsto \text{SHA-256}(s) + 1(\text{mod } 2^{256})$. The bracket (...) denotes the smallest nonnegative representative.

It should be noted that **cryptographic** functions are not automatically suitable. The output 105
function $s \mapsto \text{SHA-256}(s) + \text{SHA-256}(s) \text{ mod } 2^{256}$, for instance, is **cryptographic** but obviously

weak because the least significant bit is always 0. Furthermore, consider par. 118.

- 106 [Definition of a **request**] Many **DRNG** designs only produce a sequence of **internal random numbers** of a fixed bit size (e.g., 128 bits). Other **DRNG** designs, such as the **NIST**-approved **Deterministic random bit generators (DRBGs)**, allow for generating random bits of different lengths (e.g., when the **DRNG** is called by an application that requires a certain amount of random bits). Upon receiving an external **request**, the **DRNG** outputs the desired number of random bits. Depending on the size of the **internal random numbers** in bits, the **DRNG** internally generates one or several **internal random numbers** to piece together the desired number of random bits. The last **internal random number** may be output only partially; cf. pars. 128 to 131 for a formal description. **Internal random number** bits not output in the **request** shall not be stored but destroyed.
- 107 [Definition of **granularity level**] In order to formulate verifiable requirements for **backward secrecy**, **forward secrecy**, and **enhanced backward secrecy**, this document uses the concept of a **granularity level** on which these security properties hold. For example, a **DRNG** satisfies *forward secrecy on the granularity level of internal random numbers* if knowledge about subsequent **internal random numbers** cannot be derived with practical computational effort from the knowledge of current or previous **internal random numbers**. Analogously, *enhanced backward secrecy on the granularity level of requests* means assurance that knowledge about previous **requests** cannot be derived with a practical computational effort from the knowledge of the current **internal state** of an **RNG**.
- 108 The security requirements for **DRNG** in this document were formulated to be applicable to as many designs as possible and make a successful evaluation of “secure” **DRNG** feasible. However, in a strict mathematical sense, they do not guarantee “ideal security” in every conceivable way. For example, **forward secrecy** on the **granularity level** of **internal random numbers** is weaker than **forward secrecy** on the **granularity level** of single output bits or a similar security property where an **adversary** knows previous as well as some subsequent output values. But **forward secrecy** on the **granularity level** of single output bits is substantially harder to prove, if not impossible. For this reason, the **DRNG** functionality classes defined in this document “only” require **forward secrecy** on the **granularity level** of **internal random numbers** paired with other requirements such as using **widely recognized cryptographic primitives** or **statistical inconspicuousness**. This gives the developer of a secure **DRNG** the practical means to convince an evaluator that the **DRNG** is indeed “secure”. It is the task of the evaluator to identify weak **DRNG** designs and reject them based on violating the requirements.
- 109 [Definition of **conceptual atomicity**] The **requests** of a **DRNG** satisfy the condition of **conceptual atomicity** if the **DRNG** finishes every **request** by the application of the state transition function before any of the requested bits are output or, at least, can be used by the application.
- 110 [justification of the **conceptual atomicity** condition (cf. par. 109)] **Enhanced backward secrecy** on the **granularity level** of **requests** (i.e., “between” **requests**) protects the **internal random numbers** generated for previous **requests** even if the current **internal state** is compromised. But it may happen that the **internal state** or the **request** state is compromised *during* a **request** (i.e., while the **DRNG** generates an **internal random numbers** for a **request**). In this case, **enhanced backward secrecy** on the **granularity level** of **requests** *does not* prevent an **adversary** from determining previous **internal random numbers** that have been generated for the current **request**. While

conceptual atomicity does not effectively prevent this situation, it assures the following condition. If the **DRNG** has not been compromised until the point in time when the output is used, then the output remains unaffected by possible compromises of the **DRNG** in the future. A more effective requirement instead of **conceptual atomicity** would be that a **request** cannot be suspended or executed within a short period of time (e.g., within one second). This requirement, however, would mean that a **DRNG** algorithm can only be evaluated with the entire environment in which it is executed. Furthermore, in many environments, this requirement cannot be guaranteed (e.g., due to CPU scheduling, virtualization, or lack of access to a clock signal).

The **conceptual atomicity** condition can be satisfied by outputting all random numbers of a **request** only after the processing for the **request** has been terminated. Alternatively, smart cards and similar hardware usually write the **internal random numbers** of a **request** to the target memory location while they are being generated. Here, the **conceptual atomicity** condition is fulfilled if the **consuming application** does not process any of the generated (and stored) **internal random numbers** until the **request** has been terminated by updating the **internal state** of the **DRNG** by the state transition function. This does, of course, limit the maximal size of a **request** to the size of the memory available to buffer generated data. 111

[**conceptual atomicity** condition (par. 109); exception] If the **DRNG** provides **enhanced backward secrecy** on the **granularity level** of **internal random numbers** (i.e., “within” **requests**), the **conceptual atomicity** condition can be dropped. More precisely, it then suffices that the **request** state is destroyed (e.g., by secure overwriting) after an **internal random number** has been generated. In order to actually achieve **enhanced backward secrecy** for the implementation of a **DRNG** (and not just its conceptual model), the guidance in par. 134 needs to be taken into account. Note: The Hash_ DRBG (cf. Subsection 5.3.1), for example, provides **enhanced backward secrecy** on the **granularity level** of **requests** but not within **requests**. 112

The following paragraph contains an informative summary of the requirements on the maximum number of random bits within a **request**, the minimum size of the **effective internal state**, and its minimal **entropy** after the **seeding procedure** (or **reseeding procedure**) for functionality classes DRG.2, DRG.3, or DRG.4. These requirements also affect functionality class DRT.1, albeit partially in a modified form. The normative requirements are contained in the definitions of the functionality classes below. 113

[DRNG: Minimal requirements]; cf. the Requirements DRG.x.2, DRG.x.3, DRG.x.4 ($x \in \{2, 3, 4\}$) and DRG.4.10 114

- Each **request** shall comprise a maximum of 2^{19} bits.
- The **effective internal state** shall comprise at least 246 bits.
- The **min-entropy** of the initial **effective internal state** after the **seeding procedure**, resp. after the **reseeding procedure**, shall be at least 240 bits. Alternatively, 250 bits of **Shannon entropy** suffice, provided that the **raw random numbers** of the true **RNG** are **stationarily distributed**. For the **reseeding procedure**, the most conservative assumption is assumed that an **adversary** knows the **internal state** before reseeding.

A more precise formulation of the size condition would be the following: The set $\{0, 1\}^{246}$ can be embedded into the set of all admissible **effective internal states** S_{eff} , i.e., there exists an injective mapping $\chi: \{0, 1\}^{246} \rightarrow S_{\text{eff}}$.

Similarly, a more precise formulation of the **entropy** condition would be the following: Assume that the **random variable** X takes on values in S_{eff} and describes the (random) value of the **effective internal state** after the **seeding procedure**, resp. after the **reseeding procedure**. For the **seeding procedure** the third bullet point reads $H_{\min}(X) \geq 240$ or $H(X) \geq 250$, respectively. Assume further that the **random variable** X' describes the (random) value of the **effective internal state** before the **reseeding procedure**. Then the third bullet point implies $H_{\min}(X \mid X') \geq 240$ or $H(X \mid X') \geq 250$, respectively.

- 116 The lower bounds for the **entropy** after the **seeding procedure**/**reseeding procedure** and, for the minimal size of the **effective internal state** defined in par. 114 shall repel, among other things, potential threats by quantum computers and by **multi-target attacks**. It should be noted that to prevent **multi-target attacks**, it would suffice to increase the size of the **internal state** accordingly (instead of the **effective internal state**), provided that all parts of the **internal state** affect the **internal random numbers**.
- 117 If the applicant claims **Shannon entropy** for the **seeding procedure** or the **reseeding procedure** with a **PTRNG**, an additional condition has to be met, namely the stationarity, or more precisely, the **time-local stationarity** of the **raw random numbers**; cf. DRG.2.4, DRG.3.4, DRG.4.4, DRT.1.6, and application note par. 191). The stationarity condition shall prevent well-known pathologies that may arise when probability distributions are (extremely) unbalanced; cf. par. 663, for example.
- 118 In the presence of quantum computers, **DRNG** designs whose security relies on the hardness of factoring or the discrete logarithm problem will likely become insecure (due to being susceptible to quantum attacks). Until then, such designs may be used but, of course, parameter(s) shall be selected so that the instances resist all known factoring algorithms or algorithms that compute the discrete logarithm. This document discourages using **DRNG** designs based on the hardness of factoring or the discrete logarithm problem.
- 119 The **entropy** of the initial **internal state** is an upper bound for the overall **entropy** of the generated random numbers or subsets thereof if no **fresh entropy** is mixed into the **internal state**.
- 120 As already explained in par. 93, **DRNGs** are ineffective against an **adversary** with unlimited computational resources. Relative to an **adversary** with unlimited computational resources, the **effective internal state** would contain **entropy** only if the **adversary** did not know more than a few **internal random number** bits (**information-theoretic security**) or had not otherwise observed information about the **seed material** to test their hypotheses. But, if the **DRNG** is computationally secure, then a resource-limited **adversary** observing random numbers will still be unable to determine the information contained in the (effective) **internal state**.
- 121 There are two generic attacks to guess an unknown k -bit output string x of a **DRNG**, namely blind guessing of x and blind guessing of the initial **internal state** (or **seed material**, whichever has less **entropy**). If an **adversary** guesses the initial **internal state** correctly, this **adversary** can compute all random numbers for the current **seedlife** (assuming they also know any **additional input** data that was provided). Requirements DRG.2.4, DRG.3.4, DRG.4.4, DRT.1.6, and DRT.1.7 ensure

that the second attack is (at least approximately) a 240-bit problem. For an “ideal” DRNG, guessing x (first attack) would be a k -bit problem if $k < 240$ (first attack) and (at least) a 240-bit problem, otherwise.

In the context of forward secrecy, backward secrecy, and enhanced backward secrecy, for k -bit output strings the security strength (of cryptographic mechanisms) shall not be significantly lower than for the ideal case (cf. par. 120), i.e., 240 bits. Furthermore, the security strength (of cryptographic mechanisms) of the DRNG shall not significantly decrease during its lifetime. 122

The entropy of the internal state can decrease during the lifetime of a DRNG instance if the state transition function ϕ is not bijective. To comply with functionality class DRG.3, ϕ must be a one-way function. The effective internal state shall contain sufficient entropy within the whole life cycle of the instance to prevent successful guessing attacks. 123

Ideally, after the seeding procedure/reseeding procedure, the probabilities of all possible values of the effective internal state would be the same. From an adversary’s point of view (trying to guess the effective internal state), this represents the worst-case, as every value would be equally likely. Since the state transition function ϕ is usually not bijective, the probability distributions of the internal states (or rather, the effective internal states) might become more and more imbalanced over time, thereby reducing the entropy of the internal state. If an adversary had precise knowledge of these distributions, they could be leveraged to speed up guessing attacks by selecting the most probable internal states after n iterations of ϕ . 124

However, such a deep understanding of the iterated application of ϕ would likely also allow analytical attacks on the forward secrecy property. If theoretical statements on the distributions of future internal states are possible, this shall be considered in the security proofs of the algorithmic properties of the DRNG. 125

Note: Section 4.4 treats random mappings.

An adversary without such deep understanding (of the distributions of the internal state/effective internal state after the iterated application of ϕ) could only try to mount a generic guessing attack to exploit a growing imbalance of the distribution of the (effective) internal states. For example, to obtain a guess of the internal state s after the n^{th} iteration of ϕ , the adversary could randomly select an element of S and apply the state transition function ϕ n times. In this case, each single guess is much more costly than a single “blind” guess of the internal state after n iterations of ϕ . For a hash function that is a widely recognized cryptographic primitive (e.g., for SHA-256), it is assumed that even with precomputations, the entropy loss due to the iterated application is not practically exploitable. 126

If the DRNG design allows the computation of the n -fold composition of ϕ that is significantly faster than a step-by-step evaluation of ϕ or other ways to speed up guessing, this shall be considered in the evaluation. 127

DRNGs usually have a core function that generates blocks of internal random numbers of a size prescribed by the chosen cryptographic primitive (e.g., designs based on AES usually generate blocks of 128 bits). For simple DRNG designs, this core function coincides with the output function (which means that the DRNG can only generate random numbers of a fixed size). But in practice, the logic to generate random numbers of a requested length is often already integrated 128

into the **DRNG**. In order to formulate security requirements, we use the following formal model.

129 The algorithmic structure of a **DRNG** can be described by a 9-tuple

$$(S, S_{req}, R, A, I, \phi, \phi_{req}, \phi_0, \psi) \quad (\text{describing 9-tuple of the } \mathbf{DRNG}) \quad (3.1)$$

The components of (3.1) have the following meaning:

- S = set of admissible **internal states** (typically, $S = \{0, 1\}^n$)
- S_{req} = set of admissible (temporary) **internal request states**
- R = set of admissible output values (**internal random numbers**), $R = \{0, 1\}^k$ for some $k \in \mathbb{N}$.
- A = set of admissible **additional input** values (typically, $A = \{0, 1\}^*$, where ϵ denotes the empty string (= no **additional input**))
- I = set of admissible **request** lengths, counted in bits
- $\phi: S \times A \times I \rightarrow S$ (state transition function, logically computed at the end of a **request**)
- $\phi_{req}: S \times A \rightarrow S_{req}$ (generates the **internal request state**)
- $\phi_0: S_{req} \times A \rightarrow S_{req}$ (**request** state transition function)
- $\psi: S_{req} \rightarrow R$ (output function)

Note: The set S is also denoted as **internal state space**, and, similarly, the set of all admissible **effective internal states** S_{eff} is called **effective internal state spaces**.

- 130 If public parameters and/or **secret parameters** affect any of the mappings ϕ , ϕ_{req} , ϕ_0 , or ψ , then these data items are part of S .
- 131 If the **request** requires the generation of $p \in I$ bits, then $m := \lceil \frac{p}{k} \rceil$ **internal random numbers** are generated (where k denotes the bit length of the **internal random numbers** as defined in par. 131). In pseudocode, a **request** implies the following steps:

$$\begin{aligned} & s_{req} := \phi_{req}(s, a) \\ & \text{for } j := 1 \text{ to } m \text{ do } \{ \\ & \quad r_j := \psi(s_{req}); \\ & \quad s_{req} := \phi_0(s_{req}, a); \\ & \} \\ & s := \phi(s, a, p) \end{aligned} \quad (3.2)$$

Here, r_j denotes the j^{th} **internal random number** generated within the current **request**. Depending on p (number of requested random bits), the m^{th} **internal random number** may be truncated when output. More precisely, the right-most $k \lceil \frac{p}{k} \rceil - p$ bits of the last **internal random number** are not output.

Note: Often, physically $S_{req} \subseteq S$. In this case, the internal **request** state s_{req} “vanishes” automatically right after the **request** has ended. Prominent examples are the Hash_DRBG and the HMAC_DRBG [SP800-90A]. Otherwise, if $S_{req} \not\subseteq S$, the internal **request** state s_{req} shall be overwritten or erased when the **request** has been terminated. Usually, for $j = m$, the step $s_{req} := \phi_0(s_{req}, a)$ of the loop need not be executed.

If the **DRNG** does not allow **additional input**, then $A = \{o\}$ and ϕ, ϕ_{req} , and ϕ_0 actually do not depend on A . 132

The 9-tuple describes the *conceptual structure* of the **DRNG**. As already mentioned in the note of par. 131, within a **request** S_{req} often physically coincides with a subset of S ; cf. par. 135. 133

Note: For the Hash_DRBG the **request** state S_{req} equals the first component of the **internal state space** $S = S' \times S' \times \mathbb{N}$, and the impact of ϕ_0 is encoded in the output function; cf. Subsection 5.3.1, par. 1017 ff.

For the analysis of **enhanced backward secrecy**, the evaluator shall not only consider the 9-tuple (i.e., the conceptual structure of the **DRNG**) but also the **DRNG** itself and (if applicable) its implementation. The conceptual structure can be used to verify that certain information that practically allows computing previous output values does not remain in the **internal state**. The evaluator shall verify that this is indeed the case. For example, if a **DRNG** derives the **request** state s_{req} from the state s (cf. par. 131) and claims **enhanced backward secrecy** on the **granularity level of internal random numbers** (i.e., within **requests**), then s needs to be deleted or overwritten. Analogously, after a **request** has been completed, all information on the **request** state s_{req} needs to be deleted from memory in order to achieve **enhanced backward secrecy**. 134

Note: The set S_{req} is by definition part of the **internal state** of the **DRNG**, although it may not be not apparent in the formal model using S .

[9-tuples, simplification] For pure **DRNGs**, **additional input** is not allowed. In this case, it is permitted to simplify the definition of the state transition function to $\phi: S \times I \rightarrow S$ and of ϕ_{req} to $\phi_{req}: S \rightarrow S_{req}$ (instead of $\phi: S \times A \times I \rightarrow S$ and $\phi_{req}: S \times A \rightarrow S_{req}$). 135

If, additionally, by design only **requests** of $\leq k$ bits are allowed, only one **internal random number** is generated. In particular, the function $\phi_{req}: S \rightarrow S_{req}$ is never applied and thus can be defined arbitrarily, e.g., as the identity mapping. Furthermore, it is usually convenient to set $S_{req} = S$.

Example: see Subsection 5.2.1 (pure **DRNG**, **request** length $\leq k$). 136

A **DRNG** derives its initial **internal state** from a randomly selected seed value (**seed material**). The “seed **entropy**”, that is, the **entropy** contained in the **seed material** used for the **seeding procedure**, the **reseeding procedure** (if applicable), and a description of how the **seed material** was generated must be covered by the deliverables from the applicant; cf. pars. 180, 205, and 221. 137

[**TRNG** seeding **DRNG**] If the **seed material** (for the **seeding procedure** or **reseeding procedure**) is generated by a **PTRNG** (compliant with PTG.2 or PTG.3) or by an **NPTRNG** (compliant with NTG.1), the Requirements PTG.2.2, PTG.3.5, or NTG.1.5 guarantee lower **entropy** bounds per **internal random number** bits. The Requirements PTG.2.2 and PTG.3.5 allow claims in both **Shannon entropy** and **min-entropy** while NTG.1.5 prescribes **min-entropy**. 138

139

[TRNG seeding DRNG] The **entropy** requirements for seeding, reseeding, and inserting additional high-**entropy** input are quantified in **min-entropy**. Under suitable conditions, namely, if the **seed material** is generated by a PTRNG and if its **raw random numbers** are time-locally **stationarily distributed** (cf. **time-local stationarity**; fulfilled for functionality classes PTG.2 and PTG.3), the **min-entropy** claim can be substituted by a **Shannon entropy** claim; cf. Requirements DRG.2.4, DRG.3.4, DRG.4.4, DRG.4.10 for details.

140 [DRNG seeding DRNG] This case is treated in pars. 163 to 170.

141 [Personalization string] An optional input parameter to the **seeding procedure** is a **personalization string**. This denotes a freely chosen value, often derived from information related to a specific DRNG instance. Unique **personalization strings** can inhibit side-channel analysis and may prevent an **adversary** in control of the **seed material** from identifying the **internal state** of the DRNG. Furthermore, if the **personalization string** is kept secret, it may be the last resort if the **seed material** (cf. pars. 143 to 144) is compromised.

142 The 9-tuple (3.1) describes the algorithmic structure of the DRNG when it is in operation. Similarly, the **seeding procedure** and the **reseeding procedure** can formally be described.

143 The following 4-tuple describes the algorithmic aspects of the **seeding procedure**:

$$(SM, PS, S, \phi_{seed}) \quad (\text{describing 4-tuple of the seeding procedure}) \quad (3.3)$$

The components of (3.3) have the following meaning:

- SM = set of admissible values of the **seed material** (typically, $SM = \{0, 1\}^s$)
- PS = set of **personalization strings** (may contain public and secret parts)
- S = set of admissible **internal states**
- $\phi_{seed}: SM \times PS \rightarrow S$ (**seeding procedure**)

144 The security of the **seeding procedure** shall not be based on the **entropy** of the **personalization string** (even if it contains **secret parameters**). The **seed material** itself shall contain enough **entropy** to meet the requirements (DRG.2.4, DRG.3.4, DRG.4.4, DRT.1.6).

145 The simplest **seeding procedure** is the following: (i) $\phi_{seed}(sm, o) := sm$. The seed is copied into the **internal state**.

146 Other, more complex **seeding procedures** exist that use **cryptographic** mechanisms. Examples:
 (i) $\phi_{seed}(sm, o) := \phi(sm, o)$. The **seed material** is copied into the **internal state** and the state transition function ϕ is applied once.

(ii) NIST-approved DRBG designs in [SP800-90A]

Note: **Seeding procedures** and **reseeding procedures** that use **cryptographic** mechanisms are recommended. One advantage is that statistical weaknesses of the **seed material** as bias or short-term dependencies are blurred.

147

The following 4-tuple describes the algorithmic aspects of the **reseeding procedure**:

$$(SM, PS, S, \phi_{\text{reseed}}) \quad (\text{describing 4-tuple of the } \textbf{reseeding procedure}) \quad (3.4)$$

The components of (3.4) have the following meaning:

- SM = set of admissible **seed material** values (typically, $SM = \{0, 1\}^s$)
- PS = set of **personalization strings** (may contain public and secret parts)
- S = set of admissible **internal states**
- $\phi_{\text{reseed}}: S \times SM \times PS \rightarrow S$ (**reseeding procedure**)

The security of the **reseeding procedure** shall not be based on the **entropy** of the current **internal state** or the **personalization string** (even if it contains **secret parameters**). The **seed material** itself shall contain enough **entropy** to meet the requirements (DRG.2.4, DRG.3.4, DRG.4.4, DRT.1.6). This document requires using the current **internal state** as an additional parameter. 148

Note: The knowledge of the precise probability distribution on the set of all admissible **effective internal states** after the **seeding procedure** and the **reseeding procedure** would allow the determination of the resulting **entropy**. However, except for very simple **seeding procedures** (e.g., when using bijections, as in par. 145), this is, in general, practically infeasible. The same is true for the distribution on S , the set of all admissible **internal states**. Thus, unlike in the 6-tuple in [AIS2031An_11], the distribution on S after (re-)seeding is not a component of the describing 9-tuple. Instead, the Requirements DRG.2.4, DRG.3.4, and DRG.4.4 aim at the **entropy** induced on the set of all admissible **effective internal states**. 149

In order to verify requirements concerning the **entropy** of the initial **effective internal state** (DRG.2.4, DRG.3.4, DRG.4.4, DRT.1.6), specifying the precise probability distribution is not required. Instead, it suffices to specify a lower **entropy** bound. The developer shall present a justified estimate, e.g., by modeling **widely recognized cryptographic primitives** such as hash functions or block ciphers as random mappings or random bijections. The evaluator decides whether simplifications made in this model are acceptable and whether the properties of random mappings and random bijections can be used as assumptions. Section 4.4 collects properties of random mappings and random bijections; cf. also several application notes to Requirement PTG.3.5. 150

The characteristics of random mappings usually do not apply if the definition range is too small. To give an extreme example, it is not permitted to model a randomly selected mapping $\{0, 1\}^{16} \rightarrow \{0, 1\}^{16}$ as a random mapping and to draw conclusions from Section 4.4. 151

In Subsection 5.2.2, pars. 988 to 990, it is shown that if the state transition function of a **hybrid DRNG** is too simple, an **adversary** who is able to manipulate **additional input** data might be able to control the evolution of the **internal state**. This can weaken a **DRNG** completely. 152

Subsection 5.2.2, par. 992 provides an example of a disastrous interaction of the state transition function and the output function. A similar example that accepts **additional input** data is 153

$\phi(s, a) = \text{SHA-256}(s) + 1 + a \bmod 2^{256}$, $\psi(s) = \text{SHA-256}(s)$. This feature weakens the DRNG completely.

- 154 If **additional input** is permitted, it shall not weaken the algorithmic strength of the DRNG. The **hybrid DRNG** version shall not be less secure than the **pure DRNG** version of the DRNG that does not allow **additional input** (DRG.2.7, DRG.3.8, DRG.4.8). The **hybrid DRNG** mentioned in par. 153 does not meet these requirements.
Note: This problem seems to primarily affect DRNGs of functionality class DRG.2 (e.g., with a simple state transition function).
- 155 A reasonable design strategy to prevent (chosen) **additional input** from weakening the DRNG is to select a state transition function whose “core” is a hash function and use it to mix the **additional input** into the **internal state**. However, depending on the DRNG design, simpler state transition functions may also be appropriate. Pitfalls where **additional input** weakens the DRNG should primarily be an issue for DRG.2-compliant DRNGs.
- 156 To ensure **enhanced forward secrecy**, **additional input** with sufficient **entropy** or a **reseeding procedure** is needed (cf. Requirement DRG.4.10). One-time high-**entropy** input cannot be compensated by many low-**entropy additional inputs** within several **requests**.
Example: Assume that a random byte is mixed into the **internal state** of the DRNG in each iteration, i.e., whenever an **internal random number** is generated. Assume further that an **adversary** knows the current **internal state** and that the DRNG generates 128-bit **internal random numbers**, which the application uses as **AES** keys. If an **adversary** knows a plaintext/ciphertext pair for each **AES** key, it would allow successive key guesses, e.g., the next 2^{10} **AES** keys could be guessed with $\leq 2^{10} \cdot 2^8 = 2^{18}$ trials.
- 157 It shall not be possible to distinguish sequences of **internal random numbers** that are generated by a DRNG that is compliant with the DRG.2, DRG.3, or DRG.4 functionality class from output sequences of an **ideal RNG** by statistical tests. Of course, “unfair” statistical tests that exploit knowledge of the **internal state** of the DRNG are excluded.
- 158 **Widely recognized cryptographic primitives** should not show any statistical weaknesses.
- 159 Example: **AES**, CBC mode: ciphertext blocks (interpreted as a bit sequence) of arbitrary plaintexts should not show any statistical weaknesses.
- 160 Usually, evidence that the DRNG fulfills Requirements DRG.2.9, DRG.3.10, or DRG.4.11, can be given by theoretical arguments about the cryptographic primitives. Then, the application of statistical (**black box**) tests to output sequences of the DRNG is not necessary.
- 161 Note that in the presence of implementation or design flaws, the use of **widely recognized cryptographic primitives** alone does not preclude the existence of statistical weaknesses. Extreme example: The output of a hash function is (accidentally) concatenated with itself or **bit strings** are filled up with zeroes.
- 162 If the evaluator suspects that a given DRNG design does not have the property of **statistical inconspicuousness**, targeted statistical tests should be applied to test for these possibilities.

[DRNG seeding DRNG] Especially with software implementations on PCs, servers, etc., it can happen that DRNGs are needed but there is no TRNG available that could generate seed material. To give a simple example: The DRNG of the operating system has been seeded by an NPTRNG, and the applications call this DRNG for seed material to seed their own DRNGs. In practice, more complex scenarios can occur.

Example 1: In OpenSSL, threads instantiate (and uninstantiate) two DRNGs of their own, which are seeded by another DRNG.

Example 2: Since kernel version 5.6, Linux /dev/random is no longer compliant with functionality class NTG.1. Instead, the cryptographic post-processing algorithm (base ChaCha20) is interpreted as a (hybrid) DRG.3-compliant DRNG; cf. Section 5.6. When using /dev/random, as of kernel version 5.18.1, upon request, CPU-local DRNGs provide the random numbers. These second-level ChaCha20 DRNGs are seeded and reseeded from the base ChaCha20 DRNG; cf. Fig. 36.

[DRNG seeding DRNG] For these reasons, a DRNG can optionally also be seeded/reseeded by another DRNG, *provided that certain conditions are fulfilled*. These conditions, specified in functionality class DRT.1, shall mitigate additional security threats and risks (exemplarily addressed in pars. 170 to 172) that arise from the fact that the seeding procedures and reseeding procedures do not use seed material generated from a TRNG.

[DRNG seeding DRNG, notation] We begin with a DRNG that receives its seed material from a TRNG or a DRNG that is reseeded to provide enhanced forward secrecy immediately before the seed material is generated. This DRNG is called *root DRNG*. The root DRNG can generate seed material for other DRNGs, and each of these DRNGs can generate seed material for further DRNGs etc. *The following definitions apply to seeding (but not to reseeding!)*.

DRNG B is called a *direct seed successor* of DRNG A if DRNG B has received seed material for the seeding procedure from DRNG A. Vice versa, DRNG A is called the *direct seed predecessor* of DRNG B. DRNG D is a *seed successor* of DRNG A if a chain of direct seed successors exists from DRNG A to DRNG D. Then, DRNG A is a *seed predecessor* of DRNG D. DRNG F is called a *sibling* of DRNG B if both have the same direct seed predecessor (here: DRNG A). Fig. 6 illustrates these definitions.

[DRNG seeding DRNG, DRNG tree] Par. 165 describes how DRNGs can successively be instantiated using seed material from other DRNGs. In this document, a *DRNG tree* is composed of a single root DRNG and its successor DRNGs. The DRNG tree includes information about the direct seed predecessors (and thus, also about the direct seed successors). The root DRNG receives seed material from an *initial randomness source* (of the DRNG tree) that is either a TRNG or a DRNG that is reseeded to provide enhanced forward secrecy immediately before the seed material is generated. The *initial randomness source* is not considered part of the DRNG tree. The root DRNG is the only DRNG in the DRNG tree that receives seed material from the initial randomness source. All other DRNGs receive their seed material from other DRNGs in the DRNG tree. The direct seed predecessor of a DRNG is its *randomness source*.

[DRNG seeding DRNG, seed graph, instantiation] Relevant seeding and reseeding information can be stored and visualized by a *seed graph*. For each DRNG in the DRNG tree, the seed graph includes information about its direct seed predecessor and all DRNGs that have ever generated seed material for reseeding this DRNG. When a new DRNG in the DRNG tree is instantiated,

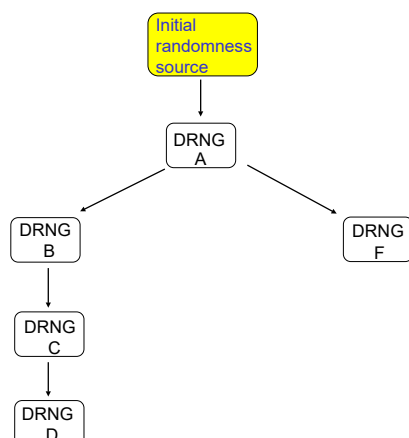


Figure 6: Example of a **DRNG tree**: The directed edges correspond to the property “has generated **seed material** for the **seeding procedure** for”. The figure illustrates the definitions “direct seed predecessor”, “direct seed successor”, “seed predecessor”, “seed successor”, and “sibling”; cf. also par. 167, Note 2.

a node is added to the **seed graph**. In the **seed graph** a directed edge is drawn from the direct seed predecessor of the instantiated **DRNG** to the instantiated **DRNG**. The new node and the new edge are painted in black. Fig. 7 shows a **seed graph**.

Note 1: We identify the existing (i.e., not yet uninstantiated) **DRNGs** (physical objects) in the **DRNG tree** with nodes in the **seed graph**. In particular, we use the same name for a **DRNG** of the **DRNG tree** and the corresponding node in the **seed graph**, e.g., by **DRNG F**. The root node of the **seed graph** corresponds to the root **DRNG** of the **DRNG tree**. The node in the **seed graph** that corresponds to the direct seed predecessor of a **DRNG** in the **DRNG tree** is also called the *parent node*.

Note 2: In the context of graphs, we speak of nodes and (directed) edges. It should be mentioned that in the literature, the term vertex is often used instead of node, whereas (directed) edges are also called directed lines, directed links, and arrows.

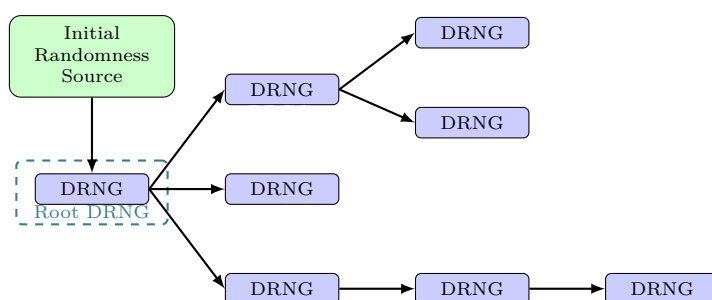


Figure 7: An example of a **seed graph**. The **initial randomness source** provides the **entropy** for the whole **DRNG tree**. Arrows point to the direct seed successor (a child node in the **seed graph**). The **initial randomness source** is not part of the **DRNG tree**.

[DRNG seeding DRNG, seed graph, uninstantiation] DRNGs of a DRNG tree can be uninstantiated; ideally they are deleted then and their internal states are overwritten. Although they no longer physically exist, the knowledge of uninstantiated DRNGs is still relevant as will become apparent below. Therefore, although uninstantiated DRNGs disappear from the DRNG tree, the corresponding nodes and the directed edges to and from these nodes remain in the seed graph. The node corresponding to an uninstantiated DRNG is painted in gray from now on. The edges stay black. Fig. 8 illustrates the concept using an example.

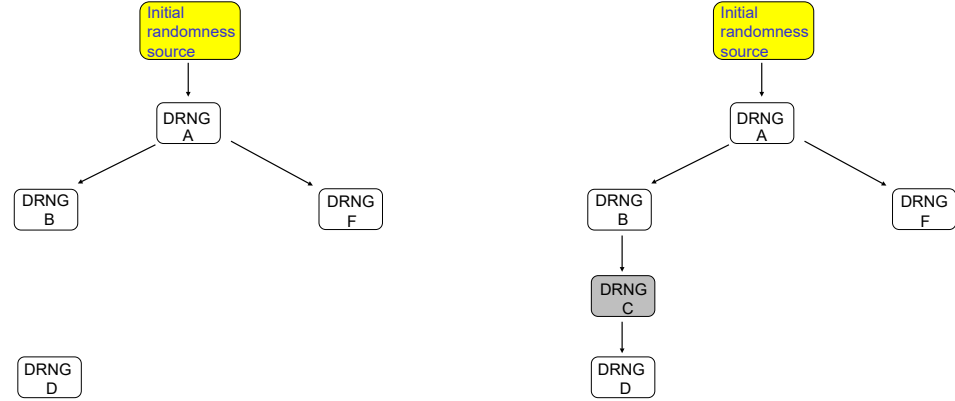


Figure 8: The left drawing shows the DRNG tree from Fig. 6 at a later time after DRNG C has been uninstantiated. The right drawing shows the corresponding seed graph.

[DRNG seeding DRNG, seed graph, reseeding] If DRNG F generates seed material for reseeding DRNG C, a red edge from DRNG F to DRNG C is added to the seed graph unless this edge already exists (either in black or red); cf. Fig. 9 for illustration. If a DRNG in the DRNG tree is later uninstantiated in the seed graph, the red edges from and to this node remain in the seed graph and stay red. The seed graph in Fig. 9 contains uninstantiated DRNGs and reseed edges. Note 1: The coloring of the nodes (black, gray) allows distinguishing DRNGs that still exist from DRNGs that have already been uninstantiated. The coloring of the edges (black, red) allows distinguishing ‘seeding edges’ from ‘reseeding edges’. In particular, the only directed black edge to a DRNG comes from its direct seed predecessor.

Note 2: A black edge from DRNG B to DRNG C (cf. Fig. 9) says that DRNG B has generated the seed material for seeding DRNG C (and possibly seed material for one or several reseeding procedures). A red edge from DRNG F to DRNG C implies that DRNG C has received seed material for reseeding from DRNG F at least once and that DRNG F is not the direct seed predecessor of DRNG C.

Note 3: All nodes of a seed graph are black or gray. All nodes, together with the black edges, form a subgraph of the seed graph that contains the seeding information when the DRNGs were instantiated. This subgraph is always a tree (“instantiation tree”) because the seed graph has only one root node, and each DRNG other than the root DRNG has exactly one direct seed predecessor. This motivated the intuitive but admittedly loose naming “DRNG tree”. It should be noted that in the terminology of graph theory, the DRNG tree can develop from a tree into a forest due to the uninstantiation of DRNGs.

Note 4: As long as no DRNG has been uninstantiated, there is a 1-1 correspondence between the DRNG tree and the subgraph defined in Note 3. Recall that this subgraph does not contain any reseeding information.

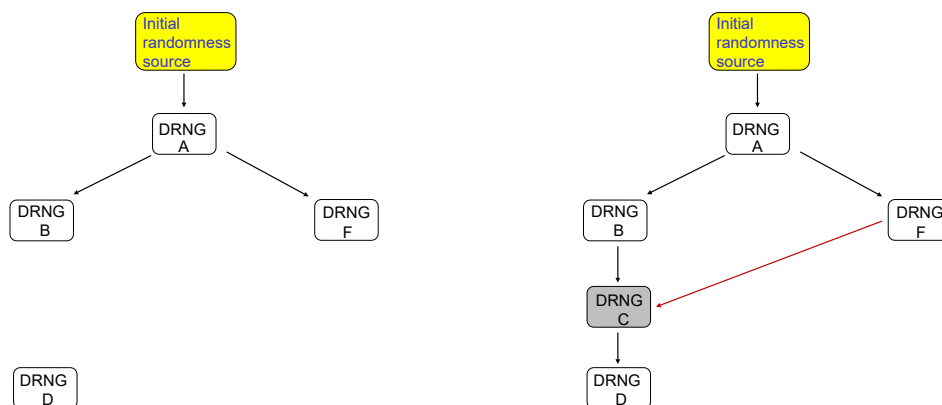


Figure 9: The left drawing shows the **DRNG tree** from Fig. 8. The **seed graph** shows that, unlike in Fig. 8, **DRNG C** had been reseeded by **DRNG F** before it was uninstantiated.

- 170 [DRNG seeding DRNG; additional security risks] Compared to the usual way of (re-)seeding a DRNG with a TRNG, (re-)seeding with a DRNG bears additional security risks. When (re-)seeding with a TRNG, it suffices that the TRNG works properly at the time when the seed material is generated. Possible entropy defects or successful attacks in the past or in the future are not relevant for this purpose. This changes if the seeding procedure or reseeding procedure uses seed material that was generated by a DRNG. Successful attacks in the future on the DRNG that generated the seed material do not affect the (computational) security of currently generated seed material if this DRNG provides enhanced backward secrecy. The situation becomes much worse if the attack has happened in the past. An (undetected) compromise of the internal state of that DRNG (i.e., an adversary gained knowledge of the internal state) would affect all DRNGs that receive (directly or indirectly) seed material from the compromised DRNG after that time. This means that the security breach would spread transitively in the DRNG tree. Note: Due to the sketched security risks, it is recommended to use a TRNG for (re-)seeding a DRNG if available. DRNG trees should be used only in scenarios where this is not the case or if the availability cannot be guaranteed.
- 171 [DRNG seeding DRNG, seed loop] A major issue is the DRNGs from which a DRNG can accept seed material (if applicable) for a reseeding procedure. An important security goal is to prevent “seed loops” (i.e., directed cycles in the seed graph) because in that case a DRNG would transitively reseed itself. A seed loop would be problematic for several reasons. First of all, the situation would become weird and extremely complex, usually prohibiting a solid security evaluation.
- 172 [DRNG seeding DRNG, seed loop] Another, even more important reason to prevent seed loops is the following: Assume that the internal state of DRNG B has been compromised but not the internal state of its direct seed predecessor, DRNG A. Then reseeding with seed material from DRNG A heals the compromise of DRNG B (unless, of course, the adversary still has access to the internal state of DRNG B but, in that case, then also reseeding with a TRNG would not help).

If **DRNG B** has provided **seed material** after its **internal state** has been compromised, all direct seed successors of **DRNG B** that are instantiated after this instant are compromised, too, and this problem spreads transitively. For this reason, reseeding **DRNG B** may not heal the compromise if a seed loop exists, namely if reseeding is accomplished using output from a seed successor of **DRNG B**.

If the compromised **DRNG B** provides **seed material** for reseeding **DRNG C** (after **DRNG C** was initially seeded securely by **DRNG B**), this does not make things worse. Of course, if **DRNG B** has been compromised, reseeding does not increase the security of **DRNG C**.

The **seed graph** in Fig. 10 contains a seed loop between **DRNG B** and **DRNG C**. The **seed graph** in Fig. 11 illustrates a situation where a sibling node provides **seed material** for reseeding. Here **DRNG E**, which is a sibling of **DRNG F**, provides **seed material** for reseeding **DRNG G** (whose parent is **DRNG F**).

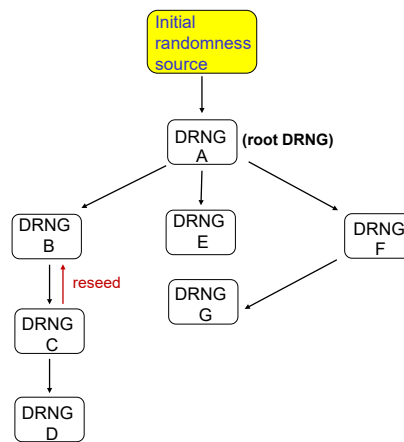


Figure 10: **Seed graph** with a seed loop between **DRNG B** and **DRNG C**.

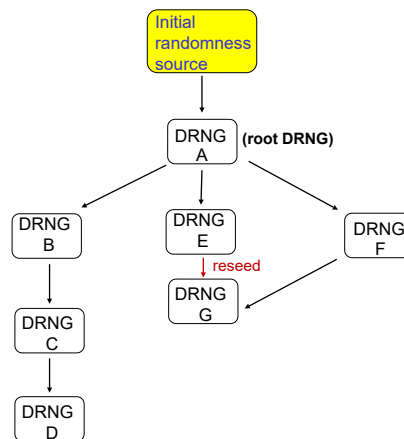


Figure 11: **Seed graph**. **DRNG G** is reseeded by **DRNG E**, which is a sibling of its **direct seed predecessor DRNG F**.

[DRNG seeding DRNG, functionality class DRT.1] Functionality class DRT.1 specifies requirements for DRNG trees. *The objects of this functionality class are not the individual DRNGs but whole DRNG trees.* This is due to the fact that in a DRNG tree, security cannot be guaranteed locally by evaluating a DRNG and its direct seed successor. As already explained above, the output of the DRNGs can be compromised by security problems further up the DRNG tree. DRNG trees can be static or dynamic in the sense that DRNGs can be instantiated or uninstantiated during the lifetime of the DRNG tree.

- 174 [DRNG seeding DRNG, functionality class DRT.1] A seed graph can be a useful tool for the evaluation of a DRNG tree to illustrate the seeding and reseeding information of a DRNG tree, especially when compositions of DRNG trees with subtrees are concerned; for definition and examples cf. application notes par. 265 to par. 273. It is not necessary to explicitly set up and maintain the seed graph during the lifetime of the DRNG tree. By Requirement DRT.1.9 (cf. the specification of functionality class DRT.1) it suffices to keep “local information”, namely on the direct seed predecessor of a DRNG and the siblings of its direct seed predecessor. Note: In fact, in many scenarios maintaining a dynamic seed graph during the lifetime of a DRNG tree would at least be burdensome.
- 175 [DRNG seeding DRNG, DRT.1-compliant DRNG trees] Functionality class DRT.1 specifies requirements for DRNG trees. This paragraph summarizes central features; for details cf. functionality class DRT.1.
- (i) All nodes of the DRNG tree shall be algorithmically compliant with functionality class DRG.3.
 - (ii) The root DRNG shall either be seeded by a TRNG, or by a DRG.4-compliant DRNG that provides enhanced forward secrecy immediately before it generates the seed material. In particular, this includes functionality classes PTG.2, PTG.3, NTG.1, and DRG.4.
Note 1: Only the initial randomness source provides entropy. The other randomness sources in the DRNG tree are (DRG.3-compliant) DRNGs and thus cannot provide entropy.
Note 2: The initial randomness source is not part of the DRNG tree.
 - (iii) Each DRNG in the DRNG tree (except for the root DRNG) shall be seeded with seed material of length ≥ 256 bits. Since the seed material is generated by a DRNG, a substitute condition for Requirement DRG.3.4 is needed to prevent inappropriate seeding procedures or reseeding procedures. This is the motivation of Requirement DRT.1.7.
 - (iv) Apart from the root, each DRNG in the DRNG tree shall only be reseeded by its direct seed predecessor or by a sibling DRNG of its direct seed predecessor. The sibling node can be instantiated before or after the parent node.
 - (v) A (re-)seeded DRNG shall use the received seed material only for the seeding procedure or reseeding procedure.
 - (vi) Each DRNG in a DRNG tree can accept additional input, but it shall not be credited with entropy as for seeding or reseeding.
 - (vii) The RNG that generates seed material for the root DRNG (“initial randomness source”) and all DRNGs of the DRNG tree shall be implemented and operated inside the same security boundary.

Note 1: Requirement (iv), or more precisely, Requirement DRT.1.9, prevents seed loops; cf. par. 260. In particular, this eliminates the need to store a static or maintain a dynamic **seed graph** during the lifetime of the **DRNGs**. If only the seed predecessor is used for reseeding, it suffices that each **DRNG** “remembers” its **direct seed predecessor**. If sibling nodes of the direct predecessor can be used instead, this requires additional “local” information on the **DRNG tree**.
 Note 2: Although **additional input** is not credited with **entropy**, it may be advisable to use **additional input** that, e.g., stems from **non-physical noise sources**, even if the guaranteed amount of **entropy** is low.

Note 3: The security of the scenario “**DRNG seeding DRNG**” may depend on further aspects, which are “out of scope” for this document. This, in particular, refers to the secure transport of **seed material** from one **DRNG** to another **DRNG** (e.g., integrity, authenticity, confidentiality of the **seed material** as well as its secure deletion after the **seeding procedure** or the **reseeding procedure** has been performed). These security aspects shall be covered by the overall security evaluation of the product.

3.3.2 Functionality Class DRG.2

Functionality class DRG.2 defines requirements for **deterministic RNG**. 176

DRG.2-compliant **DRNGs** are primarily recommended for cryptographic applications for which the disclosure of previous random numbers due to a compromise of the **internal state** is not an issue (e.g., for challenges in challenge-response protocols). 177

Note: A compromise of the **internal state** constitutes a serious security incident that is supposed to be prevented by the overall **TOE**.

The **TOE Security Functionality (TSF)** has to protect the **internal state** of the **RNG** from being compromised. 178

Note: Par. 180 does not replace an evaluation methodology. 179

DRG.2-specific deliverables by the applicant 180

The security architecture description and developer evidence shall contain at least

- a formal description of the algorithmic behavior of the **DRNG** by a 9-tuple (3.1),
- a formal description of the **seeding procedure** (3.3) and (if applicable) the **reseeding procedure** (3.4),
- a description of how the **seed material** for the **seeding procedure** and (if applicable) **reseeding procedure** is generated (DRG.2.1),
- proofs that the **DRNG** design and the (re-)seeding **TRNG** fulfill Requirements DRG.2.1 to DRG.2.8.
- evidence that DRG.2.9 is fulfilled.

181

DRG.2: Security functional requirements

Security functional requirements of functionality class DRG.2 are defined by component **FCS_RNG.1** with specific operations as given below.

182 **FCS_RNG.1** Random number generation (Class DRG.2)

FCS_RNG.1.1 The **TSF** shall provide a *deterministic* random number generator that implements:

- (DRG.2.1) The *seed material* (for the *seeding procedure* and *reseeding procedure*) shall be generated by [selection, choose one of:
 - a **PTRNG** of class PTG.2,
 - a **PTRNG** of class PTG.3,
 - an **NPTRNG** of class NTG.1,
 - a [selection, choose one of: **PTRNG**, **NPTRNG**] that generates random bits with a *min-entropy* of [assignment: amount of *entropy*] per bit,
 - a **PTRNG** with *stationarily distributed raw random numbers* that generates random bits with a *Shannon entropy* of [assignment: amount of *entropy*] per bit,
 - a **DRNG** of class DRG.4].
- (DRG.2.2) The length of a single *request* shall comprise a maximum of 2^{19} bits.
- (DRG.2.3) The *effective internal state* shall comprise at least 246 bits.
- (DRG.2.4) The *seed material* shall be generated by [selection, choose one of:
 - a **TRNG** and the initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) shall have [selection: *min-entropy* ≥ 240 bits, *Shannon entropy* ≥ 250 bits],
 - a **DRNG** that is DRG.4-compliant and has been provided with *enhanced forward secrecy* before generating the *seed material*, and the following conditions shall be fulfilled: The *seed material* shall comprise ≥ 256 bits. If the *seed material* had ≥ 250 bits of *min-entropy*, then the initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) would have *min-entropy* ≥ 240 bits].If *Shannon entropy* is claimed, the *raw random numbers* of the *seed material*-generating **PTRNG** shall be *stationarily distributed*.
- (DRG.2.5) The **DRNG** shall provide *forward secrecy* on the *granularity level* of *internal random numbers*.
- (DRG.2.6) The **DRNG** shall provide *backward secrecy* on the *granularity level* of *internal random numbers*.
- (DRG.2.7) If applicable: *additional input* shall not weaken the strength of the **DRNG** even if an *adversary* is able to control the *additional input*.
- (DRG.2.8) The state transition function ϕ , the output function ψ , or both shall be *cryptographic*.

FCS_RNG.1.2 The **TSF** shall provide random numbers that meet:

(DRG.2.9) *The **internal random numbers** shall have **statistical inconspicuousness**.*

Application notes

[DRG.2.1] Usually, the **seeding procedure** and the **reseeding procedure** use a **PTRNG** that is 183
compliant with PTG.2 or PTG.3, or an **NPTRNG** that is compliant with NTG.1. The entropy
requirement per bit is meant to be understood as in par. 335, Note 1.

[DRG.2.1] It is permitted to use a **TRNG** that is not compliant with functionality classes PTG.2, 184
PTG.3, or NTG.1 to generate the **seed material**. Of course, the verification of Requirement
DRG.2.1 then usually requires significantly greater effort. In particular, the applicant has to
give evidence that the **seed material** (for the **seeding procedure** and **reseeding procedure**) indeed
contains the claimed amount of **entropy**. This includes a reliable verification that the **TRNG** is
working properly at the time when the **seed material** was generated; cf. par. 185 for an extreme
example. For functionality classes DRG.2 and DRG.3, a formal **stochastic model** for the seeding
PTRNG is recommended but not mandatory. Anyway, the **entropy** claim has to be explained
and justified, based on the design of the **physical noise source**. Without a **stochastic model**, more
conservative (pessimistic) **entropy** estimates shall be applied. **Black box** tests are not sufficient.
By default, **min-entropy** has to be claimed. Claiming **Shannon entropy** requires a **stochastic**
model, and the **raw random numbers** need to be (time-locally) **stationarily distributed** (cf. **time-**
local stationarity).

Note: We refrain from making a **stochastic model** mandatory, because for **NPTRNGs** a **stochastic**
model is not mandatory, either.

[DRG.2.1] Extreme example: Assume that a seeding **PTRNG** requires, for instance, a delay of 185
100.000 **internal random number** bits to reliably detect non-tolerable **entropy** defects or **total**
failures of the **noise source**. In this case, it would be acceptable to generate and to store the
seed material, apply an appropriate test on the next 100.000 random bits, and to output the
seed material in case of a positive test result. Of course, this **PTRNG** would not be compliant
with functionality class PTG.2, because the above-mentioned **online test** would be too slow for
general applications.

[DRG.2.1, DRG.4] If the **seed material** is generated by a DRG.4-compliant **DRNG**, this DRG.4- 186
compliant **DRNG** shall have been provided with **enhanced forward secrecy** before it generates
the **seed material**.

[DRG.2.2] For further explanations regarding **requests**, see pars. 106 to 114. Unlike for func- 187
tionality class DRG.3, for functionality class DRG.2, no **conceptual atomicity** condition (cf.
Requirement DRG.3.7) exists, because functionality class DRG.2 only ensures **backward secrecy**
and **forward secrecy** on the level of the **internal random numbers** (cf. the Requirements DRG.2.5
and DRG.2.6). The **conceptual atomicity** of a **request** is only relevant in the context of **enhanced**
backward secrecy.

[DRG.2.2] Requirement DRG.2.2 makes a counter for the number of **internal random numbers** 188
necessary per **request**.

[DRG.2.3] The **effective internal state** and its size shall be determined under the assumption that the **adversary** knows a large number of **internal random numbers**. In particular, the following (conceivable) reasoning *will not be accepted* for verifying Requirement DRG.2.3: Since immediately after the **seeding procedure/reseeding procedure** an **adversary** did not have a chance to collect any information about the **DRNG**, any part of the **internal state** (apart from publicly known input) is unknown and thus effective.

- 190 For the **reseeding procedure**, the least favorable scenario shall be assumed where an **adversary** knows the previous **internal state**.
- 191 [DRG.2.4] The verification of Requirement DRG.2.4 is split into two steps. In the first step, a lower **entropy** bound for the **seed material** has to be determined. If a certified **TRNG** compliant with functionality class PTG.2, PTG.3, or NTG.1 is used for the **seeding procedure** or **reseeding procedure**, this is an easy task. For functionality class PTG.2, Requirement PTG.2.3 ensures that the **Shannon entropy** per bit is ≥ 0.9998 if a **Shannon entropy** claim has been certified, and if a **min-entropy** claim has been certified, the **min-entropy** per bit is ≥ 0.98 . For functionality class PTG.3, the applicant has several options to claim **PTRNG**-specific **entropy** bounds, either in terms of **Shannon entropy**, **min-entropy**, or both (cf. Requirement PTG.3.5). If the **seeding procedure** or the **reseeding procedure** applies a PTG.3-compliant **PTRNG** without a specific **entropy** claim (corresponding to the first three selections in Requirement PTG.3.5), the **entropy** claims of the **intermediate random numbers** can be used instead (i.e., ≥ 0.9998 bit **Shannon entropy** and/or ≥ 0.98 bit **min-entropy**), although the **cryptographic post-processing** may increase the **entropy** defect per bit. For functionality class NTG.1, only **min-entropy** is permitted. If a **TRNG** is used that is not compliant with functionality classes PTG.2, PTG.3, or NTG.1, this requires (at least a partial) evaluation of the **TRNG** to derive a reliable lower **entropy** bound per bit.
- 192 [DRG.2.4] To verify Requirement DRG.2.4, after having determined a lower **entropy** bound for the **seed material** (cf. par. 191), the impact of the **seeding procedure** or **reseeding procedure** on the **entropy** on the **effective internal state** has to be taken into account. This can be very easy; cf., e.g., the example discussed in Subsection 5.2.1. If the **seeding procedure** and the **reseeding procedure** apply cryptographic mechanisms, the task can be more challenging. If the **seed material** has (a lot of) surplus **entropy**, this may allow rough estimates that facilitate the task. If necessary, the developer and the evaluator can apply more sophisticated methods as they are used to verify **entropy** claims for functionality class PTG.3; cf. the application notes to Requirement PTG.3.5 and there in particular par. 410.
- 193 [DRG.2.4] For **PTRNGs** that belong to functionality classes PTG.2 or PTG.3, the **raw random numbers** satisfy the **time-local stationarity** condition; cf. Requirements PTG.2.1 and PTG.3.1. In particular, **Shannon entropy** can be claimed in place of **min-entropy**.
- 194 [DRG.2.4] If the **seed material** is generated by a DRG.4-compliant **DRNG**, this **DRNG** shall be reseeded or high **entropy additional input** shall be included immediately before generating the **seed material**. This means that the DRG.4-compliant **DRNG** has been provided **enhanced forward secrecy**. Relative to the **internal random numbers** that the DRG.4-compliant **DRNG** has previously output, the situation is close to the case where a **TRNG** generates the **seed material**. Usually, the **entropy** requirements for **TRNGs** (cf. Requirement DRG.2.4) should be fulfilled, too, but this cannot be guaranteed in a strict sense. In fact, Requirements DRG.4.4 and DRG.4.10

ensure that after providing **enhanced forward secrecy**, the **effective internal state** of the DRG.4-compliant **DRNG** has 240 bits of **min-entropy** (or, if applicable, alternatively 250 bits **Shannon entropy**) against an **adversary** who knows the previous **internal state** (cf. par. 190). However, to be precise, (re-)seeding another **DRNG** usually causes a loss of a few bits of **entropy**. This is the reason for the substitute condition in Requirement DRG.2.4 when the **seed material** is generated by a DRG.4-compliant **DRNG**.

Example: The Hash_ DRBG and the HMAC_ DRBG fulfill this requirement if they are used with a hash function listed in (5.34) or (5.81), respectively; cf. pars. 1074, Note 1, and 1113, Note 3.

Note: With regard to the future output of the **seed material**-generating DRG.4-compliant **DRNG**, there is no equivalence to **TRNGs** in the sense of information theory unless this DRG.4-compliant **DRNG** would be reseeded immediately after it has generated the **seed material**. For reasons of efficiency, functionality classes DRG.2, DRG.3, and DRT.1 resign on such a requirement. This is justified by the fact that the DRG.4-compliant **DRNG** provides **computational security** (**enhanced backward secrecy**), and also the (re-)seeded **DRNG** can only provide **computational security**.

[DRG.2.[5,6,7]] The Requirements DRG.2.5, DRG.2.6, and DRG.2.7 shall be guaranteed by the algorithmic properties of the **DRNG**, i.e., by the interaction of the state transition function and the output function. A lack of algorithmic properties cannot be compensated for by other measures, e.g., by high-**entropy additional input**. 195

[DRG.2.[5,6]] The **DRNG** may support generating output of variable length (by concatenating **internal random numbers**, cf. par. 131). Irrespective of that, DRG.2.5 and DRG.2.6 only require **forward secrecy** and **backward secrecy** on the **granularity level** of the **internal random numbers**. This means the following: Assume that an **adversary** knows a sequence of **internal random numbers** generated within one or several **requests**. The sequence need not start nor terminate a **request**. The task of the **adversary** is to compute or guess the **internal random number** that follows or precedes this sequence. The **security strength** (of cryptographic mechanisms) should be considered in the way explained in pars. 121 and 122. 196

[DRG.2.[5,6]] The focus on **internal random numbers** in DRG.2.5 and DRG.2.6 (instead of considering strings of **internal random numbers** bits of arbitrary length) will simplify the evaluation. The focus on **internal random numbers** is motivated by the fact that the **internal random numbers** are the basic building blocks of a **request**. Thus, **forward secrecy** and **backward secrecy** should extend from the **granularity level** of **internal random numbers** to the **granularity level** of strings of **internal random number** bits of arbitrary length with corresponding **computational security** strength. It is part of the evaluation to identify “pathological” **DRNG** designs for which this is not the case. 197

[DRG.2.7] The term “if applicable” in Requirement DRG.2.7 refers to **DRNGs** that accept **additional input**. 198

[DRG.2.8] In case of doubt, the certification body decides whether a function is considered **cryptographic**. 199

[DRG.2.9] This conclusion shall be based on theoretical considerations, theoretical considerations supported by statistical tests, or statistical tests with justification of the choice. Furthermore, we refer to pars. 157 to 162. 200

3.3.3 Functionality Class DRG.3

201 Functionality class DRG.3 defines requirements for **deterministic RNG**. The differences from functionality class DRG.2 are explained in par. 210.

202 DRG.3-compliant **DRNGs** are suitable for all cryptographic applications except for those that require guaranteed **fresh entropy**.

203 The **TSF** has to protect the **internal state** of the **RNG** from being compromised.

204 Note: Par. 205 does not replace an evaluation methodology.

205 **DRG.3-specific deliverables by the applicant**

The security architecture description and developer evidence shall contain at least

- a formal description of the algorithmic behavior of the **DRNG** by a 9-tuple (3.1),
- a formal description of the **seeding procedure** (3.3) and (if applicable) the **reseeding procedure** (3.4),
- a description of how the **seed material** for the **seeding procedure** and (if applicable) **reseeding procedure** is generated (DRG.3.1),
- proofs that the **DRNG** design and the seeding **TRNG** fulfill Requirements DRG.3.1 to DRG.3.9.
- evidence that DRG.3.10 is fulfilled.

206 **DRG.3: Security functional requirements**

Security functional requirements of functionality class DRG.3 are defined by component **FCS_RNG.1** with specific operations as given below.

207 **FCS_RNG.1** Random number generation (Class DRG.3)

FCS_RNG.1.1 The **TSF** shall provide a *deterministic* random number generator that implements:

(DRG.3.1) *The **seed material** (for the **seeding procedure** and **reseeding procedure**) shall be generated by [selection, choose one of:*
*a **PTRNG** of class PTG.2,*
*a **PTRNG** of class PTG.3,*
*an **NPTRNG** of class NTG.1,*
*a [selection, choose one of: **PTRNG**, **NPTRNG**] that generates random bits with a **min-entropy** of [assignment: amount of **entropy**] per bit,*
*a **PTRNG** with **stationarily distributed raw random numbers** that generates random bits with a **Shannon entropy** of [assignment: amount of **entropy**] per*

bit,
a *DRNG* of class DRG.4].

(DRG.3.2) The length of a single *request* shall comprise a maximum of 2^{19} bits.

(DRG.3.3) The *effective internal state* shall comprise at least 246 bits.

(DRG.3.4) The *seed material* shall be generated by [selection, choose one of:
a *TRNG* and the initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) shall have [selection: *min-entropy* $\geq$ 240 bits, *Shannon entropy* $\geq$ 250 bits],
a *DRNG* that is DRG.4-compliant and has been provided with *enhanced forward secrecy* before generating the *seed material*, and the following conditions shall be fulfilled: The *seed material* shall comprise $\geq$ 256 bits. If the *seed material* had $\geq$ 250 bits of *min-entropy*, then the initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) would have *min-entropy* $\geq$ 240 bits].
If *Shannon entropy* is claimed, the *raw random numbers* of the *seed material*-generating *PTRNG* shall be *stationarily distributed*.

(DRG.3.5) The *DRNG* shall provide *forward secrecy* on the *granularity level* of *internal random numbers*.

(DRG.3.6) The *DRNG* shall provide *backward secrecy* on the *granularity level* of *internal random numbers*.

(DRG.3.7) The *DRNG* shall provide [selection, choose one of: *enhanced backward secrecy* on the *granularity level* of *internal random numbers*, *enhanced backward secrecy* on the *granularity level* of *requests* and the *requests* shall satisfy the condition of *conceptual atomicity*].

(DRG.3.8) If applicable: *additional input* shall not weaken the strength of the *DRNG* even if an *adversary* is able to control the *additional input*.

(DRG.3.9) The state transition function ϕ and the output function ψ shall be *cryptographic*. The state transition function shall be a *one-way function*.

FCS_RNG.1.2 The *TSF* shall provide random numbers that meet:

(DRG.3.10) The *internal random numbers* shall have *statistical inconspicuousness*.

Application notes

[DRG.3 vs. DRG.2] Class DRG.3 includes the requirements of class DRG.2. The Requirements 208
DRG.2.1 and DRG.3.1, DRG.2.2 and DRG.3.2, DRG.2.3 and DRG.3.3, DRG.2.4 and DRG.3.4,
DRG.2.5 and DRG.3.5, DRG.2.6 and DRG.3.6, DRG.2.7 and DRG.3.8, DRG.2.9 and DRG.3.10
coincide.

[DRG.3 vs. DRG.2] Therefore, the corresponding application notes 183, 184, 185, 186, 188, 189, 209
190, 191, 192, 193, 194, 195, 196, 197, 198, 199, and 200 are valid for DRG.3.y instead of DRG.2.x
as well, where x and y correspond as described in par. 208.

[DRG.3 vs. DRG.2] In addition to the DRG.2 requirements, functionality class DRG.3 requires **enhanced backward secrecy** (DRG.3.7), and DRG.3.9 extends DRG.2.8.

- 211 [DRG.3.[5,6,7,8]] Same as in par. 195, the Requirement DRG.3.5 to DRG.3.8 shall be guaranteed by the algorithmic properties of the **DRNG**, i.e., by the interaction of the state transition function and the output function. Missing algorithmic properties cannot be compensated by other measures, e.g., by **high-entropy additional input**
- 212 [DRG.3.7] **Enhanced backward secrecy** (DRG.3.7) is an algorithmic property. It cannot be compensated for or supported by technical security measures that (are claimed to) prevent the **internal state** from being compromised or modified. Clause DRG.3.7 essentially requires a state transition function that is a **one-way function** concerning an **adversary** who knows the **internal state** and (if relevant) the last **additional input**. The **security strength (of cryptographic mechanisms)** should be as explained in pars. 121 and 122.
- 213 [DRG.3.7] The **DRNG** may support output of variable length (by concatenating **internal random numbers**, cf. par. 131). Clause DRG.3.7 requires **enhanced backward secrecy** on the **granularity level of requests**. (If the **DRNG** outputs **requests** that do not comprise more than one **internal random number**, then DRG.3.7 trivially guarantees **enhanced backward secrecy** on the **granularity level of internal random numbers**.) This means the following: Assume that an **adversary** gains access to the current **internal state** and (if applicable) to the **additional input** during the previous **request**. In this case, Requirement DRG.3.7 prevents an **adversary** from computing or guessing **internal random numbers** from previous **requests**. However, Requirement DRG.3.7 does not prevent an **adversary** who has learned the **internal state** before the state transition function has been applied to compute or to guess all **internal random numbers** of this **request**. The impact of this attack is mitigated by the **conceptual atomicity** condition (primarily) and by the length restriction of a **request**. If the **DRNG** provides **enhanced backward secrecy** on the **granularity level of internal random numbers**, the **conceptual atomicity** condition is dropped.
- Note 1: Obviously, **enhanced backward secrecy** on the **granularity level of internal random numbers** implies **enhanced backward secrecy** on the **granularity level of requests**.
- Note 2: **Enhanced backward secrecy** on the **granularity level of internal random numbers** can be ensured by using a **one-way function** as a **request** state transition function ϕ_0 , or by design, if the **DRNG** allows only **requests** that comprise one **internal random number**.
- 214 [DRG.3.7] Size and **entropy** requirements for the **effective internal state** shall prevent successful attacks against **forward secrecy** and **backward secrecy**. Concerning **enhanced backward secrecy**, by assumption, the **adversary** knows the current **internal state**. However, attacking the **enhanced backward secrecy** shall be about as difficult as attacks against **forward secrecy** and **backward secrecy**. This means that computing or guessing n bits of previously generated **internal random number** bits is at least about a $\min\{240, n\}$ bit problem (non-quantum **attacker**). For an **adversary** with access to a (sufficiently powerful) quantum computer, the effort can be smaller (e.g., by Grover's algorithm), but a successful attack shall still be infeasible.
- 215 [DRG.3.9] For functionality class DRG.3, both the state transition function ϕ and the output function ψ shall be **cryptographic**. Additionally, ϕ shall be a **one-way function**. We refer the reader to pars. 97 to 105. Note that the final decision on whether a function is considered **cryptographic** rests with the certification body.

3.3.4 Functionality Class DRG.4

Functionality class DRG.4 defines requirements for **deterministic RNG**. These requirements can only be fulfilled by **hybrid DRNGs**. The differences from functionality class DRG.3 are explained in pars. 225 and 227. 216

DRG.4-compliant **DRNGs** are suitable for all cryptographic applications except for those that require a **TRNG**. 217

DRG.4-compliant **DRNGs** have access to a **PTRNG**, and are able to trigger the **seeding procedure**, **reseeding procedures**, and/or the insertion of high-**entropy additional input**. Furthermore, the **additional input** may also include data from sources without an **entropy** claim. 218

The **TSF** has to protect the **internal state** of the **RNG** from being compromised. 219

Note: Par. 221 does not replace an evaluation methodology. 220

DRG.4-specific deliverables by the applicant 221

The security architecture description and developer evidence shall contain at least

- a formal description of the algorithmic behavior of the **DRNG** by a 9-tuple (3.1),
- a formal description of the **seeding procedure** (3.3) and (if applicable) the **reseeding procedure** (3.4),
- a specification of the internal **PTRNG** and the mechanisms to trigger a **seeding procedure** and/or a **reseeding procedure**, and/or to obtain high-**entropy additional input**,
- a description of how the **seed material** for the **seeding procedure** and (if applicable) **reseeding procedure** is generated (DRG.4.1),
- proofs that the **DRNG** design and the internal **PTRNG** fulfill Requirements DRG.4.1 to DRG.4.10
- evidence that DRG.4.11 is fulfilled.

DRG.4: Security functional requirements 222

Security functional requirements of functionality class DRG.4 are defined by component **FCS_RNG.1** with specific operations as given below.

FCS_RNG.1 Random number generation (Class DRG.4) 223

FCS_RNG.1.1 The **TSF** shall provide a *hybrid deterministic* random number generator that implements:

- (DRG.4.1) The *seed material* (for the *seeding procedure* and *reseeding procedure*) and (if applicable) the *high-entropy additional input* shall be generated by [selection, choose one of:
a *PTRNG* of class PTG.2,
a *PTRNG* of class PTG.3,
a *PTRNG* that generates random bits with a *min-entropy* of [assignment: amount of *entropy*] per bit,
a *PTRNG* with *stationarily distributed raw random numbers* that generates random bits with a *Shannon entropy* of [assignment: amount of *entropy*] per bit].
- (DRG.4.2) The length of a single *request* shall comprise a maximum of 2^{19} bits.
- (DRG.4.3) The *effective internal state* shall comprise at least 246 bits.
- (DRG.4.4) The initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) shall have [selection: *min-entropy* ≥ 240 bits, *Shannon entropy* ≥ 250 bits].
If *Shannon entropy* is claimed, the *raw random numbers* of the *seed material*-generating *PTRNG* shall be *stationarily distributed*.
- (DRG.4.5) The *DRNG* shall provide *forward secrecy* on the *granularity level* of *internal random numbers*.
- (DRG.4.6) The *DRNG* shall provide *backward secrecy* on the *granularity level* of *internal random numbers*.
- (DRG.4.7) The *DRNG* shall provide [selection, choose one of: *enhanced backward secrecy* on the *granularity level* of *internal random numbers*, *enhanced backward secrecy* on the *granularity level* of *requests* and the *requests* satisfy the condition of *conceptual atomicity*].
- (DRG.4.8) If applicable: *additional input* shall not not weaken the strength of the *DRNG* even if an *adversary* is able to control the *additional input*.
- (DRG.4.9) The state transition function ϕ and the output function ψ shall be *cryptographic*. The state transition function shall be a *one-way function*.
- (DRG.4.10) The *DRNG* shall provide *enhanced forward secrecy* [selection: on demand, on condition [assignment: condition], after [assignment: time]]. This is achieved by the *reseeding procedure* (cf. DRG.4.4) or by *high-entropy additional input* such that the *effective internal state* has [selection: *min-entropy* ≥ 240 bits, *Shannon entropy* ≥ 250 bits]. If *Shannon entropy* is claimed, the *raw random numbers* of the *seed material*-generating *PTRNG* shall be *stationarily distributed*.
The *DRNG* may apply both above-mentioned methods. Minimum requirement: Until the next *reseeding procedure*, or the next *high-entropy additional input* at most 2^{17} *internal random number* bits shall be generated.

FCS_RNG.1.2 The *TSF* shall provide random numbers that meet:

- (DRG.4.11) The *internal random numbers* shall have *statistical inconspicuousness*.

Application notes

- [DRG.4 vs. DRG.3 vs. DRG.2] Class DRG.4 includes the requirements of class DRG.3 and thus also of DRG.2. The Requirements DRG.2.2 and DRG.4.2, DRG.2.3 and DRG.4.3, DRG.2.4 and DRG.4.4, DRG.2.5 and DRG.4.5, DRG.2.6 and DRG.4.6, DRG.2.7 and DRG.4.8, DRG.2.9 and DRG.4.11 coincide. Furthermore, DRG.3.7 and DRG.4.7, DRG.3.9 and DRG.4.9 coincide. 224
- [DRG.4 vs. DRG.3 vs. DRG.2] Requirement DRG.4.1 limits the selection of a **TRNG** in DRG.2.1 and the selection of a **TRNG** or **DRNG** in DRG.3.1 to **PTRNGs** for functionality class DRG.4. 225
- [DRG.4 vs. DRG.3 vs. DRG.2] Thus, the application notes 187, 188, 189, 190, 191, 192, 193, 195, 196, 197, 198, 199, and 200 remain valid if we replace DRG.2.x by DRG.4.y with regard to the correspondences from par. 224. Moreover, the application notes 210, 211, 212, 213, 214, and 215 remain valid if one substitutes DRG.3.x by DRG.4.x. 226
- [DRG.4 vs. DRG.3] Class DRG.4 includes the requirements of class DRG.3. Additionally, DRG.4 requires that the **DRNG** has the capability to ensure **enhanced forward secrecy** (DRG.4.10). 227
- [DRG.4.1] Functionality class DRG.4 requires a **PTRNG** for the **seeding procedure** (DRG.4.4), the **reseeding procedure** (DRG.4.4, DRG.4.10), and for high-**entropy additional input** (DRG.4.10). Unlike for functionality classes DRG.2 and DRG.3, **NPTRNGs** and **DRNGs** are not allowed. This is justified by the fact that for **NPTRNGs** the environment, the platform, etc. are not under the control of the designer or evaluator. Moreover, the devices on which **NPTRNGs** run (PCs, servers, mobile devices, etc.) are usually more vulnerable to implementation attacks than, e.g., smart cards; cf. Subsection 3.5.1. For these reasons, in general, our trust in **NPTRNGs** is lower than our trust in **PTRNGs**. 228
- [DRG.4.1] Usually, the **seed material** (for the **seeding procedure** or **reseeding procedure**) and high-**entropy additional input** is generated with a **PTRNG** that is compliant with functionality classes PTG.2 or PTG.3. 229
- [DRG.4.1] It is permitted to use a **PTRNG** that is not compliant with PTG.2 or PTG.3. In this case, the verification of Requirement DRG.4.1 usually requires significantly greater effort than if a certified **PTRNG** is used that is compliant with functionality class PTG.2 or PTG.3. In particular, the applicant has to give evidence that the **seed material** (for the **seeding procedure** and **reseeding procedure**) contains the claimed amount of **entropy**. This includes evidence that the **PTRNG** is working properly at the time of the **seeding procedure**/**reseeding procedure**. Unlike for functionality classes DRG.2 and DRG.3, a **stochastic model** of the **PTRNG** is mandatory. Claiming **Shannon entropy** requires that the **raw random numbers** are (time-locally) **stationarily distributed** (cf. **time-local stationarity**). 230
- [DRG.4.4, DRG.4.10] Two methods are allowed for mixing **fresh entropy** into the **internal state**: by reseeding and by inserting high-**entropy additional input**. The requirements are essentially identical and comply with those for seeding; cf. DRG.4.4, DRG.4.10 231
- [DRG.4.4, DRG.4.10] In the context of functionality class DRG.4, the intuitive but imprecise wording “high **entropy additional input**” means that **enhanced forward secrecy** is ensured by 232

additional input. This can only be achieved in combination with an appropriate state transition function but implies that the **additional input** must have at least 240 bits of **min-entropy**, or, if the substitution condition applies, at least 250 bits of **Shannon entropy**.

- 233 [DRG.4.10] The **entropy** claim in Requirement DRG.4.10 refers to the least favorable case where an **adversary** knows the previous **internal state**; cf. par. 190.
- 234 [DRG.4.10] **Enhanced forward secrecy** can only be achieved for **internal random numbers** that are generated after the next **reseeding procedure** or the next mixing of high-**entropy additional input** into the **internal state** by the state transition function. If high-**entropy additional input** is used, in general the **internal random numbers** of the **request** in which high-**entropy additional input** is mixed in, do not provide **enhanced forward secrecy**. Designs exist, for which these **internal random numbers** are already provided with **enhanced forward secrecy**. This is the case if ϕ_{req} (the function that generates S_{req}), the **request** state transition function ϕ_0 and the output function ψ “mix” the high-**entropy additional input** suitably into the generation of **internal random numbers**, providing them with sufficient **entropy**. Such examples are the Hash_ DRBG and the HMAC_ DRBG; see (5.45), (5.46), (5.47), and (5.95), (5.96), and (5.97). If this is not the case, the **DRNG** shall apply the state transition function ϕ first (corresponds to an “empty request”).
- 235 [DRG.4.10, high-**entropy additional input**] If requirement **enhanced forward secrecy** is intended to be achieved by high-**entropy additional input**, the same **PTRNG** that was used for the **seeding procedure** and (if applicable) the **reseeding procedures** shall be used. In particular, Requirement DRG.4.1 applies. This is unlike the case of using “arbitrary” **additional input**, which is covered by DRG.4.8.
- 236 [DRG.4.10, high-**entropy additional input**] When achieving **enhanced forward secrecy** by high-**entropy additional input**, the requirements are rather similar to that for the **reseeding procedure**, although, of course, the state transition function is applied in place of the (re-)seeding procedure. **Additional input** strings derived from both reliable **RNGs** (e.g., from a PTG.2-compliant **PTRNG**) and “arbitrary” **noise sources** (e.g., time stamps) are permitted. Of course, **additional input** from “arbitrary” **noise sources** may also contribute some **entropy**, but this cannot be taken into account for the evaluation of Requirement DRG.4.10. **Additional input** that is not high **entropy additional input** is not credited in the security analysis.
- 237 [DRG.4.10] Like the **seed material** in the **seeding procedure** and **reseeding procedure**, the high-**entropy additional input** data (DRG.4.10) must also be protected to ensure secrecy, integrity, and authenticity; cf. application note 386. The verification of these properties is part of the overall evaluation of the **TOE**.
- 238 [DRG.4.10] **Enhanced forward secrecy** “on demand” is triggered by the requesting application. Requirement DRG.4.10 defines a condition that always has to be considered, namely that at most 2^{17} **internal random number** bits shall be generated before the next **reseeding procedure** or the next high-**entropy additional input**; cf. par. 239. The **DRNG** can apply further design-specific “on condition” requirements, e.g., realized by a smaller number of **internal random number** bits than 2^{17} . Finally, “after time” conditions require reliable time measurement (real-time). Note: 2^{17} **internal random number** bits correspond to 512 **internal random numbers** that consist of 256 bits each or 1024 **internal random numbers** that consist of 128 bits each, for example.

[DRG.4.10] It is not necessary to interrupt an ongoing **request** when 2^{17} **internal random number** bits have been generated since the previous **high-entropy additional input**, since the previous **reseeding procedure** (or since the **seeding procedure**). The current **request** can be completed, but the **DRNG** shall receive fresh **high-entropy additional input** or shall be reseeded before it generates further output.
 Note: For large **requests**, Requirement DRG.4.2 can dominate the “ 2^{17} bit condition”.

3.3.5 Functionality Class DRT.1

Functionality class DRT.1 defines requirements for **deterministic RNG** trees (short: **DRNG trees**, see pars. 165 ff for the notation used in this section).

DRT.1-compliant **DRNG trees** are an option for software implementations that need several **DRNGs**, although not all of them have access to a **TRNG**.

The **TSF** has to protect the **internal states** of all **DRNGs** in the **DRNG tree** from being compromised.

Note: Par. 244 does not replace an evaluation methodology.

DRT.1-specific deliverables by the applicant

The security architecture description and developer evidence shall contain at least

- proofs (including design and implementation rules) that Requirements DRT.1.1, DRT.1.2, and DRT.1.9 are fulfilled,
- for all appearing **DRNG** designs, a formal description of their algorithmic behavior by 9-tuples (3.1),
- for all appearing **DRNG** designs, a formal description of their **seeding procedure** (3.3) and (if applicable) the **reseeding procedure** (3.4),
- a description of how the **seed material** for the **seeding procedure** and (if applicable) **reseeding procedure** of the root **DRNG** is generated (DRT.1.3),
- for all appearing **DRNG** designs, proofs that the Requirements DRT.1.4 to DRT.1.8 and DRT.1.10 to DRT.1.14 are fulfilled,
- for all appearing **DRNG** designs, evidence that DRT.1.15 is fulfilled.

DRT.1: Security functional requirements

Security functional requirements of functionality class DRT.1 are defined by component **FCS_RNG.1** with specific operations as given below.

FCS_RNG.1 Random number generation (Class DRT.1)

- FCS_RNG.1.1 The **TSF** shall provide a *deterministic* random number generator that implements:
- (DRT.1.1) The **TSF** shall implement a **DRNG tree** that consists of ≥ 1 **DRNGs**, together with the relation “is *direct seed predecessor*”. One of these **DRNGs**, called root **DRNG**, shall be distinguished and remain distinguished during the lifetime of the **DRNG tree**. During the lifetime of the **DRNG tree**, all **DRNGs** can be instantiated and uninstantiated. The root **DRNG** shall not be replaced during the lifetime of the **DRNG tree**.
 - (DRT.1.2) The **RNG** that generates *seed material* for the root **DRNG** (“initial randomness source”) and all **DRNGs** of the **DRNG tree** shall be implemented and operated inside the same *security boundary*.
 - (DRT.1.3) The *seed material* for the root **DRNG** (for the *seeding procedure* and *reseeding procedures*) shall be generated by [selection, choose one of:
 - a **PTRNG** of class PTG.2,
 - a **PTRNG** of class PTG.3,
 - an **NPTRNG** of class NTG.1,
 - a [selection, choose one of: **PTRNG**, **NPTRNG**] that generates random bits with a *min-entropy* of [assignment: amount of *entropy*] per bit,
 - a **PTRNG** with *stationarily distributed raw random numbers* that generates random bits with a *Shannon entropy* of [assignment: amount of *entropy*] per bit,
 - a **DRNG** of class DRG.4].
 - (DRT.1.4) For each **DRNG** in the **DRNG tree**, the length of a single *request* shall comprise at most 2^{19} bits.
 - (DRT.1.5) For each **DRNG** in the **DRNG tree**, the *effective internal state* shall comprise at least 246 bits.
 - (DRT.1.6) The *seed material* for the root **DRNG** shall be generated by [selection, choose one of:
 - a **TRNG**, and the initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) shall have [selection: *min-entropy* ≥ 240 bits, *Shannon entropy* ≥ 250 bits],
 - a **DRNG** that is DRG.4-compliant and has been provided with *enhanced forward secrecy* before generating the *seed material*, and the following condition shall be fulfilled: The *seed material* shall comprise ≥ 256 bits. If the *seed material* had ≥ 250 bits of *min-entropy*, then the initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) would have *min-entropy* ≥ 240 bits].
 - (DRT.1.7) For each **DRNG** in the **DRNG tree**, except the root **DRNG**, the *seed material* shall be generated by a **DRNG** in the **DRNG tree**, and the following conditions shall be fulfilled: The *seed material* shall comprise ≥ 256 bits. If the *seed material* had ≥ 250 bits of *min-entropy*, then the initial *effective internal state* (after the *seeding procedure* or the *reseeding procedure*) would have *min-entropy* ≥ 240 bits.
 - (DRT.1.8) Each **DRNG** in the **DRNG tree** shall use the received *seed material* only for the *seeding procedure* or the *reseeding procedure*.

- (DRT.1.9) For each *DRNG* in the *DRNG tree*, except the root *DRNG*, the *seed material* for *reseeding procedures* shall be generated by the same *DRNG* that has generated the *seed material* for the *seeding procedure*, i.e., by the *direct seed predecessor* (parent node in the *seed graph*), or, alternatively, by a sibling *DRNG* of the *direct seed predecessor*.
- (DRT.1.10) Each *DRNG* in the *DRNG tree* shall provide *forward secrecy* on the *granularity level* of *internal random numbers*.
- (DRT.1.11) Each *DRNG* in the *DRNG tree* shall provide *backward secrecy* on the *granularity level* of *internal random numbers*.
- (DRT.1.12) Each *DRNG* in the *DRNG tree* shall either provide *enhanced backward secrecy* on the *granularity level* of *internal random numbers*, or *enhanced backward secrecy* on the *granularity level* of *requests* and the *requests* shall satisfy the condition of *conceptual atomicity*.
- (DRT.1.13) For each *DRNG* in the *DRNG tree* (if applicable), *additional input* shall not weaken the strength of the *DRNG* even if an *adversary* is able to control the *additional input*.
- (DRT.1.14) For each *DRNG* in the *DRNG tree*, the state transition function ϕ and the output function ψ shall be *cryptographic*. The state transition function shall be a *one-way function*.

FCS_RNG.1.2 The *TSF* shall provide random numbers that meet:

- (DRT.1.15) For each *DRNG* in the *DRNG tree*, the *internal random numbers* shall have *statistical inconspicuousness*.

Application notes

[General aspects] The *DRNG tree* consists of a set of *DRNGs*, together with the information about the *direct seed predecessors* (and, thereby, also about the direct seed successors). This set can be static or dynamic. If it is static, no new *DRNGs* are instantiated after the instantiation of the *DRNG tree*, and no *DRNGs* are uninstantiated before the whole *DRNG tree* is uninstantiated. In the best case, the structure of the *DRNG tree* is already known at the time of evaluation, which makes its evaluation easier. *DRNG trees* can be highly dynamic, instantiating and uninstantiating many *DRNGs* during their lifetime. The *RNG* that generates *seed material* for the root *DRNG*, also called “initial randomness source” (of the *DRNG tree*), is not part of the *DRNG tree*. This is analogous to functionality classes DRG.2 and DRG.3. 247

[General aspects] A *DRNG tree* can be interpreted as a big *deterministic RNG* with several interfaces to the particular *DRNGs*, which justifies choosing “deterministic random number generator” in FCS_RNG.1.1. 248

[General aspects] In a *DRNG tree* the security of the outputted *internal random numbers* cannot be guaranteed “locally” by an isolated evaluation of the output-generating *DRNG*, including its algorithmic properties, the *seeding procedure*, and the *reseeding procedure*. Security breaches further up the *DRNG tree* (i.e., towards the root; cf. par. 170) can lead to security problems of 249

the **DRNG** under consideration. If such a security breach leads to a compromise of its **internal state**, the generated **internal random numbers** are totally insecure.

- 250 [General aspects] Therefore, **DRNG trees** as a whole and not the individual **DRNGs** are the objects of functionality class DRT.1. DRT.1-compliant **DRNG trees** shall output **internal random numbers** that provide **forward secrecy**, **backward secrecy**, and **enhanced backward secrecy**.
- 251 [General aspects] DRT.1.1, DRT.1.2, and DRT.1.9 are “global requirements”, whereas the other requirements are “local requirements” (concerning the individual **DRNGs**).
- 252 [General aspects] Especially, if the **DRNG tree** is dynamic, at the time of evaluation, the evaluator does not know the exact structure of the **DRNG tree** (which can change significantly) during its lifetime. Design and implementation rules of the **TOE** shall guarantee that, at any time, the **DRNG tree** (in particular) satisfies the “global” Requirements DRT.1.1, DRT.1.2, and DRT.1.9.
- 253 [General aspects, **seed graph**] The **seed graph** contains the seed and reseed history of the **DRNG tree**. The **seed graph** describes where the **DRNGs** in the **DRNG tree** have received their **seed material** from when they were instantiated, including those **DRNGs** that have already been uninstantiated. The color of the nodes says which **DRNGs** still exist and which **DRNGs** have already been uninstantiated. Furthermore, the **seed graph** shows which **DRNGs** other than the **direct seed predecessor** have generated **seed material** for reseeding. For a more comprehensive treatment of **seed graphs** with illustrative examples, cf. pars. 167 ff.
- 254 [General aspects, **seed graph**] A **seed graph** can be a helpful tool for the evaluation of a **DRNG tree** to illustrate the seeding and reseeding information of a **DRNG tree**, especially when compositions of **DRNG trees** with **DRNG** subtrees are concerned. By Requirement DRT.1.9, it suffices that each **DRNG** “knows” its **direct seed predecessor** and, possibly, the siblings of the **direct seed predecessor**. It is not necessary to maintain the **seed graph** during the lifetime of the **DRNG tree**, which would be a challenging task for very dynamic **DRNG trees**.
- 255 [DRT1.1] The root **DRNG** is an essential component of a **DRNG tree**. Thus, the root **DRNG** shall not be intentionally uninstantiated. If the root **DRNG** is accidentally inaccessible, e.g., due to a failure of the operating system, one may expect that the whole **DRNG** is inaccessible, which would end the lifetime of the **DRNG tree**. If this should not be the case, i.e., that parts of the **DRNG tree** are accessible but not its root **DRNG**, the root **DRNG** shall not be replaced by another **DRNG**, not even with a new instance of itself. If possible, it is recommended to uninstantiate the remaining parts of the **DRNG tree**.
- 256 [DRT1.2] Requirement DRT.1.2 formulates restrictions regarding the implementation of **DRNG trees**. These requirements are intended to limit the potential security risks of the **DRNG tree**. Possibly counter-examples for this requirement include the following.
- The **DRNG tree** is distributed over multiple computing platforms belonging to different operators with different (incompatible) **security boundaries**.
 - The **internal state** of **DRNG** is backed up to a storage location outside the **security boundary**.

- The **seed material** for a **DRNG** within the **DRNG tree** is generated outside the **security boundary**.
- The communication between **DRNGs** of **DRNG tree** is encrypted, but can be disturbed by an **adversary**.

Note: In case of doubt, the certification body decides if the requirement is satisfied.

[DRT1.6] Requirement DRT.1.6 coincides with Requirements DRG.2.4 and DRG.3.4. In particular, **Shannon entropy** requires that the **raw random numbers** are **stationarily distributed**; cf. Requirement DRT.1.3. The case that a DRG.4-compliant **DRNG** is used for seeding or reseeding was discussed in par. 194. 257

[DRT.1.7] When seeding or reseeding is performed within the **DRNG tree**, **entropy** requirements on the **seed material** would not make sense, because if **DRNGs** of the **DRNG tree** have output some **internal random numbers**, then from the view of **information-theoretic security** (i.e., assuming unlimited computing power), the **internal state** of the root **DRNG** and the **internal states** of all its seed successors (i.e., the whole **DRNG tree**) is compromised. Nevertheless, inappropriate **seeding procedures** and **reseeding procedures** have to be prevented. Thus, Requirement DRT.1.7 provides a substitute requirement for the usual **entropy** condition. 258

Note: The aim of this substitute requirement is to ensure the plausibility that sufficient **randomness** is “transported” from the **seed material** to the **effective internal state** of the (re-)seeded **DRNG**. It is in line with Requirement DRG.2.4, DRG.3.4, and DRG.1.6 as far (re-)seeding with a DRG.4-compliant **DRNG** is concerned.

Example: Hash_DRBG and HMAC_DRBG; see par. 194, Example.

[DRT.1.7] **DRNGs** in a **DRNG tree** can generate **internal random numbers** for seeding, reseeding, and for other purposes as well. 259

[DRT.1.9] In par. 169, Note 3, it was mentioned that the set of all nodes together with the black edges form not only a subgraph of the **seed graph** but even a subtree. In a tree, the depth of a node is the length of the path to the root **DRNG**. Requirement DRT.1.9 prevents seed loops because the depth of the parent node and of its siblings in the subtree is decreased by 1 when compared to the depth of the **DRNG**, for which the **seed material** is generated. 260

Note 1: Siblings remain siblings for all time, but additional siblings can appear.

Note 2: Whether a **DRNG tree** satisfies Requirement DRG.1.9 can be read from the **seed graph**. A **DRNG tree** satisfies Requirement DRT.1.9, if to each node directed red edges only exist from siblings of its **direct seed predecessor**.

Example: The **DRNG tree** from Fig. 10 does not satisfy Requirement DRT.1.9. The **DRNG tree** from Fig. 11 satisfies Requirement DRT.1.9.

[DRT.1.9] If the developer wants to use the option that a sibling of the **direct seed predecessor** generates **seed material** for the **reseeding procedure**, *this requires that sibling nodes can reliably be identified*, e.g., by the operating system. Allowing the use of sibling **DRNGs** of the **direct seed predecessor** is often motivated by load balancing between several CPUs; cf. par. 163, Example 2. 261

Note: If the **direct seed predecessor** of a **DRNG** has been uninstantiated, or is unavailable for other reasons, and if no sibling **DRNG** of the **direct seed predecessor** is available, this **DRNG** cannot be reseeded. From a logical point of view, a solution would be to uninstantiate the **DRNG**

that cannot be reseeded and to instantiate a new DRNG in the DRNG tree. Fig. 11 illustrates such a situation, e.g., with DRNG C to be reseeded and its direct seed predecessor DRNG B. Siblings of DRNG B are DRNG E and DRNG F.

- 262 [DRT.1.12] The **granularity level** is a local requirement. It can be selected individually for each DRNG of a DRNG tree. For those DRNGs that provide **enhanced backward secrecy** on the level of **internal random numbers**, the **conceptual atomicity** condition is dropped; cf. par 213.
- 263 [local requirements] All DRNGs in a DRNG tree can algorithmically be identical (including the **seeding procedure** and the **reseeding procedure**), but this need not be the case. For each type of DRNG that occurs in the DRNG tree, the applicant has to provide evidence that the Requirements DRT.1.3 (root DRNG), DRT.1.4, DRT.1.5, DRT.1.6 (root DRNG), DRT.1.7 (non-root DRNGs), DRT.1.8, DRT.1.10, DRT.1.11, DRT.1.12, DRT.1.13, DRT.1.14, and DRT.1.15 are met. This includes the describing 9-tuple, a formal description of the **seeding procedure**, and (if applicable) the **reseeding procedure**.
- 264 [local requirements] The Requirements DRT.1.3 and DRG.3.1, DRT.1.4 and DRG.3.2, DRT.1.5 and DRG.3.3, DRT.1.6 and DRG.3.4, DRT.1.10 and DRG.3.5, DRT.1.11 and DRG.3.6, DRT.1.12 and DRG.3.7, DRT.1.13 and DRG.3.8, DRT.1.14 and DRG.3.9, and DRT.1.15 and DRG.3.10 coincide. The corresponding application notes for functionality class DRG.3 apply. Requirement DRT.1.7 is a substitute for Requirement DRG.3.4.
Note 1: Algorithmically, the DRNGs of a DRNG tree are compliant with functionality class DRG.3.
Note 2: Requirements DRT.1.8 and DRT.1.9 are specific for DRNG trees. These requirements do not exist for the “stand-alone” DRNGs that are not part of a DRNG tree.
Note 3: Although not explicitly mentioned for DRNGs that are not part of a DRNG tree, Requirement DRT.1.8 is meaningful and should also be considered there. Due to the larger complexity, it is required for DRNG trees.
- 265 [composition rules] Often, a developer would like to attach a DRNG or, more generally, a subtree S to an existing DRNG tree M . Both M and S can be static or dynamic. The question is whether their composition or a subtree of their composition that contains the subtree S , satisfies Requirement DRT.1.9. Of course, if M and/or S are dynamic, it shall be ensured that Requirement DRT.1.9 is satisfied all the time.
Note 1: A subtree of a DRNG tree consists of a subset of DRNGs that (together with the relation “is direct seed successor”) forms a tree. We use the term “DRNG subtree” if the subtree contains the (unique) root DRNG, otherwise we speak of a “subtree”.
Note 2: A single DRNG can be interpreted as a subtree.
Note 3: In practice, the existing DRNG tree M or a DRNG subtree thereof may have already been certified earlier; cf. par. 272 and 273.
- 266 [composition rules, notation] In pars. 267 to 270 appropriate and non-appropriate compositions are discussed. To avoid clumsy formulations, we introduce definitions for DRNG trees and DRNG subtrees. Figs. 12, 13, and 14 show possible corresponding seed graphs.
- DRNG tree M : comprises the DRNGs A,B,C,D,E,F,G.
 - DRNG subtree K of M : comprises the DRNGs A,F.
 - DRNG subtree L of M : comprises the DRNGs A,E,F,G.

- subtree S : comprises the DRNGs P, Q, R (to be attached).
- Composite DRNG tree T : DRNG tree M + subtree S .
- Composite DRNG subtree U of T : DRNG tree K + subtree S .
- Composite DRNG subtree V of T : DRNG tree L + subtree S .

Note 1: The adjective “composite” is not a technical term but refers to the property that the subtree S is attached to a DRNG tree.

Note 2: To avoid confusion below, we address the seed graph that corresponds to DRNG tree M by $M_{(sg)}$. The seed graphs that belong to other DRNG trees are denoted accordingly.

Note 3: Below, the nodes of the seed graphs M, K, L, S, T, U, V are always the same but the corresponding seed graphs are not because the sets of edges are not equal.

Note 4: To ease reading below, we repeat several times which DRNGs the DRNG (sub-)trees comprise.

[composition rules] In pars. 268 to 270 five compositions of DRNG trees are discussed. The corresponding seed graphs are shown in Figs. 12, 13, and 14. We limit our considerations on Requirement DRT.1.9. 267

Note 1: The subtree S itself is not a DRNG tree because the seed material for DRNG P is not generated by the initial randomness source. More formally, Requirement DRT.1.6 is not satisfied. In particular, the subtree S cannot be evaluated independently from the main DRNG tree M .

Note 2: Any composition that is compliant with functionality class DRT.1 must contain the root DRNG A because this is the only DRNG in the DRNG tree that fulfills Requirement DRT.1.6.

Note 3: The local requirements on the DRNGs do not depend on the composition.

[composition rules, Fig. 12, left and right drawing] The seed graphs shown in the left and the right drawing of Fig. 12 illustrate favorable situations. The seed graph $M_{(sg)}$ in the left drawing does not contain red edges. This means that no other DRNGs have generated seed material for reseeding than the direct seed predecessors. Hence, $M_{(sg)}$ is even a tree. In the right drawing, DRNG C has received seed material for the seeding procedure from its direct predecessor DRNG B . Furthermore, DRNG C has received seed material for reseeding from the siblings of its direct seed predecessor, DRNG E and DRNG F . (Possibly, also the direct seed predecessor DRNG B has generated seed material for reseeding but this is not relevant for the assessment.) Conclusion: In both cases, the composite DRNG tree T (comprises the DRNGs $A, B, C, D, E, F, G, P, Q, R$) fulfills Requirement DRT.1.9. 268

[composition rules, Fig. 13, left drawing] The situation is less favorable than in Fig. 12 because the seed graph $T_{(sg)}$ contains a seed loop between DRNG B and DRNG C . Therefore, the DRNG tree M and thus also the composite DRNG tree T violate Requirement DRT.1.9. Consequently, both are not compliant with functionality class DRT.1. The composite DRNG subtree U (comprises the DRNGs A, F, P, Q, R) of T fulfills Requirement DRT.1.9, and, additionally, the seed cycle in T does not affect U . The same is true for the composite DRNG subtree V (comprises the DRNGs A, E, F, G, P, Q, R) of T . 269

Conclusion: The composite DRNG tree U and the composite DRNG tree V fulfill Requirement DRT.1.9.

Note 1: The composite DRNG tree U is the smallest composite DRNG subtree of T that contains the subtree S .

Note 2: This interpretation of Requirement DRG.1.9 implies that the evaluator does not need to know details about parts of the DRNG tree, respectively of parts of the main seed graph $M_{(sg)}$

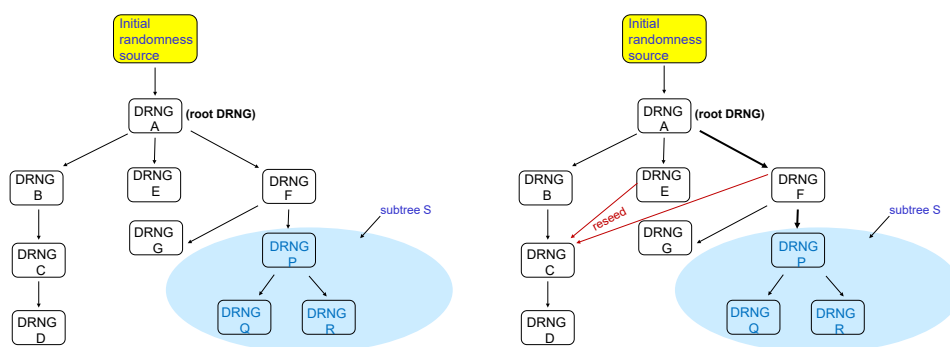


Figure 12: The **DRNG** subtree S is attached to the **DRNG** tree M (comprises the **DRNGs** A, B, C, D, E, F, G). Both drawings show possible **seed graphs** of the composite **DRNG** tree T (good compositions).

(including possibly attached subtrees), that have no impact on the composite subtree U (or V); cf. pars. 272 and 273.

Example to Note 2: A subtree attached to M that does not generate random numbers for cryptographic applications but, e.g., for stochastic simulations. Note that for non-cryptographic applications usually only statistical properties are relevant. Fig. 15 illustrates a similar situation where a subtree S' is attached to the **DRNG** tree L .)

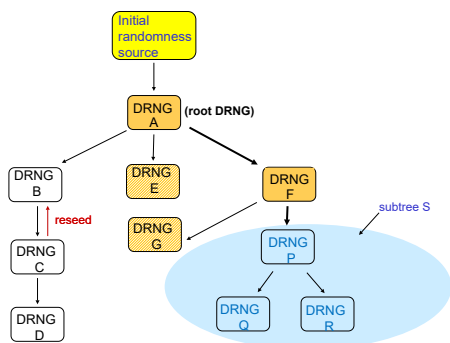


Figure 13: **DRNG** subtree S is attached to the **DRNG** tree M (comprises the **DRNGs** A, B, C, D, E, F, G). The drawing shows another possible **seed graph** of **DRNG** tree T than Fig. 12. The **DRNG** tree T does not satisfy Requirement DRT.1.9, but its **DRNG** subtrees V (composition of **DRNG** subtree L (**DRNGs** A, E, F, G) with subtree S) and U (composition of **DRNG** subtree K (**DRNGs** A, F) with subtree S) do.

- 270 [composition rules, Fig. 14, left and right drawing] In both cases, the **seed graph** $M_{(sg)}$ violates Requirement DRT.1.9. The left drawing shows a seed loop between the **DRNGs** F and G . In the right drawing, **DRNG** F has received **seed material** from both **DRNG** A and **DRNG** E , although **DRNG** A and **DRNG** E are not siblings, thereby violating Requirement DRT.1.9. In contrast with the drawing in Fig. 13, the violations of Requirement DRT.1.9 affect the composite subtrees U (composition of **DRNG** subtree K (**DRNGs** A, F) with subtree S) and V (composition of **DRNG** subtree L (**DRNGs** A, E, F, G) with subtree S).

Conclusion: The **DRNG trees** T, U, and V violate Requirement DRT.1.9. In particular, they are not compliant with functionality class DRT.1. Note: It would not change the situation if **DRNG G** (Fig. 14, left drawing) or **DRNG E** (Fig. 14, right drawing) were uninstantiated.

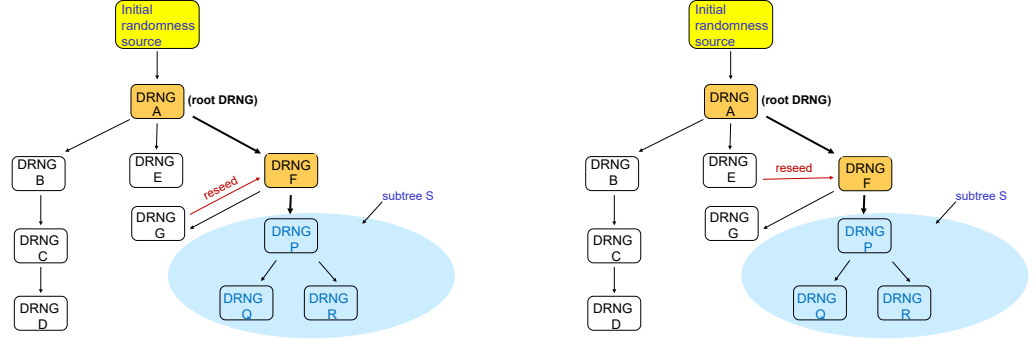


Figure 14: **DRNG subtree S** is attached to the **DRNG tree M**. Both drawings show possible **seed graphs** of the **DRNG tree T**. Neither the **DRNG tree T** nor any **DRNG subtree** that comprises subtree S fulfills Requirement DRT.1.9.

[DRT.1.13] **Additional input** cannot substitute a **reseeding procedure**, regardless of its **entropy**. 271

[results from previous evaluations] Results from previous certification processes can be used. This may concern, for example, the local requirements of particular **DRNGs** that belong to the **DRNG tree** or whether **DRNG tree** (without any attached subtrees) is compliant with functionality class DRT.1. 272

[results from previous evaluations] Example: Fig. 15 shows a scenario similar to that discussed in pars. 265 to 270. We use the same notation. Again, a subtree S shall be attached. We assume that a certificate confirms that the **DRNG tree L** (consisting of the **DRNGs** A, E, F, G; printed in orange) is compliant with functionality class DRT.1. Furthermore, a subtree S' that consists of **DRNG B** and its seed successors has been attached (or could be attached in the future) to the **DRNG tree L**. The applicant and the designer do not know details of the structure of the subtree S' , but only that it is ensured that no **DRNGs** in the **DRNG tree L** and in the subtree S receive **seed material** from any **DRNG** in S' . As in pars. 268 the composite **DRNG trees U** (consisting of the **DRNGs** A,F,P,Q,R) and **V** (consisting of the **DRNGs** A,E,F,G,P,Q,R) then satisfy Requirement DRT.1.9. 273

3.4 PTRNGs: Functionality Classes

Subsections 3.4.2 and 3.4.3 define functionality classes PTG.2 and PTG.3, respectively. The differences between this document and its predecessor [AIS2031An_11] are explained in the appendix, Subsection 6.1.2. Subsection 3.4.1 contains explanations that are relevant for both PTG.2 and PTG.3. We begin with general remarks. 274

We distinguish between **pure PTRNGs** and **hybrid PTRNGs**. Roughly speaking, the security 275

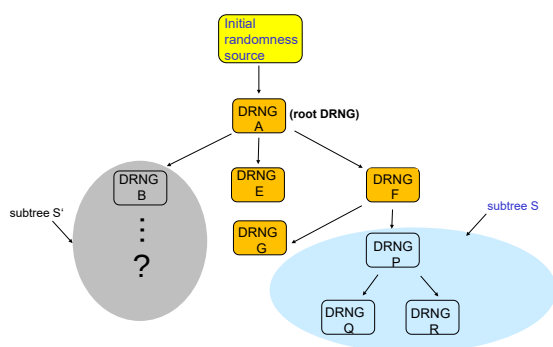


Figure 15: A **DRNG tree** with two attached **DRNG** subtrees **S** and **S'**.

of a **pure PTRNG** is essentially based on the **entropy** of the **raw random numbers** (to which an appropriate **post-processing** algorithm needs to be applied (the identity mapping is principally possible), whereas **hybrid PTRNGs** have two security anchors, namely **entropy** from the **noise source** and **computational security**, the latter provided by a **cryptographic post-processing** algorithm (**with memory**) that by itself is a **DRNG**.

- 276 This classification is not precise (and not relevant for the evaluation). Usually, **pure PTRNGs** apply non-cryptographic **post-processing** algorithms (typically, a simple **algorithmic post-processing** to increase the **entropy** per data bit), but **cryptographic post-processing** is also allowed. **PTRNGs** that use cryptographic constructions for their **post-processing** algorithm but not **with memory** (i.e., those constructions are not **DRNGs**) are generally considered to be **pure PTRNGs** because they become practically insecure if the **noise source** becomes weak or breaks down completely. **Hybrid PTRNGs** apply **cryptographic post-processing**, which according to the definition in this document, always means **with memory**. By data compression, at the cost of performance, **cryptographic post-processing** can be applied to increase the **entropy** per bit, but **cryptographic post-processing** algorithms (with or without data compression) are also an additional security layer that is based on **computational security**.
- 277 A **PTRNG** that is compliant with functionality class PTG.2 is basically a well-understood **physical noise source** that exploits physical phenomena that provide a quantified amount of **entropy** with very high assurance. Together with a **total failure test** and an **online test**, this allows the generation of **internal random numbers** with an **entropy** per bit that is very close to 1. Functionality class PTG.3 is basically a PTG.2-compliant **PTRNG** combined with a DRG.3-compliant **cryptographic post-processing** algorithm. A **pure PTRNG** can be compliant with functionality class PTG.2 but cannot be compliant with functionality class PTG.3 because of the requirement for the **cryptographic post-processing**.
- 278 For both functionality classes PTG.2 and PTG.3, high assurance shall be established by a **stochastic model** of the **raw random numbers**. The **stochastic model** describes the stochastic behavior of the **raw random numbers** and traces the behavior back to true physical randomness. The **stochastic model** enables statistical analysis and the quantification of the **entropy** of the **raw random numbers**. Furthermore, the **stochastic model** is important to assess **algorithmic post-processing** algorithms with regard to the **entropy** per **internal random number** bit.

279

In order to maintain this high assurance over the entire life cycle, a **PTRNG** compliant with functionality class PTG.2 or PTG.3 is required to have **total failure tests** that detect **total failures** so quickly that no **internal random numbers** are output that were generated after the **total failure** has occurred. The concrete **PTRNG** design may allow relaxations (e.g., due to buffering, in particular for PTG.3-compliant **PTRNGs**); cf. Subsection 4.5.4. Non-tolerable deviations from the desired behavior shall be detected **sufficiently soon** by the **online test**.

A **PTRNG** compliant with functionality class PTG.2 or PTG.3 delivers output with **entropy** per data bit very close to 1 with a high level of assurance; cf. par. 313 for justification. 280

A **PTRNG** compliant with functionality class PTG.3 additionally has the security properties of functionality class DRG.3. 281

It should be noted that the definitions of functionality classes PTG.2 and PTG.3 (as well as DRG.3) have been reworked in this version of the document. The definitions of functionality classes PTG.2 and PTG.3 and their objectives are similar to that in [AIS2031An_11] (which justifies maintaining the class names) although they are different in detail. An in-depth explanation of the differences to the previous definitions in [AIS2031An_11] can be found in the appendix, Subsection 6.1.2 (for DRG.3: see Section 6.1.1). 282

3.4.1 PTG.[2,3]: Definitions, Requirements, and Justification

PTRNGs use **physical noise sources** (whereas **NPTRNGs** use **non-physical noise sources**). 283

The “core” of a **PTRNG** is its **physical noise source**. The **physical noise source** extracts randomness from a physical phenomenon (or from several phenomena). The **digitization** mechanism associated with the **physical noise source** generates **raw random numbers** from the (typically) analog signals derived from a physical phenomenon (or several phenomena). The **digitization** mechanism is considered to be a part of the **physical noise source**. 284

Physical noise sources exploit physical phenomena (thermal noise, shot noise, jitter, metastability, radioactive decay, etc.) from dedicated hardware designs (using diodes, ring oscillators, etc.) or physical experiments (e.g., measuring radioactive decay times) to produce digitized random data. The dedicated hardware designs can use general-purpose components (like diodes, logic gates, etc.) if the designer is able to understand, describe, and quantify the characteristics of the design that are relevant for the generation of random numbers. 285

Usually, the **physical noise source** of a **PTRNG** is part of an electronic circuit or is realized as a physical experiment. When integrated into an electronic circuit (e.g., a microchip), the **physical noise source** consists of dedicated hardware designed for this purpose. 286

Examples: The **physical noise source** may employ Zener diodes, noisy oscillators, or ring oscillators. Or, it may exploit chaotic behavior, radioactive decay, or other quantum effects. This list of possible phenomena and design principles is not complete. We refer the reader to Section 5.4 for a more detailed treatment. 287

The central task of both PTRNG evaluations and NPTRNG evaluations is the verification that the entropy per internal random number bit exceeds a specified lower bound.

- 289 While the physical noise source of a PTRNG is “under the control” of the RNG designer, the non-physical noise source of a NPTRNG usually cannot be controlled by the RNG designer (cf. Subsec. 3.5.1). This is an important difference between PTRNGs and NPTRNGs, which significantly impacts the depth of the evaluation.
- 290 The fact that the physical noise source is based on a dedicated hardware design allows (at least in principle) precise modeling because one may assume that the same types of physical noise sources in different devices behave similarly. However, the physical noise sources generally do not behave identically in a strict sense because even digital physical noise sources usually consist of analog components. Differences may, for example, be caused by component variance (inside certain tolerance levels), aging effects, or different environmental conditions; cf. pars. 307 to 308.
- 291 [combining several physical noise sources] In the following, the document speaks of the physical noise source. However, functionality classes PTG.2 and PTG.3 also allow the use of several physical noise sources, e.g., several ring oscillators.
- 292 [combining several physical noise sources] If the PTRNG uses several physical noise sources, the physical noise sources may influence each other. Thus, all physical noise sources have to be analyzed jointly, which justifies viewing them as one (single) physical noise source. A separate analysis of the physical noise sources used is permitted only if it can be shown that the physical noise sources do not influence each other; cf. also the pars. 472 and 473, which consider the combination of several RNGs.
- 293 The analog values produced by the physical noise source are digitized at some point, providing the raw random numbers (a.k.a. das-random numbers (Digitized analog noise signal (das))). The digitization mechanism can involve simple transformations (e.g., dropping bits) and the raw random numbers may undergo several separated post-processing operations. For this reason, there may be some ambiguity as to what intermediate product should be referred to as the raw random numbers.
- 294 The developer/applicant decides which data are to be called the raw random numbers. Both PTG.2 and PTG.3 require a verifiable stochastic model for the raw random numbers that traces their stochastic behavior back to a physical phenomenon/to several physical phenomena. It is therefore strongly recommended to choose the earliest possible stage. The evaluator accepts or rejects the stochastic model and the corresponding rationale.
- 295 Examples
- The physical noise source samples noisy voltage at a high frequency and these values are digitized to 8 bits. In order to reduce statistical dependencies, the developer decides to discard every other byte. The developer then declares the remaining bytes to be the raw random numbers and provides a stochastic model describing their bias and statistical dependency as well as how they relate to the physical phenomenon causing the noise. This approach is principally permitted.

Note: This operation lowers the output rate by a factor of 2. Alternatively, the unmodified sequence can be chosen as the **raw random numbers**, and discarding every second byte may be viewed as **algorithmic post-processing**.

- A battery of ring oscillators is sampled and their output fed into a (cryptographic) hash function with a large compression factor. The developer declares the resulting hashed bits to be the **raw random numbers** and claims stochastic independence and uniform distribution as a **stochastic model**. This approach is *not permitted* because the stochastic properties of the **raw random numbers** cannot be traced back to a physical phenomenon.

Viewed as a mathematical function, **algorithmic post-processing** algorithms usually have a small domain and a small range. 296

Examples (of **algorithmic post-processing** algorithms): XORing bits or binary vectors, modular addition, **LFSRs**, and identity mapping. 297

The **raw random numbers** may or may not undergo **algorithmic post-processing** and/or **cryptographic post-processing** (finally resulting in the **internal random numbers**, i.e., the data ready for output). If the **raw random numbers** already “contain” sufficient **entropy** per bit to meet the PTG.2 requirements, the designer may choose to omit a **post-processing** algorithm. In this case, the **raw random numbers** coincide with the **internal random numbers**. Formally, a nonexistent **post-processing** algorithm can be interpreted as the identity mapping. Examples of **algorithmic post-processing** algorithms are discussed in Section 5.1; cf. also par. 297. 298

[universal families of hash functions] Alternatively, a PTG.2-compliant **PTRNG** can (under suitable conditions) also apply universal families of hash functions or **post-processing** algorithms that use **cryptographic** functions (with or without memory); cf. par. 336 ff. Universal families (as well as almost universal families) of hash functions count as **algorithmic post-processing** algorithms. To avoid misunderstanding, it should be noted that in this context “hash functions” are not cryptographic hash functions but, instead, induce simple mappings (e.g., matrix-vector multiplications). In particular, universal families of hash functions cannot be used to construct a PTG.3-compliant **PTRNG**. 299

[PTG.3] PTG.3 is the strongest functionality class in **AIS 20** and **AIS 31**. 300

[PTG.3] The usual technical realization of a PTG.3-compliant **PTRNG** is to use a PTG.2-compliant **PTRNG** whose **internal random numbers** are fed into a DRG.3-compliant **cryptographic post-processing** algorithm. The PTG.2-compliant **PTRNG** then is the central component of the PTG.3-compliant **PTRNG**. However, it is not mandatory to have a clear-cut “PTG.2-boundary” within the **PTRNG**. Of course, the lack of a clear PTG.2-boundary does not waive or relax any requirements on the **raw random numbers** and on the **entropy** verification of the **internal random numbers**. 301

[PTG.3] The data that are input to the **cryptographic post-processing** algorithm are called **intermediate random numbers**. If the **PTRNG** has a PTG.2-compliant core (the usual design, cf. par. 301), the **intermediate random numbers** of the PTG.3 design are the **internal random numbers** of the PTG.2-compliant **PTRNG**. 302

303

[PTG.3] The **cryptographic post-processing** algorithm shall not “expand” its input data (i.e., the **intermediate random numbers**). This means that the average output rate in bits of the **cryptographic post-processing** algorithm shall not be larger than its input rate (in bits). That is, the ratio between the number of intermediate bits (required for one **internal random number**) and the bit length of an **internal random number** shall be ≥ 1 . This is called the **compression rate** c_{rate} in the following. To increase the **entropy** per bit, the **compression rate** must be > 1 . If the **compression rate** is < 1 , the **PTRNG** cannot be compliant with functionality class PTG.3 (but compliance with functionality class DRG.4 is possible); for further details cf. Requirement PTG.3.4 and application note par. 388.

- 304 [PTG.3] Of course, **cryptographic post-processing** can only increase the **entropy** per bit if it compresses the input data. If the **cryptographic post-processing** algorithm can be modeled by a random mapping, the difference $c_{\text{diff}} = (\text{\#number of input bits} - \text{\#number of output bits})$ is significantly more relevant for the increase of **entropy** in the output than the **compression rate** c_{rate} . This may seem surprising at first sight, but the reason is that the ratio between the sizes of the domain and of the image space is $2^{c_{\text{diff}}}$. Section 4.4 treats this topic intensively. Note: Of course, given c_{rate} and m , it is easy to calculate n and c_{diff} . What is meant here is that (with regard to the **entropy** per bit) the case $(n, m) = (140, 128)$ is similar to $(n, m) = (268, 256)$ whereas $(n, m) = (280, 256)$ ensures a smaller **entropy** defect than $(n, m) = (140, 128)$ although c_{rate} is identical in both cases. (Expressed in other words: The term c_{diff} is more illustrative quantity than c_{rate} .)
- 305 [PTG.3] If the **cryptographic post-processing** of a PTG.3-compliant **PTRNG** would run autonomously, it would be (algorithmically) compliant with functionality class DRG.3. That means, even if the PTG.2-compliant part of a PTG.3 (assuming the usual PTG.3 design) were suddenly to deliver predictable output, the **PTRNG** would still have the security features of a DRG.3 compliant **DRNG**. This does hold, of course, only under the assumption that the **internal state** of the **cryptographic post-processing** algorithm has already received enough **entropy** from the **intermediate random numbers**. This should be the case shortly after the **PTRNG** has been started.
- 306 [PTG.3] Note that the “**DRNG** fallback” in the previous paragraph is an additional security layer. The requirements for PTG.2 and PTG.3 dictate a reliable **online test** and a reliable **total failure test** that shall prevent undetected degradation of the **entropy** of the **internal random numbers** or an undetected **total failure** of the **physical noise source** (par. 312). Further beneficial effects of **cryptographic post-processing** are described in pars. 317 and 318.
- 307 **Raw random numbers**, **intermediate random numbers**, and **internal random numbers** are interpreted as realizations (i.e., of values taken on) of **random variables**. For the concept of randomness, **random variables**, and realizations, we refer the interested reader, e.g., to Section 4.1.
- 308 [stochastic model] For **PTRNGs**, the **entropy** analysis shall be based on a **stochastic model**. The **stochastic model** takes the concrete design of the **physical noise source** into account and models its stochastic behavior. Based on this behavior, the impact of the **algorithmic post-processing** on the **internal random numbers** is analyzed. **Black box** testing of the **raw random numbers** or the **internal random numbers** is not sufficient to assess their **entropy**. If **cryptographic post-processing** is used instead of **algorithmic post-processing**, different methods are applied. Usually, the **cryptographic post-processing** algorithm is modeled by a random

mapping or a random bijection; cf. Section 4.4 and several application notes to Requirement PTG.3.5. Universal families of hash functions are considered in Section 4.3, pars. 641 to 648.

The formulation, verification, and analysis of the **stochastic model** is the crucial part of a **PTRNG** evaluation. We refer the reader to detailed explanations in Section 4.5, and to Section 5.4 for illustrating examples. 309

When the **PTRNG** has been started, a **start-up test** shall check for a **total failure** and severe statistical weaknesses; cf. Subsection 4.5.5. 310

The **entropy** per **internal random number** bit shall be large enough when the **PTRNG** is in operation. For functionality classes PTG.2 and PTG.3, the indistinct term “large enough” is quantified; cf. Requirements PTG.2.2 and PTG.3.5. When the **PTRNG** is operation, the **entropy** claim shall be assured by an **online test**; cf. Subsection 4.5.3. The effectiveness of the **online test** shall be verified based on the **stochastic model** of the **physical noise source**. 311

During operation, a **total failure** of the **physical noise source** can occur. A **total failure** would imply that future **raw random numbers** contain (almost) no **entropy**. A **total failure test** shall detect a **total failure** of the **physical noise source** virtually immediately (relaxations are possible, depending on the design of the **PTRNG**); cf. Subsection 4.5.4. This means, a **total failure test** must detect a **total failure** in time to prevent the output of low-**entropy internal random numbers**. 312

Ideal RNGs do not exist. Even if **ideal RNGs** existed, it would be impossible to verify them. Thus, functionality classes PTG.2 and PTG.3 allow small **entropy** defects. Compliance with functionality classes PTG.2 and PTG.3 guarantees a lower **entropy** bound per random bit close to 1. 313

Assume that $Z_1, Z_2, \dots$ are **stationarily distributed** binary-valued **random variables**. One way to quantify k -step dependencies is to evaluate $|\text{Prob}(Z_{j+k} = 0 \mid Z_j = 0) - \text{Prob}(Z_{j+k} = 0 \mid Z_j = 1)|$. 314

The PTG.2 class specification tolerates a small **entropy** defect, e.g., caused by a bias or by dependencies of the **internal random numbers**. For many cryptographic applications (e.g., for the generation of **AES** keys, challenges, IVs, etc.) such defects should not impact security in practice. 315

Yet for other applications, such defects might bear security risks. For **ECDSA** signatures, for example, the ephemeral keys are linked by an underdetermined system of linear equations over a finite field. An **adversary** might try to combine information from many signatures. Although no concrete attack is known to date that could leverage the small **entropy** deficiency allowed for PTG.2-compliant **PTRNGs**, at least in theory, this represents a security risk. 316

Consequently, the direct employment of PTG.2-compliant **PTRNGs** for arbitrary cryptographic applications is not recommended. Generally, PTG.2-compliant **PTRNGs** should be used to seed **DRNGs** or serve as a “core” of a PTG.3-compliant **PTRNG**. 317

Furthermore, the **cryptographic post-processing** of PTG.3-compliant **hybrid PTRNGs** should also increase their resistance to side-channel and fault attacks (e.g., induced transient breakdowns of 318

the **physical noise source** or enforcing specific values). Implementation attacks are not covered by AIS 31 but, of course, are relevant in the overall evaluation of the **TOE**; cf. Section 2.1, par. 28.

- 319 Principally, the **entropy** defects tolerated by functionality class PTG.2 could have been set significantly smaller. We have refrained from doing so for two reasons: First of all, this would have increased the requirements concerning the verification of the **stochastic model**. Furthermore, it would increase the difficulties of implementing efficient **online tests** (with reasonable sample sizes) that would effectively detect when the **entropy** falls below the specified **entropy** bound.
- 320 For functionality classes PTG.2 and PTG.3, the **entropy** of the **raw random numbers** can be quantified in **Shannon entropy** and/or **min-entropy**. **Shannon entropy** is justified by the fact that the **raw random numbers** are **stationary** (cf. Section 4.3) and that the **Shannon entropy** satisfies useful functional equations (4.100) and (4.101), which simplify the **entropy** calculation for dependent **random variables**. For functionality classes PTG.2 and PTG.3, **entropy** claims for the **internal random numbers** and the **intermediate random numbers** (only for PTG.3) **Shannon entropy**, **min-entropy**, or both **Shannon entropy** and **min-entropy**, are permitted. This is different from functionality class NTG.1, which only allows **min-entropy** claims.

3.4.2 Functionality Class PTG.2

- 321 Functionality class PTG.2 defines requirements for physical **RNGs**.
- 322 Roughly speaking, PTG.2-compliant **RNGs** generate high-**entropy internal random numbers**. The **entropy** shall, in particular, prevent successful guessing attacks, but the **internal random numbers** may be practically distinguishable from an **ideal RNG** when testing large amounts of data.
- 323 The **TSF** has to protect the **internal state** (if applicable) of the **RNG** from being compromised.
- 324 Note: Par. 325 does not replace an evaluation methodology.
- 325 **PTG.2-specific deliverables by the applicant**
The security architecture description and developer evidence shall contain at least
- a description of the **physical noise source** (including the **digitization** mechanism),
 - a comprehensive description of the “algorithmic behavior” of the **PTRNG**, beginning with the **digitization** of the **raw random numbers**,
 - a **stochastic model** of the **raw random numbers** with substantiated justification, statistical evidence, and thorough analysis,
 - evidence that PTG.2.1 and PTG.2.2 are fulfilled,
 - a description of the **start-up test** and evidence that PTG.2.3 is fulfilled,
 - a description of the **online test** and evidence that PTG.2.4 is fulfilled,

- a description of the **total failure test** and evidence that PTG.2.5 is fulfilled,
- evidence that PTG.2.6 is fulfilled.

PTG.2: Security functional requirements

326

Security functional requirements of functionality class PTG.2 are defined by component **FCS_RNG.1** with specific operations as given below.

FCS_RNG.1 Random number generation (Class PTG.2)

327

FCS_RNG.1.1 The **TSF** shall provide a *physical* random number generator that implements the following:

- (PTG.2.1) The **TSF** shall generate **raw random numbers** that can be viewed as realizations of a (time-local) **stationary** stochastic process $R_1, R_2, \dots$
- (PTG.2.2) The **internal random numbers** shall be interpreted as realizations of **random variables** $Y_1, Y_2, \dots$. If the **random variables** Y_j are binary-valued, it shall be $\text{Prob}(Y_j = 1) \in (0.493, 0.507)$. If the **random variables** Y_j assume binary vectors, this condition shall be met by the projections onto the particular bits. Furthermore, it shall be guaranteed that [selection, choose one of:
the **Shannon entropy** per **internal random number** bit is greater than or equal to 0.9998,
the **min-entropy** per **internal random number** bit is greater than or equal to 0.98,
the **Shannon entropy** per **internal random number** bit is greater than or equal to 0.9998 and the **min-entropy** per **internal random number** bit is greater than or equal to 0.98].
- (PTG.2.3) The **start-up test** shall be applied after the **RNG** has been started. It shall be designed to detect a **total failure** of the **physical noise source** and severe statistical weaknesses. The **TSF** shall not output any **internal random numbers** before the **start-up test** has been passed.
- (PTG.2.4) The **online test** shall check the quality of the **raw random numbers** while the **RNG** is in operation. The **online test** shall detect non-tolerable **entropy** defects of the **raw random numbers** sufficiently soon. The **TSF** shall not output any **internal random numbers** if a non-tolerable **entropy** defect has been detected.
- (PTG.2.5) The **total failure test** shall detect if a **total failure** of the **physical noise source** occurs while the **PTRNG** is in operation. The **total failure test** shall prevent the output of **internal random numbers** that depend on any **raw random number** that has been generated after the **total failure** of the **physical noise source**. If the **PTRNG** applies a **cryptographic post-processing** algorithm (*with memory*) that is compliant with functionality classes DRG.2 or DRG.3, then this relaxes this requirement: After the **total failure** has occurred, the output of **internal random numbers** bits whose **entropy** per bit is not close to 1 shall be prevented.

FCS_RNG.1.2 The **TSF** shall provide [selection, choose one of: *bits, octets of bits, integers* [assignment: *format of the numbers*]] that meet:

(PTG.2.6) The *internal random numbers* shall pass test suite T_{irn} .

Application notes

- 328 [stochastic model, refers to PTG.2 and PTG.3] The evaluation of a PTRNG shall be based on a verifiable, substantiated stochastic model. There is only one level of detail in the description of the stochastic model, irrespective of the chosen EAL. We refer to Section 4.5, which provides additional information, illuminates the mathematical background, and discusses elementary examples of stochastic models. More complex “real-world” examples of stochastic models can be found in Section 5.4.
- 329 [stochastic model, refers to PTG.2 and PTG.3] The evaluator checks the stochastic model, its justification, and the analysis provided by the applicant. The justification should be supported by theoretical considerations, findings in literature, and typically, by empirical studies. The empirical studies should be supported by statistical tests and possibly by predictors that are tailored to the stochastic model. The evaluator may perform additional tests if they feel that it might be relevant. It is encouraged that the evaluator considers predictors on the raw random numbers that are tailored to the stochastic model. Predictors cannot verify the entropy claim(s) but can falsify them if a predictor is significantly more successful than expected. In the positive case, as statistical tests, predictors can help to increase the confidence in the stochastic model; cf. Section 4.5 and Section 4.6.
- 330 [stochastic model, refers to PTG.2 and PTG.3] Functionality classes PTG.2 and PTG.3 do not limit the “depth” of possible dependencies of the raw random numbers. This is motivated by the fact that otherwise, widespread types of physical noise sources would have been excluded. However, usually the verification of the stochastic models should be significantly easier if (at most) short-term dependencies exist. Apart from Requirement PTG.2.1, which says that the raw random numbers shall be time-local stationarily distributed, functionality classes PTG.2 and PTG.3 have no further requirements on the raw random numbers. For this reason, functionality classes PTG.2 and PTG.3 do not prescribe statistical black box tests and black box predictors for raw random numbers.
- 331 [several physical noise sources, refers to PTG.2 and PTG.3] Functionality classes PTG.2 and PTG.3 allow to use several physical noise sources instead of one. In general, it is *not sufficient* to analyze these physical noise sources separately, unless it can be given evidence that the physical noise sources can be viewed as independent. Such an independence claim would be part of the security evaluation of the TOE, or more precisely, of the noise source. Otherwise, if mutual influence of the physical noise sources cannot be excluded, *all physical noise sources* have to be analyzed and evaluated jointly; cf. pars. 291 and 292.
- 332 [several physical noise sources, refers to PTG.2 and PTG.3] If already a single physical noise source provides enough entropy to meet the requirements of PTG.2 or PTG.3, respectively, and if it can be shown that the other physical noise sources do not negatively affect its entropy, then it suffices to evaluate this single physical noise source and disregard the other physical noise sources.
- 333 [PTG.2.1] The verification of Requirement PTG.2.1 shall be based on the stochastic model. For

the analysis of the **stochastic model**, usually time-local **stationarily distributed random variables** can be treated as if they were **stationarily distributed**; cf. Subsection 4.5.1, pars. 788 to 790, **time-local stationarity**.

[PTG.2.1] If the **raw random numbers** are not binary-valued but ℓ -bit vectors, it may be the case 334
that the **random variables** that describe the random behavior of the ℓ -bit vectors are time-locally **stationarily distributed**, but not the individual bits. Of course, such a feature has to be taken into account in the analysis.

[PTG.2.2] The developer may claim the **Shannon entropy** bound, the **min-entropy** bound, or 335
both. For both **Shannon entropy** and **min-entropy**, functionality class PTG.2 only allows fixed, class-specific values. The verification of the **min-entropy** claim may require additional efforts.
Note 1: If the **internal random numbers** comprise k bits, the **entropy** requirements can alternatively be expressed as follows: “The **Shannon entropy** per **internal random numbers** is greater than or equal to $0.9998k$ ” and “The **min-entropy** per **internal random numbers** is greater than or equal to $0.98k$ ”. That is to say, on average, the individual bits of a single **internal random number** satisfy the **entropy** condition. Note that here, “average” does not mean “average in time” (over several **internal random numbers**).

Note 2: The individual bits of the **internal random numbers** need not be identically distributed. The condition that each projected **internal random number** bit shall assume the value 1 with a probability in $(0.493, 0.507)$ shall prevent too great imbalancedness.

Note 3: Although the computed **entropy** values (based on the **stochastic model**) should normally exceed the specified **entropy** bounds significantly, class PTG.2 does not allow larger **entropy** claims; cf. par. 319.

[PTG.2.2] [**post-processing**] To verify Requirement PTG.2.2, the effect of **post-processing** algo- 336
rithm (applied to the **raw random numbers**) on the **entropy** of the **internal random numbers** has to be taken into account. Several types of **post-processing** algorithms are permitted:

- No **post-processing** algorithm: This option requires that the **raw random numbers** already have enough **entropy** to satisfy Requirement PTG.2.2. Formally, “no **post-processing**” can be interpreted as the identity mapping.
- **Algorithmic post-processing** algorithm: By an **algorithmic post-processing** algorithm, we mean a relatively simple mapping such as XORing consecutive, non-overlapping pairs of **raw random numbers**. An **algorithmic post-processing** algorithm should be selected with regard to the possible distributions of the **raw random numbers**; i.e., it should be tailored to the **stochastic model**. The impact of the **algorithmic post-processing** algorithm on the **entropy** has to be analyzed. Examples of **algorithmic post-processing** algorithms are discussed in Section 5.1, in particular in Subsections 5.1.1 to 5.1.3, and in several application notes below.
- Universal families of hash functions: Universal families of hash functions also belong to the class of **algorithmic post-processing** algorithms. Nevertheless, they have specific features; see pars. 337 to 339 for further details.
Note: Even if it is not explicitly mentioned, this bullet point and the following corresponding paragraphs also include almost universal families of hash functions.
- **Post-processing** algorithm using a **cryptographic** function (stateful (**with memory**) or state-

less); cf. par. 343.

The bit length of the **internal random numbers** should not be greater than 2048. Otherwise, the certification body decides whether the construction is acceptable.

The first three types of **post-processing** algorithms are not compliant with the requirements of functionality class PTG.3. If the **post-processing** applies a **cryptographic** function **with memory**, it can satisfy Requirements PTG.3.2 and PTG.3.4, but need not.

Note: If the **post-processing** algorithm applies a **cryptographic** function, it is recommended to maintain an **internal state** and make it a proper **cryptographic post-processing** algorithm (i.e., **with memory**) because this covers further security features; cf. Requirement PTG.2.5 (and functionality class PTG.3).

- 337 [PTG.2.2] [universal families of hash functions] Universal families of hash functions (and, more generally, almost universal families of hash functions) are treated in pars. 641 to 648, and in Subsection 5.1.4. Let $\mathcal{H} = \{h_k : \{0, 1\}^n \rightarrow \{0, 1\}^m \mid k \in K\}$ denote a ϵ -almost universal family of hash functions with $\epsilon = \frac{1+\alpha}{2^m}$. The generation of an **internal random number** y_j consists of two steps. In the first step, an element of $\mathcal{H}$, a so-called hash function, is randomly selected using a random number u_j , and in the second step, a vector of **raw random numbers**, denoted by $\vec{r}_j$, of bit length n is used as input of the selected hash function: $y_j = h_{u_j}(\vec{r}_j)$.
Note: The elements of $\mathcal{H}$ are called hash functions. They should not be confused with cryptographic hash functions.

- 338 [PTG.2.2] [universal families of hash functions] As usual, the values u_j, r_j and y_j are interpreted as realizations of **random variables** U_j, R_j and Y_j . From the leftover hash lemma, a lower **min-entropy** bound for the **internal random number** Y_j can be derived, that is reached or exceeded *on average*, if the hash functions are selected with the uniform distribution (4.144). This implies several problems; cf. Subsection 5.1.4. As a consequence, the following requirements shall be fulfilled (cf. par. 951):

- a) For each **internal random number** y_j , the hash function h_j shall be selected randomly.
- b) Let $\chi(U_j) := H_{\min}(h_{U_j}(\vec{R}_j))$. The **random variables** $\chi(U_1), \chi(U_2), \dots$ shall satisfy the strong law of large numbers.
- c) Recall that $y_j \in \{0, 1\}^m$. Then

$$\text{Prob} \left(H_{\min} \left(h_{U_j}(\vec{R}_j) \right) < 0.98 \right) \leq 2^{-16} \quad (\text{min-entropy claim}) \quad (3.5)$$

$$\text{Prob} \left(H \left(h_{U_j}(\vec{R}_j) \right) < 0.9998 \right) \leq 2^{-16} \quad (\text{Shannon entropy claim}) \quad (3.6)$$

shall hold.

In other words: The class-specific **entropy** bound shall not be undercut with probability of $> 2^{-16}$.

- d) The random numbers u_j shall be kept secret at any time.
Note: “at any time” includes the time after the **PTRNG** has been uninstantiated. Therefore, the index counter shall be overwritten or erased when the **PTRNG** is uninstantiated.

e) $|\mathcal{H}| \geq 2^{200}$

f) The knowledge of U_j shall give little information on $U_{j+\ell}$ unless ℓ is very small. More precisely, the conditional **min-entropy** shall satisfy the condition (3.7). Here, $t = \log_2(|\mathcal{H}|)$.

$$H_{\min}(U_j) - H_{\min}(U_j | U_{j+\ell}), H_{\min}(U_j) - H_{\min}(U_j | U_{j-\ell}) \leq \begin{cases} 0.10t & \text{for } \ell \geq 2 \quad \text{if } 200 \leq t \leq 500 \\ 0.30t & \text{for } \ell \geq 2 \quad \text{if } t > 500. \end{cases} \quad (3.7)$$

[PTG.2.2] [universal families of hash functions] The reason for Requirement e) is that the probability of a very bad hash function (e.g., a matrix with a low rank) shall be kept very small. Unlike for **cryptographic post-processing** algorithms, or more generally, for **cryptographic** functions, for non-cryptographic hash functions it is usually easy to determine whether a given hash function is weak. The adjective “weak” can be a property of the hash function itself or the hash function in combination with the **physical noise source**. 339

Inequality (4.148) allows a straightforward generalization of the leftover hash lemma (4.151) for non-uniformly distributed random variables. Therefrom, the generalized **min-entropy** inequalities (4.152) and (4.153) can be derived. In this case the random variables $U_1, U_2, \dots$ are iid (although not uniformly distributed in general), and in particular the Requirements b) and f) are fulfilled. If the applicant applies the ‘usual’ leftover hash lemma (4.139), or more precisely, the **min-entropy** inequality (4.144), the **random variables** $U_1, U_2, \dots$ shall be (approximately) **uniformly distributed** on the index set K , thereby inducing (approximately) the uniform distribution on $\mathcal{H}$. Pars. 951 to 955 discuss a possible solution that satisfies the strong law of large number although the random variables $U_1, U_2, \dots$ are usually not iid. Requirement f) ensures that the ‘memory’ of the previous indices fades out rapidly. 340

Note: If $h_1, h_2, \dots, h_{|\mathcal{H}|}$ denotes an enumeration of $\mathcal{H}$, the **random variables** U_j can be **uniformly distributed** on $\{1, \dots, |\mathcal{H}|\}$. For the case that $|\mathcal{H}|$ is not a power of 2 additional problems occur; e.g., [TR-02102], Subsection B.4, can be helpful.

[PTG.2.2] [universal families of hash functions] Assume that the **random variable** U is **uniformly distributed** on K . Requirement (3.5) can be verified with inequality (4.144). Assume that $E_U(H_{\min}(h_{U_j}(\vec{R}_j))) \geq (1-a)m$ for some $a \in (0, 1)$ and let $b := \text{Prob}(H_{\min}(h_{U_j}(\vec{R}_j)) < 0.98m)$. Then 341

$$(1-a)m \leq E_U \left(H_{\min}(h_{U_j}(\vec{R}_j)) \right) \leq b \cdot 0.98m + (1-b)m = m(1 - 0.02b). \quad (3.8)$$

Inequality (3.8) implies $b \leq \frac{a}{0.02} = 50a$. Thus, for $a = \frac{2^{-16}}{50}$ it is $b \leq 2^{-16}$, satisfying requirement (3.5).

Note: Inequality (3.8) is rather coarse and might be improved, allowing a larger value $a \in (0, 1)$.

[PTG.2.2] [universal families of hash functions] The random indices $u_1, u_2, \dots$ can be generated with a PTG.3-compliant **PTRNG** with **min-entropy** claim of $1 - 2^{-32}$ bits of **min-entropy** per **internal random number** bit. Subsection 5.1.4, pars. 953 to 955, explains and investigates a design that uses the same **physical noise source** to generate the random indices and the **raw random numbers**. If it satisfies the requirements formulated in par. 338, this design can be used. It is *not permitted* to generate the random indices with a DRG.3-compliant **DRNG**. The main reason is that this would protect previously generated indices only computationally, and forthcoming indices not at all. It is also *not permitted* to use the same hash function permanently and to argue 342

with the leftover hash lemma. In this case, the selected hash function $h \in \mathcal{H}$ would be viewed as a “usual” **algorithmic post-processing** algorithm, and thus, the developer has to quantify a lower **entropy** bound for the **internal random numbers** based on the **stochastic model**.

- 343 [PTG.2.2] **post-processing** with a **cryptographic** function] For the **entropy** analysis of **post-processing** algorithms that use **cryptographic** functions (stateful (**with memory**) or stateless), the same methods can be applied as for functionality class PTG.3 (there: for the DRG.3-compliant **cryptographic post-processing** algorithms); cf. par. 391 to 413, and Section 4.4. In particular, the PTG.2-specific **entropy** bounds (0.9998 bits of **Shannon entropy**, 0.98 bits of **min-entropy**) shall be undercut at most with probability 2^{-16} ; cf. par. 394. An advantage is that it is not necessary to know a class of probability distributions that contains the true distribution of the **raw random numbers** at each time. Instead, it suffices to determine a lower **min-entropy** bound for the **raw random numbers**. Disadvantages are that **cryptographic** functions are usually more costly than **algorithmic post-processing** algorithms (implementation, computation time etc.). Furthermore, the ratio between the **raw random numbers** bits needed as input and the generated **internal random number** bits can be smaller than for a tailored **algorithmic post-processing** algorithm. Example (stateless **post-processing** algorithm): Apply the SHA-256 hash function to n **raw random number** bits. The number n shall be selected with regard to the **stochastic model**, or more precisely, with regard to a lower **entropy** bound for the **raw random numbers**.
- 344 [PTG.2.2] Pars. 344 and 345 refer to both **Shannon entropy** and **min-entropy**. **Min-entropy** is mentioned in brackets. If the **Shannon entropy** per **raw random number** bit is below 0.9998 (the **min-entropy** is below 0.98), the **algorithmic post-processing** algorithm must increase the **entropy** per **internal random number** bit. This is not possible without data compression. The evaluator has to verify that the **Shannon entropy** per **internal random number** bit is ≥ 0.9998 (the **min-entropy** is ≥ 0.98). It is not necessary to quantify the exact **entropy** value.
- 345 [PTG.2.2] If the **entropy** per **raw random number** bit already is equal or larger than the PTG.2-specific boundary (**Shannon entropy**: 0.9998, **min-entropy**: 0.98), it suffices to show that the **post-processing** does not decrease the **entropy** per bit. Usually, this is much easier than quantifying the gain of **entropy** per bit. An example is a **post-processing** algorithm (**with memory**) that is injective for each admissible value of the memory, and for which the bit length of the elements of the range is less than or equal to the bit length of the elements of the domain. In this case, the **post-processing** algorithm maintains the **entropy** per bit.
If the **post-processing** algorithm uses **widely recognized cryptographic primitives** (not necessary for functionality class PTG.2), then **post-processing** can often be modeled as a random bijection or as a random mapping, just as for functionality classes DRG.2, DRG.3, and DRG.4; cf. pars. 97 to 104.
- 346 [PTG.2.2] Exemplarily, pars. 351 to 354 discuss Requirement PTG.2.2 by two **stochastic models**. Note that in par. 351 to 354 the **stochastic models** *are only claimed but not justified*. **Stochastic models** are treated in detail in Sections 4.5 and 5.4.
- 347 [PTG.2.2] The verification of Requirement PTG.2.2 shall be supported by statistical tests of the **raw random numbers**. The tested **raw random numbers** shall be generated under representative relevant environmental conditions (cf. par. 367).
Note: The evaluator may also decide to apply adapted statistical tests to the **internal random numbers**.

[PTG.2.2] This paragraph gives advice on how the **Shannon entropy** and the **min-entropy** can be computed for **independent and identically distributed (iid) random variables** and Markov chains. For more information, we refer to Subsections 4.3.1 and 5.1. We consider exemplarily three cases. (i) The **random variables** $Z_1, Z_2, \dots$ are **iid** and $\text{Prob}(Z_j = 1) \in [0.4931, 0.5069]$. In particular, the developer can apply the one-dimensional **entropy** formulas. Then, $H(Z_j) > 0.9998$ and $H_{\min}(Z_j) > 0.98$. (ii) The **random variables** $Z_1, Z_2, \dots$ form a homogeneous Markov chain on $\Omega = \{0, 1\}$ with state transition matrix P . The **Shannon entropy** and the **min-entropy** can be computed by (4.107) and (4.135), respectively. If, for example, $\text{Prob}(Z_j = 1) \in [0.494, 0.506]$ and $|\text{Prob}(Z_{j+1} = 0 \mid Z_j = 0) - \text{Prob}(Z_{j+1} = 0 \mid Z_j = 1)| \leq 0.001$, then $H(Z_j) > 0.99989$ and $H_{\min}(Z_j) > 0.981$. (iii) The **random variables** $Z_1, Z_2, \dots$ form a homogeneous Markov chain on a finite state space Ω . Then (4.124) (in place of (4.135)) can be applied to determine a set of appropriate parameters that meet the **min-entropy** bound. If $|\Omega| > 2$ the **min-entropy** per **internal random number** bit (averaged over all bit positions) is relevant. If $Z_1, Z_2, \dots$ form a 2-step Markov chain on Ω , at first a (1-step) Markov chain has to be constructed as described in par. 638.

[PTG.2.2] Par. 348 can be applied to both the **raw random numbers** and the **internal random numbers**, or more precisely, to the corresponding **random variables** $R_1, R_2, \dots$ and $Y_1, Y_2, \dots$, if these **random variables** are **iid** or form a homogeneous Markov chain. If the **raw random numbers** already fulfill Requirement PTG.2.2 and if the **post-processing** neither reduces the **Shannon entropy** nor the **min-entropy**, the **entropy** claim can directly be transferred to the **internal random numbers**. If the **PTRNG** provides significantly more **entropy** than needed, it may be reasonable to apply non-optimal (but easy-to-prove) **entropy** estimates. To give an example: For Markov chains, the average gain of **min-entropy** per bit is trivially bounded from below by $-\log_2(\max_{i,j}\{p_{ij}\})$, where $P = (p_{ij})$ denotes the transition matrix. If necessary, the more accurate formulae (4.135) and (4.124) can be applied.

[PTG.2.2] It may be the case that even for unlocked test devices, the evaluator does not have access to the **raw random numbers**. This can constitute a serious (unsolvable) problem for the evaluation of a **PTRNG** that should have been considered during the design phase of the **PTRNG**. In consultation with the evaluator, the developer may try to capture the necessary data using external measurement equipment (e.g., a logic analyzer). In exceptional cases, it might be possible to test the **internal random numbers** instead, provided that this allows well-founded conclusions on the stochastic properties (e.g., **entropy**, bias, dependencies) of the **raw random numbers**. In any case, the applicant must be able to define, verify, and analyze a **stochastic model** of the **internal random numbers**.

Note: This option is not recommended. The certification process will most probably fail in practice if the requirements mentioned above are not fulfilled.

[PTG.2.2: Example A] The **raw random numbers** are interpreted as realizations of binary-valued **random variables** $R_1, R_2, \dots$. Based on the **stochastic model**, the developer provides evidence that the **random variables** $R_1, R_2, \dots$ are **stationarily distributed**. (Requirement PTG.2.1 is satisfied.) Furthermore, based on the **stochastic model** and supported by tailored statistical tests, the developer provides evidence that a bias may exist, but no (significant) k -step dependencies for $k \geq 1$. Altogether, it is reasonable to consider the **random variables** $R_1, R_2, \dots$ to be **iid**. Assume that $\text{Prob}(R_j = u) \in (0.5 - \epsilon_0, 0.5 + \epsilon_0)$ for $u \in \{0, 1\}$.

A.1 Assume that

$$\epsilon_0 = 0.003 \quad \text{or, equivalently,} \quad \text{Prob}(R_j = 1) \in (0.497, 0.503). \quad (3.9)$$

Then $H(R_j) \geq 0.99997$ and $H_{\min}(R_j) \geq 0.991$.

Conclusion: If the **post-processing** algorithm does not reduce the **entropy** per bit, the **PTRNG** fulfills Requirement PTG.2.2 (**Shannon entropy** claim and **min-entropy** claim).

Note: Alternatively, the developer could point to par. 348(i), saving their own calculations.

A.2 Assume that $\epsilon_0 = 0.03$.

Without a data-compressing **post-processing** algorithm, this **PTRNG** violates Requirement PTG.2.2. XORing non-overlapping pairs of **raw random number** bits, i.e., $Y_1 = R_1 \oplus R_2, Y_2 = R_3 \oplus R_4, \dots$, guarantees $\text{Prob}(Y_j = 1) \in (0.4982, 0.5018)$, $H(Y_j) \geq 0.99999$, and $H_{\min}(Y_j) \geq 0.994$.

Conclusion: The **PTRNG** fulfills Requirement PTG.2.2 (**Shannon entropy** claim and **min-entropy** claim).

352 [PTG.2.2, Example A continued] Assertion A.1 follows by substituting the least favorable parameters into the one-dimensional **Shannon entropy** formula (4.88) and the one-dimensional **min-entropy** formula (4.89). The second claim of Assertion A.2 follows from (5.2) with $k = 2$; cf. pars. 925 and 926.

353 [PTG.2.2: Example B] The **raw random numbers** are interpreted as realizations of binary-valued **random variables** $R_1, R_2, \dots$. Based on the **stochastic model**, the developer provides evidence that the **random variables** $R_1, R_2, \dots$ are **stationarily distributed**. (Requirement PTG.2.1 is satisfied.) Furthermore, based on the **stochastic model** and supported by tailored statistical tests, the developer gives evidence that a bias and 1-step dependencies may exist but no (significant) k -step dependencies for $k \geq 2$. Altogether, it is reasonable to consider the **random variables** $R_1, R_2, \dots$ to be Markovian distributed. Assume that $\text{Prob}(R_j = u) \in (0.5 - \epsilon_0, 0.5 + \epsilon_0)$ for $u \in \{0, 1\}$ and $|\text{Prob}(R_{j+1} = 0 \mid R_j = 0) - \text{Prob}(R_{j+1} = 0 \mid R_j = 1)| \leq \epsilon_1$.

B.1 Assume that

$$\epsilon_0 = 0.004, \quad \text{or, equivalently,} \quad \text{Prob}(R_j = 1) \in (0.496, 0.504), \quad \epsilon_1 = 0.003 \quad (3.10)$$

Then $H(R_j \mid R_{j-1}, \dots, R_1) = H(R_j \mid R_{j-1}) \geq 0.99995$ and $\frac{H_{\min}(R_{n+1}, \dots, R_{n+m})}{m} \rightarrow_{m \rightarrow \infty} 0.9845$.

Conclusion: If the **algorithmic post-processing** does not reduce the **entropy** per bit, the **PTRNG** fulfills Requirement PTG.2.2 (**Shannon entropy** claim and **min-entropy** claim).

B.2 Assume that $\epsilon_0 = 0.01$, and $\epsilon_1 = 0.012$.

Without a data-compressing **post-processing** algorithm, this violates Requirement PTG.2.2, e.g., because the bias is too large. XORing non-overlapping pairs of **raw random number** bits (= **algorithmic post-processing**), i.e., $Y_1 = R_1 \oplus R_2, Y_2 = R_3 \oplus R_4, \dots$, guarantees $H(Y_{n+1} \mid Y_1, \dots, Y_n) \geq 0.99989$, and $H_{\min}(Y_j) \geq 0.9712$, and $\text{Prob}(Y_j = 1) \in (0.4938, 0.5062)$.

Conclusion: The **PTRNG** satisfies the **Shannon entropy** condition but not the **min-entropy** condition of PTG.2.2; cf. par. 354. The bias of the **internal random numbers** lies in the permitted interval.

[PTG.2.2, Example B continued] Assertion B.1 and the first claim of Assertion B.2 of par. 353 follow by substitution into the Shannon entropy formula and in the min-entropy formula for Markov chains; cf. par. 348(ii). The Shannon entropy claim and the distribution of Y_j in assertion B.2 follow from (5.5) and the inequality (5.6) with $k = 2$; cf. pars. 926 and 927. The result on the min-entropy was obtained as in (5.6). In particular,

$$H_{\min}(Y_{m+1} \mid Y_1, \dots, Y_m) \geq H_{\min}(R_{2m+1} + R_{2m+2}(\bmod 2) \mid R_m) \geq -\log_2(\max\{p_{ij}p_{jk} + p_{i(1-j)}p_{(1-j)(1-k)} \mid 0 \leq i, j, k \leq 1\}) . \quad (3.11)$$

provides a lower min-entropy bound. A larger min-entropy bound may be achievable but would require a more sophisticated approach.

[PTG.2.2] In par. 353, Example B.2, the entropy per bit is increased by XORing non-overlapping pairs of raw random numbers. Another option would be to thin out the raw random numbers by a factor of 2, i.e., by outputting only every second raw random number bit. This would also slightly increase the entropy per bit. The internal random numbers then would be Markovian with transition matrix P^2 instead of P .

Note: Thinning the raw random numbers out does not reduce the bias. Generally speaking, thinning the raw random numbers out is not very efficient unless the 1-step dependencies are rather large. It is an option, however, to thin out beforehand (as part of the digitization mechanism, equivalent to reducing sample rate) and interpret the resulting values as the raw random numbers.

[PTG.2.3] The start-up test shall be applied when the RNG is started after the TOE has been powered up, reset, rebooted, etc., or after the operation of the RNG has been stopped (e.g., to reduce the power consumption of the TOE). If the physical noise source requires a warm-up period after powering up, the start-up test shall be applied after the warm-up period. It is important that no internal random numbers are output (or stored and output later) before the start-up test has been passed. The start-up test shall be designed to detect a total failure of the physical noise source and severe statistical weaknesses; cf. Subsection 4.5.5. The start-up test can apply the online test, possibly with different evaluation rules; cf. Subsection 4.5.5.

[PTG.2.4] When the PTRNG is in operation, the online test shall detect if the entropy Requirement PTG.2.2 is violated. If a defect occurs, it should usually affect Requirement PTG.2.2. This cannot (or at least not reliably) be achieved by black box testing without considering the nature of the physical noise source. Instead, the online test shall be tailored to the stochastic model, and its effectiveness shall be proven on the basis of the stochastic model.

[PTG.2.4] Of course, if the developer claims both Shannon entropy and min-entropy, the online test shall detect if any claim is violated. In particular, the developer needs to specify appropriate parameters. The online test shall detect sufficiently soon when the PTRNG leaves the specified set of appropriate parameters (implicitly given by the class requirements). If the PTRNG generates internal random numbers that have significantly more entropy than required, this usually simplifies the task of designing an effective (and efficient) online test. These aspects are discussed and explained in detail in Subsection 4.5.3.

[PTG.2.4] To be able to fulfill the tasks addressed in par. 358, the impact of the post-processing algorithm on the entropy of the internal random numbers has to be analyzed. An effective online

test should have a low probability of failing if the true distribution of the **raw random numbers** is appropriate (**false positive**) and shall have a high probability of triggering a **noise alarm** if the distribution is inappropriate. The reader is referred to Subsection 4.5.3 for detailed explanations.

- 360 [PTG.2.4] The **online test** should be applied to the **raw random numbers**. In exceptional cases, it might be possible to test the **internal random numbers** instead. This requires that the applicant is able to determine the possible distributions of the **internal random numbers**, i.e., to formulate, justify, and analyze a **stochastic model** of the **internal random numbers**. This may be possible in favorable cases (e.g., for **iid stochastic models** with simple **algorithmic post-processing**, but usually the analysis will be more difficult than for **online tests** on the **raw random numbers**. Note: This approach is not recommended and can fail in practice.
- 361 [PTG.2.4] The **online test** may be applied continuously, at regular (short) intervals, or upon specified internal events. The analysis shall take into account the calling scheme of the **online test** for the verification of its suitability. The applicant shall specify the consequences of a **noise alarm**. These aspects are also a subject of the evaluation. For general considerations, further explanations, and examples, we refer to Subsection 4.5.3.
- 362 [PTG.2.5] A **total failure** of the **physical noise source** implies that without intervention, Requirement PTG.2.2 would drastically be violated (e.g., because the next **raw random number** bits have no **entropy** at all or, at best, have very low **entropy**). The **total failure test** may include statistical tests, but other solutions (voltage sensors, etc.) may be acceptable as well. The effectiveness of the **total failure test** shall be proven based on a substantiated failure analysis of the **physical noise source** and the impact of (if applicable) further design features; cf. pars 363 and 364. For general considerations, the reader is referred to Subsection 4.5.4.
- 363 [PTG.2.5] Buffering the **internal random numbers** before they are output and/or a suitable **post-processing** algorithm can increase the permitted detection and reaction time. Unlike for **cryptographic post-processing** algorithms (cf. pars. 855, 414 to 421), for stateful **algorithmic post-processing** algorithms the **internal state** cannot be interpreted as an “**entropy buffer**”. Instead, precise analysis is needed that **raw random numbers** that have been generated after the **total failure** do not spoil the generated **internal random numbers**, e.g., cf. the examples in par. 853. Further considerations to FIFOs and **algorithmic post-processing** algorithms are explained, and examples are discussed in Subsection 4.5.4; cf. in particular pars. 849, 850, 852, 853, and 854.
- 364 [PTG.2.5] If the **PTRNG** design uses a **cryptographic post-processing** algorithm that is compliant with functionality class DRG.2 or DRG.3, Requirement PTG.2.5 factually coincides with PTG.3.8. The application notes pars. 414 to 421 can be applied accordingly if the **intermediate random numbers** are included by a **reseeding procedure** or as **high-entropy additional input**; cf. the last two sentences of Requirement PTG.3.5.
- 365 [PTG.2.4, PTG.2.5] If the **total failure test** and/or the **online test** are not part of the **TOE** but are to be implemented later as an external security measure, then the applicant must submit an accurate specification of the **online test** and/or of the **total failure test**, as well as a reference implementation. The tasks concerning the verification that PTG.2.4 and/or PTG.2.5 are fulfilled remain unaffected. The specification of the tests shall be part of the user manual (guidance documents). The **online test** of the final **PTRNG** implementation shall exactly fulfill the specification of the user manual (to be checked later in a composite evaluation) in order to be

PTG.2-compliant.

[PTG.2.6] The statistical (**black box**) test suite T_{irn} (cf. Subsection 4.6.3) shall be applied 366
under representative environmental conditions (cf. par. 367). Depending on the **PTRNG** design,
the developer or evaluator may apply further statistical tests. For functionality class PTG.2,
the importance of comprehensive statistical tests is incomparably higher than for functionality
classes DRG.2, DRG.3, DRG.4, DRT.1, PTG.3, and NTG.1. It can happen that an **algorithmic**
post-processing algorithm transforms statistical defects of **raw random numbers** to (statistical)
defects of the **internal random numbers**. If a **cryptographic post-processing** algorithm is applied,
statistical defects of the **internal random numbers** are not likely, but the evaluator shall apply
the test suite T_{irn} in any case.

[environmental conditions] Environmental conditions (temperature, voltage, etc.) are viewed as 367
relevant in this context if they belong to the specified range of permitted operating conditions. A
selection of parameter sets (temperature, voltage, etc.) is representative if the tests under these
environmental conditions allow drawing conclusions on the behavior of the **raw random numbers**
(or the **internal random numbers**) under other environmental conditions within the permitted
operating conditions.

Note: Within the vulnerability analysis, one may perform tests for environmental conditions
that lie outside of the permitted range. If the **physical noise source** works properly under these
environmental conditions, too, then this may, to some extent, relax the requirements on the
anti-tamper measures (e.g., voltage sensor for attacks on the power supply). This is, however,
not part of AIS 20/31; cf. pars. 24 and 25.

3.4.3 Functionality Class PTG.3

Functionality class PTG.3 defines requirements for **hybrid PTRNGs**. The differences from func- 368
tionality classes PTG.2 and DRG.4 are explained in par. 378.

Functionality class PTG.3 is the strongest functionality class. It defines requirements for **RNGs** 369
that are appropriate for any cryptographic application. Unlike for PTG.2-compliant **PTRNGs**,
the security of PTG.3-compliant **PTRNGs** does not only rely on **information-theoretic security**
ensured by the **physical noise source** (in combination with **algorithmic post-processing**) but, addi-
tionally, also on **computational security** ensured by **cryptographic post-processing**. In particular,
the **internal random numbers** will not show any bias or short-term dependencies. The **crypto-**
graphic post-processing can reduce the **entropy** defect per **intermediate random number** bit by
data compression.

Functionality class PTG.3 demands a **cryptographic post-processing** algorithm that (interpreted 370
as a **DRNG**) is DRG.3-compliant even if its input data, the **intermediate random numbers**, would
become predictable at some point in time. **Intermediate random numbers** can be used as **seed**
material for the **reseeding procedure** or **additional input**.

The **TSF** has to protect the **internal state** of the **RNG** from being compromised. 371

Note: Par. 373 does not replace an evaluation methodology. 372

373 PTG.3-specific deliverables by the applicant

The security architecture description and developer evidence shall contain at least

- a description of the **physical noise source** (including the **digitization** mechanism),
- a comprehensive description of the “algorithmic behavior” of the **PTRNG** beginning with the **digitization** of the **raw random numbers**,
- a **stochastic model** of the **raw random numbers** with substantiated justification, statistical evidence, and thorough analysis,
- evidence that PTG.3.1, PTG.3.3, and PTG.3.5 are fulfilled,
- a description of the **cryptographic post-processing** algorithm and evidence that PTG.3.2, and PTG.3.4 are fulfilled,
- a description of the **start-up test** and evidence that PTG.3.6 is fulfilled,
- a description of the **online test** and evidence that PTG.3.7 is fulfilled,
- a description of the **total failure test** and evidence that PTG.3.8 is fulfilled,
- evidence that PTG.3.9 is fulfilled.

374 PTG.3: Security functional requirements

Security functional requirements of functionality class PTG.3 are defined by component **FCS_RNG.1** with specific operations as given below.

375 FCS_RNG.1 Random number generation (Class PTG.3)

FCS_RNG.1.1 The **TSF** shall provide a *hybrid physical* random number generator that implements:

- (PTG.3.1) The **TSF** shall generate **raw random numbers** that can be viewed as realizations of a (time-local) **stationary** stochastic process $R_1, R_2, \dots$
- (PTG.3.2) If the **cryptographic post-processing** algorithm runs autonomously it shall algorithmically be compliant with functionality class **DRG.3**.
- (PTG.3.3) The **intermediate random numbers** that are input to the **cryptographic post-processing** algorithm [selection, choose one of:
are generated by a PTG.2-compliant **PTRNG**,
are generated by a **PTRNG** that fulfills Requirement PTG.3.1 and guarantees that [selection, choose one of:
the **Shannon entropy** per **intermediate random number** bit is greater than or equal to [assignment: $v_S \in [0.4, 0.9998]$],
the **min-entropy** per **intermediate random number** bit is greater than or equal to [assignment: $v_m \in [0.1, 0.98]$],
the **Shannon entropy** per **intermediate random number** bit is greater than or

equal to [assignment: $v_S \in [0.4, 0.9998]$] and the *min-entropy* per *intermediate random number* bit is greater than or equal to [assignment: $v_m \in [0.1, 0.98]$]. The *intermediate random numbers* are input into the *cryptographic post-processing* algorithm by the *reseeding procedure* or as *high-entropy additional input*. The *PTRNG* may apply both input methods to include *intermediate random numbers*.

(PTG.3.4) The *intermediate random numbers* can included by the *reseeding procedure* or as *additional input*. The *cryptographic post-processing* algorithm shall not expand its input sequence. In other words: The input rate (the *intermediate random numbers*, counted in bits) shall be greater than or equal to the output rate (the *internal random numbers*, counted in bits). The *compression rate* C_{rate} is the ratio between input rate and output rate and it shall be ≥ 1 . The compression shall be appropriate.

(PTG.3.5) The *TSF* shall guarantee that [selection, choose one of:
the *Shannon entropy* per *intermediate random number* bit is greater than or equal to 0.9998 and the *cryptographic post-processing* does not expand its input,
the *min-entropy* per *intermediate random number* bit is greater than or equal to 0.98 and the *cryptographic post-processing* does not expand its input,
the *Shannon entropy* and *min-entropy* per *intermediate random number* bit is greater than or equal to 0.9998 and 0.98, respectively, and the *cryptographic post-processing* does not expand its input,
the *Shannon entropy* per *internal random number* bit is greater than or equal to [assignment: $v_S \in [0.9998, 1 - 2^{-32}]$],
the *min-entropy* per *internal random number* bit is greater than or equal to [assignment: $v_m \in [0.98, 1 - 2^{-32}]$],
the *Shannon entropy* per *internal random number* bit is greater than or equal to [assignment: $v_S \in [0.9998, 1 - 2^{-32}]$] and the *min-entropy* per *internal random number* bit is greater than or equal to [assignment: $v_m \in [0.98, 1 - 2^{-32}]$]]]

(PTG.3.6) The *start-up test* shall be applied after the *RNG* has been started. It shall be designed to detect a *total failure* of the *physical noise source* and severe statistical weaknesses. The *TSF* shall not output any *internal random numbers* before the *start-up test* has successfully been passed.

(PTG.3.7) The *online test* shall check the quality of the *raw random numbers* while the *RNG* is being operated. The *online test* shall be designed to detect non-tolerable *entropy* defects of the *raw random numbers* sufficiently soon. The *TSF* shall not output any *internal random numbers* if a non-tolerable *entropy* defect has been detected.

(PTG.3.8) The *total failure test* shall detect if a *total failure* of the *physical noise source* occurs while the *PTRNG* is in operation. After the *total failure* has occurred, the output of *internal random numbers* bits whose *entropy* per bit is not close to 1 shall be prevented.

FCS_RNG.1.2 The *TSF* shall provide [selection, choose one of: *bits*, *octets of bits*, *numbers* [assignment: *format of the numbers*]] that meet:

(PTG.3.9) The *internal random numbers* shall have *statistical inconspicuousness*.

Application notes

- 376 [PTG.3 vs. PTG.2] The general application notes in pars. 328, 329, 330 (stochastic model), 331, 332 (several physical noise sources), and 367 (environmental conditions) apply to functionality class PTG.3 as well.
- 377 [PTG.3 vs. PTG.2] Functionality class PTG.3 includes many of the requirements of functionality class PTG.2. In particular, the Requirements PTG.2.1 and PTG.3.1, PTG.2.3 and PTG.3.6, and PTG.2.4 and PTG.3.7 coincide. The corresponding application notes for functionality class PTG.2 apply to PTG.3, too. The Requirements PTG.2.6 and PTG.3.8 partially coincide.
- 378 [PTG.3 vs. PTG.2, PTG.3 vs. DRG.4] By using cryptographic post-processing rather than only algorithmic post-processing, functionality class PTG.3 assures computational security even if the raw random numbers or the intermediate random numbers are compromised, provided that the internal state of the cryptographic post-processing algorithm has already received enough entropy by the intermediate random numbers, which should be the case shortly after the PTRNG has been started. In contrast with functionality class DRG.4, functionality class PTG.3 does not allow the cryptographic post-processing algorithm to “extend” its input data.
- 379 [PTG.3.1] Requirements PTG.3.1 and PTG.3.3 concern the raw random numbers and the intermediate random numbers, which are computed from the raw random numbers. More precisely, Requirement PTG.3.3 focuses on the entropy of the intermediate random numbers. For the “typical design” (where the intermediate random numbers are generated by a PTG.2-compliant “inner” PTRNG), both requirements are/were already part of the PTG.2-evaluation.
- 380 [PTG.3.2] Requirement PTG.3.2 concerns the algorithmic properties of the cryptographic post-processing algorithm. It refers to functionality class DRG.3, or more precisely, to the Requirements DRG.3.3 and DRG.3.5 to DRG.3.8.
- 381 [PTG.3.2] The post-processing algorithm shall not be stateless, i.e., the state shall not be deleted after generating one or more random numbers because otherwise, the PTG.3 would lose its “with memory” property, which is a security feature. If the internal state is deleted, a start-up test has to be applied before the next internal random numbers can be output (PTG.3.6). The intermediate random numbers shall influence the next internal state.
- 382 [PTG.3.3] Usually, a PTG.2-compliant PTRNG generates the intermediate random numbers for the cryptographic post-processing algorithm. The entropy requirement per bit is meant to be understood as in par. 335, Note 1.
For PTG.3-designs without an “inner” PTG.2-compliant PTRNG, the Shannon entropy claim for the intermediate random numbers can be smaller than 0.9998 per bit, and/or the min-entropy claim can be smaller than 0.98 per bit (PTG.2-specific entropy bounds; cf. PTG.2.2). Higher entropy claims for the intermediate random numbers than specified in functionality class PTG.2, Requirement PTG.2.2, are not accepted. The reasons are explained for functionality class PTG.2 in par. 319. The case that the intermediate random numbers are generated by a PTG.3-compliant PTRNG is treated in Section 3.6, par. 476.
- 383 [PTG.3.3] The intermediate random numbers that are fed into the cryptographic post-processing

algorithm shall be untampered with (integrity), authentic, and kept secret. This, of course, is essential for the **entropy** claim of the **internal random numbers**. Verification of these security claims is part of the overall evaluation of the **TOE**; cf. application note 237.

[PTG.3.4] By Requirement PTG.3.4, **fresh entropy** can be introduced by the **reseeding procedure** and/or as **additional input**. If the **reseeding procedure** is used to include the **intermediate random numbers**, the state transition function shall be applied after the last **internal random number** has been generated. When the **PTRNG** starts, the **internal state** of the (DRG.3-compliant) **cryptographic post-processing** algorithm has not yet received any **entropy** from **intermediate random numbers**. Thus, *only at the start of the PTRNG* a **seeding procedure** could be used in place of a **reseeding procedure**. 384

[PTG.3.4] Assume that the **PTRNG** outputs v **internal random numbers** (k -bit vectors) per **intermediate random number** (n bits). Here, $v = 1$ may be typical, but $v > 1$ is possible. We set $m = v \cdot k$. Of course, for $v = 1$ it is $k = m$. For a fixed **internal state** s , the **cryptographic post-processing** of an **intermediate random number** can be viewed as a mapping $\chi_s: \{0, 1\}^n \rightarrow \{0, 1\}^m$, depending on the **internal state** s . The mapping χ_s describes the generation of v random numbers. Depending on the method for feeding the **intermediate random numbers** into the **PTRNG**, this mapping may include the **reseeding procedure** of the **cryptographic post-processing** algorithm, the application of the state transition function and the output function. 385

[PTG.3.4, compression] Requirement PTG.3.4 separates PTG.3-compliant **PTRNGs** from DRG.4-compliant **DRNGs** because $n \geq m$ is demanded, or equivalently, $c_{\text{rate}} \geq 1$. To increase the **entropy** per **internal random number** bit beyond the **entropy** per **intermediate random number** bit, it is necessary to apply a **compression rate** $c_{\text{rate}} > 1$. 386

[PTG.3.4, compression] The **compression rate** c_{rate} is computed per **intermediate random number**, i.e., the bit length n of an **intermediate random number** is divided by the number of **internal random number** bits (may belong to several **internal random numbers**) that are generated before the next **intermediate random number** is input. If the bit length of the **intermediate random numbers** is not constant, n is set to the minimal guaranteed bit size. In order to not make the evaluation too complicated, the use of a sliding average over several **intermediate random numbers** is not allowed. 387

[PTG.3.4, appropriate compression] The calculation of the **compression rate** c_{rate} alone is not sufficient. Instead, it needs to be effective in the following sense. The following example may be pathological but points to the fundamental problem. 388

Example: Assume that the **seed material** used for the **reseeding procedures** of the **cryptographic post-processing** algorithm is generated by a PTG.2-compliant **PTRNG** and comprises 4096 bits. Assume further that the **internal state** of the **cryptographic post-processing** algorithm only comprises 256 bits. Outputting 4096 **internal random number** bits after each **reseeding procedure** would result in $c_{\text{rate}} = 1$. However, this data compression is not effective. The output of more than 256 **internal random number** bits is not permitted because the **internal state** obviously cannot “store” more than 256 bits of **entropy**.

Note 1: The requirement that the effective **compression rate** is ≥ 1 is important if the developer decides on using one of the three first selections in PTG.3.5. If in PTG.3.5 the developer chooses a concrete **entropy** claim, this problem would become evident in the **entropy** analysis.

Note 2: The requirement that the compression shall be effective shall exclude pathological de-

signs. It is closely related to the concept of “narrowest internal width” in [SP800-90B].

- 389 [PTG.3.4] Example: We determine (n, m) and c_{rate} for several designs and explain the calculations. In this paragraph, we assume that the **cryptographic post-processing** algorithm is the **DRNG** that is defined in pars. 994 and 995. We summarize its relevant features: It is $S = S_{\text{req}} = R = \{0, 1\}^{256}$, and both the state transition function $\phi_{(H2)}: S \times A \times I \rightarrow S$ and the output function $\psi_{(H2)}: S_{\text{req}} \rightarrow R$ essentially apply the SHA-256 hash function. More precisely, if s and a denote the current **internal state** and the current **intermediate random number** (treated as **additional input**), the next **internal random number** equals $\text{SHA-256}(s||00||a)$, and the next **internal state** is given by $\text{SHA-256}(s||11||a)$. In this example the length of a **request** is limited by 256 bits, the bit length of a single **internal random number**. Alternatively, the **intermediate random numbers** can be used as **seed material** for the **reseeding procedure** of the **cryptographic post-processing** algorithm; cf. Requirement PTG.3.4. In this example, the **reseeding procedure** is rather simple: A 256-bit **seed material** string is XORed to the current **internal state**. The **internal state** comprises 256 bits and thus cannot store more than 256 bits of **entropy**. As a first consequence, only one **internal random number** can be output per **reseeding procedure**. Otherwise, Requirement PTG.3.4 would be violated. Furthermore, we assume that the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG**. Below, five examples are briefly discussed that differ only with regard to the **additional input**.

- [(i)] Per **intermediate random number** a with bit length $|a| = 256$ (used as input either for the **reseeding procedure**, or as **additional input**), one **internal random number** is output.
Conclusion: Requirement PTG.3.4 is fulfilled with $(n, m) = (256, 256)$ and $c_{\text{rate}} = 1$.
- [(ii)] An **intermediate random number** a with $|a| = 280$ is used as **additional input**, and one **internal random number** is output.
Conclusion: Requirement PTG.3.4 is fulfilled with $(n, m) = (280, 256)$, and thus $c_{\text{rate}} = 280/256 = 1.09 \geq 1$.
- [(iii)] 512-bit **intermediate random numbers** are used as **seed material** for the **reseeding procedures**. Two **internal random numbers** are output between two successive **reseeding procedures**.
Conclusion: It is $c_{\text{rate}} = 512/512 = 1$, but Requirement PTG.3.4 is violated because the **internal state** cannot store more than 256 bits of **entropy**.
Note: The output of only one **internal random number** between two successive **reseeding procedures** would satisfy Requirement PTG.3.4.
- [(iv)] Periodically, **intermediate random numbers** a with $|a| = 320$ and $|a| = 270$ are used as **additional input**, and one **internal random number** is output.
Conclusion: Requirement PTG.3.4 is fulfilled with $(n, m) = (\min\{320, 270\}, 256)$, and thus $c_{\text{rate}} = 270/256 = 1.05 \geq 1$; cf. par. 387.
- [(v)] Periodically, the **additional input** is given by **intermediate random numbers** a with $|a| = 1024$, $a = o$ (“empty string”, i.e., no **additional input**), $a = o$, $a = o$, and after each **additional input** (including the empty **additional inputs**), an **internal random number** is output.
Conclusion: This design does not fulfill Requirement PTG.3.4 because the **internal state** cannot “store” more than 256 bits of **entropy**.
Note: Instead, a sequence of **intermediate random numbers** with $|a| = 512$, $a = o$, $|a| = 512$, $a = o$, etc. would be possible with $(n, m) = (512, 512)$, and thus $c_{\text{rate}} = 1$.

Justification: The **entropy** of the first 256 bits of the non-empty **intermediate random number** is “directly” used for the generation of an **internal random number** while its second 256 bits provides **fresh entropy** to the **internal state**. The second **internal random number** “uses” this **fresh entropy**. In the notation of par. 995 the two outputted **internal random numbers** can be expressed as follows: $\chi_s: \{0, 1\}^{512} \rightarrow \{0, 1\}^{512}$, $\chi_s(a) = (\psi_{(H_2)}(\phi_{req(H_2)}(s, a)), \psi_{(H_2)}(\phi_{req(H_2)}(\phi_{(H_2)}(a, s), \{\}))$. Here, a denotes a non-empty **additional input** value of length 512 bits. The two components of $\chi_s(a)$ can be viewed as “incompatible”.

[PTG.3.4] Example: Below, three further examples are discussed. As in par. 389, we assume 390 that the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG**.

[(i)] Hash_ DRBG, see Subsection 5.3.1. An **intermediate random number** a with $|a| \geq outlen$ is used as **additional input**, and one **internal random number** is output.

Conclusion: Requirement PTG.3.4 is fulfilled with $(n, m) = (|a|, outlen)$, and thus $c_{rate} \geq 1$. Only one **internal random number** can be output, regardless of $|a|$.

Justification: The **additional input** (here: an **intermediate random number**) is first mapped to an $outlen$ -bit value $f(v, a)$, cf. (5.43), (5.45), (5.47). The **entropy** of $f(v, a)$ is limited by $outlen$ bits.

[(ii)] Hash_ DRBG, see Subsection 5.3.1. An **intermediate random number** a with $|a| \geq seedlen$ is input by the **reseeding procedure**.

Conclusion: If one **internal random number** is output, Requirement PTG.3.4 is fulfilled with $(n, m) = (\geq seedlen, outlen)$, and thus $c_{rate} \geq seedlen/outlen \geq 1$.

Special case: For $Hash = SHA-384$ we have $seedlen > 2 \cdot outlen$. If two **internal random numbers** are output, Requirement PTG.3.4 is fulfilled with $(n, m) = (seedlen, 2 \cdot outlen) = (888, 768)$, and thus, $c_{rate} = 888/768 = 1.16 \geq 1$.

Justification: cf. Example (i)

[(iii)] The **cryptographic post-processing** is performed by the **DRNG**. that is described in pars. 996 to 998. Per **intermediate random number** a with $|a| = 128$ (**additional input**), one **internal random number** is output.

Conclusion: Requirement PTG.3.4 is fulfilled with $(n, m) = (128, 128)$, and thus $c_{rate} = 1$.

[PTG.3.5] Requirement PTG.3.5 is central because it considers the **entropy** of the **internal random numbers**. In the first three choices, this is done “indirectly” because no concrete lower **entropy** bound is quantified. In contrast, the last three choices allow (in certain limits) individual **entropy** claims. There, the **entropy** per **intermediate random number** bit shall be greater than or equal to some specified threshold value. This pertains to both **Shannon entropy** and **min-entropy**. A **min-entropy** claim for the **internal random numbers** is only possible if there is a (verified) **min-entropy** claim for the **intermediate random numbers**. The same applies to **Shannon entropy** claims. Par. 413 addresses a (coarse) transfer of **min-entropy** claims to **Shannon entropy** claims, which is always possible. 391

[modeling by a random mapping] A **cryptographic post-processing** algorithm can usually be described by a mapping $\chi_s: \{0, 1\}^n \rightarrow \{0, 1\}^m$, or more generally, by a composition of mappings $\chi_w \circ \dots \circ \chi_1$ ($w \geq 1$). The case $w > 1$ usually occurs when the **intermediate random numbers** are used to reseed the **cryptographic post-processing** algorithm. If the **intermediate random** 392

numbers are used as high-entropy additional input, often (but not always) $w = 1$. Of course, it supports (eases) the evaluation, if each mapping χ_j either is bijective or can be modeled by a random mapping. To be more precise: The second case means that χ_j can be interpreted as a realization of a random mapping, or for short, as being randomly selected. A random mapping is a random variable that is uniformly distributed on $\mathcal{F}_{\{0,1\}^n, \{0,1\}^m} = \{\chi: \{0,1\}^n \rightarrow \{0,1\}^m\}$; cf. Section 4.4. The parameters n and m depend on the concrete scenario. Then in particular, the methods described in par. 405 and 406 can be applied; cf. also par. 407. In the remainder of this paragraph, we assume that $f \in \{\chi_1, \dots, \chi_w\}$.

The following subitems specify a positive list of typical scenarios for which it is permitted to view f as a randomly selected mapping.

(i) The mapping f is a widely recognized dedicated hash function (widely recognized cryptographic primitive). This, in particular, applies to SHA-224, SHA-512/224, SHA-256, SHA-512/256, SHA-384, SHA-512, SHA3-224, SHA3-256, SHA3-384, SHA3-512.

(ii) The mapping f is the projection of a mapping $\tilde{f}$ onto a subset of output bits if $\tilde{f}$ can be viewed as a randomly selected mapping; cf. par. 738.

(iii) It is $f = (f_1, f_2)$, and f_1 and f_2 can be viewed as a randomly selected mappings. Furthermore, f_1 and f_2 must be (computationally) “incompatible”; cf. par. 738.

Counterexample: $f_1 = f_2$.

Note: This list is not complete. In case of doubt, the certification body should be contacted.

393 [modeling by a composition of random mappings] In this paragraph, the mapping f is given by an HMAC, or more precisely, $f(x) := \text{HMAC}_k(x) := \text{Hash}((k \oplus \text{opad}) \parallel \text{Hash}((k \oplus \text{ipad}) \parallel x))$. Here, Hash denotes a widely recognized dedicated hash function; cf., e.g., par. 392(i). As usual, ipad and opad are known constants. Furthermore, the key k is also assumed to be known. The HMAC construction cannot be modeled as a randomly selected mapping. Instead, we express $f: \{0,1\}^n \rightarrow \{0,1\}^h$ as the composition of two mappings χ_1 and χ_2 , i.e., $f = \chi_2 \circ \chi_1$. Here, h denotes the output length of Hash , and $\chi_1: \{0,1\}^n \rightarrow \{0,1\}^h$ and $\chi_2: \{0,1\}^h \rightarrow \{0,1\}^h$ are given by $\chi_1(x) := \text{Hash}((k \oplus \text{ipad}) \parallel x)$ and $\chi_2(y) := \text{Hash}((k \oplus \text{opad}) \parallel y)$, respectively. Both, χ_1 and χ_2 can be modeled by randomly selected mappings, and f can be viewed as the composition of two randomly selected mappings.

394 [modeling by a random mapping] Below, the impact of many cryptographic post-processing algorithms is thoroughly analyzed. For a randomly selected mapping $f: \{0,1\}^n \rightarrow \{0,1\}^m$, the (min-) entropy of the output values can be very small, even 0 if f is constant. Fortunately, such events are extremely rare. In any case, lower min-entropy bounds can only be guaranteed with probability $\geq 1 - p$, which means that the lower min-entropy bound is undercut at most with probability p . For functionality class PTG.3.5, we use $p = 2^{-16}$, i.e., for each randomly selected mapping, the min-entropy claim is satisfied with probability $\geq 1 - 2^{-16}$. Or in other words: The true min-entropy undercuts this lower bound only with probability $\leq p = 2^{-16}$.

Note 1: It is tolerable that the lower entropy bound is undercut with probability $\leq p = 2^{-16}$, in particular because the internal state s changes each time after fresh entropy has been included. Thus, the effect of “unfavorable” mappings should average out when generating several random numbers; cf. par. 680.

Note 2: The threshold $1 - 2^{-16}$ holds for any modeling of the cryptographic post-processing algorithm, not only if it can be modeled by a random mapping.

Note 3: Applying the results on random mappings from Section 4.4 (without additional proofs) requires $n, m \geq 128$; cf. par. 695, Note 1.

395

[PTG.3.5, Typical design] Usually, a PTG.2-compliant “inner” PTRNG generates the **intermediate random numbers** that are used as input for **cryptographic post-processing**. The PTRNG then can be viewed as a composition of a PTG.2-compliant PTRNG and the DRG.3-compliant **cryptographic post-processing** algorithm. For PTG.2-compliant “inner” PTRNGs, Requirement PTG.2.2 specifies class-specific **entropy** thresholds. If the applicant aims at one of the three first **entropy** claims of Requirement PTG.3.5, i.e., that the **entropy** of the **intermediate random numbers** is greater than or equal to the PTG.2-specific **entropy** bound(s) and that **cryptographic post-processing** does not expand the input data (**intermediate random numbers**), the verification of Requirement PTG.3.5 usually is rather simple. Indeed, this does not require **entropy** analysis of the impact of **cryptographic post-processing**. Concrete **entropy** claims (last three choices of Requirement PTG.3.5) for the **internal random number** bits requires **entropy** analysis. The next paragraphs address the relevant aspects.

[PTG.3.5] PTG.3-compliant designs without a clear inner “PTG.2-boundary” are also permitted 396 if the **raw random numbers** fulfill Requirement PTG.2.1 (or PTG.3.1, respectively) and Requirement PTG.3.3. If the **entropy** of the **intermediate random numbers** does not meet Requirement PTG.2.2, this has to be compensated for by data compression. In particular, the first three choices of Requirement PTG.3.5 and $n = m$ (or equivalently, $c_{\text{rate}} = 1$) is not permitted then. Of course, the lower **entropy** bound per **intermediate random number** bit has to be taken into account when verifying concrete **entropy** bounds for the **internal random number** bits. As will become clear below, the **entropy** analysis is essentially the same as for the “typical design”.

[PTG.3.5, $n = m$] For $n = m$, i.e., for $c_{\text{rate}} = 1$, the **cryptographic post-processing** cannot 397 increase the **entropy** per bit. Analogously to the case $n > m$, we describe the **cryptographic post-processing** algorithm by a mapping $\chi_s: \{0, 1\}^n \rightarrow \{0, 1\}^n$ where s again denotes the **internal state**. Most designs fall into one of the following two categories:

(i) $\chi_s: \{0, 1\}^n \rightarrow \{0, 1\}^n$ can be interpreted as a randomly selected mapping, or χ_s can be interpreted as a composition of randomly selected mappings. Although such **post-processing** algorithms decrease the **entropy** per bit to some degree, they usually have positive effects. This is because they remove possible bias and short-term dependencies of the **intermediate random number** by “smearing” the weaknesses over the **internal random number**, thereby counteracting practical attacks and increasing the **computational security**.

Note: If χ_s can be interpreted as a randomly selected mapping, for each $a_2 \in \{0, 1\}^n$, the pre-image size $|\chi_s^{-1}(\{a_2\})|$ can be interpreted as a realization of a **random variable** that is Poisson distributed with parameter $\tau = 1$; cf. par. 704.

Example: The **internal random numbers** result from a cryptographic hash of the current **internal state** and the **intermediate random number**.

(ii) The mapping $\chi_s: \{0, 1\}^n \rightarrow \{0, 1\}^n$ is bijective for each s . Then the **cryptographic post-processing** algorithm preserves the **entropy** of the **intermediate random numbers**.

[PTG.3.5, $n = m$, typical design] Assume that the **intermediate random numbers** are generated 398 by a PTG.2-compliant PTRNG. If par. 397 (i) applies, the developer can choose one of the first three choices in PTG.3.5 (depending on the **entropy** claim of the used PTG.2). A concrete **entropy** bound of the **internal random number** bits (last three choices) cannot be claimed because χ_s reduces the **entropy** of the **intermediate random numbers**. If par. 397 (ii) applies, all choices are possible. (Of course, **min-entropy** claims are limited to $v_m = 0.98$, while **Shannon entropy** claims are limited to $v_S = 0.9998$.)

399

[PTG.3.5, $n = m$, typical design] The following example uses the analysis from pars. 389 and 390.

Example: Assume that the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** that satisfies both the **Shannon entropy** condition and the **min-entropy** condition; cf. Requirement PTG.2.2.

The following designs allow the claim of the third choice of Requirement PTG.3.5: par. 389[(i)], par. 389[(ii)], par. 389[(iv)], par. 389[(v)] (only the scenario that is described in the note), par. 390[(i)], par. 390[(ii)], par. 390[(iii)].

The following designs allow the claim of the third or the sixth choice of Requirement PTG.3.5: par. 389[(ii)], par. 389[(iv)], par. 390[(ii)], par. 390[(iii)] (with $v_S = 0.9998$, $v_m = 0.98$).

- 400 [PTG.3.5] Functionality class PTG.3 allows the claim of a **PTRNG**-specific **entropy** bound. Unless the **cryptographic post-processing** is bijective, this requires $c_{\text{rate}} > 1$. However, when determining a lower **entropy** bound of the **internal random numbers**, it may be necessary to analyze an intermediate step with $n \leq m$. This can, e.g., occur when **fresh entropy** is inserted by a **reseeding procedure** and if the **effective internal state** is much larger than the size m of the **internal random numbers**. Prominent examples are Hash_DRBG and HMAC_DRBG. Below, general strategies are explained and illustrated by several examples. In case of doubt, the certification body should be contacted.
- 401 [PTG.3.5] For **algorithmic post-processing** algorithms, the **stochastic model** of the **raw random numbers** has to be taken into account (functionality class PTG.2). In contrast, the analysis of **cryptographic post-processing** algorithms does not need to consider the **stochastic model** of the **intermediate random numbers** or of the underlying **raw random numbers**, respectively. Instead, only the **entropy** claim of the **intermediate random numbers** has to be considered. This allows composite evaluations where, for example, a software developer uses the output of a certified PTG.2-compliant **PTRNG** that was designed and manufactured by another company. The software developer does not need to know any details of the **PTRNG** design (the usual scenario). Possibly, specifications from the user manual have to be implemented (e.g., the **online test** or the **total failure test**; cf. application note 365). Of course, within the composite evaluation, the designer and evaluator have to give evidence that the design fulfills all requirements of functionality class PTG.3 that were not already certified by the PTG.2-evaluation.
- 402 [PTG.3.5] Functionality class PTG.3 defines true **RNGs**. In particular, it only credits the gain of **entropy** of the **internal random numbers** implied by the latest **intermediate random number**. The reason is, from an **information-theoretic security** point of view, that the previous **internal random numbers** reveal at least parts of the **internal state**. This strategy also covers worst-case scenarios where an **adversary** has been able to temporarily compromise the **internal state**.
- 403 [PTG.3.5] As above, we describe the **cryptographic post-processing** of an **intermediate random number** by a mapping $\chi_s: \{0, 1\}^n \rightarrow \{0, 1\}^m$ that is parametrized by the **internal state** s . Again, n denotes the bit length of the **intermediate random number**, and m stands for the number of generated **internal random number** bits. (Usually, $m = k$ or a small integer multiple of the bit size k of the **internal random numbers**.) This description applies to all admissible techniques (i.e., to the **reseeding procedure** and the insertion of high-entropy additional input). To verify an **entropy** claim for the **internal random numbers**, the **cryptographic post-processing** algorithm usually has to be modeled; bijective mappings (cf. pars. 397 (ii) and 398) are exceptions that are easy to handle, but they do not increase the **entropy** per bit ($n = m$). Often, the **crypto-**

graphic post-processing can be modeled as a randomly selected mapping $\chi_s: \{0,1\}^n \rightarrow \{0,1\}^m$; see Section 4.4.2 for a comprehensive treatment of random mappings, and cf. pars. 404 ff. Especially, if fresh entropy is included by reseeding, the entropy analysis usually expresses χ_s by a composition of several mappings, which can either be modeled by randomly selected mappings or are bijective. Different scenarios are explained and analyzed below. Bijective mappings have already been treated in pars. 397 to 399.

Note: In order not to overstress the model (e.g., of random mappings), Requirement PTG.3.5 limits both the min-entropy claim and the Shannon entropy claim per internal random number bit to $1 - 2^{-32}$.

Note: If the min-entropy per bit is $\geq 1 - 2^{-32}$, [SP800-90C] speaks of “full entropy”.

[PTG.3.5, random mappings] Subsection 4.4.2, pars. 702 to 708, treat the impact of random mappings on the Shannon entropy. Pars. 709 to 742 (end of Subsection 4.4.2) considers the impact on min-entropy. The results on the min-entropy are much more extensive. Random mappings $\{0,1\}^n \rightarrow \{0,1\}^m$ of four different types are analyzed: (i) ($n > m$, $\gamma \gg 1$), (ii) ($n \leq m$, or $n \gtrapprox m$), (iii) ($n = m$, scenario 3), and (iv) ($n = m$, scenario 4). For background information and concrete formulae, the reader is referred to Subsec. 4.4. To be more precise, the following paragraphs are relevant:
(i) pars. 714 to 729, (ii) pars. 712, 713, 730, and 731, (iii) pars. 739 and 740 (Case (iii)), (iv) pars. 741 and 742. In the application notes below, all four cases are explained and illustrated by concrete examples.

[PTG.3.5, $\gamma \gg 1$, random mappings] For uniformly distributed input data, par. 719, formula (4.214), allows specifying concrete min-entropy bounds for the output bits that are undercut only with some (small) probability p . Tab. 4 (par. 720) provides exemplary data. As already mentioned in par. 394, for functionality class PTG.3 the value $p = 2^{-16}$ is relevant. In the context of functionality class PTG.3, the input data are intermediate random numbers. Of course, for “real-world” PTRNGs, the intermediate random numbers are not uniformly distributed (at least not perfectly). Formula (4.216) quantifies the number of extra input bits (compared to uniformly distributed input data) that are necessary to guarantee the same min-entropy bound; cf. pars. 724 to 729. Usually, the developer first specifies the values m (given by the intended cryptographic post-processing algorithm) and the desired maximum entropy defect e_{def} per internal random number bit. By (4.214) a (minimal) value for n can be calculated, for which the desired entropy defect e_{def} is exceeded at most with probability $p = 2^{-16}$ (for uniformly distributed input data). Therefore, in the first step, by (4.215), the threshold value $z_{2^{-16}}$ is calculated and substituted into (4.214). Finally, the minimum bit length of the intermediate random numbers is calculated with (4.216).

The evaluator can apply formula (4.216) in the reverse direction. First, by (4.225) $n^- := \lfloor n \cdot h_m \rfloor$ is computed where h_m for the moment denotes the min-entropy per intermediate random number bit. That is, for a given bit length n of the intermediate random numbers, the designer/evaluator determines an input bit length n^- for a fictitious PTRNG design, for which the intermediate random numbers are generated from an ideal RNG and the cryptographic post-processing χ_s (or mappings χ_j within a composition of mappings; cf. par. 392) is given by a randomly selected mapping. As shown in Subsection 4.4.2, in terms of min-entropy per internal random number bit, the real-world PTRNG is at least as good as the fictitious PTRNG.

If $n^- \geq m + 16$ (cf. par. 407(ii)), in a second step (4.214) can be applied with $z = z_{2^{-16}}$; cf. par. 720. This provides an upper bound for the min-entropy defect (or, equivalently, a lower min-entropy bound) per internal random number bit of the PTRNG under evaluation; the latter undercut only with probability $\leq 2^{-16}$. Pars. 408, 409, and 411 illustrate the procedure by

several examples.

406 [PTG.3.5, $n \leq m$, or $n \gtrsim m$, random mappings] The formulae (4.216) and (4.224) coincide. As a consequence, the “conversion” from input length n to fictitious input length n^- is analogous to the case $\gamma \gg 1$, which is described in par. 405. In particular, $n^- := \lfloor n \cdot h_m \rfloor$. Provided that $n^- \leq m + 6$ in a second step par. 712, formula (4.202), can be applied with $p = 2^{-16}$. This provides an upper bound for the **min-entropy** defect (or, equivalently, a lower **min-entropy** bound) per **internal random number** bit of the **PTRNG** under evaluation. Pars. 410 and 411 illustrate the procedure by examples.

407 [PTG.3.5, random mappings, requirements, and limitations]

(i) The approach described in pars. 405 and 406 is generic and can be applied if the **intermediate random numbers** are compliant with Requirement PTG.3.3. It is not limited to the “typical design” where **intermediate random numbers** are generated by an “inner” PTG.2-compliant **PTRNG**.

(ii) The case $n > m$, $\gamma \gg 1$: A necessary condition to apply this approach is $n^- \geq m + 16$ (fictitious parameters (n^-, m)). This mainly results from the condition $\frac{2^{n^- - m}}{m \log(2)} \rightarrow \infty$ (par. 717).

(iii) The case $n \leq m$, or $n \gtrsim m$: A necessary condition to apply this approach is $n^- \leq m + 6$ (fictitious parameters (n^-, m)). This mainly results from the fact that in par. 712 a binomial distribution is approximated by a Poisson distribution.

(iv) In scenario 3 and scenario 4 (cf. par. 404 and pars. 739 to 742) it is $n = m$.

(v) Of course, functionality class PTG.3 does not exclude the case $m + 6 < n^- < m + 16$ but the applicant has to provide their own (further) arguments that support the **entropy** claim.

Note: The **Shannon entropy** claim refers to the average **entropy** per bit (averaged over a sequence of **internal random numbers**), while the **min-entropy** claim holds for the **entropy** per bit of any **internal random number** (averaged over all bit positions) with probability $\geq 1 - 2^{-16}$. Since the **internal state** changes permanently, smaller **entropy** values for very few **internal random numbers** are averaged out and are overcompensated; cf. par. 680.

408 [PTG.3.5, $n > m$, $\gamma \gg 1$] Example: Assume that n -bit **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** with the **min-entropy** claim ≥ 0.98 . Assume further that $(n, m) = (327, 256)$. We first conclude that the **min-entropy** per **intermediate random number** is at least $327 \cdot 0.98 = 320.45 > 320 =: n^-$. Applying (4.214) with $(n^-, m, z = z_{2^{-16}})$ gives an upper bound for the **min-entropy** defect per **internal random number** bit of $2^{-32.93}$ (cf. Tab. 4, first row, last column) that is only exceeded with probability 2^{-16} for the fictitious **ideal RNG**. This justifies the following **min-entropy** claim for the (real) **PTRNG** under evaluation: The **min-entropy** per **internal random number** bit exceeds $1 - 2^{-32.93}$.

Note 1: Requirement PTG.3.5 allows at most a **min-entropy** claim for $v_m = 1 - 2^{-32}$. Larger **min-entropy** claims are not permitted. Thus, the above analysis justifies a **min-entropy** claim for $v_m = 1 - 2^{-32}$.

Note 2: Assume that the **min-entropy** per **intermediate random number** bit is only ≥ 0.6 (would also satisfy Requirement PTG.3.3). By (4.216), the bit length of the **intermediate random numbers** then must be at least $\lceil \frac{320}{0.6} \rceil = 534$.

409 [PTG.3.5, $n > m$, $\gamma \gg 1$] Example: We assume the “typical design” where the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG**.

(i) (par. 389[(ii)]): Here $(n, m) = (280, 256)$. The fictitious parameters are $n^- = \lfloor 280 \cdot 0.98 \rfloor = \lfloor 274.4 \rfloor = 274$ and $m = 256$. Applying (4.214) with $(n^-, m, z = z_{2^{-16}})$ gives an upper bound

for the **min-entropy** defect per **internal random number** bit of $2^{-9.81}$ that is exceeded only with probability $\leq 2^{-16}$ for the fictitious **ideal RNG**. This justifies the following **min-entropy** claim for the (real) **PTRNG** under evaluation: The **min-entropy** per **internal random number** bit is greater than or equal to $v_m = 1 - 2^{-9.81} \approx 0.9989$.

(ii) (par. 389[(iv)]): Here $(n, m) = (270, 256)$; cf. par. 389[(iv)]. The fictitious parameters are $n^- = \lfloor 270 \cdot 0.98 \rfloor = \lfloor 264.4 \rfloor = 264$ and $m = 256$. Since $6 < n^- - m = 264 - 256 = 8 < 16$ neither the method described in par. 405 nor the method from par. 406 can be applied, at least not without further argumentation. The problem can be overcome as follows: Instead of $(n^- = 264, 256)$, consider the less favorable parameters $(n^{(2)-} := 262, 256)$. Then par. 407(iii) allows to use the method from par. 406. Finally, this leads to the **min-entropy** claim $v_m = 1 - 0.0155 = 0.9845$.

[PTG.3.5, $n \leq m$, or $n \gtrsim m$, **reseeding procedure**] Example:

410

We consider the same **cryptographic post-processing** algorithm as in par. 389. Only the **reseeding procedure** is different: It is given by $\phi_{\text{reseed}}: \{0, 1\}^{256} \times \{0, 1\}^* \times ps \rightarrow \{0, 1\}^{256}$, $\phi_{\text{reseed}} := \text{SHA-256}(11 \| s \| a \| ps)$. The **internal state** coincides with the **effective internal state**. After the **reseeding procedure**, one **internal random number** is generated by $\psi(s) := \text{SHA-256}(s \| 00)$. In (i) to (iii), we compute the **min-entropy** of $S = \{0, 1\}^{256}$ after reseeding (as usual, relative to a privileged **adversary** who knows the previous **internal state** s and the **personalization string** ps).

(i) The **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** with **min-entropy** claim, and $|a| = 266$. Then $(n, m) = (266, 256)$, and by (4.225) $n^- = \lfloor 266 \cdot 0.98 \rfloor = \lfloor 260.68 \rfloor = 260$; i.e., $n^- = 260$. By (4.202) the **min-entropy** of the **internal state** space $\{0, 1\}^{256}$ equals 252.85 bits. In particular, (for $p = 2^{-16}$) the **min-entropy** defect per bit equals 0.012 bit.

(ii) The **intermediate random numbers** are generated by a **PTRNG** that fulfills the Requirements PTG.3.1 and PTG.3.3. The **intermediate random numbers** comprise 655 bits with **min-entropy** ≥ 0.4 per bit. Here $(n, m) = (655, 256)$. Analogous calculations as in (i) lead to $n^- = 262$, and the **min-entropy** of the **internal state** space $\{0, 1\}^{256}$ equals 253.92 bits. In particular, (for $p = 2^{-16}$) the **min-entropy** defect per bit equals 0.008 bit.

(iii) The **intermediate random numbers** are generated by a **PTRNG** that fulfills the Requirements PTG.3.1 and PTG.3.3. The **intermediate random numbers** comprise 665 bits with **min-entropy** claim is 0.4 per bit. Here $(n, m) = (665, 256)$. Analogous calculations as in (i) lead to $n^- = 268$. Here, $n^- > m + 6$ and $n^- < m + 16$ so that none of the discussed methods can directly be applied. An option for the applicant is to “downsize” $n^- = 266$ to $n^{(2)-} = 262$ (as in par. 409), which allows to apply the method from par. 406. A larger **min-entropy** claim would require additional justifications.

Note: If $n^- \geq m + 16$ the method for $\gamma \gg 1$ applies.

(iv) Hash_DRBG with $\text{Hash} = \text{SHA-256}$; cf. par. 390[(i)] and Subsection 5.3.1:

The **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** with **min-entropy** claim. The **intermediate random numbers** are included by the **reseeding procedure**. The **seed material**, the **personalization_string**, and the **internal state** before reseeding are denoted by a, pm and s' . Further, $\phi_{\text{reseed},(s', pm)}(\cdot)$ denotes the reseeding function of the Hash_DRBG where s' and pm are included as parameters in the definition of $\phi_{\text{reseed},(s', pm)}(\cdot)$. The **internal state** after the **reseeding procedure** is $s_1 = (v_1, c_1, 1) \in \{0, 1\}^{440} \times \{0, 1\}^{440} \times \mathbb{Z}_{2^{19}}$. The **effective internal state** comprises $2 \cdot 440$ bits; cf. par. 1068. Because c_1 is a function value of s_1 , the first component $S' = \{0, 1\}^{440}$ contains the whole **entropy** of the **effective internal state** space. Thus, for the **entropy** analysis only the first component is relevant.

Assume that the bit length of the **intermediate random numbers** is 337. Then $(n, m) = (337, 440)$, and $n^- = \lfloor 337 \cdot 0.98 \rfloor = 330$. By (4.202) the **min-entropy** of the first component of the **effective internal state** space, $S' = \{0, 1\}^{440}$, equals 328 bits (for $p = 2^{-16}$).

Note: In this scenario, the loss of **min-entropy** is rather small.

(v) Hash_DRBG with $Hash = \text{SHA-384}$; cf. par. 390[(ii)] and Subsection 5.3.1:

The **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** with **min-entropy** claim. The **intermediate random numbers** are included by the **reseeding procedure**. We use the notation from (iv). Again, $\phi_{\text{reseed},(s',pm)}(\cdot)$ denotes the reseeding function of the Hash_DRBG where s' and pm are included into the definition of $\phi_{\text{reseed},(s',pm)}(\cdot)$ as parameters. The **internal state** after the **reseeding procedure** is $s_1 = (v_1, c_1, 1) \in \{0, 1\}^{888} \times \{0, 1\}^{888} \times \mathbb{Z}_{2^{19}}$. The **effective internal state** comprises $2 \cdot 888$ bits; cf. par. 1068. Because c_1 is a function value of s_1 , the first component $S' = \{0, 1\}^{888}$ contains the whole **entropy** of the **internal state**. Thus, as in (iv) for the computation of the **entropy**, only the first component is relevant.

Assume that the bit length of the **intermediate random numbers** is 870. Then $(n, m) = (870, 888)$, and $n^- = \lfloor 870 \cdot 0.98 \rfloor = 852$. By (4.202), the **min-entropy** of the **effective internal state** space $\{0, 1\}^{888}$ equals 847 bits (for $p = 2^{-16}$).

411 [PTG.3.5, $n \leq m$, $n \gtrsim m$, and $\gamma \gg 1$ (i.e., $n \gg m$)] Example: In this example, we assume the “typical design” where the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** (with **min-entropy** claim). For “non-typical” designs, the **min-entropy** can be computed analogously; cf. par. 410 (ii), (iii).

(i) [continues par. 410(i)] After the **internal state** has been reseeded one **internal random number** $r = \chi(s) := \text{SHA-256}(s)$ is generated. By par. 392 (i), we can model $\chi(\cdot)$ by a randomly selected mapping ($n = m$). By par. 410(i), the **min-entropy** of the **internal state** is 252.85. Thus, $n^- = 252$, and the **min-entropy** per **internal random number** bit is $246.91/256 = 0.965$ bits. Thus, a specific **min-entropy** claim is not possible. On the other hand, $\phi_{\text{reseed}}: \{0, 1\}^{266} \rightarrow \{0, 1\}^{256}$ and $\chi: \{0, 1\}^{256} \rightarrow \{0, 1\}^{256}$ do not extend data. In particular, the input rate of $\chi_1 \circ \phi_{\text{reseed}}: \{0, 1\}^{266} \rightarrow \{0, 1\}^{256}$ is greater than the output rate. Hence, the **PTRNG** satisfies the second choice of Requirement PTG.3.5. If the PTG.2-compliant **PTRNG** additionally satisfies the PTG.2-specific **Shannon entropy** claim, the **PTRNG** also satisfies the third choice of Requirement PTG.3.5.

Note: Even if the **min-entropy** of the **internal state** was 256 (ideal situation), the **min-entropy** per **internal random number** bit would only be ≥ 0.977 (with probability $\geq 1 - 2^{-16}$); cf. par. 713. Consequently, a specific **min-entropy** claim is not possible, regardless of how long the **seed material** is. In any case, a particular **min-entropy** claim can only be achieved by truncating some output bits.

(ii) [continues par. 410(iv)] The next **internal random number** is given by $\text{SHA-256}(v_1 + 1 \pmod{2^{256}})$ (no **additional input!**) where v_1 denotes the first component of the **internal state** after the **reseeding procedure**. By par. 392 (i), we can model $\chi(\cdot)$ by a randomly selected mapping ($n = m$). By par. 410(iv), the **min-entropy** of the **internal state** is 328. Here, $n^- = 328$, and the **min-entropy** per **internal random number** bit is $\geq 1 - 2^{-36.7}$. Thus, it is possible to claim the fifth choice of Requirement PTG.3.5 for $v_m = 1 - 2^{-32}$.

Note 1: Obviously, per **reseeding procedure** only one **internal random number** can be output.

Note 2: Assume that the **min-entropy** per **intermediate random number** bit is only ≥ 0.4 (would also satisfy Requirement PTG.3.3). To claim **min-entropy**, by (4.216) the bit length of the **intermediate random numbers** must be at least $\lceil \frac{330}{0.4} \rceil = 825$.

(iii) [continues par. 410(v)] As in par. 410(v), we assume that the **intermediate random numbers** comprise 870 bits that are generated by a PTG.2-compliant **PTRNG**. In this example, per **reseeding procedure** two **internal random numbers** $r_1, r_2 \in \{0, 1\}^{384}$ are output. We distinguish two cases. In Case 1, we assume that r_1, r_2 are generated without applying the state transition function in between, while Case 2 treats the scenario that the state transition function is applied in between. The **entropy** analysis for Case 2 is considerably more difficult than for (ii) or Case 1.

We split the generation of (r_1, r_2) into several steps. The first step (**reseeding procedure**) was already studied in par. 410(v). In particular, the first fictitious parameter for the **seed material** was $n^{-(1)} = 852$, and after reseeding the **min-entropy** of the first component of the **internal state space**, $S' = \{0, 1\}^{888}$, is 847.

Case 1: Here, $(r_1, r_2) = (\text{SHA-384}(v_1), \text{SHA-384}(v_1 + 1 \bmod 2^{888}))$. Due to the properties of the dedicated hash function SHA-384, the mappings $t \mapsto \text{SHA-384}(t)$ and $t \mapsto \text{SHA-384}(t + 1 \bmod 2^{888})$ can be viewed as randomly selected mappings, which are “incompatible”. By par. 392 (iii), the mapping $\tau(t) := (t \mapsto \text{SHA-384}(t), t \mapsto \text{SHA-384}(t + 1 \bmod 2^{888}))$ can be viewed as randomly selected mapping. $\tau: \{0, 1\}^{888} \rightarrow \{0, 1\}^{768}$ with fictitious parameters $(n^{-(2)}, m) = (847, 768)$. Consequently, the **min-entropy** per **internal random number** bit is $\geq 1 - 2^{-40.9}$.

Case 2: It is $r_1 = \chi_1(v_1)$ and $r_2 = \chi_1 \circ \phi_{B; c_1}(v_1)$. Here $\chi_1: \{0, 1\}^{888} \rightarrow \{0, 1\}^{384}$, $\chi_1(t) = \text{SHA-384}(t)$ while $\phi_B: \{0, 1\}^{888} \rightarrow \{0, 1\}^{888}$ is given by $\phi_B(v_1) := v_1 + c(v_1) + 1 + g(v_1) \bmod 2^{888}$. The value $c_1 := c(v_1)$ is computed from v_1 by a **one-way function** $c: \{0, 1\}^{888} \rightarrow \{0, 1\}^{888}$, $g(v_1) = \text{SHA-384}(0x03\|v_1)$, and the term “+1” come from the `reseed_counter`.

In the second step, we study the mapping $\psi: \{0, 1\}^{888} \rightarrow \{0, 1\}^{384} \times \{0, 1\}^{888}$, $\psi(v) = (\psi_1(v), (\psi_2(v)) := (\chi_1(v), v + c(v) + 1 \bmod 2^{888}))$. By pars. 739 and 740, ψ_2 can be viewed as a randomly selected mapping because the addition “+1(mod 2⁸⁸⁸)” induces a bijection. Consequently, by par. 392(iii), the mapping ψ itself can be modeled by a random mapping. Let the **random variable** V describe the distribution on $\{0, 1\}^{888}$ after reseeding. The second fictitious parameter is $n^{-(2)} = 847$. The usual calculations give $H_{\min}(\psi(V)) = 845.4$, $H_{\min}(\psi_1(V)) = 384$, and $-\log_2(\min. \text{pre-image size of } \psi_1(V)) = 384$; cf. par. 734. (The second and third number deviate only negligibly from 384.) By par. 628, the conditional **min-entropy** $H_{\min}(\psi_2(V) \mid \psi_1(V)) = 845.4 - 384 = 461.4$. The **entropy** analysis of the first component $\psi_1(V)$ is completed, but we still have to analyze the further progress of the second component $\psi_2(V)$. In the next step, we consider the mapping $\{0, 1\}^{888} \rightarrow \{0, 1\}^{888}$, $w \mapsto w + g(w) \bmod 2^{888}$ with a randomly selected mapping $g: \{0, 1\}^{888} \rightarrow \{0, 1\}^{384}$. By (4.236) and (4.232) we obtain that the remaining **min-entropy** is at least $880.73 + 461.4 - 888 = 454.1$. Finally, the impact of a randomly selected mapping $\chi: \{0, 1\}^{888} \rightarrow \{0, 1\}^{384}$ has to be considered with fictitious parameter $n^{-(4)} = 454$. The **min-entropy** per bit of r_2 (under the condition that r_1 is known) is greater or equal than $1 - 2^{-35.96}$. Altogether, these considerations allow the **min-entropy** claim that r_1 and r_2 are greater or equal than $1 - 2^{-32}$.

Note 3: For Case 1, **seed material** with smaller **min-entropy** would suffice for the **min-entropy** claim $1 - 2^{-32}$.

Note 4: As usual, in (i) to (iii) all steps of the **entropy** analysis are valid with probability $1 - 2^{-16}$.

Note 5: The **entropy** analysis in Examples (i) to (ii) does not need to consider the impact of the state transition function that is applied after the last **internal random number** has been generated.

[PTG.3.5, HMAC] In this paragraph, the HMAC construction from par. 393 with $Hash = \text{SHA-256}$ is analyzed. The key $k = \tau(s)$ is a function of the **internal state** s while the **fresh entropy** is included only by the argument x (**intermediate random number**, included as **high-entropy additional input**). Here, $\chi_s := f = \chi_2 \circ \chi_1$ would be the output function, while the state transition function is not considered here. Furthermore, the **min-entropy** per **intermediate random number** is ≥ 280 . (For the “typical design” where the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** (with **min-entropy** claim), this is guaranteed if x comprises ≥ 286 bits.) Following the strategy from par. 393, we analyze the HMAC construction $\chi_s := f = \chi_2 \circ \chi_1$ in two steps.

Since $280 > 256 + 16$, we apply the method from par. 405 for $(n^-, m) = (280, 256)$. By this, the

min-entropy of the intermediate value after applying χ_1 is ≥ 255.96 bits. To quantify the impact of χ_2 , we apply the method from par. 406 with $(n^{-(2)}, m) = (255, 256)$. The **min-entropy** per **internal random number** is ≥ 249.33 bits, so that the **min-entropy** per **internal random number** bit is ≥ 0.974 bit.

Note 1: The **min-entropy** per **internal random number** bit does not suffice to apply to the fifth or sixth **entropy** claim of Requirement PTG.3.5.

Note 2: For the “typical design” (with **min-entropy** claim), this design allows to apply to the second and possibly also to the third **entropy** claim of Requirement PTG.3.5.

Note 3: The analyzed **cryptographic post-processing** algorithm is not appropriate to apply to the fifth or sixth **entropy** claim of Requirement PTG.3.5, regardless of the **min-entropy** of the **intermediate random numbers**. Even if the **min-entropy** after the application of the mapping χ_1 would be maximal (i.e., 256 bits), the **min-entropy** per **internal random number** bit does not exceed 0.9771.

Note 4: The situation from Note 3 changes if SHA-256 is replaced by SHA-384. If the **min-entropy** of the intermediate state after the application of the mapping χ_1 would be maximal (i.e., 384 bits), then the **min-entropy** per **internal random number** bit would be ≥ 0.983 . If the **min-entropy** of the **intermediate random numbers** is sufficiently large (e.g., ≥ 400 bits), it is possible to apply to the fifth possibly also to the sixth **entropy** claim of Requirement PTG.3.5.

Note 5: The analysis performed in this paragraph also applies for fixed key k when the HMAC construction is a **cryptographic post-processing** algorithm *without memory* (although clearly in terms of content, strictly speaking, this formulation contradicts our definition of **cryptographic post-processing**). Of course, this would be not compliant with functionality class PTG.3 but, in principle, would be a **post-processing** candidate for functionality class PTG.2. By Note 3, the HMAC construction with $Hash = \text{SHA-256}$ cannot be used for functionality class PTG.2, unless some output bits are canceled, or equivalently, the **internal random numbers** bits are projected onto a subset; cf. par. 392 (ii).

- 413 [PTG.3.5, **min-entropy**, **Shannon entropy**] Assume that the evaluation process has confirmed that the **min-entropy** per **internal random number** bit is at least $v_m \in [0.98, 1 - 2^{-32}]$. Since $H_{\min}(X) \leq H(X)$ for each **random variable** X , this (trivially) implies that the **Shannon entropy** v_s per **internal random number** bit is $\geq v_m$.
- 414 [PTG.3.8] Requirement PTG.3.8 customizes PTG.2.5 to the given situation. Unlike for (specific) **algorithmic post-processings** algorithms, for **cryptographic post-processings** algorithms it is not possible to identify a subset of bits of the **internal state** that does not have an impact on the next **internal random number**. Instead, the **effective internal state** can be viewed as an “**entropy** buffer”. The intuition behind this is that the **effective internal state** should have accumulated **entropy** since the start of the PTRNG; cf. par. 855.
- 415 [PTG.3.8, evaluation rule] For the evaluation of the **total failure** test, the developer/evaluator can assume that each **effective internal state** bit has a maximal **entropy** of 1. This assumption applies to **Shannon entropy** and **min-entropy** as well. The following evaluation rule can be used: If t' bits of **entropy** after the **total failure** remain, Requirement PTG.3.8 still allows the output of no more than t' **internal random number** bits.
Note: The term “remaining bits of **entropy**” is explained in pars. 419 to 421 by examples.
- 416 [PTG.3.8] Note 1: Requirement PTG.3.8 and the evaluation rule stated in par. 415 might seem to be inconsistent. In fact, a more stringent formulation of PTG.3.8 would be: The **internal**

random numbers that are output after a total failure shall satisfy the entropy claim selected in PTG.3.5. If choices 4, 5, or 6 were selected, this would require an entropy analysis similar to that for Requirement PTG.3.5. The evaluation rule from par. 415 still allows outputting as many internal random numbers bits as the remaining entropy of the effective internal state. In many cases, this evaluation rule simplifies the tasks of the designer or evaluator, e.g., because the impact of the previous state transition function on the entropy does not have to be considered. Note 2: This simplification does not lead to security problems. First of all, the entropy per internal random number bit is close to 1 anyway, and additionally, the cryptographic post-processing algorithm provides computational security. Furthermore, a total failure is assumed to be a rare event.

[PTG.3.8] In the “typical design”, the intermediate random numbers are generated by an “inner” PTRNG that is compliant with functionality class PTG.2. In particular, the inner PTRNG shall satisfy Requirement PRTG.2.5. If no intermediate random number is available, the PTG.3 shall not output any further internal random numbers. Further considerations are not necessary. Justification: Two cases are possible:

Case 1: The PTG.2-compliant inner PTRNG outputs no intermediate random numbers bits (no post-processing, algorithmic post-processing, and/or output buffer; cf. par. 364) that depend on the raw random numbers that have been generated since a total failure.

Case 2: The PTG.2-compliant inner PTRNG applies a cryptographic post-processing algorithm (with memory) with an effective internal state of t' bits that is compliant with functionality class DRG.2 or DRG.3. Then, at most t' intermediate random number bits are available that have been generated after the total failure using the effective internal state as an “entropy buffer”; cf. pars. 414 and 415.

[PTG.3.8] Pars. 419 to 421 treat PTRNG designs where no (appropriate) total failure test has already been implemented for the physical noise source. We tacitly assume that there is some mechanism (sensor, dedicated statistical test, or similar) that detects (with some delay) a total failure of the physical noise source. The examples in pars. 419 to 421 explain exemplarily how Requirement PTG.3.8, and in particular the evaluation rule from par. 415, can be satisfied.

[PTG.3.8, high-entropy additional input] Example: We assume that the effective internal state of the DRG.3-compliant cryptographic post-processing algorithm comprises t bits, and that it has essentially t bits of entropy; cf. par. 414. The intermediate random number a_{n+1} influences the next internal random number r_{n+1} as an argument of the output function, i.e., $r_{n+1} = \psi(s_n, a_{n+1})$, and further a_{n+1} is mixed into the internal state s_n via $s_{n+1} = \phi(s_n, a_{n+1})$. (Here, s_n and s_{n+1} denote the n^{th} and the $(n+1)^{\text{th}}$ internal state, respectively.) Assume that after the computation of $\psi(s_n, a_{n+1})$ or $\phi(s_n, a_{n+1})$, a total failure of the physical noise source has occurred. Since the fresh entropy was “directly” mixed into the internal random number r_{n+1} , the “remaining entropy” of the effective internal state can be assumed to be t bits. The evaluation rule from par. 415 allows the output of $\leq t$ internal random number bits.

Justification: Before the additional input a_{n+1} is mixed into the internal state, the entropy of the effective internal state space is $\approx t$.

[PTG.3.8, reseeding procedure] Example: As in par. 419, we assume that the effective internal state of the DRG.3-compliant cryptographic post-processing algorithm comprises t bits, and that it has essentially t bits of entropy. Unlike in par. 419, we assume that the intermediate random numbers are mixed into the internal state by permanent reseeding. After the intermediate random

number a_{n+1} has been mixed into the internal state by reseeding, one internal random number is generated via $r_{n+1} = \psi(s_{n+1})$. Assume that after the computation of $r_{n+1} = \psi(s_{n+1}, a_{n+1})$ or $s_{n+1} = \phi(s_n, a_{n+1})$ a total failure of the physical noise source has occurred. Here, the “remaining entropy” of the effective internal state can only be assumed to be $t - k$ bits where k denotes the output length of the internal random numbers. The reason is that the internal random number r_{n+1} has already been generated from the current internal state.

Example: Hash_DRBG with $Hash = \text{SHA-256}$; cf. par. 411 (ii)

Here $t = 440$ and $k = 256$. This means that at most $440 - 256 = 184$ internal random number bits can be output after a total failure has occurred. In other words, one further internal random number can be generated, but at most 184 bits thereof can be output. This requires that the total failure test detects a total failure before the next internal random number is output.

- 421 [PTG.3.8, reseeding procedure] As in par. 420, we assume that the intermediate random numbers are mixed into the internal state by reseeding, and per reseeding procedure one internal random number is generated and output. The bit size of the effective internal state of the DRG.3-compliant cryptographic post-processing algorithm is t , and the size of the internal random numbers is k . In this paragraph, we additionally assume that $t \geq 2k$. (Depending on the concrete design and the entropy claim, provided that the intermediate random numbers contain enough entropy, possibly more than one internal random number can be output per reseeding procedure; cf. the example below) We assume that the effective internal state received enough entropy when the PTRNG started (justifying the assumption that it essentially has maximum entropy t at this time), and that each intermediate random number has sufficient entropy such that the output internal random number satisfies the entropy claim. As pointed out in par. 420, the “remaining entropy” of the effective internal state can be assumed to be $t - k$ bits. This means that at least one internal random number can be output, relaxing the conditions on the detection and reaction time on a total failure.

Example: Hash_DRBG with $Hash = \text{SHA-384}$; cf. par. 411 (iii)

In par. 411 (iii), it was shown that even two internal random numbers can be output per reseeding procedure with entropy defect $\leq 2^{-32}$ per bit if the seed material contains at least 852 bits of min-entropy. If by design only one internal random number per reseeding procedure is output, it suffices that the seed material contains at least 450 bits of min-entropy. By the evaluation rule from par. 415, $888 - 384 = 504$ internal random number bits can be output after a total failure. This is one complete internal random number, and from the second internal random number at most $504 - 384 = 120$ bits can be output.

- 422 [PTG.3.9] For PTRNG designs where the intermediate random numbers are generated by a PTG.2-compliant PTRNG (the standard case, cf. par. 385), the statistical black box test suite T_{irn} was applied to the internal random numbers of the PTG.2-compliant PTRNG within its evaluation. Otherwise, if the developer has selected any other choice of Requirement PTG.3.3 than the first, the test suite T_{irn} is waived. The reason is that a cryptographic post-processing algorithm is applied to the intermediate random numbers.
- 423 [PTG.3.9] For the statistical properties of the internal random numbers, the justification can focus on the DRG.3-compliant cryptographic post-processing algorithm; cf. DRG.3.10 and par. 200.

3.5 NPTRNGs: Functionality Classes

Subsection 3.5.2 defines functionality class NTG.1. The differences between this document and its predecessor [AIS2031An_11] are explained in the appendix in Subsection 6.1.3. Subsection 3.5.1 contains explanations that are relevant for functionality class NTG.1. We begin with general remarks about NPTRNGs.

NPTRNGs generate “true” random bits, but unlike PTRNGs, they do not employ dedicated hardware designs or physical experiments as noise sources. Instead, NPTRNGs prevalently exploit non-physical noise sources such as system data or human interaction. From the point of view of the RNG, these non-physical noise sources may be viewed as “external” although they belong to the device or are exploited by the device on which the NPTRNG is implemented. The distribution of the output data from non-physical noise sources (i.e., raw random numbers) usually cannot be modeled as precisely as the raw random numbers generated by dedicated physical noise source designs of PTRNGs. Thus, their entropy shall be conservatively estimated.

NPTRNGs are used to generate “true” random numbers if PTRNGs with dedicated physical noise sources are not available. If one compares the evaluation of NPTRNGs with the evaluation of PTRNGs, certain fundamental differences become apparent. First of all, non-physical noise sources used by NPTRNGs often only work well under specific circumstances, and the NPTRNG often is unable to check whether these conditions are met. Secondly, the entropy estimate is usually based on complex assumptions about the knowledge and capabilities of an adversary and the operational environment (cf. pars. 429 and 432).

Note: For these reasons, the BSI generally has lower trust in NPTRNGs than in PTRNGs. As a consequence, functionality class DRG.4 prohibits the use of NPTRNGs for the seeding procedure, the reseeding procedures, and high-entropy additional input.

It should be noted that the definition of functionality class NTG.1 has been reworked in this version of the document. The definition of functionality class NTG.1 and the objectives are similar to that in [AIS2031An_11] (which justifies maintaining the class names), although it is different in detail.

3.5.1 NTG.1: Definitions, Requirements, and Justification

Typically, NPTRNGs are used if “true” random numbers are needed (e.g., for generating cryptographic keys or (re-)seeding a DRNG), but no PTRNG is available. Therefore, NPTRNGs use any available noise sources that are hard to predict. Since NPTRNGs are often computer programs, the output data of non-physical noise sources are usually system data and data produced by the interaction of human users or other external entities. A common approach is using time stamps from a high-resolution timer at “random” points in time.

For any TRNG evaluation, the central task is to verify that the entropy per internal random number bit exceeds a specified lower bound. For NPTRNGs, this means assessing how much entropy the collected raw random numbers contain relative to external observers. Unlike in the

case of **physical noise sources**, the unpredictability of the **raw random numbers** collected by a **NPTRNGs** from **non-physical noise sources** may, to a large degree, depend on the platform and the operational environment. Furthermore, the unpredictability depends on the means of an **adversary** to monitor or influence the **noise sources**. As a result of these problems and of further problems and potential security risks addressed below, the **entropy** estimation of an **NPTRNG** as a general rule cannot be based on a precise **stochastic model** but instead on conservative estimates assuming (realistic) worst-case conditions.

- 430 Analogous to **physical noise sources**, the output data of **non-physical noise sources** are called **raw random numbers**.
- 431 Examples of attack paths and risks.
1. A software-implemented **NPTRNG** is not secure against an **adversary** with full system access (in particular, read access to the **internal state**). An **adversary** with a lower access level can try to monitor the **noise sources** (e.g., perform coarse time measurements) or generate predictable **raw random numbers** (e.g., by running an unprivileged process on the same CPU core).
 2. An **adversary** may connect a malicious device that generates predictable events (e.g., keyboard and mouse events or network traffic).
 3. A software **NPTRNG** may be operated in an environment for which it was not intended, i.e., on a CPU where instructions behave differently, in a virtual machine, or in a scenario where no or only a subset of the **noise sources** are present.
- 432 In order to determine possible attack vectors as well as stating necessary operational conditions (e.g., the **adversary's** privileges), the **security boundary** of the **TOE** must be precisely specified. It should be noted that **NPTRNGs** usually have more and stronger operational security requirements than **PTRNGs** (cf. par. 429); cf. Section 5.6, par. 1365, for example (Linux /dev/random). Furthermore, the **attacker** should not have root access privileges.
- 433 The **raw random numbers** collected by an **NPTRNG** are often huge in data size compared to their estimated **entropy**. Since the **entropy** estimate is usually made with heuristic rules that assume a (realistic) worst-case scenario, the **raw random numbers** may contain more **entropy** in practice (e.g., if real-world **adversaries** are not as knowledgeable as assumed or if the **noise sources** are “more random” than assumed). Thus, **NPTRNGs** often compress and mix the collected **raw random numbers** into a large intermediate data structure, the **entropy** pool, in order to reduce the data size while still preserving the extra **entropy**. When the **NPTRNG** generates **internal random numbers**, data from the **entropy** pool is extracted and possibly compressed again such that the estimated **entropy** per **internal random number** almost equals the bit length.
- 434 An **NPTRNG** shall not generate more **internal random number** bits than the estimated overall **entropy** of the collected **raw random numbers**. **NPTRNGs** usually feature an **entropy** counter to keep track of how much **entropy** has entered the **entropy** pool and how much **entropy** has been extracted; the latter value corresponds to the number of **internal random number** bits that have been output. The counter is capped at the maximum amount of **entropy** that the mixing function can insert into the **entropy** pool. If a request for random bits exceeds the available amount of

entropy contained in the entropy pool, the NPTRNG shall block the output. The request can be served only after sufficient additional entropy has been gathered.

[generic design] There are many conceivable designs of NPTRNGs. This paragraph describes a generic design that uses typical components. The design is exemplary, and variations are possible. 435

- Non-physical noise sources are used to gather or generate raw random numbers from system data or interaction with external entities. The entropy of the raw random numbers is estimated.
- Periodically, or driven by events, the raw random numbers are mixed into the entropy pool. The entropy counter is increased accordingly (taking into account the maximal amount of entropy that the entropy pool can store).
- Upon a request for random bits, data from the entropy pool is extracted. If the entropy pool contains insufficient entropy, the request is refused or blocked (suspended). After data extraction, the entropy counter is decreased accordingly. To achieve enhanced backward secrecy, the previous value of the entropy pool is erased or cryptographically overwritten after each output (cf. par. 438).
- The function that extracts data from the entropy pool and generates output (internal random numbers) can be stateless (e.g., simply a hash function) or can have memory that persists between calls (e.g., a DRNG for cryptographic post-processing). The use of a DRNG does not waive the requirement of blocking in case of insufficient entropy.

[NTG.1] As an analogy to functionality class PTG.3, we call the data that are input to the cryptographic post-processing algorithm intermediate random numbers. 436

For stateful extraction, stateful output functions, or multiple entropy pools, the entropy must be counted consistently over all data structures to prevent generating pseudorandom output (cf. par. 434). This can be accomplished by having multiple entropy counters or fixed transfer sizes. Note that in order to achieve enhanced backward secrecy, the previous values of each data structure involved in generating output need to be erased or cryptographically overwritten (cf. par. 438) after providing entropy to the next stage. 437

Similar to pure PTRNGs without cryptographic post-processing algorithms, an NPTRNG can principally be stateless, i.e., collect a certain amount of entropy, generate output, and then completely erase its internal state in order to achieve enhanced backward secrecy. It is, however, recommended (and required for functionality class NTG.1) that an internal state is maintained in such a way that the NPTRNG exhibits the computational security properties of a (properly seeded) DRNG. This provides an additional security anchor in case the entropy of some raw random numbers is overestimated. 438

To be “secure” even if at some point in time, the raw random numbers of the non-physical noise sources do not contain enough entropy, a DRNG within the NPTRNG must be properly seeded. In particular, it must receive a sufficient amount of entropy before generating output. Otherwise, even if the raw random numbers would contain some entropy, the NPTRNG, when viewed as 439

a **hybrid DRNG**, would potentially be susceptible to the generic guessing attack described in par. 156.

- 440 A **DRNG** security anchor can be achieved, for example, by designing the **NPTRNG** to be a **DRNG** with the **internal state** being the **entropy** pool. Alternatively, the **NPTRNG** can consist of an **entropy** pool for collecting **entropy** and a dedicated **DRNG** for **cryptographic post-processing** that is continually reseeded from the **entropy** pool. In order to achieve **enhanced backward secrecy**, the **entropy** pool as well as the **internal state** of the **DRNG** need to be updated after having generated output (cf. par. 437). Using a dedicated **DRNG** for **cryptographic post-processing** can simplify the security evaluation of **DRNG** properties.
- 441 The concept of having a dedicated **DRNG** in par. 440 roughly corresponds to building a PTG.3 by combining a PTG.2 with DRG.3-compliant **cryptographic post-processing**.
- 442 For a long time, a prominent example of an NTG.1-compliant **NPTRNG** (under suitable operational conditions) has been the mechanism behind /dev/random; to be precise, until Linux kernel version 5.5; cf. [Linux_RNG_overview]. In later kernel versions, /dev/random delivers pseudorandom bits. Under suitable assumptions, for several kernel versions /dev/random is compliant with functionality class DRG.3, while later kernel versions show features of **DRNG trees**; see Section 5.6 for details. The document [RNG_virtual_env] considers the generation of random numbers in virtualized environments.

3.5.2 Functionality Class NTG.1

- 443 Functionality class NTG.1 defines requirements for **NPTRNGs** that rely on **information-theoretic security** (similar to **PTRNGs**) but use external input signals as **noise sources**. Additionally, a suitable **cryptographic post-processing** algorithm shall provide an additional security anchor.
- 444 NTG.1-compliant **NPTRNGs** are usually operated on devices like PCs, servers, etc. that do not have access to a **PTRNG**.
- 445 The **TSF** has to protect the **internal state** of the **RNG** from being compromised.
- 446 Note: Par. 447 does not replace an evaluation methodology.
- 447 **NTG.1-specific deliverables by the applicant**
The security architecture description and developer evidence shall contain
- a description of the required operational conditions and a specification of the **security boundary**,
 - a description of the **noise sources** and a justification for **entropy** estimates,
 - a comprehensive description of the “algorithmic behavior” of the **NPTRNG**,
 - evidence that NTG.1.1 through NTG.1.6 are fulfilled.

NTG.1: Security functional requirements

448

Security functional requirements of functionality class NTG.1 are defined by the component **FCS_RNG.1** with specific operations as given below.

FCS_RNG.1 Random number generation (Class NTG.1)

449

FCS_RNG.1.1 The **TSF** shall provide a **non-physical true RNG** that implements:

- (NTG.1.1) The **NPTRNG** shall collect and test the **raw random numbers** provided by the used **noise sources** in order to estimate the **entropy** and detect failures of the used **noise sources**
- (NTG.1.2) The **NPTRNG** shall have an **entropy** pool and an **entropy** counter that tracks the estimated amount of **entropy** currently stored in the **entropy** pool. The **NPTRNG** shall never generate more **internal random number** bits than indicated by the **entropy** counter.
- (NTG.1.3) The **NPTRNG** shall apply a **cryptographic post-processing algorithm with memory**. Viewed as a **hybrid DRNG**, the **NPTRNG** is compliant with functionality class DRG.3 (cf. pars. 438, 439, and 440). The **fresh entropy** can be input by the **reseeding procedure** or as **additional input**. The **NPTRNG** may apply both above-mentioned methods to input **fresh entropy**.
- (NTG.1.4) The **NPTRNG** shall not generate any **internal random numbers** until the following condition has been met. The **entropy** pool has collected at least 240 bits of **min-entropy** from at least two different **noise sources** each. These two **noise sources** shall employ different principles to provide randomness. If the **internal state** does not coincide with the **entropy** pool, the **DRNG** shall be (re-)seeded after this condition has been fulfilled before **internal random numbers** are output. Viewed as a **DRNG**, the **NPTRNG** has been seeded using 240 bits of **min-entropy** from each of the two **noise sources**.
- (NTG.1.5) The estimated **min-entropy** per **internal random number** bit shall be greater than or equal to $\lceil \text{assignment } v \in [0.98, 1 - 2^{-32}] \rceil$.

FCS_RNG.1.2 The **TSF** shall provide random numbers that meet:

- (NTG.1.6) The **internal random numbers** shall have **statistical inconspicuousness**.

Application notes

[noise sources] Usually, an NTG.1-compliant **NPTRNG** uses **non-physical noise sources**. If the **NPTRNG** has temporary access (e.g., when the **NPTRNG** is started; cf. Requirement NTG.1.4) or permanent access to a **PTRNG** that is compliant with functionality class PTG.2 or PTG.3, this **PTRNG** can be used as a substitute for a **non-physical noise source**. If the **NPTRNG** has temporary or permanent access to a **PTRNG** or a **physical noise source** that has not been certified,

450

this PTRNG/physical noise source has to be evaluated first. Assuming a positive evaluation, the PTRNG/physical noise source can be used as a substitute for a non-physical noise source.

Note: Usually, only the developer of a PTRNG/physical noise source can provide the evidence required for an evaluation.

- 451 [NTG.1.1] An NTG.1-compliant NPTRNG may utilize any source of data as noise source for which there is a compelling technical explanation of why the data are hard to predict by an adversary. The explanation shall specify the necessary operational requirements for the noise sources to function (e.g., the type of CPU, whether virtualization is allowed, or assumptions regarding the security features that protect against an adversary) and deliver a conservative lower bound for the expected amount of entropy.
Note: By Requirement NTG.1.5, an NTG.1-compliant NPTRNG needs (at least temporarily) access to at least two noise sources.
- 452 [NTG.1.1] The applicant's documentation shall specify all possible failure modes for the noise sources. The explanation shall comprise a heuristic analysis of the noise sources as a justification for the entropy estimated operation.
- 453 [NTG.1.1, overall evaluation] The explanation shall survey conceivable attack vectors on the noise sources (cf. par. 431) and assess (under realistic assumptions) the ability of an adversary to influence or observe the data and the impact on its entropy.
- 454 [NTG.1.1] The entropy estimator may assign a constant value as an entropy estimate to data from a noise source (unless a failure has been detected) or, alternatively, heuristically determine an entropy value. Sets of data that are estimated to contain zero entropy may also be added to the entropy pool unless this weakens the security of the NPTRNG (cf. par. 457).
- 455 [NTG.1.1, several noise sources] If the entropy pool receives raw data from several noise sources, it does not suffice to analyze the particular noise sources separately and to add their entropy contributions. Instead, the joint entropy has to be considered. In particular, the NPTRNG shall not overestimate the collected entropy by processing entropy contribution from the same event multiple times through different noise sources.
- 456 [NTG.1.1, non-evaluated noise sources] Non-evaluated noise sources (e.g., data from an NTP server) are permitted to provide input to the entropy pool if
(i) the non-evaluated noise sources are not credited with entropy.
(ii) the raw data are mixed into the entropy pool using a cryptographic function.
Note: Condition (ii) shall prevent that the entropy of the non-credited noise sources reduce the overall entropy of the raw data.
- 457 [NTG.1.1, overall evaluation] The evaluator shall consider the different noise sources and an adversary's ability to weaken the NPTRNG through the insertion of malicious raw random numbers. An adversary outside the security boundary shall not be able to weaken the security of the NPTRNG, provided that the operational requirements (e.g., necessary assumptions made for the evaluation that become part of the user guidance) are met.
- 458 [NTG.1.2] The entropy counter of the entropy pool shall start from zero. It shall be increased

by the estimated amount of **entropy** provided by the **raw random numbers** that are mixed into the **entropy** pool and decreased by the bit length of the output random numbers (**intermediate random numbers**) when the bits are extracted from the **entropy** pool. The incrementation/reduction of the **entropy** counter shall be consistent with the mixing function. The value of the **entropy** counter shall never exceed the maximal amount of **entropy** that the **entropy** pool can store. The maximal amount of storable **entropy** is determined by the data size of the **entropy** pool as well as the function that transfers **entropy** from **raw random numbers** into the **entropy** pool.

[NTG.1.2] If the **NPTRNG** has separate logical data structures in which **entropy** is stored (i.e., **entropy** pools), then the (local) **entropy** counter for each **entropy** pool and the (global) **entropy** counter for the sum of the **entropy** in all the **entropy** pools must be kept consistent (cf. par. 437). Note that NTG.1.5 (minimal amount of **entropy** of the **internal random numbers**) implies that extracting **entropy** from multiple data structures in order to produce output also needs to be kept consistent. 459

[NTG.1.3] The documentation to be provided by the developer for a DRG.3 evaluation comprises a formal description of how the **DRNG** updates its **internal state** and generates output. The same modeling is thus required for an NTG.1 evaluation; see par. 440. The **internal state** of the **cryptographic post-processing** algorithm can coincide with the **entropy** pool, but this need not be the case. 460

Note: In the first case, the **internal state** is updated by **raw random numbers** or (possibly stateless) post-processed **raw random numbers**.

[NTG.1.3, NTG.1.4] Requirements NTG.1.3 and NTG.1.4 shall ensure that the **NPTRNG** is at least as secure as a properly seeded DRG.3 (cf. par 438). To fulfill them, the corresponding requirements of DRG.3 shall be checked with the following modification. Because of the issues described in pars. 429 and 432, it is required to use two different **noise sources** that shall each provide a sufficient amount of data for the **seeding procedure**. In order to increase resilience, it may be advisable to collect even more data and from more **noise sources** before generating the first **internal random number**. 461

[NTG.1.4] This requirement for the **seeding procedure** of the **DRNG** security anchor also stipulates that the **NPTRNG** shall have at least two different **noise sources**. It also means that the **NPTRNG** cannot generate random bits until the slower of the two **noise sources** has produced the required amount of **entropy**. If the **NPTRNG** possesses more than two **noise sources**, Requirement NTG.1.4 is satisfied when the two fastest **noise sources** have each contributed enough **entropy**. The amount of **entropy** contributed by any other **noise sources** may be less than that provided by the two fastest **noise sources**. 462

[NTG.1.4] Seeding the **entropy** pool requires bits from at least two different **noise sources** and thus, the *second fastest* **noise source** determines the delay until the first **internal random number** can be generated. After this seeding step, it is not required to wait until the second fastest **noise source** has produced enough **entropy**. If the NTG.1 has a **noise source** that delivers a lot of **entropy** per time period, it may then continue to produce output with this (or even greater) bandwidth. This means that after the seeding step, only the total **entropy** is relevant, regardless of how many **noise sources** have contributed. 463

Note: In order to increase resilience and not to depend on a single **noise source**, however, it may

be advisable to prevent designs where the **NPTRNG** is dominated by a single **noise source** whose **entropy** rate greatly exceeds that of the other **noise sources**.

- 464 [NTG.1.4] **NPTRNGs** often exploit time points or time intervals of random events. Different types of events, e.g., events driven by user interaction, incoming network packages, or system events, can be interpreted as different **non-physical noise sources**, even if they apply the same sampling mechanism (time stamps of interrupts).
- 465 [NTG.1.4] The joint **entropy** of the raw data input to the **entropy** pool shall not be smaller than $2 \cdot 240$ bits of **min-entropy**.
- 466 [NTG.1.5] Requirement NTG.1.5 is the equivalent to Requirement PTG.3.5. Unlike for functionality class PTG.3, only **min-entropy** is allowed. While the **entropy** claim for functionality class PTG.3 is based on a **stochastic model**, the **entropy** claim for class NTG.1 is derived from heuristic **entropy** estimates. The entropy requirement per bit is meant to be understood as in par. 335, Note 1.
- 467 [NTG.1.5] As for the PTG.3 functionality class, the results from Subsection 4.4.2 can be utilized to analyze the impact of the **cryptographic post-processing** algorithm on the **entropy**. Par. 407 also applies here.
- 468 [NTG.1.6] Regarding the statistical properties of the **internal random numbers**, the justification can focus on the DRG.3-compliant **cryptographic post-processing**; cf. DRG.3.10 and par. 200.

3.6 Cross-class Topics

- 469 In Sections 3.3, 3.4, and 3.5, seven functionality classes for **DRNGs**, **PTRNGs**, and **NPTRNGs** have been defined. This section considers “cross-class” topics in which **RNGs** from different classes are involved.
- 470 [PTG.3 to DRG.3] After a **total failure**, a PTG.3-compliant **PTRNG** shall not output further **internal random numbers**. To be precise, its **internal state** allows (to some degree) a delayed reaction; cf. Requirement PTG.3.8 and several application notes. Unless the **total failure** has occurred immediately after the start of the **PTRNG** (i.e., before the **internal state** has been seeded with sufficient **entropy**), the **internal state** of the **cryptographic post-processing** algorithm may be assumed to have maximal **entropy**. In principle, after the **total failure** the **RNG** could restart the output of **internal random numbers**, but then the **RNG** would no longer be compliant with functionality class PTG.3. Instead, the **RNG** would drop down to functionality class DRG.3.
- 471 [PTG.3 to DRG.3] In specific scenarios (e.g., in the case of safety requirements), shutting down an **RNG** is not a valid option. From the perspective of a system design, it may be preferable to let a **PTRNG** compliant with functionality class PTG.3 continue to operate even if a failed **online test** or a failed **total failure test** indicates that the **noise source** has become inadequate. The fact that in such systems, **noise alarms** are basically ignored, does not, however, waive the respective requirements for functionality class PTG.3. In order to be compliant with functionality

class PTG.3, the **PTRNG** MUST be able to detect weaknesses of the random numbers and **total failures** AND signal them to the **consuming application** immediately. From the moment the **PTRNG** asserts that the **noise source** is not delivering the required **entropy** to satisfy the requirements of PTG.3, the **PTRNG** is no longer in conformance with functionality class PTG.3 but may continue to operate. Whether the device's response is suitable or not for the application is outside the scope of this document.

Note: "Downgrading" to functionality class DRG.3 has to be handled with the greatest care and should be considered only in very specific scenarios. In particular, the device must never "forget" that the **RNG** has been downgraded to a deterministic **DRNG**, even if the device has been turned off and restarted.

[combining several **RNGs**] It is possible to combine several individually evaluated **RNGs** if they are *logically and physically independent*. Logical independence means that there are no correlations by design; an extreme example of logical dependence is given, for example, by two instances of the same **DRNG** that were initialized identically. Usually, logical independence applies to **DRNGs**. Physical independence means that the **physical noise sources** (including the **digitization** mechanism) of the particular **RNGs** are not influenced by the other **RNGs**; this relates in particular to the other **physical noise sources**. Within the evaluation process, the developer has to give evidence that these assumptions are valid. In many cases, this task may be rather easy, in other cases very difficult, and in some cases there is indeed a mutual interference of the **physical noise sources** (e.g., between several ring oscillators). In the last case, of course, an "independence proof" cannot exist. If several **physical noise sources** exist and if it is not possible to give evidence that they are physically independent, *the physical noise sources shall be evaluated jointly*. In particular, it is necessary to develop and analyze a *joint stochastic model*. If **PTRNGs** and **NPTRNGs** are involved, there should usually be little interference between the **PTRNGs** and the **NPTRNGs**, but possibly within both groups.

Note 1: The verification of logical independence is usually rather easy, whereas the verification of physical independence can be much more difficult.

Note 2: This paragraph refers to the combination of several individually evaluated **RNGs**. The case where a single **TRNG** uses several **noise sources** has to be treated within its evaluation according to the functionality class PTG.2, PTG.3, or NTG.1.

[combining several **RNGs**] This paragraph provides several examples of how to combine **RNGs**. It is assumed that the **RNGs** are physically and logically independent. For simplicity, we further assume that the **internal random numbers** of the **RNGs** have been concatenated to form binary strings $y_{1(i)}, y_{2(i)}, \dots$. The index (i) refers to the **RNG** ($i = 1, 2$). The output sequence of the combined **RNG** is denoted by $z_1, z_2, \dots$.

- (a) **RNG** no. 1: PTG.3-compliant, **RNG** no. 2: **DRNG**:
 $z_j = y_{j(1)} + y_{j(2)} \pmod{2}$ for $j = 1, 2, \dots$ (corresponding bits are XORed).
 The combined **RNG** belongs to functionality class PTG.3.
- (b) **RNG** no. 1: PTG.2-compliant, **RNG** no. 2: DRG.3-compliant:
 $z_j = y_{j(1)} + y_{j(2)} \pmod{2}$ for $j = 1, 2, \dots$ (corresponding bits are XORed).
 The combined **RNG** belongs to functionality classes PTG.2 and DRG.3.
- (c) **RNG** no. 1: PTG.3-compliant, **RNG** no. 2: PTG.2-compliant:
 $z_j = y_{j(1)} + y_{j(2)} \pmod{2}$ for $j = 1, 2, \dots$ (corresponding bits are XORed).
 The combined **RNG** belongs to functionality class PTG.3.

- (d) **RNG** no. 1: PTG.3-compliant, **RNG** no. 2: NTG.1-compliant:
 $z_j = y_{j(1)} + y_{j(2)} \pmod{2}$ for $j = 1, 2, \dots$ (corresponding bits are XORed).
 The combined **RNG** belongs to the functionality classes PTG.3 and NTG.1.
- (e) **RNG** no. 1: DRG.3-compliant, **RNG** no. 2: DRG.2-compliant:
 $z_j = y_{j(1)} + y_{j(2)} \pmod{2}$ for $j = 1, 2, \dots$ (corresponding bits are XORed).
 The combined **RNG** belongs to functionality class DRG.3.

Note 1: Composition (b) is not compliant with functionality class PTG.3 because the **internal state** of the of the **DRNG** is not regularly updated with **fresh entropy**.

Note 2: The following considerations apply accordingly to more than two **RNGs**.

- 474 From a logical point of view, it might seem to be reasonable to extend the definition of functionality class PTG.3 by composition (b) in par. 473. However, with regard to the resistance against implementation attacks, Construction (b) has a disadvantage compared to the design demanded by functionality class PTG.3 because the **DRNG** never gets **fresh entropy**. This feature might make an attack on the **DRNG** easier: An **adversary** might try to mount a side-channel attack on the DRG.3-compliant **DRNG** first, in order to learn its **internal state** and to determine (and remove) its contribution to the XOR sum. In a second step, the **adversary** could, e.g., try to perform a fault injection attack on the remaining PTG.2-compliant **RNG**. For a PTG.3-compliant **PTRNG**, implementation attacks on the physical part and on the deterministic part cannot be carried out separately in this way.
- 475 [combining several **RNGs**] In the examples of par. 473, the bitwise XOR operation may be replaced by other group operations. For instance, one could divide the sequences $(y_{j(1)})_{j \in \mathbb{N}}$ and $(y_{j(2)})_{j \in \mathbb{N}}$ into non-overlapping k -bit blocks and apply a group operation to these blocks (e.g., the addition modulo 2^8 to 8-bit blocks).
- 476 [PTG.3, additional **cryptographic post-processing**] In this paragraph, we assume that **RNG** no. 1 is PTG.3-compliant, and that **RNG** no. 2 is DRG.3-compliant. Furthermore, the output sequence of **RNG** no. 1 is fed into the **DRNG** in compliance with Requirement PTG.3.4. We consider the combined **RNG** to be compliant with functionality class PTG.3 (without a specific **entropy** claim).
 Note 1: (i) If **RNG** no. 1 were PTG.2-compliant, this would directly follow from Requirement PTG.3.5.
 (ii) If the developer (the applicant for a certificate) aims for an **RNG-specific entropy** claim for the overall **PTRNG** (cf. Requirement PTG.3.5), this requires a specific **entropy** claim for **RNG** no. 1.
 Note 2: Unlike in Requirement PTG.3.3, here, **entropy** values can be assigned to the **intermediate random numbers** (the output of a PTG.3) that are larger than 0.9998 or 0.98.

4 Mathematical Background

Chapter 4 introduces, collects, and explains central mathematical concepts that are relevant and can be useful for the evaluation of RNGs according to AIS 20 and AIS 31. Sections 4.1 and 4.2 provide a large number of definitions and useful facts from probability theory and stochastics. In particular, random variables and stochastic processes are treated. Section 4.3 considers the concepts of entropy and work factor, while Section 4.4 deals intensively with random mappings. In Section 4.5 the concept of a stochastic model, the “core” of any PTRNG evaluation, is introduced, explained, and motivated. Furthermore, online tests and total failure tests are addressed. Finally, Section 4.6 contains general considerations to statistical tests and predictors, and a black box test suite for functionality class PTG.2 is specified. The concepts and their central ideas are illustrated by examples within the sections but also later in Chapter 5.

477

4.1 Randomness and Random Experiments

True randomness is a crucial requirement for any RNG. For non-deterministic (true) random number generators (TRNGs), loosely speaking, the noise source “generates” randomness. For deterministic random number generators (DRNGs), the randomness is extracted from the seed material.

478

Probability theory describes, analyzes, and quantifies randomness by means of abstract mathematical objects, in particular by random variables and stochastic processes (cf. Section 4.2). The core of any PTRNG evaluation is the stochastic model (Section 4.5).

479

Statistics links abstract mathematical models with real-world RNGs through experiments. These experiments may be used to estimate parameters that describe the model or to test hypotheses deduced from this model.

480

A statistical test checks whether the output sequence of an experiment is “typical” in a specified sense. Any finite collection of statistical tests can only check finitely many criteria for “regularity”. Hence, it is important to understand the nature of the noise source to rate the randomness of random number generation.

481

An experiment is called *unpredictable* if the observable outcome of the experiment is (to a certain extent) unknown before it is conducted. In this document, we denote the outcome of an experiment as *random* if it is unpredictable, i.e., if it cannot be predicted with certainty. Note: Deterministic behavior can be viewed as a special case of randomness that is described by a one-point distribution.

482

After the experiment has been performed, the degree of uncertainty depends on the observer’s ability to predict or even observe the outcome. Entropy (cf. Section 4.3) quantifies the degree of unpredictability relative to an observer.

483

Experiments are called *independent* if the outcomes of previous experiments do not influence the outcome of the current experiment.

484

485 Real-world **RNGs** cannot generate ideal randomness; they can, at most, approximately achieve this goal. Roughly speaking, the key point of any **TRNG** evaluation is to verify that the **TRNG** is “sufficiently close” to an **ideal RNG**.

4.2 Probability, Stochastics, and Random Variables

486 In Subsection 4.2.1, we introduce definitions and concepts from probability theory and stochastics that allow the qualitative statements from the previous section to be precise in a mathematical sense. Furthermore, Subsection 4.2.2 collects useful facts that are needed in this document or can be used in evaluations of real-world **RNGs**. In the context of this document, an important field of application is **stochastic models** of **physical noise sources** (cf. Section 4.5).

487 Below, definitions and well-known facts are given without references, while for less well-known facts, references are added. Furthermore, several facts and results are proven if no reference is known. At least for some of them, it is likely that they are already known in the literature.

4.2.1 Definitions and Basic Concepts

488 In the following, Ω denotes a non-empty set.

489 In this document Ω , usually represents the admissible values of random numbers, random experiments, or measurements. Usually, Ω is finite (typically, $\Omega = \{0, 1\}^k$ or $\Omega = \mathbb{Z}_n$) or it equals $\mathbb{R}^m$ or a subset of $\mathbb{R}^m$ ($m \geq 1$).

Note 1: Experiments with finite Ω are, for example, coin tosses and dice rolls. In the context of **RNGs**, random numbers are important examples that assume values in a finite or in a countable set Ω , e.g., $\Omega = \{0, 1\}$ and $\Omega = \mathbb{N}_0$.

Note 2: Examples for $\Omega \subseteq \mathbb{R}^m$ are timing measurements and voltage measurements.

490 $\mathcal{P}(\Omega)$ denotes the power set of Ω . The power set contains all subsets of Ω . If Ω is finite then $|\mathcal{P}(\Omega)| = 2^{|\Omega|}$.

491 Paragraphs 492 to 501 contain basic definitions and facts from probability and measure theory, which will be needed below for proper definitions of independence or **stationary** stochastic processes, for example. However, these concepts are rather “technical”. Paragraphs 503 to 506 provide a “light version” thereof, which should suffice to understand the subsequent definitions and concepts.

492 A σ -algebra $\mathcal{A}$ over Ω is a set of subsets of Ω , i.e., $\mathcal{A} \subseteq \mathcal{P}(\Omega)$, that fulfills the following conditions:

(a) $\Omega \in \mathcal{A}$

(b) If $A \in \mathcal{A}$, then also its complement $A^c := \Omega \setminus A \in \mathcal{A}$

(c) If $A_1, A_2, \dots \in \mathcal{A}$ then $\bigcup_{n \geq 1} A_n \in \mathcal{A}$

Note: Par. 492, Condition (c), includes finite sequences $A_1, A_2, \dots, A_k$. Note that such a finite sequence can formally be extended by $A_{k+1} = A_{k+2} = \dots = \{\}$ to an infinite sequence with the same union set. 493

Example: (i) $\mathcal{P}(\Omega)$ is a σ -algebra over Ω . 494

(ii) The Borel σ -algebra $\mathcal{B}(\mathbb{R})$ over $\mathbb{R}$ is the smallest σ -algebra that contains the open intervals (equivalently, that contains the open subsets of $\mathbb{R}$).

(iii) More generally, for $m \geq 1$, the Borel σ -algebra $\mathcal{B}(\mathbb{R}^m)$ over $\mathbb{R}^m$ is the smallest σ -algebra that contains the open subsets of $\mathbb{R}^m$.

A probability measure ν on a σ -algebra $\mathcal{A}$ is a mapping $\nu: \mathcal{A} \rightarrow [0, 1]$ with the following properties 495

(a) $\nu(\Omega) = 1$

(b) If the sets $A_1, A_2, \dots \in \mathcal{A}$ are mutually disjoint, $\nu\left(\bigcup_{n \geq 1} A_n\right) = \sum_{n \geq 1} \nu(A_n)$.
(The sequence $A_1, A_2, \dots$ may be finite or countable.)

More generally, if a mapping $\nu: \mathcal{A} \rightarrow [0, \infty]$ fulfills Condition (b) from par. 495, and if $\nu(\Omega) < \infty$, we refer to ν as a finite measure; otherwise, ν is an infinite measure. If there exists a countable sequence $C_1 \subseteq C_2 \subseteq C_3 \dots \in \mathcal{A}$ such that $\nu(C_n) < \infty$ for all $n \in \mathbb{N}$ and $\bigcup_{n \geq 1} C_n = \Omega$, then ν is a σ -finite measure. 496

Any $A \in \mathcal{A}$ is said to be an event or a measurable set. A pair $(\Omega, \mathcal{A})$ is denoted as a measurable space, while the triple $(\Omega, \mathcal{A}, \nu)$ is called a measure space. If ν is a probability measure, the triple $(\Omega, \mathcal{A}, \nu)$ is a probability space. 497

Example: (i) Let $B(n, p)$ denote a binomial distribution with parameters n and p . Then $B(n, p)$ is a probability measure on $\mathcal{P}(\{0, \dots, n\})$. 498

(ii) The Lebesgue measure λ is a σ -finite measure on $\mathcal{B}(\mathbb{R})$.

Note: The Lebesgue measure corresponds to the “geometric” measure on $\mathbb{R}$, i.e., $\lambda([a, b)) = b - a$ if $a \leq b$.

(iii) The standard normal distribution (standard Gaussian distribution) $N(0, 1)$ is a probability measure on $\mathcal{B}(\mathbb{R})$.

(iv) The Lebesgue measure λ_m on $\mathbb{R}^m$ is a σ -finite measure.

If there is no ambiguity about the σ -algebra $\mathcal{A}$, we often loosely speak of “measures on Ω ”. Unless otherwise stated, in this document $\mathcal{A} = \mathcal{P}(\Omega)$ for countable Ω (finite or infinite), and for $\mathbb{R}$, $\mathbb{R}^m$, and measurable subsets $\Omega \subseteq \mathbb{R}^m$, we use the Borel σ -algebras $\mathcal{A} = \mathcal{B}(\mathbb{R})$, $\mathcal{A} = \mathcal{B}(\mathbb{R}^m)$, or $\mathcal{A} = \mathcal{B}(\Omega)$, respectively. 499

Assume that $(\Omega_1, \mathcal{A}_1, \nu)$ is a probability space and that $(\Omega_2, \mathcal{A}_2)$ is a measurable space. Furthermore, let $\phi: \Omega_1 \rightarrow \Omega_2$ be a mapping. We call ϕ measurable (or more precisely, $(\mathcal{A}_1, \mathcal{A}_2)$ -measurable) if for each $A' \in \mathcal{A}_2$ the pre-image $\phi^{-1}(A') \in \mathcal{A}_1$. If ν is a measure on $\mathcal{A}_1$, then 500

$\nu^\phi(A') := \nu(\phi^{-1}(A'))$ for all $A' \in \mathcal{A}_2$ defines a measure on $\mathcal{A}_2$. We denote ν^ϕ the image measure (or: transformed measure) of ν under ϕ .

- 501 Assume that $\mathcal{A}_1$ and $\mathcal{A}_2$ are σ -algebras over Ω_1 and Ω_2 . A **random variable** X is a measurable mapping $X: \Omega_1 \rightarrow \Omega_2$. In our context, usually Ω_2 is finite, countable, or a subset of $\mathbb{R}^m$.
- 502 Outside of mathematical proofs, the probability space of a **random variable** is usually not explicitly stated. We point out that a **random variable** $X: \Omega_1 \rightarrow \Omega_2$ with probability space $(\Omega_1, \mathcal{A}_1, \nu)$ can also be interpreted as a **random variable** on the measure space $(\Omega_2, \mathcal{A}_2, \nu^X)$. Here, ν^X denotes the image measure (or: transformed measure) of X , i.e., $\nu^X(A_2) = \nu(X^{-1}(A_2))$ for all $A_2 \in \mathcal{A}_2$. Furthermore, $\text{Prob}(X \in A_1) = \nu(A_1)$ quantifies the probability that the **random variable** X assumes a value in A_1 .
- 503 [“light version” of pars. 492 to 501] As already mentioned above, these definitions and concepts are needed for mathematically precise definitions in the following. Fortunately, in the context of **RNG** evaluations, problems concerning measurability hardly occur. The paragraphs 504 to 506 thus provide a “light version”. This light version should suffice for at least an intuitive understanding of the following definitions and concepts and to apply them correctly. This, in particular, refers to the material collected in Subsection 4.2.2.
- 504 [“light version” of pars. 492 to 501 ctd.] Some of the following definitions and conditions refer to “measurable subsets” of some space Ω (equivalently, to elements of a σ -algebra on Ω). If Ω is finite or countable, all subsets of Ω are measurable (to be precise: with regard to the σ -algebra $\mathcal{P}(\Omega)$). If $\Omega \subseteq \mathbb{R}^m$ one may think of “regular” subsets as (depending on the dimension m) intervals, rectangles, circles, cuboids, balls, etc. and countable unions thereof. (There exist further measurable and non-measurable subsets, but this should be of little importance for **RNG** evaluations.)
- 505 [“light version” of pars. 492 to 501 ctd.] In this document and, more generally, in the context of the evaluation of **RNGs**, **random variables** usually assume values in finite or countable sets or in subsets of $\mathbb{R}$ or $\mathbb{R}^m$. We may speak of **random variables** on finite or countable set Ω (e.g., $\Omega = \{0, 1\}$), or **random variables** on $\mathbb{R}$ (also: “real-valued **random variables**”), **random variables** on $\mathbb{R}^m$, or **random variables** on Ω .
- 506 [“light version” of pars. 492 to 501 ctd.] The expression $\text{Prob}(X \in A)$ quantifies the probability that the **random variable** X assumes a value in the set $A \subseteq \Omega$.
- 507 $X \sim \nu$ means that the **random variable** X has distribution ν , i.e., that $\text{Prob}(X \in A) = \nu(A)$. The term $\text{Prob}(X \in A)$ quantifies the probability that X assumes a value in the set A . Values that are assumed (or: taken on) by a **random variable** X are called *realizations* of X .
- 508 [Notation] In this document, we denote **random variables** by capital letters and their realizations usually by the corresponding small letters.
- 509 Example: Assume that the **random variable** X models the tossing of a fair coin. Then $\text{Prob}(X = 0) = \text{Prob}(X = 1) = 0.5$ if we identify “head” and “tail” with 1 and 0. These probabilities quantify the knowledge on the outcome of a future coin toss (and on a past experiment to a

person who does not know its outcome). Possible realizations of X are 0 and 1.

In this document, we model non-deterministic phenomena by **random variables**. Their realizations are observable as random numbers, voltage, or timing, for example. 510

Definition: The term $B(n, p)$ denotes the binomial distribution with parameters n and p , which is given by 511

$$\text{Prob}(X = k) = \binom{n}{k} p^k (1 - p)^{n-k} \quad \text{for } k = 0, \dots, n. \quad (4.1)$$

Definition: The Poisson distribution with parameter $\tau > 0$ is given by 512

$$\text{Prob}(X = k) = \frac{\tau^k}{k!} e^{-\tau} \quad \text{for } k \in \mathbb{N}_0. \quad (4.2)$$

Note: The parameter $\tau > 0$ equals the mean number of events per time interval of length 1.

Definition: The geometric distribution $\mathcal{G}_p$ with parameter $p \in (0, 1]$ is a discrete distribution on $\mathbb{N}$. More precisely, 513

$$\mathcal{G}_p(k) := p(1 - p)^{k-1} \quad \text{for } k \in \mathbb{N}. \quad (4.3)$$

The term $\mathcal{G}_p(k)$ equals the probability that a sequence of **iid** Bernoulli trials with individual success probability p is successful for the first time in the k^{th} trial.

Note: In the literature, there also exists an alternative definition of the geometric distribution that only counts the number of failures, i.e., $k - 1$ in place of k .

Definition: The letters λ and λ_m denote the Lebesgue measures on $\mathbb{R}$ or $\mathbb{R}^m$, respectively. It is $\lambda([a, b)) = b - a$ if $a \leq b$. Accordingly, $\lambda_m\left(\prod_{j=1}^m [a_j, b_j)\right) = \prod_{j=1}^m (b_j - a_j)$ if $a_j \leq b_j$ for $1 \leq j \leq m$. 514

Note: The Lebesgue measure λ corresponds to the “geometric” measure on $\mathbb{R}$ (not to be mixed up with the geometric distribution defined in par. 513).

Definition: The term $N(\mu, \sigma^2)$ denotes the normal (Gaussian) distribution with expectation μ and variance σ^2 . It has the density 515

$$\phi(x) := \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}. \quad (4.4)$$

In particular, $N(0, 1)$ is called *standard normal distribution*. Its cumulative distribution function $\Phi(\cdot)$ is given by

$$\Phi(x) := \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{t^2}{2}} dt. \quad (4.5)$$

Note 1: To be precise, “density” means “Lebesgue density”. In this document, densities with respect to other measures than the (one-dimensional or multidimensional) Lebesgue measure are not relevant. For this reason, we briefly speak of “density” in place of “Lebesgue density” in the following.

Note 2: Normal distributions exist in $\mathbb{R}^k$ for each $k \geq 1$ (multivariate normal distributions); cf. par. 542.

Definition: The Gamma distribution with the shape parameter $\alpha > 0$ and rate parameter $\beta > 0$ has the density

$$\gamma_{\alpha,\beta}(x) := \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x} \quad \text{for } x > 0. \quad (4.6)$$

Note: Occasionally, the Gamma distribution is not characterized by a shape parameter and a rate parameter but by a shape parameter and a scale parameter. Thus, caution is advised when results from different books and papers are applied. The scale parameter is the reciprocal value of the shape parameter.

517 The **random variable** X is called discrete if Ω is countable (finite or infinite). If Ω is finite, we also call X a finite **random variable**. Examples are binomially distributed **random variables** and Poisson-distributed **random variables**. Section 4.4 treats random mappings. There, the realizations of the **random variables** are mappings between sets.

518 Let X be a **random variable** that assumes values in a finite set Ω . We say that X is **uniformly distributed** (or equivalently: **unbiased**, equidistributed) if it assumes all $\omega \in \Omega$ with the same probability, namely $\text{Prob}(X = \omega) = |\Omega|^{-1}$. Otherwise, X is said to be **biased**.
Note: Precisely formulated, it should actually read $\text{Prob}(X = \{\omega\})$ instad of $\text{Prob}(X = \omega)$. However, the shorter expression “ $\text{Prob}(X = \omega)$ ” is common for finite and countable Ω .

519 A **random variable** X has density $f: \Omega \rightarrow [0, \infty]$ with respect to a measure τ if $\text{Prob}(X \in A) = \int_A f(\omega) d\tau(\omega)$ for all measurable sets A . Equivalently, a measure ν has density $f: \Omega \rightarrow [0, \infty]$ with respect to a measure τ if $\nu(A) = \int_A f(\omega) d\tau(\omega)$ for all measurable A .
Note: Densities do not exist for each pair of measures (ν, τ) .

520 In our context, usually $\Omega \subseteq \mathbb{R}^m$ with $m \geq 1$, and $\tau = \lambda_m$. Then

$$\text{Prob}(X \in A) = \int_A f(x) \lambda_m(dx) = \int_A f(x) dx. \quad (4.7)$$

521 Let X denote a **random variable** that assumes values in $\mathbb{R}^m$ and has distribution ν . If the integral

$$E(X) := \int_{\Omega} x \nu(dx) \quad (4.8)$$

exists (i.e., if $\int_{\Omega} |x| \nu(dx) < \infty$), then $E(X)$ is called the expectation of X .

Note: The expectation $E(X)$ does not exist for every **random variable**. Counterexamples are, for example, Cauchy-distributed **random variables**.

522 For discrete **random variables** X with values in $\Omega \subseteq \mathbb{R}$ (e.g., $\Omega = \{0, 1\}$, $\mathbb{N}$, or $\mathbb{Z}$), formula (4.8) simplifies to

$$E(X) := \sum_{x \in \Omega} x \text{Prob}(X = x). \quad (4.9)$$

If X assumes values in $\mathbb{R}^m$ and has Lebesgue density f then (4.8) reads

$$E(X) := \int_{\mathbb{R}^m} x f(x) dx. \quad (4.10)$$

In the context of **PTRNG** evaluations, we are usually faced with these two special cases.

Note: For **random variables** with values in $\{0,1\}^n$, no meaningful definition for the mean is evident. 523

The *variance* of a real-valued **random variable** X is defined by 524

$$\text{Var}(X) := E(E(X) - X)^2. \quad (4.11)$$

provided that both expectations exist. This is not always the case.

Assume that $\text{Var}(X)$ exists. Then 525

$$\sigma_X := \sqrt{\text{Var}(X)} \quad (4.12)$$

is the standard deviation of X .

[sum of normal distributions] If X_1 and X_2 denote independent normally distributed **random variables** with expectations μ_1, μ_2 and variances σ_1^2, σ_2^2 , then $X_1 + X_2$ is normally distributed with expectation $\mu_1 + \mu_2$ and variance $\sigma_1^2 + \sigma_2^2$. More generally, if the **random variables** $X_1, \dots, X_n$ are **iid** $N(\mu, \sigma^2)$ -distributed then the sum $X_1 + \dots + X_n$ is $N(n\mu, n\sigma^2)$ -distributed. 526

[Gamma distribution] The Gamma distribution with the shape parameter $\alpha > 0$ and rate parameter $\beta > 0$ has the density $\gamma_{\alpha,\beta}(\cdot)$, cf. par. 516. A **random variable** that is Gamma distributed with parameters α and β has mean $\mu = \alpha/\beta$ and variance $\sigma^2 = \alpha/\beta^2$. 527

[sum of Gamma distributions] If X and Y are independent **random variables** with densities $\gamma_{\alpha_1,\beta}(\cdot)$ and $\gamma_{\alpha_2,\beta}(\cdot)$, respectively, then $X + Y$ is Gamma-distributed with density $\gamma_{\alpha_1+\alpha_2,\beta}(\cdot)$. Consequently, if the **random variables** $X_1, \dots, X_n$ are **iid** Gamma distributed with parameters α and β then the sum $X_1 + \dots + X_n$ is Gamma distributed with parameters $n\alpha$ and β . 528

The **random variables** $X_1, X_2, \dots, X_k$ are said to be *independent* if for *each* k -tuple $(A_1, \dots, A_k)$ of measurable sets the equality 529

$$\text{Prob}(X_1 \in A_1, \dots, X_k \in A_k) = \prod_{j=1}^k \text{Prob}(X_j \in A_j). \quad (4.13)$$

holds.

More generally, the (infinite) sequence $X_1, X_2, \dots$ of **random variables** is said to be *independent* if for *each* integer $k' \geq 1$ and for *each* k' -tuple $(A_1, \dots, A_{k'})$ of measurable sets, condition (4.13) is valid (with k' in place of k). 530

Note: Independence can be generalized to uncountable index sets.

For discrete **random variables** $X_1, X_2, \dots$ with values in Ω , condition (4.13) simplifies to 531

$$\text{Prob}(X_1 = x_1, \dots, X_k = x_k) = \prod_{j=1}^k \text{Prob}(X_j = x_j) \quad (4.14)$$

for each k -tuple $(x_1, \dots, x_k) \in \Omega^k$.

- 532 In the context of **random variables** $X_1, X_2, \dots$, the abbreviation **iid** stands for “independent and identically distributed”.
- 533 Mathematically, a sequence of **iid uniformly distributed random variables** $X_1, X_2, \dots$ on a finite set Ω (e.g., $\Omega = \{0, 1\}$) describes an ideal **RNG**.
- 534 Assume that the **random variables** $X_1, X_2, \dots, X_n$, resp. $X_1, X_2, \dots$ are independent. If $X_j \sim \nu_j$, the joint distribution of $(X_1, X_2, \dots, X_n)$, resp., of the sequence $X_1, X_2, \dots$ is given by the product measure $\otimes_{j=1}^n \nu_j$, resp. by $\otimes_{j=1}^\infty \nu_j$. These product measures are characterized by the conditions from pars. 529 and 530. If the **random variables** $X_1, X_2, \dots$ are identically distributed, i.e., if $\nu_1 = \nu_2 = \dots = \nu_n$, we alternatively also use the notation ν^n and $\nu^\mathbb{N}$.
- 535 Assume that for the real-valued **random variables** X and Y , expectations and variances exist. Then the right-hand sides of (4.15) and (4.16) exist

$$\text{Cov}(X, Y) := E(XY) - E(X)E(Y) \quad (\text{covariance}) \quad (4.15)$$

$$\text{corr}(X, Y) := \frac{\text{Cov}(X, Y)}{\sigma_X \cdot \sigma_Y} = \frac{E(XY) - E(X)E(Y)}{\sqrt{\text{Var}(X)}\sqrt{\text{Var}(Y)}} \quad (\text{correlation coefficient}) \quad (4.16)$$

If $\text{Cov}(X, Y) = 0$, we say that X and Y are *uncorrelated*.

- 536 Independence implies uncorrelatedness but, in general, the converse is not true (cf. pars. 537 and 543).
- 537 Counterexample ([Geor15], Beispiel (4.26)): Assume that X and Y are **random variables** that assume values in $\Omega_1 = \{-1, 0, 1\}$ and in $\Omega_2 = \{0, 1\}$, respectively. Assume further that $\text{Prob}(X = 1, Y = 0) = \text{Prob}(X = 0, Y = 1) = \text{Prob}(X = -1, Y = 0) = 1/3$. Hence, $\text{Prob}(X = 0) = \text{Prob}(X = 1) = \text{Prob}(X = -1) = 1/3$ and thus $E(X) = 0$. Similarly, $\text{Prob}(Y = 0) = 2/3$, $\text{Prob}(Y = 1) = 1/3$ and thus $E(Y) = 1/3$. Finally,

$$\begin{aligned} \text{Cov}(X, Y) &= E(XY) - 0 \cdot \frac{1}{3} = \sum_{x \in \Omega_1, y \in \Omega_2} xy \text{Prob}(X = x, Y = y) - 0 = \\ &= \left(1 \cdot 0 \cdot \frac{1}{3} + 0 \cdot 1 \cdot \frac{1}{3} - 1 \cdot 0 \cdot \frac{1}{3}\right) = 0. \end{aligned}$$

Thus, X and Y are uncorrelated, but since $\text{Prob}(X = 1, Y = 1) = 0 \neq 1/9 = \text{Prob}(X = 1) \cdot \text{Prob}(Y = 1)$, the **random variables** X and Y are not independent.

- 538 [Strong law of large numbers] Assume that the **random variables** $X_1, X_2, \dots$ have expectations $E(X_1), E(X_2), \dots$. We say that the sequence $X_1, X_2, \dots$ satisfies the *strong law of large numbers* if

$$\text{Prob} \left(\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N (X_j - E(X_j)) = 0 \right) = 1. \quad (4.17)$$

If the **random variables** $X_1, X_2, \dots$ are identically distributed with expectation $E(X_j) = \mu$, (4.17)

simplifies to

$$\text{Prob} \left(\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N X_j = \mu \right) = 1. \quad (4.18)$$

[Strong law of large numbers] If the **random variables** $X_1, X_2, \dots$ fulfill one of the following conditions, they satisfy the strong law of large numbers. 539

(i) The **random variables** $X_1, X_2, \dots$ are **iid** with expectation $E(X_j) = \mu$.

(ii) The **random variables** $X_1, X_2, \dots$ are identically distributed with expectation $E(X_j) = \mu$ and mutually independent.

Note 1: Condition (ii) generalizes condition (i).

Note 2: Under suitable conditions, the strong law of large numbers also applies to dependent random variables.

[Convolution of probability measures] Let X and Y denote independent real-valued **random variables** that are η -distributed and τ -distributed, respectively. The distribution of the sum $X + Y$ is the convolution product of η and τ , denoted by $\eta * \tau$. If X has density $g(\cdot)$ and Y has density $h(\cdot)$, then $X + Y$ has density 540

$$f(x) := \int_{-\infty}^{\infty} g(x-y)h(y) dy. \quad (4.19)$$

Note 1: The convolution of special distributions is discussed in pars. 1202 to 1206.

Note 2: The term τ^{*k} denotes the k -fold convolution product $\tau * \dots * \tau$.

[Convolution of probability measures on groups] The convolution of probability measures is not only defined on $\mathbb{R}$ but also on groups. In our context, finite groups are relevant. Assume that the independent **random variables** X and Y take on values on a finite (not necessarily commutative) group G with group operation “ $\circ$ ”, and that X and Y are τ -distributed and η -distributed, respectively. In particular, 541

$$\tau * \eta(g) = \sum_{h \in G} \eta(g \circ h^{-1})\tau(h) \quad \text{for each } g \in G. \quad (4.20)$$

Example:

a) $(G, \circ) = (\mathbb{Z}_n, +(\text{mod } n))$.

b) $(G, \circ) = (\{0, 1\}^t, \oplus)$, where “ $\oplus$ ” denotes the bitwise addition mod 2.

c) $(G, \circ) = \text{GL}(n, \text{GF}(2))$, the group of invertible $(n \times n)$ -matrices over $\text{GF}(2)$. Here, the group operation is the matrix multiplication. The group $\text{GL}(n, \text{GF}(2))$ is not commutative for $n \geq 2$.

Note 1: In an additive group, $X \circ Y$ and $g \circ h^{-1}$ read $X + Y$ and $g - h$, respectively.

Note 2: Pars. 596 to 603 provide useful theorems.

Definition: Let C is a positive definite $(k \times k)$ -matrix. The term $N(\vec{\mu}, C)$ denotes the k -dimensional normal (Gaussian) distribution with expectation $\vec{\mu}$ and covariance matrix C . It has the k -dimensional density 542

$$f_C(x) := \frac{1}{\sqrt{(2\pi)^k \det C}} e^{-0.5(\vec{x} - \vec{\mu})^T C^{-1}(\vec{x} - \vec{\mu})}. \quad (4.21)$$

Note 1: If $\vec{X} = (X_1, \dots, X_k)$ is $N(\vec{\mu}, C)$ -distributed, its expectation $E(\vec{X}) = \vec{\mu} \in \mathbb{R}^k$, and $C = (c_{ij})_{1 \leq i, j \leq k} = (\text{Cov}(X_i, X_j))_{1 \leq i, j \leq k}$ is the covariance matrix of $\vec{X}$.

Note 2: Definition (4.21) generalizes (4.4). In the one-dimensional case, $\vec{\mu} = \mu$, and the covariance matrix C equals the variance σ^2 .

Note 3: If the **random variables** $X_1, \dots, X_k$ are **iid** $N(0, 1)$ -distributed, the covariance matrix is given by the k -dimensional identity mapping.

Note 4: Above, we assume that the covariance matrix C is regular. It should be noted that normal distributions with singular covariance matrices C also exist. These distributions do not have k -dimensional densities.

543 Assume that the **random variables** X and Y are *bivariate normally distributed*. If X and Y are uncorrelated, X and Y are independent.

Note: In general, this conclusion is not true; cf. pars. 536 and 537.

544 Let $(\Omega, \mathcal{A}, P)$ be a probability space. Formally, a *stochastic process* $(X_t)_{t \in T}$ with state space Ω is a collection of real-valued **random variables** $\{X_t \mid t \in T\}$, where the index t is usually interpreted as “time”.

545 If $T \subseteq \mathbb{R}$ is an interval (e.g., $T = (a, b)$, $T = [0, \infty)$ or $T = \mathbb{R}$), we speak of (time-)continuous stochastic processes. If $T \subseteq \Delta\mathbb{Z}$ for some $\Delta > 0$, e.g., $T = \mathbb{Z}$, $T = \mathbb{N}$ or $T = \mathbb{N}_0$, the stochastic process is called (time-)discrete.

546 Example: Markov chains (time-discrete stochastic process; cf. par. 578), Gaussian process (time-continuous stochastic process; cf. par. 547), Wiener process (time-continuous stochastic process).

547 A time-continuous real-valued stochastic process $(X_t)_{t \in T}$ is called a *Gaussian process* if the random vector $(X_{t_1}, \dots, X_{t_k})$ is (multivariate) normally (Gaussian) distributed for each finite index set $\{t_1, \dots, t_k\} \subseteq T$; cf. par. 542.

548 A stochastic process $(X_t)_{t \in T}$ is called *stationary* (or: **stationary** in a strict sense) if

$$\begin{aligned} \text{Prob}(X_{t_1} \in A_1, X_{t_2} \in A_2, \dots, X_{t_k} \in A_k) &= P(X_{t_1+\tau} \in A_1, X_{t_2+\tau} \in A_2, \dots, X_{t_k+\tau} \in A_k) \quad (4.22) \\ &\text{for each } k \in \mathbb{N}, \tau > 0, \text{ all } t_1 < \dots < t_k \text{ with } t_j, t_j + \tau \in T \ (j \leq k), \\ &\text{and all measurable sets } A_1, \dots, A_k. \end{aligned}$$

If the **random variables** X_j are discrete, (4.22) simplifies to

$$\begin{aligned} \text{Prob}(X_{t_1} = x_1, X_{t_2} = x_2, \dots, X_{t_k} = x_k) &= \text{Prob}(X_{t_1+\tau} = x_1, X_{t_2+\tau} = x_2, \dots, X_{t_k+\tau} = x_k) \quad (4.23) \\ &\text{for each } k \in \mathbb{N}, \tau > 0, \text{ all } t_1 < \dots < t_k \text{ with } t_j, t_j + \tau \in T \ (j \leq k), \text{ and all } x_1, \dots, x_k \in \Omega. \end{aligned}$$

549 Stationarity means that the distribution of the stochastic process is time-invariant. In other words: For admissible shifts τ (that is, $T + \tau \subseteq T$), the stochastic processes $(X_t)_{t \in T}$ and $(X_{t+\tau})_{t \in T}$ are identically distributed. If $T = \mathbb{R}$ or $T = [0, \infty)$, for example, any $\tau > 0$ is admissible. For $T = \mathbb{Z}$ or $T = \mathbb{N}$ (time-discrete stochastic processes), the shift parameter τ must be a positive integer.

550

A stochastic process $(X_t)_{t \in T}$ is *stationary in a weak sense* (or: *stationary* in a wide sense) if

$$E(X_t) = E(X_{t+\tau}) \quad (4.24)$$

$$E((X_{t_1} - \mu)(X_{t_2} - \mu)) = E((X_{t_1+\tau} - \mu)(X_{t_2+\tau} - \mu)) \quad (4.25)$$

for all $t, t + \tau \in T$, $\tau > 0$.

Assume that $(X_t)_{t \in T}$ is a stochastic process on the state space $\mathbb{R}$ for which $E(X_t)$ and $\text{Var}(X_t)$ exist for all $t \in T$. Then

$$\bar{K}_X(t_1, t_2) := E((X_{t_1} - E(X_{t_1}))(X_{t_2} - E(X_{t_2}))) \quad (t_1, t_2 \in T) \quad (4.26)$$

is the *autocovariance function* of $(X_t)_{t \in T}$. Furthermore,

$$\bar{\gamma}_X(t_1, t_2) := \frac{E((X_{t_1} - E(X_{t_1}))(X_{t_2} - E(X_{t_2})))}{\sqrt{\text{Var}(X_{t_1})\text{Var}(X_{t_2})}} \quad (t_1, t_2 \in T) \quad (4.27)$$

defines the *autocorrelation function* of $(X_t)_{t \in T}$.

Assume that $(X_t)_{t \in T}$ is a stochastic process on the state space $\mathbb{R}$ for which $E(X_t)$ and $\text{Var}(X_t)$ exist for all $t \in T$. If $(X_t)_{t \in T}$ is *stationary* or *stationary* in a weak sense, (4.26) and (4.27) only depend on $|t_1 - t_2|$.

Analogously to par. 551, we define the *autocovariance function*

$$K_X(h) := E((X_t - E(X_t))(X_{t+h} - E(X_{t+h}))) \quad (h \geq 0; t, t + h \in T) \quad (4.28)$$

and the *autocorrelation function (ACF)*

$$\gamma_X(h) := \frac{E((X_t - E(X_t))(X_{t+h} - E(X_{t+h})))}{\sqrt{\text{Var}(X_t)\text{Var}(X_{t+h})}} \quad (h \geq 0; t, t + h \in T). \quad (4.29)$$

Note: If $T = [0, \infty)$, for example, $h \in [0, \infty)$ while $h \in \mathbb{N}_0$ if $T \in \{\mathbb{Z}, \mathbb{N}_0, \mathbb{N}\}$.

Pars. 585 and 587 to 590 collect useful facts on *stationary* stochastic processes. Stationarity plays an important role in *stochastic models* for *PTRNGs*; cf. Section 4.5. It captures the desired feature that if a *PTRNG* is analyzed in a certain period in time, its stochastic behavior should be the same at different times.

Note: For *stochastic models* of *physical noise sources*, the requirement is relaxed to *time-local stationarity*; cf. pars. 788 to 790.

If a (time-continuous or time-discrete) *stationary* stochastic process is ergodic, statistical properties of this stochastic process can be deduced from a single, sufficiently long realization of this stochastic process with probability 1.

Note 1: In the context of the evaluation of *PTRNGs*, this feature is exploited for the estimation of parameters, by *online tests* and by statistical tests applied by the evaluator, for example.

Note 2: There exist several equivalent formal definitions for ergodicity, e.g., that the invariant events are attained with probability 0 or 1. We refer the interested reader to the relevant literature, e.g., to [KaTa75], Chapter 9.

Note 3: Pars. 555 and 556 provide an example of an ergodic process and a counterexample. Loosely speaking, to ensure ergodicity, long-term dependencies of the stochastic process need to decrease sufficiently fast.

Example: Assume that the **random variables** $X_1, X_2, \dots$ are **iid** $B(1, p)$ -distributed. If we observe a realization sequence $x_1, x_2, \dots$, the empirical mean $n^{-1} \sum_{j=1}^n x_j$ converges to p with probability 1 (Strong law of large numbers; cf. pars. 538 and 539). If the **random variables** model the repeated tossing of a particular coin (cf. Subsec. 4.5.2), a sequence of realizations can easily be obtained by tossing this coin several times, which allows the estimation of the (unknown) parameter p . The **random variables** $X_1, X_2, \dots$ define a **stationary** ergodic process (cf. par. 556).

556 Counterexample: Assume that the **random variables** $X_1, X_2, \dots$ are identically $B(1, p)$. Unlike in par. 555, these **random variables** are not independent but fully dependent, namely $X_1 = X_2 = \dots$. Then the realization of X_1 determines the whole realization sequence. In this case, one can only observe the realization sequences $1, 1, \dots$ (with probability p) or $0, 0, \dots$ (with probability $1 - p$). Hence, it is not possible to estimate p on the basis of a single realization sequence. This stochastic process is **stationary** but not ergodic.

557 [empirical mean and empirical variance] Assume that $x_1, x_2, \dots, x_m$ are realizations of the **iid random variables** $X_1, X_2, \dots, X_m$. Assume further that the expectation $\mu = E(X_j)$ and the variance $\sigma^2 = \text{Var}(X_j)$ exist. The *arithmetic mean* $\bar{x}$ and the *empirical variance* $\bar{s}^2$ of $x_1, x_2, \dots, x_m$ are given by

$$\bar{x} := \frac{x_1 + x_2 + \dots + x_m}{m}, \quad (4.30)$$

$$\bar{s}^2 := \frac{1}{m-1} \sum_{j=1}^m (x_j - \bar{x})^2. \quad (4.31)$$

$\bar{x}$ and $\bar{s}^2$ are *unbiased* estimators of μ and σ^2 . In this context, unbiased means, that if the sample values x_j in the right-hand sides of (4.30) and (4.31) are replaced by **random variables** X_j , the expectation of these terms is μ and σ^2 , respectively.

Note: Occasionally, formula (4.31) is used with factor $1/m$ in place of $1/(m-1)$. In this case, the estimator is biased (but asymptotically unbiased).

558 [empirical mean and empirical variance] Assume that the **random variables** $X_1, X_2, \dots, X_m$ are **iid** $N(\mu, \sigma^2)$ -distributed. Then

$$\frac{X_1 + X_2 + \dots + X_m}{m} \sim N\left(\mu, \frac{\sigma^2}{m}\right) \quad \text{and} \quad (4.32)$$

$$\frac{m-1}{\sigma^2} \cdot \frac{1}{m-1} \sum_{j=1}^m (X_j - \bar{X})^2 \sim \chi_{m-1}^2 \quad (4.33)$$

where χ_{m-1}^2 denotes the χ^2 -distribution with $m-1$ degrees of freedom. Formula (4.33) is a well-known corollary from Cochran's Theorem.

559 [empirical mean and empirical variance] If the **random variables** $X_1, X_2, \dots, X_m$ are **iid** (but not

necessarily normally distributed), then

$$E \left(\frac{1}{m-1} \sum_{j=1}^m (X_j - \bar{X})^2 \right) = \sigma^2 \quad (4.34)$$

$$\text{Var} \left(\frac{1}{m-1} \sum_{j=1}^m (X_j - \bar{X})^2 \right) = \frac{1}{m} \left(E((X - \mu)^4) - \frac{m-3}{m-1} \sigma^4 \right) \quad (4.35)$$

[Allan variance] When estimating the jitter of digital clock signals, for example, the empirical variance may overestimate the jitter if low frequency noise such as flicker noise is present. In such scenarios, often the (empirical) Allan variance is used instead; cf., e.g., [ASPB+18]. Assume that the measurement values $x_1, x_2, \dots, x_m$ are taken at times $\tau, 2\tau, \dots, m\tau$ (in practice, the x_j often are fractional frequencies that have been averaged over an interval of length τ). The (empirical) Allan variance of $x_1, x_2, \dots, x_m$ is defined by 560

$$\overline{\text{AVar}} = \frac{1}{2(m-1)} \sum_{j=1}^{m-1} (x_j - x_{j+1})^2. \quad (4.36)$$

Note 1: By construction, the Allan variance is only a little sensitive to slow drifts of the distributions of the corresponding random variables $X_1, X_2, \dots, X_m$.

Note 2: The definition of the Allan variance is not uniform in the literature.

[Allan variance] Assume that the measurement values $x_1, x_2, \dots$ are realizations of the random variables $X_1, X_2, \dots$. If the random variables are stationarily distributed and uncorrelated (i.e., $\text{Cov}(X_i, X_j) = 0$ for $i \neq j$), the Allan variance coincides with the “usual” variance [ASPB+18], Theorem 1. 561

4.2.2 Useful Theorems and Facts

Subsection 4.2.2 provides facts and theorems that can be useful in the context of this document. 562

[Stirling’s approximation] 563

$$\sqrt{2\pi n} \left(\frac{n}{e} \right)^n e^{\frac{1}{12n+1}} < n! < \sqrt{2\pi n} \left(\frac{n}{e} \right)^n e^{\frac{1}{12n}} \quad (\text{Stirling’s approximation}) \quad (4.37)$$

[Stirling’s approximation] If n, k , and $n - k$ are large, applying the lower bound in Stirling’s formula (4.37) to the factorials of $\binom{n}{k}$ yields the approximation 564

$$\binom{n}{k} \approx \sqrt{\frac{n}{2\pi k(n-k)}} \cdot \frac{n^n}{k^k (n-k)^{n-k}}. \quad (4.38)$$

[Expectation: computation rules] Assume that for the (not necessarily independent nor identically distributed) **random variables** $X_1, \dots, X_k$, the expectations $E(X_j)$ exist. Let $Y = a_1X_1 + \dots + a_kX_k$ with $a_1, \dots, a_k \in \mathbb{R}$. Then the expectation of Y exists. More precisely,

$$E(Y) = E(a_1X_1 + \dots + a_kX_k) = \sum_{j=1}^k a_j E(X_j). \quad (4.39)$$

If the **random variables** are **iid** and $a_j = 1/k$ for each $j \leq k$, then $E(Y) = E(X_1) = \dots = E(X_k)$.

566 [Variance: computation rules] Assume that for the independent (but not necessarily identically distributed) **random variables** $X_1, \dots, X_k$, the variances $\text{Var}(X_j)$ exist. Let $Y = a_1X_1 + \dots + a_kX_k$ for $a_1, \dots, a_k \in \mathbb{R}$. Then the expectation of Y exists. More precisely,

$$\text{Var}(Y) = \text{Var}(a_1X_1 + \dots + a_kX_k) = \sum_{j=1}^k a_j^2 \text{Var}(X_j). \quad (4.40)$$

If we drop the assumption that the **random variables** $X_1, \dots, X_k$ are independent, then (4.40) becomes more complicated

$$\text{Var}(Y) = \text{Var}(a_1X_1 + \dots + a_kX_k) = \sum_{j=1}^k a_j^2 \text{Var}(X_j) + \sum_{i \neq j} a_i a_j \text{Cov}(X_i, X_j). \quad (4.41)$$

567 Example: Expectation and variance of $B(n, p)$ -distributed **random variables**.
The **random variable** $Y = Y_1 + \dots + Y_n \sim B(n, p)$ if $Y_1, \dots, Y_n$ are **iid** $B(1, p)$ -distributed. By (4.39) and (4.40) we conclude that $E(Y) = E(Y_1) + \dots + E(Y_n) = np$ and $\text{Var}(Y) = \text{Var}(Y_1) + \dots + \text{Var}(Y_n) = np(1 - p)$.

568 [Central Limit Theorem (CLT)] Assume that the real-valued **random variables** $X_1, X_2, \dots$ are **iid** with expectation μ and variance σ^2 . For $n = 1, 2, \dots$

$$S_n^* := \frac{X_1 + \dots + X_n - n\mu}{\sqrt{n}\sigma} \quad (4.42)$$

define normalized partial sums. The Central Limit Theorem (CLT) applies to the sequence $X_1, X_2, \dots$. More precisely,

$$\lim_{n \rightarrow \infty} \text{Prob}(S_n^* \leq x) = \Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{t^2}{2}} dt \quad \text{for all } x \in \mathbb{R}. \quad (4.43)$$

569 [tail of the standard normal distribution] For $x > 0$, it is

$$\left(\frac{1}{x} - \frac{1}{x^3} \right) \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \leq 1 - \Phi(x) = \Phi(-x) \leq \frac{1}{x} \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \quad (4.44)$$

([GaSt77], Lemma 1.19.2).

570

[CLT, parameter estimation] Assume that $X_1, X_2, \dots$ are **iid** $B(1, p)$ -distributed. Then the CLT implies

$$\begin{aligned} \text{Prob} \left(\left| \frac{1}{N} \sum_{j=1}^N X_j - p \right| > \epsilon \right) &= \text{Prob} \left(\left| \frac{\sum_{j=1}^N X_j - Np}{N} \right| > \epsilon \right) = \\ \text{Prob} \left(\left| \frac{\sum_{j=1}^N X_j - Np}{\sqrt{N} \sqrt{p(1-p)}} \right| > \frac{\epsilon \sqrt{N}}{\sqrt{p(1-p)}} \right) &= 2\Phi \left(\frac{-\epsilon \sqrt{N}}{\sqrt{p(1-p)}} \right) \leq 2\Phi(-2\epsilon \sqrt{N}). \end{aligned} \quad (4.45)$$

[CLT] Par. 568 formulates the Central Limit Theorem (CLT) for **iid random variables**. The CLT is very robust and holds under weak conditions. Under suitable conditions, the **iid** assumption and even the independence property can be dropped. Some special cases are covered in paragraphs 572, 583, and 584.

Background information: If the CLT applies, the **random variables** $S_1^*, S_2^*, \dots$ converge to $N(0, 1)$ in distribution. We do not go deeper but refer the interested reader to ([Geor15], Subsection 5.3).

[CLT] Assume that the real-valued **random variables** $X_1, X_2, \dots$ are independent (but not necessarily **iid**) with expectations $E(X_j) = \mu_j$ and variances $\text{Var}(X_j) = \sigma_j^2$ for $j \in \mathbb{N}$. For $n = 1, 2, \dots$

$$S_n^* := \frac{\sum_{j=1}^n (X_j - \mu_j)}{\sqrt{s_n^2}} \quad \text{with } s_n^2 := \sum_{j=1}^n \sigma_j^2 \quad (4.46)$$

defines normalized partial sums. Assume further that the Lindeberg condition holds

$$\lim_{n \rightarrow \infty} L_n(\delta) = 0 \quad \text{for all } \delta > 0 \quad \text{where } L_n(\delta) := \frac{1}{s_n^2} \sum_{j=1}^n E((X_j - \mu_j)^2 1_{\{|X_j - \mu_j| \geq \delta s_n\}}). \quad (4.47)$$

Then the Central Limit Theorem (CLT) applies to the sequence $X_1, X_2, \dots$. In particular,

$$\lim_{n \rightarrow \infty} \text{Prob}(S_n^* \leq x) = \Phi(x) \quad \text{for all } x \in \mathbb{R}. \quad (4.48)$$

[CLT] Assume that the **random variables** $X_1, X_2, \dots$ are **iid** and that besides $E(X_1)$ and $E(X_1^2)$, also the third moment $E(X_1^3)$ exist. Then the well-known Berry-Esséen-Theorem provides an upper bound for the maximal difference between the exact cumulative distribution function of S_n^* and $\Phi(\cdot)$. It is

$$(\text{Berry-Esséen-Theorem}) \quad |\text{Prob}(S_n^* \leq x) - \Phi(x)| \leq C \frac{E(|X_1 - E(X_1)|^3)}{(\text{Var}(X_1))^{1.5}} \frac{1}{\sqrt{n}} \quad \text{for each } x \in \mathbb{R} \quad (4.49)$$

for a suitable constant C (cf. [Geor15], Bemerkung (5.31), with $C = 0.8$). In [Shev11], it is proved that $C < 0.4748$. In particular, (4.49) says that the rate of convergence is $O(n^{-0.5})$.

A sequence $X_1, X_2, \dots$ of **random variables** is called q -dependent if the random vectors $(X_1, \dots, X_u)$ and $(X_v, \dots, X_n)$ are independent for all $1 \leq u < v \leq n$ with $v - u > q$.

Note: The components of each vector need not be independent.

[CLT for q -dependent random variables, [HoRo48]] Let $X_1, X_2, \dots$ be a q -dependent (not necessarily stationary) sequence of random variables such that $E(|X_i|^3)$ is uniformly bounded for all $i \in \mathbb{N}$.

$$A_i := \text{Var}(X_{i+q}) + 2 \sum_{j=1}^q \text{Cov}(X_{i+q-j}, X_{i+q}) \quad \text{for } i \in \mathbb{N}. \quad (4.50)$$

If the limit $A := \lim_{u \rightarrow \infty} u^{-1} \sum_{h=1}^u A_{i+h}$ exists uniformly for all $i \in \mathbb{N}$, then

$$\lim_{n \rightarrow \infty} \text{Prob} \left(\frac{\sum_{j=1}^n (X_j - E(X_j))}{\sqrt{An}} \leq x \right) = \Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{t^2}{2}} dt \quad \text{for all } x \in \mathbb{R}. \quad (4.51)$$

576 [CLT for q -dependent random variables] If the random variables $X_1, X_2, \dots$ in par. 575 are stationary, the necessary conditions simplify considerably: It suffices that $E(|X_i|^3)$ and

$$A = \sigma^2 := \text{Var}(X_1) + 2 \sum_{j=1}^q \text{Cov}(X_1, X_{1+j}). \quad (4.52)$$

exist. In particular, for $\mu := E(X_1)$, we obtain the equivalent to (4.51)

$$\lim_{n \rightarrow \infty} \text{Prob} \left(\frac{\sum_{j=1}^n (X_j - \mu)}{\sigma \sqrt{n}} \leq x \right) = \Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{t^2}{2}} dt \quad \text{for all } x \in \mathbb{R}. \quad (4.53)$$

577 [CLT, dependent random variables] The CLT may even hold if $X_1, X_2, \dots$ has no finite memory, provided that the dependencies decrease sufficiently fast. If the sequence $X_1, X_2, \dots$ is stationary and, e.g., strongly mixing, the CLT holds if some further conditions are fulfilled. If needed, the reader is referred to [Jone04], Section 4, for details.

578 [Markov chains] Assume that the random variables $X_0, X_1, \dots$ take on values in a countable set Ω . Assume that

$$\text{Prob}(X_{n+1} = x_{n+1} \mid X_1 = x_1, \dots, X_n = x_n) = \text{Prob}(X_{n+1} = x_{n+1} \mid X_n = x_n) \quad (4.54)$$

for each $n \in \mathbb{N}_0$ and all $x_0, x_1, \dots, x_{n+1} \in \Omega$, provided that both conditional probabilities in (4.54) are well-defined (note: the latter is the case if $\text{Prob}(X_1 = x_1, \dots, X_n = x_n) > 0$). We say that $X_0, X_1, \dots$ is a (time-discrete) Markov chain on the state space Ω . If the right-hand conditional probabilities in (4.54) do not depend on n , the Markov chain is *homogeneous*.

579 [Markov chains] Condition (4.54) says that X_{n+1} may depend on X_n but any further information on the preceding random variables $X_0, \dots, X_{n-1}$ does not provide additional information on the outcome of X_{n+1} .

Note 1: Equation (4.54) does not imply that X_{n+1} and X_{n-1} are independent.

Note 2: Condition (4.54) can be generalized to time-continuous stochastic processes on arbitrary state spaces (Markov processes).

580 [Markov chains] Assume that $X_0, X_1, \dots$ is a homogeneous Markov chain on the finite state space $\Omega = \{\omega_1, \dots, \omega_k\}$. The transition matrix $P = (p_{ij})_{1 \leq i, j \leq k}$ is defined by $p_{ij} = \text{Prob}(X_{n+1} = \omega_j \mid$

$X_n = \omega_i$). If the row vector ν_j denotes the distribution of X_j , then $\nu_{n+1} = \nu_n P$.

Note: In the literature on Markov chains, traditionally row vectors are used instead of column vectors.

[Markov chains] Assume that $X_0, X_1, \dots$ is a homogeneous Markov chain on the finite state space $\Omega = \{\omega_1, \dots, \omega_k\}$ with transition matrix P . Assume further that there is an integer $m \in \mathbb{N}$ for which all entries of P^m are positive. 581

Then for each initial distribution ν_0 , the sequence of distributions $\nu_0, \nu_1, \dots$ converges to a limit distribution ν with $\nu(\omega_j) > 0$ for all $j \leq k$. The limit distribution ν is the unique left eigenvector of P to the eigenvalue 1. The convergence rate is exponentially (e.g., [Geor15], Subsection 6.3.1).

If the Markov chain from par. 581 already starts in the equilibrium state, namely if $\nu_0 = \nu$, 582 then $\nu_0 = \nu_1 = \dots = \nu$. Then the Markov chain $X_0, X_1, \dots$ is **stationary** and ergodic (see, e.g., [Geor15], Subsection 6.3.1).

[CLT, Markov chain] Assume that $X_0, X_1, \dots$ is a homogeneous Markov chain on the finite 583 state space $\Omega = \{\omega_1, \dots, \omega_k\}$ with transition matrix P . Assume further that the Markov chain converges to a limit distribution ν regardless of ν_0 (as in par. 581). Let $g: \Omega \rightarrow \mathbb{R}$ any mapping. Then the Central Limit Theorem applies to $g(X_1), g(X_2), \dots$. The normalized partial sums (see par. 568) are given by

$$S_n^* := \frac{g(X_1) + \dots + g(X_n) - n\mu}{\sqrt{n}\sigma} \quad \text{with} \quad (4.55)$$

$$\mu := \mathbb{E}(g(X_1)) \quad \text{and} \quad \sigma^2 := \text{Var}(g(X_1)) + 2 \sum_{k=1}^{\infty} \text{Cov}(g(X_1), g(X_{1+k})) .$$

The version of the CLT from par. 583 also applies under more general conditions. 584

Note: If the **random variables** $X_1, X_2, \dots$ are independent (special case of a Markov chain), (4.55) coincides with (4.42), applied to the **random variables** $g(X_1), g(X_2), \dots$.

[Stationarity, stationarity in a weak sense] 585

a) Assume that a stochastic process $(X_t)_{t \in T}$ is **stationary** and that $E(X_t)$ and $\text{Var}(X_t)$ exist. Then $(X_t)_{t \in T}$ is also **stationary** in a weak sense.

b) Assume that a *Gaussian* stochastic process $(X_t)_{t \in T}$ (cf. par. 542) is **stationary** in a weak sense. Then $(X_t)_{t \in T}$ is **stationary**.

Note 1: In the context of **PTRNGs**, the condition that expectation and variance exist should usually be fulfilled.

Note 2: Assertion b) can be very useful in the evaluation of **PTRNGs**.

[Stationarity] The pars. 587 to 590 provide useful facts on **stationary** stochastic processes for 586 which stationarity is “inherited”. These results can be very useful for the analysis of **PTRNGs**.

[Stationarity] Proposition: Assume that the stochastic process $(X_t)_{t \in T}$ is **stationary** on the state 587 space $\Omega = \mathbb{R}^m$ for some integer $m \geq 1$. Furthermore, let $f: \Omega \rightarrow \Omega' := \mathbb{R}^s$ denote a measurable mapping (as usual, we consider the Borel- σ -algebras on $\mathbb{R}^m$ and $\mathbb{R}^s$).

a) The stochastic process $(f(X_t))_{t \in T}$ is **stationary**, too.

b) Let $\Delta t > 0$. The stochastic process $(Y_t)_{t \in T}$, given by $Y_t := X_{t+\Delta t} - X_t$, defines a **stationary**

process on $\mathbb{R}^m$ (we tacitly assume that $T + \Delta t \subseteq T$).

c) Let $(Z_t)_{t \in T'}$ with $T' \subseteq T$ and $Z_t = X_t$ for each $t \in T'$. Then $(Z_t)_{t \in T'}$ is **stationary**, too. Proof: a) Since f is measurable, $f^{-1}(B') \in \mathcal{B}(\mathbb{R}^m)$ for all $t \in T$ and $B' \in \mathcal{B}(\mathbb{R}^s)$, and thus the stationarity of $(X_t)_{t \in T}$ implies that $(f(X_t))_{t \in T}$ is **stationary**, too.

b) For given $t_1, \dots, t_k, \tau$ we temporarily define the sets $S := \{t_1, t_1 + \Delta t, t_2, t_2 + \Delta t, \dots, t_k, t_k + \Delta t\}$ and $S_\tau := \{t_1 + \tau, t_1 + \Delta t + \tau, t_2 + \tau, t_2 + \Delta t + \tau, \dots, t_k + \tau, t_k + \Delta t + \tau\}$. Assume for the moment that $|S| = 2k$ (and thus also $|S_\tau| = 2k$). The stationarity of $(X_t)_{t \in T}$ implies

$$\begin{aligned} \text{Prob}(X_{t_1 + \Delta t} \in B_1, X_{t_1} \in C_1, \dots, X_{t_k + \Delta t} \in B_k, X_{t_k} \in C_k) = \\ \text{Prob}(X_{t_1 + \Delta t + \tau} \in B_1, X_{t_1 + \tau} \in C_1, \dots, X_{t_k + \Delta t + \tau} \in B_k, X_{t_k + \tau} \in C_k) = \\ \text{for all } B_1, C_1, \dots, B_k, C_k \in \mathcal{B}(\mathbb{R}^m) \end{aligned} \quad (4.56)$$

Since the set $\{B'_1 \times C'_1 \times B'_2 \times \dots \times C'_k \mid B'_j, C'_j \in \mathcal{B}(\mathbb{R}^m)\}$ is stable under intersections and generates the product σ -algebra $\mathcal{B}(\mathbb{R}^m) \otimes \mathcal{B}(\mathbb{R}^m) \otimes \dots \otimes \mathcal{B}(\mathbb{R}^m) = \mathcal{B}(\mathbb{R}^{2km})$ on $\mathbb{R}^{2km}$, the random vectors $\vec{Z}' := (X_{t_1 + \Delta t}, X_{t_1}, \dots, X_{t_k + \Delta t}, X_{t_k})$ and $\vec{Z}'_\tau := (X_{t_1 + \Delta t + \tau}, X_{t_1 + \tau}, \dots, X_{t_k + \Delta t + \tau}, X_{t_k + \tau})$ are identically distributed on $\mathbb{R}^{2km}$. Consequently, $\vec{Y} := (Y_{t_1}, \dots, Y_{t_k})$ and $\vec{Y}_\tau := (Y_{t_1 + \tau}, \dots, Y_{t_k + \tau})$ are identically distributed on $\mathbb{R}^{mk}$. In particular,

$$\begin{aligned} \text{Prob}(Y_{t_1} \in A_1, \dots, Y_{t_k} \in A_k) = \text{Prob}(Y_{t_1 + \tau} \in A_1, \dots, Y_{t_k + \tau} \in A_k) \\ \text{for all } A_1, \dots, A_k \in \mathcal{B}(\mathbb{R}^m). \end{aligned} \quad (4.57)$$

For the case $|S| < 2k$, an analogous procedure (considering **random variables** $\vec{Z}'$ and $\vec{Z}'_\tau$ on $\mathbb{R}^{|S|m}$) also verifies (4.57). Since $t_1, \dots, t_k, \tau$ were arbitrary, this completes the proof of b).

c) Compared to $(X_t)_{t \in T}$, the “restricted” process $(Z_t)_{t \in T'}$ only has to satisfy a subset of conditions of type (4.22) to be **stationary**.

588 [Stationarity] Proposition: Assume that the stochastic process $(X_n)_{n \in \mathbb{N}}$ is **stationary** on a state space $\Omega = \mathbb{R}^m$ for some integer $m \geq 1$.

a) The stochastic process $\vec{Y}_1, \vec{Y}_2, \dots$ with $\vec{Y}_n := (X_{(n-1)t+1}, \dots, X_{nt})$ is **stationary** on $\mathbb{R}^{mt}$. (The random vectors $\vec{Y}_1, \vec{Y}_2, \dots$ are non-overlapping.)

b) The stochastic process $\vec{Z}_1, \vec{Z}_2, \dots$ with $\vec{Z}_n := (X_n, \dots, X_{n+t-1})$ is **stationary** on $\mathbb{R}^{mt}$. (The random vectors $\vec{Z}_1, \vec{Z}_2, \dots$ are overlapping.)

Proof: For given $j_1, \dots, j_k, \tau$ we temporarily define the set $M = \{j_1, j_1 + 1, \dots, j_1 + t - 1, j_2, \dots, j_k + t - 1\}$. The stationarity of $(X_n)_{n \in \mathbb{N}}$ implies

$$\begin{aligned} \text{Prob}(\vec{Y}_{j_i} \in (B_{(j_i-1)t+1} \times \dots \times B_{j_it}) \text{ for } i \leq k) = \text{Prob}(X_i \in B_i \text{ for } i \in M) = \\ \text{Prob}(X_{i+\tau} \in B_i \text{ for } i \in M) = \text{Prob}(\vec{Y}_{j_i+\tau} \in (B_{(j_i-1)t+1} \times \dots \times B_{j_it}) \text{ for } i \leq k) \\ \text{for each } \tau \in \mathbb{N} \text{ and all } B_i \in \mathcal{B}(\mathbb{R}^m), i \in M \end{aligned} \quad (4.58)$$

Since the set $\{B'_1 \times B'_2 \times \dots \times B'_{kt} \mid B'_i \in \mathcal{B}(\mathbb{R}^m)\}$ is stable under intersections and generates $\mathcal{B}(\mathbb{R}^{kmt})$, the vectors $(\vec{Y}_1, \dots, \vec{Y}_k)$ and $(\vec{Y}_{1+\tau}, \dots, \vec{Y}_{k+\tau})$ are identically distributed on $\mathbb{R}^{kmt}$. Therefore, (4.58) implies

$$\text{Prob}(\vec{Y}_{j_i} \in A_{j_i} \text{ for } i \leq k) = \text{Prob}(\vec{Y}_{j_i+\tau} \in A_{j_i} \text{ for } i \leq k) \text{ for each } \tau \in \mathbb{N} \text{ and } A_{j_i} \in \mathcal{B}(\mathbb{R}^{mt}). \quad (4.59)$$

b) The proof of b) is analogous to the proof of a). In the second and the third term of (4.59), components X_i can appear several times. This is not a problem. It is, for example, $(X_i \in A', X_i \in A'')$ equivalent to $X_i \in A' \cap A''$.

[Stationarity] Proposition: Given a (not necessarily **stationary**) stochastic process $(X_n)_{n \in \mathbb{N}}$ with state space $\Omega = \mathbb{R}^m$ for some integer $m \geq 1$. Furthermore, assume that the stochastic processes $(\vec{Y}_n)_{n \in \mathbb{N}}$ (non-overlapping random vectors with t components) and $(\vec{Z}_n)_{n \in \mathbb{N}}$ (overlapping random vectors with t components) are defined as in par. 588.

a) Assume that the stochastic process $(\vec{Y}_n)_{n \in \mathbb{N}}$ is **stationary**. Then the stochastic process $(X_n)_{n \in \mathbb{N}}$ need not be **stationary**.

b) If the stochastic process $(\vec{Z}_n)_{n \in \mathbb{N}}$ is **stationary**, the underlying stochastic process $(X_n)_{n \in \mathbb{N}}$ is **stationary**, too.

Proof: a) Counterexample ($m = 1$): Let $X_1, X_2, \dots$ denote independent binary-valued **random variables**. Furthermore, $X_i \sim B(1, 0.5)$ for $i \in \{t, 2t, \dots\}$ and $X_i \sim B(1, 0)$ else. Then $(\vec{Y}_n)_{n \in \mathbb{N}}$ is **stationary** but not $(X_n)_{n \in \mathbb{N}}$.

b) The second equality of (4.60) follows from the stationarity of $(\vec{Y}_n)_{n \in \mathbb{N}}$.

$$\begin{aligned} \text{Prob}(X_{j_1} \in A_1, \dots, X_{j_k} \in A_k) &= \text{Prob}(\vec{Y}_{j_1} \in A_1 \times \Omega^{t-1}, \dots, \vec{Y}_{j_k} \in A_k \times \Omega^{t-1}) = \\ \text{Prob}(\vec{Y}_{j_1+\tau} \in A_1 \times \Omega^{t-1}, \dots, \vec{Y}_{j_k+\tau} \in A_k \times \Omega^{t-1}) &= \text{Prob}(X_{j_1+\tau} \in A_1, \dots, X_{j_k+\tau} \in A_k) \end{aligned} \quad (4.60)$$

[Stationarity] Proposition: Assume that the stochastic processes $(X_t)_{t \in T}$ and $(Y_t)_{t \in T}$ is **stationary** on the state space $\Omega = \mathbb{R}^m$ for some integer $m \geq 1$. 590

a) If the stochastic processes $(X_t)_{t \in T}$ and $(Y_t)_{t \in T}$ are independent and $\odot \in \{+, -\}$, the stochastic process $(Z_t)_{t \in T}$ with $Z_t = X_t \odot Y_t$ is **stationary**, too.

b) Let $m = 1$. If the stochastic processes $(X_t)_{t \in T}$ and $(Y_t)_{t \in T}$ are independent and $\odot \in \{+, -, \cdot, /\}$, the stochastic process $(Z_t)_{t \in T}$ with $Z_t = X_t \odot Y_t$ is **stationary**, too.

For $\odot = /$, the additional condition $\text{Prob}(X_t = 0) = 0$ is required.

c) If the stochastic processes $(X_t)_{t \in T}$ and $(Y_t)_{t \in T}$ are not independent, in general a) and b) are not true.

Proof: a) For given $t_1, \dots, t_k, \tau$ stationarity and independence imply

$$\begin{aligned} \text{Prob}(X_{t_1} \in B_1, Y_{t_1} \in C_1, \dots, X_{t_k} \in B_k, Y_{t_k} \in C_k) &= \\ \text{Prob}(X_{t_1} \in B_1, \dots, X_{t_k} \in B_k) \cdot \text{Prob}(Y_{t_1} \in C_1, \dots, Y_{t_k} \in C_k) &= \\ \text{Prob}(X_{t_1+\tau} \in B_1, \dots, X_{t_k+\tau} \in B_k) \cdot \text{Prob}(Y_{t_1+\tau} \in C_1, \dots, Y_{t_k+\tau} \in C_k) &= \\ \text{Prob}(X_{t_1+\tau} \in B_1, Y_{t_1+\tau} \in C_1, \dots, X_{t_k+\tau} \in B_k, Y_{t_k+\tau} \in C_k) &= \\ \text{for all } B_1, C_1, \dots, B_k, C_k \in \mathcal{B}(\mathbb{R}^m) & \end{aligned} \quad (4.61)$$

Since the set $\{B_1 \times C_1 \times \dots \times B_k \times C_k \mid B_j, C_j \in \mathcal{B}(\mathbb{R}^m)\}$ is stable under intersection and generates the σ -algebra $\mathcal{B}(\mathbb{R}^{2km})$, and since the componentwise addition or subtraction of random vectors is measurable, as in par. 587, proof of assertion b), this implies

$$\begin{aligned} \text{Prob}(Z_{t_1} \in A_1, \dots, Z_{t_k} \in A_k) &= \text{Prob}(Z_{t_1+\tau} \in A_1, \dots, Z_{t_k+\tau} \in A_k) \\ \text{for all } A_1, \dots, A_k \in \mathcal{B}(\mathbb{R}^m) & \end{aligned} \quad (4.62)$$

Since $t_1, \dots, t_k, \tau$ were arbitrary, this completes the proof of a).

b) Assertion b) can be proved as a).

c) Counterexample: Let $V_0, V_1, \dots$ denote **iid** $B(1, 0.5)$ -distributed **random variables**, $Z_n = X_n + Y_n$ for $n \in \mathbb{N}$, $T = \mathbb{N}$, $\odot = +$, and $m = 1$. Let further $X_n = V_n$ for $n \geq 1$, $Y_1 = V_0$, and $Y_n = V_n$ for $n \geq 2$. Then

$$\begin{aligned} \text{Prob}(X_1 + Y_1 = 1) &= \text{Prob}(V_1 = 1, V_0 = 0) + \text{Prob}(V_1 = 0, V_0 = 1) = 0.5 \quad \text{but} \\ \text{Prob}(X_n + Y_n = 1) &= \text{Prob}(V_n + V_n = 1) = 0 \quad \text{for } n \geq 2 \end{aligned} \quad (4.63)$$

Hence, the stochastic process $(Z_n)_{n \in \mathbb{N}}$ is not **stationary**. Analogously, the stochastic process defined by $Z'_n = X_n - Y_n$ is not **stationary**. Similarly, let $X'_n = e^{V_n}$ for $n \geq 1$, $Y'_1 = e^{V_0}$, and $Y'_n = e^{V_n}$ for $n \geq 2$. Then the stochastic processes defined by $Z'''_n := X'_n \cdot Y'_n$ and $Z''''_n := X'_n / Y'_n$, respectively, are not **stationary**.

- 591 [Renewal process] Assume that $T_1, T_2, \dots$ denote **iid** non-negative **random variables**, and that the expectation $E(T_j) > 0$ exists. Furthermore, unless otherwise stated, $\text{Prob}(T_j = 0) = 0$, and there is no $\Delta > 0$ such that $\text{Prob}(T_j \in \{j\Delta \mid j \in \mathbb{N}\}) = 1$. Then $Z(t) := \inf\{k \mid T_1 + \dots + T_k > t\}$ defines a *renewal process*, where t ranges in $[0, \infty)$. The **random variables** T_j are often interpreted as lifetimes of machines and called the j^{th} holding time. In the context of **physical noise sources**, the **random variable** T_j often quantifies the intermediate time between the $(j-1)^{\text{th}}$ and the j^{th} event; see Subsections 5.4.2, 5.4.3, and 5.4.4.

- 592 [Renewal process] A *delayed renewal process* considers independent, non-negative **random variables** $T_0, T_1, T_2, \dots$. Again, the **random variables** $T_1, T_2, \dots$ are **iid**, while T_0 can have a different distribution. Furthermore, it is assumed that $E(T_0) \geq 0$ and $E(T_j) > 0$ exist. Also, let

$$J_n := T_0 + T_1 + \dots + T_n. \quad (4.64)$$

The delayed renewal process is given by

$$Z(t) := \inf\{k \mid T_0 + T_1 + \dots + T_k > t\}. \quad (4.65)$$

Note: The renewal process from par. 591 can be interpreted as a special case of a delayed renewal process with $T_0 \equiv 0$. It is also called a *non-delayed renewal process*.

- 593 [**Stationary** renewal process] A delayed renewal process is a *stationary renewal process* (or: *equilibrium renewal process*) if the increments $(Z(t_2) - Z(t_1), Z(t_3) - Z(t_2), \dots, Z(t_{m+1}) - Z(t_m))$ have the same distribution as $(Z(t_2 + t) - Z(t_1 + t), Z(t_3 + t) - Z(t_2 + t), \dots, Z(t_{m+1} + t) - Z(t_m + t))$ for all $m \in \mathbb{N}$, $0 \leq t_1 < \dots < t_{m+1}$, and $t > 0$. Then the distribution of $J_{Z(t)} - t$ converges to a limiting distribution as t tends to ∞ . More precisely, if G_T denotes the cumulative distribution function of the **random variables** T_j , then

$$G_{T,\infty}(x) := \lim_{t \rightarrow \infty} \text{Prob}(J_{Z(t)} - t \leq x) = \frac{1}{\mu} \int_0^x (1 - G_T(u)) \, du. \quad (4.66)$$

If the T_j have density $g(\cdot)$, then

$$G_{T,\infty}(\cdot) \quad \text{has density } g_\infty(x) := (1 - G_T(x))/\mu. \quad (4.67)$$

The formulae (4.66) and (4.67) are well-known, cf. [Fel65], Chapter XI, (4.10).

If the distribution of T_0 equals the limiting distribution, i.e., if $\text{Prob}(T_0 \leq x) = G_{T,\infty}(x)$, then the renewal process is **stationary**.

Note 1: This property is fulfilled if the **random variables** $\dots, J_{-1}, J_0$ are in equilibrium.

Note 2: In the context of **physical noise sources**, **stationary** renewal processes are of particular interest; cf. Subsections 5.4.2, 5.4.4, and, in particular, 5.4.3.

- 594 [**Stationary** renewal process] Assume that $\{Z(t) \mid t \geq 0\}$ defines a **stationary** renewal process for

which $\sigma^2 = \text{Var}(T_j)$ exist. Then (e.g., [Cox62], Section 4.5, Formula (18)),

$$E(Z(t)) = \frac{t}{\mu}, \quad (4.68)$$

$$\text{Var}(Z(t)) = \left(\frac{\sigma^2}{\mu^3}\right)t + \frac{1}{6} + \frac{\sigma^4}{2\mu^4} - \frac{E((T - \mu)^3)}{3\mu^3} + o(1). \quad (4.69)$$

Note: Of course, for large t , the expectation and the variance of the non-delayed renewal process are rather similar to (4.68) and (4.69). In particular, for the non-delayed renewal process (4.68) applies only asymptotically.

[Stationary renewal process] If $\{Z(t) \mid t \in [0, \infty)\}$ defines a stationary renewal process, then 595

$$((T_{Z(t)}, T_{Z(t)+1}, \dots), T_{Z(t)} - t) \text{ is stationary in } t. \quad (4.70)$$

For (4.70) the requirement that the random variables $T_1, T_2, \dots$ are iid can be relaxed to the assumption that $T_1, T_2, \dots$ are stationarily distributed and ergodic; cf. [Lall86], (1.5), with $\chi = [0, \infty)$ while $\xi: [0, \infty)^{\mathbb{Z}} \rightarrow [0, \infty)$ is given by the projection onto the 0th component.

Note: Reference [Lall86] considers doubly infinite sequences of random variables; in our notation $\dots, T_{-1}, T_0, T_1, \dots$

[Convolution of probability measures on groups] Let X and Y denote independent random variables on a finite (not necessarily commutative) group G with group operation “ $\circ$ ”. If X is τ -distributed and Y is η -distributed, then $X \circ Y$ is $\tau * \eta$ -distributed (cf. par. 541) and $Y \circ X$ is $\eta * \tau$ -distributed. Recall that the convolution product $\tau * \eta$ is given by 596

$$\tau * \nu(g) = \sum_{h \in G} \tau(g \circ h^{-1})\eta(h) \quad \text{for each } g \in G. \quad (4.71)$$

If the group G is commutative, $\tau * \eta = \eta * \tau$. For ν , the uniform distribution on the (commutative or non-commutative) group G , it is $\tau * \nu = \nu * \tau = \nu$ for each τ .

[Convolution of probability measures on groups, notation] Let $X_1, X_2, \dots$ denote iid τ -distributed random variables on a finite group G with group operation “ $\circ$ ”. Then the random variable $X_1 \circ X_2 \circ \dots \circ X_k$ is $\tau^{*k} := \tau * \tau * \dots * \tau$ -distributed. 597

[Convolution of probability measures on groups] Let τ denote a probability measure on a finite group G and ν the uniform distribution on G . Then the following assertions hold. 598

- (a) ([Diac88], Chapter 3, p. 23) If the mass of τ is not concentrated on a proper subgroup of G or on a coset of a proper subgroup of G , then τ^{*k} converges to the uniform distribution ν .
- (b) ([Diac88], Chapter 4, Theorem 1, p. 73) Assume that there is some $n_0 \in \mathbb{N}$ and a real number $c \in (0, 1)$ such that

$$\tau^{*k}(g) \geq c\nu(g) = c \cdot \frac{1}{|G|} \quad \text{for all } g \in G \text{ and all } k \geq n_0. \quad (4.72)$$

Then the variation distance (cf. par. 613) is bounded by

$$\delta(\tau^{*k}, \nu) \leq (1 - c)^{\lfloor \frac{k}{n_0} \rfloor} \quad \text{for all } k \in \mathbb{N}. \quad (4.73)$$

(c) As in par. 647, inequality (4.73) enables an easy derivation of a lower min-entropy bound.

$$\begin{aligned} H_{\min}(X_1 \circ \cdots \circ X_k) &\geq -\log_2 \left(\frac{1}{|G|} + \delta(\tau^{*k}, \nu) \right) \geq \\ &-\log_2 \left(\frac{1}{|G|} + (1 - c)^{\lfloor \frac{k}{n_0} \rfloor} \right) \quad \text{for all } k \in \mathbb{N}. \end{aligned} \quad (4.74)$$

Note: Assertions (a) and (b) generalize to compact groups if the uniform distribution is substituted by the Haar measure; cf., e.g., [Diac88], Chapter 3, Theorem 1, p. 23.

599 [Convolution of probability measures on groups] The rate of convergence ensured by the right-hand sides of (4.73) and (4.74) is often very pessimistic. The reason is that the element, for which $\tau^{*n_0}(\cdot)$ is minimal, determines the constant $c \in (0, 1)$; it is $c \leq \min_{g \in G} \{\tau^{*k}(g)\}/|G|$. In many cases of interest, better bounds can be derived; cf. par. 600f. and 955.

600 [Convolution of probability measures on groups] Again, assume that τ is a probability measure on a finite group G . Then, by (4.71)

$$\tau * \tau(g) = \sum_{h \in G} \tau(g \circ h^{-1}) \tau(h) \quad \text{for all } g \in G. \quad (4.75)$$

The sum in (4.75) consists of $|G|$ summands, and both the arguments of the left factors and of the right factors are permutation of the group elements. We enumerate the elements of G such that

$$\tau(g_1) \leq \tau(g_2) \leq \cdots \leq \tau(g_{|G|}) \quad (4.76)$$

are ordered increasingly. By the rearrangement inequality we obtain

$$\sum_{j=1}^{|G|} \tau(g_j) \tau(g_{|G|+1-j}) \leq \tau * \tau(g) \leq \sum_{j=1}^{|G|} \tau(g_j) \tau(g_j) \quad \text{for all } g \in G. \quad (4.77)$$

Note that the right-hand side of (4.77) is the collision probability of τ ; cf. par. 611.

601 [Convolution of probability measures on groups, special case] Example: Assume that G is given by the set $\{0, 1\}^t$, equipped with some group operation “ $\circ$ ”. (The group operation can be the bitwise XOR-operation or the addition modulo 2^t , for example.) Assume that $X_1 = (B_1, \dots, B_t)$ and $X_2 = (B_{t+1}, \dots, B_{2t})$ with iid $B(1, p)$ -distributed random variables $B_1, B_2, \dots, B_{2t}$. Furthermore, let $p = 0.5 + 0.5s'$ where $s' \in [0, s]$ for some fixed $s \in [0, 1)$. Since $\binom{t}{j} = \binom{t}{t-j}$ the right-hand side of (4.77) equals

$$\begin{aligned} \sum_{j=1}^{|G|} \tau(g_j)^2 &= \sum_{j=0}^t \binom{t}{j} (0.5 + 0.5s')^{2j} (0.5 - 0.5s')^{2t-2j} \\ &= \sum_{j=0}^t \binom{t}{j} ((0.5 + 0.5s')^2)^j ((0.5 - 0.5s')^2)^{t-j} \\ &= (0.5 + 0.5s'^2)^t = 0.5^t (1 + s'^2)^t \leq 0.5^t (1 + s^2)^t. \end{aligned} \quad (4.78)$$

The second to last term in (4.78) term equals the collision probability of X_1 while the last term provides an upper bound for the least favorable parameters. In particular,

$$\begin{aligned} H_{\min}(X_1 \circ X_2) &= -\log_2 \left(\max_{g \in G} \tau * \tau(g) \right) \geq H_2(X_1) = t \left(1 - \log_2 (1 + s'^2) \right) \\ &\geq t \left(1 - \log_2 (1 + s^2) \right). \end{aligned} \quad (4.79)$$

Similarly, the left-hand side of (4.77) equals

$$\begin{aligned} \sum_{j=1}^{|G|} \tau(g_j) \tau(g_{|G|+1-j}) &= \sum_{j=0}^t \binom{t}{j} (0.5 - 0.5s')^j (0.5 + 0.5s')^{t-j} (0.5 - 0.5s')^{t-j} (0.5 + 0.5s')^j \\ &= \sum_{j=0}^t \binom{t}{j} (0.5 - 0.5s')^t (0.5 + 0.5s')^t \\ &= 0.5^t (1 - s')^t (1 + s')^t = 0.5^t (1 - s'^2)^t \geq 0.5^t (1 - s^2)^t. \end{aligned} \quad (4.80)$$

Note: Under the conditions of this paragraph, i.e., for iid $B(1, p)$ -distributed **raw random numbers** B_j , inequality (4.78) should be used to determine a constant c_2 for the convolution product τ^{*2} , which is significantly larger than c^2 where the constant c belongs to τ . Since $\tau^{*2k} = (\tau^{*2})^{*k}$ the inequalities (4.73) and (4.74) provide better bounds for the variational distance and for the **min-entropy**, respectively. In fact,

$$c = \frac{(0.5 - 0.5s)^t}{0.5^t} = (1 - s)^t \quad \text{but} \quad c_2 = \frac{(0.5 - 0.5s^2)^t}{0.5^t} = (1 - s^2)^t. \quad (4.81)$$

[Convolution of probability measures on groups] Let $Y_0, Z_1, Z_2, \dots$ denote independent **random variables** on a (not necessarily commutative) finite group G . Furthermore, Y_0 is uniformly ν -distributed, whereas $Z_1, Z_2, \dots$ are τ -distributed. For $n \in \mathbb{N}$, let further $Y_{n+1} = Y_n \circ Z_{n+1}$. Then 602

- a) It follows from par. 596 by induction that the **random variables** $Y_0, Y_1, \dots$ are **uniformly distributed**.
- b) The **random variables** $Y_0, Y_1, \dots$ are **stationarily distributed**.
- c) Unless $\tau = \nu$, the **random variables** $Y_0, Y_1, \dots$ are not independent.

[Convolution of probability measures on groups] Let the **random variables** $Z_1, Z_2, \dots$ and $Y_0, Y_1, \dots$ be defined as in par. 602. Furthermore, $f: G \rightarrow [0, M] \subseteq \mathbb{R}$ is a real-valued function on G . Then 603

$$E(f(Y_j)) = \frac{1}{|G|} \sum_{g \in G} f(g) =: \mu \quad \text{for } j = 0, 1, \dots \quad (4.82)$$

Assume that τ satisfies condition (4.72). Then

$$\begin{aligned}
 \text{Cov}(f(Y_0), f(Y_k)) &= E(f(Y_0) \cdot f(Y_k)) - E(f(Y_0)) \cdot E(f(Y_k)) = \\
 &= \sum_{g \in G} \sum_{h \in G} \text{Prob}(Y_0 = g) \cdot \text{Prob}(Y_k = h) f(g) f(h) - \mu^2 = \\
 &= \sum_{g \in G} \sum_{h \in G} \text{Prob}(Y_0 = g) \cdot \text{Prob}(Z_1 + \dots + Z_k = g^{-1} \circ h) f(g) f(h) - \mu^2 = \\
 &= \sum_{g \in G} \sum_{h \in G} \nu(g) \tau^{*k}(g^{-1} \circ h) f(g) f(h) - \mu^2 = \\
 &= \sum_{g \in G} \sum_{k \in G} \nu(g) \tau^{*k}(k) f(g) f(g \circ k) - \mu^2 = \\
 &= \sum_{g \in G} \sum_{k \in G} \nu(g) (\nu(k) + \Delta(k)) f(g) f(g \circ k) - \mu^2 = \\
 &= \frac{1}{|G|} \sum_{g \in G} \sum_{k \in G} \Delta(k) f(g) f(g \circ k) \quad \text{with } \Delta(g') := \tau^{*k}(g') - \nu(g') \text{ for all } g' \in G. \quad (4.83)
 \end{aligned}$$

Let $A_+ := \{g \in G \mid \Delta(g) > 0\}$ and $A_- := \{g \in G \mid \Delta(g) < 0\}$. With this notation, by par. 598(b)

$$\begin{aligned}
 |\text{Cov}(f(Y_0), f(Y_k))| &= \\
 &= \left| \frac{1}{|G|} \sum_{g \in G} \sum_{k \in A_+} \Delta(k) f(g) f(g \circ k) - \frac{1}{|G|} \sum_{g \in G} \sum_{k \in A_-} |\Delta(k)| f(g) f(g \circ k) \right| \leq \\
 &= \frac{1}{|G|} \sum_{g \in G} \sum_{k \in A_+} \Delta(k) M^2 = \sum_{k \in A_+} \Delta(k) M^2 = \delta(\tau^{*k}, \nu) M^2 \leq M^2 (1 - c)^{\lfloor \frac{k}{n_0} \rfloor}. \quad (4.84)
 \end{aligned}$$

(The two summands in the second line of (4.84) have opposite sign.) Recall that $\text{Cov}(f(Y_0), f(Y_0)) = \text{Var}(f(Y_0))$, and $\text{Var}(f(Y_0)) \leq M^2$. From (4.84) we conclude

$$\sum_{k=0}^{\infty} |\text{Cov}(f(Y_0), f(Y_k))| \leq \sum_{k=0}^{\infty} M^2 (1 - c)^{\lfloor \frac{k}{n_0} \rfloor} = n_0 M^2 \sum_{j=0}^{\infty} (1 - c)^j = \frac{n_0 M^2}{c} < \infty. \quad (4.85)$$

Since the random variables $Y_1, Y_2, \dots$ are stationarily distributed, (4.85) implies

$$\begin{aligned}
 0 &\leq \text{Var} \left(\frac{1}{n} \sum_{k=1}^n f(Y_k) \right) = E \left(\left(\frac{1}{n} \sum_{k=1}^n f(Y_k) - \mu \right)^2 \right) = E \left(\left(\frac{1}{n} \sum_{k=1}^n f(Y_k) - \mu \right)^2 \right) \leq \\
 &= \frac{1}{n^2} \left(\sum_{k=1}^n E((f(Y_k) - \mu)^2) + 2 \sum_{\ell=1}^{n-1} \sum_{t=\ell+1}^n |E((f(Y_\ell) - \mu)(f(Y_t) - \mu))| \right) \\
 &\leq \frac{1}{n} \left(\text{Var}(f(Y_1)) + \sum_{k=1}^{n-1} 2 |\text{Cov}(f(Y_1), f(Y_{k+1}))| \right) \leq \frac{1}{n} \cdot \frac{2n_0 M^2}{c}. \quad (4.86)
 \end{aligned}$$

4.3 Entropy and Guesswork

The central goal in the evaluation of **TRNGs** is to quantify the amount of randomness of the generated random numbers. In this section, the concepts of **entropy**, guesswork, and work factor are introduced, and their relation is explained.

4.3.1 Entropy

For the evaluation of **RNGs**, **Shannon entropy** (4.88) and **min-entropy** (4.89) play an outstanding role. Both can be viewed as special cases of **Renyi entropy**, a more general definition of **entropy**. Collision **entropy** (4.90) also has some relevance.

Let X be a **random variable** that assumes values in the finite set $\Omega = \{\omega_1, \omega_2, \dots, \omega_k\}$. The most general notion of **entropy** is the **Renyi entropy** of order α , denoted by H_α , where

$$H_\alpha(X) = \frac{1}{1-\alpha} \log_2 \sum_{i=1}^k (\text{Prob}(X = \omega_i))^\alpha, \quad 0 \leq \alpha < \infty. \quad (4.87)$$

Formula (4.87) comprises infinitely many different definitions of **entropy**. Its most important representatives are **Shannon entropy**, **min-entropy**, and **collision entropy**. For a given **random variable** X , the **entropy** values $H_\alpha(X)$ are monotonically decreasing in α .

The **entropy** $H_\alpha(X)$ only depends on the distribution μ of X . Thus, we synonymously use the notation $H_\alpha(\mu)$.

The special case $\alpha = 1$ yields the well-known **Shannon entropy**. In particular, *L'Hopital's rule* then leads to the usual definition of **Shannon entropy**

$$H_1(X) = H(X) = - \sum_{i=1}^k \text{Prob}(X = \omega_i) \log_2(\text{Prob}(X = \omega_i)). \quad (4.88)$$

If $\text{Prob}(X = \omega_i) = 0$, by convention $\text{Prob}(X = \omega_i) \log_2(\text{Prob}(X = \omega_i)) = 0$. As usual, we use H in place of H_1 to indicate the **Shannon entropy**.

Shannon entropy $H = H_1$ is sometimes also called average **entropy** or simply **entropy** due to its prevalence in information theory.

The parameter $\alpha = \infty$ yields the **min-entropy**.

$$\lim_{\alpha \rightarrow \infty} H_\alpha(X) = - \log_2 \left(\max_{1 \leq i \leq k} \{\text{Prob}(X = \omega_i)\} \right) = H_{\min}(X). \quad (4.89)$$

Besides $H_{\min}(\cdot)$, the notation $H_\infty(\cdot)$ is also common.

Finally, H_2 defines **collision entropy**. Let X and X' be two independent and identically-distributed **random variables** with values in a finite set Ω . The *collision probability* is $\text{Prob}(X = X') = \sum_{x \in \Omega} (\text{Prob}(X = x))^2$, and the *collision entropy* equals

$$H_2(X) = - \log_2 \left(\sum_{\omega \in \Omega} (\text{Prob}(X = \omega))^2 \right). \quad (4.90)$$

- 612 The inequality (4.91) explains the relation between Shannon entropy, min-entropy, and collision entropy.

$$H_{\min} \leq H_2 \leq H_1, \quad H_{\min} \leq H_2 \leq 2H_{\min}. \quad (4.91)$$

By par. 606, the min-entropy is the most conservative entropy measure. Figure 16 plots H_1 , H_2 , and H_∞ for binary-valued random variables.

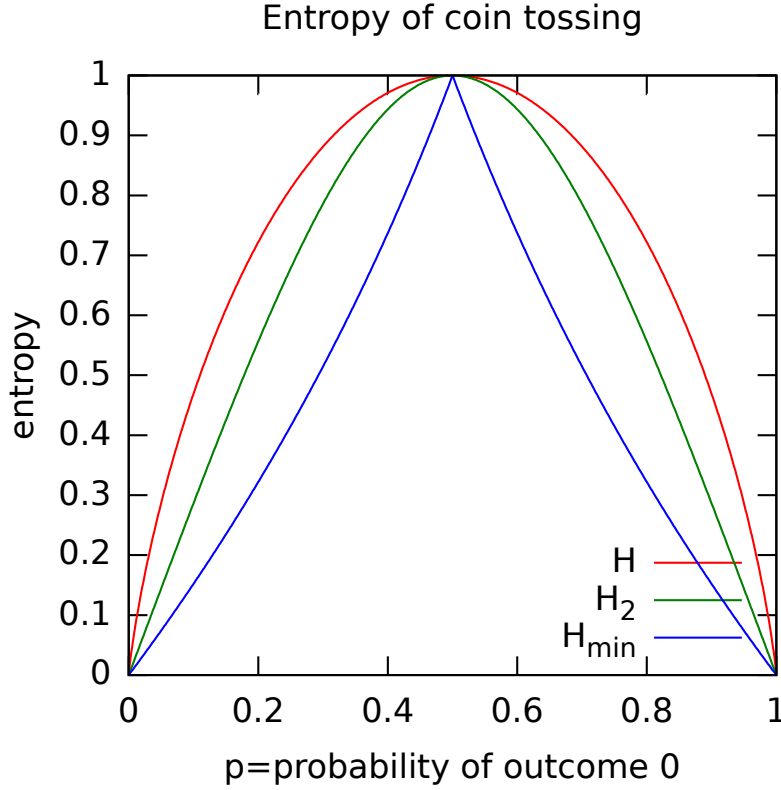


Figure 16: min-entropy $H_{\min}$ (blue), collision entropy H_2 (green), and Shannon entropy H (red) for $B(1, p)$ -distributed random variables

- 613 The variation distance (a.k.a. statistical distance) between two probability distributions $\tau = (\tau(\omega_1), \dots, \tau(\omega_k))$ and $\eta = (\eta(\omega_1), \dots, \eta(\omega_k))$ on the finite set Ω is defined by

$$\delta(\tau, \eta) := \max_{A \subseteq \Omega} \{|\tau(A) - \eta(A)|\} = \sum_{\omega \in \Omega | \tau(\omega) > \eta(\omega)} (\tau(\omega) - \eta(\omega)). \quad (4.92)$$

Note that $\delta(\tau, \tau) = 0$ and

$$\delta(\tau, \eta) = 0.5 \|\tau - \eta\|_1 \quad (4.93)$$

where $\|\cdot\|_1$ denotes the L^1 -norm. For $\nu = (\frac{1}{k}, \dots, \frac{1}{k})$ (uniform distribution on Ω), the variation

distance simplifies to

$$\delta(\tau, \nu) = \tau(A_+) - \frac{|A_+|}{k}, \quad \text{with } A_+ = \left\{ a \mid \tau(a) > \frac{1}{k}, a \in \Omega \right\}. \quad (4.94)$$

Inequality (4.95) provides an inequality for the variation distance

614

$$\sum_{x \in \Omega} (\text{Prob}(X = x))^2 \geq \frac{1 + 4(\delta(\tau, \nu))^2}{|\Omega|} \quad ([\text{CFPZ09}]). \quad (4.95)$$

For the uniform distribution ν on Ω , we have

615

$$\max_{\omega \in \Omega} \{\tau(\omega)\} \leq \frac{1}{|\Omega|} + \delta(\tau, \nu). \quad (4.96)$$

Inequality (4.96) implies an inequality for the **min-entropy** of τ

$$H_{\min}(\tau) \geq -\log_2 \left(\frac{1}{|\Omega|} + \delta(\tau, \nu) \right). \quad (4.97)$$

Note: Inequality (4.96) and thus also inequality (4.97) are sharp. The upper bound in (4.96) is attained if $A_+ = \{\omega_*\}$ and $\tau(\omega_*) = \frac{1}{|\Omega|} + \delta(\tau, \nu)$.

[Renyi **entropy** of order α] For independent **random variables** $X_1, \dots, X_m$, (4.98) follows immediately from definition (4.87). Of course, this result matches with the intuition.

616

$$H_\alpha(X_1, \dots, X_m) = H_\alpha(X_1) + \dots + H_\alpha(X_m). \quad (4.98)$$

Pars. 630 to 640 consider **min-entropy** in the context of homogeneous Markov chains, improving the general inequality (4.109).

[**Shannon entropy**: joint **entropy**] Assume that X_1 and X_2 denote (not necessarily independent) **random variables** that assume values in Ω_1 and Ω_2 , respectively. Then the (joint) **Shannon entropy** of X_1 and X_2 is given by

617

$$\begin{aligned} H(X_1, X_2) &= - \sum_{i,j=1}^k \text{Prob}(X_1 = \omega_i, X_2 = \omega_j) \log_2 (\text{Prob}(X_1 = \omega_i, X_2 = \omega_j)) = \\ &= - \sum_{i,j=1}^k \text{Prob}(X_2 = \omega_j \mid X_1 = \omega_i) \text{Prob}(X_1 = \omega_i) \log_2 (\text{Prob}(X_2 = \omega_j \mid X_1 = \omega_i) \text{Prob}(X_1 = \omega_i)). \end{aligned} \quad (4.99)$$

Since $\log_2(ab) = \log_2(a) + \log_2(b)$ (here, $a = \text{Prob}(X_2 = \omega_j \mid X_1 = \omega_i)$ and $b = \text{Prob}(X_1 = \omega_i)$), by rearranging the terms, we obtain the useful functional equation

$$H(X_1, X_2) = H(X_2 \mid X_1) + H(X_1) \quad \text{where} \quad (4.100)$$

$$H(X_2 \mid X_1) = - \sum_{i=1}^k \text{Prob}(X_1 = \omega_i) H(X_2 \mid X_1 = \omega_i). \quad (4.101)$$

Here, $H(X_2 | X_1 = x_1)$ denotes the **entropy** of X_2 under the condition that X_1 assumes the value x_1 . The term $H(X_2 | X_1)$ is the *conditional (Shannon) entropy* of X_2 und X_1 . It quantifies the average **entropy** of X_2 if X_1 is known. Clearly,

$$\min_{\omega_1 \in \Omega_1} \{H(X_2 | X_1 = \omega_1)\} \leq H(X_2 | X_1) \leq \max_{\omega_1 \in \Omega_1} \{H(X_2 | X_1 = \omega_1)\}. \quad (4.102)$$

618 The functional equation (4.100) generalizes to several **random variables**. More precisely,

$$H(X_1, \dots, X_{m+1}) = H(X_1, \dots, X_m) + H(X_{m+1} | X_1, \dots, X_m). \quad (4.103)$$

Formula (4.103) is well-known and very useful for the evaluation of physical **RNGs**.

619 Depending on the distribution of the **random variables** $X_1, X_2, \dots$, the formula for the conditional **entropy** in (4.103) may simplify considerably. In particular,

$$H(X_{m+1} | X_1, \dots, X_m) \leq H(X_{m+1}), \quad (4.104)$$

$$H(X_{m+1} | X_1, \dots, X_m) = H(X_{m+1}) \quad \text{if } X_1, X_2, \dots \text{ are independent,} \quad (4.105)$$

$$H(X_{m+1} | X_1, \dots, X_m) = H(X_{m+1} | X_m) \quad \text{if } X_1, X_2, \dots \text{ are Markovian.} \quad (4.106)$$

620 Assume that $X_1, X_2, \dots$ defines a homogeneous ergodic Markov chain with limiting distribution ν . If the Markov chain has already (at least almost) reached equilibrium, then (4.106) simplifies to

$$H(X_{m+1} | X_m) = \sum_{i=1}^k \nu(\omega_i) H(X_{m+1} | X_m = \omega_i) \quad (4.107)$$

where $H(X_{m+1} | X_m = \omega_i)$ depends on the i^{th} row of the transition matrix.

621 If the sequence of **random variables** $X_1, X_2, \dots$ is **stationary**, for each shift parameter t and for each $h \leq m$

$$H(X_{m+t+1} | X_{m+t-h+1}, \dots, X_{m+t}) = H(X_{m+1} | X_{m-h+1}, \dots, X_m). \quad (4.108)$$

622 [conditional Renyi **entropy**] For Renyi parameters $\alpha \neq 1$, no equivalent to the chain rule (4.100) exists. In the literature, several definitions of *conditional Renyi entropy* $H_\alpha(X_{m+1} | X_1, \dots, X_m)$ exist; cf., e.g., [TwSh13], Subsection 2.2. For any definition of conditional Renyi **entropy** $H_\alpha(X_{m+1} | X_1, \dots, X_m)$ that is reasonable (in our context), inequality (4.102) should generalize to

$$\min_{\omega_{i_1}, \dots, \omega_{i_m}} \{H_\alpha(X_{m+1} | X_1 = \omega_{i_1}, \dots, X_m = \omega_{i_m})\} \leq H_\alpha(X_{m+1} | X_1, \dots, X_m) \leq \max_{\omega_{i_1}, \dots, \omega_{i_m}} \{H_\alpha(X_{m+1} | X_1 = \omega_{i_1}, \dots, X_m = \omega_{i_m})\}. \quad (4.109)$$

623 [conditional Renyi **min-entropy**] Example: This example shows that the lower and the upper bound of inequality (4.109) can be rather different. Let the mapping $f: \{1, \dots, 7\} \rightarrow \{a, b\}$ be given by $f(1) = a$ and $f(j) = b$ for $j = 2, \dots, 7$. The **random variable** X assumes values in $\{1, \dots, 7\}$ with $\text{Prob}(X = 1) = 1/4$ and $\text{Prob}(X = j) = 1/8$ for $j = 2, \dots, 7$, and finally

$Y = f(X)$. Then $H_{\min}(X) = 2$, $H_{\min}(X \mid Y = a) = -\log(1) = 0$, and $H_{\min}(X \mid Y = b) = -\log(1/6) = \log_2(6) > H_{\min}(X)$.

[conditional **min-entropy**] In this document, besides conditional **Shannon entropy** ($\alpha = 1$) we focus on conditional **min-entropy** ($\alpha = \infty$). While the definition of the conditional **min-entropy** of $Y \mid X = x$ is obvious, this is not the case for the conditional **min-entropy** of $Y \mid X$. Equations (4.110) (“worst case”) and (4.111) (“average case”) provide two different definitions of conditional **min-entropy**; cf. [IwSh13], eqs. (12), (11). Equation (4.112) corresponds to [IwSh13], eq. (2), with $\alpha = \infty$.

$$H_{\min}(Y \mid X) := \min_{x \in \Omega_1} H_{\min}(Y \mid X = x) = \left(-\log_2 \left(\max_{x \in \Omega_1, y \in \Omega_2} \text{Prob}(Y = y \mid X = x) \right) \right)$$

(“worst case” conditional **min-entropy**). (4.110)

$$H_{\min(av)}(Y \mid X) := -\log_2 \left(\sum_{x \in \Omega_1} \text{Prob}(X = x) \max_{y \in \Omega_2} \text{Prob}(Y = y \mid X = x) \right) =$$

$$= -\log_2 \left(\sum_{x \in \Omega_1} \text{Prob}(X = x) 2^{-H_{\min}(Y \mid X = x)} \right)$$

(average conditional **min-entropy**). (4.111)

$$H_{\min(av,alt)}(Y \mid X) := \sum_{x \in \Omega_1} \text{Prob}(X = x) H_{\min}(Y \mid X = x)$$

(average conditional **min-entropy**; *alternative definition*). (4.112)

[conditional **min-entropy**]

With regard to the definition of the conditional **Shannon entropy** (4.101), definition (4.112) might seem to be more natural than (4.111). Inequality (4.113) shows the relationship between all definitions.

$$H_{\min}(Y \mid X) \leq H_{\min(av)}(Y \mid X) \leq H_{\min(av,alt)}(Y \mid X) \quad (4.113)$$

The left inequality in (4.113) is obvious because in (4.110) the maximum is calculated over all pairs (x, y) . The function $x \mapsto -\log_2(x)$ is convex on $(0, \infty)$, and thus the second inequality follows from Jensen’s inequality.

$$\begin{aligned} H_{\min(av)}(Y \mid X) &= -\log_2 \left(\sum_{x \in \Omega_1} \text{Prob}(X = x) 2^{-H_{\min}(Y \mid X = x)} \right) \\ &\leq \sum_{x \in \Omega_1} \text{Prob}(X = x) \left(-\log_2 \left(2^{-H_{\min}(Y \mid X = x)} \right) \right) \\ &= \sum_{x \in \Omega_1} \text{Prob}(X = x) H_{\min}(Y \mid X = x) = H_{\min(av,alt)}(Y \mid X). \end{aligned} \quad (4.114)$$

Inequality (4.114) shows that $H_{\min(av)}(\cdot \mid \cdot)$ is more conservative than $H_{\min(av,alt)}(\cdot \mid \cdot)$. In the context of **PTRNGs**, the difference between $H_{\min(av)}(Y \mid X)$ and $H_{\min(av,alt)}(Y \mid X)$ should usually be small. In [DORR08], the authors discuss an extreme example for which both definitions differ significantly. In the following (depending on the context), we only use definitions (4.110) and (4.111) but not (4.112).

[conditional **min-entropy**]

Both definitions (4.110) and (4.111) fulfill inequality (4.109) for $\alpha = \infty$. Definition (4.110) also satisfies a weak chain rule:

$$H_{\min}(Y, X) \geq H_{\min}(X) + H_{\min}(Y | X). \quad (4.115)$$

(4.115) is a “weak” equivalent to the chain rule (4.100), which holds for the **Shannon entropy**.

627 [conditional **min-entropy**] The weak chain rule (4.115) allows a “step-by-step” verification of a joint lower **min-entropy** bound for random variables $X_1, \dots, X_k$. Unless otherwise stated, in this document the definition (4.110) of (worst case) conditional **min-entropy** $H_{\min}(\cdot | \cdot)$ is used when we speak of conditional **min-entropy**.

Note 1: If, for example, (long) sequences of **stationarily distributed random variables** $X_1, X_2, \dots, X_m$ are considered, the gain of **min-entropy** per random variable X_j does not significantly depend on the applied definition of conditional **min-entropy**, $H_{\min}(\cdot | \cdot)$ or $H_{\min(av)}(\cdot | \cdot)$. Unless m is too small, the difference between both definitions marginalizes to a “boundary effect”; cf. pars. 638 to 640.

Note 2: If only two variables X and Y (or more generally, a small number of random variables) are considered, it can make a big difference whether $H_{\min}(\cdot | \cdot)$ or $H_{\min(av)}(\cdot | \cdot)$ is applied.

628 Par. 628 provides a useful inequality for the lower bound in (4.109) for the special case $m = 2$. Elementary calculations yield

$$\begin{aligned} H_{\min}(X_2 | X_1 = \omega_1) &= -\log_2 \left(\max_{\omega_2 \in \Omega_2} \text{Prob}(X_2 = \omega_2 | X_1 = \omega_1) \right) = \\ &= -\log_2 \left(\max_{\omega_2 \in \Omega_2} \frac{\text{Prob}(X_2 = \omega_2, X_1 = \omega_1)}{\text{Prob}(X_1 = \omega_1)} \right) \geq \\ &= -\log_2 \left(\frac{\max_{\omega_2 \in \Omega_2, \omega_1 \in \Omega_1} \text{Prob}(X_2 = \omega_2, X_1 = \omega_1)}{\min_{\omega_1 \in \Omega_1} \text{Prob}(X_1 = \omega_1)} \right) = \\ &= H_{\min}(X_2, X_1) + \log_2 \left(\min_{\omega_1 \in \Omega_1} \text{Prob}(X_1 = \omega_1) \right) \\ &\quad \text{for all } \omega_1 \in \Omega_1 \text{ with } \text{Prob}(X_1 = \omega_1) > 0. \end{aligned} \quad (4.116)$$

Special case: X_1 is **uniformly distributed** on Ω_1 . Then

$$H_{\min}(X_2 | X_1 = \omega_1) \geq H_{\min}(X_2, X_1) - \log_2(|\Omega_1|) = H_{\min}(X_2, X_1) - H_{\min}(X_1). \quad (4.117)$$

629 Assume that the binary-valued **random variable** X is $B(1, 0.5 + 0.5\epsilon)$ -distributed. If $|\epsilon|$ is small, the Taylor expansion of the natural logarithm $\log(\cdot)$ yields

$$\log_2(0.5 \pm 0.5\epsilon) = \log_2(0.5(1 \pm \epsilon)) = \log_2(0.5) + \frac{\log(1 \pm \epsilon)}{\log(2)} = -1 + \frac{\pm\epsilon - 0.5\epsilon^2 + O(\epsilon^3)}{\log(2)}. \quad (4.118)$$

Elementary, but careful computations show

$$H_{\min}(X) = 1 - \frac{\epsilon - 0.5\epsilon^2}{\log(2)} + O(\epsilon^3) \quad \text{and} \quad H(X) = 1 - \frac{0.5\epsilon^2}{\log(2)} + O(\epsilon^3). \quad (4.119)$$

630

[Markov chain, **min-entropy**] Assume that $X_1, X_2, \dots$ defines a homogeneous ergodic Markov chain on the finite state space Ω with transition matrix P and limiting distribution ν . If the distribution of X_{n+1} has (at least almost) reached equilibrium ν , then the joint **min-entropy** is given by

$$H_{\min}(X_{n+1}, \dots, X_{n+m}) = -\log_2 \left(\max \left\{ \nu(\omega) p_{\omega, \omega_{n+2}} \cdots p_{\omega_{n+m-1}, \omega_{n+m}} \mid \omega, \omega_{n+2}, \dots, \omega_{n+m} \in \Omega \right\} \right). \quad (4.120)$$

The **min-entropy** per random number equals

$$\frac{H_{\min}(X_{n+1}, \dots, X_{n+m})}{m}. \quad (4.121)$$

[Markov chain, **min-entropy**] Par. 630 considers the case where X_n is unknown while in par. 632 X_n is assumed to be known. In our context, the scenario considered in par. 632 is important.

[Markov chain, **min-entropy**] The *conditional average min-entropy* of m consecutive **random variables** is given by

$$\begin{aligned} H_{\min(av)}(X_{n+1}, \dots, X_{n+m} \mid X_n, \dots, X_{n-k}) &= H_{\min(av)}(X_{n+1}, \dots, X_{n+m} \mid X_n) = \\ &= \sum_{\omega \in \Omega} \nu(\omega) H_{\min}(X_{n+1}, \dots, X_{n+m} \mid X_n = \omega) = \\ &= -\log_2 \left(\sum_{\omega \in \Omega} \nu(\omega) \max \left\{ p_{\omega, \omega_{n+1}} \cdots p_{\omega_{n+m-1}, \omega_{n+m}} \mid \omega_{n+1}, \dots, \omega_{n+m} \in \Omega \right\} \right) = \\ &= -\log_2 \left(\sum_{\omega \in \Omega} \nu(\omega) 2^{-H_{\min}(X_{n+1}, \dots, X_{n+m} \mid X_n = \omega)} \right). \end{aligned} \quad (4.122)$$

The first equation follows from the Markov property. The *average gain* of **min-entropy** per random number (for randomly selected x_n) equals

$$\frac{H_{\min(av)}(X_{n+1}, \dots, X_{n+m} \mid X_n)}{m}. \quad (4.123)$$

[Markov chain, **min-entropy**] Analogously to (4.122), we obtain a formula for the conditional worst case **min-entropy**

$$\begin{aligned} H_{\min}(X_{n+1}, \dots, X_{n+m} \mid X_n, \dots, X_{n-k}) &= H_{\min(av)}(X_{n+1}, \dots, X_{n+m} \mid X_n) = \\ &= -\log_2 \left(\max \left\{ p_{\omega, \omega_{n+1}} \cdots p_{\omega_{n+m-1}, \omega_{n+m}} \mid \omega, \omega_{n+1}, \dots, \omega_{n+m} \in \Omega \right\} \right). \end{aligned} \quad (4.124)$$

[Markov chain] Special cases:

(i) $|\Omega| = 2$: Then the **random variables** $X_1, X_2, \dots$ quantify the distribution of random bits (e.g., of **raw random numbers**).

(ii) $X_1, X_2, \dots$ forms a homogeneous k -step Markov chain. Then the random vectors $\vec{Y}_1, \vec{Y}_2, \dots$, given by $\vec{Y}_j = (X_j, \dots, X_{j+k-1})^t$, form a homogeneous Markov chain on the state space Ω^k .

[Markov chain, $|\Omega| = 2$] The next goal is to develop easy-to-use formulae for (4.122) (and thus for (4.123)) for arbitrary $m \geq 1$. After re-ordering, any product $p_{\omega, \omega_1} \cdots p_{\omega_{m-1}, \omega_m}$ of transition probabilities in (4.122) is of the form to $p_{00}^a p_{01}^b p_{10}^c p_{11}^d$ with integers $a, b, c, d \geq 0$ such that $a + b + c + d = m$. Our task is to determine the maximum for both $\omega = 0$ and $\omega = 1$. This requires case distinctions for the parameters $p_{00}, p_{01}, p_{10}, p_{11}$, or more precisely, only for p_{01}, p_{10} because $p_{00} = 1 - p_{01}$ and $p_{11} = 1 - p_{10}$.

636 [Markov chain, $|\Omega| = 2$, ctd.] To simplify the notation, we introduce the following definition

$$\max_{P,2}(\omega, m) := \max \{p_{\omega, \omega_1} \cdots p_{\omega_{m-1}, \omega_m} \mid \omega, \omega_1, \dots, \omega_m \in \Omega\}. \quad (4.125)$$

We extend (4.125) to the case $m = 0$ by $\max_{P,2}(\omega, 0) := 1$. Below, we assume that $m \geq 1$ and $p_{01} \leq p_{10}$. The case $p_{01} \geq p_{10}$ can be handled analogously; concrete formulae can be derived without further work by relabeling the state space $\Omega = \{0, 1\}$.

- Case I: $p_{01} \leq p_{10} \leq 0.5$. Thus, $p_{01} \leq p_{10} \leq 0.5 \leq p_{11} \leq p_{00}$, and

$$\max_{P,2}(0, m) := p_{00}^m, \quad \max_{P,2}(1, m) = \max\{p_{10}p_{00}^{m-1}, p_{11}^m\}. \quad (4.126)$$

- Case II: $0.5 \leq p_{01} \leq p_{10}$. Thus, $p_{11} \leq p_{00} \leq 0.5 \leq p_{01} \leq p_{10}$, and

$$\max_{P,2}(0, m) := \begin{cases} (p_{01}p_{10})^{m/2} & \text{for even } m \\ (p_{01}p_{10})^{(m-1)/2} p_{01} & \text{for odd } m \end{cases} \quad (4.127)$$

$$\max_{P,2}(1, m) := \begin{cases} (p_{10}p_{01})^{m/2} & \text{for even } m \\ (p_{10}p_{01})^{(m-1)/2} p_{10} & \text{for odd } m \end{cases} \quad (4.128)$$

- Case III: $p_{01} \leq 0.5 \leq p_{10}$. Hence, $p_{01}, p_{11} \leq 0.5 \leq p_{10}, p_{00}$. We distinguish two subcases:

- Subcase III₁: $p_{00} \geq p_{10}$. Thus $p_{01} \leq p_{11} \leq 0.5 \leq p_{10} \leq p_{00}$. Then

$$\max_{P,2}(0, m) := p_{00}^m, \quad \max_{P,2}(1, m) = p_{10}p_{00}^{m-1} \quad (4.129)$$

- Subcase III₂: $p_{00} \leq p_{10}$. Thus, $p_{11} \leq p_{01} \leq 0.5 \leq p_{00} \leq p_{10}$. Then

$$\max_{P,2}(0, m) := \begin{cases} \max \{p_{00}^m, (p_{01}p_{10})^{m/2}\} & \text{for even } m \\ \max \{p_{00}^m, (p_{01}p_{10})^{(m-1)/2} p_{00}\} & \text{for odd } m \end{cases} \quad (4.130)$$

$$\max_{P,2}(1, m) := \begin{cases} \max \{p_{10}p_{00}^{m-1}, (p_{10}p_{01})^{m/2}\} & \text{for even } m \\ \max \{p_{10}p_{00}^{m-1}, (p_{10}p_{01})^{(m-1)/2} p_{10}\} & \text{for odd } m \end{cases} \quad (4.131)$$

637 [Markov chain, $|\Omega| = 2$, ctd.] The results from par. 636 allow the simplification of the formulae (4.121), (4.123) and (4.124). The average gain of **min-entropy** per random bit of $X_{n+1}, \dots, X_{n+m}$ equals

$$\frac{H_{\min}(X_{n+1}, \dots, X_{n+m})}{m} = \frac{-\log_2(\max\{\nu(0) \max_{P,2}(0, m-1), \nu(1) \max_{P,2}(1, m-1)\})}{m}. \quad (4.132)$$

Similarly,

$$\frac{H_{\min(av)}(X_{n+1}, \dots, X_{n+m} \mid X_n)}{m} = \frac{-\log_2(\sum_{\omega \in \Omega} \nu(\omega) \max_{P,2}(\omega, m))}{m} \quad (4.133)$$

and

$$\frac{H_{\min}(X_{n+1}, \dots, X_{n+m} \mid X_n)}{m} = \frac{-\log_2(\max\{\max_{P,2}(0, m), \max_{P,2}(1, m)\})}{m}. \quad (4.134)$$

[Markov chain, $|\Omega| = 2$, ctd.] Par. 637 provides manageable formulae to determine the **min-entropy** per bit for Markov chains on the state space $\Omega = \{0, 1\}$. Their derivation requires careful considerations with case distinctions. It is obvious that for 2-step Markov chains, the necessary efforts increase significantly. In our usual context (**raw random numbers** generated by **PTRNGs**), one usually is interested in the average **entropy** within long sequences of **raw random numbers** or **internal random numbers**. Then we may apply the simpler formula (4.135), neglecting complicating “boundary effects”, which can play a role for small m . In fact, the functional equation of the logarithm function, $\log_2(pq) = \log_2(p) + \log_2(q)$, yields

$$\lim_{m \rightarrow \infty} \frac{H_{\min}(X_{n+1}, \dots, X_{n+m})}{m} = -\max\{\log_2(p_{00}), \log_2(p_{11}), 0.5 \log_2(p_{01}p_{10})\}. \quad (4.135)$$

[Markov chain] Consider a Markov chain on a finite state space Ω with transition matrix P . We call $(\omega_1, \dots, \omega_\ell, \omega_{\ell+1})$ a loop if $\omega_1 = \omega_{\ell+1}$ while $\omega_1, \dots, \omega_\ell$ are mutually distinct. Here, ℓ denotes the length of the loop. Formula (4.136) equals Theorem 2 in [ASPB+18].

$$\lim_{m \rightarrow \infty} \frac{H_{\min}(X_{n+1}, \dots, X_{n+m})}{m} = \min_{\ell} \min_{(\omega_1, \dots, \omega_\ell, \omega_{\ell+1}) \in \mathcal{C}_\ell} \frac{1}{\ell} \sum_{j=1}^{\ell} \log_2 \left(\frac{1}{p_{\omega_j \omega_{j+1}}} \right). \quad (4.136)$$

Here, $\mathcal{C}_\ell$ denotes the set of all loops of length ℓ .

Note: For $\Omega = \{0, 1\}$, there are two loops of length 1 (namely, $(0, 0), (1, 1)$) and two loops of length 2 (namely, $(0, 1, 0), (1, 0, 1)$). Substituting into (4.136) yields (4.135).

[Markov chain, ctd.] If $X_0, X_1, \dots$ forms a homogeneous k -step Markov chain on Ω , then $\vec{Y}_0 = (X_0, X_1, \dots, X_{k-1})$, $\vec{Y}_1 := (X_1, X_2, \dots, X_k)$ is a homogeneous 1-step Markov chain on the product state space Ω^k . In particular, (4.124) and (4.136) can be applied to the Markov chain $\vec{Y}_0, \vec{Y}_1, \dots$

[universal families of hash functions] The next paragraphs treat universal families of hash functions. Pars. 642 to 648 provide central definitions and results from [Shou08], Chapter 8, and immediate conclusions thereof. The definitions, or more precisely the used letters, are adapted to our conventions.

[universal families of hash functions] Let K, S , and T denote non-empty finite sets and $\mathcal{H} = \{h_k : S \rightarrow T \mid k \in K\}$. The elements of K are called *keys* while the functions h_k are called *hash functions*. Furthermore, let Z denote a **uniformly distributed random variable** over K . In particular, $h_Z(s) = h_k(s)$ if the **random variable** Z assumes the value k , and $h_Z(s)$ is a **random variable** assuming values in T for each fixed $s \in S$.

Note: The hash functions mentioned here are not cryptographic hash functions! Likewise, the so-called keys are not keys in a cryptographic sense but essentially indices.

- 643 [universal families of hash functions] We say that the family of hash functions $\mathcal{H}$ is *pairwise independent* if the family of **random variables** $\{h_Z(s) \mid s \in S\}$ is pairwise independent, and if the **random variables** $h_Z(s)$ are **uniformly distributed** on T for all $s \in S$. We say that $\mathcal{H}$ is *universal* if

$$\text{Prob}(h_Z(s) = h_Z(s')) \leq \frac{1}{|T|} \quad \text{for all } s, s' \in S \text{ with } s \neq s'. \quad (4.137)$$

Note: If a family of hash functions $\mathcal{H}$ is pairwise independent, $\mathcal{H}$ is universal.

- 644 [universal families of hash functions] We say that $\mathcal{H}$ is ϵ -almost universal if

$$\text{Prob}(h_Z(s) = h_Z(s')) \leq \epsilon \quad \text{for all } s, s' \in S \text{ with } s \neq s'. \quad (4.138)$$

Note 1: The family $\mathcal{H}$ is universal iff it is $\frac{1}{|T|}$ -almost universal.

- 645 [universal families of hash functions] (Leftover hash lemma; cf. [Shou08], Theorem 8.37). Let $\mathcal{H}$ be a $\frac{1+\alpha}{|T|}$ -almost universal family of hash functions from S to T . Let the **random variables** Z and X be independent where Z is **uniformly distributed** over the keys (index set) K while X assumes values in S . Furthermore, β denotes the collision probability of X , i.e., $\beta = \sum_{s \in S} \text{Prob}(X = s)^2$, and the independent **random variables** U_K and U_T are **uniformly distributed** on K and T , respectively. Inequality (4.139) provides an upper bound for the variation distance of $(Z, h_Z(X))$ from the uniform distribution on $K \times T$.

$$\delta((Z, h_Z(X)), (U_K, U_T)) \leq 0.5 \cdot \sqrt{|T|\beta + \alpha}. \quad (4.139)$$

- 646 [universal families of hash functions] We rewrite (4.139) and adapt it to our needs. Since Z and X are assumed to be independent

$$\begin{aligned} \text{Prob}((Z, h_Z(X)) = (k, t)) &= \text{Prob}(Z = k) \cdot \text{Prob}(h_Z(X) = t \mid Z = k) = \\ &= \text{Prob}(Z = k) \cdot \text{Prob}(h_k(X) = t) \quad \text{for each } (k, t) \in K \times T. \end{aligned} \quad (4.140)$$

Furthermore, by (4.93) and (4.140)

$$\begin{aligned} \delta((Z, h_Z(X)), (U_K, U_T)) &= 0.5 \cdot \|(Z, h_Z(X)) - (U_K, U_T)\|_1 = \\ &= 0.5 \cdot \sum_{k \in K} \sum_{t \in T} |\text{Prob}((Z, h_Z(X)) = (k, t)) - \text{Prob}((U_K, U_T) = (k, t))| \\ &= 0.5 \cdot \sum_{k \in K} \sum_{t \in T} |\text{Prob}(Z = k) \cdot \text{Prob}(h_k(X) = t) - \text{Prob}(U_K = k) \cdot \text{Prob}(U_T = t)| = \\ &= 0.5 \cdot \sum_{k \in K} \sum_{t \in T} \left| \frac{1}{|K|} \cdot \text{Prob}(h_k(X) = t) - \frac{1}{|K|} \cdot \text{Prob}(U_T = t) \right| = \\ &= \frac{1}{|K|} \cdot 0.5 \cdot \sum_{k \in K} \sum_{t \in T} |\text{Prob}(h_k(X) = t) - \text{Prob}(U_T = t)| = \\ &= \frac{1}{|K|} \cdot \sum_{k \in K} 0.5 \cdot \|h_k(X) - U_T\|_1 = \frac{1}{|K|} \cdot \sum_{k \in K} \delta(h_k(X), U_T). \end{aligned} \quad (4.141)$$

Or, equivalently,

$$\delta((Z, h_Z(X)), (U_K, U_T)) = E_Z(\delta(h_Z(X), U_T)) . \quad (4.142)$$

The right-hand side of (4.142) is the expectation of $\delta(h_Z(X), U_T)$ with respect to the uniform distribution on the index set K .

Note: In other words: The inequality (4.139) holds on average but (depending on the distribution of X) can be violated for particular hash functions h_k . This observation is important for cryptographic applications; cf. Subsection 5.1.4.

[universal families of hash functions] From (4.139) and (4.142), we obtain

647

$$E_Z(\delta(h_Z(X), U_T)) \leq 0.5 \cdot \sqrt{|T| \cdot 2^{-H_2(X)} + \alpha} \leq 0.5 \cdot \sqrt{|T| \cdot 2^{-H_{\min}(X)} + \alpha} . \quad (4.143)$$

The **min-entropy** $H_{\min}(h_k(X))$ depends on the key (index) $k \in K$ and on the distribution η of X . Let η be fixed. Inequality (4.143) implies

$$\begin{aligned} E_Z(H_{\min}(h_Z(X))) &\geq E_Z\left(-\log_2\left(\frac{1}{|T|} + \delta(h_Z(X), U_T)\right)\right) \geq \\ &-\log_2\left(\frac{1}{|T|} + E_Z(\delta(h_Z(X), U_T))\right) \geq \\ &-\log_2\left(\frac{1}{|T|} + 0.5 \cdot \sqrt{|T| \cdot 2^{-H_2(X)} + \alpha}\right) \geq \\ &-\log_2\left(\frac{1}{|T|} + 0.5 \cdot \sqrt{|T| \cdot 2^{-H_{\min}(X)} + \alpha}\right) . \end{aligned} \quad (4.144)$$

The first inequality follows from (4.97). Since the mapping $y \mapsto -\log_2(y)$ is convex, the second inequality follows from Jensen's inequality.

Note 1: Inequality (4.144) is rather coarse. This is due to the first step, which considers the worst case for each $z \in K$. Depending on the specific scenario, it may be possible to improve the lower bound in (4.144). Presumably, this requires knowledge about $\mathcal{H}$ and (at least partial) knowledge about the distribution of X . Note 2: For $\alpha = 0$, neglecting the term $\frac{1}{|T|}$ in the last line of (4.144) provides a slightly smaller but more intuitive lower average **min-entropy** bound.

$$E_Z(H_{\min}(h_Z(X))) \geq -\log_2\left(0.5 \cdot \sqrt{|T| \cdot 2^{-H_{\min}(X)}}\right) = 0.5H_{\min}(X) + 1 - 0.5\log_2(|T|) . \quad (4.145)$$

[universal families of hash functions] Example ([Shou08], Example 8.42, slightly modified and extended): Here $S = \{0, 1\}^{1000}$, $T = \{0, 1\}^{64}$, and $\mathcal{H}$ is a universal family of hash functions from S to T . Furthermore, it is assumed that the **random variables** Z and X are independent, $H_{\min}(X) \geq 160$, and that Z is **uniformly distributed** on the finite set K . Then $H_2(X) \geq 160$, and thus $\beta \leq 2^{-160}$. Substituting these values into (4.139) (with $\alpha = 0$) gives

648

$$\delta((Z, h_Z(X)), (U_R, U_T)) \leq 0.5 \cdot \sqrt{2^{64} \cdot 2^{-160} + 0} = 2^{-49} . \quad (4.146)$$

By (4.144), the **min-entropy** of the **internal random numbers** satisfies

$$E_Z(H_{\min}(h_Z(X))) \geq -\log_2(2^{-64} + 2^{-49}) \approx 49 . \quad (4.147)$$

Of course, with regard to functionality class PTG.2, the lower bound 49 is much too small for 64-bit **internal random numbers**.

[universal families of hash functions] [hint: par. 648] So far, we assumed that the indices (keys) of the hash functions are drawn uniformly from the set K , described by a random variable Z that is uniformly distributed on K . Below, we assume that the selection of the indices is described by an (arbitrarily distributed) random variable Z' on K . We first note that

$$E_{Z'}(\delta(h_{Z'}(X), U_T)) \leq 2^{\log_2(|K|) - H_{\min}(Z')} \cdot E_Z(\delta(h_Z(X), U_T)) . \quad (4.148)$$

This can be seen as follows.

$$E_{Z'}(\delta(h_{Z'}(X), U_T)) = \sum_{k \in K} \text{Prob}(Z' = k) \delta(h_k(X), U_T) \leq \sum_{k \in K} 2^{-H_{\min}(Z')} \delta(h_k(X), U_T) = (4.149)$$

$$2^{\log_2(|K|) - H_{\min}(Z')} \sum_{k \in K} 2^{-\log_2(|K|)} \delta(h_k(X), U_T) = 2^{\log_2(|K|) - H_{\min}(Z')} \cdot E_Z(\delta(h_Z(X), U_T)) (4.150)$$

Note 1: The proof of (4.148) is the same as the proof of Theorem 6 in [HaTs15] apart from the fact that there the L^1 -norm is considered.

Note 2: If Z' is uniformly distributed on K then $2^{\log_2(|K|) - H_{\min}(Z')} = 1$.

650 [universal families of hash functions] (generalized leftover hash lemma; cf. par. 645). Substituting (4.148) into (4.143) yields the more general inequalities, which holds for any distribution of Z' .

$$\begin{aligned} E_{Z'}(\delta(h_{Z'}(X), U_T)) &\leq 0.5 \cdot 2^{\log_2(|K|) - H_{\min}(Z')} \cdot \sqrt{|T| \cdot 2^{-H_2(X)} + \alpha} \leq \\ &0.5 \cdot 2^{\log_2(|K|) - H_{\min}(Z')} \cdot \sqrt{|T| \cdot 2^{-H_{\min}(X)} + \alpha} . \end{aligned} \quad (4.151)$$

651 [universal families of hash functions] The first and the second inequality of (4.152) follow from (4.97) and Jensen's inequality. The remaining inequalities follow by (4.151).

$$\begin{aligned} E_{Z'}(H_{\min}(h_{Z'}(X))) &\geq E_{Z'}\left(-\log_2\left(\frac{1}{|T|} + \delta(h_{Z'}(X), U_T)\right)\right) \geq \\ &-\log_2\left(\frac{1}{|T|} + E_{Z'}(\delta(h_{Z'}(X), U_T))\right) \geq \\ &-\log_2\left(\frac{1}{|T|} + 2^{\log_2(|K|) - H_{\min}(Z') - 1} \cdot \sqrt{|T| \cdot 2^{-H_2(X)} + \alpha}\right) \geq \\ &-\log_2\left(\frac{1}{|T|} + 2^{\log_2(|K|) - H_{\min}(Z') - 1} \cdot \sqrt{|T| \cdot 2^{-H_{\min}(X)} + \alpha}\right) . \end{aligned} \quad (4.152)$$

Inequality (4.152) generalize (4.144) to non-uniformly distributed random variables Z' . Loosely speaking, the term $2^{\log_2(|K|) - H_{\min}(Z')}$ quantifies the “penalty”. If Z' is uniformly distributed on K , (4.152) coincides with (4.144).

Note: For $\alpha = 0$, neglecting the term $\frac{1}{|T|}$ in the last line of (4.152) provides a slightly smaller but more intuitive lower average **min-entropy** bound; cf. par 647.

$$\begin{aligned} E_{Z'}(H_{\min}(h_{Z'}(X))) &\geq -\log_2\left(2^{\log_2(|K|) - H_{\min}(Z') - 1} \cdot \sqrt{|T| \cdot 2^{-H_{\min}(X)}}\right) = \\ &H_{\min}(Z') + 0.5H_{\min}(X) + 1 - \log_2(|K|) - 0.5\log_2(|T|) . \end{aligned} \quad (4.153)$$

652

[universal families of hash functions] Example: In this paragraph we fix the output length to $m = 512$ bits. The aim is to determine the minimum bit length n of the input data of the hash functions for which (4.152) ensures that $H_{\min}(h_{Z'}(X))$ is at least $0.99m$ on average. More formally, $S = \{0, 1\}^n$ and $T = \{0, 1\}^m$. We assume that the universal family of hash functions is given by $\mathcal{H} = \{h: \text{GF}(2)^n \rightarrow \text{GF}(2)^m \mid h(x) = Tx\}$ where T denotes a Toeplitz matrix. In particular, $|K| = 2^{n+m-1} = 2^{n+m-1}$, $\alpha = 0$, and (4.152) simplifies to

$$E_{Z'}(H_{\min}(h_{Z'}(X))) \geq -\log_2 \left(2^{-m} + 2^{(n+m-1)-H_{\min}(Z')-1} \cdot \sqrt{2^m \cdot 2^{-H_{\min}(X)}} \right) = \\ -\log_2 \left(2^{-m} + 2^{\frac{m}{2}-1} \cdot 2^{(n+m-1)\left(1-\frac{H_{\min}(Z')}{n+m-1}\right)-n\frac{H_{\min}(X)}{2n}} \right). \quad (4.154)$$

It is easy to verify that the right-hand side of (4.154) is strictly monotonically increasing in n iff $\frac{H_{\min}(Z')}{n+m-1} + \frac{H_{\min}(X)}{2n} > 1$. We consider seven sets of parameters. Recall that $m = 512$ is fixed and that the entropy bound $0.99m$ shall be reached on average: Below, for several parameter sets the minimal values for n are given that are necessary to ensure this **entropy** bound.

- a) $\frac{H_{\min}(Z')}{n+m-1} = 1$ (ideal RNG) and $\frac{H_{\min}(X)}{n} = 1$ (ideal RNG). Then $n \geq 1524$.
- b) $\frac{H_{\min}(Z')}{n+m-1} = 1$ (ideal RNG) and $\frac{H_{\min}(X)}{n} = 0.9$. Then $n \geq 1694$.
- c) $\frac{H_{\min}(Z')}{n+m-1} = 1$ (ideal RNG) and $\frac{H_{\min}(X)}{n} = 0.8$. Then $n \geq 1905$.
- d) $\frac{H_{\min}(Z')}{n+m-1} = 1$ (ideal RNG) and $\frac{H_{\min}(X)}{n} = 0.7$. Then $n \geq 2177$.
- e) $\frac{H_{\min}(Z')}{n+m-1} = 0.9$ and $\frac{H_{\min}(X)}{n} = 0.9$. Then $n \geq 2323$.
- f) $\frac{H_{\min}(Z')}{n+m-1} = 0.8$ and $\frac{H_{\min}(X)}{n} = 0.8$. Then $n \geq 4321$.
- g) $\frac{H_{\min}(Z')}{n+m-1} = 0.7$ and $\frac{H_{\min}(X)}{n} = 0.7$. Then $n \geq 18305$.

Note: In the scenarios e) – g) it is tacitly assumed that the random numbers used to select the hash function and the input data are generated by the same **PTRNG**, which should be typical. The above condition implies that increasing the input length n is only successful iff $\frac{H_{\min}(Z')}{n+2m-1}, \frac{H_{\min}(X)}{n} > \frac{2}{3}$.

4.3.2 Guesswork and Work Factor

Although guesswork and work factor do not appear in the definition of the functionality classes, we briefly explain these concepts. 653

As in Subsection 4.3.1, X denotes a **random variable** that assumes values in $\Omega = \{\omega_1, \omega_2, \dots, \omega_k\}$. W.l.o.g. we may assume that 654

$$\text{Prob}(X = \omega_1) \geq \text{Prob}(X = \omega_2) \geq \dots \geq \text{Prob}(X = \omega_k). \quad (4.155)$$

A reasonable goal is to estimate the effort to guess the outcome of an experiment that is viewed as a realization of X .

If the **random variable** X has distribution τ , then $\tau(\omega_j) := \text{Prob}(X = \omega_j)$. Recall the notation $\tau := (\tau(\omega_1), \dots, \tau(\omega_k))$. 655

The λ -work-factor $w_\lambda(X)$ denotes the minimum number of guesses to get the correct result with probability $\geq \lambda$ (with $0 < \lambda < 1$) if the optimal guessing strategy (beginning with $\omega_1, \omega_2, \dots$) is applied. That is,

$$w_\lambda(X) = \min \left\{ \ell \mid \sum_{i=1}^{\ell} \tau(\omega_i) \geq \lambda \right\}. \quad (4.156)$$

657 The *guesswork* $W(X)$ denotes the expected number of guesses until a success if the optimal guessing strategy is applied

$$W(X) = \sum_{i=1}^k i \tau(\omega_i). \quad (4.157)$$

658 The guesswork and the λ -work-factor (for a suitable parameter λ) seem to be appropriate criteria to assess the strength of a **TRNG** that is used for cryptographic applications. However, in many scenarios of practical relevance, it can be very difficult to sort the probabilities of the admissible outcomes in descending order as in (4.155), in particular if X is a random vector $(X_1, \dots, X_m)$ with dependent components. Usually, the calculation of the **entropy** (or at least the determination of a useful lower bound) is easier. In the next paragraphs, we explain the relation between **entropy**, guesswork, and work factor.

659 For $\lambda = 0.5$, the work factor of the optimal guessing strategy satisfies the following inequality [Plia99]

$$\left\lfloor \frac{1}{2 \max \{ \tau(x_j) \mid 1 \leq j \leq k \}} \right\rfloor \leq w_{\frac{1}{2}}(X) \leq \lceil (1 - \delta(\tau, \nu) \cdot k) \rceil. \quad (4.158)$$

As above, $\nu = (\frac{1}{k}, \dots, \frac{1}{k})$ denotes the uniform distribution on Ω .

660 For the most general case, the following inequality provides tight bounds for the guesswork

$$\frac{k}{2} \delta(\tau, \nu) \leq \frac{k-1}{2} - W(X) \leq k \delta(\tau, \nu). \quad (4.159)$$

661 A memoryless binary-valued **stationary** random source can be described by independent identically $B(1, p)$ -distributed **random variables** $X_1, X_2, \dots, X_n$. For large n , the guesswork $w_{\frac{1}{2}}((X_1, \dots, X_n))$ for n random bits, or equivalently for the random vector $\vec{X} = (X_1, \dots, X_n)$, can be estimated by the **Shannon entropy** [Maur92]

$$\log_2 w_{\frac{1}{2}}(\vec{X}) \approx n \cdot H(X_1). \quad (4.160)$$

662 More generally, for an ergodic **stationary** binary random source, the ratio between the guesswork and the length of a sequence tends asymptotically to **Shannon entropy** [Maur92]

$$\lim_{n \rightarrow \infty} \frac{\log_2 w_\alpha(\vec{X})}{n} = H(X_1), \text{ for } 0 < \alpha < 1. \quad (4.161)$$

[Example] Let τ denotes the probability distribution on $\Omega = \{0, 1\}^{128}$ for which $\tau((0, \dots, 0)) = 0.5$, $\tau((1, \dots, 1)) = 2^{-128}$, and $\tau(\omega) = 2^{-129}$ else. Then $H_{\min}(X) = 1$, whereas

$$H(X) = - (0.5 \log_2(0.5) + 2^{-128} \log_2(2^{-128}) + (2^{128} - 2) \cdot 2^{-129} (\log_2(2^{-129}))) = (4.162) \\ 0.5 + 2^{-128} \cdot 128 + (2^{128} - 2) \cdot 2^{-129} \cdot 129 = 0.5 + 2^{-121} + 64.5 - 2^{-128} \approx 65.$$

For $\lambda \leq 0.5$, we have $w_\lambda(X) = 1$, which would be disastrous for cryptographic applications. On the other hand,

$$W(X) = 1 \cdot 0.5 + 2 \cdot 2^{-128} + \sum_{i=3}^{2^{128}} i \cdot 2^{-129} = 0.5 + 2^{-127} + \left(\frac{2^{128}(2^{128} + 1)}{2} - 3 \right) 2^{-129} = \\ 2^{126} + 0.75 + 2^{-129} \approx 2^{126}. \quad (4.163)$$

is rather large.

Note: The guesswork for the uniform distribution is $W(X) = 2^{127} + 0.5 \approx 2^{127}$.

[Example] Let τ denote the probability distribution on $\Omega = \{0, 1\}^{128}$ for which $\tau((0, \dots, 0)) = 2^{-127}$, $\tau((1, \dots, 1)) = 0$, and $\tau(\omega) = 2^{-128}$ else. Then $H_{\min}(\tau) = 127$, while 664

$$H(X) = - (2^{-127} \log_2(2^{-127}) + 0 + (2^{128} - 2) \cdot 2^{-128} (\log_2(2^{-128}))) = (4.164) \\ 2^{-127} \cdot 127 + (2^{128} - 2) \cdot 2^{-128} \cdot 128 = 128 - 2^{-127} \approx 128.$$

For $\lambda = 2^{-127}$, we have $w_\lambda(X) = 1$ (in place of $= 2$ for the uniform distribution), but for $\lambda = 2^{-100}$, for instance, $w_\lambda(X) = 2^{28} - 1$ (instead of 2^{28} for the uniform distribution). Furthermore, the guesswork equals

$$W(X) = 1 \cdot 2^{-127} + \sum_{i=2}^{2^{128}-1} i \cdot 2^{-128} = 2^{-127} + \left(\frac{(2^{128} - 1) \cdot 2^{128}}{2} - 1 \right) 2^{-128} = \\ 2^{127} - 0.5 + 2^{-128} \approx 2^{127}, \quad (4.165)$$

which is very close to the guesswork of the uniform distribution.

The example in par. 663 shows that for very unbalanced distributions, **Shannon entropy** and 665 the guesswork may tremendously overestimate the resistance against guessing attacks. On the other hand in the example in par. 664, **Shannon entropy** and the guesswork provide a realistic assessment of the strength against guessing attacks, while the **min-entropy** underestimates this strength unless extremely small parameters λ are considered.

Example: Assume that the **random variables** $X_1, \dots, X_n$ are **iid** $B(1, p)$ -distributed and $\vec{X} = 666 (X_1, \dots, X_n)$. Since the **random variables** X_i are **iid**, we obtain $H_1(\vec{X}) = n \cdot H_1(X_1) = -n \cdot (p \log_2(p) + (1-p) \log_2(1-p))$ and $H_{\min}(\vec{X}) = -\log_2(\max\{p^n, (1-p)^n\}) = -n \log_2(\max\{p, 1-p\})$. If $p \geq 0.5$, the most likely vector is $(1, \dots, 1)$ but $(0, \dots, 0)$ otherwise. Furthermore, $Y = \text{ham}(X_1, \dots, X_n)$, the Hamming weight of the random vector $(X_1, \dots, X_n)$, is $B(n, p)$ -distributed, and $\text{Prob}(Y = y) = \binom{n}{y} p^y \cdot (1-p)^{n-y}$.

Example from par. 666 continued: The work factor $w_\lambda(\vec{X})$ from (4.156) can be efficiently 667 computed because $\text{Prob}(\vec{X} = \vec{x})$ only depends on the Hamming weight of $\vec{x}$. In particular, only

$n+1$ different probabilities occur. W.l.o.g. we may assume that $p \geq 0.5$. At first, for the targeted success probability λ , one determines

$$\alpha(\lambda) := \max \left\{ i \geq 0 \mid \sum_{j=i}^n \binom{n}{j} p^j (1-p)^{n-j} \geq \lambda \right\}. \quad (4.166)$$

Then

$$w_\lambda(\vec{X}) = \sum_{j=\alpha(\lambda)+1}^n \binom{n}{j} + \left\lceil \frac{\lambda - \sum_{j=\alpha(\lambda)+1}^n \binom{n}{j} p^j (1-p)^{n-j}}{p^{\alpha(\lambda)} (1-p)^{n-\alpha(\lambda)}} \right\rceil. \quad (4.167)$$

Unless n or j are rather small, Stirling's approximation formula (4.37) and (4.38) can be applied to compute the factorials and the binomial coefficients.

4.4 Random Mappings

668 Section 4.4 treats random mappings. We focus on aspects that are relevant in the context of AIS 20 and AIS 31. Subsection 4.4.1 summarizes well-known statistical properties for the iteration of random mappings. In Subsection 4.4.2, the impact of randomly selected mappings on the work factor, the Shannon entropy, and, in particular, on the min-entropy is analyzed. Further results on the work factor can be found in the appendix, Section 6.2.

669 The results from this section can be useful for all functionality classes. For DRNGs (functionality classes DRG.2, DRG.3, DRG.4, and DRT.1), this refers to the seeding procedure, the reseeding procedure, and high-entropy additional input. For TRNGs (functionality classes PTG.2, PTG.3, and NTG.1), the results support the analysis of the cryptographic post-processing algorithm or of a post-processing algorithm that uses a cryptographic function (with or without memory). The main area of application is functionality class PTG.3.

670 [Notation] In this section, A , A_1 , and A_2 denote finite sets and $\mathcal{F}_{A_1, A_2} := \{f' : A_1 \rightarrow A_2\}$. For given sets A_1 and A_2 , a random mapping F is a random variable that assumes values uniformly in a specified subset $V \subseteq \mathcal{F}_{A_1, A_2}$; cf. par. 671 and 678.

Note: Assume that F is a random variable over the probability space $(\Omega, \mathcal{A}, P)$. Then $F(\omega) \in \mathcal{F}_{A_1, A_2}$ for each $\omega \in \Omega$, $F(\omega)(x) \in A_2$, and $F(x)$ is a random variable over $(\Omega, \mathcal{A}, P)$ that assumes values in A_2 .

4.4.1 Iteration of Random Mappings: Statistical Properties

671 In Subsection 4.4.1, $A_1 = A_2 = A$ and F is a random mapping that is uniformly distributed on a subset $V \subseteq \mathcal{F}_{A, A}$. We focus on the special cases $V = \mathcal{F}_{A, A}$ and $V = \text{Perm}_A$, the set of all permutations on A (i.e., the bijective mappings of $\mathcal{F}_{A, A}$).

672 In the context of AIS 20, the results of this subsection may be applicable to the state transition functions of DRNGs. These results are particularly interesting for pure DRNGs. For hybrid

DRNGs, the situation should be more favorable since **additional input** usually causes the **internal state** to “jump” between cycles of the pure version (i.e., without **additional input**) of the DRNG.

For a given $\omega \in \Omega$, the term $F(\omega)$ denotes a fixed mapping in V . We consider the sequence $t_{n+1} := F(\omega)(t_n)$ with $t_0 = t$ for some $t \in A$ and $n \geq 0$. In terms of the functional graph of $F(\omega)$, this sequence $t_0, t_1, \dots$ describes a path in A that ends in a cycle. The functional graph consists of components, each of which consists of one cycle that is connected to several trees (0 trees are possible).

Table 1 collects well-known results on random mappings on $\mathcal{F}_{A,A}$; see, e.g. [Flod89] for details. If $|A| = n$, then $|\mathcal{F}_{A,A}| = n^n$. In Table 1, it is assumed that the mappings are selected randomly. For the last two features, additionally an element of $A_2 = A$ is selected randomly. It should be noted that the asymptotic distributions (for $n \rightarrow \infty$) provide more information than the expected values (expectations) given in Table 1. This is especially the case if the expected value is not “typical”, i.e., if the variance is “large”; an example is the predecessor size. If relevant for the evaluation, we refer the designer/the evaluator to the literature.

Table 2 collects well-known results on random permutations (random bijections) that are chosen uniformly from the set of all $n!$ permutations $V = \text{Perm}_A$ (cf. [Golo64] for details). If $|A| = n$, then $|\text{Perm}_A| = n!$. In Table 2 it is assumed that the permutations are selected randomly. For the last feature, additionally an element of $A_2 = A$ is selected randomly.

4.4.2 Impact on the Work Factor, on **Shannon entropy**, and **min-entropy**

In Subsection 4.4.2, we analyze the impact of randomly selected mappings on the work factor (pars. 686 to 701), **Shannon entropy** (pars. 703 to 708), and **min-entropy** (pars. 709 to 742). The contributions to **Shannon entropy** and in particular to **min-entropy** have the greatest importance in context of the AIS 20/31. Further results on the work factor are outsourced to the appendix, cf. Section 6.2.

In this subsection, A_1, A_2 denote finite sets with cardinality $|A_1| = b_1$ and $|A_2| = b_2$. In particular, $|\mathcal{F}_{A_1, A_2}| = b_2^{b_1}$.

[**random variables**] In this subsection, F denotes a **random variable** that assumes values uniformly in $\mathcal{F}_{A_1, A_2} := \{f' : A_1 \rightarrow A_2\}$. The **random variable** X assumes values in A_1 , and F and X are independent. Unless otherwise stated, in this subsection the random variable X is **uniformly distributed** on A_1 while the random variable X' is allowed to be non-uniformly distributed. Furthermore, U denotes a **random variable** that is **uniformly distributed** on A_2 .

The results of this section are in particular useful for the evaluation of Requirement PTG.3.5. This mainly concerns the impact of data compression but data extension can also be relevant.

For PTG.3-compliant **PTRNGs**, **fresh entropy** is introduced by **intermediate random numbers**, either as **seed material** for **reseeding procedures** or as high-entropy **additional input**. Usually, a PTG.2-compliant **PTRNG** generates n -bit **intermediate random numbers** $x_1, x_2, \dots$ (The n -bit random number x_j can be the concatenation of n random bits.) The **intermediate random num-**

Table 1: Statistics of random mappings on A , $|A| = n$; cf. [Flod89]

Characteristic	Expected value as $n \rightarrow \infty$	Definition and comments
Number of components	$\frac{1}{2} \log(n)$	A component consists of one cycle and several trees connected to this cycle. (0 trees are possible.)
Largest component	$\approx 0.75782n$	
Number of cyclic nodes	$\sqrt{\frac{\pi n}{2}}$	$\sqrt{\frac{\pi n}{2}} \approx 1.253314\sqrt{n}$
Cycle length (μ)	$\sqrt{\frac{\pi n}{8}}$	The number of edges in the cycle is called the cycle length of t , denoted $\mu(t)$, $\sqrt{\frac{\pi n}{8}} \approx 0.626657\sqrt{n}$.
Maximum cycle length	$\approx 0.78248\sqrt{n}$	
Tail length (λ)	$\sqrt{\frac{\pi n}{8}}$	The number of edges in the path to the cycle is called the tail length of t , denoted $\lambda(t)$, $\sqrt{\frac{\pi n}{8}} \approx 0.626657\sqrt{n}$.
Maximum tail length	$\approx 1.73746\sqrt{n}$	
Rho length (ρ)	$\sqrt{\frac{\pi n}{2}}$	$\rho(t) = \lambda(t) + \mu(t)$, number of steps until a node on the path repeats, $\sqrt{\frac{\pi n}{2}} \approx 1.253314\sqrt{n}$.
Maximum rho length	$\approx 2.4119\sqrt{n}$	
Tree size	$\frac{n}{3}$	Tree size of a node t means the size of the maximal tree (rooted to the cycle) containing this node t .
Largest tree	$\approx 0.48n$	
Number of terminal nodes	$e^{-1}n$	Number of nodes without predecessor, $e^{-1}n \approx 0.367879n$.
Number of image points	$(1 - e^{-1})n$	$ f(A) $ = number of nodes that have a predecessor, $(1 - e^{-1})n \approx 0.632121n$.
Number of k -th iterate image points	$(1 - r_k)n$, $r_0 = 0$, $r_{k+1} = e^{-1+r_k}$	$ f^k(A) $ = number of nodes after application of f^k .
Component size	$\frac{2n}{3}$	of a randomly selected point
Predecessor size	$\sqrt{\frac{\pi n}{8}}$	The predecessor size of the node t is the size of the tree rooted at node t , or equivalently, the number of iterated pre-images of t .

Table 2: Statistics of random permutations on A , $|A| = n$, cf. [Golo64; PuWi68]

Characteristic	Expected value as $n \rightarrow \infty$	Distribution as $n \rightarrow \infty$
Number of cycles	$\log(n) + C + o(1)$ $C = 0.57721566 \dots$	Number ω_n of cycles of the permutation $\text{Prob}(\omega_n = k) = \frac{e^{-\frac{(k-\log(n))^2}{2\log(n)}}}{\sqrt{2\pi\log(n)}} (1 + o(1))$
Cycle length	$\frac{n}{\log(n) + C + o(1)}$	Number $\omega_{n\ell}$ of cycle of length ℓ $\text{Prob}(\omega_{n\ell} = k) = P_{1/\ell}(k) = \frac{e^{-\frac{1}{\ell}}}{\ell^k k!}$ (Poisson distribution with parameter $\tau = \frac{1}{\ell}$).
Length of the largest cycle	$\approx 0.6243n$	
Expected cycle length of a random element	$\frac{n+1}{2}$	Probability that a random element x lies on a cycle of size k , $k \leq n$, is $\text{Prob}(\omega(x) = k) = \frac{1}{n}$.

bers $x_1, x_2, \dots$ are interpreted as realizations of random variables $X_1, X_2, \dots$.

Assume that the current intermediate random number x_j is mixed as additional input into S_{req} of the cryptographic post-processing (a DRG.3-compliant DRNG), and finally also into the internal state. If s_j denotes the current internal state of the cryptographic post-processing algorithm in the notation of Chapter 3, the next values in S_{req} and S can be described by the random variables $\phi_{req}(s_j, X_j)$ and $\phi(s_j, X_j)$, respectively. Similarly, the outputted internal random number r_j can be interpreted as a realization of the random variable $\psi(\phi_{req}(s_j, X_j))$. Let the mapping $f_s: A_1 = \{0, 1\}^n \rightarrow A_2 = \{0, 1\}^m$ be given by $f_s(x) := \psi(\phi_{req}(s, x))$. This yields a sequence of (different) functions $f_{s_1}, f_{s_2} \dots \in \mathcal{F}_{A_1, A_2}$.

If the current intermediate random number x_j is used as seed material for a reseeding procedure, the impact on the internal random numbers can analogously be described by mappings $f'_{s_1}, f'_{s_2} \dots \in \mathcal{F}_{A_1, A_2}$.

This observation motivates the study of random mappings. In the following, most of the time uniformly distributed random variables X are considered. However, pars. 722 to 729 transfer results on the min-entropy to non-uniform distributions. 681

In view of functionality class PTG.3, we are interested in the impact of the cryptographic post-processing on the stochastic properties of the internal random numbers. This comprises Shannon entropy and min-entropy. Additionally, we consider the impact on the work factor. 682

For a fixed mapping $f \in \mathcal{F}_{A_1, A_2}$, the term $f(X)$ describes the transformed random variable X . Similarly, $F(X)$ denotes the random variable that is given when a mapping $f \in \mathcal{F}_{A_1, A_2}$ is uniformly selected (modeled by the random variable F) and applied to the random variable X , which models the intermediate random numbers. (The pair of random variables (X, F) assumes values in $(A_1, \mathcal{F}_{A_1, A_2})$ while $F(X)$ assumes values in A_2 .) In particular, $H(F(X))$ and $H_{min}(F(X))$ denote the Shannon entropy and the min-entropy of $F(X)$, respectively. 683

Below, we assume that a mapping from $\mathcal{F}_{A_1, A_2}$ is selected randomly. Many properties of random mappings are “typical” in the sense that they are shared by “nearly all” mappings. This 684

property is important for stateless **post-processing** algorithms (with a fixed cryptographic output function, as permitted by functionality class PTG.2) and for **cryptographic post-processing** algorithms, where the **adversary** knows the complete current **internal state** (which is the most favorable scenario from the **adversary's** point of view). For PTG.3-compliant **PTRNGs**, the output function can be interpreted as a random mapping of the **intermediate random numbers** that is parametrized by the current **internal state** of the **post-processing** algorithm; cf. par. 680. The constantly changing **internal state** acts as an “averaging operator” on $\mathcal{F}_{A_1, A_2}$, compensating the effects of individual “weak” mappings.

685 For functionality class NTG.1, the situation is similar to functionality class PTG.3 in the sense that true random numbers are post-processed. A significant difference is that for NTG.1-compliant **NPTRNGs**, the distribution of the **raw random numbers** usually has only little **entropy** per output bit, which requires a higher compression rate; cf. par. 727.

686 Impact on the work factor

Although the work factor does not appear in the class Requirements of PTG.2 and PTG.3, we at first consider the impact on the work factor. More precisely, we determine the expected (average) work factor of $F(X)$ in A_2 and compare it to the work factor of a **uniformly distributed random variable** U on A_2 .

687 Let $f \in \mathcal{F}_{A_1, A_2}$ be fixed for the moment. For $s \in \mathbb{N}_0$, we introduce the definitions

$$V_{(f)s} := \{z \in A_2 : |f^{-1}(z)| = s\} \quad \text{and} \quad v_{(f)s} := |V_{(f)s}|. \quad (4.168)$$

That is to say, $V_{(f)s}$ denotes the set of elements of A_2 that have exactly s pre-images, and $v_{(f)s}$ quantifies its cardinality.

688 Since X is **uniformly distributed** on A_1 , we have $\text{Prob}(f(X) = z) = \frac{s}{b_1}$ for each $z \in V_{(f)s}$. As an immediate consequence

$$\text{Prob}(f(X) \in V_{(f)s}) = \frac{sv_{(f)s}}{b_1} \quad \text{and} \quad \text{Prob}\left(f(X) \in \bigcup_{s=r}^{b_1} V_{(f)s}\right) = \sum_{s=r}^{b_1} \frac{sv_{(f)s}}{b_1}. \quad (4.169)$$

689 In our context, the values $f(\cdot)$ are random numbers. The best guessing strategy for an **adversary** (without additional knowledge) is to try those $a_2 \in A_2$ first that have the most pre-images under f . Thus, the work factor corresponding to the success probability $\text{Prob}\left(f(X) \in \bigcup_{s=r}^{b_1} V_{(f)s}\right)$ is given by $\sum_{s=r}^{b_1} sv_{(f)s}$, since by assumption, X is **uniformly distributed** on A_1 .

690 [Notation] Subsequently, $E_F(\cdot)$ and $\text{Prob}_X(\cdot)$ denote the expectation with regard to the random mapping F and the probability with regard to the **random variable** X , respectively. In particular, for a mapping $h: \mathcal{F}_{A_1, A_2} \rightarrow \mathbb{R}$, this means

$$E_F(h(F)) = \frac{1}{|\mathcal{F}_{A_1, A_2}|} \sum_{f \in \mathcal{F}_{A_1, A_2}} h(f). \quad (4.170)$$

691

The term

$$e_r := \text{Prob}_X (|F^{-1}(F(X))| \geq r) \quad (4.171)$$

quantifies the average probability that a realization of $F(X)$ has $\geq r$ pre-images.

If $s \neq t$, then $V_{(f)s}$ and $V_{(f)t}$ are disjoint. Hence,

692

$$\begin{aligned} e_r &= E_F \left(\text{Prob}_X \left(F(X) \in \bigcup_{s=r}^{b_1} V_{(F)s} \right) \right) = \sum_{s=r}^{b_1} E_F \left(\text{Prob}_X (F(X) \in V_{(F)s}) \right) = \sum_{s=r}^{b_1} \frac{s E_F(v_{(F)s})}{b_1} \\ &= \sum_{s=r}^{b_1} \frac{s E_F \left(\sum_{z \in A_2} 1_{\{s\}} (|F^{-1}(z)|) \right)}{b_1} = \sum_{s=r}^{b_1} \frac{s \sum_{z \in A_2} \text{Prob}(|F^{-1}(z)| = s)}{b_1} \\ &= \sum_{s=r}^{b_1} \frac{s b_2}{b_1} \binom{b_1}{s} p^s (1-p)^{b_1-s} = \sum_{s=r}^{b_1} \binom{b_1-1}{s-1} p^{s-1} (1-p)^{(b_1-1)-(s-1)} \\ &= \text{Prob}(Y \geq r-1) \end{aligned} \quad (4.172)$$

with $p = \frac{1}{b_2}$, and Y is a $B(b_1-1, p)$ -distributed **random variable**. The third equation results from interchanging the order of integration (with regard to F and X).

The corresponding e_r -work factor (or more precisely, $e_{r(F)}$, averaged over all $f' \in \mathcal{F}_{A_1, A_2}$) equals

693

$$w_{e_r}(F(X)) = E_F \left(\left| \bigcup_{s=r}^{b_1} V_{(F)s} \right| \right) = \sum_{s=r}^{b_1} b_2 \binom{b_1}{s} p^s (1-p)^{b_1-s} = b_2 \text{Prob}(Y' \geq r) \quad (4.173)$$

where Y' denotes a $B(b_1, p)$ -distributed **random variable**.

In particular, $E(Y) = \frac{b_1-1}{b_2}$ and $\text{Var}(Y) = \frac{b_1-1}{b_2} \left(1 - \frac{1}{b_2}\right)$. Similarly, $E(Y') = \frac{b_1}{b_2}$ and $\text{Var}(Y') = \frac{b_1}{b_2} \left(1 - \frac{1}{b_2}\right)$.

694

[Notation] In the remainder of this subsection, we assume

695

$$A_1 = \{0, 1\}^n \quad \text{and} \quad A_2 = \{0, 1\}^m \quad \text{with large } n \text{ and } m. \quad (4.174)$$

Such sets are relevant in the context of cryptographic algorithms. In particular, $b_1 = 2^n$ and $b_2 = 2^m$. To simplify the notation, we define

$$\gamma := 2^{n-m}. \quad (4.175)$$

Below, we distinguish the cases $\gamma \gg 1$ (strong data compression), $\gamma \gtrsim 1$ (moderate data compression), $\gamma = 1$, and $\gamma < 1$ (data expansion).

Note 1: The term “large” in (4.174) is not precise. This document uses results from Section 4.4 in the context of cryptographic mappings (**cryptographic post-processing** algorithms, **post-processings** algorithms that use **cryptographic** functions, **seeding procedures**, **reseeding procedures**). In this context, one may assume that $n, m \geq 128$.

Note 2: The results on random mappings do not apply to **algorithmic post-processing** algorithms because there the parameters (n, m) are too small or because all mappings are linear etc. For

algorithmic post-processing algorithms, the entropy estimation of the internal random numbers shall be based on a stochastic model; cf. functionality class PTG.2.

- 696 Since n, m are assumed to be large, $(b_1 - 1)/b_1 = (2^n - 1)/2^n \approx 1$ and $1 - \frac{1}{b_2} = 1 - 2^{-m} \approx 1$. Using these approximations, we obtain

$$E(Y) = E(Y') = \text{Var}(Y) = \text{Var}(Y') = \gamma. \quad (4.176)$$

In the remainder, we identify the distributions of Y and Y' .

- 697 Let U denote a uniformly distributed random variable on $A_2 = \{0, 1\}^m$. The equations (4.177) and (4.178) provide useful relations

$$\text{Prob} \left(U \in \left(\bigcup_{s=r}^{b_1} V_{(F)s} \right) \right) = \frac{w_{e_r}(F(X))}{b_2} = \text{Prob}(Y' \geq r) = e_{r+1}, \text{ and thus (4.177)}$$

$$w_{e_r}(F(X)) = w_{e_{r+1}}(U) \text{ and } w_{e_{r-1}}(F(X)) = w_{e_r}(U). \quad (4.178)$$

- 698 Equations (4.177) and (4.178) imply (4.179). This term quantifies the *relative work factor defect* between U and $F(X)$ (on the elements of A_2 with pre-image size $\geq r$);

$$\frac{w_{e_r}(U) - w_{e_r}(F(X))}{w_{e_r}(U)} = \frac{w_{e_{r-1}}(F(X)) - w_{e_r}(F(X))}{w_{e_{r-1}}(F(X))} = \frac{e_r - e_{r+1}}{e_r} = \frac{\text{Prob}(Y = r - 1)}{\text{Prob}(Y \geq r - 1)} \quad (4.179)$$

- 699 [Case $\gamma \gg 1$] On average, each $a_2 \in A_2$ has γ pre-images. Unless r is very small or very large compared to γ , the Central Limit Theorem (with correction factor “ ± 0.5 ”, see [Geor15], Korollar (5.23)) equation (4.172) implies

$$e_r = 1 - \Phi \left(\frac{r - 1 - 0.5 - \gamma}{\sqrt{\gamma}} \right) = \Phi \left(\frac{\gamma - r + 1.5}{\sqrt{\gamma}} \right) \quad (4.180)$$

$$w_{e_r}(F(X)) = 2^m \left(1 - \Phi \left(\frac{r - 0.5 - \gamma}{\sqrt{\gamma}} \right) \right) = 2^m \Phi \left(\frac{\gamma - r + 0.5}{\sqrt{\gamma}} \right) \text{ and } \quad (4.181)$$

$$w_{e_r}(U) = 2^m \Phi \left(\frac{\gamma - r + 1.5}{\sqrt{\gamma}} \right). \quad (4.182)$$

- 700 [Case $\gamma \gg 1$] From (4.181) and (4.182) we obtain the work factor defect between U and $F(X)$ (on the elements of A_2 with pre-image size $\geq r$)

$$w_{e_r}(U) - w_{e_r}(F(X)) = 2^m \left(\Phi \left(\frac{\gamma - r + 1.5}{\sqrt{\gamma}} \right) - \Phi \left(\frac{\gamma - r + 0.5}{\sqrt{\gamma}} \right) \right). \quad (4.183)$$

Substituting (4.181) and (4.182) into (4.179) yields the relative work factor defect between U and $F(X)$ (on the elements of A_2 with pre-image size $\geq r$)

$$\frac{w_{e_r}(U) - w_{e_r}(F(X))}{w_{e_r}(U)} = \frac{\Phi \left(\frac{\gamma - r + 1.5}{\sqrt{\gamma}} \right) - \Phi \left(\frac{\gamma - r + 0.5}{\sqrt{\gamma}} \right)}{\Phi \left(\frac{\gamma - r + 1.5}{\sqrt{\gamma}} \right)}. \quad (4.184)$$

For further results on the work factor, the interested reader is referred to the appendix, Section 6.2. 701

Impact on the Shannon entropy 702

After this excursion to the work factor, we return to our main goal, the impact of randomly selected mappings on entropy. Our analysis begins with Shannon entropy. As before, $\gamma = 2^{n-m}$ with “large” parameters n and m , and the random variable X is assumed to be uniformly distributed on A_1 .

[Shannon entropy] In the context of AIS 20 and AIS 31, we are interested in the entropy $H(f(X))$ for given mappings f . For one-way functions (such as SHA-256), it is infeasible to determine this value exactly. Instead, we compute the expected Shannon entropy value if the mapping f is selected randomly; cf. pars. 681 to 685; basic considerations were already explained, e.g., in [Schi09b], Example 3.11. We view f as a realization of the random variable F . Straightforward considerations yield 703

$$\begin{aligned}
 E(H(F(X))) &= - \sum_{f \in \mathcal{F}_{A_1, A_2}} \frac{1}{|\mathcal{F}_{A_1, A_2}|} \sum_{a_2 \in A_2} \left(\sum_{a \in f^{-1}(a_2)} \text{Prob}(X = a) \right) \cdot \log_2 \left(\sum_{a \in f^{-1}(a_2)} \text{Prob}(X = a) \right) \\
 &= - \sum_{f \in \mathcal{F}_{A_1, A_2}} \frac{1}{|\mathcal{F}_{A_1, A_2}|} \sum_{r=1}^{b_1} |\{y \in A_2 \mid |f^{-1}(y)| = r\}| \cdot \frac{r}{b_1} \cdot \log_2 \left(\frac{r}{b_1} \right) \\
 &= - \sum_{r=1}^{b_1} E_F(|\{y \in A_2 \mid |f^{-1}(y)| = r\}|) \cdot \frac{r}{b_1} \cdot \log_2 \left(\frac{r}{b_1} \right) \\
 &= - \sum_{r=1}^{b_1} b_2 \text{Prob}(Y' = r) \cdot \frac{r}{b_1} \cdot \log_2 \left(\frac{r}{b_1} \right). \tag{4.185}
 \end{aligned}$$

The random variable Y' is $B(2^n, 2^{-m})$ -distributed. To be precise, $E(H(F(X)))$ denotes the average Shannon entropy with regard to F , i.e., $E_F(H(F(X)))$.

The first line in (4.185) provides the formula for the general case where X has arbitrary distribution on A_1 . Of course, the expectation $E(H(F(X)))$ exists for non-uniformly distributed random variables X , too, but the computations become significantly more complicated. For the special case where X is uniformly distributed on A_1 , it suffices to consider the size of the pre-images. This simplifies the computations significantly (second line of (4.185)). Since $b_1 = b_2\gamma$, we obtain $\log_2(\frac{r}{b_1}) = \log_2(\frac{r}{\gamma}) - m$, and $E(Y') = \frac{b_1}{b_2}$ (4.185) implies

$$\begin{aligned}
 E(H(F(X))) &= -b_2 \sum_{j=1}^{b_1} \text{Prob}(Y' = r) \left(-m \frac{r}{b_1} + \frac{r}{b_2\gamma} \log_2 \left(\frac{r}{\gamma} \right) \right) = \\
 &\quad \frac{b_2}{b_1} m E(Y') - \frac{b_2}{b_2} E \left(\frac{Y'}{\gamma} \log_2 \left(\frac{Y'}{\gamma} \right) \right) = m - E \left(\frac{Y'}{\gamma} \log_2 \left(\frac{Y'}{\gamma} \right) \right) \tag{4.186}
 \end{aligned}$$

[Shannon entropy, $n \leq m$, or $n \gtrsim m$] As in par. 1388, one concludes that the random variable 704

Y' can be viewed as Poisson distributed with parameter $\gamma = 2^{n-m}$. Then (4.186) equals

$$E(H(F(X))) = m - e^{-\gamma} \sum_{r=0}^{\infty} \frac{\gamma^r}{r!} \frac{r}{\gamma} \log_2 \left(\frac{r}{\gamma} \right) = m - e^{-\gamma} \sum_{r=1}^{\infty} \frac{\gamma^{r-1}}{(r-1)!} \log_2 \left(\frac{r}{\gamma} \right), \quad (4.187)$$

with $0 \log_2(0) := 0$ as usual. The second term of (4.187) quantifies the average **Shannon entropy** defect that occurs if a random mapping is applied to a **uniformly distributed random variable** X .

705 [**Shannon entropy**, special case $\gamma = 1$] Here $b_1 = b_2 = 2^n$ and $\gamma = 1$. Hence, (4.187) simplifies to

$$E(H(F(X))) = m - e^{-1} \sum_{r=1}^{\infty} \frac{1}{(r-1)!} \log_2(r). \quad (4.188)$$

Numerical computations show that

$$e^{-1} \sum_{r=1}^{\infty} \frac{1}{(r-1)!} \log_2(r) \approx e^{-1} \sum_{r=1}^{10} \frac{\log_2(r)}{(r-1)!} \approx 0.827. \quad (4.189)$$

This means that the average **Shannon entropy** defect per output bit is $0.827/m$.

Example: For $n = m = 256$, the average **Shannon entropy** defect per output bit is ≈ 0.003 .

706 [**Shannon entropy**, $n \leq m$, or $n \gtrsim m$] Example: $n = 260$, $m = 256$: By (4.187), the average **Shannon entropy** defect per output bit is ≈ 0.00018 .

707 [**Shannon entropy**, $\gamma \gg 1$] Since n and m are assumed to be large, Y' can be approximated by a normal distribution $N(\gamma, \gamma)$ with $\gamma = 2^{n-m}$, cf. par. 696. Then

$$\text{Prob}(Y' = r) \approx \frac{1}{\sqrt{2\pi\gamma}} e^{-\frac{(r-\gamma)^2}{2\gamma}}, \quad (4.190)$$

and (4.186) can be approximated by

$$E(H(F(X))) = m - \frac{1}{\sqrt{2\pi\gamma} \log(2)} \int_0^{\infty} e^{-\frac{(r-\gamma)^2}{2\gamma}} \cdot \frac{r}{\gamma} \log \left(\frac{r}{\gamma} \right) dr. \quad (4.191)$$

708 [**Shannon entropy**, $\gamma \gg 1$] Because the exact calculation of the right-hand integral in (4.191) appears to be quite difficult, we apply Jensen's inequality to the expectation in (4.186). Recall that $Y' \sim B(2^n, p)$ -distributed with $p = 2^{-m}$ (cf. par. 693), and furthermore, $E(Y') = \gamma$ and $E(Y'^2) = \text{Var}(Y') + E^2(Y') = \gamma(1-p) + \gamma^2$ (par. 696). By this,

$$E \left(\frac{Y'}{\gamma} \log_2 \left(\frac{Y'}{\gamma} \right) \right) = \sum_{j=0}^{b_1} \frac{j \text{Prob}(Y' = j)}{\gamma} \log_2 \left(\frac{j}{\gamma} \right) = \sum_{j=0}^{b_1} q_j \log_2 \left(\frac{j}{\gamma} \right) \quad (4.192)$$

where $q_j := j \text{Prob}(Y' = j)/\gamma$. Since $\gamma = E(Y')$, it is $q_0, \dots, q_{b_1} \geq 0$ and $q_0 + \dots + q_{b_1} = 1$. Furthermore, since $x \mapsto \log(x/\gamma)$ is concave on $\mathbb{R}_+$ Jensen's inequality implies

$$\begin{aligned} \sum_{j=1}^{b_1} q_j \log_2 \left(\frac{j}{\gamma} \right) &\leq \log_2 \left(\sum_{j=1}^{b_1} q_j \frac{j}{\gamma} \right) = \log_2 \left(\sum_{j=1}^{b_1} \frac{\text{Prob}(Y' = j) j^2}{\gamma^2} \right) = \log_2 \left(\frac{E(Y'^2)}{\gamma^2} \right) = \\ &\log_2 \left(\frac{\gamma(1-p) + \gamma^2}{\gamma^2} \right) < \log_2 \left(1 + \frac{1}{\gamma} \right) \leq \frac{1}{\log(2)\gamma}. \end{aligned} \quad (4.193)$$

(Since $q_0 = 0$ and by convention $0 \cdot \log_2(0) = 0$, the sums in (4.193) start with index $j = 1$.) Equation (4.193) matches with our intuition that the **entropy** defect becomes negligible if the compression difference $n - m$ and thus $\gamma = 2^{n-m}$ increases. Altogether, by (4.186)

$$E(H(F(X))) \geq m - \frac{1}{\log(2)\gamma}. \quad (4.194)$$

Dividing (4.194) by m provides a lower bound for the average **Shannon entropy** per **internal random number** bit:

$$\frac{E(H(F(X)))}{m} \geq 1 - \frac{1}{\log(2)\gamma m}. \quad (4.195)$$

Note: Jensen's inequality applies to the case $\gamma = 1$, too, but there (4.188) is more suitable as it quantifies the **entropy** defect more precisely than (4.193).

Impact on **min-entropy**

709

Finally, we consider the impact of randomly selected mappings on the **min-entropy**. As before, $\gamma = 2^{n-m}$ and the **random variable** X is assumed to be **uniformly distributed** on A_1 . The following results have the highest relevance for functionality classes PTG.3 and NTG.1. It yet can be useful for functionality class PTG.2, too, if the **PTRNG** uses a **post-processing** algorithm that applies a **cryptographic** function (usually without memory). This also applies to the results on non-uniformly distributed **random variables**, which are discussed in pars. 722 ff.

[**Min-entropy**] For a mapping $f: A_1 \rightarrow A_2$, the **min-entropy** of $f(X)$ results from the most probable pre-image $y^* \in A_2$, i.e., $H_{\min}(f(X)) = \max\{-\log_2(\text{Prob}(f(X) = y')) \mid y' \in A_2\}$. Since X is assumed to be **uniformly distributed**, for given $f \in \mathcal{F}(A_1, A_2)$ the probability simplifies to $\text{Prob}(f(X) = y') = \frac{|f^{-1}(y')|}{2^n}$ for each $y' \in A_2$. Thus, it suffices to study the size of the pre-images $f^{-1}(y') \subseteq A_1$. Obviously,

710

$$\left(\max_{y' \in A_2} |f^{-1}(y')| > \gamma(1 + \tau) \right) \iff \left(H_{\min}(f(X)) < -\log_2 \left(\frac{\gamma(1 + \tau)}{2^n} \right) = m - \log_2(1 + \tau) \right) \quad \text{for } \tau > 0. \quad (4.196)$$

Below, we analyze the distribution of the size of the largest pre-image if the mapping f is selected randomly.

Note: Any useful upper bound for the maximum pre-image size (corresponding with a useful lower **min-entropy** bound for $f(X)$) of a randomly selected mapping f can only be guaranteed with a certain probability. (Think, for example, of the worst case, where f maps A_1 onto a single element $y_0 \in A_2$. Of course, this case is very unlikely but its probability is larger than 0.)

First, we study the cases $n \leq m$ and $n \gtrsim m$. The case $n < m$ can be relevant when **DRNGs** are (re-)seeded, especially if the **effective internal states** is large. Prominent examples are the Hash_DRBG and the HMAC_DRBG; cf. Subsections. 5.3.1 and 5.3.2. This scenario applies to the functionality classes DRG.2, DRG.3, DRG.4, and DRT.1, and also to functionality classes PTG.3 (and NTG.1) if the **intermediate random number** are used for reseeding. Later, we treat the case $\gamma \gg 1$, which is needed to generate **internal random numbers** with a small **min-entropy** defect.

711

[**min-entropy**, $n \leq m$, or $n \gtrsim m$] Recall that $A_1 = \{0, 1\}^n$, $A_2 = \{0, 1\}^m$ while F is a random mapping from $A_1 \rightarrow A_2$. To simplify notation, in this paragraph, we use the following definition:

712

For $b \in A_2$, we define the **random variable** $X_b := |F^{-1}(\{b\})|$. As already pointed out in pars. 703 and 704, $X_b \sim B(2^n, 2^{-m})$, and X_b is approximately Poisson distributed P_γ with parameter $\gamma = 2^{n-m}$. Furthermore, let $t \in \mathbb{N}$, and for $b \in A_2$, the binary-valued **random variable** Y_b is given by $Y_b = 1$ if $X_b > t$ and $Y_b = 0$ else. Using these definitions,

$$\max\{X_b \mid b \in A_2\} > t \iff \max\{Y_b \mid b \in A_2\} = 1 \iff \sum_{b \in A_2} Y_b \geq 1. \quad (4.197)$$

Since X_b is (approximately) P_γ Poisson distributed for each $b \in A_2$, in particular,

$$\text{Prob}(Y_b = 1) = \text{Prob}(X_b > t) = 1 - e^{-\gamma} \sum_{k=t+1}^{\infty} \frac{\gamma^k}{k!}. \quad (4.198)$$

Using the equivalences (4.197), straightforward considerations yield

$$\begin{aligned} \text{Prob}(\max\{Y_b \mid b \in A_2\} = 1) &= \sum_{j=1}^{2^n} \text{Prob}\left(\sum_{b \in A_2} Y_b = j\right) \leq \\ &\sum_{j=1}^{2^n} \text{Prob}\left(\sum_{b \in A_2} Y_b = j\right) j = E\left(\sum_{b \in A_2} Y_b\right) = \sum_{b \in A_2} E(Y_b) = 2^m E(Y_b). \end{aligned} \quad (4.199)$$

The linearity of the expectation implies the last but one equation in (4.199). To make further calculations easier, we provide an upper bound for $E(Y_b)$

$$\begin{aligned} E(Y_b) &= \text{Prob}(Y_b = 1) = \text{Prob}(X_b > t) = \\ &e^{-\gamma} \sum_{k=t+1}^{\infty} \frac{\gamma^k}{k!} = \frac{e^{-\gamma} \gamma^{t+1}}{(t+1)!} \left(1 + \frac{\gamma^1}{t+2} + \frac{\gamma^2}{(t+2)(t+3)} + \dots\right) \leq \\ &\frac{e^{-\gamma} \gamma^{t+1}}{(t+1)!} \left(1 + \frac{\gamma^1}{t+2} + \frac{\gamma^2}{(t+2)^2} + \dots\right) = \frac{e^{-\gamma} \gamma^{t+1}}{(t+1)!} \cdot \frac{1}{1 - \frac{\gamma}{t+2}} = \frac{e^{-\gamma} \gamma^{t+1} (t+2)}{(t+1)! (t+2-\gamma)}. \end{aligned} \quad (4.200)$$

Putting the pieces together gives

$$\begin{aligned} \text{Prob}(\max\{X_b \mid b \in A_2\} > t) &= \text{Prob}(\max\{Y_b \mid b \in A_2\} = 1) \leq 2^m E(Y_b) = \\ &\leq 2^m \frac{e^{-\gamma} \gamma^{t+1} (t+2)}{(t+1)! (t+2-\gamma)}. \end{aligned} \quad (4.201)$$

Finally, for given probability $p > 0$

$$\begin{aligned} \left(2^m \frac{e^{-\gamma} \gamma^{t+1} (t+2)}{(t+1)! (t+2-\gamma)} < p\right) &\implies (\text{Prob}(\max\{X_b \mid b \in A_2\} > t) < p) \iff \\ &\left(\text{Prob}\left(H_{\min}(F(X)) \geq -\log_2\left(\frac{t}{2^n}\right)\right) \geq 1 - p\right). \end{aligned} \quad (4.202)$$

Note: If the term $2^m E(Y_b)$ is small, the inequality (4.199) should be sharp because then for $j > 1$ the probabilities $\text{Prob}(\sum_{b \in A_2} Y_b = j)$ should be small.

713 **[min-entropy, $n = m$]** Example: $n = m = 256$ and $p = 2^{-16}$. Then $\text{Prob}(H_{\min}(F(X)) \geq -\log_2(\frac{59}{2^{256}})) \geq 1 - 2^{-16}$. In particular, the **min-entropy** per output bit $\frac{H_{\min}(F(X))}{256} \geq 0.977$ with probability $\geq 1 - 2^{-16}$.

Let $f \in \mathcal{F}_{A_1, A_2}$ be fixed. For the remainder of this section, the term $M(f) := \max_{y' \in A_2} |f^{-1}(y')|$ denotes the maximal pre-image size for f (a.k.a. maximal occupancy). In pars. 715 to 717, we state three known limiting distributions of $M(F)$; see [KoSC78] for details. Interestingly, the limiting behavior depends on the scale parameter

$$\frac{|A_1|}{|A_2| \log(|A_2|)} = \frac{2^n}{2^m \log(2^m)} = \frac{\gamma}{m \log(2)}. \quad (4.203)$$

In the pars. 715 to 717, we assume that $n, m \rightarrow \infty$.

Note: For our purposes, only the case $\frac{\gamma}{m \log(2)} \rightarrow \infty$ is relevant. For the sake of completeness, in pars. 715 and 716, we briefly address the other cases, too.

[Case $\frac{\gamma}{m \log(2)} \rightarrow 0$] ([KoSC78], Section II 6, Theorem 1, p. 96) This case covers data expansion 715 ($n < m$), the case $\gamma = 1$, and small compression $\gamma > 1$. Interestingly, the asymptotic distribution is concentrated at most two values. More precisely, assume that $r = r(m)$ is chosen so that $r > \gamma$ and $2^m \nu_{P_\gamma}(r)$ converges to $\lambda > 0$. Here, ν_{P_γ} denotes the Poisson distribution with parameter γ . Then

$$\text{Prob}(M(F) = r - 1) \rightarrow e^{-\lambda} \quad (4.204)$$

$$\text{Prob}(M(F) = r) \rightarrow 1 - e^{-\lambda} \quad (4.205)$$

[Case $\frac{\gamma}{m \log(2)} \rightarrow x > 0$] ([KoSC78], Section II 6, Theorem 2, p. 96) In this case, the asymptotic 716 distribution is discrete with infinite range. Assume that $r = r(m)$ is chosen so that $r > \gamma$ and $2^m \nu_{P_\gamma}(r)$ converges to $\lambda > 0$. Then

$$\text{Prob}(M(F) \leq r + j) \xrightarrow{r \rightarrow \infty} e^{-\frac{\lambda \eta^{j+1}}{1-\eta}} \quad \text{for } j \in \mathbb{Z} \quad (4.206)$$

where η is the zero of the equation

$$\eta + x(\log(\eta) - \eta + 1) = 0 \quad (4.207)$$

in $(0, 1)$.

Note: In (4.207), x denotes the value specified in the Case condition.

[Case $\frac{\gamma}{m \log(2)} \rightarrow \infty$] ([KoSC78], Section II 6, Theorem 3, p. 96) This is the most relevant 717 case for us since it treats the case of large compression. Here, the asymptotic distribution is continuous. It is an extreme value distribution of the double exponential type, often named ‘‘Gumbel distribution’’. More precisely,

$$\begin{aligned} & \text{Prob} \left(\frac{M(F) - \gamma - \gamma u \left(\frac{1}{\gamma} \left(\log(2^m) - \frac{\log \log(2^m)}{2} \right) \right)}{\sqrt{\frac{\gamma}{2} \log(2^m)}} + \frac{1}{2} \log(4\pi) \leq z \right) = \\ & \text{Prob} \left(\frac{M(F) - \gamma - \gamma u \left(\frac{1}{\gamma} \left(m \log(2) - \frac{\log(m \log(2))}{2} \right) \right)}{\sqrt{\frac{\gamma}{2} m \log(2)}} + \frac{1}{2} \log(4\pi) \leq z \right) \rightarrow e^{-e^{-z}} \end{aligned} \quad (4.208)$$

The function $u: (0, \infty) \rightarrow (0, \infty)$ is implicitly defined by

$$-u(t) + (1 + u(t)) \log(1 + u(t)) = t. \quad (4.209)$$

More precisely, $u(t)$ is the positive solution of (4.209). By ([KoSC78], Section II 6, Lemma 5 (8), p. 102), for small $t > 0$ the function $u(t)$ has the expansion

$$u(t) = \sqrt{2t} + \frac{1}{3}t - \frac{\sqrt{2}}{36}t^{3/2} + \dots \quad (4.210)$$

In (4.208) it is $t = \frac{1}{\gamma} \left(m \log(2) - \frac{\log(m \log(2))}{2} \right)$. The expansion (4.210) applies to large compression rates γ . (Then $\frac{m}{\gamma} = m \cdot 2^{n-m} \ll 1$.)

Note: There seems to be a typo in [KoSC78], Section II 6, Theorem 3, p. 96. As in Theorem 7, p. 112, it should read $\alpha / \ln N \rightarrow \infty$ in place of $\alpha / \ln N \rightarrow 0$. (In our notation, $\alpha / \ln N$ reads $\gamma / (m \log(2))$.)

718 $[\gamma = 1]$ The Case $\frac{\gamma}{m \log(2)} \rightarrow 0$ from par. 715 applies to $\gamma \leq 1$ (no data compression). However, par. 712 provides more concrete results.

719 [numerical example, $\gamma \gg 1$] The largest pre-image determines the **min-entropy**. Formula (4.208) in par. 717 gives an upper bound for the maximal occupancy $M(F)$, which in turn provides a lower bound for the **min-entropy**. The expansion of $u(t)$ until the third term should be enough by far because $t = O(\gamma^{-1})$ in (4.208). Furthermore, the **min-entropy** bound depends on the choice of z . (Of course, in (4.208) probability 1 cannot be achieved because constant mappings map A_1 onto a single element.) Solving the term inside $\text{Prob}(\dots)$ in (4.208) for $M(F)$ gives

$$M(F) \leq \sqrt{\frac{\gamma}{2} m \log(2)} \left(z - \frac{1}{2} \log(4\pi) \right) + \gamma + \gamma u \left(\frac{1}{\gamma} \left(m \log(2) + \frac{\log(m \log(2))}{2} \right) \right). \quad (4.211)$$

Then the term

$$M_z^*(F) := \sqrt{\frac{\gamma}{2} m \log(2)} \left(z - \frac{1}{2} \log(4\pi) \right) + \gamma + \gamma u \left(\frac{1}{\gamma} \left(m \log(2) + \frac{\log(m \log(2))}{2} \right) \right) \quad (4.212)$$

defines an upper bound for the maximal pre-image size that is not exceeded with probability $e^{-e^{-z}}$ (provided, of course, that the asymptotic behavior is already valid). Therefrom, by (4.196) with $M_z^*(F) = \gamma(1 + \tau)$, we conclude that the **min-entropy** per m -bit output block does not undercut

$$-\log_2 \left(\frac{M_z^*(F)}{2^n} \right) = m - \log_2 \left(\frac{M_z^*(F)}{\gamma} \right). \quad (4.213)$$

with probability $e^{-e^{-z}}$. Dividing (4.213) by m implies that the (average) **min-entropy** per output bit does not undercut

$$h_{\min, (n, m, z)} := 1 - \frac{\log_2 \left(\frac{M_z^*(F)}{\gamma} \right)}{m} \quad (4.214)$$

with probability $e^{-e^{-z}}$. Consequently, the **min-entropy defect per output bit**, $e_{\text{def}} := \frac{\log_2 \left(\frac{M_z^*(F)}{\gamma} \right)}{m}$, is exceeded only with complementary probability $1 - e^{-e^{-z}}$. (As before, X and F are **uniformly distributed** on $\{0, 1\}^n$ and on $\mathcal{F}_{\{0, 1\}^n, \{0, 1\}^m}$, respectively.)

720 The parameter z occurs on the left and on the right side of (4.208). The function $z \mapsto e^{-e^{-z}}$ is strictly monotonically increasing on $\mathbb{R}$. To given z , let $p_z := 1 - e^{-e^{-z}}$ denote the probability that the **min-entropy** defect exceeds e_{def} . Solving this equation by z yields

$$(p_z := 1 - e^{-e^{-z}}) \iff (-\log(1 - p_z) = e^{-z}) \iff (-\log(-\log(1 - p_z)) = z). \quad (4.215)$$

To given probability p , the right-hand term in (4.215) assigns the corresponding value z_p . This value z_p can be substituted into (4.214). Then the **min-entropy** defect exceeds $\frac{\log_2\left(\frac{M_{z_p}^*(F)}{\gamma}\right)}{m}$ only with probability p . Table 3 exemplarily provides several pairs (p, z_p) .

Table 3: Case $\frac{\gamma}{m \log(2)} \rightarrow \infty$ (cf. par. 717): Pairs (p, z_p) for which the **min-entropy** defect per output bit, $\frac{\log_2\left(\frac{M_{z_p}^*(F)}{\gamma}\right)}{m}$, is exceeded only with probability p . The table values are computed with formula (4.215).

p	0.02	2^{-8}	2^{-10}	2^{-12}	2^{-16}
z_p	3.90	5.54	6.93	8.31	11.09

Table 4 provides the **min-entropy** defect per output bit (4.214) for several parameter sets (n, m, p) . Table 4 uses the approximation $u(t) \approx \sqrt{2t} + \frac{1}{3}t - \frac{\sqrt{2}}{36}t^{3/2}$; cf. (4.210). (Cutting off $u(t)$ after the first term, for example, would yield slightly different figures.) As explained in par. 720, to given probability p first the corresponding value z_p is calculated with formula (4.215). In a second step, the value z_p is substituted into (4.214). The entry in the last column of row 1 (Table 4), for example, says that the **min-entropy** defect per bit exceeds $2^{-32.67}$ only with probability 2^{-16} , or equivalently, that (average) **min-entropy** per output bit is greater than or equal to $h_{\min, (n, m, z_{2^{-16}})} = 1 - 2^{-32.67}$ with probability $1 - 2^{-16}$. Table 4 and further numerical experiments indicate that the **min-entropy** defect per output bit essentially depends on the difference $n - m$ or, equivalently, on $\gamma = 2^{n-m}$. Roughly speaking, ensuring a **min-entropy** defect of 2^{-t} requires $n - m \approx 2t$. The probability p seems to be less relevant for this question.

Table 4: Case $\frac{\gamma}{m \log(2)} \rightarrow \infty$ (cf. par. 717): The **min-entropy** defect per output bit, $\frac{\log_2\left(\frac{M_{z_p}^*(F)}{\gamma}\right)}{m}$, is exceeded only with probability p . The table values are computed with formulae (4.215) and (4.214).

(n, m)	$p = 2^{-8}$	$p = 2^{-12}$	$p = 2^{-16}$
(320, 256)	$2^{-33.59}$	$2^{-33.05}$	$2^{-32.67}$
(256, 128)	$2^{-65.09}$	$2^{-64.56}$	$2^{-64.17}$
(192, 128)	$2^{-33.09}$	$2^{-32.56}$	$2^{-32.17}$

[non-uniformly distributed pre-images] For PTG.3-compliant **PTRNGs**, the **cryptographic post-processing** algorithm often can be modeled by a randomly selected mapping or a composition of randomly selected mappings. The first scenario typically applies if the **intermediate random numbers** are used as **high-entropy additional input**, while the second scenario is typical if the **intermediate random numbers** are used for reseeding; cf. the application notes to Requirement PTG.3.5 and pars. 679 to 684. So far, in this section we have assumed that the random vector X is **uniformly distributed** on $\{0, 1\}^n$. For “real-world” **PTRNGs**, the **intermediate random numbers** (typically generated by a PTG.2-compliant **PTRNG**) are not uniformly distributed but their bits usually are (to some degree) **biased** and correlated. An **intermediate random number** can again be interpreted as a realization of a random vector X , which yet cannot be assumed to be **uniformly distributed** on $\{0, 1\}^n$. This means that some values in $\{0, 1\}^n$ are attained by X with

small probabilities and others with large probabilities. One might expect that the application of a random mapping averages this imbalancedness out to a large extent in the image space $\{0, 1\}^m$. In the following, we treat the question how deviations from the uniform distribution influence the above results. We focus on **min-entropy**.

723 [non-uniformly distributed pre-images] Principally, one might try to adapt the strategies for the uniform distribution but then the computations became too complicated; in particular, it would no longer suffice to count the number of pre-images. Below, we show that the deviations of X from the uniform distribution can be compensated by increasing the input length from n to $n^* > n$. In pars. 724 to 729 we discuss the case ($n > m$, $\gamma \gg 1$). Later, in pars. 730 to 731 the case $n \leq m$, or $n \approx m$ is considered.

724 [non-uniformly distributed pre-images] For the moment, we assume for simplicity that the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** (typical design) satisfying the **min-entropy** claim; i.e., the **min-entropy** per **internal random number** bit of the PTG.2-compliant **PTRNG** is greater than or equal to 0.98. We further assume that an **intermediate random number** is a realization of a **random variable** X' . Let

$$n^* = \min\{n' \geq n \mid H_{\min}(X') \geq n\} \quad \text{and} \quad \Delta n := n^* - n. \quad (4.216)$$

In the following, we assume that $n - m$ and (n, m) are sufficiently large such that the asymptotic formula (4.208) applies. Furthermore, $A_1^* = \{0, 1\}^{n^*}$, and, as before, $A_1 = \{0, 1\}^n$ and $A_2 = \{0, 1\}^m$.

Note 1: Functionality class PTG.3 requires $n \geq m + 16$.

In the following, we compare the case where an n^* -bit random vector X' is mapped (by a randomly selected mapping $\in \mathcal{F}_{A_1^*, A_2}$) to m -bit output vectors with the case where a **uniformly distributed** n -bit random vector X is mapped (by a randomly selected mapping $\in \mathcal{F}_{A_1, A_2}$) to an m -bit output vector. The second case has already been studied above.

Note 2: The requirement that the **intermediate random numbers** are generated by a PTG.2-compliant **PTRNG** will be relaxed in par. 729.

725 [non-uniformly distributed pre-images] Example: Assume that each **intermediate random number** can be interpreted as a realization of sequences of binary-valued **iid random variables** $B_1, B_2, \dots, B_{n^*}$ with $\text{Prob}(B_j = 1) = 0.5 + \epsilon$ for some $\epsilon \geq 0$. Then $H_{\min}(B_j) = -\log_2(0.5 + \epsilon)$, and $n^* = \left\lceil \frac{n}{-\log_2(0.5 + \epsilon)} \right\rceil$.

Numerical example: $(n, \epsilon, n^*, \Delta n) = (256, 0.01, 264, 8), (256, 0.007, 262, 6), (320, 0.007, 327, 7)$.

726 [non-uniformly distributed pre-images, $\gamma \gg 1$] Let $\gamma^* = 2^{n^* - m}$ and $a_2 \in A_2$. We assume that $f \in \mathcal{F}_{A_1^*, A_2}$ is selected randomly. The pre-image size $v_{a_2} := |f^{-1}(a_2)|$ can be interpreted as a realization of a **random variable** $V_{a_2} \sim B(2^{n^*}, 2^{-m}) \approx N(2^{n^* - m}, 2^{n^*} \cdot 2^{-m}(1 - 2^{-m})) \approx N(\gamma^*, \gamma^*)$ (CLT). We may assume that the pre-image $f^{-1}(\{a_2\}) = \{a'_1, \dots, a'_v\}$ is a randomly selected subset of A_1 of size v_{a_2} . For $j = 1, \dots, v_{a_2}$, we define the **random variables** $T_j := \text{Prob}(X' = a'_j)$. Then

$$E(T_j) = \sum_{a \in A_1} 2^{-n^*} \text{Prob}(X' = a) = 2^{-n^*} \quad (4.217)$$

and, similarly,

$$E(T_j^2) = \sum_{a \in A_1} 2^{-n^*} \text{Prob}(X' = a)^2 = 2^{-n^*} 2^{-H_2(X')}. \quad (4.218)$$

Recall that $H_2(X')$ denotes the collision **entropy** of X' . Since $V_{a_2} \sim N(\gamma^*, \gamma^*)$, it is $|v| = \gamma^* + O(\sqrt{\gamma^*}) \ll 2^{n^*}$. This justifies the assumption that the **random variables** $T_1, T_2, \dots$ are **iid**.

[non-uniformly distributed pre-images, $\gamma \gg 1$] We set $T_{a_2} := \text{Prob}(X' \in f^{-1}(a_2)) (= \text{Prob}(f(X') = a_2))$, or equivalently, $T_{a_2} = T_1 + \dots + T_{v_{a_2}}$. Wald's Theorem implies

$$E(T_{a_2}) = E\left(\sum_{j=1}^{V_{a_2}} T_j\right) = E(V_{a_2})E(T_j) = \gamma^* 2^{-n^*} = 2^{-m}. \quad (4.219)$$

Similarly, although with greater effort, (4.220) follows. From (4.218) we obtain

$$\begin{aligned} E(T_{a_2}^2) &= E\left(\left(\sum_{j=1}^{V_{a_2}} T_j\right)^2\right) = \sum_{v=0}^{2^{n^*}} \text{Prob}(V_{a_2} = v) E(T_{a_2}^2 | V_{a_2} = v) = \\ &= \sum_{v=0}^{2^{n^*}} \text{Prob}(V_{a_2} = v) E((T_1 + \dots + T_v)^2) = \\ &= \sum_{v=0}^{2^{n^*}} \text{Prob}(V_{a_2} = v) (v E(T_1^2) + (v^2 - v) (E(T_1))^2) = \\ &= \sum_{v=0}^{2^{n^*}} \text{Prob}(V_{a_2} = v) (v \cdot 2^{-n^*} 2^{-H_2(X')} + (v^2 - v) 2^{-2n^*}) = \\ &= E(V_{a_2}) \cdot 2^{-n^*} 2^{-H_2(X')} + (E(V_{a_2}^2) - E(V_{a_2})) 2^{-2n^*} = \\ &= \gamma^* \cdot 2^{-n^*} 2^{-H_2(X')} + \gamma^{*2} 2^{-2n^*} = 2^{-m} \cdot 2^{-H_2(X')} + 2^{-2m}. \end{aligned} \quad (4.220)$$

(It is $E(V_{a_2}^2) - E(V_{a_2}) = E(V_{a_2}^2) - E(V_{a_2})^2 + E(V_{a_2})^2 - E(V_{a_2}) = \gamma^* + \gamma^{*2} - \gamma^* = \gamma^{*2}$.) Finally, from (4.219) and (4.220) we obtain

$$\text{Var}(T_{a_2}) = \text{Var}\left(\sum_{j=1}^{V_{a_2}} T_j\right) = 2^{-m} \cdot 2^{-H_2(X')} + 2^{-2m} - 2^{-2m} = 2^{-m} \cdot 2^{-H_2(X')} \quad \text{and (4.221)}$$

$$\begin{aligned} \sigma_{T_{a_2}} &= \sqrt{\text{Var}\left(\sum_{j=1}^{V_{a_2}} T_j\right)} = \sqrt{2^{-m} \cdot 2^{-H_2(X')}} = \\ &= 2^{-0.5m} \cdot 2^{-0.5 \cdot H_2(X')} = 2^{-0.5(n+m)} \cdot 2^{-0.5(H_2(X')-n)} \end{aligned} \quad (4.222)$$

Example: If $X' = (B_1, \dots, B_{n^*})$ with **iid** $B(1, 0.5 + \epsilon)$ -distributed **random variables** $B_1, \dots, B_{n^*}$ (cf. par. 725), then $H_2(X') = -n^* \log_2(0.5 + 2\epsilon^2)$ and $H_{\min}(X') = -n^* \log_2(0.5 + \epsilon)$.

[non-uniformly distributed pre-images, $\gamma \gg 1$, comparison to uniform input] As in par. 724, we assume that the **random variable** X is **uniformly distributed** on $\{0, 1\}^n$ while X' assumes values in $\{0, 1\}^{n^*}$. The **random variables** F and F' describe **uniformly distributed** random mappings on $\mathcal{F}_{A_1, A_2}$ and on $\mathcal{F}_{A_1^*, A_2}$, respectively, while for **uniformly distributed** input, the **random variable** $T_{a_{2,0}}$ is the equivalent to T_{a_2} . In particular, $E(F(X) = a_2) = \gamma = E(F'(X') = a_2)$, or equivalently, $\text{Prob}(T_{a_{2,0}}) = \text{Prob}(T_{a_2}) = 2^{-m}$. Furthermore, the **random variable** $T_{a_{2,0}}$

has standard deviation $\sigma_{T_{a_2,0}} = 2^{-0.5(n+m)}$ while T_{a_2} has standard deviation $\sigma_{T_{a_2}} (= \sigma_{T_{a_2,0}} \cdot 2^{-0.5(H_2(X')-n)})$. The factor $2^{-0.5(H_2(X')-n)}$ is the ratio of the average standard deviations $\sigma_{T_{a_2,0}}$ and $\sigma_{T_{a_2}}$. By (4.216), $H_2(X')-n \geq H_{\min}(X')-n \geq n-n=0$, and thus $2^{-0.5(H_2(X')-n)} \leq 1$, and finally $\sigma_{T_{a_2}} \leq \sigma_{T_{a_2,0}}$. Since the **min-entropy** is calculated from the largest value, the inequality $\sigma_{T_{a_2}} \leq \sigma_{T_{a_2,0}}$ is an indicator that $H_{\min}(F'(X'))$ falls below a particular bound h_0 is not more probable than for $H_{\min}(F(X))$ (at cost of Δn **additional input** bits). This heuristics can be supported by a further argument. The **random variable** $T_{a_2,0}$ can be viewed as $N(2^{-m}, \sigma_{T_{a_2,0}})$ -distributed (CLT). If the normal approximation also applies to the **random variable** T_{a_2} , for $t > 2^{-m}$ we obtain

$$\text{Prob}(T_{a_2,0} > t) = \Phi\left(-\frac{t - 2^{-m}}{\sigma_{T_{a_2,0}}}\right) = \Phi\left(\frac{2^{-m} - t}{\sigma_{T_{a_2,0}}}\right) \geq \Phi\left(\frac{2^{-m} - t}{\sigma_{T_{a_2}}}\right) = \text{Prob}(T_{a_2} > t) \quad (4.223)$$

Inequality (4.223) says that for a randomly selected a_2 , the probability that $T_{a_2,0}$ exceeds some bound $t > 2^{-m}$ is $\geq$ than the corresponding probability for T_{a_2} .

729 [non-uniformly distributed pre-images, $\gamma \gg 1$] In the derivation of formula (4.222), we used the assumption that the **random variables** $T_1, T_2, \dots, T_{v_{a_2}}$ are **iid** (cf. par. 727). Although it may not be true in a strict sense, two properties justify this assumption. At first, $|f^{-1}(\{a_2\})|/|A_1| \approx 2^{-m}$. In other words: for parameters n, m that are relevant for functionality class PTG.3 (and NTG.1), the size of $f^{-1}(\{a_2\})$ is very small compared to the number of elements in A_1 . Secondly, by assumption the **intermediate random numbers** have been generated by a PTG.2-compliant **PTRNG** (cf. par. 724), which means that their distribution is not very far from the uniform distribution. For very extreme input distributions, the independence assumption might be invalid.

Note 1: Functionality class PTG.3 allows to apply the above-mentioned method if the **intermediate random numbers** are either generated by a PTG.2-compliant **PTRNG** or by a **PTRNG** that satisfies Requirement PTG.3.3.

Note 2: Functionality class NTG.1 allows to apply the above-mentioned method, too. There, the numerical value Δ usually is very large.

Note 3: In both cases, $n \geq m + 16$ is required; cf. application notes pars. 407 (ii), and 467.

730 [non-uniformly distributed pre-images, $n \leq m$, or $n \gtrapprox m$] In this paragraph, we treat two scenarios, namely that the bit length n of the input data is smaller than or equal to the bit length m of the output data, and that n can be a bit larger than m . As for the case $\gamma \gg 1$, which was treated in pars. 722 to 729, we consider non-uniformly distributed **intermediate random numbers**. Just like there, we assume that the **intermediate random numbers** were generated by a PTG.2-compliant **PTRNG**, and we compute

$$n^* = \min\{n' \geq n \mid H_{\min}(X') \geq n\} \quad \text{and} \quad \Delta n := n^* - n. \quad (4.224)$$

Analogously to the case $\gamma \gg 1$, we conclude that for X' the situation is not worse than for the ideal case (at the cost of Δn **additional input** bits); cf. par. 728. The main difference is that the **random variable** V_{a_2} should be approximated by a Poisson distribution (P_{γ^*}) rather than by a normal distribution ($N(\gamma^*, \gamma^*)$; cf. par. 726. In particular, formula (4.222) is also valid in this case. Recall that expectation and variance of P_{γ} -distributed **random variable** equals γ .

Note: As already mentioned in par. 711, the case $n < m$ can be relevant when **DRNGs** are reseeded. This applies to functionality classes DRG.2, DRG.3, DRG.4, DRT.1, and PTG.3.

731

[non-uniformly distributed pre-images, $n \leq m$, or $n \gtrsim m$] As in par. 729, we relax the requirement that the **intermediate random numbers** have been generated by a PTG.2-compliant PTRNG.

Note 1: Functionality class PTG.3 allows to apply the method described in par. 730 if the **intermediate random numbers** are generated by a PTG.2-compliant that satisfies Requirement PTG.3.3.

Note 2: Functionality class PTG.3 requires $n^- \leq m + 6$; cf. application notes par. 407(iii).

[conversion: non-uniformly distributed input data to **uniformly distributed** input data] So far, 732
we started with a parameter set (n, m) and **uniformly distributed** input data, described by a **random variable** X . Formulae (4.216) and (4.224) therefrom determined a parameter set (n^*, m) that corresponds to non-uniformly distributed input data, described by a **random variable** X' . This “conversion formula” can be inverted in a straightforward manner: Begin with a parameter set (n, m) for the *non-uniformly distributed input data* and therefrom compute a corresponding parameter set (n^-, m) for **uniformly distributed** input data.

$$n^- := \lfloor H_{\min}(X') \rfloor \quad (4.225)$$

Since $n = (n^-)^*$, the argumentations from above remain valid, i.e., the parameter set (n^-, m) for **uniformly distributed** input data should provide a lower **min-entropy** bound for the non-uniformly distributed input data with parameters (n, m) . In other words, it should be $H_{\min}(F(X)) \leq H_{\min}(F'(X'))$ with high probability where F and F' denote (**uniformly distributed**) random mappings on $\mathcal{F}_{A_1^-, A_2}$ and $\mathcal{F}_{A_1, A_2}$, respectively. Here $A_1^- = \{0, 1\}^{n^-}$, $A_1 = \{0, 1\}^n$, and $A_2 = \{0, 1\}^m$.

Note 1: For evaluation purposes, the conversion formula (4.225) should be more convenient than (4.216) and (4.224); cf. par. 405.

Note 2: For certain scenarios, conversion formula (4.225) has to be applied instead of (4.224); cf. par. 740.

[minimum pre-image size] Formula (4.212) provides an upper bound $M_z^*(F)$ for the maximal 733
pre-image size that is exceeded only with probability $\leq p$. Functionality classes PTG.3 and NTG.1 use the value $p = 2^{-16}$. For specific problems (cf. par. 628), a threshold $M_{z^*}^*(F)$ for the minimum pre-image size is useful that is undercut only with probability $\leq p$. This topic is treated in par. 734 (**uniformly distributed** input data) and in par. 735 (non-uniformly distributed input data). We focus on the case of (strong) data compression $\frac{\gamma}{m \log(2)} \rightarrow \infty$.

[minimum pre-image size, **uniformly distributed** input, Case $\frac{\gamma}{m \log(2)} \rightarrow \infty$] ([KoSC78], Sec- 734
tion II 6, Theorem 7, p. 112) The asymptotic formula for the minimum occupancy number $M^-(F)$ is similar to the asymptotic formula (4.208) for the maximum occupancy $M(F)$ number.

$$\text{Prob} \left(\frac{M^-(F) - \gamma - \gamma v \left(\frac{1}{\gamma} \left(m \log(2) - \frac{\log(m \log(2))}{2} \right) \right)}{\sqrt{\frac{\gamma}{2} m \log(2)}} - \frac{1}{2} \log(4\pi) \leq -z \right) \rightarrow 1 - e^{-e^{-z}} \quad (4.226)$$

The function $v: [0, 1] \rightarrow \mathbb{R}$ is the negative solution of (4.209). By ([KoSC78], Section II 6, Lemma 5 (9), p. 102), for small $t > 0$ the function $v(t)$ has the expansion

$$v(t) = -\sqrt{2t} + \frac{1}{3}t + \frac{\sqrt{2}}{36}t^{3/2} + \dots \quad (4.227)$$

In analogy to (4.212), the term

$$M_z^{-,*}(F) := -\sqrt{\frac{\gamma}{2} m \log(2)} \left(z - \frac{1}{2} \log(4\pi) \right) + \gamma + \gamma v \left(\frac{1}{\gamma} \left(m \log(2) + \frac{\log(m \log(2))}{2} \right) \right) \quad (4.228)$$

defines a lower bound for the minimal pre-image size that is undercut with probability $1 - e^{-e^{-z}}$ (provided, of course, that the asymptotics take effect). Recall that $M_z^*(F)$ is exceeded only with probability $1 - e^{-e^{-z}}$. For small $t > 0$, the functions $u(t)$ and $v(t)$ are dominated by $\pm\sqrt{2t}$. However, the second terms of (4.210) and (4.227) are not the negative of each other. Consequently, for $\gamma \gg 1$ the values $M_z^{-,*}(F)$ and $M_z^*(F)$ lie roughly symmetrically around the average pre-image size γ but there is no perfect symmetry.

The value

$$-\log_2 \left(\frac{M_z^{-,*}(F)}{2^n} \right) = m - \log_2 \left(\frac{M_z^{-,*}(F)}{\gamma} \right) \quad (4.229)$$

is undercut only with probability $e^{-e^{-z}}$. For concrete calculations, the approximation (4.227) can be applied.

735 [minimum pre-image size, non-uniformly distributed input data, $\gamma \gg 1$] The transfer from $F(X')$ (with parameters (n, m)) to $F(X)$ (with parameters (n^-, m)) works as for the maximum pre-image size because the minimum occupancy number (as the maximum occupancy number) refers to extreme values.

736 Some useful facts

The pars 737 to 742 provide several facts and results that can be useful for the evaluation of RNGs.

737 If $f, g \in \mathcal{F}_{A,A}$, usually $|g(f(A))| < |f(A)|, |g(A)|$; Table 1 considers the image size for k -fold iterates of f . Analogous shrinking effects arise in the general case where $f \in \mathcal{F}_{A_1, A_2}$ and $g \in \mathcal{F}_{A_2, A_3}$. Consequently, the composition $G \circ F$ of (uniformly distributed) random mappings F and G , is *not* a (uniformly distributed) random mapping. This observation has impact on the entropy analysis; cf. pars. 393, 410, 411, 412.

738 Let $g \in \mathcal{F}_{A_1, A'_2 \times A''_2}$, while $g' := pr_{A'_2}(g) \in \mathcal{F}_{A_1, A'_2}$ and $g'' := pr_{A''_2}(g) \in \mathcal{F}_{A_1, A''_2}$ denote its projections onto A'_2 and A''_2 , respectively. Then g is of the form $g(a_1) = (g'(a_1), g''(a_1))$. The mapping $\psi: \mathcal{F}_{A_1, A'_2 \times A''_2} \rightarrow \mathcal{F}_{A_1, A'_2} \times \mathcal{F}_{A_1, A''_2}$, $\psi(g)(a_1) := (g'(a_1), g''(a_1))$ is injective. Since $|\mathcal{F}_{A_1, A'_2 \times A''_2}| = (b'_2 \cdot b''_2)^{b_1} = b'^{b_1}_2 \cdot b''^{b_1}_2$ equals $|\mathcal{F}_{A_1, A'_2}| \cdot |\mathcal{F}_{A_1, A''_2}|$, the mapping ψ is bijective. In particular, if F' and F'' are (uniformly distributed) random mappings on $\mathcal{F}_{A_1, A'_2}$ and $\mathcal{F}_{A_1, A''_2}$, respectively, then $F' \times F''$ is a (uniformly distributed) random mapping on $\mathcal{F}_{A_1, A'_2 \times A''_2}$. Vice versa, if F is a (uniformly distributed) random mapping on $\mathcal{F}_{A_1, A'_2 \times A''_2}$, then the projections $pr_{A'_2}(F)$ and $pr_{A''_2}(F)$ onto the first and the second component are independent (uniformly distributed) random mappings on $\mathcal{F}_{A_1, A'_2}$ and $\mathcal{F}_{A_1, A''_2}$, respectively.

739 [scenario 3] Let $A_1 = A_2 = \{0, 1\}^n = A$ and $g, g_1, g_2 \in \mathcal{F}_{A,A}$. Further, $\text{id}: A \rightarrow A$ denotes the identity mapping. If we interpret the elements of A (bit strings) as binary representations of integers, then $\text{id} + g(\text{mod } 2^n) \in \mathcal{F}_{A,A}$ for each $g \in \mathcal{F}_{A,A}$. Obviously,

$$\begin{aligned} (a + g_1(a) \equiv a + g_2(a) \pmod{2^n} \quad \text{for all } a \in A) &\iff \\ (g_1(a) = g_2(a) \pmod{2^n} \quad \text{for all } a \in A) &\iff (g_1 = g_2). \end{aligned} \quad (4.230)$$

By (4.230) the mapping $\mathcal{F}_{A,A} \rightarrow \mathcal{F}_{A,A}$, $g \mapsto \text{id} + g(\text{mod } 2^n)$ is injective and thus also bijective. In other words,

$$\mathcal{F}_{A,A} = \text{id} + \mathcal{F}_{A,A}(\text{mod } 2^n). \quad (4.231)$$

In particular, if F is a (uniformly distributed) random mapping on $\mathcal{F}_{A,A}$, then $F^*(X) := X + F(X)(\text{mod } 2^n)$ is also a (uniformly distributed) random mapping on $\mathcal{F}_{A,A}$.

Note 1: The same conclusions apply to other group operations on $\{0,1\}^n$ than the addition modulo 2^n , too.

Note 2: The name “scenario 3” and “scenario 4” below is motivated by the fact that so far random mappings with $\gamma \gg 1$ and $(n \leq m, \text{ or } n \gtrsim m)$ have been analyzed.

[scenario 3, non-uniformly distributed input data] We use the definitions from par. 739 with X' in place of X . As usual, the random variable X' is not (necessarily) uniformly distributed on A . Since (4.231) does not depend on the distribution of the input data, this allows to simplify the problem. Instead of $X' \mapsto X' + F(X')(\text{mod } 2^n)$, an “ordinary” (uniformly distributed) random mapping F^* on $\mathcal{F}_{A,A}$ can be evaluated (here, $A = \{0,1\}^n$). This, in turn, allows to compute n^- by (4.225). Finally, par. 712 can be applied to (n^-, n) .

Note: The definition $\text{id} + F(\text{mod } 2^n)$ needs $n = m$. Thus, the argumentation has to start with non-uniformly distributed input data. After having “reduced” the problem to an “ordinary” (uniformly distributed) random mapping F^* , the conversion to uniformly distributed input data can be done as usual.

[scenario 4, uniformly distributed input data] Let $A_1 = A_2 = \{0,1\}^n = A$, $A_3 = \{0,1\}^k$ with $k < n$ while F_3 denotes a uniformly distributed random mapping on $\mathcal{F}_{A,A_3}$. As in par. 739, $\text{id}: A \rightarrow A$ denotes the identity mapping, The random variable X is uniformly distributed on A . In this paragraph, we analyze the random mapping $F = \text{id} + F_3(\text{mod } 2^n)$. (Note that F is not uniformly distributed on $\mathcal{F}_{A,A_3}$.) Again, we interpret A (bit strings) as binary representations of integers, and we identify A and A_3 with $\mathbb{Z}_{2^n}$ and $\mathbb{Z}_{2^k}$, respectively. For $b \in A$, let $C_z = \{z - 2^k + 1(\text{mod } 2^n), z - 2^k + 2(\text{mod } 2^n), \dots, z\}$. Obviously, C_z is the set of all possible pre-images of z for $x \mapsto x + g(x)(\text{mod } 2^n)$ for any mapping $g \in \mathcal{F}_{A,A_3}$, or equivalently, the set of possible solutions of the equation $z = x + F_3(x)(\text{mod } 2^n)$; cf. par. 670. Furthermore, for $b \in A$ we define the random variable $X_b := |\{x \in C_b \mid x + F_3(x) \equiv b \text{ mod } 2^n\}|$. Since X is uniformly distributed on C_b and $F_3(x)$ is uniformly distributed on A_3 for each $x \in A$, we conclude $X_b \sim B(2^k, 2^{-k})$. Unless k is very small, a case, which is not relevant for our purposes, X_b is approximately P_1 Poisson distributed for all $b \in A$ (cf. par. 712). Consequently, the subsequent steps in par. 712 can be carried out analogously. In particular, (4.197), (4.198), (4.199), (4.200), and (4.201) are valid. Finally, (4.202) applies with $\gamma = 1$. For given probability $p > 0$

$$\begin{aligned} \left(2^m \frac{e^{-1}(t+2)}{(t+1)!(t+1)} < p \right) &\implies (\text{Prob}(\max\{X_b \mid b \in A_2\} > t) < p) \iff \\ &\left(\text{Prob} \left(H_{\min}(F(X)) \geq -\log_2 \left(\frac{t}{2^n} \right) \right) \geq 1 - p \right). \end{aligned} \quad (4.232)$$

Note: Interestingly, the loss of entropy does not depend on the parameter k .

[scenario 4, continues par. 741, non-uniformly distributed input data] As in par. 741, we analyze the scenario $F = X' + F_3(\text{mod } 2^n)$ but unlike there, we assume that the input data is a realization of a random variable X' that is not uniformly distributed. For $x, y \in A$, we define the family of

random variables $\{Y_{x,y} \mid x, y \in A\}$. Formula (4.233) provides a useful inequality.

$$\begin{aligned}
 v_z &= \text{Prob}(X' + F_3(X') \equiv z \bmod 2^n) = \sum_{x \in C_z} \text{Prob}(X' + F_3(X') \equiv z \bmod 2^n, X' = x) = \\
 &\sum_{x \in C_z} \text{Prob}(X' = x) \text{Prob}(X' + F_3(X') \equiv z \bmod 2^n \mid X' = x) = \\
 &\sum_{x \in C_z} \text{Prob}(X' = x) \text{Prob}(x + F_3(x) \equiv z \bmod 2^n) \leq \\
 &\frac{p_{\max, X'}}{2^{-n}} \sum_{x \in C_z} \text{Prob}(X = x) \text{Prob}(x + F_3(x) \equiv z \bmod 2^n) = \\
 &\frac{p_{\max, X'}}{2^{-n}} \sum_{x \in C_z} \text{Prob}(X = x) \text{Prob}(X + F_3(X) \equiv z \bmod 2^n \mid X = x) = \\
 &\frac{p_{\max, X'}}{2^{-n}} \sum_{x \in C_z} \text{Prob}(X + F_3(X) \equiv z \bmod 2^n, X = x) = \\
 &\frac{p_{\max, X'}}{2^{-n}} \text{Prob}(X + F_3(X) \equiv z \bmod 2^n) =: v_{0,z} \\
 &\text{with } p_{\max, X'} = \max\{\text{Prob}(X' = x) \mid x \in A\}.
 \end{aligned} \tag{4.233}$$

Inequality (4.233) implies

$$v_z \leq \frac{p_{\max, X'}}{2^{-n}} v_{0,z} \quad \text{for all } z \in A, \text{ and thus} \tag{4.234}$$

$$\max\{v_z \mid z \in \{0, 1\}^n\} \leq \frac{p_{\max, X'}}{2^{-n}} \max\{v_{0,z} \mid z \in \{0, 1\}^n\}. \tag{4.235}$$

$$\begin{aligned}
 H_{\min}(F(X')) &= -\log_2(\max\{V_z \mid z \in \{0, 1\}^n\}) \geq -\log_2\left(\frac{p_{\max, X'}}{2^{-n}} \max\{V_{0,z} \mid z \in \{0, 1\}^n\}\right) = \\
 &H_{\min}(F(X)) + H_{\min}(X') - n.
 \end{aligned} \tag{4.236}$$

Formula (4.236) traces the problem back to the case of **uniformly distributed** random input data. As explained in par. 741, a lower **min-entropy** bound for $H_{\min}(F(X))$ can be computed with formula (4.232).

4.5 Stochastic Model, Online Test, Total Failure Test, and Start-up Test

- 743 **TRNGs** such as **PTRNGs** and **NPTRNGs** shall provide **information-theoretic security**. The degree of randomness can be quantified by the **entropy** of the generated random numbers.
- 744 The evaluation processes for **PTRNGs** and **NPTRNGs** are, however, very different.
- 745 The main reason is that **PTRNGs** use **physical noise sources**. A **physical noise source** exploits physical phenomena (thermal noise, shot noise, jitter, metastability, radioactive decay, etc.) from dedicated hardware designs (using diodes, ring oscillators, etc.) or physical experiments to produce digitized random data. Dedicated hardware designs can use general-purpose components

(like diodes, logic gates, etc.) if the designer is able to understand, describe, and quantify the characteristics of the design that are relevant for the generation of random numbers.

In contrast, **NPTRNGs** exploit **non-physical noise sources**. **Non-physical noise sources** typically exploit system data (interrupts, system time, etc.) and/or user interaction (e.g., mouse movement, key strokes) to produce digitized random data.

Different specimens of **physical noise sources** (e.g., within a chip series) are identically designed, and therefore their stochastic behavior may not be identical but essentially similar. In contrast, **non-physical noise sources** are not under the control of the designer. For **NPTRNGs** running on different platforms, the behavior might be very different.

The central task of **PTRNG** and **NPTRNG** evaluations is to provide a lower **entropy** bound per **internal random number** bit. For **PTRNGs**, **AIS 31** demands a so-called **stochastic model**. In **CC** evaluations, the evaluated **PTRNGs** are usually based on electronic circuits. Note: For **non-physical noise sources**, it is usually infeasible to develop and to verify a sound **stochastic model**. Therefore, **AIS 31** does not require **stochastic models** for **NPTRNGs**.

The **stochastic model** is the core of any **PTRNG** evaluation according to **AIS 31**. In Subsection 4.5.1, the concept is motivated and explained. In Subsection 4.5.2, the theoretical explanations are illustrated by an elementary example.

Subsections 4.5.3, 4.5.4, and 4.5.5 treat **online tests**, **total failure tests**, and **start-up tests**. **AIS 31** requires that the **online test** is tailored to the **stochastic model**. The **start-up tests** should also consider the **stochastic model**, while the **total failure tests** shall be based on a failure analysis of the **physical noise source**.

In the literature, **stochastic models** of many real-world **PTRNG** designs have already been studied. In Section 5.4, several **stochastic models** of real-world **physical noise sources** and two generic **stochastic models** are discussed, and references are provided.

4.5.1 Stochastic Model: Motivation and Definition

The random numbers delivered by a **PTRNG** to the consuming cryptographic application (**external random numbers**) shall be suitable. That is, they shall meet the security requirements or the assumptions of the **consuming application**. Usually, the **consuming application** expects that the **external random numbers** behave (at least approximately) like realizations of **iid** and **uniformly distributed** (i.e., **unbiased**) **random variables**. In the terminology of **AIS 31**, **internal random numbers** are the finalized random numbers ready for output that are still inside the **RNG security boundary**. The **external random numbers** are subsets (usually subsequences) of the generated **internal random numbers** that are passed to the requesting application outside the **security boundary** of the **RNG**.

The goal of an evaluation is to guarantee a lower **entropy** bound per bit of the **internal random numbers**.

Unfortunately, there do not exist effective (reliable) estimators or **black box** tests for the **entropy** of a given, arbitrary sequence of random numbers without further information, i.e., without stochastic assumptions on its distribution.

- 755 This is because **entropy** is a property of **random variables** but not of their realizations (here: random numbers); see Section 4.3, for example. For this purpose (verification of a lower **entropy** bound), functionality classes PTG.2 and PTG.3 of AIS 31 require a **stochastic model**.
- 756 We interpret the **raw random numbers** $r_1, r_2, \dots$ and the **internal random numbers** $y_1, y_2, \dots$ as realizations of **random variables** $R_1, R_2, \dots$ and $Y_1, Y_2, \dots$, respectively. Analogously, we interpret observable and measurement values of the **physical noise source** (if relevant for the development, justification, and verification of the **stochastic model**) as realizations of **random variables**, too. The **random variables** R_j and Y_j are discrete. The **random variables** R_j assume values in $\{0, 1\}$, $\{0, 1\}^k$, or $\mathbb{Z}$, while the Y_j are $\{0, 1\}^m$ -valued. Here, k and m denote suitable integers. The **random variables** that quantify the stochastic behavior of observables usually are real-valued.
- 757 [use of language] If there is no risk of misunderstanding, we loosely speak of the **entropy** per **raw random number**, per **raw random number** bit, per **internal random number**, etc. A more precise but more clumsy formulation, of course, would be “the **entropy** per corresponding **random variable**” or even better “the average gain of **entropy** per corresponding **random variable**” if dependent **random variables** are concerned.
Note: This applies to **Shannon entropy** and **min-entropy**.
- 758 A **stochastic model** provides a partial mathematical description (of the relevant properties) of a **physical noise source** using **random variables**. The **stochastic model** shall allow the verification of a lower **entropy** bound for the **internal random numbers** or for **intermediate random numbers** during the lifetime of the **PTRNG**, even if the quality of the **raw random numbers** goes down.
- 759 If there is no **post-processing**, the **raw random numbers** and the **internal random numbers** coincide. Formally, the **post-processing** algorithm can be interpreted as the identity mapping.
- 760 Of course, a precise analysis of the impact of a (DRG.3-compliant) **cryptographic post-processing** on the **entropy** per random bit is infeasible. Instead, the **cryptographic post-processing** algorithm often can be interpreted as a randomly selected mapping (with particular properties) or as a random permutation. Section 4.4 provides many results on random mappings, which can be useful for this purpose.
Note: This scenario is relevant for functionality classes PTG.3 and NTG.1. There, the input values to the **cryptographic post-processing** algorithm are called **intermediate random numbers**. They can be included as high-**entropy additional input** or in a **reseeding procedure**. It should be noted that functionality class PTG.2 also allows **post-processing** algorithms that apply a **cryptographic** function (stateful or stateless). Their analysis is similar to that of **cryptographic post-processing** algorithms for functionality class PTG.3.
- 761 For functionality classes PTG.2 and PTG.3, the first part of the evaluation (i.e., the **stochastic model**) is identical.
- 762 For “real-world” **PTRNGs**, the distribution of the underlying **random variables** $R_1, R_2, \dots$ and

$Y_1, Y_2, \dots$ (cf. par. 756) is usually unknown. To some degree, the distribution may vary over time, e.g., due to aging effects, changing environmental conditions, etc.; cf. par. 788 to 791.

[stochastic model, optimal case] Ideally, a stochastic model is a family of probability distributions that contains the true distribution of the raw random numbers (cf. 757) or of suitably defined auxiliary random variables during the lifetime of the PTRNG, even if the quality of the digitized data goes down. This family of distributions can depend on one or several parameters (typically, one to three). In Subsection 4.5.2 and in Subsections 5.4.2 to 5.4.6, such scenarios are discussed. 763

[stochastic model, non-optimal case] It may not always be possible to achieve the ambitious goal formulated in par. 763. It then suffices if the stochastic model describes and quantifies parts of the entropy contributions *provided that this allows the quantification of a lower entropy bound for the raw random numbers or auxiliary random variables (and, finally, for the internal random numbers)*. Of course, neglecting other contributions of randomness requires evidence that the neglected contributions do not decrease the entropy that is guaranteed by an isolated analysis of the focused contribution(s). Neglected contributions of randomness are not credited. Therefore, it can be the case that the entropy analysis is rather pessimistic in the non-optimal case. 764
Example: Assume that for a physical noise source, several effects contribute to the entropy of the raw random numbers and thus influence their distribution. It is permitted that the stochastic model focuses on one such contribution of randomness (e.g., on thermal noise) if evidence can be given that the other contributions (e.g., flicker noise) are independent of the focused one.

In the best case, the stochastic model would also provide a family of distributions that contain the true (but unknown) distribution of the internal random numbers, or more precisely, of the corresponding random variables $Y_1, Y_2, \dots$ 765

From an abstract point of view, the algorithmic post-processing transforms a stochastic model of the raw random numbers into a stochastic model of the internal random numbers. Under favorable circumstances, if the algorithmic post-processing is not too complicated, it may be possible to explicitly formulate and to analyze the “transformed” stochastic model. This is trivially the case, of course, in the absence of an algorithmic post-processing algorithm. Non-trivial examples are given in Subsections 5.1.1, 5.1.2, and 5.1.3 (XORing independent raw random numbers, von Neumann transformation, thinning out of homogeneous Markov chains). 766

However, in many cases the algorithmic post-processing algorithm (in combination with the distribution of the raw random numbers) is too complicated for an explicit formulation of a stochastic model for the internal random numbers. Par. 919 in Section 5.1 provides an elementary counterexample where the raw random number bits are XORed to the feedback value of an LFSR. Note: A sound and reliable “direct” verification of a stochastic model for the internal random numbers (without a careful analysis of the raw random numbers) is unlikely to be possible. 767

For these reasons AIS 20/31, *does not require a stochastic model of the internal random numbers*. Instead, AIS 31 demands a stochastic model of the raw random numbers (or of suitably defined auxiliary random variables), and on the basis of this stochastic model, a lower entropy bound for the internal random numbers shall be derived. 768

As stated above, it ultimately suffices to derive a lower entropy bound for the internal random numbers on the basis of the stochastic model. In the LFSR example from par. 919, the raw 769

random numbers are mapped 1-1 to the internal random numbers (depending on the initial state of the LFSR). Hence, the (average) entropy per bit is trivially the same for both the raw random numbers and the internal random numbers. On the other hand, even for iid $B(1, p)$ -distributed raw random numbers, it is hardly feasible to provide an explicit description of the distribution of the internal random numbers, unless $p = 0.5$.

- 770 In some PTRNG designs, the stage where the raw random numbers occur first may not be uniquely identifiable. In such cases, different interpretations are permitted, but we strongly recommend selecting an early stage because this usually simplifies the justification and verification of the stochastic model, see pars. 781, 782, 783, and 784.
- 771 As an alternative to a stochastic model for the raw random numbers, in some scenarios, it can be favorable to consider a stochastic model for suitably defined “auxiliary” random variables. If this stochastic model finally allows the derivation of a lower entropy bound for the internal random numbers, this approach is permitted. An example is discussed in Subsection 5.4.2.
- 772 Different specimens of a PTRNG design (e.g., PTRNGs on chips of some series) can, to different degree, behave differently. This may be caused by tolerances of components (of the physical noise source). Even the distribution of raw random numbers generated by a single PTRNG changes to some degree during its lifetime due variations of the environmental conditions (temperature or voltage, for example), and aging effects, for example.
- 773 [stochastic model, optimal case] In the optimal case (addressed in par. 763), the stochastic model shall contain all distributions that can occur under any (admissible) environmental condition and at any time for any specimen of the PTRNG that uses the design under consideration (usually running on essentially the same hardware). Different parameters correspond to different distributions. It is usually not too difficult to compute the entropy per raw random number or per auxiliary random variable for the admissible distribution, or for the corresponding set of parameters, respectively. Of course, also in the non-optimal case (addressed in 764) it must be possible to decide whether the entropy of the raw random numbers (and, finally, the entropy of internal random numbers) is indeed greater than or equal to a claimed lower entropy bound.
- 774 [stochastic model, optimal case] Assume that $\mathcal{S}_1$ denotes a stochastic model for some PTRNG, and that $\mathcal{S}_2$ is a superset of $\mathcal{S}_1$, i.e., that each distribution of $\mathcal{S}_1$ is also contained in $\mathcal{S}_2$. Then $\mathcal{S}_2$ is a stochastic model for this PTRNG, too.
- 775 The use of a “large” stochastic model (depending on many parameters) has both advantages and disadvantages. The advantage is that the verification of the stochastic model can become easier, and thus the proof that the true distribution(s) of the raw random numbers (for all specimens, under all conditions of use) are contained in the admissible set of distributions (Example: $\mathcal{S}_2 \cong$ Markovian model vs. $\mathcal{S}_1 \cong$ iid model). For the verification of the entropy bound, the larger stochastic model usually should not cause serious additional problems apart from the fact that the entropy estimation formula becomes more difficult, see par. 805. An obvious disadvantage of this approach is that the online test must cover a wider range of admissible distributions. This possibly reduces the effectiveness of the online test. In pars. 805 to 807, an example is discussed.
- 776 [Advantages of a stochastic model] It is a notable advantage of stochastic models that they (ideally) comprise (parametric) families of distributions. First of all, the justification/verification

that a whole class of distributions contains the true distribution is easier than showing that it matches with a particular single distribution. Moreover, as already pointed out in par. 762, even the distribution of a single PTRNG varies to some degree while the PTRNG is in operation. Finally, the distributions contained in a stochastic model usually allow a unified analysis since they only differ by their parameters. These advantages apply accordingly to the non-optimal case.

[defective states] In the optimal case, the stochastic model also contains distributions that correspond to defective states of the physical noise source that yield non-tolerable weaknesses of the internal random numbers (i.e., the entropy defects are too large). Also in the non-optimal case, defective states must be considered. In both cases, non-tolerable defective behavior must be detected when the PTRNG is in operation. Therefore, suitable online tests and total failure tests are required. Online tests and total failure tests are explained in Subsections 4.5.3 and 4.5.4. 777

The last mentioned feature of stochastic models in par. 776 concerns the efficient estimation of entropy. In a first step, the parameters of the true distribution are estimated on the basis of observed raw random numbers (or auxiliary random numbers). This parameter estimate is substituted into an entropy formula that fits the distribution of the stochastic model. This yields an estimate for the entropy per random number (or more precisely, per random variable). In Subsection 4.5.2, this procedure is illustrated by an elementary example (coin tossing). For example, par. 620 provides the Shannon entropy formula for homogeneous Markov chains while pars. 630 to 640 consider min-entropy. 778

Note: Entropy may be underestimated but must never be overestimated.

The experiments and the entropy estimations shall be performed under different environmental conditions. The evaluated prototypes shall meet the requirements of functionality class PTG.2 or functionality class PTG.3 under all admissible environmental conditions. 779

When the PTRNG is in operation, the online test shall guarantee that non-tolerable weaknesses of the random numbers lead to a noise alarm (see Subsection 4.5.2). 780

[verification of the stochastic model] Ultimately, (in the ideal case) a stochastic model is a claim that the random values (usually, raw random numbers) produced by some physical experiment or an electronic circuit follow a probability distribution that is contained in a specified family of distributions. For the non-optimal case, the situation is similar. There, the claim usually focuses on a particular entropy contribution (e.g., thermal noise). As already mentioned, the suitability and correctness of the stochastic model has to be justified and verified. 781

[verification of the stochastic model] The stochastic model shall be supported by technical arguments based on the design of the physical noise source and findings in the literature. This requires at least a qualitative understanding of the physical noise source. 782

[verification of the stochastic model] Empirical data gained from the physical noise source (analog data like voltage or timing variations, raw random numbers, etc.) shall be used to develop, confirm, and adjust the claimed stochastic model or subclaims thereof. Different environmental conditions (temperature, voltage, etc.) shall be considered. This may be done by statistical tests that are tailored to the physical noise source and the stochastic model. (These statistical 783

tests are applied *in addition* to the evaluator **black box** test suite T_{irn} , which is described in Subsection 4.6.2). The goal of these investigations is to increase the understanding of the **physical noise source** that is used by the **RNG** and support the confidence in the **stochastic model**. For very simple and theoretically well-understood designs (e.g., for the coin tossing example or if the **PTRNG** exploits certain physical experiments), the evaluator might leave out or at least reduce such experimental investigations.

- 784 [verification of the **stochastic model**] An interesting question is to what extent the **raw random numbers** depend on variations of the environmental conditions (e.g., temperature, voltage) and of the characteristics of the **physical noise source**. Such dependencies may be very different (and difficult to quantify). A high-resolution measurement of the power consumption, for example, might reveal correlations with the **raw random numbers**. The analysis of the **TOE** should consider the question of whether variations of some parameters can cause significant changes of relevant statistical properties of the **raw random numbers** because such a behavior might be exploited by an **adversary**.
- 785 The developer has to specify the allowed ranges of the environmental conditions. The evaluation shall verify that the **entropy** of the **internal random numbers** remains large enough as long as the parameters stay in the permitted ranges.
- 786 [stage of the **stochastic model**] Considering the **stochastic model** at an early stage of the random bit generation usually has the advantage that random data can still be clearly distinguished from ideal output in case of a significant **entropy** defect. Furthermore, the supporting technical rationale usually allows the evaluator to confirm that technical arguments predict the general shape of the distribution of random output.
- 787 [stage of the **stochastic model**] In contrast, assume that a **PTRNG** is analyzed at a stage where the random output is indistinguishable from ideally distributed output (e.g., after **cryptographic post-processing**) almost irrespective of the amount of true **entropy** contained therein. For example, random numbers could be (almost) pseudorandom. This cannot lead to a successful evaluation.
- 788 For non-**stationary** stochastic processes, the sound and trustworthy verification of a **stochastic model** and the estimation of the parameters is rather difficult and can be practically infeasible. **Stationary** behavior facilitates the tasks in an evaluation considerably, in particular, since many transformations maintain stationarity; cf. Section 4.1, pars. 587 to 589, and Section 5.4, for real-world examples. Stationarity means that the process behaves time-invariantly throughout the entire time. Demanding stationarity in a strict mathematical sense would be too restrictive since the parameters of the true distributions can vary to some degree over time (due to aging effects, varying environmental conditions, etc). Nevertheless, the property that *observing a **physical noise source** at one point in time is representative for other points in time* is an important prerequisite for a meaningful **PTRNG** evaluation. Therefore, this document requires that the **raw random numbers** (resp., auxiliary random numbers) belong to a “time-local **stationary**” stochastic process; cf. par. 789.
- 789 “**Time-local stationarity**” is an **AIS 31**-specific term. It means that the **raw random numbers** (resp., the auxiliary random numbers), or more precisely, the corresponding **random variables** can be viewed as **stationarily distributed** over “short” time-scales that are “large” compared to the sample size of the **online tests** and the evaluator tests (e.g., to estimate parameters). Within such

periods, the relevant distribution parameters shall change at most marginally, allowing to treat the **raw random numbers** (or more precisely, the underlying **random variables**) as **stationarily distributed**.

This approach takes advantage of the properties of (mathematical) stationarity but also takes into consideration that, for real-world **PTRNGs**, stationarity in a strict mathematical sense may not exist due to reasons, which have already been discussed above. **PTRNGs** can generate hundreds of kBits or even MBits of **raw random numbers** per second so that within a few seconds a very large amount of random numbers are generated.

The aging effects of the analog components may slowly change the distribution of the parameters. However, such effects are not relevant over short time scales. Transient effects on the parameters during the start-up of the **physical noise source** may be ignored provided that no **internal random numbers** are output. These **raw random numbers** can still be used to seed the **internal state** of the **algorithmic post-processing** algorithm (if it is stateful) and the **cryptographic post-processing** algorithm but no **entropy** is credited.

As already mentioned in par. 779, the parameters of the underlying distribution and, based on that, the **entropy** of the **raw random numbers** (or, the auxiliary random numbers) shall be estimated under different environmental conditions. Minor variations of the estimated parameters under changing environmental conditions are expected and tolerable as long as the **entropy** remains large enough. Even then, if the parameter estimates vary “significantly”, this might be a starting point for a fault attack on the **physical noise source** and thus should be considered in the overall evaluation of the **TOE** (cf. Section 2.1).

In the analysis of **PTRNGs**, we assume that the **adversary** knows the design details but does not have knowledge of any “**internal state**” of the **physical noise source**. The **adversary**, for instance, does not know the current analog state of a Zener diode.

4.5.2 Example: Stochastic Model for Coin Tossing

In this subsection, we illustrate the concept of a **stochastic model** by analyzing an elementary example. A single coin is tossed repeatedly by a human operator. For simplicity, we assume that this operator neither has the opportunity to cheat (i.e., to intentionally and successfully influence the outcome of the coin tossings, e.g., due to a minimum throw height of each valid coin toss) nor is interested in cheating. Furthermore, pars. 805 to 807 consider a Markovian model.

It should be noted that this experiment *would not be viewed as a PTRNG in the sense of AIS 31* because of the significant impact of the human operator. A **physical noise source** is based on underlying physical phenomena. However, coin tossing provides an appropriate example to illustrate the concept of **stochastic models**.

We identify the outcomes “head” and “tail” with “1” and “0”, respectively. We assume that the outcomes $x_1, x_2, \dots$ are realizations of binary-valued **random variables** $X_1, X_2, \dots$. In this example, these values are our **raw random numbers**.

Since a coin has no memory and since the physical structure of the coin remains identical (at least during reasonable time periods), we may assume that the **random variables** $X_1, X_2, \dots$ are **iid** $B(1, p)$ -distributed with unknown parameter p . In this example, the **stochastic model** is given by a one-parameter family of distributions.

Note: An alternative, more formal description of this **stochastic model** is given by $\{B(1, p)^n \mid p \in [0, 1]\}$. The n -fold product measure $B(1, p)^n$ describes the distribution of the random vector $(X_1, \dots, X_n)$.

798 This **stochastic model** does not only apply to a single coin but to any coin (even though with different parameters).

799 For real-world **PTRNGs**, the verification of the **stochastic model** is more difficult. In Section 5.4, several examples are discussed.

800 When tossing the coin N times, the mean value provides an estimate $\tilde{p}$ for the unknown parameter p

$$\tilde{p} := \frac{1}{N} \sum_{j=1}^N x_j. \quad (4.237)$$

The strong law of large numbers guarantees that the right-hand side converges “almost surely” to the parameter p as N tends to infinity.

801 By formula (4.45), the estimation error in (4.237) is $\leq \epsilon$ with probability $1 - 2\Phi(-2\epsilon\sqrt{N})$. Numerical example: For $(\epsilon, N) = (0.01, 10000)$, we have $-2\epsilon\sqrt{N} = -2.0$, and $2\Phi(-2.0) \approx 0.046$. Increasing N from 10.000 to 100.000 reduces the last term from 0.046 to 10^{-9} .

802 [Shannon entropy] Since the **random variables** $X_1, X_2, \dots, X_N$ are **iid**, $H(X_1, X_2, \dots, X_N) = NH(X_1)$. Substituting $\tilde{p}$ from (4.237) into the one-dimensional **Shannon entropy** formula yields the **entropy** estimate for $H(X_1)$,

$$\tilde{H}(X_1) := -(\tilde{p} \log_2(\tilde{p}) + (1 - \tilde{p}) \log_2(1 - \tilde{p})). \quad (4.238)$$

803 [Shannon entropy] Linear Taylor expansion of $H(X_1)$ (viewed as a function of p) gives an approximation of the estimation error

$$\tilde{H}(X_1) - H(X_1) \approx \Delta p (-\log_2(p) + \log_2(1 - p)) \quad (4.239)$$

where $\Delta p := \tilde{p} - p$.

804 [Min-entropy] Analogously to par. 802,

$$\tilde{H}_{min}(X_1) := -\log_2(\max\{\tilde{p}, 1 - \tilde{p}\}) \quad (4.240)$$

provides an estimator for the **min-entropy**.

805 [Markovian model] Now consider another **stochastic model** that assumes that the **random variables** $X_1, X_2, \dots$ form a homogeneous Markov chain with transition matrix $P = (p_{ij})_{0 \leq i, j \leq 1}$.

This **stochastic model** depends on two parameters, namely $p_{01} := \text{Prob}(X_{n+1} = 1 \mid X_n = 0)$ and $p_{10} := \text{Prob}(X_{n+1} = 0 \mid X_n = 1) \in [0, 1]$. The transition matrix P reads

$$P = \begin{pmatrix} 1 - p_{01} & p_{01} \\ p_{10} & 1 - p_{10} \end{pmatrix}. \quad (4.241)$$

If $p_{10} = 1 - p_{01}$, then both rows of P are identical, which means that the **random variables** $X_1, X_2, \dots$ are **iid**. In particular, the Markovian **stochastic model** is a superset of the **iid stochastic model** discussed above, and thus also is a valid **stochastic model** for the coin tossing experiment (cf. par. 774). Here, the **stochastic model** depends on two parameters. The set $\{(p_{01}, p_{10}) \mid 0 \leq p_{01}, p_{10} \leq 1\}$ specifies the admissible parameters.

Note: The subset $\{(p_{01}, p_{10}) \mid p_{10} = 1 - p_{01}, 0 \leq p_{01} \leq 1\}$ describes an **iid** model.

[Markovian model, continuation of par. 805] If $0 < p_{01} + p_{10} < 2$, the Markov chain is ergodic, and the distributions $\nu_1, \nu_2, \dots$ converge to the limiting distribution $\nu = (\frac{p_{10}}{p_{01} + p_{10}}, \frac{p_{01}}{p_{01} + p_{10}})$ (par. 581). The special cases $p_{01} = p_{10} = 0$ and $p_{01} = p_{10} = 1$ correspond to **noise sources** that generate constant **raw random number** bit sequences or alternating **raw random number** bit sequences $\dots, 0, 1, 0, 1, \dots$, respectively. If the Markov chain is in equilibrium (i.e., if X_m is ν -distributed), by (4.101), we obtain the conditional **Shannon entropy**

$$\begin{aligned} H(X_{m+1} \mid X_m) &= -\frac{p_{10}}{p_{01} + p_{10}} (p_{01} \log_2(p_{01}) + (1 - p_{01}) \log_2(1 - p_{01})) - \\ &\quad -\frac{p_{01}}{p_{01} + p_{10}} ((1 - p_{10}) \log_2(1 - p_{10}) + p_{10} \log_2(p_{10})). \end{aligned} \quad (4.242)$$

The conditional **Shannon entropy** (4.242) quantifies the average increase of **Shannon entropy** per **raw random number** bit.

Note 1: The gain of **entropy** by the next **raw random number** bit depends on the current random **raw random number** bit, thus on the first or on the second row of the transition matrix P .

Note 2: If **min-entropy** is claimed, pars. 630 to 640 in Section 4.3 can be useful.

[Markovian model, continuation of par. 806] For the Markovian **stochastic model**, not only one parameter p (as in the **iid** model) but two parameters p_{01}, p_{10} have to be estimated and then substituted into the corresponding **entropy** formula; cf. (4.242), (4.135), and (4.136).

4.5.3 Online Test

The task of the **online test** (of a **PTRNG**) is to detect **sufficiently soon** if the quality of the random numbers becomes too low (compared to the requirements of functionality classes PTG.2 or PTG.3) while the **PTRNG** is in operation. An effective **online test** is mandatory for functionality classes PTG.2 and PTG.3.

Ideally, the **stochastic model** contains all possible distributions of the **raw random numbers** (or, alternatively, of auxiliary **random variables**; cf. Subsection 5.4.2) that may occur during the lifetime of the **PTRNG**. As already pointed out in Subsection 4.5.2, even for a given **PTRNG** some variation of the parameters during its lifetime is normal, and different **PTRNGs** of the same type can differ to some extent. The **online test** shall detect if the true distribution has left the subset of appropriate parameters A_{good} (see pars. 810 and 811). All parameters in A_{good} provide

enough **entropy** (cf. PTG.2.2, resp. PTG.3.5). The **online test** can guess the parameters, or it can pursue the goal indirectly by a statistical test that fails with large probability if the true parameters leave the set A_{good} .

Note 1: Depending on the **entropy** claim, **entropy** means **Shannon entropy**, **min-entropy**, or both.

Note 2: If the **stochastic model** only covers partial aspects of the **physical noise source** (non-optimal case), the preceding applies accordingly.

- 810 Ideally, the **online test** would never fail if the true parameter(s) belong to A_{good} and always fail whenever the parameter(s) lie outside of A_{good} , i.e., if they lie in its complement A_{bad} . Fig. 17 illustrates the failure probabilities of an ideal test. Of course, this aim cannot be achieved because the discriminatory power of statistical tests (with finite sample size) is not infinite. Fig. 18 shows a more realistic picture.

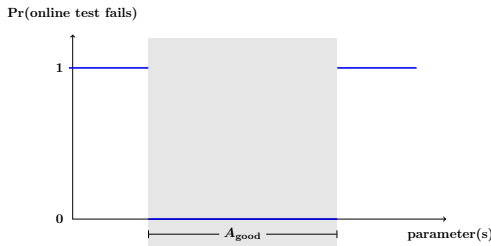


Figure 17: ideal **online test**: never fails if the true parameter(s) is in A_{good} and always fails otherwise

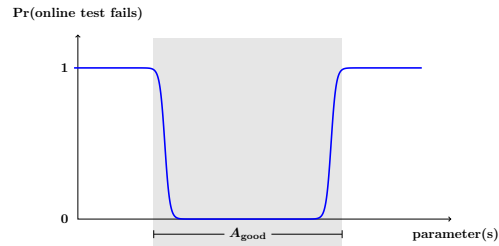


Figure 18: more realistic **online test**: significant failure probability near the border between A_{good} and A_{bad} , and large failure probability outside A_{good}

- 811 In Fig. 18, the failure probability for the parameter set A_{bad} is appropriate but also for certain parameters inside A_{good} (implicitly defined by PTG.2.2 or PTG.3.5), namely for the “border region”, the failure probability is rather large. This is not a security problem but affects the availability of the **PTRNG**.
- 812 For **PTRNG** evaluations, only the behavior of the **online test** on the set A_{bad} is relevant. In particular, the applicant has to give evidence that Requirement PTG.2.4 or PTG.3.7 (as appropriate), is fulfilled.
- 813 From a security point of view, the behavior of the **online test** on the set A_{good} is irrelevant. However, availability is an important feature of IT products. In pars. 814 to pars. 819, we formulate some thoughts about how to combine security (effectiveness of the **online test**) with availability (not too many “false” failures of the **online test**). In Section 5.5, concrete examples are discussed.
- 814 Often, the **entropy** of the **raw random numbers** is larger than required by functionality classes PTG.2 and PTG.3. Assume for the moment that during operation, the true parameters of all (properly working) specimens of the **PTRNG** design under evaluation are contained in a subset $A_{real} \subseteq A_{good}$, which contains “very good” parameters that parametrize “very good” distributions. From a security point of view, failures of the **online test** on the difference set

$A_{good} \setminus A_{real}$ are neither necessary nor harmful, and also the availability is not affected because these parameters should never occur for properly working PTRNGs.

If the designer assumes that for all (properly working) PTRNGs under consideration, the true parameters indeed always stay in such a subset $A_{real} \subseteq A_{good}$, this property can be utilized to design an effective online test. The applicant does not need to provide evidence that the true parameters of the PTRNG specimens are indeed always contained in the set A_{real} . Overly optimistic assumptions, however, may limit the availability of the PTRNG, but this is primarily an issue for the applicant to consider.

Of course, the smaller the set A_{real} (i.e., the larger the difference set $A_{good} \setminus A_{real}$), the easier it is to design an efficient (and effective) online test, which on the one hand detects sufficiently soon when the true parameter(s) leave A_{good} and on the other hand hardly reduces the availability of appropriate PTRNGs. Fig. 19 shows an example where the online test rarely fails if the true parameters are in A_{real} , while the failure probability is large for A_{bad} . Fig. 20 illustrates the relation between these subsets for a stochastic model that depends on two parameters (e.g., a Markovian model).

Note: A_{good} includes the green and the yellow area.

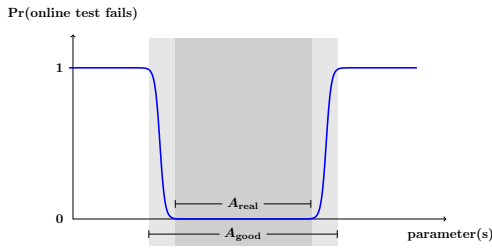


Figure 19: appropriate online test: low failure probability on A_{real} and large failure probability on A_{bad}

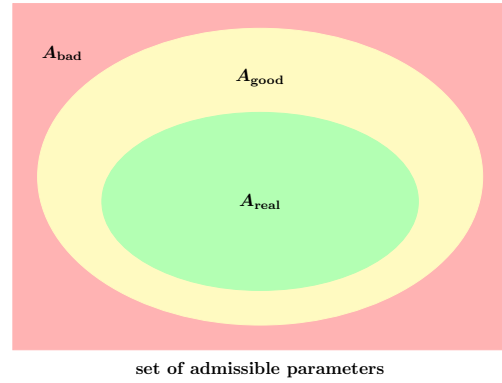


Figure 20: Relation between A_{real} , A_{good} , and A_{bad}

Of course, assuming a smaller subset A_{real} increases the technical requirements on the PTRNG design, including aging effects, tolerances of components, and the dependence on environmental conditions.

Note: As already mentioned in Chapter 3, the lower entropy bound defined in PTG.2.2 and PTG.3.5 could have been set even closer to 1. One reason for the chosen values of the bounds is to enable effective and efficient online tests.

[stochastic model, optimal case] There is a 1-1 correspondence between the admissible parameters of the stochastic model and the possible distributions of the raw random numbers (or, alternatively, to distributions of the auxiliary random variables). This allows the identification of parameters with distributions. We may view A_{real} as the (composite) null hypothesis of the online test, and the inappropriate parameters A_{bad} as the (composite) alternative hypothesis.

- 819 [stochastic model, non-optimal case] In the non-optimal case the **stochastic model** should parametrize the focused **entropy** contributions (e.g., thermal noise). Here, the set A_{bad} comprises all parameters for which it cannot be guaranteed that the (overall) **entropy** is greater than or equal to the specified threshold. The set $A_{real} \subseteq A_{good}$ is defined accordingly, and A_{good} again is the complement of A_{bad} .
- 820 [terminology] The **online test**, or more precisely, the **online test scheme** (synonymously, the **online test procedure**) consists of one or several statistical tests (applied simultaneously or consecutively when the **PTRNG** is in operation), evaluation rules, a calling scheme (cf. par. 834), and a specification of what happens if the **online test** fails (“consequences of a **noise alarm**”; cf. par. 837).
- 821 [terminology] If it is unambiguous, we alternatively use the term “**online test**” for the applied statistical test(s) but for simplicity also in place of “**online test scheme**” or “**online test procedure**”.
- 822 The developer shall provide evidence that the **online test** (i.e., the **online test scheme**) is appropriate, i.e., that it fulfills Requirement PTG.2.4 or PTG.3.7, respectively.
- 823 Which random numbers should be tested by the **online test**? This is a natural question. The general advice is to apply the **online test** to the **raw random numbers** even if the **stochastic model** considers “auxiliary” **random variables**. Principally, also the analog signals of the **physical noise source** can be tested.
- 824 If the **PTRNG** applies an **algorithmic post-processing** algorithm, AIS 31 principally allows **online tests** of the **internal random numbers**. However, usually the design of the **online test** and the verification of its effectiveness, if possible at all, are at least significantly more difficult if the **online test** is applied to the **internal random numbers**. Exceptions are possible if the transformed **stochastic model** can be described explicitly (cf. pars. 766 and 767) or if the **post-processing** can be reversed (most favorable case, cf. par. 826). However, in the first scenario, it should be more favorable to test the **raw random numbers**, cf. par. 831. It is easy to see that, at least if the **post-processing** algorithm is stateless, for each **online test** on the **internal random numbers**, an **online test** on the **raw random numbers** exists, that is at least as effective.
Note: From a formal point of view, the **online test** on the **internal random numbers** can then be interpreted as an **online test** on the **raw random numbers**.
- 825 **Online tests** on the output of a **cryptographic post-processing** algorithm (as a substitute for an **online test** on the **raw random numbers**) are meaningless and therefore not credited. In particular, such tests cannot substitute for **online tests** on the **raw random numbers**.
- 826 Example: Consider the **PTRNG** design described in par. 919 where the **raw random number** bits are XORed to the feedback value of an **LFSR**. Even if the **physical noise source** would fail completely, generating constant sequences of 0’s or 1’s (**total failure**; cf. Subsection 4.5.4), the statistical properties of the **internal random numbers** should pass common statistical tests (apart from linear complexity tests, of course) unless the **LFSR** is too short. Effective **online tests** on the **internal random numbers** have to reverse the effect of the **LFSR**, which means that finally the **raw random numbers** would be tested anyway.

827

The example in par. 826 underlines the general advice from par. 823 that the raw random numbers should be tested.

The online test shall be tailored to the stochastic model.

828

Example: A monobit test counts the number of 0's and 1's. A monobit test is appropriate for the stochastic model from par. 797 (coin tossing, iid model). The monobit test shall detect if the parameter p moves or lies outside the permitted area. Of course, a monobit test is not appropriate for the Markovian stochastic model that was discussed in pars. 805 to 807. Assume, for example, that $p_{01} = p_{10} = 0.1$. By (4.242), the conditional Shannon entropy $H(X_{m+1} | X_m) = 0.468$ is rather low although, by par. 806, the limiting distribution is $\nu = (0.5, 0.5)$. Hence, a monobit test would not detect this dramatic entropy defect. In a Markovian stochastic model, an appropriate online test must consider the transition probabilities.

829

Note: The other direction, selecting an online test that tests properties beyond the given stochastic model, is permitted. For an iid model, a poker test can be a suitable choice. Apart from the bias, it would detect (small) dependencies that are not covered by the stochastic model. Large dependencies should not occur, since otherwise the stochastic model would be inappropriate.

Note 2: It is not required that the online test detects weaknesses outside the stochastic model.

Generally, the more comprehensive a stochastic model, i.e., the more parameters it includes, the easier its verification. On the negative side, the specification of an effective and efficient online test can become more difficult.

830

Assume that a physical noise source generates iid $B(1, p)$ -distributed raw random number bits for which $|p - 0.5|$ is too large so that the entropy per bit is insufficient. To increase the entropy per random bit, non-overlapping pairs of raw random number bits are XORed (algorithmic post-processing). In this scenario, it would also be easy to formulate a stochastic model for the internal random numbers (cf. par. 766 and Subsection 5.1.1), because the transformed stochastic model is again iid. But even in this scenario, it is more favorable to test the raw random numbers in place of the internal random numbers: XORing non-overlapping pairs of raw random numbers reduces the sample size of the online test to 50%, and the algorithmic post-processing reduces the “distance” between A_{real} and A_{bad} , thereby reducing the discriminatory power of the online test.

831

Note: An example is discussed in Subsection 5.5.2.

The sample size of an online test is usually much smaller than the sample size of a typical evaluator test that is applied to some prototypes of the PTRNG during evaluation. Consequently, the discriminatory power of a single online test is much smaller. In Subsection 5.5.2, an online test (or, more precisely, an online test scheme) is discussed. One of its features is a “history variable” that compensates for this effect to some degree as it captures long-term effects that result from deviations of the expectation of the test value.

832

The designer is free to choose the online test as long as it fulfills its purpose. The online test can be a “normal” statistical test, an entropy estimator, or a parameter estimator, for example.

833

[calling schemes] Apart from the choice of an appropriate online test, the calling scheme is important for the effectiveness of the online test scheme. The online test can, for example, be applied to all raw random numbers, to the raw random numbers that are used to generate the

834

output data of the current **request**, or to the **raw random numbers** that precede those **raw random numbers** that are used to generate the current **request**. The developer has to justify that the selected method is appropriate in the given scenario.

- 835 [calling schemes] A bad (i.e., not acceptable) solution would be to apply an **online test** to a sample of random numbers and output (non-tested) random numbers that were generated much later. The reason is that the behavior of the **physical noise source** might have changed considerably after the last application of the **online test**.
- 836 [calling schemes] The scenario from par. 835 changes if the generated **raw random numbers** are tested with a suitable **online test**, and if the **internal random numbers** are securely stored. (Of course, the respective memory had to be protected against manipulation and readout all the time. This is not covered by AIS 31 itself but, of course, is a task within the overall evaluation; cf. Section 2.1.)
- 837 [**noise alarm** and **false positives**] Due to the nature of statistical tests, an **online test** could fail even if the **entropy** per bit is sufficiently large. In fact, failures would also occur for ideal **RNGs**. If the **online test** fails, this causes a **noise alarm**. Erroneous (or accidental) **noise alarms** occur with some probability. It is not obvious what should happen after a **noise alarm**.
- 838 [consequences of a **noise alarm**] Depending on the concrete scenario, different reactions to a **noise alarm** (triggered by the **online test**) can be appropriate. For example:
- a) The most rigorous measure, clearly, is to stop the output of random numbers forever.
 - b) The device could be subjected to an “emergency test” (without outputting **internal random numbers**). The emergency test would be used to determine whether the **noise alarm** was accidental or justified. In the first case, the **RNG** can again output random numbers after the emergency test has been passed, while in the second case, the emergency test should fail, too, and then further output of random numbers is permanently prohibited.
 - c) A human operator checks the quality of the **RNG** (typically by appropriate statistical tests) before further output is allowed.
 - d) **Noise alarms** may be logged, but the **RNG** continues the output of random numbers (cp. par. 471).
 - e) etc.

Which options are possible (and effective) depends on the concrete deployment environment where the **RNG** runs. The third option, for example, is only possible in specific scenarios.

- 839 [consequences of a **noise alarm**] The developer has to justify the suitability of the specified consequences of a **noise alarm**. This belongs to the evaluation of the **online test**.
- 840 [consequences of a **noise alarm**] It is not a valid option, of course, to just perform **online tests** until one **online test** is (accidentally) passed and then to continue as before.

4.5.4 Total Failure Test

The **total failure test** shall detect failures of the **physical noise source** that imply that the **entropy** per **raw random number** bit has decreased to (essentially) 0. A failed **total failure test** causes a **total failure alarm**. 841

The **total failure test** shall detect naturally occurring (usually permanent) **total failures** of the **physical noise source**. That is, the aim of a **total failure test** is testing for possible ways for the device to fail rather than detecting targeted attacks such as fault-injection attacks that cause a (usually transient) failure mode. **Total failure tests** might be able to detect targeted attacks, but in general additional countermeasures are required. If targeted attacks are relevant for the **TOE** in the intended usage, then potential attacks and the implemented countermeasures shall be analyzed within the overall evaluation of the **TOE**, cf. Section 2.1. 842

After a **total failure**, the **entropy** per **raw random number** bit is essentially 0. Hence, a **total failure** must be quickly detected, cf. pars. 849 ff. In particular, no weak **internal random numbers** shall be output when a **total failure** has been detected. 843

The developer shall give evidence that the implemented **total failure test** is appropriate. 844

A thorough failure analysis of the **physical noise source** is indispensable. This analysis shall clarify which failures are technically plausible, and their impact on the **raw random numbers** should be described. 845

Technically, the **total failure test** can be realized by sensors or by statistical tests. **AIS 31** does not specify any other requirements besides effectiveness. 846

In the simplest scenario, a **total failure** of the **physical noise source** implies constant sequences of **raw random numbers** (0's or 1's), e.g., due to a stuck flip flop. Of course, this behavior could be detected by the **online test** or by a statistical test that fails if the last (for example) 40 **raw random number** bits were constant. The choice of the threshold (e.g., 40) should consider the throughput of the **RNG** within its lifetime to prevent erroneous **total failure alarms**. A little bit more general is the repetition count test defined in [SP800-90B], Subsection 4.4.1, which can handle **bit strings** in place of bits. 847

Depending on the **physical noise source** and its **digitization** mechanism, it is possible that a **total failure** may lead to more complicated error patterns, e.g., alternating sequences $\dots, 0, 1, 0, 1, \dots$ or even sequences containing some remaining noise. For example, if a Zener diode fails, the analog-to-digital converter may yet receive some noise from an amplification circuit. Despite some remaining **entropy**, a failure of the Zener diode still constitutes a **total failure** of the **noise source**. 848

Immediately after a **total failure** has been detected, no “weak” **internal random numbers** (containing only little **entropy**) shall be output. In particular, if the **post-processing** algorithm of the **PTRNG** belongs to one of the following types: 849

- (i) no **post-processing** algorithm,
- (ii) simple **algorithmic post-processing** algorithm (without memory),
- (iii) **post-processing** by universal families of hash functions, or
- (iv) stateless **post-processing** algorithm using a **cryptographic** function,

this means that no **internal random numbers** shall be output that have been generated after a **total failure**. For

- (v) **algorithmic post-processing with memory**

exceptions are possible; cf. par. 853.

- 850 [relaxations] Detecting a **total failure** immediately, is a great challenge. By suitable design features/security measures, the permitted delayed detection and reaction time of a **total failure** can be extended, thereby relaxing the requirements on the **total failure test**. This, in particular, concerns FIFO buffers (par. 852) and **cryptographic post-processing** algorithms (par. 855). Under suitable conditions, this also applies to **algorithmic post-processing** algorithms (pars. 853 and 854). All design features can be combined.
- 851 [detection mechanisms] A dedicated statistical **total failure test** (based on the failure analysis of the **physical noise source**), e.g., checking whether the last t **raw random number** bits have been identical, should be considered instead of using the **online test** (applied with a specific rejection area that is adjusted to the **total failure** case) because the sample size of the **online tests** is usually much larger than the sample size of a dedicated **total failure test**. However, usually an immediate detection of a **total failure** can most effectively be attained by sensors or similar technical measures but not by statistical tests.
- 852 [FIFO buffers] Assume that (i) the **internal random numbers** are stored in a FIFO buffer before they are output, and that (ii), by design, always at least t **internal random number** bits are stored in the FIFO buffer. In this case, it suffices to stop the output of **internal random numbers** at the latest after t **internal random number** bits have been generated since the **total failure** has occurred. Compared to par. 849, this relaxes the requirement that the **total failure** has to be detected immediately.
- 853 [**algorithmic post-processing**] If a stateful **algorithmic post-processing** algorithm is applied, under suitable conditions its **internal state** can be viewed as a buffer (similar to par. 852), increasing the permitted delayed detection time. This requires careful analysis.
 Example 1: Assume that the PTRNG applies the **LFSR post-processing** design in par. 919 with a t -bit **internal state**. If the **LFSR** generates binary-valued **internal random numbers**, further output has to be prevented at the latest after t **raw random number** bits have been generated since the **total failure**. If the **LFSR** outputs k -bit **internal random numbers**, the threshold of t **raw random number** bits decreases to $t - k + 1$ because, in the worst-case, the **total failure** occurs when the last **raw random number** bit needed for an **internal random number** was generated.
 Example 2: Assume that the **internal state** consists of 6 bits and that 6 fresh **raw random numbers**

are XORed to the **internal state** before the updated **internal state** is multiplied by a 4×6 -matrix A over $\text{GF}(2)$. The result of the matrix-vector-multiplication is the next **internal random number** (4 bits). Here, no **internal random number** shall be output that have been generated after a **total failure**; cf. par. 849.

[FIFO, **algorithmic post-processing**] In this paragraph, we assume that a FIFO buffer is always filled with at least t bits. If no **post-processing** algorithm or a stateless **algorithmic post-processing** is applied, the permitted delayed detection time is essentially determined by the FIFO. However, data-compression can be taken into account to compute a threshold in terms of **raw random numbers** that can at most be generated since a **total failure**. 854

Example: Assume that the FIFO is always filled with at least t **internal random numbers** bits and that the **PTRNG** XORs non-overlapping pairs of **raw random number** bits. Then the permitted delayed detection time increases from t to $2t - 1$ **raw random number** bits.

[**cryptographic post-processing**] Assume that the **PTRNG** applies a **cryptographic post-processing** algorithm (**with memory**) that, viewed as a **DRNG**, belongs to functionality classes DRG.2 or DRG.3. This scenario applies to PTG.3-compliant **PTRNGs** but also to particular PTG.2-designs. In case of a **total failure**, the **effective internal state** can be used as an “**entropy** buffer”. This is justified by the intuitive idea that the **internal state** should have accumulated **entropy** since the start of the **PTRNG**. Precise evaluation rules are explained in pars. 414 to 421. 855

4.5.5 Start-up Test

When the **PTRNG** has been started, a so-called **start-up test** (a.k.a. **self test**) is performed. The **start-up test** shall test for **total failures** and severe statistical weaknesses. 856

For these reasons, the **start-up test** shall be tailored to the **PTRNG**. Usually, a reasonable choice is to apply the **online test** (possibly with a different evaluation rule). 857

4.6 Statistical Tests and Predictors

Statistical tests are used as **online tests**, **start-up tests**, and partly as **total failure tests**; cf. Subsections 4.5.3, 4.5.5, and 4.5.4. In Section 4.6, the use of statistical tests and **predictors** within the evaluation process of **PTRNGs** or as part of the **black box** test suites is considered. 858

Section 4.6 is structured as follows: Subsection 4.6.1 summarizes basic facts and properties of statistical tests and **predictors**. Subsection 4.6.2 provides generic advice how to use statistical tests or **predictors** in the evaluation of the **stochastic model**. Finally, in Subsection 4.6.3, several **black box** tests and **black box predictors** are specified, which are prescribed by the Requirement PTG.2.6. 859

4.6.1 General Considerations

- 860 First, we recall some basics from statistics that should ease the reading of this section. Readers who are familiar with these concepts may skip the next paragraphs and proceed with pars. 871 ff., which treat **predictors**.
- 861 In conclusive statistics, the observed data (in our context: **raw random numbers** or **internal random numbers**) are interpreted as realizations of **random variables** with unknown distributions. Usually, it is assumed that the (unknown) true distribution is contained in a set of admissible hypotheses Θ (a collection of admissible probability distributions), which splits into the null hypothesis H_0 and its complement in Θ , the alternative hypothesis H_A .
- 862 A test value c is interpreted as a realization of a **random variable** C . The distribution of C depends on the statistical test and, of course, on the distribution of the tested random numbers, or more precisely: on the distribution of the underlying **random variables**.
- 863 For **online tests** and statistical tests in the evaluation process that are tailored to the **physical noise source**, single null hypotheses H_0 are not appropriate because real-world **RNGs** are not ideal. For this reason, functionality classes PTG.2 and PTG.3 tolerate, to some degree, statistical deficiencies of the **raw random numbers**, and functionality class PTG.2 also for the **internal random numbers**. In the presence of a **stochastic model**, H_0 and H_A can be identified with subsets of the set of admissible parameters (optimal case). These aspects have already been discussed in Subsection 4.5.3 in the context of **online tests**.
- 864 If a statistical test fails, the null hypothesis is rejected. On the other hand, even if the test value is not very unlikely, this does not confirm the null hypothesis. *Statistical tests cannot confirm the null hypothesis*. But, absence of evidence is not absence of knowledge. The statistician decides under consideration of the number of performed tests whether he continues or stops further testing. The criteria that cause the end of statistical testing depend on the aimed assurance that no deviation of the **RNG** from the null hypothesis H_0 has been found.
- 865 The evaluator decides on the basis of test values whether the true distribution is not contained in the null hypothesis. Roughly speaking, the null hypothesis is rejected if the test data indicate that the null hypothesis is sufficiently unlikely. One can select between two approaches to define the probability of type 1 error, yielding a **false positive**.
- 866 Predefined level of significance: The significance level α is selected before the experiments are conducted. If the test value c is contained in the critical set K_α , the statistician (evaluator) rejects the null hypothesis. By definition, the significance level α of a test (with given null hypothesis H_0) is defined by

$$\alpha = \sup_{C \in H_0} \text{Prob}(C \in K_\alpha) . \quad (4.243)$$

In other words: The significance level α equals the largest probability (if a maximum exists) among all hypotheses in H_0 that the null hypothesis is rejected (although it is true).

- 867 The significance level α should be selected with regard to the test scenario.
Note: For many (non-cryptographic) statistical applications, $\alpha = 0.05$ and $\alpha = 0.01$ are typical

significance levels.

The evaluator may commit two types of errors. Error type 1 gives a **false positive** while error type 2 yields a false negative. Table 5 illustrates the situation. 868

Table 5: Statistical tests: error types

	Reality	
	Null hypothesis is true	Null hypothesis is false
Test rejects the null hypothesis H_0	Error Type 1 (false positive) (with probability α) $\text{Prob}(C \in K_\alpha \mid H_0) \leq \alpha$	correct decision (with probability $1 - \beta$) $\text{Prob}(C \in K_\alpha \mid H_A) \geq 1 - \beta$
Test does not reject the null hypothesis H_0	correct decision (with probability $1 - \alpha$) $\text{Prob}(C \notin K_\alpha \mid H_0) \geq 1 - \alpha$	Error Type 2 (false negative) (with probability β) $\text{Prob}(C \notin K_\alpha \mid H_A) \leq \beta$

p-value approach: The **p-value** quantifies the probability that the test value is at least as extreme as the value that has just been observed (tail probability) if the null hypothesis is true. 869

Example: Assume that the random variables $B_1, B_2, \dots, B_N$ are **iid** $B(1, p')$ -distributed with unknown parameter p' . For simplicity, we assume $H_0 = \{p' = 0.5\}$ and $H_A = \{p' \in [0, 1] \mid p \neq 0.5\}$. For $p' = 0.5$, (4.45) yields $\text{Prob}\left(\left|\frac{1}{N} \sum_{j=1}^N X_j - 0.5\right| > \epsilon\right) = 2\Phi\left(-2\epsilon\sqrt{N}\right)$. For $N = 10.000$ and $p' = 0.5 + 0.012$, for example, the right-hand equals $2\Phi(-2.4) = 0.016$. Hence, $p(0.512) = p(488) = 0.016$.

If the **p-value** is smaller than a pre-defined bound, the statistician rejects the null hypothesis or he continues testing. 870

In the remainder of Subsection 4.6.1, $x_1, x_2, \dots$ denotes a sequence of random numbers that assume values in the state space Ω . These random numbers can be single bits, integers, or bit vectors. Then $\Omega = \{0, 1\}$, $\Omega = \{0, 1\}^m$, or $\Omega \subseteq N_0$. 871

Note: In our context, $x_1, x_2, \dots$ usually denotes a sequence of **raw random numbers** or **internal random numbers**.

A mapping

$$\phi: \Omega^r \rightarrow \Omega^s, \quad (x_t, \dots, x_{t+r-1}) \mapsto \phi(x_{t+r}, \dots, x_{t+r+s-1}), \quad r \in \mathbb{N}_0 \cup \{\infty\}, s \in \mathbb{N} \quad (4.244)$$

is called an (r, s) -**predictor**.

The goal of an (r, s) -**predictor** is to predict the (unknown) next s elements $x_{t+r}, \dots, x_{t+r+s-1}$ on the basis of $x_t, \dots, x_{t+r-1}$. The prediction $\phi(x_t, \dots, x_{t+r-1})$ is *successful* iff $\phi(x_t, \dots, x_{t+r-1}) = (x_{t+r}, \dots, x_{t+r+s-1})$ (correct prediction). 873

Note: The parameter $r = \infty$ means that the **predictor** considers the whole previous history, i.e., $x_1, \dots, x_{t+r-1}$. In contrast, parameter $r = 0$ means that the **predictor** does not take previous values into account at all.

- 874 Assume that an **adversary**/evaluator successively applies a (fixed) (r, s) -**predictor** $\phi(\cdot)$ to predict the random numbers $x_{n+1}, x_{n+2}, \dots, x_{n+m}$ while $\delta(\cdot, \cdot)$ denotes Dirac's delta function, that is, $\delta(u, v) = 1$ if $u = v$, and $= 0$, else. Equations (4.245) and (4.246) define the relative success rates for overlapping and non-overlapping **predictors**.

$$\begin{aligned} \text{success rate (overlapping guesses)} = \\ \frac{1}{m-s+1} \sum_{j=1}^{m-s+1} \delta(\phi(x_{n+j-r}, \dots, x_{n+j-1}), (x_{n+j}, \dots, x_{n+j+s-1})) \end{aligned} \quad (4.245)$$

$$\begin{aligned} \text{success rate (non-overlapping guesses)} = \\ \frac{1}{m/s} \sum_{j=1}^{m/s} \delta(\phi(x_{n+(j-1)s-r+1}, \dots, x_{n+(j-1)s}), (x_{n+(j-1)s+1}, \dots, x_{n+j s})) . \end{aligned} \quad (4.246)$$

Note: In (4.246), it is assumed that m is a multiple of s .

- 875 In the remainder of this subsection, we assume that $x_1, x_2, \dots$ are realizations of **stationarily distributed random variables** $X_1, X_2, \dots$. The **random variables** $X_1, X_2, \dots$ are binary-valued, integer-valued, or binary vector-valued. By par. 588 a), b), and par. 587 c), the sequences of random vectors $Y_1 := (X_1, \dots, X_k), Y_2 := (X_2, \dots, X_{k+1}), \dots$ (overlapping k -tuples), $Y_1 = (X_1, \dots, X_k), Y_{k+1} = (X_{k+1}, \dots, X_{2k}), \dots$ (non-overlapping k -tuples), and $Z_1 := Y_1, Z_2 := Y_{t+1}, Z_3 := Y_{2t+1} \dots$ (regularly thinned out stochastic process) are also **stationarily distributed**.
- 876 In the remainder of this subsection, we assume that $r < \infty$ and $k = r + s$. To the **predictor** $\phi: \Omega^r \rightarrow \Omega^s$, we assign the function

$$f_\phi: \Omega^k \rightarrow \mathbb{R}, \quad f_\phi(u_1, \dots, u_{r+s}) := \delta(\phi(u_1, \dots, u_r), (u_{r+1}, \dots, u_{r+s})). \quad (4.247)$$

Furthermore, we assume that the stationary processes $(X_n)_{n \in \mathbb{N}}$, $(Y_n)_{n \in \mathbb{N}}$, and $(Z_n)_{n \in \mathbb{N}}$ (with $t = s$) are ergodic.

Note: The ergodicity assumption is not very restrictive. If the ergodicity assumption is not satisfied, it does make sense to test sequences of random numbers.

- 877 The stationarity and ergodicity of the stochastic processes $(Y_n)_{n \in \mathbb{N}}$ and $(Z_n)_{n \in \mathbb{N}}$ imply

$$\begin{aligned} & \lim_{m \rightarrow \infty} \frac{1}{m-s+1} \sum_{j=1}^{m-s+1} \delta(\phi(x_{n+j-r}, \dots, x_{n+j-1}), (x_{n+j}, \dots, x_{n+j+s-1})) \\ &= \lim_{m \rightarrow \infty} \frac{1}{m-s+1} \sum_{j=1}^{m-s+1} f_\phi(x_{n+j-r}, \dots, x_{n+j-1}, x_{n+j}, \dots, x_{n+j+s-1}) \\ &= E(f_\phi(X_1, \dots, X_{r+s})) \quad \text{with probability 1,} \end{aligned} \quad (4.248)$$

and similarly,

$$\begin{aligned}
& \lim_{m \rightarrow \infty} \frac{1}{m/s} \sum_{j=1}^{m/s} \delta(\phi(x_{n+(j-1)s-r+1}, \dots, x_{n+(j-1)s}), (x_{n+(j-1)s+1}, \dots, x_{n+js})) \\
&= \lim_{m \rightarrow \infty} \frac{1}{m/s} \sum_{j=1}^{m/s} f_{\phi}(x_{n+(j-1)s-r+1}, \dots, x_{n+(j-1)s}, x_{n+(j-1)s+1}, \dots, x_{n+js}) \\
&= E(f_{\phi}(X_1, \dots, X_{r+s})) \quad \text{with probability 1.}
\end{aligned} \tag{4.249}$$

Note: Equations (4.248) and (4.249) show that asymptotically the success rate is the same for overlapping and non-overlapping **predictors**.

The *optimal* (r, s) -**predictor** $\phi_{opt}(\cdot)$ assigns to $(x_t, \dots, x_{t+r-1})$ an s -tuple $(x_{t+r}^*, \dots, x_{t+r+s-1}^*)$ that maximizes the conditional probability 878

$$\text{Prob}(X_{t+r} = x_{t+r}, \dots, X_{t+r+s-1} = x_{t+r+s-1} \mid X_t = x_t, \dots, X_{t+r-1} = x_{t+r-1}). \tag{4.250}$$

By the definition of **min-entropy**, for given $x_t, \dots, x_{t+r-1}$ the maximal conditional probability equals

$$2^{-H_{\min}(X_{t+r}, \dots, X_{t+r+s-1} \mid X_t = x_t, \dots, X_{t+r-1} = x_{t+r-1})}. \tag{4.251}$$

In particular, (4.248) and (4.249) are valid for the optimal **predictor** ϕ_{opt} . To simplify notation, let for the moment 879

$$\begin{aligned}
cp^*(x_1, \dots, x_r) &:= \\
&\max_{x'_{r+1}, \dots, x'_{r+s} \in \Omega} (\text{Prob}(X_{r+1} = x'_{r+1}, \dots, X_{r+s} = x'_{r+s} \mid X_1 = x_1, \dots, X_r = x_r))
\end{aligned} \tag{4.252}$$

Using this notation, we conclude

$$\begin{aligned}
E(f_{\phi_{opt}}(X_1, \dots, X_{r+s})) &= \\
&\sum_{x_1, \dots, x_r \in \Omega} \text{Prob}(X_1 = x_1, \dots, X_r = x_r) cp^*(x_1, \dots, x_r) = \\
&\sum_{x_1, \dots, x_r \in \Omega} \text{Prob}(X_1 = x_1, \dots, X_r = x_r) \cdot 2^{-H_{\min}(X_{r+1}, \dots, X_{r+s} \mid X_1 = x_1, \dots, X_r = x_r)} = \\
&2^{-H_{\min(av)}(X_{r+1}, \dots, X_{r+s} \mid X_1, \dots, X_r)}.
\end{aligned} \tag{4.253}$$

The expected success probability of any (r, s) -**predictor** is limited by the right-hand side of (4.253), which is a function of the average conditional **min-entropy** $H_{\min(av)}(X_{r+1}, \dots, X_{r+s} \mid X_1, \dots, X_r)$.

From (4.253) and (4.114) it follows immediately that 880

$$2^{-H_{\min(av)}(X_{r+1}, \dots, X_{r+s} \mid X_1, \dots, X_r)} = E(f_{\phi_{opt}}(X_1, \dots, X_{r+s})) \leq 2^{-H_{\min}(X_{r+1}, \dots, X_{r+s} \mid X_1, \dots, X_r)}. \tag{4.254}$$

[Example] Let $X_1, X_2, \dots$ denote a homogeneous Markov chain with transition matrix 881

$$P = \begin{pmatrix} 0.5 & 0.5 \\ 0.7 & 0.3 \end{pmatrix} \tag{4.255}$$

and state space $\{0, 1\}$. The limiting distribution ν is given by $\nu = (\frac{7}{12}, \frac{5}{12})$. We assume that the Markov chain is in equilibrium, i.e., that X_1 has distribution ν . The optimal (1, 1)-**predictor** ϕ_{opt} is given by $\phi_{opt}(0) = 1$ and $\phi_{opt}(1) = 0$. (Of course, $\phi_{opt}(0) = 0$ would be fine, too.). This yields the corresponding function $f_{\phi_{opt}}$,

$$f_{\phi_{opt}}(0, 0) = f_{\phi_{opt}}(1, 1) = 0, \quad f_{\phi_{opt}}(0, 1) = f_{\phi_{opt}}(1, 0) = 1. \quad (4.256)$$

Substituting into (4.110), (4.111), and (4.112) yields

$$H_{\min}(X_2 | X_1) = -\log_2(0.7) \approx 0.515 \quad (4.257)$$

$$H_{\min(av)}(X_2 | X_1) = -\log_2\left(\frac{7}{12} \cdot 0.5 + \frac{5}{12} \cdot 0.7\right) \approx 0.778 \quad (4.258)$$

$$H_{\min(av, alt)}(X_2 | X_1) = \frac{7}{12} \cdot (-\log_2(0.5)) + \frac{5}{12} \cdot (-\log_2(0.7)) \approx 0.798. \quad (4.259)$$

Finally,

$$E(f_{\phi_{opt}}(X_1, X_2)) = \frac{7}{12} \cdot 0.5 + \frac{5}{12} \cdot 0.7 \approx 0.583 = 2^{-H_{\min(av)}(X_2 | X_1)} < 2^{-H_{\min}(X_2 | X_1)} = 0.7. \quad (4.260)$$

882 [Example] Let $X_1, X_2, \dots$ denote a homogeneous Markov chain with transition matrix

$$P = \begin{pmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{pmatrix} \quad (4.261)$$

and state space $\{0, 1\}$. The limiting distribution ν is given by $\nu = (\frac{1}{2}, \frac{1}{2})$. As in the example from par. 881, we assume that the Markov chain is in equilibrium.

a) The optimal (0, 1)-**predictor** ϕ_{opt} is given by $\phi_{opt} = 1$. (Of course, $\phi_{opt} = 0$ would be fine, too.). The success probability is 0.5.

b) The optimal (1, 1)-**predictor** ϕ_{opt} is given by $\phi_{opt}(x) = x$. Here, the success probability is 0.9.

c) The optimal (2, 1)-**predictor** ϕ_{opt} is given by $\phi_{opt}(x_1, x_2) = x_2$. Here, the success probability is 0.9, too.

Note: (0, 1)-**predictors** are useful only if the **random variables** $X_1, X_2, \dots$ are **iid**. For (one-step) Markov chains considering more than one predecessor does not increase the success probability.

4.6.2 Verification of the Stochastic Model

883 The verification of the **stochastic model** is the central part of an evaluation of a **PTRNG**; cf. Subsection 4.5.3 and in particular pars. 781 ff. This comprises technical arguments based on the design of the **physical noise source** and findings in the literature.

884 Additionally, the developer / applicant for a certificate should propose statistical tests and parameter estimators that are tailored to the **stochastic model**. (Of course, the evaluator can apply additional statistical tests or **predictors** if he feels that this would be reasonable.) This can concern the **raw random numbers** or claims and assumptions on the analog data that lead to the **stochastic model**. The latter could be properties of the analog signal, e.g., intermediate times

between events such as consecutive 0-1-crossings (cf. Subsection 5.4.2) or a claimed distribution of jittered cycle lengths.

Because the applied statistical test shall be tailored to the concrete design, it is not possible to provide comprehensive advice that could serve as a ready-to-use checklist. 885

Aside from statistical tests, the evaluator can apply **predictors**. **Predictors** cannot verify a **stochastic model** in a strict sense but they can possibly falsify it. This would be the case if, for given (estimated) parameters, some **predictor** turned out to be (significantly) more successful than indicated by (4.253). Of course, the evaluator may apply AI-based **predictors**. 886
Note: In the best case, statistical tests and **predictors** can increase the confidence in the argumentation and the **stochastic model**.

Example: The **stochastic model** says that the **raw random numbers** form a homogeneous Markov chain on a finite state space Ω . The statistical tests and the **predictors** should be applied to the **raw random numbers**. They should be selected such that they are able to falsify or “confirm” the Markov property. **Predictors** that exploit several preceding bits should not be more successful than the optimal **predictors** that only use the immediate predecessor; in particular, their success probability should not significantly exceed the right-hand side of (4.253). If this should be the case, the Markov assumption is obviously wrong. Similarly, transition probabilities from the last two bits should not depend on the last but one bit. 887

Example: a) Assume that the **raw random numbers** $r_1, r_2, \dots$ can be viewed as realizations of time-locally **stationarily distributed** binary-valued **random variables** $R_1, R_2, \dots$. As in par. 875, the random vectors $V_1 = (R_1, \dots, R_s), V_2 = (R_{s+1}, \dots, R_{2s}), \dots$, the random vectors $V_1, V_{k+1}, \dots$, and the random vectors $V_1, V_{t+1}, \dots$ can be viewed as time-locally **stationarily distributed** as well. These random vectors can be statistically tested and targets of **predictors**. In general, these random vectors are not **iid**. 888

b) For **iid random variables** $X_1, X_2, \dots$, it is $\text{Prob}(A \cap B) - \text{Prob}(A) \cdot \text{Prob}(B) = 0$ for all events A and B for which A only depends on $X_1, \dots, X_n$ while B only depends on $X_{n+1}, X_{n+2}, \dots$. For non-**iid random variables**, this is usually not the case.

c) Additionally to Example b), we assume that there is an integer $h > 0$ such that, loosely speaking, $\sup_{A,B} |\text{Prob}(A \cap B) - \text{Prob}(A) \cdot \text{Prob}(B)| \approx 0$ where the supremum is calculated over all events A and B for which A only depends on $X_1, \dots, X_n$ while B only depends on $X_{n+h}, X_{n+h+1}, \dots$. For $k = s$ and $t \geq s + h$, the random vectors $V_1 = (R_1, \dots, R_s), V_{t+1} = (R_{t+1}, \dots, R_{t+s}), \dots$ are time-locally **stationarily distributed** and “almost” **iid** (but not necessarily their components). An **iid** assumption usually simplifies the computation of the test statistics. If $R_1, R_2, \dots$ forms a homogeneous Markov chain with transition matrix P , for example, this does usually not imply that $h = 2$ fulfills the above condition. In fact, the transition from step n to step $n + 2$ is quantified by the transition matrix P^2 .

4.6.3 Statistical Tests and Predictors

Statistical **black box** tests and **black box predictors** only play a supplementary role for the evaluation of **PTRNGs**. The application of statistical **black box** tests and **black box predictors** can be seen as a sanity check of the **stochastic model**. If statistical **black box** test suites fail or 889

if **black box predictors** are too successful, this points to errors in the evaluation process. **Black box** test suites and **black box predictors** cannot verify a **stochastic model** but, in the worst-case, can falsify it.

890 It is an important feature of the **black box** test suite T_{irn} (including the specified evaluation rules) that appropriate **PTRNGs**, i.e., **PTRNGs** that satisfy the Requirements PTG.2,1 to PTG.2.5, should not accidentally fail the test suite. The respective rejection probability should be negligible. It is important that the test results (“pass” or “fail”) can be reproduced in different trials with large probability.

891 In the following, four statistical tests is specified. This comprises the name of the test, the input data (a sequence of bits), the test value, and remarks concerning the distribution of the test value. These statistical tests form the basis of the the **black box** test suite T_{irn} (cf. Subsection 4.6.4). Test T3 and Test T4 are based on predictors. As part of the harmonization between AIS 20/31 and NIST SP 800-90 series, some tests were adopted from NIST.

892 In this section, test values are denoted by c_j , where j labels the statistical test. All statistical tests apply to binary sequences.

893 [general assumption] As usual, we interpret the test values c_j as realizations of random variables, denoted by the capital letter C_j . The distribution of C_j depends on the distribution of the input random variables $B_1, \dots, B_m$ that quantify the stochastic properties of the input sequence $b_1, \dots, b_m$. That is, we assume that $b_1, \dots, b_m$ are realizations of the random variables $B_1, \dots, B_m$.

894 **Test T1:** monobit test
input data: $b_1, \dots, b_m \in \{0, 1\}$
assumption: $B_1, \dots, B_m$ are **iid** binary-valued random variables.

$$\text{test value: } c_1 = \sum_{j=1}^m b_j. \quad (4.262)$$

895 [monobit test] distribution of the test value: Under mild assumptions on the input random variables $B_1, B_2, \dots$ the Central Limit Theorem (CLT) applies to the distribution of the test variable C_1 ; cf. Subsection 4.2.2. More precisely, if $B_j \sim B(1, p)$ -distributed then

$$\frac{C_1 - mp}{\sqrt{m}\sqrt{p(1-p)}} \text{ is approximately } N(0, 1)\text{-distributed.} \quad (4.263)$$

unless the sample size m is too small. For the special case $p = 0.5$, (4.262) implies that

$$\frac{C_1 - 0.5m}{0.5\sqrt{m}} \text{ is approximately } N(0, 1)\text{-distributed.} \quad (4.264)$$

896 **Test T2:** poker test
input data: $b_1, \dots, b_m \in \{0, 1\}$; m is a multiple of 4
assumption: The random variables $B_1, \dots, B_m$ are **iid uniformly distributed** on $\{0, 1\}$.

$$\text{test value: } c_2 = \sum_{i=0}^{15} \frac{(f[i] - \frac{m}{64})^2}{\frac{m}{64}} \quad \text{with} \quad (4.265)$$

$$w_j = 8b_{4j-3} + 4b_{4j-2} + 2b_{4j-1} + b_{4j} \quad \text{and} \quad (4.266)$$

$$f[i] = \left| \left\{ j \mid w_j = i, 1 \leq j \leq \frac{m}{4} \right\} \right| \quad (\text{frequency counter}) \quad (4.267)$$

[poker test] distribution of the test value: If the input random variables $B_1, B_2, \dots$ are **iid uniformly distributed** on $\{0, 1\}$, the test variable 897

$$C_2 \text{ is asymptotically } \chi^2\text{-distributed with 15 degrees of freedom.} \quad (4.268)$$

Test T3: MultiMMC Prediction Estimate 898
input data: $b_1, \dots, b_m \in \{0, 1\}$ with $m = L \cdot \log_2 k$ and k being a power of 2.

Note: The following description of the MultiMMC Prediction Estimate is identical to that in 899
 [SP800-90B] in order to enable re-use of test tools. The MultiMMC Prediction Estimate uses
 $D = 16$ predictors based on Markov Model with Counting (MMC) in order to predict the next
 value based on observed transitions. The input of Test T3 is a vector of L values $s_1, \dots, s_L \in A$.
 The success of the prediction is then used to derive a (pessimistic) estimate for the **min-entropy**.

Given the input $S = (s_1, \dots, s_L) = (b_1, \dots, b_m)$, where $s_i \in A = \{x_1, \dots, x_k\}$ (e.g., $s_1 =$ 900
 $(b_1, \dots, b_8)$ for $k = 256$),

1. Let $D = 16$, and $N = L - 2$. Let *subpredict* be a list of D values, each initialized to *Null*.
 Let *correct* be an array of N values, each initialized to 0. Let *entries* be an array of D
 values, each initialized to 0, and let *maxEntries* = 100000.
2. For $d = 1$ to D , let M_d be a set of counters, where $M_d[x, y]$ denotes the number of observed
 transitions from output x to output y for the d^{th} -order MMC.
3. Let *scoreboard* be a list of D counters, each initialized to 0. Let *winner* = 1.
4. For $i = 3$ to L :
 - a. For $d = 1$ to D :
 - i. If $d < i - 1$:
 1. If $[(s_{i-d-1}, \dots, s_{i-2}), s_{i-1}]$ is in M_d , increment $M_d[(s_{i-d-1}, \dots, s_{i-2}), s_{i-1}]$ by
 1.
 2. Else if $\text{entries}_d < \text{maxEntries}$, add a counter for $[(s_{i-d-1}, \dots, s_{i-2}), s_{i-1}]$ to
 the set, let $M_d[(s_{i-d-1}, \dots, s_{i-2}), s_{i-1}] = 1$ and increment entries_d by 1.
 - b. For $d = 1$ to D :
 - i. If $d < i$, find the y value that corresponds to the highest $M_d[(s_{i-d}, \dots, s_{i-1}), y]$
 value, and denote that y as *y_{max}*. If there is a tie, let *y_{max}* be the greatest y in
 the tie. Let *subpredict_d* = *y_{max}*. If all possible values of $M_d[(s_{i-d}, \dots, s_{i-1}), y]$
 are 0, then let *subpredict_d* = *Null*.

- c. Let $prediction = subpredict_{winner}$.
- d. If $(prediction = s_i)$, let $correct_{i-2} = 1$.
- e. Update the scoreboard. For $d = 1$ to D :
 - i. If $(subpredict_d = s_i)$
 1. Let $scoreboard_d = scoreboard_d + 1$.
 2. If $scoreboard_d \geq scoreboard_{winner}$, let $winner = d$.
5. Let C be the number of ones in $correct$.
6. Calculate the predictor's global performance as $P_{global} = \frac{C}{N}$. The upper bound of the 99 % confidence interval on P_{global} , denoted P'_{global} is calculated as:

$$P'_{global} = \begin{cases} 1 - 0.01^{\frac{1}{N}}, & \text{if } P_{global} = 0 \\ \min(1, P_{global} + 2.576\sqrt{\frac{P_{global}(1-P_{global})}{N-1}}), & \text{otherwise} \end{cases} \quad (4.269)$$

where 2.576 corresponds to the $Z_{(1-0.005)}$ value.

7. Calculate the predictor's local performance, based on the longest run of correct predictions. Let r be one greater than the length of the longest run of ones in $correct$. Use a binary search to solve the following for P_{local} :

$$0.99 = \frac{1 - P_{local}x}{(r + 1 - rx)q} \times \frac{1}{x^{N+1}}, \quad (4.270)$$

where

$$q = 1 - P_{local} \quad (4.271)$$

and $x = x_{10}$, derived by iterating the recurrence relation

$$x_j = 1 + qP_{local}^r x_{j-1}^{r+1} \quad (4.272)$$

for j from 1 to 10, and $x_0 = 1$.

8. The test value is the estimated **min-entropy** given by the negative logarithm of the greater performance metric

$$\text{test value: } c_4 = -\log_2(\max(P'_{global}, P_{local}, \frac{1}{k})). \quad (4.273)$$

901 **Test T4:** LZ78Y Prediction Estimate

input data: $b_1, \dots, b_m \in \{0, 1\}$ with $m = L \cdot \log_2 k$ and k being a power of 2.

- 902 Note: The following description of the LZ78Y Prediction Estimate is identical to that in [SP800-90B] in order to enable re-use of test tools. The LZ78Y Prediction Estimate manages a dictionary similar to the LZ78 compression algorithms and uses this dictionary to predict the next value. The success of the prediction is then used to derive a (pessimistic) estimate for the **min-entropy**.

- 903 Given the input $S = (s_1, \dots, s_L) = (b_1, \dots, b_m)$, where $s_i \in A = \{x_1, \dots, x_k\}$ (e.g., $s_1 = (b_1, \dots, b_8)$ for $k = 256$),

1. Let $B = 16$, and $N = L - B - 1$. Let *correct* be an array of N Boolean values, each initialized to 0. Let *maxDictionarySize* = 65536.
2. Let D be an empty dictionary. Let *dictionarySize* = 0.
3. For $i = B + 2$ to L :
 - a. For $j = B$ down to 1:
 - i. If $(s_{i-j-1}, \dots, s_{i-2})$ is not in D , and *dictionarySize* < *maxDictionarySize*:
 1. Let $D[s_{i-j-1}, \dots, s_{i-2}]$ be added to the dictionary.
 2. Let $D[s_{i-j-1}, \dots, s_{i-2}][s_{i-1}] = 0$.
 3. *dictionarySize* = *dictionarySize* + 1
 - ii. If $(s_{i-j-1}, \dots, s_{i-2})$ is in D ,
 1. Let $D[s_{i-j-1}, \dots, s_{i-2}][s_{i-1}] = D[s_{i-j-1}, \dots, s_{i-2}][s_{i-1}] + 1$.
 - b. Use the dictionary to predict the next value, s_i . Let *prediction* = *Null*, and let *maxcount* = 0. For $j = B$ down to 1:
 - i. Let *prev* = $(s_{i-j}, \dots, s_{i-1})$.
 - ii. If *prev* is in the dictionary, find the $y \in \{x_1, \dots, x_k\}$ that has the highest $D[\text{prev}][y]$ value. In the event of a tie, let the y be the symbol with the higher byte value. For example, if $D[\text{prev}][1]$ and $D[\text{prev}][5]$ both have the highest value, then $y = 5$.
 - iii. If $D[\text{prev}][y] > \text{maxcount}$:
 1. *prediction* = y .
 2. *maxcount* = $D[\text{prev}][y]$.
 - c. If (*prediction* = s_i), let *correct* _{$i-B-1$} = 1.
4. Let C be the number of ones in *correct*. Calculate the predictor's global performance as $P_{\text{global}} = \frac{C}{N}$. The upper bound of the 99 % confidence interval on P_{global} , denoted P'_{global} is calculated as:

$$P'_{\text{global}} = \begin{cases} 1 - 0.01^{\frac{1}{N}}, & \text{if } P_{\text{global}} = 0 \\ \min(1, P_{\text{global}} + 2.576\sqrt{\frac{P_{\text{global}}(1-P_{\text{global}})}{N-1}}), & \text{otherwise} \end{cases} \quad (4.274)$$

where 2.576 corresponds to the $Z_{(1-0.005)}$ value.

5. Calculate the predictor's local performance, based on the longest run of correct predictions. Let r be one greater than the length of the longest run of ones in *correct*. Use a binary search to solve the following for P_{local} :

$$0.99 = \frac{1 - P_{\text{local}}x}{(r + 1 - rx)q} \times \frac{1}{x^{N+1}}, \quad (4.275)$$

where

$$q = 1 - P_{\text{local}} \quad (4.276)$$

and $x = x_{10}$, derived by iterating the recurrence relation

$$x_j = 1 + qP_{\text{local}}^r x_{j-1}^{r+1} \quad (4.277)$$

for j from 1 to 10, and $x_0 = 1$.

6. The test value is the estimated **min-entropy** given by the negative logarithm of the greater performance metric

$$\text{test value: } c_5 = -\log_2(\max(P'_{\text{global}}, P_{\text{local}}, \frac{1}{k})). \quad (4.278)$$

4.6.4 Black Box Test Suite T_{irn}

- 904 The T_{irn} **black box** test suite applies to the **internal random numbers**. The **internal random numbers** can be bits or bit vectors.
- 905 By PTG.2.6, the T_{irn} **black box** test suite is mandatory for functionality class PTG.2.
- 906 It should be noted, however, that the core of a PTG.2 evaluation is the verification of a **stochastic model**. Passing the T_{irn} **black box** test suite is a necessary requirement but plays the role of a sanity check (cf. par. 889). The parameters of the T_{irn} **black box** test suite were chosen in such a way that any suitable **RNG** should pass it, unless its random numbers have serious defects (e.g., the rejection values in T3 and T4 differ from the **entropy** values from class PTG.2). In particular, if an **RNG** passes the T_{irn} **black box** test suite, then this alone does not warrant a meaningful statement about the quality of its random numbers.
- 907 The T_{irn} **black box** test suite applies the **black box** tests T1 to T4 from Subsection 4.6.3. The parameters of these tests are customized and rejection criteria are specified.
- 908 [T_{irn} : evaluation rules]
 (a) The T_{irn} **black box** test suite is passed if all individual statistical tests have been passed.
 (b) If at least one individual statistical test has failed, the T_{irn} test suite is failed.
 (c) If the T_{irn} **black box** test suite has failed, the evaluator applies the T_{irn} **black box** test suite a second time. If the second trial is successful, the T_{irn} **black box** test suite is viewed as passed. Another repetition of the T_{irn} **black box** test suite is not allowed.
 (d) Rules (a) to (c) apply separately to all relevant environmental conditions, for which T_{irn} is applied.
- 909 Each test from Subsection 4.6.3 is applied once. Thus the test suite T_{irn} requires 2 040 000 bits. The bits for each test should be consecutive (generated without gaps).
- 910 [preprocessing] If the **internal random numbers** are bit vectors, the **internal random numbers** are interpreted as sequences of bits.
- 911 $T_{IRN(1)}$ [monobit test] cf. par. 894, Test T1
parameter: $m = 20\,000$
input data: $b_1, \dots, b_m \in \{0, 1\}$

$$\text{test value: } t_{IRN(1)} = c_1 = \sum_{j=1}^m b_j. \quad (4.279)$$

with c_1 as in (4.262).

decision rule: The test fails if $t_{IRN(1)} \notin \{9655, \dots, 10345\}$.

- 912 $T_{IRN(2)}$ [poker test] cf. pars. 896 to 897, Test T2
parameter: $m = 20\,000$

input data: $b_1, \dots, b_m \in \{0, 1\}$

$$\text{test value: } t_{IRN(2)} = c_2 = \sum_{i=0}^{15} \frac{(f[i] - \frac{625}{2})^2}{\frac{625}{2}} = \quad \text{with} \quad (4.280)$$

$$w_j = 8b_{4j-3} + 4b_{4j-2} + 2b_{4j-1} + b_{4j} \quad \text{and} \quad (4.281)$$

$$f[i] = |\{j \mid w_j = i, j = 1, \dots, 5000\}| \quad (\text{frequency counter}) \quad (4.282)$$

with c_2 as in (4.267).

decision rule: The test fails if $t_{IRN(2)} < 1.03$ or $t_{IRN(2)} > 57.4$.

$T_{IRN(3)}$ [MultiMMC Prediction Estimate] cf. pars. 898 to 900, Test T3 913

parameter: $m = 1\,000\,000$

input data: $b_1, \dots, b_m \in \{0, 1\}$ with $L = m$ and $k = 2$

$$\text{test value: } t_{IRN(3)} = c_3. \quad (4.283)$$

with c_3 as in (4.273).

decision rule: The test fails if $t_{IRN(3)} < 0.95$.

$T_{IRN(4)}$ [Z78Y Prediction Estimate] cf. pars. 901 to 903, Test T4 914

parameter: $m = 1\,000\,000$

input data: $b_1, \dots, b_m \in \{0, 1\}$ with $L = m$ and $k = 2$

$$\text{test value: } t_{IRN(4)} = c_4. \quad (4.284)$$

with c_4 as in (4.278).

decision rule: The test fails if $t_{IRN(4)} < 0.95$.

5 Examples

- 915 This chapter discusses examples from several areas and illustrates general concepts that have been introduced in the previous chapters. We begin with **algorithmic post-processing** algorithms (Section 5.1), and then we discuss exemplary verifications of algorithmic requirements of functionality classes DRG.2, DRG.3, and DRG.4 (Section 5.2). In Section 5.3, the conformity of the Hash_DRBG [SP800-90A] and the HMAC_DRBG [SP800-90A] to the algorithmic requirements of class DRG.3 are verified. Section 5.4 investigates **stochastic models** for real-world designs of **physical noise sources**. In Section 5.5, strategies for **online tests** are discussed, and Section 5.6 deals with Linux /dev/random and /dev/urandom. Applicants, designers, and evaluators can refer to the discussed examples and to the results that are provided in this chapter.

5.1 Examples of Algorithmic Post-processing Algorithms

- 916 In this section, we discuss several examples of **algorithmic post-processing** algorithms that can be used by PTG.2-compliant **PTRNGs**. For further explanations, we refer the interested reader, e.g., to [Schi09b], section 3.5, or to [DiBi07; Lach08], for example.
- 917 By an **algorithmic post-processing** algorithm, we mean a relatively simple mapping, which (ideally) has been selected with regard to the admissible distributions of the input data, usually the **raw random numbers**. In other words: The **algorithmic post-processing** should be tailored to the **stochastic model**. The task is to determine the impact of the **algorithmic post-processing** on the **entropy** per random bit, i.e., to determine a lower **entropy** bound per **internal random number**.
- 918 The (average) **entropy** per bit cannot be increased by injective mappings.
- 919 Example: A **noise source** that outputs one **raw random number** bit per clock cycle. An **LFSR** is clocked synchronously, and the **raw random number** bit is XORed to the feedback value of the **LFSR**. The outputs of the **LFSR** are the **internal random numbers**. Ignoring the initial state of the **LFSR**, this **post-processing** algorithm is injective. It thus cannot increase the **entropy** per bit but transforms weaknesses of the **raw random number** bits into other statistical defects, e.g., a bias into dependencies (see, e.g., [Schi09b], Example 3.7)., In particular, if the binary-valued **random variables** $R_1, R_2, \dots$ and $Y_1, Y_2, \dots$ model the **raw random numbers** and the **internal random numbers**, respectively, for any distribution of $R_1, R_2, \dots$ (at least in average) we have

$$H(Y_{n+1} \mid Y_1, \dots, Y_n) = H(R_{n+1} \mid R_1, \dots, R_n). \quad (5.1)$$

Note: Actually, in (5.1) one could replace “=” by “ $\geq$ ” if **entropy** can be assigned to the initial value of the **LFSR**. But even then, essentially “=” should hold if the **RNG** outputs many **raw random numbers**.

- 920 Generally speaking, if the **raw random numbers** already have enough **entropy**, it suffices to prove that $H(Y_{n+1} \mid Y_1, \dots, Y_n) \geq H(R_{n+1} \mid R_1, \dots, R_n)$. Otherwise, the gain of **entropy** per random number has to be verified, which is usually more difficult.

921

To increase the **entropy** per bit, one has to compress the input stream, resulting in a lower output rate.

The examples discussed below do not have an **internal state**, which means that they have no memory. Of course, designs **with memory** are also possible; cf. par: 919, for example. 922

Subsection 5.1.4 treats universal families of hash functions. Universal families of hash functions also count as **algorithmic post-processing** algorithms. However, they have special features. 923

5.1.1 Fixed Compression Rate

In this subsection, we treat several examples of **algorithmic post-processing** algorithms with fixed compression rates. 924

[Xoring non-overlapping k -bit subsequences, **iid**] If the **random variables** $R_1, R_2, \dots$ are **iid** $B(1, 0.5 + 0.5\epsilon)$ -distributed for some $\epsilon \in [0, 1)$, the **random variables** $Y_j := R_{k(j-1)+1} + \dots + R_{kj} \pmod{2}$ are **iid**, too. More precisely, 925

$$\begin{aligned} \text{Prob}(Y_j = 0) &= \sum_{i=0; i \text{ even}}^k \binom{k}{i} (0.5 + 0.5\epsilon)^i (0.5 - 0.5\epsilon)^{k-i}, \\ \text{Prob}(Y_j = 1) &= \sum_{i=0; i \text{ odd}}^k \binom{k}{i} (0.5 + 0.5\epsilon)^i (0.5 - 0.5\epsilon)^{k-i}, \text{ and thus} \\ \epsilon_k := \text{Prob}(Y_j = 1) - \text{Prob}(Y_j = 0) &= - \sum_{i=0}^k (-1)^i \binom{k}{i} (0.5 + 0.5\epsilon)^i (0.5 - 0.5\epsilon)^{k-i} \\ &= -(-(0.5 + 0.5\epsilon) + (0.5 - 0.5\epsilon))^k = (-1)^{k+1} \epsilon^k. \end{aligned} \quad (5.2)$$

Formula (5.2) says that the bias vanishes exponentially fast in the number k of XORed bits. On the negative side, the output rate decreases by the factor k . The greatest practical significance has the case $k = 2$. For any integer $k \geq 2$, we have

$$|\text{Prob}(Y_j = 1) - 0.5| = 0.5\epsilon^k. \quad (5.3)$$

Note: The **random variables** Y_j are $B(1, 0.5 + 0.5\epsilon)^{*k}$ -distributed (k -fold convolution product of the Bernoulli distribution $B(1, 0.5 + 0.5\epsilon)$).

[Xoring k non-overlapping bits of a Markov chain] Assume that the **random variables** $R_0, R_1, R_2, \dots$ form a homogeneous binary-valued ergodic Markov chain with transition matrix P , and, as in par. 925, that the **random variables** $Y_1, Y_2, \dots$ are given by $Y_j := R_{k(j-1)+1} + \dots + R_{kj} \pmod{2}$. The **random variables** $Y_1, Y_2, \dots$ usually are no longer Markovian. However, the random vectors $\vec{R}_1 := (R_1, \dots, R_k), \vec{R}_2 := (R_{k+1}, \dots, R_{2k}), \dots$ are Markovian with a $(2^k \times 2^k)$ -transition matrix Q . As for the special case $k = 2$ in [Schi09b], Example 3.31, we obtain a lower **entropy** bound 926

$$H(Y_{n+1} | Y_n, \dots, Y_1) \geq H(Y_{n+1} | \vec{R}_n, \dots, \vec{R}_1) = H(Y_{n+1} | \vec{R}_n) = H(Y_{n+1} | R_{nk}). \quad (5.4)$$

The inequality follows from the fact that Y_j is a function of $\vec{R}_j$. The Markov property of $\vec{R}_1, \vec{R}_2, \dots$ and $R_1, R_2, \dots$ implies the equation signs. As in par. 925, the case $k = 2$ has the greatest practical significance.

- 927 [Xoring k non-overlapping bits of a Markov chain, ctd.] If all transition probabilities, i.e., all entries of the transition matrix P , are positive, this is the case for Q , too, and the Markov chain $\vec{R}_1, \vec{R}_2, \dots$ is ergodic with invariant distribution $\nu_{(k)}$. If we assume that the random variables $R_1, R_2, \dots$ are in equilibrium, then $\vec{R}_1, \vec{R}_2, \dots$ are in equilibrium, too. Furthermore, the random vectors $Y_1, Y_2, \dots$ are **stationarily distributed** with distribution η . Since R_0 assumes the values 0 or 1,

$$\eta_0 = \sum_{i=0}^1 \nu_i \sum_{\substack{j_1, \dots, j_k \\ j_1 + \dots + j_k \equiv 0 \pmod{2}}} p_{ij_1} p_{j_1 j_2} \cdots p_{j_{k-1} j_k}, \quad \eta_1 = 1 - \eta_0. \quad (5.5)$$

In particular, by (5.4)

$$H(Y_{n+1} | Y_n, \dots, Y_1) \geq H(Y_{n+1} | R_{nk}) = \sum_{i=0}^1 \nu_i H(Y_{n+1} | R_{nk} = i). \quad (5.6)$$

- 928 [Xoring 2 non-overlapping bits of a Markov chain] In pars. 928 to 929 we take a closer look at the distribution of the random variables $Y_1, \dots, Y_n$ for $k = 2$. Again, we assume that the random variables $R_0, R_1, R_2, \dots$ form a homogeneous Markov chain with 2×2 -transition matrix $P = (p_{ij})_{0 \leq i, j \leq 1}$ with positive transition probabilities. The limiting distribution is given by $\nu = (\nu_0, \nu_1) = \left(\frac{p_{10}}{p_{01} + p_{10}}, \frac{p_{01}}{p_{01} + p_{10}} \right)$. Under the assumption that the Markov chain $R_0, R_1, \dots$ is in equilibrium, we conclude

$$\text{Prob}(Y_1 = y_1, \dots, Y_n = y_n) = \sum_{j=0}^1 \nu_j \text{Prob}(Y_1 = y_1, \dots, Y_n = y_n | R_0 = j) = \quad (5.7)$$

$$\sum_{j=0}^1 \nu_j \text{Prob}(Y_n = y_n | Y_1 = y_1, \dots, Y_{n-1} = y_{n-1}, R_0 = j) \text{Prob}(Y_1 = y_1, \dots, Y_{n-1} = y_{n-1} | R_0 = j)$$

Exploiting the Markov property of $R_0, R_1, \dots$, the last but one conditional probability in (5.7) can be expressed as follows:

$$\begin{aligned} \text{Prob}(Y_n = y_n | Y_1 = y_1, \dots, Y_{n-1} = y_{n-1}, R_0 = j) &= \\ \sum_{i=0}^1 \text{Prob}(Y_n = y_n, R_{2n-2} = i | Y_1 = y_1, \dots, Y_{n-1} = y_{n-1}, R_0 = j) &= \\ \sum_{i=0}^1 \text{Prob}(Y_n = y_n | R_{2n-2} = i) \cdot \text{Prob}(R_{2n-2} = i | Y_1 = y_1, \dots, Y_{n-1} = y_{n-1}, R_0 = j) &= \\ \sum_{i=0}^1 (p_{i0} p_{0, y_n} + p_{i1} p_{1, 1-y_n}) \cdot \text{Prob}(R_{2n-2} = i | Y_1 = y_1, \dots, Y_{n-1} = y_{n-1}, R_0 = j) & \end{aligned} \quad (5.8)$$

[Xoring 2 non-overlapping bits of a Markov chain, special cases]

(i) The first special case is given if $p_{00} = p_{10}$ because then the **random variables** R_j are **iid**. This special case has already been covered by par. 925.

(ii) Another special case is given if $p_{01} = p_{10}$. Then $\nu = (0.5, 0.5)$, i.e., the **random variables** $R_0, R_1, \dots$ are **unbiased** but dependent. The equality $p_{01} = p_{10}$ implies $p_{00} = p_{11}$, and thus $p_{ik} = p_{1-i, 1-k}$. Thus, the term $(p_{i0}p_{0,y_n} + p_{i1}p_{1,1-y_n})$ does not depend on i , simplifying (5.8) to $(p_{00}p_{0,y_n} + p_{01}p_{0,y_n}) = p_{0,y_n}$. By induction, we obtain from (5.7) and (5.8)

$$\begin{aligned} \text{Prob}(Y_1 = y_1, \dots, Y_n = y_n) &= \sum_{j=0}^1 \nu_j \text{Prob}(Y_1 = y_1, \dots, Y_n = y_n \mid R_0 = j) = \\ p_{0,y_n} \sum_{j=0}^1 \nu_j \text{Prob}(Y_1 = y_1, \dots, Y_{n-1} = y_{n-1} \mid R_0 = j) &= p_{0,y_n} \text{Prob}(Y_1 = y_1, \dots, Y_{n-1} = y_{n-1}) = \\ p_{0,y_n} p_{0,y_{n-1}} \cdots p_{0,y_1}. \end{aligned} \quad (5.9)$$

Hence, the **random variables** $Y_1, Y_2, \dots$ are **iid** with $\text{Prob}(Y_j = i) = p_{0i}$. Interestingly, unlike in special case (i) where XORing non-overlapping pairs of **raw random number** bits increases the **entropy** per bit significantly, in special case (ii) it does not. Compared to the Markov chain $R_0, R_1, \dots$, the **entropy** per bit does not increase; this refers to both **Shannon entropy** and **min-entropy**; cf. pars. 806 and 638.

Note: It should be noted that the special case (ii) can be verified directly without considering the complicated formulae in par. 928. In fact, the probability that $R_{2n} = R_{2n-1}$, or equivalently, that $Y_n = 0$, is p_{00} or p_{11} ($= p_{00}$), regardless of $y_1, \dots, y_{n-1}$. This in turn implies

$$\text{Prob}(Y_1 = y_1, \dots, Y_n = y_n) = p_{0,y_n} \text{Prob}(Y_1 = y_1, \dots, Y_{n-1} = y_{n-1}). \quad (5.10)$$

The rest follows by induction.

Of course, algorithmic compression functions are not limited to XORing single bits. Another approach is to group the sequence $R_1, R_2, \dots$ into non-overlapping blocks of t bits; $\vec{R}_1 := (R_1, \dots, R_t)$, $\vec{R}_2 := (R_{t+1}, \dots, R_{2t})$, ... and to interpret their realizations as values in a finite group G with 2^t elements (e.g., modular arithmetic). A possible **algorithmic post-processing** algorithm would be $\vec{Y}_j := \vec{R}_{2j-1} + \vec{R}_{2j} \pmod{2^t}$. Applying this group operation provides a stronger mixture of the particular components than the bitwise XOR operation. 930

A straightforward example is $G = \mathbb{Z}_{2^t}$ together with the addition modulo 2^t . For the special cases $t = 4$ and $t = 8$, also $G = \mathbb{Z}_{2^t+1}^*$, equipped with the multiplication modulo $2^t + 1$ as group operation, is a valid example because 17 and 257 are prime. The integer 0 is identified with 2^t . 931

[Counterexample] This paragraph discusses an example where data compression reduces the **entropy** per bit to 0. This example explains why the distribution of the input data (**raw random numbers**) has to be considered. 932

Assume that $U_1, U_2, \dots$ are **iid random variables** that are **uniformly distributed** on $\{0, 1\}$. Further, $X_{2j-1} = X_{2j} = U_j$ and $Y_j = X_{2j-1} + X_{2j} \pmod{2}$ for $j = 1, 2, \dots$. Then $H_{\min}(U_1, \dots, U_n) = H_{\min}(X_1, \dots, X_{2n}) = n$ for all $n \in \mathbb{N}$ but $H_{\min}(Y_1, \dots, Y_n) = 0$. In other words: The **min-entropy** of the **random variables** $X_1, X_2, \dots$ is 0.5 while the **min-entropy** of the post-processed **random variables** $Y_1, Y_2, \dots$ equals 0.

Note: The property that subsequent **random variables** X_{2j-1} and X_{2j} are identical could arise,

for example, from overclocking a flip-flop by a factor of 2 that stores the output data of a **physical noise source**.

933 The pars. 934 to 939 may, in particular, be useful if the **noise source** generates k -bit **raw random number** vectors $\vec{r}_1, \vec{r}_2, \dots$

934 Assume that $\Omega_1 = \{x_1, \dots, x_n\}$, $\Omega_2 = \{y_1, \dots, y_n\}$, and $\Omega = \{z_1, \dots, z_n\}$. The **random variables** X and Y are independent and take on values in Ω_1 and Ω_2 , respectively, with probabilities $\text{Prob}(X = x_j) = p_j$ and $\text{Prob}(Y = y_j) = q_j$ for $j = 1, \dots, n$. Without loss of generality, we may assume that $p_1 \leq \dots \leq p_n$ and $q_1 \leq \dots \leq q_n$. (Otherwise, relabel the elements of Ω_1 and Ω_2 .) Furthermore, $f: \Omega_1 \times \Omega_2 \rightarrow \Omega$ and $Z = f(X, Y)$.

935 Assume further that the mapping $f: \Omega_1 \times \Omega_2 \rightarrow \Omega$ is invertible in the second argument (i.e., for each fixed first argument). Hence, for each pair (i, j) of indices, there exists a unique index k such that $z_i = f(x_j, y_k)$. In other words: For each $i \leq n$, the function f generates a permutation π_i of $\{1, \dots, n\}$ that is given by $z_i = f(x_j, y_{\pi_i(j)})$. Since X and Y are independent

$$\text{Prob}(Z = z_i) = \sum_{j=1}^n \text{Prob}(X = x_j, Y = y_{\pi_i(j)}) = \sum_{j=1}^n p_j q_{\pi_i(j)} \quad \text{for } 1 \leq i \leq n. \quad (5.11)$$

936 Applying the re-arrangement inequality [HaLP34] to the right-hand side of (5.11) provides the inequality

$$\sum_{j=1}^n p_j q_{n-j+1} \leq \text{Prob}(Z = z_i) = \sum_{j=1}^n p_j q_{\pi_i(j)} \leq \sum_{j=1}^n p_j q_j. \quad (5.12)$$

937 In pars. 937 to 939, we additionally assume that $\Omega_1 = \Omega_2$ and that $p_j = q_j$ for $j = 1, \dots, n$, i.e., that X and Y are identically distributed. Inequality (5.12) implies

$$H_{\min}(Z) = -\log_2(\max\{\text{Prob}(Z = z_i) \mid i = 1, \dots, n\}) \leq -\log_2\left(\sum_{j=1}^n p_j^2\right) = H_2(X) \quad (5.13)$$

$$\text{with equality if } \max\{\text{Prob}(Z = z_i) \mid i = 1, \dots, n\} = \sum_{j=1}^n p_j^2 \quad (5.14)$$

which ties the collision **entropy** of X to the **min-entropy** of $Z = f(X, Y)$.

938 Assume that $\Omega_1 = \Omega_2 = \{0, 1\}^k$ and $f(x, y) = x \oplus y$ (bitwise XOR operation). Then $z = f(x, y) = \vec{0}$ iff $x = y$, and thus

$$\max\{\text{Prob}(Z = z_i) \mid i = 1, \dots, 2^k\} = \text{Prob}(Z = \vec{0}) = \sum_{j=1}^{2^k} p_j^2. \quad (5.15)$$

Finally, (5.13) and (5.14) imply

$$H_{\min}(Z) = -\log_2\left(\sum_{j=1}^{2^k} p_j^2\right) = -\log_2(\text{Prob}(Z = \vec{0})) = H_2(X). \quad (5.16)$$

Equation (5.16) simplifies the estimation of the **min-entropy** of Z to the estimation of the probability $\text{Prob}(Z = \vec{0})$. This can be interesting for **noise sources** that output independent k -bit **raw random number** vectors. Alternatively, portions of k bits may be taken from **stationary** binary-valued **raw random numbers** such that consecutive vectors can be assumed to be independent. In particular, (5.16) suggests a simple **online test** that checks the proportion of pairs of k -bit input vectors that are identical. (This is equivalent to counting the number of 0's of the output sequence.) 939

5.1.2 Von Neumann Unbiasing

Von Neumann unbiasing works asynchronously, i.e., it receives pairs of binary-valued **raw random numbers** $\vec{r}_k = (r_{2k}, r_{2k+1})$ as input but does not generate **internal random number** bit for all input pairs. More precisely, let 940

$$r'_k := \begin{cases} 0 & \text{if } \vec{r}_k = (0, 1) \\ 1 & \text{if } \vec{r}_k = (1, 0) \\ o & \text{if } \vec{r}_k = (0, 0) \\ o & \text{if } \vec{r}_k = (1, 1) \end{cases} \quad (5.17)$$

The **internal random number** bits $y_1, y_2, \dots$ are given by the concatenation of all $r'_k \in \{0, 1\}$.

It is well-known (and easy to prove) that the **internal random numbers** $Y_1, Y_2, \dots$ are **iid** $B(1, 0.5)$ -distributed if the **random variables** $R_1, R_2, \dots$ are **iid** $B(1, p)$ -distributed. This means that the von Neumann unbiasing algorithm removes the bias completely. 941

The main difficulty in the context of **PTRNG** evaluations is the verification that the **random variables** $R_1, R_2, \dots$ are indeed (at least almost) **iid**. 942

However, there are further disadvantages: The output rate drops down to $p(1-p) \leq 0.25$ of the input rate ($p = \text{Prob}(R_1 = 1)$), and from a technical point of view, it can cause problems because it is impossible to guarantee the generation of an **internal random number** bit within a fixed time interval. 943

[Generalized von Neumann unbiasing] The technique described above can be generalized to transform pairs of integer values $\vec{r}_k = (r_{2k}, r_{2k+1})$ into **internal random number** bits by the following rule. 944

$$r'_k := \begin{cases} 0 & \text{if } r_{2k} < r_{2k+1} \\ 1 & \text{if } r_{2k} > r_{2k+1} \\ o & \text{if } r_{2k} = r_{2k+1} \end{cases} \quad (5.18)$$

5.1.3 Thinning Out

Assume that the sequence $R_1, R_2, \dots$ has only a small bias but non-negligible dependencies. A straightforward strategy is to use only each t^{th} **raw random number**, i.e., $Y_n := R_{nt}$. This approach should reduce the dependencies.

- 946 Assume that $R_1, R_2, \dots$ form a homogeneous ergodic Markov chain on the finite state space $\Omega_R := \{\omega_1, \dots, \omega_k\}$ with state transition matrix P (typically, $k = 2$). Then $Y_1, Y_2, \dots$ also form a homogeneous ergodic Markov chain but with state transition matrix P^t . The rows of the powers $P, P^2, \dots$ converge exponentially fast to the limiting distribution ν . Thus $H(\nu)$ (or $H_{\min}(\nu)$, respectively) provide upper **entropy** bounds.

5.1.4 Universal Families of Hash Functions

- 947 In Subsection 5.1.4, we assume that $\mathcal{H} = \{h_k: \{0, 1\}^n \rightarrow \{0, 1\}^m \mid k \in K\}$ is an ϵ -almost universal family of hash functions with $\epsilon = \frac{1+\alpha}{2^m}$. For the definition of (ϵ -almost) universal families of hash functions and for relevant facts, the reader is referred to pars. 641 to 648. As in the whole Section 5.1, the focus lies on the application of ϵ -almost universal families of hash functions as **algorithmic post-processing** algorithms for **PTRNGs** that shall be compliant with functionality class PTG.2. An advantage over the **algorithmic post-processing** algorithms discussed in Subsections 5.1.1 to 5.1.3 is that for the evaluation, the knowledge of the distribution of the **raw random numbers** is not needed but only a lower **min-entropy** bound. Problems and disadvantages are explained in pars. 951 ff.
- 948 Assume that the vectors $\vec{r}_1, \vec{r}_2, \dots \in \{0, 1\}^n$ are vectors that are given by non-overlapping concatenations of **raw random numbers** $r_1, r_2, \dots$, which are binary-valued or, e.g., byte-valued. We assume further that $\vec{r}_1, \vec{r}_2, \dots$ are realization of random vectors $\vec{R}_1, \vec{R}_2, \dots$ that are **iid** η -distributed. Clearly, $H_{\min}(\vec{R}_j) \geq v \in [0, n]$. For simplicity, for the moment, we assume that the **random variables** $Z_1, Z_2, \dots$ are **iid uniformly distributed** on a finite set K . As pointed out in par. 647, for fixed distribution η (of the random vectors $\vec{R}_j$) the **min-entropy** $H_{\min}(h_k(\vec{R}_j))$ can be interpreted as a function $\chi: K \rightarrow \mathbb{R}$, $\chi(k) := H_{\min}(h_k(\vec{R}_1))$. The **random variables** $\chi(Z_1), \chi(Z_2), \dots$ are **iid** on $[0, m]$, or more precisely, on the finite subset $\chi(K) \subseteq [0, m]$. Furthermore, we assume that the random vector $\vec{R}$ is also η -distributed and independent of $\vec{R}_1, \vec{R}_2, \dots$. If $z_1, z_2, \dots$ denote realizations of $Z_1, Z_2, \dots$, by the strong law of large numbers and (4.144)

$$\text{Prob} \left(\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N H_{\min}(h_{z_j}(\vec{R}_j)) = E_Z \left(H_{\min}(h_Z(\vec{R})) \right) \right) = 1, \quad \text{and thus} \quad (5.19)$$

$$\text{Prob} \left(\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N H_{\min}(h_{z_j}(\vec{R}_j)) \geq -\log_2 \left(\frac{1}{|T|} + 0.5 \cdot \sqrt{|T| \cdot 2^{-H_2(\vec{R})} + \alpha} \right) \right) = 1, \quad (5.20)$$

$$\text{Prob} \left(\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N H_{\min}(h_{z_j}(\vec{R}_j)) \geq -\log_2 \left(\frac{1}{|T|} + 0.5 \cdot \sqrt{|T| \cdot 2^{-H_{\min}(\vec{R})} + \alpha} \right) \right) = 1. \quad (5.21)$$

- 949 If the developer applies the generalized leftover hash lemma (4.151), and, consequently the **min-entropy** inequality (4.152), the random variables $Z_1, Z_2, \dots$ are iid but not uniformly distributed.

Equation (5.19) remains unchanged, and (5.20) and (5.21) generalize to

$$\text{Prob} \left(\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N H_{\min}(h_{z_j}(\vec{R}_j)) \geq -\log_2 \left(\frac{1}{|T|} + 2^{\log_2(|K|) - H_{\min}(Z') - 1} \cdot \sqrt{|T| \cdot 2^{-H_2(\vec{R})} + \alpha} \right) \right) = 1, \quad (5.22)$$

$$\text{Prob} \left(\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{j=1}^N H_{\min}(h_{z_j}(\vec{R}_j)) \geq -\log_2 \left(\frac{1}{|T|} + 2^{\log_2(|K|) - H_{\min}(Z') - 1} \cdot \sqrt{|T| \cdot 2^{-H_{\min}(\vec{R})} + \alpha} \right) \right) = 1. \quad (5.23)$$

Pars. 951 to 955 describe and analyze a method that allows to apply the “usual” leftover hash lemma (4.139) and the **min-entropy** inequality (4.144). Recall that par. 652 provides numerical examples for several parameter sets for the generalized leftover hash lemma. 950

One problem is that (5.20) and (5.21) provide lower **min-entropy** bounds for the **internal random numbers** $y_1 := h_{z_1}(\vec{r}_1), y_2 := h_{z_2}(\vec{r}_2), \dots$ that are satisfied *on average*. A further problem is that (in a strict sense) **iid** uniformly distributed random variables do not exist. Two conditions must be satisfied. 951

- a) The **random variables** $\chi(Z_1), \chi(Z_2), \dots$ shall satisfy the strong law of large numbers (cf. par. 538) where the expectation $E_Z \left(H_{\min}(h_Z(\vec{R})) \right)$ is computed with regard to the uniform distribution on K . In par. 948, for simplicity, it is assumed that the **random variables** $Z_1, Z_2, \dots$ are **iid uniformly distributed** on K , which, of course, is an idealization. The strong law of large numbers holds under weaker conditions, too. The condition that the **random variables** Z_j are (at least approximately) **uniformly distributed** on K cannot be weakened, unless the applicant for the certificate presents (and proves) a version of the leftover hash lemma that fits the actual distribution of the **random variables** $Z_1, Z_2, \dots$.
- b) For some indices z_j , the **min-entropy** $\chi(z_j) = H_{\min}(h_{z_j}(\vec{R}_j))$ can be (significantly) smaller than the average value $E_Z \left(H_{\min}(h_Z(\vec{R})) \right)$. For these $z_j \in K$, the **internal random number** $h_{z_j}(\vec{R}_j)$ can be vulnerable to guessing attacks. Which indices are affected and how large the **entropy** defect is, depends on the distribution η of the **raw random numbers**, or more generally, on the class of distributions to which η belongs. An **adversary** with knowledge and understanding of the **physical noise source** could identify hash functions h_k that are “weak” for the distribution η . Thus, a central requirement is that the random numbers $z_1, z_2, \dots$ must be kept secret, and that an **adversary** shall not have the opportunity to determine these values, neither later nor in advance. To support this goal, the **random variable** Z_j shall contain only little information on $Z_{j-\ell}$ and $Z_{j+\ell}$ unless ℓ is “small”. In other words, The “history” of the selected indices should fade out sufficiently fast.

A valid solution would be to generate the random numbers $z_1, z_2, \dots$ with a **PTRNG** that is compliant with functionality class PTG.3 and has $1 - 2^{-32}$ bits of **min-entropy** per **internal** 952

random number bit. However, it is not economic to continually use a PTG.3-compliant PTRNG to construct another PTRNG that is compliant only with functionality class PTG.2. Pars. 953 to 954 discuss an alternative approach that allows, under certain conditions, the use of the same physical noise source that generates the input for the selected hash functions.

- 953 [Example] Assume that a physical noise source generates binary-valued stationarily distributed raw random numbers. Furthermore, $\mathcal{H} := \{h_k: \{0,1\}^n \rightarrow \{0,1\}^m \mid k \in K\}$ defines a universal family of hash functions. For simplicity, we assume that $|K| = 2^t$, which allows us to identify the sets K and $\{0,1\}^t$. We equip the set $\{0,1\}^t$ with a group operation $\circ$, giving a group G . (Example: If the elements of $\{0,1\}^t$ are interpreted as binary representations of integers, the addition modulo 2^t defines the group operation $\circ$.) From the raw bit stream, non-overlapping bit vectors $z_{-s+1}, z_{-s+2}, \dots, z_0, z_1, \vec{r}_1, z_2, \vec{r}_2, \dots$ are generated with $z_j \in \{0,1\}^t$ and $\vec{r}_j \in \{0,1\}^n$. These vectors are interpreted as realizations of independent random vectors $Z_{-s}, Z_{-s+1}, \dots, Z_0, Z_1, \vec{R}_1, Z_2, \vec{R}_2, \dots$. Depending on the physical noise source, it may be necessary to leave “gaps” between neighbored bit vectors. (If due to the stochastic model the raw random number bits have no significant dependencies if the gap is larger than 2, for example, the gaps between consecutive bit vectors shall be at least 3.) Then the random variables $Z_{-s}, Z_{-s+1}, \dots, Z_0, Z_1, Z_2, \dots$ are iid τ -distributed while the random vectors $\vec{R}_1, \vec{R}_2, \dots$ are iid η -distributed. We further assume that τ fulfills condition (4.72) for some $c \in (0,1)$. In a “warm-up phase”, the bit vectors $z_{-s+1}, z_{-s+2}, \dots, z_0$ initialize an “index register” that is used for the selection of the hash functions. The current value of this register is denoted by u_j . In the beginning, we initialize the index register with the t -bit value z_{-s+1} , and we set $u_{-s+1} := z_{-s+1}$. We successively refresh the register via $u_{-j+1} := u_{-j} \circ z_{-j+1}$ until u_0 has been computed, which completes the warm-up phase. The integer s shall be selected with regard to the distribution τ such that the random variable $U_0 = Z_{-s+1} \circ Z_{-s+2} \circ \dots \circ Z_{-0}$ is “almost” uniformly distributed on G . For $j \geq 1$, the value u_j is used to select the j^{th} hash function while r_j is the input of the selected hash function. The internal random numbers are $y_1 := h_{u_1}(\vec{r}_1), y_2 := h_{u_2}(\vec{r}_2), \dots$
- 954 [Example; continues par. 953] In this paragraph, we show that the construction from par. 953 fulfills the Requirements a) and b) from par. 951. The function $\chi: \{0,1\}^t \rightarrow [0,m] \subseteq \mathbb{R}$ is defined as in par. 951. By par. 602, a), b), the random variables $U_0, U_1, \dots$ can be assumed to be uniformly and stationarily distributed on K . Consequently, the random variables $\chi(U_1), \chi(U_2), \dots$ are stationarily distributed, too, and by (4.86) $\text{Var}\left(\frac{1}{n} \sum_{j=1}^n \chi(U_j)\right) = O(n^{-1})$. In particular, although not independent, the random variables $\chi(U_1), \chi(U_2), \dots$ satisfy Requirement a) from par. 951. It is $U_{j+k} = U_j \circ Z_{j+1} \circ \dots \circ Z_{j+k}$. Loosely speaking, the “history” and the “future” of the selected indices fade away if τ^{*k} converges to the uniform distribution. In other words, the index values u_j satisfy Requirement b) from par. 951 if the convolution products $\tau, \tau^{*2}, \dots$ converge “sufficiently fast” to the uniform distribution ν on $\{0,1\}^t$. Note: If the convolution powers $\tau, \tau^{*2}, \dots$ do not converge fast enough to the uniform distribution, this can easily be repaired by modifying the update process of the index register. In place of $u_1 = u_0 \circ z_1, u_2 = u_1 \circ z_2, \dots$ we set $u_1 = u_0 \circ z_1 \circ z_2 \dots z_b, u_2 = u_1 \circ z_{b+1} \circ z_{b+2} \dots z_{2b}, \dots$ where $b \geq 1$ is adjusted to the distribution τ of the random variables Z_j .
- 955 [Example (special case); continues par. 953f.] Assume that the raw random numbers $r_1, r_2, \dots$ are binary-valued and iid $B(1,p)$ -distributed with $p \in [0.5 - 0.5s, 0.5 + 0.5s]$ for some $s \in [0,1)$. The least favorable case is given by $p_w = 0.5 + 0.5s$ (or, likewise, $p_w = 0.5 - 0.5s$). As before, the set $\mathcal{H}$ is an ϵ -universal of hash functions (for simplicity, in this paragraph we assume that

$\epsilon = 0$), and $\vec{r}_1, \vec{r}_2, \dots$ are vectors of n consecutive **raw random numbers** (= bits). In particular, $H_{\min}(\vec{R}_j) \geq -n \log_2(p_w)$, and by substituting in (4.144) (with $T = \{0, 1\}^m$)

$$\begin{aligned} E_Z \left(H_{\min}(h_Z(\vec{R})) \right) &\geq -\log_2 \left(2^{-m} + 0.5 \sqrt{2^m \cdot 2^{-H_{\min}(\vec{R})} + 0} \right) \geq \\ &-\log_2 \left(2^{-m} + 2^{-1+0.5m} \cdot p_w^{n/2} \right). \end{aligned} \quad (5.24)$$

Furthermore, the distributions τ of the **random variables** Z_j fulfill condition (4.72) with $n_0 = 1$ and $c \geq \left(\frac{0.5-0.5s}{0.5} \right)^t = (1-s)^t$ (worst case). By (4.73) we obtain

$$\delta(\tau^{*k}, \nu) \leq (1 - (1-s)^t)^k \quad \text{for all } k \in \mathbb{N}, \quad (5.25)$$

and, analogously, by (4.74)

$$H_{\min}(Z_1 \circ \dots \circ Z_k) \geq -\log_2 \left(\frac{1}{2^t} + (1 - (1-s)^t)^k \right) \quad \text{for all } k \in \mathbb{N}. \quad (5.26)$$

Numerical Example: $(n, m, t, s, k) = (48, 12, 59, 0.1, 10)$:

By (5.24), $E_Z(H_{\min}(h_Z(\vec{R}))) / m = E_Z(H_{\min}(Y_1)) / m \geq 0.991$ provides a lower (average) **min-entropy** bound per **internal random number** bit; cf. par 341.

Substituting s into (5.25) and (5.26) gives the inequalities $\delta(\tau^{*10}, \nu) \leq 0.982$ and $H_{\min}(Z_1 \circ \dots \circ Z_{10}) / t \geq 0.000489$. The upper bound for the variation distance of τ^{*k} to the uniform distribution ν is terribly large while the lower bound for the **min-entropy** of $Z_1 \circ \dots \circ Z_{10}$ is very small.

However, in this special case, much better bounds can be obtained. First of all, $H_{\min}(Z_j) / t = -\log_2(0.55) \approx 0.86$, and for $k = 2$ inequality (4.79) yields $H_{\min}(Z_1 \circ Z_2) / t \geq 0.986$.

Note: It can easily be shown that the sequence $H_{\min}(\tau), H_{\min}(\tau^{*2}), H_{\min}(\tau^{*3}), \dots$ increases monotonically.

5.2 Evaluation of DRNGs: Miscellaneous Aspects

Section 5.2 discusses several **pure DRNG** and **hybrid DRNG** designs. The focus lies on the exemplary verification of requirements of functionality classes DRG.2 and DRG.3. Furthermore, we also illustrate some pitfalls that can occur when a **DRNG** is designed carelessly. These examples may serve as a warning to evaluators. 956

In this section, we focus on the algorithmic aspects. We do not cover **entropy** issues associated with **seeding procedures** and **reseeding procedures**. 957

The Hash_DRBG and the HMAC_DRBG from the **NIST** document [SP800-90A] are analyzed in Section 5.3, which is a section of its own. 958

5.2.1 AES in **OFB** mode

In this subsection, a simple **pure DRNG** design is analyzed. This illustrates how typical proofs can be organized. We show that this **DRNG** is compliant with functionality class DRG.2 but 959

not with DRG.3. In particular, the DRNG provides backward secrecy and forward secrecy but not enhanced backward secrecy.

- 960 The “core” is the block cipher AES-256. The DRNG calls the AES-256 cipher once during each iteration (full OFB mode). Its plaintext and ciphertext space are given by $S_B := \{0, 1\}^{128}$ while $S_K = \{0, 1\}^{256}$ denotes the key space. For simplicity, we further assume that this DRNG only accepts requests of at most 128 bits, the bit length of a single internal random number. This means that always one internal random number is generated between two applications of the state transition function. Below, we formulate the describing 9-tuple $(S, S_{req}, R, A, I, \phi, \phi_{req}, \phi_0, \psi)$; cf. (3.1).
- 961 [describing 9-tuple] It was explained in par. 135 that arguments can be omitted if they are not relevant for the description of the 9-tuple. In this example, the components of the 9-tuple are as follows: $S = S_B \times S_K$, $S_{req} = S$, $R = S_B$, $A = \{o\}$ (no external input, pure DRNG), $I = \{1, \dots, 128\}$ (requests have length ≤ 128 bits), $\phi: S \rightarrow S, \phi(r, k) := (\text{AES-256}(r, k), k)$ (state transition function), $\phi_{req}: S \rightarrow S_{req}, \phi_{req}(s) = s$ (identity mapping), $\phi_0: S_{req} \rightarrow S_{req}, \phi_0(s) = s$ (the definition of ϕ_0 is irrelevant because a request comprises only one internal random number; cf. pars. 131 and 135), and $\psi: S \rightarrow R, \psi(r, k) := r$ (output function).
- 962 We assume that the seed material sm , a bit string of $128 + 256 = 384$ bits, is generated by a PTRNG that is compliant with functionality class PTG.2 or PTG.3, or by an NPTRNG compliant with functionality class NTG.1. In this example, both the seeding procedure and the reseeding procedure are kept rather simple to illustrate the method; cf. par. 146.
- 963 The seed material sm initializes the first internal state $s_1 := (r_1, k)$ of the DRNG. In terms of the seeding procedure describing 4-tuple (SM, PS, S, ϕ_{seed}) (cf. (3.3)), the seeding procedure reads as follows. $SM = \{0, 1\}^{384}$, $PS = \{o\}$, $\phi_{seed}: SM \times PS \rightarrow S, \phi_{seed}(sm, o) = sm$ (seeding procedure, projection onto the first component).
- 964 For the reseeding procedure, the seed material sm , a bit string of 384 bits, is generated by a PTRNG that is compliant with functionality class PTG.2 or PTG.3, or by an NPTRNG compliant with functionality class NTG.1. In terms of the reseeding procedure describing 4-tuple $(SM, PS, S, \phi_{reseed})$ (cf. (3.4)), the reseeding procedure reads as follows: $SM = \{0, 1\}^{384}$, $PS = \{o\}$, $\phi_{reseed}: S \times SM \times PS \rightarrow S, \phi_{reseed}(s, sm, o) = s \text{ XOR } sm$ (reseeding procedure).
- 965 The second component of the internal state space $S = S_B \times S_K$ remains constant during the seedlife of the seed and serves as a long-term key for AES-256. The output function is the projection onto the first component of the internal state. Since each random number reveals the current first component, the first 128 bits of the internal state ($= S_B$) are potentially public. Thus, the unknown part of the internal state comprises 256 bits; cf. par. 971.
- 966 Now assume that an adversary knows the random numbers $r_i, \dots, r_j$. The task is to determine or to guess the successor r_{j+1} or the predecessor r_{i-1} of this subsequence.
- 967 [forward secrecy] The subsequence $r_i, \dots, r_j$ can be written in the following form: $r_{i+1} = \text{AES-256}(r_i, k), r_{i+2} = \text{AES-256}(r_{i+1}, k), \dots, r_j = \text{AES-256}(r_{j-1}, k)$. If this information would suffice to determine $r_{j+1} = \text{AES-256}(r_j, k)$, this would mean that a chosen plaintext attack on

AES-256 (for the (plaintext/ciphertext) pairs $(r_i, r_{i+1}), \dots, (r_{j-1}, r_j)$) would be feasible. However, the cryptographic community has analyzed the **AES** cipher for more than two decades, and no such cryptanalytic attack has been found. This common knowledge about AES-256 can be used to conclude that the **DRNG** has **forward secrecy**.

[**forward secrecy**] Par. 967 provides a typical security proof for **DRNGs**. The desired security property of the **DRNG** is traced back to established properties of a **widely recognized cryptographic primitive**. 968

[**backward secrecy**] The proof of **backward secrecy** is analogous to the proof in par. 967. We express the known subsequence $r_i, \dots, r_j$ as follows: $r_{j-1} = (\text{AES-256})^{-1}(r_j, k)$, $r_{j-2} = (\text{AES-256})^{-1}(r_{j-1}, k)$, $\dots$, $r_i = (\text{AES-256})^{-1}(r_{i+1}, k)$. A successful attack on $r_{i-1} = (\text{AES-256})^{-1}(r_i, k)$ would imply a chosen ciphertext attack on $(\text{AES-256})^{-1}$, the decryption function of AES-256. Since no such attack is known, we conclude that the **DRNG** has **backward secrecy**. 969

[**enhanced backward secrecy**] Obviously, the **DRNG** does not have **enhanced backward secrecy**. If an **adversary** learns the **internal state** $s_n = (r_n, k)$, the preceding **internal states** $s_{n-1}, s_{n-2}, \dots$ (and the preceding random numbers $r_{n-1}, r_{n-2}, \dots$) can easily be obtained by successive encryption. 970

As a by-product of the security proofs in pars. 967 (forward security) and 969 (backward security), we conclude that the set of **effective internal states** equals the key space $S_K = \{0, 1\}^{256}$, due to the generally accepted properties of **AES**. 971

Since the **seed material** is generated by a PTG.2- or a PTG.3-compliant **PTRNG** and because the **seeding procedure** and the **reseeding procedure** are very simple, it is easy to see that Requirement DRG.2.4 (and DRG.3.4) is satisfied. 972

In the previous paragraphs, we have proved that the **DRNG** fulfills several requirements of functionality class DRG.2. In particular, this refers to Requirements DRG.2.1 (pars. 962, 963, 964), DRG.2.2 (par. 960), DRG.2.3 (par. 971), DRG.2.4 (pars. 962, 963, 964, 971, 972), DRG.2.5 (par. 967), and DRG.2.6 (par. 969). Requirement DRG.2.7 does not apply because the **DRNG** is a **pure DRNG**, and thus, DRG.2.7 is also satisfied. The state transition function ϕ is **cryptographic**, and thus DRG.2.8 is fulfilled, too. Since no statistical weaknesses of **AES-256** are known, the evaluator might argue that Requirement DRG.2.9 is fulfilled based on theoretical considerations. 973

By par. 973, the **DRNG** is compliant with functionality class DRG.2. The **DRNG** is not compliant with functionality class DRG.3 because of the missing **enhanced backward secrecy** (par. 970). 974

5.2.2 Pure and Hybrid DRNGs and a (Too) Simple State Transition Function

In this subsection, several simple **DRNG** designs are considered. In pars. 976 to 987, the DRG.2-compliance of a **pure DRNG** design is verified, and then different extensions to **hybrid DRNG** designs are discussed. Pars. 988 to 990 underline that a (too) simple state transition function may ruin the security if the **adversary** is able to control a single **additional input** value. We provide possible bug fixes but also sketch an instructive pitfall. 975

- 976 For simplicity, we assume that all **DRNGs** discussed in this subsection accept only **requests**, whose bit length is $\leq$ the bit length of an **internal random number**.
- 977 In par. 978 below, we formulate the describing 9-tuple $(S, S_{req}, R, A, I, \phi, \phi_{req}, \phi_0, \psi)$ for the “pure” version of the **DRNG**; cf. (3.1).
- 978 [**pure DRNG**, describing 9-tuple] For the **pure DRNG**, the components of the 9-tuple are as follows: $S = S_{req} = \mathbb{Z}_{2^{512}}$, $R = \{0, 1\}^{256}$, $A = \{o\}$ (no external input, **pure DRNG**), $I = \{1, \dots, 256\}$ (**requests** have length ≤ 256 bits), $\phi: S \rightarrow S, \phi(s) := (s + 1 \bmod 2^{512})$ (state transition function, modular incrementation by 1), $\phi_{req}: S \rightarrow S_{req}, \phi_{req}(s) = s$, $\phi_0: S_{req} \rightarrow S_{req}, \phi_0(s) := s$ (the definition of ϕ_0 is irrelevant because a **request** generates only one **internal random number**), and $\psi: S_{req} \rightarrow R, \psi(s_{req}) := \text{SHA-256}(s_{req})$ (output function).
- 979 We assume that **seed material** (512 bits) is generated by a **TRNG** that is compliant with functionality class PTG.2, PTG.3, or NTG.1. As in Subsection 5.2.2, the **seeding procedure** and the **reseeding procedure** are rather simple.
- 980 The first **internal state** s_1 of the **DRNG** equals the **seed material** sm . In terms of the **seeding procedure** describing 4-tuple (SM, PS, S, ϕ_{seed}) (cf. (3.3)), the **seeding procedure** can be described as follows. $SM = \{0, 1\}^{512}$, $PS = \{o\}$, $\phi_{seed}: SM \times PS \rightarrow S, \phi_{seed}(sm, o) = sm$ (**seeding procedure**, projection onto the first component).
- 981 For the **reseeding procedure**, a **seed material** string of 512 bits is generated by a **TRNG** that is compliant with PTG.2, PTG.3, or NTG.1. In terms of the **reseeding procedure** describing 4-tuple $(SM, PS, S, \phi_{reseed})$ (cf. (3.4)), the **reseeding procedure** reads as follows: $SM = \{0, 1\}^{512}$, $PS = \{o\}$, $\phi_{reseed}: S \times SM \times PS \rightarrow S, \phi_{reseed}(s, sm, o) = s \text{ XOR } sm$ (**reseeding procedure**).
- 982 Due to the **one-way function** property of SHA-256, we may assume that the **internal state** S equals the **effective internal state**. Thus, the **effective internal state** comprises 512 bits.
- 983 [**backward secrecy** and **forward secrecy**] The subsequence $r_i, \dots, r_j$ can be expressed as $r_i = \text{SHA-256}(s_i), r_{i+1} = \text{SHA-256}(s_i + 1 \bmod 2^{512}), \dots, r_j = \text{SHA-256}(s_i + j - 1 \bmod 2^{512})$. The task of an **adversary** would be to exploit this information to determine $r_{j+1} = \text{SHA-256}(s_i + j \bmod 2^{512})$ (**forward secrecy**) or $r_{i-1} = \text{SHA-256}(s_i - 1 \bmod 2^{512})$ (**backward secrecy**).
- 984 [**backward secrecy** and **forward secrecy**] If an **adversary** could determine any **internal state**, this would violate the **one-way function** property of SHA-256. Similarly, the assumption that an **adversary** would be able to determine r_{i-1} or r_{j+i+1} only on the basis of $r_i, \dots, r_{i+j}$ and relations between pre-images would also contradict the common knowledge about SHA-256. In particular, it can be assumed that the **DRNG** has **forward secrecy** and **backward secrecy**.
- 985 Yet, this **DRNG** does not provide **enhanced backward secrecy**. If an **adversary** would learn the **internal state** s_n , he could easily obtain the preceding **internal states** $s_{n-1} \equiv s_n - 1 \bmod 2^{512}$, $s_{n-2} \equiv s_n - 2 \bmod 2^{512}, \dots$ (and by this, the preceding random numbers $r_{n-1}, r_{n-2}, \dots$).
- 986 [**pure DRNG**] In the previous paragraphs, we have proved that the **pure DRNG** fulfills several

requirements of functionality class DRG.2. In particular, this refers to Requirements DRG.2.1 (pars. 979, 980, 981), DRG.2.2 (par. 976), DRG.2.3 (par. 982), DRG.2.4 (pars. 979, 980, 981, 982), DRG.2.5 (par. 984), and DRG.2.6 (par. 984). Moreover, DRG.2.7 does not apply because the DRNG is a **pure DRNG**, and thus DRG.2.7 is also satisfied. The output function ψ is **cryptographic**, and thus DRG.2.8 is fulfilled, too. Since no statistical weaknesses of SHA-256 are known, the evaluator can argue that Requirement DRG.2.9 is fulfilled on the basis of theoretical considerations.

[**pure DRNG**] By par. 986, the **pure DRNG** is compliant with functionality class DRG.2. The **DRNG** is not compliant with functionality class DRG.3 because of the missing **enhanced backward secrecy** (par. 985). 987

[**hybrid DRNG**, inadequate design] In this paragraph, the **pure DRNG** is extended to a **hybrid DRNG** design that allows **additional input**. For this, we set $A = \{0, 1\}^{512} \cup \{o\}$ and replace the state transition function ϕ and the function ϕ_{req} by 988

$$\phi_{req}(s, a) = \begin{cases} s \bmod 2^{512} & \text{if } a = o \\ s + a \bmod 2^{512} & \text{if } a \neq o, \end{cases} \quad (5.27)$$

$$\phi(s, a) = \begin{cases} s + 1 \bmod 2^{512} & \text{if } a = o \\ s + a + 1 \bmod 2^{512} & \text{if } a \neq o. \end{cases} \quad (5.28)$$

[successful attack] If the **additional input** values are generated by a strong **TRNG**, no problems should occur (512-bit strings are interpreted as binary representations of 512-bit integers). However, if the **adversary** is able to control a single **additional input** value, he is able to set the future random numbers. More precisely: Assume that $a_{j-k}, \dots, a_{j-1}$ describe the **additional inputs** at time (step) $j-k, \dots, j-1$. If at time j the **adversary** inputs $a_j = 2^{512} - a_{j-k} - \dots - a_{j-1} - k + a_{j-k}$ for some $k < j$, then $r_j = r_{j-k}$ and $s_{j+1} = s_{j-k+1}$. If $a_{j+1} = a_{j-k+1}, \dots, a_{j+k-1} = a_{j-1}$ (e.g., all = o), then not only $r_j = r_{j-k}$ but also $r_{j+1} = r_{j-k+1}, \dots, r_{j+k-1} = r_{j-1}$, which means that the **DRNG** repeats the last k **internal random numbers**. 989

The hybrid design from par. 988 violates Requirement DRG.2.7. Thus, this **hybrid DRNG** is not DRG.2-compliant. This is an example where a **hybrid DRNG** is weaker than its **pure DRNG** version. In particular, this observation motivates Requirement DRG.2.7 (resp. DRG.3.8, resp. DRG.4.8). 990

[**hybrid DRNG**, healed design] In this paragraph, we fix the buggy design from par. 988. We modify ϕ_{req} and ϕ to 991

$$\phi_{req(H)}(s, a) = \begin{cases} s & \text{if } a = o \\ s + a \bmod 2^{512} & \text{if } a \neq o, \end{cases} \quad (5.29)$$

$$\phi_{(H)}(s, a) = \begin{cases} \text{SHA-512}(s) & \text{if } a = o \\ \text{SHA-512}(s + a \bmod 2^{512}) & \text{if } a \neq o, \end{cases} \quad (5.30)$$

where “(H)” stands for “healed” to avoid confusion. The (generally accepted) security properties of SHA-512 prevent even an **adversary** with full control over the **additional input** data from selecting values that influence the **internal state** S in a targeted way unless the **internal state** is

known. By the same argument, due to the properties of the hash function SHA-256, an **adversary** is not able to influence the **internal random number** of the current **request** in a targeted way. Using the generally accepted security properties of SHA-512 and SHA-256, one concludes that the **DRNG** has **backward secrecy**, **forward secrecy**, and **enhanced backward secrecy**. In particular, this “repaired” **hybrid DRNG** is compliant with functionality class DRG.3.

Note: For a security evaluation, the reasoning should be more detailed.

992 [instructive pitfall] Depending on the device, the implementation of two different hash functions may be too expensive. This problem could be solved by replacing $R = \{0, 1\}^{256}$ by $R = \{0, 1\}^{512}$ and the output function $\psi = \text{SHA-256}$ by $\psi_* = \text{SHA-512}$. However, this design is terribly weak since $r_n = \psi(\phi_{\text{req}(H)}(s_n, a)) = \text{SHA-512}(s_n + a \bmod 2^{512}) = \phi_{(H)}(s_n, a) = s_{n+1}$. Knowledge of a single **internal random number** reveals the next **internal state**, and thus (provided that an **adversary** knows the future **additional input** data) all future **internal random numbers**.

993 Par. 992 emphasizes that it does not suffice that the functions ϕ and ψ are individually strong. Their interaction must be secure, too; cf. par. 994.

994 [**hybrid DRNG**, another design] We set $S = \{0, 1\}^{256}$ and $A = \{0, 1\}^*$, and we identify both the state space $S = Z_{2^{256}}$ and S_{req} with $\{0, 1\}^{256}$. Moreover,

$$\phi_{\text{req}(H2)}(s, a) = (s \| 00 \| a) \quad (5.31)$$

$$\psi_{(H2)}(s_{\text{req}}) = \text{SHA-256}(s_{\text{req}}) \quad (5.32)$$

$$\phi_{(H2)}(s, a, p) = \text{SHA-256}(s \| 11 \| a). \quad (5.33)$$

The strings “00” and “11” ensure that the arguments of the state transition function ϕ and output function ψ are different.

995 The describing 9-tuple reads as follows: $S = \{0, 1\}^{256}$, $S_{\text{req}} = \{0, 1\}^{256}$, $R = \{0, 1\}^{256}$, $A = \{0, 1\}^*$, $I = \{1, \dots, 256\}$, $\phi_{(H2)}$ (state transition function, cf. (5.33)), $\phi_{\text{req}(H2)}$ (cf. (5.31)), $\phi_0: S_{\text{req}} \rightarrow S_{\text{req}}$, $\phi_0(s_{\text{req}}) = s_{\text{req}}$ (the definition of ϕ_0 is irrelevant because within a **request** only one **internal random number** is generated, and $\psi_{(H2)}: S \rightarrow R$ (output function, cf. (5.32)).

996 [DRG.3-compliant **DRNG** with bijective output function] First, $p \in \{0, 1\}^{128}$ is a constant. The **additional input** $a \in A$ is a **bit string** of length $\ell \in \{0, \dots, 128\}$, and $\iota(a) := (a, 0 \dots, 0) \in \{0, 1\}^{128}$, i.e., $\iota(\cdot)$ extends a to a 128-vector by appending 0’s to the right. In particular, if $a = \emptyset$ then $\iota(a) = (0, \dots, 0)$. The output function $\psi_{(b)}: \{0, 1\}^{256} \times A \rightarrow \{0, 1\}^{128}$ is given by $\psi_{(b)}(s, a) := \text{AES-256}(p \oplus \iota(a), s)$. The key $s \in \{0, 1\}^{256}$ is the current **internal state**. The state transition function $\phi_{(b)}: \{0, 1\}^{256} \times A \rightarrow \{0, 1\}^{256}$, $\phi_{(b)}(s, a) := \text{SHA-256}(s \| p \oplus \iota(a))$. Following par. 976, we assume that **requests** are limited to 128 bits, which is the bit length of a single **internal random number**.

997 The describing 9-tuple reads as follows: $S = \{0, 1\}^{256}$, $S_{\text{req}} = \{0, 1\}^{256} \times \{0, 1\}^{128}$, $R = \{0, 1\}^{128}$, $A = \{a \in \{0, 1\}^* \mid 0 \leq |a| \leq 128\}$, $\phi_{(b)}$ (state transition function), $\phi_{\text{req}(b)}: S \times A \rightarrow S_{\text{req}}$, $\phi_{\text{req}(b)}(s, a) := (s, \iota(a))$, $\phi_0: S_{\text{req}} \rightarrow S_{\text{req}}$, $\phi_0(s) := s$ (the definition of ϕ_0 is irrelevant because a **request** requires only one **internal random number**), and $\psi_{(b)}: S_{\text{req}} \rightarrow R$ (output function).

998 [DRG.3-compliant **DRNG** with bijective output function, ctd.] We first note that the **DRNG** described in par. 996 is compliant with functionality class DRG.3: Both the state transition

function $\phi_{(b)}$ and the output function $\psi_{(b)}$ are cryptographic. Finding predecessors or successors to a given sequence of **internal random numbers** $r_i, \dots, r_j$ would require that an **adversary** were able to mount a successful chosen-plaintext attack or chosen-ciphertext attack on AES-256, which is not considered practically feasible; cf. Subsection 5.2.1. The **one-way function** property of the state transition function $\phi_{(b)}$ ensures **enhanced backward secrecy**.

5.2.3 One-way Functions Derived from the **AES** Block Cipher

Assume that $\text{Enc}(\cdot, \cdot): \{0, 1\}^t \times \{0, 1\}^t \rightarrow \{0, 1\}^t$ denotes an (ideal) block cipher for which the block length and key length are t bits. Then $\chi: \{0, 1\}^m \times \{0, 1\}^m \rightarrow \{0, 1\}^m$, $\chi(m, k) := \text{Enc}(m, k) \oplus m$ defines a one-way compression function. In one form or another, this idea is used in well-known constructions such as Davies–Meyer, Matyas–Meyer–Oseas, and Miyaguchi–Preneel. 999

For resource-constrained devices such as smart cards, designs of one-way compression functions that use **AES** (or more generally, a block cipher that is a **widely recognized cryptographic primitive**) can be an alternative to the use of dedicated hash functions. Such constructs are allowed, in principle. The applicant has to provide evidence that the class requirements are fulfilled. 1000

5.3 NIST Approved Designs [**SP800-90A**]: Conformity Analysis with Regard to DRG.3 and DRG.4

The document [**SP800-90A**] specifies three **NIST**-approved **DRNG** designs, the Hash_DRBG, the HMAC_DRBG, and the CTR_DRBG. These **DRNGs** are based on hash functions (Hash_DRBG and HMAC_DRBG) and block ciphers (CTR_DRBG). 1001

Figure 21 illustrates the generic design of these **DRBGs**. The meaning of the components will become clear in the following subsections. 1002

Subsection 5.3.1 provides a conformity proof for the Hash_DRBG to the algorithmic requirements of functionality class DRG.3. The permitted hash algorithms are listed in par. 1011. 1003

Subsection 5.3.2 provides a conformity proof for the HMAC_DRBG to the algorithmic requirements of functionality class DRG.3. The permitted hash algorithms are listed in par. 1080. In central parts, this conformity proof refers to [Kels24]. 1004

An applicant for a certificate can refer to Subsection 5.3.1, Subsection 5.3.2, or relevant paragraphs thereof. No further proof is required to show that (correct) implementations of the **NIST**-approved designs comply with the functionality classes hereinafter indicated. The resistance of the implementation against attacks, for instance, is part of the overall evaluation of the **TOE**; cf. Section 2.1. 1005

[Hash_DRBG: Conformity to DRG.3] Par. 1011 lists the permitted hash functions. By par. 1073, the Hash_DRBG fulfills the algorithmic Requirements DRG.3.2, DRG.3.3, DRG.3.5, DRG.3.6, DRG.3.7, DRG.3.8, DRG.3.9, and DRG.3.10. Par. 1075 formulates (easy to check) sufficient 1006

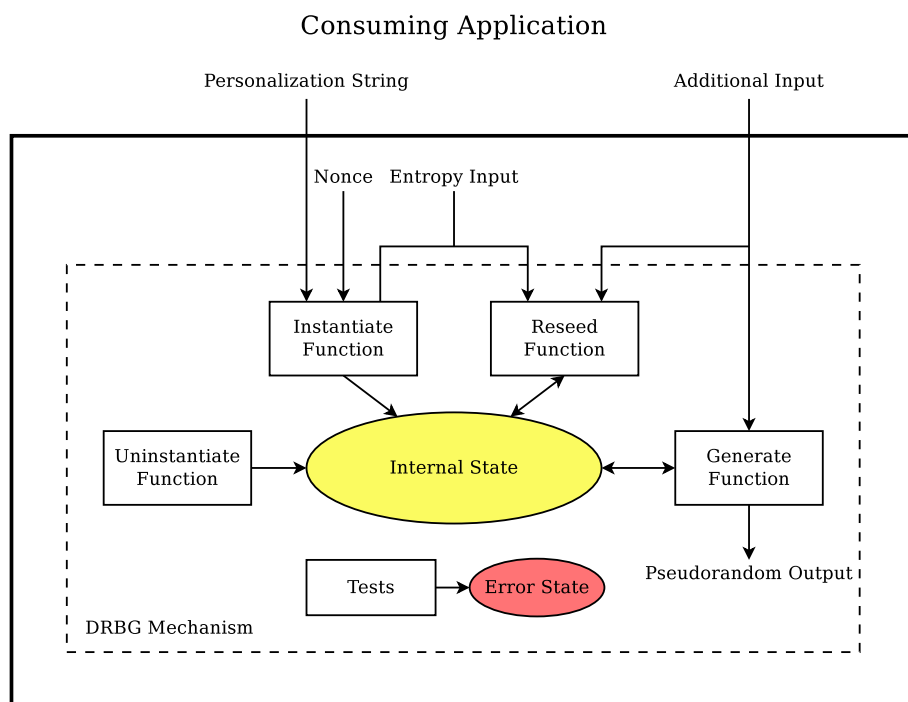


Figure 21: DRBG functional model of the NIST-approved DRBGs; source: [SP800-90A], Section 7, Figure 1

conditions that DRG.3.1 and DRG.3.4 are fulfilled if the entropy_input is generated by a TRNG that is compliant with functionality class PTG.2, PTG.3, or NTG.1.

1007 [CTR_DRBG: Conformity to DRG.3] In par. 102 it is explained that the size of the effective internal state of the CTR_DRBG, used with AES- x , $x \in \{128, 192, 256\}$, is $\leq x$ bits. In particular, when using AES-128 or AES-192, the CTR_DRBG cannot be compliant with functionality class DRG.3. This document does not contain a conformity proof for the CTR_DRBG with AES-256. This does not mean that CTR_DRBG with AES-256 is not algorithmically compliant with functionality class DRG.3 but the applicant for the certificate has to provide a conformity proof.

1008 [Conformity to DRG.4] The Hash_DRBG and the HMAC_DRBG can be “upgraded” to functionality class DRG.4. Therefore, in addition to the requirements of functionality class DRG.3, Requirements DRG.4.1 (instead of DRG.3.1) and Requirement DRG.4.10 shall be fulfilled.

5.3.1 Security Evaluation of the Hash_DRBG [SP800-90A]

1009 In this subsection, we analyze the conformity of the Hash_DRBG to the requirements of functionality class DRG.3. Pars. 1073 to 1075 summarize the results.

1010

For a detailed description of the Hash_DRBG, we refer to [SP800-90A], Subsubsection 10.1.1.

In the following, we assume

$$\text{Hash} \in \{\text{SHA}-224, \text{SHA}-512/224, \text{SHA}-256, \text{SHA}-512/256, \text{SHA}-384, \text{SHA}-512, \text{SHA3}-224, \text{SHA3}-256, \text{SHA3}-384, \text{SHA3}-512\}. \quad (5.34)$$

The hash function *Hash* outputs strings of length $\text{outlen} \in \{224, 256, 384, 512\}$ as indicated by its name; if the name includes two numbers, the output length is indicated by the second number.

If $\text{Hash} \in \{\text{SHA}-224, \text{SHA}-512/224, \text{SHA}-256, \text{SHA}-512/256\}$, then $\text{seedlen} = 440$. If $\text{Hash} \in \{\text{SHA}-384, \text{SHA}-512\}$, then $\text{seedlen} = 888$. It should be noted that use of the hash functions SHA3-224, SHA3-256, SHA3-384, SHA3-512 is not compliant with the specification of the Hash_DRBG in the current version of SP 800-90A [SP800-90A]. If seedlen is not selected as 440 or 888 (e.g., $\text{seedlen} = 512$), the conformity proof given in this subsection requires slight adaptations; cf. in particular par. 1063.

Remark: In addition to (5.34), [SP800-90A] currently allows the hash functions SHA-1.

Our first goal is to describe the Hash_DRBG by the 9-tuple (par. 129) and by the 4-tuples for the seeding procedure (par. 143) and reseeding procedure (par. 147).

In this subsection, $S' := \{0, 1\}^{\text{seedlen}}$. If additions modulo 2^{seedlen} are concerned, we tacitly identify S' with $\mathbb{Z}_{2^{\text{seedlen}}}$.

The set of internal states of the Hash_DRBG (denoted as working state in [SP800-90A]) is given by the cartesian product

$$S := S' \times S' \times \mathbb{Z}_{2^R}. \quad (5.35)$$

The third component in (5.35), $\mathbb{Z}_{2^R}$, is assumed to be large enough that it contains each reasonable reseed_counter value. The elements of the internal state space S are triples (v, c, rc) . The values v and $c = c(v_1)$ are kept secret, while the value of the reseed_counter rc is publicly known. The reseed_counter is initialized by 1 and incremented by 1 after each request.

Note 1: Since $c(v_1)$ remains constant within the seedlife, for the sake of readability, we briefly write c instead of $c(v_1)$. More precisely, the function $c(\cdot)$ is given by the derivation function Hash_{df} ; cf. par. 1029. We note that the derivation function Hash_{df} is a one-way function, which is essential to ensure enhanced backward secrecy; cf. par. 1067.

Note 2: The set $\mathbb{Z}_{2^R}$ does not have to be explicitly stated.

The product space $S' \times S'$ (cf. (5.35)) is the set of all effective internal states; cf. par. 1068.

Furthermore,

$$S_{\text{req}} := S' \quad (5.36)$$

$$A := \{0, 1\}^* \quad (5.37)$$

$$R := \{0, 1\}^{\text{outlen}} \quad (5.38)$$

$$I := \mathbb{Z}_{2^{19}}. \quad (5.39)$$

The bit length of the additional input $a \in A$ is $\leq 2^{35}$. Empty strings are possible.

At the end of each **request**, the **internal state** is updated by the state transition function ϕ .

$$\phi := \phi_B \circ (\phi_A \times \text{id}) : S \times A \times I \rightarrow S \quad \text{with} \quad (5.40)$$

$$\phi_A : S \times A \times I \rightarrow S, \quad \phi_A(v, c, rc, a, p) := (v + f(v, a)(\text{mod } 2^{\text{seedlen}}), c, rc) \quad (5.41)$$

$$\phi_B : S \times A \times I \rightarrow S, \quad \phi_B(v, c, rc, a, p) := (v + g(v) + c + rc(\text{mod } 2^{\text{seedlen}}), c, rc + 1) \quad (5.42)$$

$$\text{with } f : \{0, 1\}^{\text{seedlen}} \times \{0, 1\}^* \rightarrow \{0, 1\}^{\text{outlen}}, \quad f(v, a) := \begin{cases} \text{Hash}(0x02\|v\|a) & \text{if } a \neq \emptyset \\ 0 & \text{if } a = \emptyset \end{cases} \quad (5.43)$$

$$\text{and } g : \{0, 1\}^{\text{seedlen}} \rightarrow \{0, 1\}^{\text{outlen}}, \quad g(v) := \text{Hash}(0x03\|v). \quad (5.44)$$

Actually, in the Hash_DRBG_Generate Process ([SP800-90A], Subsection 10.1.1.4), the **internal state** space S is processed in three steps, in Step 2 (by ϕ_A), and in Step 5 and Step 6 (by ϕ_B). Step 2 is carried out before the random numbers are generated (Step 3), while Step 5 and Step 6 are performed after the random numbers have been generated. This means that during the **request**, the **internal state** assumes an intermediate value $s := \phi_A(s_{\text{old}}, a)$. Note that for the Hash_DRBG, the state transition function $\phi : S \times A \times I \rightarrow S$ does not depend on the bit length of the **request**.

1019 From a logical point of view, the **internal state** (v, c, rc) is updated in each **request** by the state transition function $\phi = \phi_B \circ \phi_A$. Its first component v is updated within each **request** (by ϕ_A and ϕ_B). The value c is a function of the first **internal state** after the **seeding procedure** or the **reseeding procedure** (cf. pars. 1029 and 1033). It remains constant until the next **reseeding procedure**. The **request** counter rc is initialized by 1 and is increased by 1 after each **request**.

1020 The temporary internal **request** state during a **request** is generated and updated by

$$\phi_{\text{req}} := S \times A \rightarrow S_{\text{req}} \quad \text{with}$$

$$\phi_{\text{req}}(v, c, rc, a) := v + f(v, a)(\text{mod } 2^{\text{seedlen}}) \quad (5.45)$$

$$\phi_0 : S_{\text{req}} \rightarrow S_{\text{req}}, \quad \phi_0(s_{\text{req}}) := s_{\text{req}} + 1(\text{mod } 2^{\text{seedlen}}) \quad (5.46)$$

The value s_{req} corresponds to “data” in the Hashgen process; cf. [SP800-90A], Subsection 10.1.1.4. Furthermore, s_{req} equals the first component of the current **internal state** space S (after ϕ_A has been applied).

1021 Finally, the output function ψ is defined by

$$\psi : S_{\text{req}} \rightarrow R, \quad \psi(s') := \text{Hash}(s'). \quad (5.47)$$

This completes the specification of the describing 9-tuple.

1022 Next, we provide formal descriptions of the **seeding procedure** and the **reseeding procedure**.

1023 In [SP800-90A], Section 10.3.1, the derivation function Hash_{df} is defined. The function Hash_{df} is the “core” of both the **seeding procedure** and the **reseeding procedure**.

1024 The derivation function Hash_{df} concatenates the *Hash* values of different input values. In the following, we assume that Hash_{df} is a **one-way function**; see par. 1040.

1025

The initial **internal state** is computed from the **seed material** (denoted by `seed_material` in [SP800-90A]). For the Hash_DRBG,

$$\text{seed_material} = (\text{entropy_input} \parallel \text{personalization_string}). \quad (5.48)$$

The maximum bit length of both the `entropy_input` and the `personalization_string` is 2^{35} [SP800-90A], Table 2. The `personalization_string` belongs to the set PS . It may contain secret parts but does not need to.

Note: The definition of `seed_material` in (5.48) refers to the upcoming version of [SP800-90A]. For the current version of [SP800-90A], the definition reads as follows

$$\text{seed_material} = (\text{entropy_input} \parallel \text{nonce} \parallel \text{personalization_string}) \quad (5.49)$$

The nonce may contain **entropy** but need not. Both nonce and `personalization_string` belong to the set PS .

In addition to the algorithmic properties, security shall be assured by the **entropy** of the string `entropy_input` (denoted as “**seed material**” in the seed describing 4-tuple from par. 143). 1026

In the notation of the 4-tuple which describes the **seeding procedure** (par. 143), 1027

$$SM = PS = \{0, 1\}^*. \quad (5.50)$$

The nonce and the `personalization_string` are constructed from the current value of PS . 1028

The first initial state (v_1, c, rc) is computed from `seed_material` via 1029

$$\begin{aligned} \phi_{seed}: \{0, 1\}^* &\rightarrow \{0, 1\}^{seedlen} \times \{0, 1\}^{seedlen} \times \mathbb{Z}_{2^R}, \quad \phi_{seed}(\text{seed_material}) \\ &:= (v_1 := \text{Hash}_{df}(\text{seed_material}, seedlen), c := \text{Hash}_{df}(0x00 \parallel v_1, seedlen), 1), \end{aligned} \quad (5.51)$$

i.e., $c = c(v_1)$. The parameter $seedlen$ depends on $Hash$.

Note: In the **seeding procedure**, the **bit string** `seed_material` (5.48) is the concatenation of values in SM (`entropy_input`) and PS (nonce and `personalization_string`).

For the **reseeding procedure**, 1030

$$\text{seed_material} = (0x01 \parallel v \parallel \text{entropy_input} \parallel \text{additional_input}). \quad (5.52)$$

The letter v denotes the first component of the **internal state** before the **reseeding procedure**. The maximum bit length of both the `entropy_input` and the `additional_input` is 2^{35} [SP800-90A], Table 2. The `additional_input` may be empty. The `additional_input` belongs to the set PS .

In addition to the algorithmic properties, security shall be assured by the **entropy** of `entropy_input`. 1031

In the notation of the 4-tuple which describes the **reseeding procedure** (par. 147), 1032

$$SM = PS = \{0, 1\}^*. \quad (5.53)$$

The first initial state (v_1, c, rc) after the **reseeding procedure** is computed from `seed_material` via

$$\begin{aligned} \phi_{\text{reseed}}: \{0, 1\}^* &\rightarrow \{0, 1\}^{\text{seedlen}} \times \{0, 1\}^{\text{seedlen}} \times \mathbb{Z}_{2^R}, & \phi_{\text{reseed}}(\text{seed_material}) \\ &:= (v_1 := \text{Hash}_{\text{df}}(\text{seed_material}, \text{seedlen}), c := \text{Hash}_{\text{df}}(0x00\|v_1, \text{seedlen}), 1) \end{aligned} \quad (5.54)$$

i.e., $c = c(v_1)$. The parameter `seedlen` depends on *Hash*.

Note: In the **reseeding procedure**, the **bit string** `seed_material` (5.52) is the concatenation of the first component of the **internal state** (v) and of values in *SM* (`entropy_input`) and *PS* (`nonce` and `personalization_string`).

1034 Below, we analyze the conformity of the `Hash_DRBG` to the requirements of functionality class DRG.3.

1035 [Notation] If the **request** y demands a **bit string** of length `reqbits` (by specification `reqbits` $\leq 2^{19}$, cf. (5.39)), we set

$$m_y := \left\lceil \frac{\text{reqbits}}{\text{outlen}} \right\rceil \text{ and } u := \text{reqbits} - (m_y - 1)\text{outlen}. \quad (5.55)$$

The output is

$$\psi(\tilde{v}_y) \|\psi(\tilde{v}_y + 1) \|\dots \|\psi(\tilde{v}_y + m_y - 2) \|\text{pr}_u(\psi(\tilde{v}_y + m_y - 1)). \quad (5.56)$$

It is $\tilde{v}_y = \phi_{\text{req}}(v_y) = s_{\text{req}}$, the value of S_{req} after being initialized by ϕ_{req} (or, equivalently, the first component of the **internal state** S after ϕ_A has been applied). Furthermore, $\text{pr}_u(\cdot)$ denotes the projection onto the leftmost u bits. In particular, *Hash* is applied m_y times.

1036 First, we investigate **forward secrecy** (DRG.3.5) and **backward secrecy** (DRG.3.6).

1037 [Notation] To simplify the notation, we denote the j^{th} random number (hash value or truncated hash value) of **request** y by $w_{(y)j}$ (cf. (5.56)). Assume that an **adversary** knows the random numbers

$$w_{(y_1)j}, \dots, w_{(y_1)m_{y_1}}, w_{(y_1+1)1}, \dots, w_{(y_2)i}. \quad (5.57)$$

The adversary's task would be to compute or guess the next random number (**forward secrecy**) or the random number that precedes this sequence (**backward secrecy**).

1038 [state transition] Assume that the triple (v_y, c, y) denotes the **internal state** at the beginning of **request** y . By (5.40), (5.41), and (5.42), the next **internal state** (after **request** y has been completed) equals

$$\begin{aligned} (v_{y+1}, c, y+1) &= \phi(v_y, c, y, a_y) = \\ (v_y + g^*(v_y, a_y) + c + y + f(v_y, a_y) \pmod{2^{\text{seedlen}}}, c, y+1) &\text{ where} \\ g^*: \{0, 1\}^{\text{seedlen}} \times \{0, 1\}^* &\rightarrow \{0, 1\}^{\text{outlen}}, \quad g^*(v_y, a_y) := g(v_y + f(v_y, a_y) \pmod{2^{\text{seedlen}}}). \end{aligned} \quad (5.58)$$

1039 [Cryptographic assumptions] In pars. 1040 to 1046, several cryptographic assumptions are formulated and justified, which will be needed below to verify the **backward secrecy**, **forward secrecy**,

and **enhanced backward secrecy** properties. These assumptions concern $\psi = \text{Hash}$ but also the mappings $\text{Hash}_{\text{df}}(\cdot, \text{seedlen}), g, f, g^*$, which are derived from Hash and are closely related.

[Cryptographic assumptions] The “core” of the following cryptographic assumptions is (5.59). 1040
Since Hash is a **widely recognized cryptographic primitive**, this justifies the following assumption

$$\begin{aligned} \psi(\cdot) = \text{Hash}(\cdot) \text{ has the pre-image resistance property.} \\ \text{It can be modeled by a random mapping.} \end{aligned} \quad (5.59)$$

Pre-image resistance means that it is practically infeasible to determine a pre-image under Hash to a given image value y , i.e., to find an x with $\text{Hash}(x) = y$. The second assumption in (5.59) refers to the modeling of $\text{Hash}(\cdot)$ in the random oracle model. This means that for given x , the value $\text{Hash}(x)$ can be viewed as a realization of a **random variable** that is **uniformly distributed** on $\{0, 1\}^{\text{outlen}}$. Furthermore, the output values of Hash for different input values can be viewed as independent.

Note 1: If the output length of a function is too small, modeling by a random mapping does not imply the pre-image resistance property.

Note 2: The justification of the cryptographic assumptions below are in a way redundant as they use “cryptographic” arguments and the modeling by random mappings.

[Cryptographic assumptions] By definition, the function $g(\cdot) = \psi(0x03\|\cdot)$ appends the argument 1041
to the fixed string “0x03” and then applies Hash . Thus, by (5.59) we may also assume

$$\begin{aligned} g: \{0, 1\}^{\text{seedlen}} \rightarrow \{0, 1\}^{\text{outlen}}, \quad g(v) = \psi(0x03\|v) \\ \text{has the pre-image resistance property. It can be modeled by a random mapping.} \end{aligned} \quad (5.60)$$

Rationale: Otherwise, finding a pre-image of $y \in \{0, 1\}^{\text{outlen}}$ under Hash would not be hard, if there exists a pre-image x of y which starts with the pre-fix byte “0x03”. If the bit lengths of the pre-images were not limited, the pre-image resistance of $g(\cdot)$ would directly follow from the pre-image resistance of Hash . In the context of the Hash_DRBG , the variable part of the pre-images under $g(\cdot)$ has fixed length seedlen , and this restriction does not simplify the problem because 2^{seedlen} is large. The second claim follows from the restriction of Hash to the domain $\{0x03\} \times \{0, 1\}^{\text{seedlen}}$.

[Cryptographic assumptions] This assumption considers the restriction of g to a domain interval 1042
 I_b of length 2^{outlen} . Based on (5.60), we also assume

$$\begin{aligned} g: \{0, 1\}^{\text{outlen}} \rightarrow \{0, 1\}^{\text{outlen}}, \quad g(v') = \psi(0x03\|v' + b(\bmod 2^{\text{seedlen}})) \quad \text{for known } b \in \mathbb{Z}_{2^{\text{seedlen}}} \\ \text{has the pre-image resistance property. It can be modeled by a random mapping.} \end{aligned} \quad (5.61)$$

Rationale: For a given $c \in \mathbb{Z}_{2^{\text{outlen}}}$, a randomly selected interval $I_b := [b, b + 2^{\text{outlen}} - 1]$ contains one pre-image x^* of c (i.e., $g(x^*) = c$) on average. Thus, if finding a pre-image of a restriction $g|_{I_b}$ was easy, an **adversary** could also find a pre-image of g in (5.60). In fact, they could randomly select an integer $b \in \mathbb{Z}_{2^{\text{seedlen}}}$, and with probability $\approx 1 - e^{-1} \approx 0.63$, the interval I_b contains a pre-image of c . In this case the **adversary** could solve (5.61). The second claim follows from restricting the domain of g to $\{0x03\} \times \{0, 1\}^{\text{outlen}}$.

[Cryptographic assumptions] If $a \neq \emptyset$, then $f(v, a) = \psi(0x02\|v\|a)$. An argument similar to the 1043

one given in par. 1041 justifies that

$$\text{if } a \neq \emptyset, \text{ then } f: \{0, 1\}^{\text{seedlen}} \times \{0, 1\}^* \rightarrow \{0, 1\}^{\text{outlen}}, f(v, a) := \begin{cases} \text{Hash}(0x02\|v\|a) & \text{if } a \neq \emptyset \\ 0 & \text{if } a = \emptyset \end{cases}$$

has the pre-image resistance property. It can be modeled by a random mapping. (5.62)

Note: In (5.62), we assume that v is unknown.

(Of course, Assumption (5.62) is not valid for $a = \emptyset$.)

- 1044 [Cryptographic assumptions] The derivation function $\text{Hash}_{\text{df}}(\cdot, \cdot)$ is given by the concatenation of one or several hash values (possibly truncated) whose pre-images only differ in the first byte; cf. [SP800-90A], Subsection 10.3.1. In both the seeding procedure and the reseeding procedure, $\text{Hash}_{\text{df}}(\cdot, \text{seedlen})$ is applied twice to compute the first and the second component of the internal state space S ; cf. (5.48) and (5.52). In these cases, at most three hash values are concatenated, depending on Hash . Thus, we assume

$$[\text{seeding procedure, reseeding procedure}] \quad \text{Hash}_{\text{df}}: \{0, 1\}^* \rightarrow \{0, 1\}^{\text{seedlen}}$$

has the pre-image resistance property. It can be modeled by a random mapping (5.63)

Rationale: The following task is not more difficult than finding a pre-image of $\text{Hash}_{\text{df}}(\cdot, \text{seedlen})$: An adversary knows three hash values $\text{Hash}(x), \text{Hash}(x'), \text{Hash}(x'')$ where x, x', x'' are in some way related (here: differences in the first input byte). The task is to find any pre-image x^* with $\text{Hash}(x^*) = \text{Hash}(x)$. If this were possible, this would point to an exploitable correlation of the hash function Hash for related input values. In particular, this would exclude the modeling of Hash by a random mapping in the random oracle model.

- 1045 [Cryptographic assumptions] Assume that an adversary knows $k < 2^{60}$ hash values

$$\psi(x), \psi(x + \delta_1 \pmod{2^{\text{seedlen}}}), \dots, \psi(x + \delta_{k-1} \pmod{2^{\text{seedlen}}}) \quad \text{with } k < 2^{60} \quad (5.64)$$

and the differences δ_j for $1 \leq j \leq k-1$ but not $x \in \mathbb{Z}_{2^{\text{seedlen}}}$ (resp., $x \in \{0, 1\}^{\text{seedlen}}$). In the remainder of this subsection, we assume that

$$\begin{aligned} &\text{To a given } \delta \in \mathbb{Z}_{2^{\text{seedlen}}} \text{ an adversary is not able to calculate } \psi(x + \delta \pmod{2^{\text{seedlen}}}) \\ &\text{unless } x + \delta \pmod{2^{\text{seedlen}}} \in \{x, x + \delta_1 \pmod{2^{\text{seedlen}}}, \dots, x + \delta_{k-1} \pmod{2^{\text{seedlen}}}\} \\ &\text{In particular, the knowledge of (5.64) does not allow finding } x. \end{aligned} \quad (5.65)$$

Rationale: This is not a “standard assumption” on hash functions (recall that $\psi = \text{Hash}$), but it is closely related to its pre-image resistance. Assumption (5.65) is similar to Assumption (5.63), although only three calls of Hash were considered rather than 2^{60} . On the other hand, $2^{60} \ll 2^{\text{outlen}}$ so that a violation of (5.65) would point to a hidden weakness of Hash , namely to correlations of Hash values for different input values. In particular, this would exclude the modeling of Hash by a random mapping. In fact, (5.65) follows when modeling of Hash by a random mapping. Altogether, these arguments make Assumption (5.65) rather plausible.

Note: Requirement DRG.3.2 does not limit the number of requests between successive (re-)seeding procedures. In the following numerical calculations, we assume an upper limit of 2^{48} requests and, consequently, an upper limit of 2^{60} internal random numbers. Larger (realistic) upper bounds can also be used and substituted into the computations. Of course, different upper bounds influence the numerical results to some degree. This applies to pars. 1056, 1057, 1058, 1059, 1060, and 1068.

[Cryptographic assumptions] Based on (5.61) we conclude

$$\{0, 1\}^{\text{outlen}} \rightarrow \{0, 1\}^{\text{outlen}}, \quad v' \mapsto v' + b + g(v' + b) \pmod{2^{\text{outlen}}} \quad \text{for known } b \in \mathbb{Z}_{2^{\text{outlen}}}$$

has the pre-image resistance property. It can be modeled by a random mapping. (5.66)

Rationale: Assumption (5.66) is reasonable because the modular addition of the identity mapping should be “incompatible” with $g(\cdot)$, or more precisely, to its restriction to I_b (cf. 5.61). When modeling as a random mapping, the modular addition of b just means that the values of $g(\cdot)$ are pointwise shifted $\pmod{2^{\text{outlen}}}$, transforming the uniform distribution to the uniform distribution.

Note: Assume that $v + g(v) = c \pmod{2^{\text{seedlen}}}$. Since $0 \leq g(v) < 2^{\text{outlen}}$, we have $v \in [c \pmod{2^{\text{seedlen}}}, c + 2^{\text{outlen}} - 1 \pmod{2^{\text{seedlen}}}]$ (modular interval). Thus, it suffices to determine $v \pmod{2^{\text{outlen}}}$, which leads to (5.66).

By definition,

$$\text{the output values of } \psi = \text{Hash}, g, g^*, f \text{ consist of } \text{outlen} \text{ bits.} \quad (5.67)$$

When proving the **backward secrecy** and the **forward secrecy** properties, we assume that an **adversary** knows a sequence of **internal random numbers**, while the (intermediate) **internal states** (v_y, c, y) are unknown for $y_1 \leq y \leq y_2$ apart from the **request** counter y . A priori, the values $g^*(v_y, a_y)$ and $f(v_y, a_y)$ are unknown, too. An **adversary** would gain additional information if they knew the modular differences

$$b_{y+1} := v_{y+1} - v_y \equiv g^*(v_y, a_y) + c + y + f(v_y, a_y) \pmod{2^{\text{seedlen}}} \quad \text{for all } y = y_1 + 1, \dots, y_2. \quad (5.68)$$

In this case, the **adversary** would know the differences modulo 2^{seedlen} between the first components of all (relevant) **internal states**. (Note that the knowledge of any **internal state** would allow an easy computation of all successors.) Within each **request**, no more than 2^{19} random bits can be output, which means that no more than $\lceil 2^{19}/\text{outlen} \rceil \leq \lceil 2^{19}/224 \rceil < 2^{12}$ hash values are computed.

[**backward secrecy** and **forward secrecy**, simpler problem] Now consider the following problem: The **adversary** knows the hash values

$$\psi(\tilde{v}_{y_1} + j), \dots, \psi(\tilde{v}_{y_1} + 2^{12} - 1), \psi(\tilde{v}_{y_1+1}), \dots, \psi(\tilde{v}_{y_2} + i) \quad (5.69)$$

and the differences $b_{y_1+1}, \dots, b_{y_2}$ as defined in (5.68)

but not the pre-images of these hash values. Furthermore, the **adversary** knows which random numbers belong to which **request** and that within one **request**, the pre-images form an interval (to be precise, an interval $\pmod{2^{\text{seedlen}}}$) of length $< 2^{12}$. The task is to determine the successor (**forward secrecy**), resp. the predecessor (**backward secrecy**), of the sequence (5.69).

[**backward secrecy** and **forward secrecy**, simpler problem] Par. 1037 formulates the task that an **adversary** has to solve in order to violate **forward secrecy** or **backward secrecy**. If the adversary additionally knows the modular differences b_{y+1} (5.68), the task does not become more difficult. In (5.69) we extended the **requests** $y_1 + 1$ to $y_2 - 1$ to 2^{12} random numbers per **request**, which is more information than in par. 1037. Furthermore, the random numbers are not truncated, and the **adversary** knows the differences $b_{y_1+1}, \dots, b_{y_2}$. Extending the **request** lengths to their

maximum does not affect the following **internal states** and thus, does not affect future random numbers.

- 1051 [**backward secrecy** and **forward secrecy**, simpler problem] Altogether, the tasks in par. 1049 cannot be more difficult than the tasks of par. 1037 because more information is available. The idea in analyzing the simpler problem is to get rid of complicated design features and to trace the problem back to the properties of *Hash*.
- 1052 [**backward secrecy** and **forward secrecy**, simpler problem] In pars. 1053 to 1064, we show that it is not practically feasible to determine the successor or the predecessor of (5.69) or to guess these values with non-negligibly greater probability than without knowledge of the sequence (5.69). This shows that the Hash_DRBG fulfills Requirements DRG.3.5 and DRG.3.6.
- 1053 [**forward secrecy**] We begin with the proof of **forward secrecy**. The successor of (5.69) either is $\psi(\tilde{v}_{y_2} + i + 1)$ if $i \leq 2^{12} - 2$, resp. $\psi(\tilde{v}_{y_2+1})$ if $i = 2^{12} - 1$. We distinguish two cases:

Case i) The pre-image of the searched random number, $\psi(\tilde{v}_{y_2} + i + 1)$ or $\psi(\tilde{v}_{y_2+1})$, is not contained in the set of the Hash pre-images in (5.69). By par. 1045, Assumption (5.65), an **adversary** is not able to exploit the knowledge of the hash values (5.69) and the modular differences between the pre-images to determine the next random number.

Case ii) The pre-image of the requested random number is contained in the set of pre-images of (5.69), i.e., a “pre-image” hit occurs. Then the **adversary**’s task is easy because both succeeding random numbers coincide. (We neglect exceptional cases where one value of a “pre-image hit pair” marks the end of a **request** while the other does not. In particular, we even overestimate the probability of a pre-image hit in the following.) Before we derive an upper bound for the probability that Case ii) occurs, we point out two facts.

- 1054 [**forward secrecy**, to par. 1053, Case i)] In Case i) by Assumption (5.65), the successor of the subsequence (5.69) cannot be determined even if all the b_y are known. Furthermore, Assumption (5.66) prevents an **adversary** from determining v_y from the knowledge of $b_{y+1} = b_{y+1}(v_y, a_y)$ (pre-image resistance of g^*). Since the value v_y is unknown, the **adversary** cannot increase his success rate by a chosen **additional input**. Furthermore, the **adversary** cannot determine $v_{y+1} = b_{y+1} + v_y$.
- 1055 [**forward secrecy**, to par. 1053, Case ii)] It is $3 \cdot \text{outlen} > \text{seedlen}$ for all admissible hash functions. Thus, a collision of triplets $(\psi(\tilde{v}_{y_2} + i - 2), \psi(\tilde{v}_{y_2} + i - 1), \psi(\tilde{v}_{y_2} + i)) = (\psi(\tilde{v}_{y_a} + j - 2), \psi(\tilde{v}_{y_a} + j - 1), \psi(\tilde{v}_{y_a} + j))$ for some **request** y_a is a strong indicator that $\tilde{v}_{y_2} + i = \tilde{v}_{y_a} + j$. Except for Hash = SHA-384, even a collision of 2-tuples should suffice.
- 1056 [**forward secrecy**, to par. 1053, Case ii)] Next, we determine an upper bound for the probability that Case ii) occurs. By induction on y , equation (5.68) implies

$$v_y \equiv v_1 + b_2 + \dots + b_y \equiv v_1 + \sum_{s=1}^{y-1} g^*(v_s, a_s) + (y-1)c + \frac{y(y-1)}{2} + \sum_{s=1}^{y-1} f(v_s, a_s) \pmod{2^{\text{seedlen}}} \quad (5.70)$$

$$\text{for } 1 \leq y \leq 2^{48}. \quad (5.71)$$

[forward secrecy, to par. 1053, Case ii)] If the pre-image $\tilde{v}_{y_2} + i + 1$ (or analogously, the pre-image $\tilde{v}_{y_2+1}$) is contained in the set of pre-images in (5.69), briefly denoted as “pre-image hit” in the following — i.e., Case ii) applies, then $\tilde{v}_{y_2} + i + 1 \in \tilde{v}_y + \{0, \dots, 2^{12} - 1\} \pmod{2^{\text{seedlen}}}$ for some request $y < y_2$ (necessary condition). This is equivalent to $v_{y_2} - v_y + i + 1 \pmod{2^{\text{seedlen}}} \in \{0, \dots, 2^{12} - 1\}$. Substituting v_{y_2} and v_y by (5.70) yields

$$\begin{aligned} & \sum_{s=y}^{y_2-1} g^*(v_s, a_s) + (y_2 - y)c + \frac{y_2(y_2 - 1)}{2} - \frac{y(y - 1)}{2} + \sum_{s=y}^{y_2-1} f(v_s, a_s) + i + 1 \\ & \equiv j \pmod{2^{\text{seedlen}}} \quad \text{for some } 1 \leq y \leq y_2 \leq 2^{48}, 1 \leq j \leq 2^{12}. \end{aligned} \quad (5.72)$$

[forward secrecy, to par. 1053, Case ii)] A quick look at (5.72) shows that $g^*(v_s, a_s), f(v_s, a_s) < 2^{\text{outlen}}$, $y \leq 2^{48}$ while $i, j \leq 2^{12}$. Consequently,

$$\sum_{s=y}^{y_2-1} g^*(v_s, a_s) + \frac{y_2(y_2 - 1)}{2} - \frac{y(y - 1)}{2} + \sum_{s=y}^{y_2-1} f(v_s, a_s) + i + 1 < 2^{\text{outlen}+49} + 2^{95}. \quad (5.73)$$

Equation (5.73) says that all terms in (5.72) except $c(y_2 - y)$ are $< 2^{\text{outlen}+50}$, and thus only affect the least $\text{outlen} + 50$ bits if we neglect carries. The most significant bits of (5.72) are determined by the term $(y_2 - y)c$.

[forward secrecy] Of course, for a pre-image hit all seedlen bits must coincide. It is not easy to determine the exact probability for a pre-image hit. Instead, motivated by the observations in par. 1058, we determine an upper bound for this probability by considering “partial” pre-image hits in the least significant outlen bits and in the most significant $\text{seedlen} - \text{outlen} - 50$ bits.

[forward secrecy] In the following, we assume that in (5.72) the sum $\sum_{s=y}^{y_2-1} (g^*(v_s, a_s) + f(v_s, a_s)) \pmod{2^{\text{outlen}}}$ behaves (stochastically) like a realization of a uniformly distributed random variable Z_y on $\{0, 1\}^{\text{outlen}}$. This is a mild assumption if we assume that the summands $(g^*(v_s, a_s) + f(v_s, a_s))$ at least approximately behave like realizations of independent random variables (recall that g^* and f assume values in $Z_{2^{\text{outlen}}}$).

[forward secrecy, pre-image hit $\pmod{2^{\text{outlen}}}$] In the following, $A_{y,\ell}$ denotes the event that a pre-image hit in the outlen least significant bits of $y_2 + i + 1$ occurs with some random number, which has been generated in the y^{th} request. Then $\text{Prob}(A_{y,\ell}) < 2^{12-\text{outlen}}$ for each $y < y_2$.

[forward secrecy, pre-image hit on the most significant bits] A pre-image hit does not only imply a hit of the outlen least significant bits but of all bits, in particular, of the $(\text{seedlen} - t)$ most significant bits. To simplify our notation we set $t := \text{outlen} + 50$, and $A_{y,m}$ denotes a hit in the $(\text{seedlen} - t)$ most significant bits. A pre-image hit for some random number within request y implies

$$\begin{aligned} & \sum_{s=y}^{y_2-1} g^*(v_s, a_s) + (y_2 - y)c + \frac{y_2(y_2 - 1)}{2} - \frac{y(y - 1)}{2} + \sum_{s=y}^{y_2-1} f(v_s, a_s) + i + 1 \\ & \pmod{2^{\text{seedlen}}} \in [0, 2^{12}). \end{aligned} \quad (5.74)$$

Furthermore, let $c = c_1 \cdot 2^{\text{outlen}} + c_0$ with $c_0 = c \pmod{2^{\text{outlen}}}$. By par. 1058

$$\begin{aligned} 0 \leq T := & \sum_{s=y}^{y_2-1} g^*(v_s, a_s) + (y_2 - y)c_0 + \sum_{s=y}^{y_2-1} f(v_s, a_s) + \\ & + \frac{y_2(y_2 - 1)}{2} - \frac{y(y - 1)}{2} + i < 3 \cdot 2^{\text{outlen}+48} < 2^{\text{outlen}+50} = 2^t, \end{aligned} \quad (5.75)$$

and since the left-hand side of (5.74) equals $(y_2 - y)c_1 2^{\text{outlen}} + T$, we conclude that

$$(y_2 - y)c_1 \cdot 2^{\text{outlen}} \pmod{2^{\text{seedlen}}} \in [(2^{\text{seedlen}-t} - 1)2^t, 2^{\text{seedlen}}) \cup [0, 2^{12}). \quad (5.76)$$

Equation (5.76) is a necessary condition for a pre-image hit of the most significant $\text{seedlen} - t$ bits.

1063 [forward secrecy] We may assume that c_1 is the realization of a random variable C_1 that is uniformly distributed on $Z_{2^{\text{seedlen}-\text{outlen}}}$. Then the random variable $X := C_1 / 2^{\text{seedlen}-\text{outlen}}$ may be modeled as uniformly distributed in the unit interval $[0, 1)$, because we are interested in the probability that X is contained in a “large” interval whose length is 2^t times $2^{-\text{seedlen}+\text{outlen}}$. Moreover, since $y_2 - y$ is an integer, the random variable $X_y := X(y_2 - y) \pmod{1}$ may be viewed as uniformly distributed on $[0, 1)$, too. The term $A'_{y,m}$ denotes the event that the random variable C_1 fulfills (5.76). Dividing the second line below by 2^{seedlen} yields

$$\begin{aligned} \text{Prob}(A_{y,m}) &< \text{Prob}(A'_{y,m}) = \\ \text{Prob}((y_2 - y)C_1 \cdot 2^{\text{outlen}} \pmod{2^{\text{seedlen}}} \in [(2^{\text{seedlen}-t} - 1)2^t, 2^{\text{seedlen}}) \cup [0, 2^{12})) &\approx \\ \text{Prob}(X_y \in [1 - 2^{-\text{seedlen}+t}, 1)) &\approx 2^{-\text{seedlen}+t} \end{aligned} \quad (5.77)$$

since $2^{12} \ll 2^t$. Putting the pieces together gives

$$\begin{aligned} \text{Prob}(\text{pre-image hit of } y_2 + i + 1) &< \sum_{y=1}^{y_2-1} \text{Prob}(A_{y,m}, A_{y,\ell}) < \sum_{y=1}^{y_2-1} \text{Prob}(A'_{y,m}, A_{y,\ell}) = \\ \sum_{y=1}^{y_2-1} \text{Prob}(A'_{y,m}) \cdot \text{Prob}(A_{y,\ell}) &\leq 2^{48} \cdot 2^{-\text{seedlen}+t} \cdot 2^{-\text{outlen}+12} = \\ 2^{-\text{seedlen}+110} &\leq 2^{-330}. \end{aligned} \quad (5.78)$$

(Since C_1 does not affect $\text{Prob}(A_{y,\ell})$, this implies that $\text{Prob}(A'_{y,m}, A_{y,\ell}) = \text{Prob}(A'_{y,m}) \cdot \text{Prob}(A_{y,\ell})$.) To be precise, for the SHA-384 and SHA-512 hash functions, the probability for a pre-image hit of $y_2 + i + 1$ is $\leq 2^{-778}$. This means that the Hash_DRBG provides forward secrecy, i.e., fulfills Requirement DRG.3.5.

1064 [backward secrecy] The proof of the backward secrecy property can be organized analogously. The only difference is that in place of the pre-images $v_{y_2} + j + 1$ or v_{y_2+1} , the pre-images $v_{y_1} + i - 1$ or $v_{y_1-1} + m_{y_1-1}$ (if $i = 0$), respectively, have to be considered. In particular, the Hash_DRNG fulfills the backward secrecy requirement, i.e., fulfills Requirement DRG.3.6.

1065 [enhanced backward secrecy] Next, we verify that the Hash_DRBG has enhanced backward secrecy. We assume that an adversary knows the internal state of the Hash_DRBG after request y , namely $(v_{y+1}, c, y + 1)$. Assume further that $\tilde{v}_y = \phi_{\text{req}}(v_y, a_y)$. To violate the enhanced backward secrecy property, an adversary has to determine any of the random numbers

$\psi(\tilde{v}_y), \psi(\tilde{v}_y + 1), \dots, pr_u(\psi(\tilde{v}_y + m_y - 1))$ of **request** y or to guess them with significantly larger probability than without knowledge of the **internal state**. (Of course, knowledge of $\tilde{v}_y$ or (v_y, a_y) would solve this problem.) In particular,

$$(v_{y+1}, c, y + 1) = \phi_B(\tilde{v}_y, c, y) = (\tilde{v}_y + g(\tilde{v}_y) + c + y \pmod{2^{\text{seedlen}}}, c, y + 1) \quad \text{and} \quad (5.79)$$

$$\tilde{v}_y = v_y + f(v_y, a_y) \pmod{2^{\text{seedlen}}}. \quad (5.80)$$

If $a_y = \emptyset$, then $v_y = \tilde{v}_y$. Assume that $a_y \neq \emptyset$. By (5.58), it would not be easier to determine v_y first instead of $\tilde{v}_y$ even if an **adversary** knew the value $f(v_y, a_y)$. In both cases, an **adversary** had to solve an equation of the following type: $v + g(v) \equiv c \pmod{2^{\text{seedlen}}}$ with known right-hand side c .

[**enhanced backward secrecy**] By the assumption in par. 1065, the values c and y are known. 1066
Hence, (5.79) allows the **adversary** to determine the sum $\tilde{v}_y + g(\tilde{v}_y)$, or more precisely, $\tilde{v}_y + g(\tilde{v}_y) \pmod{2^{\text{seedlen}}}$. (In very rare cases, $\tilde{v}_y + g(\tilde{v}_y)$ may exceed the modulus 2^{seedlen} .) As pointed out in the note of par. 1046, this allows finding an interval I_b of length 2^{outlen} , which contains the pre-image $\tilde{v}_y$. By (5.66), an **adversary** cannot solve this pre-image problem practically.

[**enhanced backward secrecy**] Recall that the **adversary** searches the image $\psi(\tilde{v}_y + i)$ for some 1067
 $i \leq 2^{12}$. By par. 1066, the knowledge of $\tilde{v}_y + g(\tilde{v}_y)$ does not suffice to determine $\tilde{v}_y$. Thus, the **adversary** is not able to guess any of the random numbers $\psi(\tilde{v}_y + i)$ with significantly greater probability than without knowledge of the current **internal state** $(v_{y+1}, c, y + 1)$. Moreover, by (5.63) $\text{Hash}_{\text{dr}}(\cdot, \text{seedlen})$ is a **one-way function**. Hence, it is not feasible to determine the first **internal state** v_1 from c or to guess it with significantly greater probability than without knowledge of c (cf. par. 1033). (Of course, the knowledge of v_1 (together with the knowledge of the **additional input** data $a_1, \dots, a_y$) would allow recovering all the previous random numbers.) Hence, the Hash_DRBG fulfills the **enhanced backward secrecy** requirement, i.e., Requirement DRG.3.7; cf. also application note par. 214.

[**effective internal state**] We reconsider the **backward secrecy** and **forward secrecy** properties of 1068
 Hash_DRBG . The set of all **internal states** is $S = S' \times S' \times \mathbb{Z}_{2^R}$. The third component equals the number of the next **request** and may be publicly known. Since $\psi(\cdot)$ is a **one-way function**, it is not possible to determine any intermediate value $\tilde{v}_y$ from the random numbers in (5.57) or (in a more favorable scenario for the **adversary**) from (5.69), even if $y_1 = 1$, $y_2 = 2^{48}$, and if for each v_y the subsequence of random numbers has maximum length. In particular, it is not possible to get $v_{y+1} = pr_{(S')}(\phi_B(\tilde{v}_y, c, y))$ (projection onto the first component of S) or $v_y \in \phi_{\text{req}}^{-1}(\tilde{v}_y)$. Furthermore, c is the image of v_1 under a **one-way function** (cf. (5.63), and pars. 1029 and 1033). Thus, it would require knowledge of v_1 to compute c . The set of **effective internal states** is given the cartesian product $S' \times S'$.

[**request** requirement] By specification, the Hash_DRBG fulfills Requirement DRG.3.2. 1069

[**additional input**] Since $f(v, a)$ (if $a \neq \emptyset$) (5.43) and $g(v)$ (5.44) are closely related to Hash , 1070
targeted chosen input attack are not feasible because v_y is unknown. Hence, the Hash_DRBG fulfills Requirement DRG.3.8.

[**cryptographic functions**] Both the state transition function ϕ and the output function ψ are 1071
cryptographic (composition of cryptographic primitives). Thus, the Hash_DRBG fulfills Requirement DRG.3.9.

- 1072 [statistical tests] The first components of the **internal states**, $v_1, v_2, \dots$, and also the intermediate values $\tilde{v}_1, \tilde{v}_2, \dots$, are images of v_1 under the repeated application of two **one-way functions** (possibly affected by **additional input** values $a_1, a_2, \dots$). For each intermediate value $\tilde{v}_y$, less than 2^{12} random numbers $\psi(\tilde{v}_y), \psi(\tilde{v}_y + 1), \dots$ are generated. The random numbers are the (possibly truncated) hash values of input values that are mutually distinct with overwhelming probability (cf. par. 1063 ff.). If the sequence of random numbers (interpreted as a binary sequence) within a **seedlife** would fail any *fair* statistical tests (i.e., tests that do not exploit the knowledge of the **internal state**) significantly more often than expected for random numbers from an ideal **RNG**, this would point to inherent weaknesses of the hash function. Since, for the permitted hash functions (5.34), no statistical weaknesses are known, we may assume that the Hash_DRBG fulfills Requirement DRG.3.10.
- 1073 [summary] The Hash_DRBG fulfills the algorithmic requirements DRG.3.2 (par. 1069), DRG.3.3 (pars. 1016, 1068), DRG.3.5 (pars. 1053 to 1063), DRG.3.6 (par. 1064), DRG.3.7 (pars. 1065 to 1067), DRG.3.8 (par. 1070), DRG.3.9 (par. 1071), and DRG.3.10 (par. 1072).
- 1074 [(re-)seeding] Compliance with functionality class DRG.3 further demands the verification of the Requirements DRG.3.1 and DRG.3.4. These requirements also concern the **RNG** that generates the **seed material** for the **seeding procedure/reseeding procedure** and thus require case-by-case considerations. General advice can be found in Subsection 4.4.2, and concrete examples, also for the Hash_DRBG, are discussed in the application notes to Requirement PTG.3.5. (There the Hash_DRBG is considered as a **cryptographic post-processing** algorithm.)
 Example: Assume that the entropy_input string is generated by a **TRNG** that is compliant with functionality class PTG.2, PTG.3, or NTG.1 and has a **min-entropy** claim ≥ 0.98 bits of **min-entropy** per **internal random number** bit. Assume further that the entropy_input string comprises at least 256 bits. Then the **min-entropy** of the entropy_input string is ≥ 250.88 . Then the **min-entropy** of the $S' = \{0, 1\}^{440}$ is ≥ 249 with probability $1 - 2^{-16}$.
 Note 1: As in the example above one shows that the entropy_input string can also be generated by a DRG.4-compliant **DRNG**. There, by assumption, the entropy_input has 250 bits of **min-entropy**, and again the **min-entropy** of the **effective internal state** $S' = \{0, 1\}^{440}$ is ≥ 249 with probability $1 - 2^{-16}$. (Note that by Requirement 2.4 the DRG.4-compliant **DRNG** has to be provided with **enhanced forward secrecy** before the generation of the entropy_input string.)
 Note 2: For a **TRNG** with **min-entropy** claim that is not compliant with any of the functionality classes PTG.2, PTG.3, or NTG.1, the minimal bit length of entropy_input depends on the guaranteed individual lower **min-entropy** bound.
 Note 3: It is also permitted to generate the entropy_input string of at least 256 bits with a PTG.2-compliant **PTRNG** that satisfies the **Shannon entropy** claim. In this case, the **Shannon entropy** of the entropy_input string is ≥ 255.95 .
 Note 4: Note 3 is intuitively clear: If the length of the entropy_input string is ≈ 256 bits (with given (fixed) nonce and personalization_string, resp. with given additional_input), the projection of $\phi_{seed}(\cdot, \cdot)$, resp. $\phi_{reseed}(\cdot, \cdot)$, onto the first component should be “almost” injective, thus losing only marginal **entropy**. If the length of the entropy_input increases, the defect caused by collisions increases much slower than the overall **entropy** of the entropy_input string.
- 1075 [(re-)seeding] The second component of the **internal state**, $c = c(v_1)$, does not increase the **entropy** of the **effective internal state** because c is a function of v_1 .

1076

[additional input, PTG.3] If the **fresh entropy** is introduced by **additional input**, the amount of **entropy** is bounded by the output length of the function $f(\cdot, \cdot)$, or equivalently, by the output length of the applied hash function *Hash*, resp. by the bit length of a single **internal random number**. This is an important observation if the Hash_DRBG is considered as a **cryptographic post-processing** algorithm for a PTG.3-compliant **PTRNG**. In contrast, introducing **fresh entropy** by the **reseeding procedure** allows the introduction of (almost) *seedlen* bits of **entropy**.

5.3.2 Security Evaluation of the HMAC_DRBG [SP800-90A]

In this subsection, we analyze the conformity of the HMAC_DRBG to the requirements of functionality class DRG.3. The relevant parts of the analysis refer to [Kels24]; cf. pars. 1101, 1102, and 1103. 1077

For a detailed description of the HMAC_DRBG we refer to [SP800-90A], Subsubsection 10.1.2. 1078

[Additional input] The function HMAC_DRBG_Update differs significantly depending on whether additional input is used or not. With regard to par. 1088, Note, par. 1103, and par. 108, we recommend to always use additional input. The additional input can be short, constant, and publicly known. 1079

In the following, we assume 1080

$$\text{Hash} \in \{\text{SHA-256}, \text{SHA-512/256}, \text{SHA-384}, \text{SHA-512}, \text{SHA3-256}, \text{SHA3-384}, \text{SHA3-512}\}. \quad (5.81)$$

The hash function *Hash* outputs strings of length *outlen* $\in \{256, 384, 512\}$ bits as indicated by its name; if the name includes two numbers the output length is indicated by the second number. Note 1: Unlike for the Hash_DRBG (5.34) the hash functions SHA-224, SHA-512/224, and SHA3-224 are not permitted. The reason is that the **effective internal state** would be too small; cf. pars. 1084 and 1104.

Note 2: As for the Hash_DRBG (cf. par. 1011) the hash functions SHA3-224, SHA3-256, SHA3-384, SHA3-512 are not compliant with the specification of the HMAC_DRBG in the current version of SP 800-90A [SP800-90A].

Our first goal is to describe the HMAC_DRBG by a 9-tuple (par. 129) and by the 4-tuples for the **seeding procedure** (par. 143) and **reseeding procedure** (par. 147). 1081

In this subsection, $S' := \{0, 1\}^{\text{outlen}}$. 1082

The internal space of the HMAC_DRBG (denoted as working state in [SP800-90A]) is given by the cartesian product 1083

$$S := S' \times S' \times \mathbb{Z}_{2^R}. \quad (5.82)$$

The third component in (5.82), $\mathbb{Z}_{2^R}$, is assumed to be large enough that it contains each reasonable *reseed_counter* value. The elements of the **internal state** space *S* are triples (k, v, rc) . The values *k* and *v* are kept secret while the value of the *reseed_counter* *rc* is publicly known. The

reseed_counter is initialized by 1 and incremented by 1 after each **request**.
 Note: The set $\mathbb{Z}_{2^R}$ does not have to be explicitly stated.

1084 The first component S' of $S = S' \times S' \times \mathbb{Z}_{2^R}$ is the **effective internal state**; cf. par. 1104.

1085 Furthermore,

$$S_{req} := S' \times S' \quad (5.83)$$

$$A := \{0, 1\}^* \quad (5.84)$$

$$R := \{0, 1\}^{outlen} \quad (5.85)$$

$$I := \mathbb{Z}_{2^{19}}. \quad (5.86)$$

The bit length of **additional input** $a \in A$ is $\leq 2^{35}$. Empty strings are possible (denoted by o).

1086 Next, we define the state transition function ϕ , the output function ψ , the function ϕ_{req} (generating the internal **request** state), and the **request** state transition function ϕ_0 . First, we repeat the definition of the function HMAC_DRBG_Update in [SP800-90A] (up to the order of its arguments). In pars. 1087 and 1088, we use an algorithmic notation to simplify reading.

1087 The function

$$\begin{aligned} \text{HMAC_DRBG_Update}: S' \times S' \times \{0, 1\}^* &\rightarrow S' \times S', \\ \text{HMAC_DRBG_Update}(k, v, a) & \end{aligned} \quad (5.87)$$

is given by

$$k := \text{HMAC}(k, v \| 0x00 \| a) \quad (5.88)$$

$$v := \text{HMAC}(k, v) \quad (5.89)$$

if $a \neq o$ then {

$$k := \text{HMAC}(k, v \| 0x01 \| a) \quad (5.90)$$

$$v := \text{HMAC}(k, v) \quad (5.91)$$

}

$$\text{return } (k, v). \quad (5.92)$$

In particular, HMAC_DRBG_Update(k, v, a) updates k and v .

1088 In the formal description by the describing 9-tuple, at the end of each **request**, the **internal state** is updated by the state transition function ϕ ; cf. par. 1089, Note. In particular,

$$\phi: S \times A \times I \rightarrow S, \quad \phi(k, v, rc, a, p) := \quad (5.93)$$

is given by

```

if  $a \neq o$ , then  $(k, v) := \text{HMAC\_DRBG\_Update}(k, v, a)$ 
for  $j := 1$  to  $m$  do {
     $v := \text{HMAC}(k, v)$ ;
     $r_j := v$ ;
}
 $(k, v) := \text{HMAC\_DRBG\_Update}(k, v, a)$ 
return  $(k, v, rc + 1)$ .

```

(5.94)

As in par. 131, it is $m := \lceil \frac{p}{\text{outlen}} \rceil$.

Note: For $a \neq o$, the “mixing properties” of `HMAC_DRBG_Update` are stronger than in the case where $a = o$ because four HMAC operations are performed instead of only two. This has impact on the state transition function and thereby on the security properties of the HMAC_DRBG; cf. [WoSh18].

The internal **request** state s_{req} is generated by

1089

$$\phi_{req}: S \times A \rightarrow S_{req}, \quad \phi_{req}(k, v, rc, a) := \begin{cases} \text{HMAC_DRBG_Update}(k, v, a) & \text{if } a \neq o \\ (k, v) & \text{if } a = o. \end{cases} \quad (5.95)$$

Finally,

1090

$$\phi_0: S_{req} \rightarrow S_{req}, \quad \phi_{req}(k, v) := (k, \text{HMAC}(k, v)) \quad (5.96)$$

$$\psi: S_{req} \rightarrow R, \quad \psi(k, v) := \text{HMAC}(k, v). \quad (5.97)$$

This completes the specification of the describing 9-tuple.

Note: From a logical point of view, the **internal state** is updated by the state transition function ϕ (cf. (5.93) and (5.94) for details), and each value $v := \text{HMAC}(k, v)$ in the for-loop of (5.92) is computed two more times (by ϕ_0 and ψ). The implementation, of course, computes each value only once (by ψ), and s_{req} equals the first two components of the current **internal state** s . The **request** counter rc is initialized by 1 and is increased by 1 after each **request**.

1091

The following paragraphs provide formal descriptions of the **seeding procedure** and the **reseeding procedure**.

1092

The initial **internal state** is computed from **seed material** (denoted by `seed_material` in [SP800-90A]). Like for the Hash_DRBG

1093

$$\text{seed_material} = (\text{entropy_input} \parallel \text{personalization_string}). \quad (5.98)$$

The requirements for the `seed_material` and its components match with those for the Hash_DRBG; cf. par. 1025 for details.

In the notation of the 4-tuple, which describes the **seeding procedure** (par. 143),

1094

$$SM = PS = \{0, 1\}^*. \quad (5.99)$$

1095 The nonce and the personalization_string are constructed from the current value of PS .

1096 The first initial state (k_1, v_1, rc) is computed from seed_material via

$$\begin{aligned} \phi_{seed}: \{0, 1\}^* &\rightarrow \{0, 1\}^{outlen} \times \{0, 1\}^{outlen} \times \mathbb{Z}_{2^R}, \\ \phi_{seed}(\text{seed_material}) &:= (\text{HMAC_DRBG_Update}(k_0, v_0, \text{seed_material}), 1) = (k_1, v_1, rc) \\ &\text{with } k_0 := 0x00 \dots, 00 \quad \text{and } v_0 := 0x01 \dots, 01. \end{aligned} \quad (5.100)$$

Note: In the **seeding procedure**, the **bit string** seed_material (5.98) is the concatenation of values in SM (entropy_input) and PS (nonce and personalization_string).

1097 For the **reseeding procedure**,

$$\text{seed_material} = (\text{entropy_input} \parallel \text{additional_input}). \quad (5.101)$$

The maximum bit length of both the entropy_input and the additional_input is 2^{35} [SP800-90A], Table 2. The additional_input may be empty. The additional_input belongs to the set PS .

1098 In the notation of the 4-tuple which describes the **reseeding procedure** (par. 147),

$$SM = PS = \{0, 1\}^*. \quad (5.102)$$

1099 The first initial state (k_1, v_1, rc) after the **reseeding procedure** is computed from seed_material via

$$\begin{aligned} \phi_{reseed}: S \times \{0, 1\}^* &\rightarrow \{0, 1\}^{outlen} \times \{0, 1\}^{outlen} \times \mathbb{Z}_{2^R} \\ \phi_{reseed}((k, v, rc), \text{seed_material}) &:= (\text{HMAC_DRBG_Update}(k, v, \text{seed_material}), 1) \\ &= (k_1, v_1, 1). \end{aligned} \quad (5.103)$$

The first two arguments of HMAC_DRBG_Update($\cdot, \cdot, \cdot$), k, v , denote the current values of the **internal state** before reseeding.

1100 When proving the conformity to functionality class DRG.3, the main difficulty lies in verifying that the **DRNG** fulfills **backward secrecy**, **forward secrecy**, and **enhanced backward secrecy**. If the conformity to functionality class DRG.4 is targeted, **enhanced forward secrecy** has to be verified, too. For a security proof of the HMAC_DRBG, we refer to [Kels24]. This paper uses the random oracle model. It is proved that specified games cannot be won by an **attacker**.

1101 [**Forward secrecy**] The HMAC_DRBG fulfills Requirement DRG.3.5 (**forward secrecy**); see [Kels24].

1102 [**Backward secrecy**] The HMAC_DRBG fulfills Requirement DRG.3.6 (**backward secrecy**); see [Kels24] for details.

1103 [**Enhanced backward secrecy**] The HMAC_DRBG fulfills Requirement DRG.3.7 (**enhanced backward secrecy**). Assume that an adversary knows the internal state (k^*, v^*, rc^*) . If $a = o$, then

guessing n bits of the unknown value v in $v^* = \text{HMAC}(k^*, v)$ (5.92) is a $\min\{240, n\}$ -bit problem, as required in par. 214. If $a \neq o$, then the HMAC_DRBG even fulfills the backtracking resistance. See [Kels24] and [WoSh18] for details.

[effective internal state] The set of admissible internal states is $S = S' \times S' \times \mathbb{Z}_{2^R}$. The third component equals the number of the next request and may be publicly known, and each random number equals the v -value of an intermediate internal state; cf. (5.93), for-loop. Hence, the admissible effective internal states are contained in $S' = \{0, 1\}^{\text{outlen}}$, the first component of the set of admissible internal states. On the other hand, due to the properties of ϕ and ψ , ultimately due to the one-way function property of the functions HMAC and Hash, the admissible effective internal states equal $S' = \{0, 1\}^{\text{outlen}}$. Hence, Requirement DRG.3.3 is fulfilled. Due to the properties of the seeding procedure and reseeding procedure, Requirement DRG.3.4 is fulfilled if the seed_material contains at least 240 bits of min-entropy. Alternatively, under suitable conditions (cf. par. 114, third bullet point) it is accepted if the seed_material contains at least 250 bits of Shannon entropy. 1104

[request requirement] By specification, the Hash_DRBG fulfills Requirement DRG.3.2. 1105

[additional input] Due to the properties of the HMAC function, the HMAC_DRBG fulfills Requirement DRG.3.8. 1106

[cryptographic functions] Both the state transition function ϕ and the output function ψ are cryptographic and composed of widely recognized cryptographic primitives. Thus, the HMAC_DRBG fulfills Requirement DRG.3.9. 1107

[statistical tests] Per request at most $2^{19}/\text{outlen} \leq 2^{19-8} = 2^{11}$ internal random numbers are generated if $\text{outlen} \geq 256$. In the following, we assume that at most 2^{48} requests are generated. Thus, at most $2^{48+11} = 2^{59}$ internal random numbers are generated until reseeding. The internal random numbers are computed from the current internal state (k, v, rc) via $\text{HMAC}(k, v)$. Below, we estimate the probability for a collision of the pre-images (k, v) of the HMAC function until reseeding. It is important to bear in mind that the first component of the internal state remains constant within a request. The probability that no collision of pre-images (k, v) in request j occurs under the condition that no collision has occurred within the requests $1, \dots, j-1$ is 1108

$$\geq \left(1 - \frac{(j-1)2^{11}}{2^{2\text{outlen}}}\right) \cdot \prod_{i=1}^{2^{11}} \left(1 - \frac{i-1}{2^{\text{outlen}}}\right). \quad (5.104)$$

The first factor covers the update mechanism of the internal state, which is performed at the end of request $j-1$ (and, for non-empty additional input, additionally at the beginning of request j) and compares the current pair (k, v) before the for-loop with all pairs (k, v) within in the requests $1, \dots, j-1$. The second factor covers the updates of the internal state within the for loop in (5.93). Since k remains fixed until the end of the request, only the second component has to be considered. In particular, this means that the denominator is 2^{outlen} . Altogether, from

(5.104) we obtain

$$\begin{aligned}
 & \text{Prob (collision of pre-images until reseeding)} \leq \\
 & 1 - \left(\prod_{j=1}^{2^{48}} \left(1 - \frac{(j-1)2^{11}}{2^{2\text{outlen}}} \right) \right) \cdot \left(\prod_{i=1}^{2^{11}} \left(1 - \frac{i-1}{2^{\text{outlen}}} \right) \right)^{2^{48}} = \\
 & 1 - \left(\prod_{j=1}^{2^{48}} \left(1 - \frac{(j-1)2^{11}}{2^{2\text{outlen}}} \right) \right) \cdot \prod_{i=1}^{2^{11}} \left(1 - \frac{(i-1)2^{48}}{2^{\text{outlen}}} \cdot \frac{1}{2^{48}} \right)^{2^{48}} \approx \\
 & 1 - e^{-2^{48+48-1+11-2\text{outlen}}} \prod_{i=1}^{2^{11}} e^{-\frac{(i-1)2^{48}}{2^{\text{outlen}}}} \approx 1 - e^{-2^{106-2\text{outlen}}} e^{-\frac{2^{21}2^{48}}{2^{\text{outlen}}}} = 1 - e^{-2^{175-3\text{outlen}}}. \quad (5.105)
 \end{aligned}$$

To estimate the first product, we applied the linear Taylor expansion of the exponential function, and for the second product, we used the well-known fact that $(1 + \frac{x}{n})^n$ converges to e^x as n tends to infinity. Hence, a collision of pre-images occurs only with negligible probability. Due to the properties of HMAC and *Hash*, we may assume that the **internal random numbers** are generated from randomly selected pre-images. Since for the permitted hash functions (5.81) no statistical weaknesses are known, we may assume that the HMAC_DRBG fulfills Requirement DRG.3.10. Note: Requirement DRG.3.2 does not limit the number of **requests** between successive (re-)seeding procedures. As in Subsection 5.3.1 (cf. par. 1045, Note), for the above computations, we assumed an upper limit of 2^{48} requests and, consequently, an upper limit of 2^{59} **internal random numbers**. Larger (realistic) upper bounds can also be used and substituted into the computations. Of course, different upper bounds influence the numerical results to some degree.

- 1109 [summary] The Hash_DRBG fulfills the algorithmic requirements DRG.3.2 (par. 1105), DRG.3.3 (pars. 1084, 1104), DRG.3.5 (pars. 1101), DRG.3.6 (par. 1102), DRG.3.7 (pars. 1103), DRG.3.8 (par. 1106), DRG.3.9 (par. 1107), and DRG.3.10 (par. 1108).
- 1110 [(re-)seeding] Compliance with functionality class DRG.3 further demands the verification of the Requirements DRG.3.1 and DRG.3.4. These requirements also concern the **RNG** that generates the **seed material** for the **seeding procedure/reseeding procedure**, and thus require case-by-case considerations; cf. pars. 1111 to 1113.
- 1111 [update function, **min-entropy**] Below we analyze the impact of the HMAC_DRBG_Update function (introduced in par. 1099) on the **min-entropy** when additional data a (e.g., **seed material**) are provided. The “core” of HMAC_DRBG_Update is the HMAC function, which is applied four times. As in par. 393 and par. 412 we assume that $\text{HMAC}(k, w) = \text{Hash}((k \oplus \text{opad}) \parallel \text{Hash}((k \oplus \text{ipad}) \parallel w))$ can be interpreted as a composition of two random mappings χ_1 and χ_2 where $\chi_1(k, w) := \text{Hash}((k \oplus \text{ipad}) \parallel w)$ and $\chi_2(k, y) := \text{Hash}((k \oplus \text{opad}) \parallel y)$. In particular, $\text{HMAC}(k, w) = \chi_2(k, \chi_1(k, w))$. The analysis of the random mappings is similar to the approach in the application notes to Requirement PTG.3.5.
- 1112 [update function, **min-entropy**] Example: In this example $\text{Hash} = \text{SHA-256}$. The additional input a is interpreted as a realization of a random variable Z with $H_{\min}(Z) = 250$. One after the other we analyze the impact of the HMAC operations (5.88) to (5.91). We begin with (5.88) and assume that the **internal state** (k, v) is known. Unlike in par. 412 we regard (k, v) as part of the input instead of parameters. The latter would be appropriate

for (5.88) but not for (5.89) to (5.91). All computations are treated as for type (ii) specified in par. 404 because $n^- < m$ in all cases. For the sake of readability we omit the constants 0x00 (5.88) and 0x01 (5.90) from the arguments of the function χ_1 . We first note that $H_{\min}(k, v, Z) = H_{\min}(Z) = 250$ so that $(n^-, m) = (250, 256)$. Applying (4.202) implies that $H_{\min}(\chi_1(k, v, Z)) \geq 245.1$ with probability $\geq 1 - 2^{-16}$. In the second step, $k^* = \chi_2(k, \chi_1(k, v, Z))$ is computed. Since $H_{\min}(k, \chi_1(k, v, Z)) = H_{\min}(\chi_1(k, v, Z))$ we set $(n^{-(2)}, m) = (245, 256)$. This results in $H_{\min}(k^*) \geq 240.7$, also with probability $\geq 1 - 2^{-16}$. Also below, even if not always mentioned, with regard to par. 394 all computed **min-entropy** bounds are undercut only with probability $\leq 2^{-16}$.

Next, (5.89) computes $v^* = \chi_2(k^*, \chi_1(k^*, v))$. Here $H_{\min}(k^*, v) = H_{\min}(k^*) \geq 240.7$, thus $(n^-, m) = (240, 256)$, and $H_{\min}(\chi_1(k^*, v)) \geq 236.1$ with probability $\geq 1 - 2^{-16}$. In the second step $\chi_2(k^*, \chi_1(k^*, v))$ is computed. Since $H_{\min}(k^*, \chi_1(k^*, v)) \geq \max\{H_{\min}(k^*), H_{\min}(\chi_1(k^*, v))\} \geq 240$ we (cautiously) set $(n^{-(2)}, m) = (240, 256)$, resulting in $H_{\min}(v^*) \geq 236.1$ with probability $\geq 1 - 2^{-16}$.

In the third step (5.90) $k^{**} = \chi_2(k^*, \chi_1(k^*, v^*, Z))$ is computed. First of all, $H_{\min}(k^*, v^*, Z) = H_{\min}(Z) = 250$ because k^* and v^* are functions of Z . Thus, as in (5.88), $(n^-, m) = (250, 256)$ and $H_{\min}(\chi_1(k^*, v^*, Z)) = 245.193$. Furthermore, $H_{\min}(k^*, \chi_1(k^*, v^*, Z)) \geq \max\{H_{\min}(k^*), H_{\min}(\chi_1(k^*, v^*, Z))\} \geq 245.1$. We set $(n^{-(2)}, m) = (245, 256)$, which results in $H_{\min}(k^{**}) \geq 240.7$.

Finally, the fourth HMAC operation (5.91) computes $v^{**} = \chi_2(k^{**}, \chi_1(k^{**}, v^*))$. Again, we conclude $H_{\min}(k^{**}, v^*) \geq \max\{H_{\min}(k^{**}), H_{\min}(v^*)\} \geq 240.7$. Hence $(n^-, m) = (240, 256)$ and thus $H_{\min}(\chi_1(k^{**}, v^*)) \geq 236.1$. Then $H_{\min}(k^{**}, \chi_1(k^{**}, v^*)) \geq \max\{H_{\min}(k^{**}), H_{\min}(\chi_1(k^{**}, v^*))\} \geq 240.7$. Finally $(n^{-(2)}, m) = (240, 256)$, and $H_{\min}(v^{**}) \geq 236.1$.

Note: In (5.89) to (5.91) we estimated the **min-entropy** of several 2-tuples downwards by the maximum of the individual **min-entropies**. It is therefore likely that the true **min-entropy** bounds for v^*, k^{**}, v^{**} are actually larger than stated above.

[Seeding procedure, reseeding procedure, enhanced forward secrecy] For $Hash = \text{SHA-256}$ it follows immediately from the example treated in par. 1112 that if the **seed material** has ≥ 250 bits of **min-entropy**, the initial **effective internal state** (after the **seeding procedure** or the **reseeding procedure**) has **min-entropy** ≥ 240 bits; cf. par. 1112 (third step) and par. 1104. The same applies if the additional high-entropy **additional input** has ≥ 250 bits of **min-entropy**. 1113

Note 1: For the hash functions in (5.81) with output size larger than 256 the **entropy** loss caused by the HMAC_DRBG_Update function (comparison of the **min-entropy** of the additional data with the **min-entropy** of the **effective internal state**) is smaller than for SHA-256. In particular, the above conclusions are valid for all hash functions in (5.81).

Note 2: **Enhanced forward secrecy** (cf. Requirement DRG.4.10) can be provided if the **seed material** for the **reseeding procedure** or high-entropy **additional input** has ≥ 250 bits of **min-entropy**.

Note 3: If the **seed material** comprises ≥ 256 bits the HMAC_DRBG satisfies the Requirements DRG.3.4 and DRG.1.6 for all hash functions in (5.81), as far as (re-)seeding with a DRG.4-compliant **DRNG** is concerned. Furthermore, Requirement DRT.1.7 is satisfied, too.

5.4 Noise Sources and Stochastic Models

Subsection 5.4.1 mentions examples of different **noise sources** that are often used by **PTRNGs** and **NPTRNGs**. In Subsection 5.4.2 to 5.4.6, several **stochastic models** are discussed and ana- 1114

lyzed. Subsections 5.4.2, 5.4.5, and 5.4.6 treat concrete designs of **physical noise sources** while Subsections 5.4.3 and 5.4.4 focus on the mathematical analysis of generic designs. Several designs of **physical noise sources** fit these generic designs.

5.4.1 Examples of Physical and Non-physical Noise Sources

- 1115 Below, a number of **noise sources** are mentioned that are used by **PTRNGs** (pars. 1117 to 1124) and **NPTRNGs** (pars. 1126 to 1129). This list does not claim to be complete and does not provide any kind of quality assessment. **AIS 20** and **AIS 31** are technology neutral. The applicant has to provide evidence that the requirements of the pursued functionality class are fulfilled.
- 1116 The quality of a **PTRNG** does not only depend on the analog part of the **physical noise source** but on the whole design, including the **digitization** mechanism. Due to (inadvertent) band-pass filtering, inherent noise, and probabilistic detection, for example, a **digitization** mechanism may undesirably blur even a physically perfect noise signal or introduce dependencies between samples. For this reason, this document assumes that the **digitization** mechanism (and, if applicable, the sampling mechanisms) is part of the **physical noise source** of a **PTRNG**. This applies to **non-physical noise sources** as well.
- 1117 [Shot **entropy** of a tube diode] The shot **entropy** of a parallel-plane temperature-limited tube diode is non-deterministic. The number of electrons emitted from the tube's cathode during a time interval follows a Poisson distribution, cf. [DaRo87], Section 7-2.
- 1118 [Thermal resistive **entropy**] The voltage between resistors varies randomly due to the vibration of atoms. Ideally, the thermal **entropy** signal has the same energy in all frequency bands (so-called "white noise"). Sampling an ideally-amplified white noise signal would generate a sequence of independent bits.
- 1119 [Semi-conductor diode breakdown **entropy**] The reverse current through semi-conductor diodes varies randomly due to the tunneling of electrons. The power of the **entropy** signal is inversely proportional to the frequency.
- 1120 [Free running oscillators] Free running oscillators generate digital signals with an edge-to-edge random analog time drift (jitter). Sampling a fast oscillator by a lower frequency oscillator generates a random bit signal. If the standard deviation of the slow oscillator is considerably greater than the fast period, the sampled bit sequence may be expected to be (nearly) uncorrelated.
- 1121 [Designs based on metastability in digital circuits] This comprises various designs where parts of a digital circuit are forced into a state between logic levels "0" and "1" to induce unpredictable behavior.
- 1122 [Chaos-based **noise source**] This comprises designs whose behavior is highly sensitive to small variations (e.g., in voltage, current, or time due to inherent noise). The **entropy** of the **raw random numbers** results from the **entropy** introduced by physical disturbances and the **noise source's** ability to amplify them and make them measurable ([BuLu08; BuLu16]). Although classical (mathematical) chaos theory only considers variations of the initial conditions, this kind

of modeling (ultimately, a **DRNG** with unlimited **entropy** in the **seed material**) is not appropriate for real-world **PTRNGs** and will not be accepted. Instead, it has to be shown that the average supply of **entropy** to the system exceeds the output rate.

[Radioactive atomic disintegration] The number of decay events (detected particles) per time interval follows a Poisson distribution; see Subsection 5.4.5. 1123

[Quantum **noise source**] Quantum **RNGs** exploit physical phenomena that contain randomness according to the laws of quantum mechanics. This document does not distinguish between quantum **entropy** and **entropy** from physical phenomena based on other physical models. **AIS** 31 considers quantum **RNGs** as **PTRNGs** already because of the **digitization** mechanism that transfers the analog data to **raw random numbers**. It should further be mentioned that most **physical noise sources** exploit effects from quantum mechanics (even if not explicitly mentioned). 1124

Pars. 1126 to 1129 consider **noise sources** for **NPTRNGs**. 1125

[General system data] A computer (e.g., a PC or a server) offers a variety of possibilities to collect data that are non-deterministic. It should be noted, however, that many **noise sources** deliver information that is comparatively easy to guess, to influence, or to determine. Examples: network data, file system or process header information, threads, current time and date, time since system start, disk I/O operations, interrupts, etc. 1126
Note: The reference [Linux_RNG_2024] treats the Linux /dev/random and /dev/urandom.

[Time stamps] If available (e.g., CPU instruction RDTSC), a highly precise time stamp counter can be used to generate data that are hard to predict by an **adversary**. The least significant bits of time stamps should be affected by all activities currently running on the computer. Under suitable conditions, virtualization does not negatively influence the suitability of time stamps as **noise sources** for **NPTRNGs**; cf. [Linux_RNG_2022; RNG_virtual_env]. 1127

[Human interaction] Input data generated by the user (e.g., mouse movement and key strokes) usually contain little **entropy**. In order to generate a (considerable) amount of **entropy** from human interaction, the computer needs to apply highly precise time resolution (similar to par. 1127). 1128

[Software execution jitter] This approach uses specially crafted software whose execution time varies greatly. The cause for variances in the execution times depends on the platform; see [Jitter-RNG]. 1129

Note: Applicants for certificates, designers, and evaluators can apply the results and strategies from Subsections 5.4.2 to 5.4.6. But, of course, it has to be verified that the assumptions are justified for the **noise source** under evaluation. 1130

5.4.2 PTRNG with Two Noisy Diodes

In Subsection 5.4.2, we discuss a **PTRNG** design that exploits two noisy diodes. The design is analyzed, relevant conclusions are summarized, and finally, a **stochastic model** is developed. For 1131

many details, we refer the interested reader to [KiSc08]. This and related designs are also treated in [Schi09b]. Some new considerations are added in the following. We mention that this design is also treated in ISO/IEC 20543 [ISO_20543], A.3.4, Example 2.

- 1132 As in [KiSc08] in Subsection 5.4.2, only Shannon entropy is considered.
- 1133 [Zener diodes] Zener diodes have a reverse avalanche effect (depending on diode type, 3 – 4 V or about 10 V) and produce more than 1mV of noisy voltage with a cutoff frequency of about 10 MHz. The flicker noise in Schottky diodes is associated with static current flow in both resistive and depletion regions, caused by traps due to crystal defects and contaminants, which randomly capture and release carriers.
- 1134 [design] Fig. 22 illustrates the PTRNG design that is discussed in Subsection 5.4.2. The circuit of

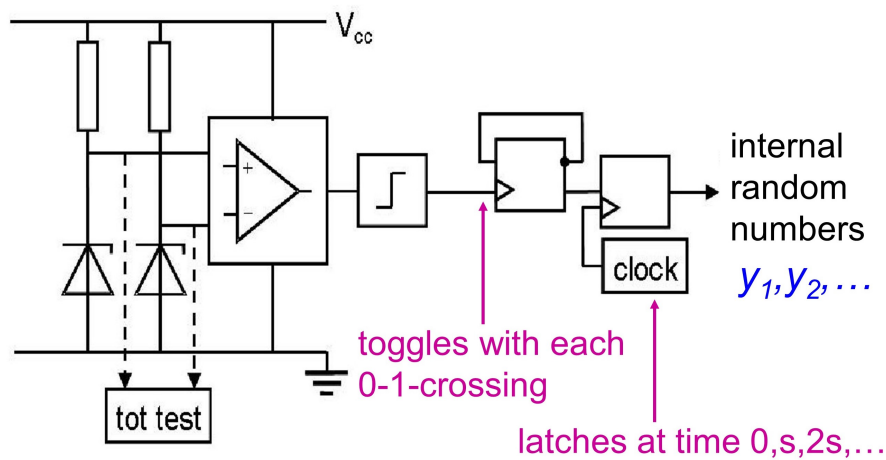


Figure 22: PTRNG with two noisy diodes (schematic design), created by W. Killmann

the AC coupling, the negative feedback for the operational amplifier, the stabilizing mechanism for the power supply, and compensating effects of temperature have been omitted for clarity in Fig. 22.

- 1135 [design] In Fig. 22, the outlets of two identical noisy diodes provide the input to an operational amplifier. The operational amplifier applies bandpass filters and amplifies the difference of the noisy voltages (with a very high amplification rate). Its output voltage is fed into a Schmitt trigger. The mean voltage of the output signal of the amplifier is approximately in the middle of the two threshold values of the Schmitt trigger. When the input voltage is below the lower threshold value, the output signal of the Schmitt trigger assumes the value “low” (= 0); when the input voltage exceeds the higher threshold value, the output value is “high” (= 1); and when the input voltage is between the two levels, the output retains its value. For the generation of random numbers, the proposed design exploits the 0-1-crossings (“up-crossings”). Each up-crossing switches the output value of the Schmitt trigger from 0 to 1 and clocks an intermediate flip-flop that inverts the D-input of a second flip-flop. The second clock is latched by a regular clock signal at equidistant times $s_0 := 0, s_1 := s, s_2 := 2s, \dots$

1136

The **physical noise source** exploits the random time intervals between subsequent 0-1-crossings. Due to the steep edges of the output of the operational amplifier and since only the 0-1-crossings are exploited, the hysteresis effect is assumed to be negligible.

[random numbers] The number of 0-1-crossings within the n^{th} clock cycle, i.e., within the time interval $I_n := (s_{n-1}, s_n] = ((n-1)s, ns]$, gives the **raw random number** $r_n \in \mathbb{N}_0$. Finally, the **internal random numbers** are given by $y_{n+1} = y_n \oplus r_{n+1} (= y_n + r_{n+1} \pmod{2})$. We denote the sequence $r_1, r_2, \dots$ as “virtual” in this context because these integers never occur. Although the **internal random numbers** (5.106) depend only on $r_n \pmod{2}$, the least significant bit of r_n , the **stochastic model** and the **online tests** should consider the (virtual) **raw random numbers** $r_1, r_2, \dots$, as they contain more information than their least significant bits. Note: y'_0 denotes the output value of the flip-flop when the “observation” (at time $t = 0$) starts.

Principally, the design in Fig. 22 would also work with a single noisy diode instead of two. A single diode is potentially more vulnerable to environmental conditions, and in particular, to an **adversary** who aims to manipulate the output voltage of the diode(s) by active attacks, e.g., by an external electromagnetic field.

Note: Designs based on single diodes are not generally unsuitable, but additional measures should be considered to mitigate these threats.

[experiments] We provide experimental results from a **PTRNG** prototype for which the design to the left of the first flip-flop equals the schematic design from Fig. 22; cf. [KiSc08], Section 5). We mention that the operations that follow the up-crossings are principally deterministic as soon as the (time) position of one 0-1-crossing relative to the regular clock signal has been fixed; cf. par. 1180.

Note: The **PTRNG** prototype was kindly provided by Frank Bergmann.

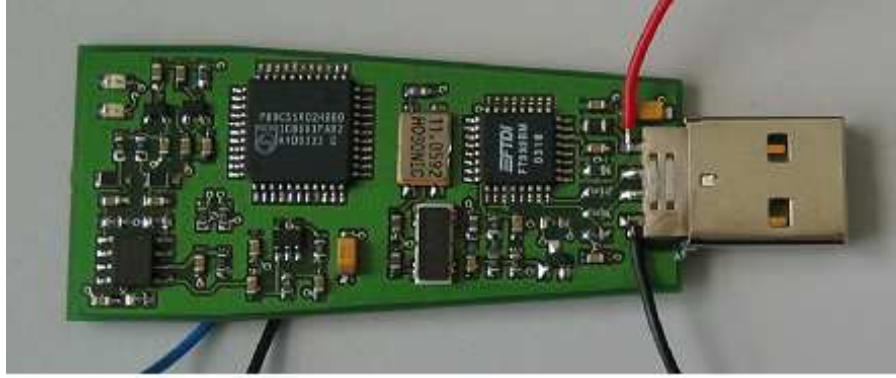


Figure 23: Hardware setup of the **PTRNG** [KiSc08], Fig. 2

Obviously,

$$y_n \equiv y_{n-1} + r_n \equiv y_0 + r_1 + \dots + r_n \pmod{2} \quad \text{for } n \geq 1 \quad (5.106)$$

, where y_0 denotes the **internal random number** at time $t = 0$. This simple **algorithmic post-processing** allows the transfer of results on the **raw random numbers** $(\text{mod } 2)$ to the **internal random numbers**.

Below, we summarize analyses and facts that are relevant for the evaluation and for the **stochastic model**. As usual, we interpret the (virtual) **raw random numbers** $r_1, r_2, \dots$ and the **internal random numbers** $y_0, y_1, \dots$, as realizations of **random variables** $R_1, R_2, \dots$ and $Y_0, Y_1, \dots$, respectively. Our goal is to (at least) determine lower bounds for the following average conditional entropies

$$H(R_{n+1} \mid R_1, \dots, R_n) \quad \text{and finally} \quad (5.107)$$

$$H(R_{n+1}(\bmod 2) \mid R_1(\bmod 2), \dots, R_n(\bmod 2)) = H(Y_{n+1} \mid Y_0, Y_1, \dots, Y_n) \quad (5.108)$$

The right-hand conditional **entropy** in (5.108) corresponds to the real-world scenario where an **adversary** knows several **internal random numbers** $y_0, y_1, y_2, \dots, y_n$.

1142 We interpret the lengths $t_1, t_2, \dots$ of the time intervals between consecutive 0-1-crossings as realizations of a stochastic process $T_1, T_2, \dots$. One may assume that the analog part of the **physical noise source** is in an equilibrium state when enough time has passed since the start of the **PTRNG** (a fraction of a second should suffice). The stochastic behavior of the **PTRNG** is determined by several operational constants (such as breakdown voltages of the noisy diodes, electronic characteristics of the amplifier, or threshold levels of the Schmitt trigger). Consequently, shortly after start-up, the stochastic process, $T_1, T_2, \dots$ should be **stationary**, or more precisely, time-locally **stationary**. Long-term drifts (caused by the feedback loop of the amplifier, by changing environmental conditions, or by aging effects) are ignored in the following, cf. pars. 788 to 791. If needed, earlier time intervals between consecutive 0-1-crossings (before equilibrium) may be denoted with negative indices $(\dots, t_{-2}, t_{-1}, t_0)$.

1143 It should be noted that in a modification of this design where both the 0-1-crossings and the 1-0-crossings are counted, the sequence of random intervals between two consecutive crossings would presumably lose the **time-local stationarity** property. The reason for that is that the random intervals between consecutive 1-0-crossings and 0-1-crossings are in general not identically distributed. And even if the **time-local stationarity** would still hold, its justification and verification would become significantly more difficult. The selected design increases the robustness and simplifies its security analysis at the cost of halving the output rate. Recall that **physical noise sources** that generate non-**stationary raw random numbers** are not compliant with functionality classes PTG.2 and PTG.3.

1144 Due to the nature of shot noise, one may assume that the stochastic process $T_1, T_2, \dots$ is q -dependent (cf. par. 574) with small q . This assumption was supported by experiments, see [KiSc08], Section 5. For the lags $\tau = 1, \dots, 5$, the autocorrelation function (ACF)

$$\rho_T(\tau) = \frac{E((T_j - E(T_j))(T_{j+\tau} - E(T_{j+\tau})))}{\sqrt{\text{Var}(T_j)}\sqrt{\text{Var}(T_{j+\tau})}} \quad (5.109)$$

of the stochastic process $T_1, T_2, \dots$ was estimated. In all cases, the absolute value was < 0.002 , which suggests that both q and the magnitude of dependencies are small. In fact, this observation is consistent with the hypothesis that the **random variables** $T_1, T_2, \dots$, are essentially **iid**. But the conclusion that the variables are indeed **iid** cannot be drawn from this observation alone. Instead, this would require further analysis. As in [KiSc08], we cautiously assume that $q \leq 1$ but especially point to results that apply if the T_j are **iid**.

1145 The q -dependence of $T_1, T_2, \dots$ ensures that a version of the CLT (Central Limit Theorem) applies to the **random variables** $T_1, T_2, \dots$; cf. par. 1157. We introduce the notation $\mu := E(T_1)$ and

$\sigma_T^2 := \text{Var}(T_1)$. Of course, $\sigma_T^2 > 0$ since otherwise, the 0-1-crossings would appear periodically, and the random numbers would not have any **entropy**. Further (natural and non-restrictive) assumptions are that $E(|T_j|^3) < \infty$ (a necessary condition for the applied version of the CLT) and $\text{Prob}(T_1 = 0) = 0$, which is ensured by the technical properties of a Schmitt trigger.

Par. 1147 considers the one-dimensional distribution of the T_j , while pars. 1148 and 1149 address the output of the operational amplifier. 1146

Fig. 24 plots the distribution of the time intervals between successive 0-1-crossings, and Fig. 25 1147 illustrates the percentiles of the distribution. (Both diagrams belong to different measurements.) These experiments verify that the **random variables** $T_1, T_2, \dots$ are approximately Gamma-distributed (cf. par. 527). In [KiSc08], Section 5, the shape parameter α and the rate parameter β were estimated to $\tilde{\alpha} = 3.0949$ and $\tilde{\beta} = 0.0240$. The mean length between successive 0-1-crossings, $E(T_j)$, was ≈ 128.85 ns, and the standard deviation ≈ 72.9 ns.

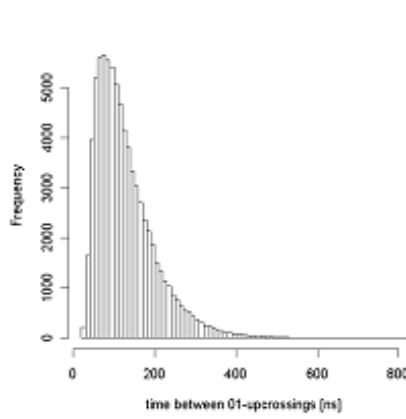


Figure 24: Empirical distribution of the time intervals between successive 0-1-crossings (in ns) [KiSc08], Fig. 3

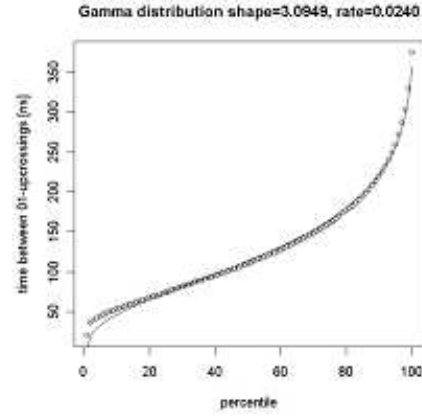


Figure 25: Percentiles of the Gamma distribution (curve) vs. the observed percentiles (circles) of the time intervals between successive 0-1-crossings (in ns) [KiSc08], Fig. 3

Fig. 26 and Fig. 27 plot the power spectrum and the autocorrelation function (ACF) of the output signal of the amplifier. 1148

Fig. 28 and Fig. 29 show typical output curves of the operational amplifier within time intervals of 1ns (resolution: 8 bits). 1149

The term w_0 denotes the time of the first 0-1-crossing after $t = 0$ (when the observation of the **raw random numbers** begins). The term z_n denotes the index of the first 0-1-crossing that follows after time $s_n = ns$ when the clock has latched the n^{th} time. Furthermore, $w_n := w_0 + t_1 + \dots + t_{z_n} - s_n$ equals the time interval from latching time s_n of the second flip-flop to the next 0-1-crossing. In particular,

$$w_0 + t_1 + \dots + t_{z_n-1} \leq s_n < w_0 + t_1 + \dots + t_{z_n}. \quad (5.110)$$

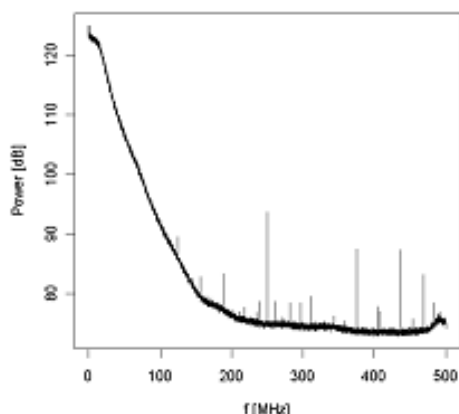


Figure 26: Mean power spectrum of the output of the amplifier (low amplification), created by W. Killmann

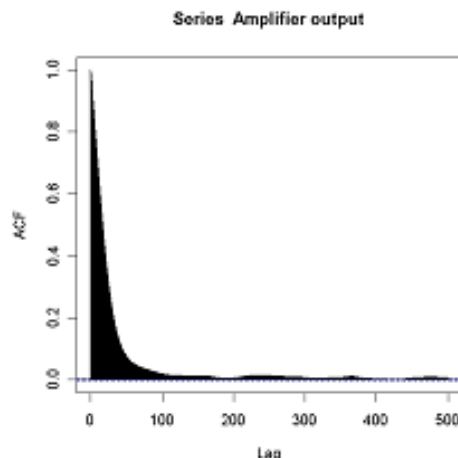


Figure 27: Autocorrelation function (ACF) of the amplified difference of noise voltages (maximum amplification, time in ns), created by W. Killmann

1151 The equations (5.111) to (5.114) show relations between several **random variables**

$$T_1, T_2, \dots \text{ are } \text{stationary} \text{ and } q\text{-dependent}, \quad (5.111)$$

$$Z_n := \min\{m \in \mathbb{N}_0 \mid W_0 + T_1 + T_2 + \dots + T_m > s_n\}, \quad (5.112)$$

$$R_n := Z_n - Z_{n-1}, \quad (5.113)$$

$$W_n := W_0 + T_1 + \dots + T_{Z_n} - s_n. \quad (5.114)$$

1152 The relations (5.111) to (5.114) fit for other **PTRNG** designs, too. The most obvious example is if the two noisy diodes are replaced by a single noisy diode. The distribution of the **random variables** $T_1, T_2, \dots$ and thus the distribution of $R_1, R_2, \dots$ and $Y_1, Y_2, \dots$, may vary significantly for different **PTRNG** designs. Thus, it is worthwhile to study the system of **random variables** that is defined by (5.111) to (5.114) under general (weak) assumptions as well as for the specific distribution of the T_j , e.g., for **iid** or Markovian **random variables** $T_1, T_2, \dots$. For our design, $q \leq 1$.

1153 A special feature of this **PTRNG** design is that under mild assumptions, the sequence $T_1, T_2, \dots$ “inherits” the stationarity property to other **random variables**; for details, see [KiSc08], Lemma 1 and Assumption 1. In particular, the **random variables**

$$(T_j)_{j \in \mathbb{N}}, (R_j)_{j \in \mathbb{N}}, (W_j)_{j \in \mathbb{N}_0}, (R_j(\bmod 2))_{j \in \mathbb{N}}, \text{ and } (Y_j)_{j \in \mathbb{N}} \\ \text{are (time-locally) } \text{stationarily distributed}. \quad (5.115)$$

The stationarity property suggests the analysis of the autocovariance and autocorrelation function (ACF) of these stochastic processes.

1154

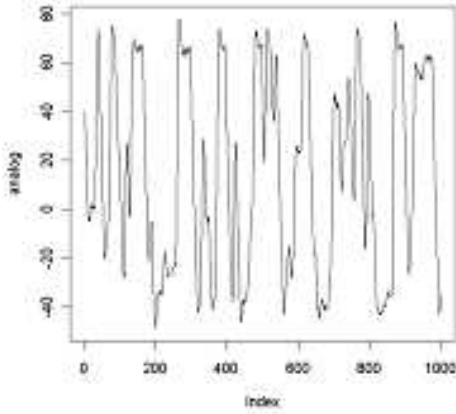


Figure 28: Output signal of the operational amplifier (low amplification), time-scale in ns, created by W. Killmann

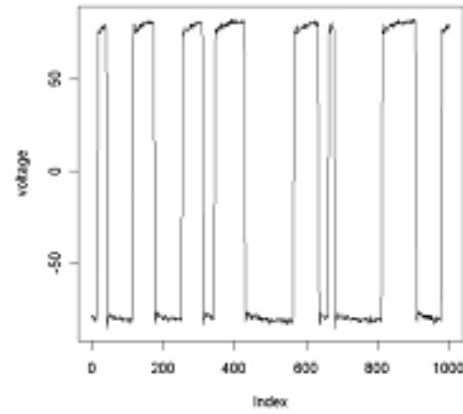


Figure 29: Output signal of the operational amplifier (maximum amplification), time-scale in ns, created by W. Killmann

[definition] The terms

$$G_T(u) := \text{Prob}(T_j \leq u) \quad \text{and} \quad G_W(u) := \text{Prob}(W_j \leq u) \quad (5.116)$$

denote the cumulative distribution functions of the **random variables** T_j and W_j . Furthermore, for $u \in (0, \infty)$ the **random variable**

$$V_{(u)} := \inf \left\{ \tau \in \mathbb{N} \mid \sum_{j=1}^{\tau+1} T_j > u \right\} = \sup \left\{ \tau \in \mathbb{N} \mid \sum_{j=1}^{\tau} T_j \leq u \right\} \quad (5.117)$$

quantifies the number of random 0-1-crossings in the interval $(0, u]$ if $W_0 \equiv 0$. The paragraphs below summarize important results from [KiSc08], Lemma 2 and Theorem 1.

For $k \geq 1$, we have

$$\text{Prob}(V_{(u)} = k) = \text{Prob}(T_1 + \dots + T_k \leq u) - \text{Prob}(T_1 + \dots + T_{k+1} \leq u) . \quad (5.118)$$

The cycle length s should be “large” compared to the mean length between two 0-1-crossings where the quantitative meaning of “large” also depends on the generalized variance of the T_j . Otherwise, the **entropy** of the **random variables** R_j might be very small, in particular, if the cycle length s is close to a small integer multiple of μ . Even if the parameter s would be selected in the middle of two integer multiples of μ , such a design would be rather sensitive to a variation of the parameter μ . The **internal random numbers** depend only on $R_j \pmod{2}$, the least significant bit of R_n . However, it is advisable to select the cycle length s so large that the distribution of R_j has several probable outcomes around the ratio s/μ . This provides distributions which are more robust against deviations of the parameters.

Since $T_1, T_2, \dots$ are assumed to be **stationary** and q -dependent (with $q \leq 1$, cf. par. 1144) and since $s \gg \mu$, the CLT may be applied to the right-hand probabilities of (5.118) so that

$$\text{Prob} \left(\frac{T_1 + \dots + T_k - k\mu}{\sqrt{k}\sigma} \leq x \right) \rightarrow_{k \rightarrow \infty} \Phi(x) \quad \text{for } x \in \mathbb{R}. \quad (5.119)$$

The variance σ^2 is computed by (4.52) with $q = 1$, while Φ denotes the cumulative distribution function of the standard normal distribution (cf. par. 568, (4.43)). The mathematical background is sketched in pars. 575 and 576. We follow [KiSc08].

1158 Let $u = v\mu$ with $v \gg 1$. By (5.118) and (5.119) we obtain

$$\text{Prob}(V_{(v\mu)} = k) \approx \Phi \left(\frac{v-k}{\sqrt{k}} \cdot \frac{\mu}{\sigma} \right) - \Phi \left(\frac{v-(k+1)}{\sqrt{k+1}} \cdot \frac{\mu}{\sigma} \right) \quad \text{for } k \geq 1 \quad (5.120)$$

$$\text{Prob}(V_{(v\mu)} = 0) \approx 1 - \Phi \left((v-1) \frac{\mu}{\sigma} \right). \quad (5.121)$$

Interestingly, the distribution of the **random variable** $V_{(v\mu)}$ (or more precisely, its normal approximation) depends only on the ratios μ/σ and $u/\mu = v$ but not on the absolute values of the parameters $\mu, \sigma^2, u = v\mu$. The mass of $V_{(v\mu)}$ is essentially concentrated on the integers k around $k \approx v$. Since $s \gg \mu$, the approximation error should be negligible.

1159 [**iid case**] If the **random variables** $T_1, T_2, \dots$ are **iid**, it is well-known (cf. par. 593) that then

$$G_W(x) := \text{Prob}(W_n \leq x) = \frac{1}{\mu} \int_0^x (1 - G_T(u)) \, du. \quad (5.122)$$

If $G_T(\cdot)$ is continuous (or equivalently, if $\text{Prob}(T_1 = y) = 0$ for all $y \in [0, \infty)$), then $G_W(\cdot)$ has density $g_W(x) := (1 - G_T(x))/\mu$.

1160 Equations (5.123) and (5.124) provide an expression for the k^{th} moment for the sum $R_1 + R_2 + \dots + R_j$.

$$E((R_1 + \dots + R_j)^k) = \int_0^{js} E((V_{(js-u)} + 1)^k \mid W_0 = u) G_W(du) \quad (5.123)$$

$$\approx \int_0^{js} E((V_{(js-u)} + 1)^k) G_W(du) \quad \text{for each } k \in \mathbb{N} \quad (5.124)$$

with equality for **iid random variables** T_j . The term “+1” in (5.123) and (5.124) is needed because the **random variables** $V_{(u)}$ do not consider the origin ($t = 0$). For $j \geq 1$, the stationarity of the R_j implies

$$E((R_1 + \dots + R_j)^2) = jE(R_1^2) + 2 \sum_{i=2}^j (j+1-i)E(R_1 R_i). \quad (5.125)$$

Beginning with $E(R_1^2)$, then in (5.123) adding successively the **random variables** $R_2, R_3, \dots$, one obtains $E(R_1 R_2), E(R_1 R_3), \dots$ in terms of expressions which are already known.

1161 Under mild regularity assumptions on the **random variables** $T_1, T_2, \dots$, heuristic arguments provide the inequality

$$\begin{aligned} H(Y_{n+1} \mid Y_1, \dots, Y_n) &= H(R_{n+1}(\bmod 2) \mid R_1(\bmod 2), \dots, R_n(\bmod 2)) \\ &\geq \min\{H(V_{(s-u)}(\bmod 2)) \mid u \in [0, \mu + a\sigma]\} G_W(\mu + a\sigma). \end{aligned} \quad (5.126)$$

where $a > 0$ should be selected such that $G_W(\mu + a\sigma) \approx 1$. The idea of the min-operation is to consider the worst-case (depending on W_n). Apart from the impact on the time remaining until the next latch by the regular clock, potential dependencies between W_n and R_{n+1} are not considered. Due to the preceding considerations, such dependencies, if existent at all, should be rather small. If the **random variables** $T_1, T_2, \dots$ are (almost) **iid**, easier formulae are derived below.

For the **raw random numbers**, we obtain

1162

$$\text{Prob}(R_{n+1} = k) \approx \int_0^s \text{Prob}(V_{(s-u)} = k-1) G_W(du) \text{ for } k \in \mathbb{N}_0 \quad (5.127)$$

$$H(R_{n+1}(\text{mod } 2)) \geq H(R_{n+1}(\text{mod } 2) \mid W_n) \approx \int_0^s H(V_{(s-u)}(\text{mod } 2)) G_W(du) \quad (5.128)$$

If the **random variables** $T_1, T_2, \dots$ are **iid** in (5.127) and (5.128), “ $\approx$ ” signs can be replaced by “ $=$ ”.

[Case: $T_1, T_2, \dots$ are **iid**] If the **random variables** $T_1, T_2, \dots$ are **iid**, then $(W_{n-1}, R_n)_{n \in \mathbb{N}}$ defines a Markov chain on the state space $\mathbb{R}_+ \times \mathbb{N}_0$. In this case, $W_0, T_1, T_2, \dots$ induces a **stationary renewal process** $Z'(t) := \inf\{k \mid W_0 + T_1 + \dots + T_k > t\}$, where t ranges in $[0, \infty)$; cf. pars. 593 to 595. In particular, $Z_n = Z'(sn)$. Even for **iid random variables** $T_1, T_2, \dots$, the **random variables** $R_1, R_2, \dots$ are usually not **iid** because large R_n (i.e., many 0-1-crossings in the n^{th} interval) makes it plausible that the last 0-1-crossing has occurred shortly before the end of this interval and thus that W_n is likely to be “large”. Hence, R_n and R_{n+1} should be weakly negatively correlated. For deeper mathematical analysis, see Subsection 5.4.3, pars. 1220 ff. However, if $s \gg \mu$ (as recommended) and if σ/μ is not “small”, the dependency between R_n and R_{n+1} should be small. Table 1 in [KSc08] underlines that this is the case for the evaluated design.

If the sequence $T_1, T_2, \dots$ is **iid** and the **random variables** $R_1, R_2, \dots$ are “almost” independent (since $\mu \ll s$), then

$$H(Y_{n+1} \mid Y_0, \dots, Y_n) \approx \int_0^s H(V_{(s-u)}(\text{mod } 2)) G_W(du) \text{ for all } n \in \mathbb{N}. \quad (5.129)$$

If $G_T(\cdot)$ is continuous (may be assumed here; cf. par. 1144), then (5.129) reads

$$H(Y_{n+1} \mid Y_0, \dots, Y_n) \approx \int_0^s H(V_{(s-u)}(\text{mod } 2)) \frac{1}{\mu} (1 - G_T(u)) du. \quad (5.130)$$

[**iid** case] For **iid** T_j , (5.122) provides an explicit formula for the cumulative distribution function $G_W(\cdot)$, and if G_T is continuous, also for the density of W_n . If $T_j \sim \gamma_{\alpha, \beta} \cdot \lambda$ (par. 1147), then

Fubini's Theorem, $\mu = \alpha/\beta$ (par. 527), and the properties of the Gamma function imply that

$$\begin{aligned}
 E(W_n^m) &= \int_0^\infty u^m \frac{1}{\mu} \int_u^\infty \gamma_{\alpha,\beta}(v) dv du = \int_0^\infty \int_0^v u^m \frac{1}{\mu} \frac{\beta^\alpha}{\Gamma(\alpha)} v^{\alpha-1} e^{-\beta v} du dv = \\
 &= \int_0^\infty \frac{1}{m+1} v^{m+1} \frac{1}{\mu} \frac{\beta^\alpha}{\Gamma(\alpha)} v^{\alpha-1} e^{-\beta v} dv = \int_0^\infty \frac{1}{m+1} \frac{1}{\mu} \frac{\beta^{\alpha+m+1} \Gamma(\alpha+m+1)}{\beta^{m+1} \Gamma(\alpha) \Gamma(\alpha+m+1)} v^{\alpha+m+1-1} e^{-\beta v} dv = \\
 &= \frac{1}{m+1} \frac{1}{\mu} \frac{\Gamma(\alpha+m+1)}{\beta^{m+1} \Gamma(\alpha)} \int_0^\infty \gamma_{\alpha+m+1,\beta}(v) dv = \frac{1}{m+1} \frac{\beta}{\alpha} \frac{(\alpha+m) \cdots \alpha}{\beta^{m+1}} \cdot 1 = \\
 &= \frac{1}{(m+1)\beta^m} (\alpha+m) \cdots (\alpha+1) \quad \text{for } m \geq 1.
 \end{aligned} \tag{5.131}$$

As an immediate consequence, we obtain

$$E(W_n) = \frac{\alpha+1}{2\beta} \quad \text{and} \quad \text{Var}(W_n) = \frac{(\alpha+2)(\alpha+1)}{3\beta^2} - \frac{(\alpha+1)^2}{4\beta^2} = \frac{\alpha^2 + 6\alpha + 5}{12\beta^2}. \tag{5.132}$$

- 1166 Subsection 5.4.3 provides a thorough analysis of a generic stochastic model for **iid random variables** $T_1, T_2, \dots$, allowing the computation of the joint distribution of $(R_1, \dots, R_m)$ for $m \geq 1$. Therefrom, both the **Shannon entropy** and the **min-entropy** of the **random variables** $(R_1 \pmod{2}, \dots, R_m \pmod{2})$ can be computed. Under suitable circumstances, these results can also be applied if the T_j are only weakly dependent; cf. Subsection 5.4.3.
- 1167 [**stochastic model**] Essential for the evaluation is the stochastic behavior of the (random) intervals between consecutive 0-1-crossings. We described these random intervals by **random variables** $T_1, T_2, \dots$. Our first task was to characterize these **random variables**. By technical arguments that consider the properties of the noisy diodes and of the operational amplifier, and supported by experiments, evidence was given in [KiSc08] that the **random variables** $T_1, T_2, \dots$ can be assumed to be **stationary** and q -dependent. It was concluded that q can be selected ≤ 1 with only small dependencies between T_j and T_{j+1} . The one-dimensional distribution of the **random variables** T_j can be approximated by a Gamma distribution; cf. pars. 1139, 1142, 1144, 1147, 1148, 1149. Further analysis depends on these findings.
- 1168 [**stochastic model**] A Gamma distribution depends on two parameters, the shape parameter α and the rate parameter β . Possibly, it can be shown that the **random variables** $T_1, T_2, \dots$ are “almost” **iid** (i.e., 0-dependent). This conclusion (as far as applicable) would require deeper analysis of the design but would simplify the further analysis to some degree.
- 1169 [**stochastic model**] Ideally, the **stochastic model** comprises a family of admissible distributions of the **raw random number** variables $R_1, R_2, \dots$, called “virtual **raw random numbers**” in [KiSc08] because they do not “really” appear. The specification of this family of distributions is not easy, not even if the T_j are **iid** (par. 1163). It should be noted that par. 1162, formula (5.127), specifies the one-dimensional distribution of R_n . If $s \gg \mu$ (as recommended), the dependencies between neighboring R_n should be weak, and it is rather likely that the $(\pmod{2})$ operation reduces dependencies additionally. In particular, the (virtual) **raw random numbers**, respectively, the corresponding random variables $R_1, R_2, \dots$ and $R_1 \pmod{2}, R_2 \pmod{2}, \dots$, are **stationary** (5.115).
- 1170 [**stochastic model**] On the other hand, the relevant conclusions from above are closely connected to the **random variables** $V_{(u)}$; cf. pars. 1160, 1161, 1162, 1164. In particular, this allows the

determination of lower **entropy** bounds for the **internal random numbers** (cf. (5.126), (5.129), (5.130)). Hence, we consider the “auxiliary” **random variables** $V_{(u)}$; cf. par. 771.

[**entropy**] Tab. 1 in [KiSc08] summarizes the results for several parameter sets. Recall that $\tilde{\mu} = 128.85$ ns (estimate of $E(T_j)$; cf. par. 1147). For the most conservative design parameter $s = 15.017\tilde{\mu}$, the conditional **Shannon entropy** per raw random bit, $R_j(\bmod 2)$, $(R_{n+1}(\bmod 2) \mid R_{n+1}(\bmod 2), \dots, R_{n+1}(\bmod 2))$, is assumed to be $> 1 - 10^{-4}$; cf. [KiSc08], Section 5. This gives an output rate of raw random bits (= output rate of **internal random numbers**) of a little more than 500 kBit/sec. 1171

[**stochastic model**] The distribution of the **random variable** $V_{(u)}$ depends on the parameters $u = v\mu$ and μ/σ , or equivalently, on $v = u/\mu$ and μ/σ , cf. (5.120) and (5.121). The distribution of the **raw random numbers** $R_1, R_2, \dots$ as well as of $R_1(\bmod 2), R_2(\bmod 2), \dots$, and their average conditional **entropy** depends on the ratios s/μ and μ/σ . 1172

[**stochastic model**] This means that the **stochastic model** of the **raw random numbers** is a 2-parameter model with parameters $(s/\mu, \mu/\sigma) \in (0, \infty)^2$. A central task of the evaluation is to specify subsets $A_{real}, A_{good}, A_{bad} = (0, \infty)^2 \setminus A_{good} \subseteq (0, \infty)^2$; cf. Subsection 4.5.3. In particular, the parameters in A_{real} and A_{good} provide enough conditional **entropy**; cf. (5.126), (5.129), (5.130). The **online test** shall detect if the true parameters leave the subset A_{real} when the **PTRNG** is in operation. 1173

Note: If s is fixed, the 2-parameter-family of admissible distributions can alternatively be parametrized by $(\mu, \mu/\sigma)$, (μ, σ^2) , or (μ, σ) .

[**online test**] The distribution of the **raw random numbers** (and thus the lower **entropy** bound for the **internal random numbers**) depends on the ratios s/μ and μ/σ . The **online test** shall detect if these values leave the set of appropriate parameters A_{good} . General considerations are found in [KiSc08], Section 6. We do not deepen this aspect here, but refer to Subsection 5.4.3. 1174

5.4.3 Sampling Events with iid Intermediate Time Intervals – Design A

This subsection does not develop and analyze a **stochastic model** for a concrete design of a **physical noise source**. Instead, a thorough analysis of a generic **stochastic model** is provided that potentially fits several different designs; cf. pars. 1184 and 1204. Unlike (5.129) and (5.130), this subsection also covers scenarios for which the **random variables** R_j are far from being independent. Note: *The developer can refer to this subsection but, of course, has to give evidence that the **stochastic model** indeed fits the design under evaluation.* 1175

Subsection 5.4.3 considers both **Shannon entropy** and **min-entropy**. The focus is on **min-entropy**. 1176

In the following, we assume that a **physical noise source** counts (design-specific) events. The time intervals between two successive events are denoted by $t_1, t_2, \dots$. The integers r_j denote the number of events within the interval $I_j := ((j-1)s, js]$, where s denotes the (fixed) length of the sampling intervals, e.g., the cycle length of a stable clock. As in Subsection 5.4.2, $s_n = sn$. Furthermore, 1177

$$y'_j = r_j(\bmod 2). \quad (5.133)$$

Note: The definition of the binary random number y'_j differs from that of y_j in Subsection 5.4.2, but the results on the **entropy** of the **random variables** Y'_j can easily be transferred to the **random variables** $Y_j = Y_0 + Y'_1 + \dots + Y'_j \pmod{2}$.

- 1178 The first goal is to determine the joint distribution of the **random variables** $R_1, \dots, R_m$. Therefrom, the joint distribution of the **random variables** $Y'_1, \dots, Y'_m$ can be deduced. This allows determining the joint **entropy** and conditional **entropy** for both **Shannon entropy** and **min-entropy**. Later, we develop an effective **online test**.
- 1179 Example: In Subsection 5.4.2, the design-specific events are 0-1-crossings of a Schmitt trigger. Likewise, such events could be radioactive decays as in Subsection 5.4.5.
- 1180 Finally, we are interested in the **entropy** of the **intermediate random numbers** $y'_1, y'_2, \dots$. However, with regard to the **online test**, we recommend to consider the **raw random numbers** $r_1, r_2, \dots$ because r_j contains much more information than y'_j . After the **raw random number** r_j has been read, the counter is reset to 0.
- 1181 As already mentioned above, the applicant/developer has to show that this generic **stochastic model** fits the concrete **physical noise source**. In this subsection, the **stochastic model** assumes that sampling is ideal in the sense that $r_1, r_2, \dots$ equal the exact numbers of events that have occurred in the intervals $I_1, I_2, \dots$. In real-world designs, occasional detection errors can occur, e.g., because an event occurs when a counting flip-flop is in the state of metastability because the setup-time or hold-time restrictions are undercut. If such detection errors occur rarely and if the **entropy** proof is not based on metastability (e.g., of the flip-flop), they can be neglected in the **stochastic model**. Furthermore, one may assume that occasional detection errors (caused by metastability) even slightly increase the **entropy** of the random numbers.
- 1182 The most critical task within the evaluation of the **physical noise source** is to verify that the **random variables** $T_1, T_2, \dots$ fulfill (at least approximately) the **iid** assumption. The physical effects contributing to the variance of T_j should be investigated. A favorable scenario is, for example, if the variance is essentially caused by thermal noise or shot noise. If (low-frequency) flicker noise has relevant impact, the situation becomes more complicated. Here, the Allan variance (cf. par. 560) could be used to estimate the size of the “useful jitter”. Perhaps the results from this subsection cannot directly be applied there, but possibly the basic ideas and strategies can be used and adjusted. “Worst-case analysis”, assuming the least favorable assumptions, may be necessary in place of the integrals developed below, presumably losing some information. For suitable designs, proving a lower **entropy** bound should be possible.
- 1183 [**iid** assumption] We assume that the time lengths $t_1, t_2, \dots$ can be viewed as realizations of **iid random variables** $T_1, T_2, \dots$. As already noted in par. 1163, the **random variables** $Z'(t) := \inf\{k \mid W_0 + T_1 + \dots + T_k > t\}$ form a **stationary** renewal process, where t ranges in $[0, \infty)$; cf. pars. 1220 ff.
- 1184 The **iid** assumption from par. 1183 applies (at least approximately) to several **noise source** designs. Example (cf. par. 1179): Subsection 5.4.2 (under the **iid** assumption; cf. par. 1163), radioactive decays (cf. Subsection 5.4.5, par. 1247, although a different sampling mechanism is used there).

1185

For each $x \in \mathbb{R}$, there exist unique $k \in \mathbb{Z}$ and $b \in [0, s)$ such that $x = ks + b$. We write $x(\bmod s) = b$. Assume that the real-valued **random variables** Y_1 and Y_2 are independent. If Y_1 is **uniformly distributed** on $[0, s)$, then $Y_1 + Y_2(\bmod s)$ is **uniformly distributed** on $[0, s)$, too, regardless of the distribution of Y_2 .

Let $S_j = T_1 + \dots + T_j(\bmod s)$. Under weak assumptions on the distribution of the T_j , the **random variables** $S_1, S_2, \dots$ converge exponentially fast to the uniform distribution on $[0, s)$. 1186

Note 1: It suffices that the distribution of T_j has a density that is > 0 on some interval $I \subseteq [0, s)$. (In fact, the interval $[0, s)$, equipped with the addition modulo s , is a compact group.

Note 2: We may assume that the **PTRNG** has started some time before (at time $-Js$ for some $J > 0$), and that $S_0, S_1, \dots$ are **uniformly distributed** on $[0, s)$ (equilibrium state).

We use the same notation as in Subsection 5.4.2. To simplify reading, we repeat the definitions. 1187
The **random variable** W_0 describes the (random) time when the first event is detected after time $t = 0$ (when the observation of the **raw random number** starts). Furthermore,

$$T_1, T_2, \dots \text{ are iid,} \quad (5.134)$$

$$Z_n := \min\{m \in \mathbb{N}_0 \mid W_0 + T_1 + T_2 + \dots + T_m > s_n\}, \quad (5.135)$$

$$R_n := Z_n - Z_{n-1}, \quad (5.136)$$

$$W_n := W_0 + T_1 + \dots + T_{Z_n} - s_n. \quad (5.137)$$

Unlike in Subsection 5.4.2, par. 1151, the **random variables** $T_1, T_2, \dots$ are not only **stationarily distributed** and q -dependent but even assumed to be **iid**.

Of course, as in Subsection 5.4.2, par. 1153 (which covers a more general case), the **random variables** 1188

$$(T_j)_{j \in \mathbb{N}}, (R_j)_{j \in \mathbb{N}}, (W_j)_{j \in \mathbb{N}_0}, \text{ and } (Y'_j = R_j(\bmod 2))_{j \in \mathbb{N}} \text{ are } \textbf{stationarily distributed}. \quad (5.138)$$

Note: The renewal process $Z'(t)$ is **stationary** (cf. par. 1183), and thus the sequence $R_1, R_2, \dots$ is **stationary** (cf. par. 593, it is $R_n = Z_n - Z_{n-1} = Z'(ns) - Z'((n-1)s)$). This, in particular, implies the stationarity of $Y'_1, Y'_2, \dots$, while the stationarity of $W_0, W_1, \dots$ follows from (4.70).

[Assumption] The distribution of the **random variables** $T_1, T_2, \dots$ has density (to be mathematically precise: Lebesgue density) $g(\cdot)$. If $G_T(\cdot)$ denotes the cumulative distribution function of T_j , then W_j has density $g_W(\cdot) = \frac{1}{\mu}(1 - G_T(\cdot))$ (par. 593). 1189

Note: In particular, $\text{Prob}(W_j > 0) = 1$ for all $j \in \mathbb{N}$.

Note: The case that the distribution of T_j has a density constitutes the most relevant case for applications. We mention that similar results can be derived if the **random variables** $T_1, T_2, \dots$ do not have a density $g(\cdot)$ (although with greater mathematical efforts). If the **random variables** T_j are discrete, the integrals below turn into sums.

For each integer $\ell \geq 1$, the term $g^{*(\ell)}(\cdot)$ denotes the ℓ -fold convolution of the density $g(\cdot)$. In particular, $g^{*(1)}(\cdot) = g(\cdot)$. 1190

Note: For each $t \in \mathbb{N}_0$ and $\ell \in \mathbb{N}$, the sum $T_{t+1} + \dots + T_{t+\ell}$ has density $g^{*(\ell)}(\cdot)$; cf. pars. 1202 and 1203.

For $m \in \mathbb{N}$ and $k_1, \dots, k_m \in \mathbb{N}_0$ it is

$$\begin{aligned} & \text{Prob}(R_1 \leq k_1, R_1 + R_2 \leq k_1 + k_2, \dots, R_1 + \dots + R_m \leq k_1 + \dots + k_m) = \\ & \text{Prob}(W_0 + T_1 + \dots + T_{k_1} > s, W_0 + T_1 + \dots + T_{k_1+k_2} > 2s, \dots, \\ & W_0 + T_1 + \dots + T_{k_1+\dots+k_m} > ms). \end{aligned} \quad (5.139)$$

1192 In the next paragraphs, we develop integral representations for the joint probabilities of $(R_1, \dots, R_m)$.

1193 $[k_1, \dots, k_m > 0]$ If $k_1, \dots, k_m > 0$, from (5.139) we obtain the integral representation

$$\begin{aligned} & \text{Prob}(R_1 \leq k_1, R_1 + R_2 \leq k_1 + k_2, \dots, R_1 + \dots + R_m \leq k_1 + \dots + k_m) = \\ & \int_{ms}^{\infty} \int_{(m-1)s}^{\infty} \dots \int_s^{\infty} \int_0^{\infty} g^{*(k_m)}(u_m - u_{m-1}) g^{*(k_{m-1})}(u_{m-1} - u_{m-2}) \dots \\ & g^{*(k_1)}(u_1 - u_0) g_W(u_0) du_0 du_1 \dots du_{m-1} du_m \end{aligned} \quad (5.140)$$

Note: The term du_0 belongs to “ $\int_0^{\infty}$ ”.

Note: Since $g_W(u), g^{*(\ell)}(u) = 0$ for $u < 0$ ($\ell \geq 1$), the integrand does not contribute to the integral unless $0 \leq u_0 \leq u_1 \leq \dots \leq u_m$. Likewise, the lower integration boundaries js could be replaced by $\max\{js, u_{j-1}\}$ (for $j = 1, \dots, m$).

1194 If $k_j = 0$ for one or several indices j , the integral representation (5.140) has to be adjusted. We begin with an example.

1195 [Example: $m = 4, k_3 = 0$] Let $(k_1, k_2, k_3, k_4) = (3, 7, 0, 2)$. By (5.139), we obtain

$$\begin{aligned} & \text{Prob}(R_1 \leq 3, R_1 + R_2 \leq 10, R_1 + R_2 + R_3 \leq 10, R_1 + R_2 + R_3 + R_4 \leq 12) = \\ & \text{Prob}(W_0 + T_1 + \dots + T_3 > s, W_0 + T_1 + \dots + T_{10} > 2s, \\ & W_0 + T_1 + \dots + T_{10} > 3s, W_0 + T_1 + \dots + T_{12} > 4s). \end{aligned} \quad (5.141)$$

Since the condition $(W_0 + T_1 + \dots + T_{10} > 3s)$ implies the weaker condition $(W_0 + T_1 + \dots + T_{10} > 2s)$, this saves one integral in the integral representation. In particular,

$$\begin{aligned} & \text{Prob}(R_1 \leq 3, R_1 + R_2 \leq 10, R_1 + R_2 + R_3 \leq 10, R_1 + R_2 + R_3 + R_4 \leq 12) = \\ & \text{Prob}(W_0 + T_1 + \dots + T_3 > s, W_0 + T_1 + \dots + T_{10} > 3s, W_0 + T_1 + \dots + T_{12} > 4s) = \\ & \int_{4s}^{\infty} \int_{3s}^{\infty} \int_s^{\infty} \int_0^{\infty} g^{*(2)}(u_4 - u_3) g^{*(7)}(u_3 - u_1) g^{*(3)}(u_1 - u_0) g_W(u_0) du_0 du_1 du_3 du_4 \end{aligned} \quad (5.142)$$

Note: Compared with the integral representation (5.140) for $k_1, k_2, k_3, k_4 > 0$, the integral “ $\int_{3s}^{\infty} \dots g^{*(k_2)}(u_3 - u_1) \dots du_3$ ” replaces “ $\int_{3s}^{\infty} \int_{2s}^{\infty} \dots g^{*(k_3)}(u_3 - u_2) g^{*(k_2)}(u_2 - u_1) \dots du_3 du_2$ ”.

1196 $[k_j = 0 \text{ for at least one index } j]$ In this and in the following paragraph, we generalize the insights from the preceding example. The goal is to derive an integral representation corresponding to (5.140). At first, isolated subsequences of 0's in $k_1, \dots, k_m$ are identified. Each of these subsequences is treated as explained in par. 1197.

Example: Let $(k_1, k_2, k_3, k_4, k_5, k_6, k_7) = (9, 0, 0, 3, 1, 5, 0)$. There are two subsequences of 0's, namely k_2, k_3 and k_7 .

1197

$[k_j = 0$ for at least one index $j]$ In this paragraph, we consider the impact of the isolated subsequences of 0's (cf. par. 1196) on the integral representation. We assume that $k_j, \dots, k_{j+t} = 0$. We distinguish four cases ($t \geq 0$).

Case (a) $1 < j, j+t < m$ and $k_{j-1}, k_{j+t+1} > 0$.

Then $W_0 + T_1 + \dots + T_{k_1 + \dots + k_{j-1}} > (j+t)s$.

Impact on the integral representation (5.140), compared to the case $k_1, \dots, k_m > 0$:

The integrals " $\int_{(j+t)s}^{\infty} \dots g^{*(k_{j-1})}(u_{j+t} - u_{j-2}) du_{j+t}$ " replace the integrals

$\int_{(j+t)s}^{\infty} \dots \int_{(j-1)s}^{\infty} \dots g^{*(k_{j+t})}(u_{j+t} - u_{j+t-1}) \dots g^{*(k_{j-1})}(u_{j-1} - u_{j-2}) \dots du_{j-1} \dots du_{j+t}$.

Case (b) $1 = j, 1+t < m$ and $k_{t+2} > 0$.

Then $W_0 > (t+1)s$, $W_0 + T_1 + \dots + T_{k_{t+2}} > (t+2)s$.

Impact on the integral representation (5.140), compared to the case $k_1, \dots, k_m > 0$:

The integrals " $\int_{(t+2)s}^{\infty} \int_{(t+1)s}^{\infty} \dots g^{*(k_{t+2})}(u_{t+2} - u_0) g_W(u_0) du_{t+2} du_0$ " replace the integrals

$\int_{(t+2)s}^{\infty} \dots \int_0^{\infty} \dots g^{*(k_{t+2})}(u_{t+2} - u_{t+1}) \dots g_W(u_0) du_0 \dots du_{t+2}$.

Case (c) $1 < j, j+t = m$ and $k_{j-1} > 0$

Then $W_0 + T_1 + \dots + T_{k_{j-1}} > ms$.

Impact on the integral representation (5.140), compared to the case $k_1, \dots, k_m > 0$:

The integrals " $\int_{ms}^{\infty} \dots g^{*(k_{j-1})}(u_m - u_{j-2}) du_m$ " replace the integrals

$\int_{ms}^{\infty} \dots \int_{js}^{\infty} \dots g^{*(k_m)}(u_m - u_{m-1}) \dots g^{*(k_j)}(u_j - u_{j-1}) du_j \dots du_m$.

Case (d) $1 = j, j+t = m$ (i.e., $k_1 = \dots, k_m = 0$).

Then $W_0 > ms$.

Impact on the integral representation (5.140), compared to the case $k_1, \dots, k_m > 0$:

The integral " $\int_{jm}^{\infty} g_W(u_0) du_0$ " replaces (5.140).

So far, we have learned how to compute probabilities of the type $\text{Prob}(R_1 \leq k_1, \dots, R_1 + \dots + R_m \leq k_1 + \dots + k_m)$. However, finally we are interested in probabilities of the type $\text{Prob}(R_1 = k_1, \dots, R_m = k_m)$. 1198

Equation (5.143) provides the desired formula. 1199

$$\begin{aligned} \text{Prob}(R_1 = k_1, R_2 = k_2, \dots, R_m = k_m) = \\ \sum_{T \subseteq \{1, \dots, m\}} (-1)^{|T|} \text{Prob}(R_1 \leq \ell_1, R_1 + R_2 \leq \ell_2, \dots, R_1 + \dots + R_m \leq \ell_m \mid \\ \ell_j = k_1 + \dots + k_j - 1 \text{ if } j \in T, \ell_j = k_1 + \dots + k_j \text{ else}) \\ \text{for } k_1, \dots, k_m \geq 0 \end{aligned} \quad (5.143)$$

We prove (5.143) by induction on m . For $m = 1$, (5.143) equals $\text{Prob}(R_1 = k_1) = \text{Prob}(R_1 \leq k_1) - \text{Prob}(R_1 \leq k_1 - 1)$, which is correct. Assume that (5.143) is valid for all $1 \leq m' \leq m$. The in-

ductive step can be verified as follows

$$\begin{aligned}
 & \text{Prob}(R_1 = k_1, R_2 = k_2, \dots, R_m = k_m, R_{m+1} = k_{m+1}) = \\
 & \text{Prob}(R_1 = k_1, R_2 = k_2, \dots, R_m = k_m, R_1 + \dots + R_{m+1} = k_1 + \dots + k_{m+1}) = \\
 & \text{Prob}(R_1 = k_1, R_2 = k_2, \dots, R_m = k_m, R_1 + \dots + R_{m+1} \leq k_1 + \dots + k_{m+1}) - \\
 & \text{Prob}(R_1 = k_1, R_2 = k_2, \dots, R_m = k_m, R_1 + \dots + R_{m+1} \leq k_1 + \dots + k_{m+1} - 1) \quad (5.144)
 \end{aligned}$$

We apply the induction hypothesis separately to the first m components of both summands of the last equation in (5.144). This leads to

$$\begin{aligned}
 & \text{Prob}(R_1 = k_1, R_2 = k_2, \dots, R_m = k_m, R_{m+1} = k_{m+1}) = \\
 & \sum_{T \subseteq \{1, \dots, m\}} (-1)^{|T|} \text{Prob}(R_1 \leq \ell_1, R_1 + R_2 \leq \ell_2, \dots, R_1 + \dots + R_{m+1} \leq k_1 + \dots + k_{m+1} \mid \\
 & \quad \ell_j = k_1 + \dots + k_j - 1 \text{ if } j \in T, \ell_j = k_1 + \dots + k_j \text{ else}) - \\
 & \sum_{T \subseteq \{1, \dots, m\}} (-1)^{|T|} \text{Prob}(R_1 \leq \ell_1, R_1 + R_2 \leq \ell_2, \dots, R_1 + \dots + R_{m+1} \leq k_1 + \dots + k_{m+1} - 1 \mid \\
 & \quad \ell_j = k_1 + \dots + k_j - 1 \text{ if } j \in T, \ell_j = k_1 + \dots + k_j \text{ else}) \quad (5.145)
 \end{aligned}$$

Actually, the right-hand probabilities of the second sum in (5.145) correspond to subsets $T' = T \cup \{m+1\} \subseteq \{1, \dots, m+1\}$. Furthermore, since $|T'| = |T| + 1$, we can combine the last two sums of (5.145), which leads to

$$\begin{aligned}
 & \text{Prob}(R_1 = k_1, R_2 = k_2, \dots, R_m = k_m, R_{m+1} = k_{m+1}) = \\
 & \sum_{T \subseteq \{1, \dots, m, m+1\}} (-1)^{|T|} \text{Prob}(R_1 \leq \ell_1, R_1 + R_2 \leq \ell_2, \dots, R_1 + \dots + R_{m+1} \leq \ell_{m+1} \mid \\
 & \quad \ell_j = k_1 + \dots + k_j - 1 \text{ if } j \in T, \ell_j = k_1 + \dots + k_j \text{ else}) \quad (5.146)
 \end{aligned}$$

This completes the proof of (5.143).

1200 [Example] Equation (5.147) illustrates the general formula (5.143) for the special case $m = 2$.

$$\begin{aligned}
 & \text{Prob}(R_1 = k_1, R_2 = k_2) = \\
 & \text{Prob}(R_1 \leq k_1, R_1 + R_2 \leq k_1 + k_2) - \text{Prob}(R_1 \leq k_1 - 1, R_1 + R_2 \leq k_1 + k_2) - \\
 & \text{Prob}(R_1 \leq k_1, R_1 + R_2 \leq k_1 + k_2 - 1) + \text{Prob}(R_1 \leq k_1 - 1, R_1 + R_2 \leq k_1 + k_2 - 1) \quad (5.147)
 \end{aligned}$$

1201 [special cases] If $k_1, \dots, k_m$ contains one or more 0's in the right-hand probabilities of (5.143), "special cases" can occur. If $\ell_j = -1$ for some j , then the whole probability is 0. If $\ell_j > \ell_{j+1}$, then ℓ_j can be replaced by ℓ_{j+1} because the random variables R_j assume non-negative values.

1202 [convolution] The expectation and the variance of the sum $T_{t+1} + \dots + T_{t+\ell}$ are $\ell E(T_j)$ and $\ell \text{Var}(T_j)$.

1203 [convolution densities, special cases] If the random variables $T_1, T_2, \dots$ are iid normally distributed, the sum $T_{t+1} + \dots + T_{t+\ell}$ is normally distributed, too (cf. par. 526). If the random variables $T_1, T_2, \dots$ are iid Gamma distributed, the sum $T_{t+1} + \dots + T_{t+\ell}$ is also Gamma distributed (par. 528). In both cases, it is easy to determine the convolution densities $g^{*(\ell)}$.

1204

[convolution densities, CLT] In the general case (for arbitrary densities $g(\cdot)$), it can be difficult to provide exact expressions for the convolution densities $g^{*(\ell)}$. However, if the CLT applies to the relevant parameters k_j (cf. par. 1208), one can use normal densities for the computation of integrals (5.140). As an additional advantage, there is no need to determine the distribution of the T_j exactly. Instead, it suffices to estimate their expectation and variance.

[convolution densities, CLT] Whether the CLT applies to the densities $f^{*(\ell)}(\cdot)$ for $\ell \geq L_0$ (where L_0 is a suitable lower bound) has to be checked from case to case. Of course, the “closer” f is to a normal distribution, the lower L_0 can be chosen. Note that the Berry-Esséen-Theorem (see, e.g., par. 573) considers the worst-case. 1205

[convolution densities, CLT] If the CLT applies to the T_j (as in Subsection 5.4.2), the results 1206 from this subsection may also be applicable to **noise sources** for which the **random variables** $T_1, T_2, \dots$ are time-locally **stationary** but show (weak) dependencies (but relevant (probable) “large” values k_j). Recall that the CLT is rather robust and, e.g., applies to q -dependent and Markovian **random variables** (cf. pars. 575, 576, 577, 583, 584).

Recall that $Y'_j = R_j \pmod{2}$. Finally, we are interested in the distribution of $Y'_1, \dots, Y'_m$. Obviously, 1207

$$\text{Prob}(Y'_1 = y'_1, \dots, Y'_m = y'_m) = \sum_{k_j \equiv y'_j \pmod{2} \text{ for } 1 \leq j \leq m} \text{Prob}(R_1 = k_1, \dots, R_m = k_m) \quad \text{for } y'_1, \dots, y'_m \in \{0, 1\}. \quad (5.148)$$

In principle, the right-hand side (5.148) comprises infinitely many probabilities. However, if 1208 k_j is “far” from the expectation $E(R_j) = \frac{s}{E(T_j)}$, the corresponding probabilities are negligible. (The quantitative meaning of “far” depends on the distribution of the T_j , in particular on their variance.) With regard to (5.143) and (5.148), it seems reasonable to calculate probabilities $\text{Prob}(R_1 \leq k_1, \dots, R_1 + \dots + R_m \leq k_1 + \dots + k_m)$ for relevant (probable) $(k_1, \dots, k_m)$ first because these probabilities may be needed several times.

Equation (5.148) provides a formula for the joint probability of the **random variables** $Y'_1, \dots, Y'_m$. 1209 Applying Bayes’s formula, we obtain the conditional probabilities

$$\text{Prob}(Y'_m = y'_m \mid Y'_1 = y'_1, \dots, Y'_{m-1} = y'_{m-1}) = \frac{\text{Prob}(Y'_1 = y'_1, \dots, Y'_m = y'_m)}{\text{Prob}(Y'_1 = y'_1, \dots, Y'_{m-1} = y'_{m-1})}. \quad (5.149)$$

Formulae (5.148) and (5.149) allow the calculation the joint **entropy** and the conditional **entropy** 1210 of the **random variables** $Y'_1, \dots, Y'_m$. This concerns both **Shannon entropy** and **min-entropy**.

[Numerical example] Table 6 provides several numerical examples. In all cases, the **random vari-** 1211 **ables** T_j are assumed to be $N(\mu, \sigma^2)$ -distributed. The figures were not gained by the evaluation in (5.140), (5.143), and (5.148). Instead, the **random variables** T_j and thus implicitly also the **random variables** R_j , were simulated (sample size N).

The conditional **Shannon entropy** is computed with formula (4.101) (with $m - 1$ in place of m), and the conditional **min-entropy** (last column of Table 6) is computed with formula (5.150) (cf.

(4.110).

$$H_{\min}(Y'_m \mid Y'_1, \dots, Y'_{m-1}) = \min\{H_{\min}(Y'_m \mid Y'_1 = y'_1, \dots, Y'_{m-1} = y'_{m-1}) \mid y'_1, \dots, y'_{m-1} \in \{0, 1\}\}. \quad (5.150)$$

Table 6: Simulation experiments (design type A): $T_j \sim N(\mu, \sigma^2)$, $\mu = 1.0$, sample size $N = 10,000,000$

$\left(\frac{s}{\mu}, \frac{\sigma}{\mu}\right)$	m	$\frac{H(Y'_1, \dots, Y'_m)}{m}$	$H(Y'_m \mid Y'_1, \dots, Y'_{m-1})$	$\frac{H_{\min}(Y'_1, \dots, Y'_m)}{m}$	$H_{\min}(Y'_m \mid Y'_1, \dots, Y'_{m-1})$
(25, 0.2)	4	0.99998	0.99998	0.9928	0.9914
(10, 0.2)	4	0.9907	0.9907	0.843	0.842
(100, 0.1)	4	0.99997	0.99997	0.9914	0.9905
(10000, 0.01)	4	0.99998	0.99998	0.9922	0.9915

1212 Tab. 6 provides exemplary figures for pairs of parameters. An important question is how sensitive the corresponding **entropy** values are with regard to deviations of the parameters. To investigate this question, in Tab. 7 we considered subsets of parameters. Let

$$A_{[\alpha_1, \alpha_2, \beta_1, \beta_2]}(\mu, \sigma) := \{(\mu', \sigma') \mid \alpha_1 \mu \leq \mu' \leq \alpha_2 \mu, \beta_1 \sigma \leq \sigma' \leq \beta_2 \sigma\}, \quad \alpha_1, \beta_1 \leq 1 \leq \alpha_2, \beta_2. \quad (5.151)$$

Note: The sensitivity of the **entropy** values is a crucial feature for the robustness of the design of a **physical noise source**. Little sensitivity against deviations of the parameters is a desirable feature (“robust design”), which reduces the requirements on the **online test**, and also the protection against active attacks should become easier.

1213 [Numerical example] Tab. 7 provides numerical examples for sets $A_{[0.9, 1.1, 0.9, 1.1]}(\mu, \sigma)$. As in Tab. 6, the **random variables** T_j are assumed to be $N(\mu, \sigma^2)$ -distributed. Again, the figures were not gained by the evaluation of the formulae (5.140), (5.143), and (5.148). Same as for Table 6, the **random variables** T_j and thus the **random variables** R_j were simulated (sample size N). We treated pairs of parameters (μ', σ') for which $\mu' \in \{0.9\mu, 0.95\mu, \mu, 1.05\mu, 1.10\mu\}$ and $\sigma' \in \{0.9\sigma, 0.95\sigma, \sigma, 1.05\sigma, 1.10\sigma\}$. The conditional **Shannon entropy** is computed with formula (4.101) (with $m - 1$ in place of m). As in Tab. 6, we computed the conditional **min-entropy** for each pair (μ', σ') with formula (5.150).

Note: The experiments confirm the intuitive understanding that $(\mu', \sigma') = (1.1\mu, 0.9\sigma)$ is the “worst-case”.

1214 [Numerical example, Gamma distribution] Subsection 5.4.2 summarizes results from [KiSc08] on a **physical noise source** design that exploits two noisy diodes. In [KiSc08] it was shown that the (one-dimensional) distribution of the T_j can be approximated by a Gamma distribution with estimated shape parameter $\tilde{\alpha} = 3.0949$ and estimated rate parameter $\tilde{\beta} = 0.0240$. Furthermore, in [KiSc08] it was concluded that consecutive T_j should be 1-dependent and only weakly autocorrelated. Under the *idealized assumption* that the T_j are **iid** Gamma distributed with parameters $\tilde{\alpha}$ and $\tilde{\beta}$, simulations (as for Tabs. 6 and 7) show that the conditional **min-entropy** $H_{\min(A)}(Y'_4 \mid Y'_1, \dots, Y'_3)$ is > 0.99 for all parameter sets considered in Tab. 1 of [KiSc08].

Table 7: Simulation experiments (design type A): $T_j \sim N(\mu, \sigma^2)$, $\mu = 1.0$, subsets $A_{[0.9, 1.1, 0.9, 1.1]}(\mu, \sigma)$, the index “(A)” indicates that sets are considered. The values in the upper line denote the average, the values in the lower line the worst-case, sample size $N = 10,000,000$,

$\left(\frac{s}{\mu}, \frac{\sigma}{\mu}\right)$	m	$\frac{H_{(A)}(Y'_1, \dots, Y'_m)}{m}$	$H_{(A)}(Y'_m \mid Y'_1, \dots, Y'_{m-1})$	$\frac{H_{\min(A)}(Y'_1, \dots, Y'_m)}{m}$	$H_{\min(A)}(Y'_m \mid Y'_1, \dots, Y'_{m-1})$
(25, 0.2)	4	0.99995	0.99995	0.9925	0.9905
		0.99975	0.99975	0.9758	0.9700
(10, 0.2)	4	0.99172	0.99169	0.8893	0.8764
		0.95734	0.95738	0.6803	0.6759
(100, 0.1)	4	0.99988	0.99988	0.9877	0.9860
		0.99896	0.99895	0.9465	0.9429
(10000, 0.01)	4	0.99998	0.99998	0.9878	0.9857
		0.99892	0.99892	0.9422	0.9422

[stochastic model] The distribution of the variables $R_1, R_2, \dots$ depends on the distribution of the iid random variables $T_1, T_2, \dots$, and, thus, also on the number of parameters that specify the admissible distributions. 1215

[stochastic model] If the T_j are normally distributed or Gamma distributed, for example, their distribution is defined by μ and σ^2 or by the shape parameter α and the rate parameter β . If the T_j are normally distributed, the distribution of the R_j depends on the triples (s, μ, σ) , while for the Gamma distribution, it depends on (s, α, β) , or equivalently, on $(s, \mu = \alpha/\beta, \sigma = \sqrt{\alpha}/\beta)$. In both cases, the distribution of the R_j remains unchanged if the triplet (s, μ, σ) is multiplied by some factor $r > 0$ (transformation theorem). Consequently, the distribution of the variables $R_1, R_2, \dots$ depends on the two parameters $(s/\mu, \mu/\sigma)$. 1216

Note 1: If the length of the sampling interval s is fixed, the 2-parameter-family of admissible distributions of the R_j can alternatively be parametrized, e.g., by $(\mu, \mu/\sigma)$, (μ, σ^2) , or (μ, σ) . This is because the parameters can be transformed into each other by bijective mappings on $[0, \infty)^2$.

Note 2: These conclusions hold approximately, if the CLT is applicable; see pars. 1204 to 1206.

Note 3: Interestingly, $E(R_j)$ and (asymptotically) $\text{Var}(R_j)$ depend only on $(s/\mu, \sigma/\mu)$ for any distribution of the T_j ; cf. pars. 1220 to 1221.

[stochastic model] If the random variables $T_1, T_2, \dots$, are iid exponentially distributed with parameter τ , it is well-known that the random variables $R_1, R_2, \dots$ are iid Poisson distributed with parameter τs . In this case, the stochastic model depends only on one parameter. 1217

Note 1: This models a physical noise source that counts radioactive decays with the intervals $I_1, I_2, \dots$, using an ideal Geiger counter.

Note 2: Also, designs with real-world (non-ideal) Geiger counters are candidates that potentially fit the stochastic model considered in this subsection; cf. Subsection 5.4.5, see in particular par. 1265.

[online test] Intuitively, it seems to be clear that s/μ and σ/μ should be “large”. This insight has been confirmed by numerical experiments above. The task of the online test is to detect if 1218

any of these conditions are violated. Generally, one should proceed as follows: In a first step, subsets A_{real} , A_{good} , and A_{bad} are specified, e.g., for the parameters $(s/\mu, \sigma/\mu)$ as described in Subsection 4.5.3. The **online test** shall detect if the true parameters leave the subset A_{good} (moving into A_{bad}) when the **PTRNG** is in operation. Usually, testing the empirical distribution of the T_j is not possible as it (at least) would require a precise clock.

1219 [online test] Intuitively, one might expect that a large variance of the **random variables** $R_1, R_2, \dots$ ensures large conditional **min-entropy** of the **internal random numbers**. This suggests testing the integer-valued **raw random numbers** $r_1, r_2, \dots$. Therefore, a closer look at $E(R_j)$ and $\text{Var}(R_j)$ is necessary.

1220 [renewal process] The **random variables** $Z'(t) := \inf\{k \mid W_0 + T_1 + \dots + T_k > t\}$ define a **stationary** renewal process ($t \in [0, \infty)$); cf. par. 1183. It is $Z_n = Z'(ns)$ for all $n \in \mathbb{N}_0$. This means that the **random variables** $Z_1, Z_2, \dots$ coincide with $Z'(t)$ at the times $t = s, 2s, \dots$. Let $s = \tau\mu$, or equivalently, $\tau = s/\mu$. From (4.68) and (4.69), we obtain

$$E(Z_n) = \frac{n\tau\mu}{\mu} = n\tau \quad (5.152)$$

$$\text{Var}(Z_n) = \left(\frac{\sigma^2}{\mu^2}\right) n\tau + \frac{1}{6} + \frac{1}{2} \left(\frac{\sigma}{\mu}\right)^4 - \frac{E((T_j - \mu)^3)}{3\mu^3} + o(1). \quad (5.153)$$

Note 1: Setting $n = 1$ in (5.152) and (5.153) yields $E(R_j)$ and $\text{Var}(R_j)$.

Note 2: Furthermore, the stationarity of the R_j implies $E(R_j + \dots, R_{j+i-1}) = E(Z_i)$ and $\text{Var}(R_j + \dots, R_{j+i-1}) = \text{Var}(Z_i)$.

1221 [renewal process] If $T_j \sim N(\mu, \sigma^2)$, for example, $E((T_j - \mu)^3) = 0$. Then (5.152) and (5.153) simplify to

$$E(R_j) = \tau \quad \text{and} \quad (5.154)$$

$$\text{Var}(R_j) \approx \left(\frac{\sigma}{\mu}\right)^2 \tau + \frac{1}{6} + \frac{1}{2} \left(\frac{\sigma}{\mu}\right)^4 \quad \text{for "large" } \tau = \frac{s}{\mu}. \quad (5.155)$$

In our applications, usually $\sigma/\mu < 1$ or even $\sigma/\mu \ll 1$ so that (5.155) further simplifies to

$$\text{Var}(R_j) \approx \left(\frac{\sigma}{\mu}\right)^2 \tau + \frac{1}{6} \quad \text{for "large" } \tau = \frac{s}{\mu}. \quad (5.156)$$

1222 [renewal process] Since the **random variables** $R_1, R_2, \dots$ are **stationary**,

$$\begin{aligned} \text{Cov}(R_n, R_{n+1}) &= 0.5 (\text{Var}(R_n + R_{n+1}) - \text{Var}(R_n) - \text{Var}(R_{n+1})) = \\ &= 0.5 (\text{Var}(R_1 + R_2) - 2\text{Var}(R_1)) = \\ &= 0.5 \left(-\frac{1}{6} - \frac{1}{2} \left(\frac{\sigma}{\mu}\right)^4 + \frac{E((T_j - \mu)^3)}{3\mu^3} + o(1) \right) \\ &= -\frac{1}{12} - \frac{1}{4} \left(\frac{\sigma}{\mu}\right)^4 + \frac{E((T_j - \mu)^3)}{6\mu^3} + o(1). \end{aligned} \quad (5.157)$$

If the third central moment $E((T_j - \mu)^3)$ vanishes (e.g., because $T_j \sim N(\mu, \sigma^2)$), if $\sigma/\mu \ll 1$, and if τ (and by this, the sampling interval s) is sufficiently large, then (5.157) simplifies to

$$\text{Cov}(R_n, R_{n+1}) \approx -\frac{1}{12}. \quad (5.158)$$

[renewal process] Equation(5.159) follows from the definition and by substituting of (5.153) and (5.157). This is an equivalent to (5.125) for variances and covariances. 1223

$$\begin{aligned} 0 &= \text{Var}(R_1 + \dots + R_j) - j\text{Var}(R_1) - 2 \sum_{i=2}^j (j+1-i) \text{Cov}(R_1, R_i) \\ &= \text{Var}(R_1 + \dots + R_j) - j\text{Var}(R_1) - 2(j-1)\text{Cov}(R_1, R_2) - \sum_{i=3}^j (j+1-i) \text{Cov}(R_1, R_i) \\ &= \left(\frac{\sigma^2}{\mu^2} \right) j\tau + \frac{1}{6} + \frac{\sigma^4}{2\mu^4} - \frac{E((T_j - \mu)^3)}{3\mu^3} + o(1) - j \left(\left(\frac{\sigma^2}{\mu^2} \right) \tau + \frac{1}{6} + \frac{\sigma^4}{2\mu^4} - \frac{E((T_j - \mu)^3)}{3\mu^3} + o(1) \right) \\ &\quad + 2(j-1) \left(-\frac{1}{12} - \frac{1}{4} \left(\frac{\sigma}{\mu} \right)^4 + \frac{E((T_j - \mu)^3)}{6\mu^3} + o(1) \right) - \sum_{i=3}^j (j+1-i) \text{Cov}(R_1, R_i) \\ &= o(1) - \sum_{i=3}^j (j+1-i) \text{Cov}(R_1, R_i) \quad \text{for } j \geq 3. \end{aligned} \quad (5.159)$$

Setting $j = 3$ (5.159) implies that $\text{Cov}(R_1, R_3) = o(1)$. By induction on j , one concludes that

$$\text{Cov}(R_1, R_j) = o(1) \quad \text{for } j \geq 3. \quad (5.160)$$

Thus, if the ratio s/μ is sufficiently large, $\text{Cov}(R_1, R_j) \approx 0$ for $j \geq 3$, which confirms the intuition.

[renewal process] The first and the third term of (5.153) can be expressed in terms of s/μ and μ/σ . For a given interval length s , both terms depend on the two first moments of the T_j , i.e., on μ and σ^2 . The forth term is a multiple of the third central moment of T_j . If the random variables T_j are normally distributed, the fourth term vanishes. Of course, if the interval length s is sufficiently large, the first term dominates anyway. 1224

[online test] In par. 1216 it was pointed out that if the random variables T_j are normally distributed or Gamma distributed, the distribution of the R_j only depends on s/μ and μ/σ . This should be approximately true if the CLT applies. Together with the observation in par. 1224, this suggests applying an online test that exploits estimates of $E(R_j)$ and $\text{Var}(R_j)$. This means that we have to determine regions A'_{real} and A'_{bad} such that $(E(R_j), \text{Var}(R_j)) \in A'_{real}$ implies $(s/\mu, \sigma/\mu) \in A_{real}$, whereas $(s/\mu, \sigma/\mu) \in A_{bad}$ implies $(E(R_j), \text{Var}(R_j)) \in A'_{bad}$. (Ideally, the implications would be equivalences.) And, of course, an appropriate online test must be able to “separate” A'_{real} from A'_{bad} . In particular, the online test shall quickly detect if $(E(R_j), \text{Var}(R_j)) \in A'_{bad}$. 1225

[online test, $T_j \sim N(\mu, \sigma^2)$] Fig. 30 illustrates the close connection between the min-entropy of the random variables Y'_j and $\text{Var}(R_j)$ for normally distributed T_j . Parameters $(s/\mu, \sigma/\mu)$, 1226

for which the conditional **min-entropy** $H_{\min}(Y'_2 | Y'_1)$ is 0.98 lie almost perfectly on the curve $\{(s/\mu, \sigma/\mu) \mid \text{Var}(R_j) = 1.05\}$, or shortly, $\{\text{Var}(R_j) = 1.05\}$. This curve was computed by (5.155). For other conditional **min-entropy** values the situation is similar; the corresponding parameters $(s/\mu, \sigma/\mu)$ lie almost perfectly on curves with constant $\text{Var}(R_j)$.

Note 1: For parameters that provide large conditional **min-entropy** values (relevant for the functionality class PTG.2), it is $H_{\min}(Y'_2 | Y'_1) \approx H_{\min}(Y'_4 | Y'_1, Y'_2, Y'_3)$.

Note 2: For parameters $(s/\mu, \sigma/\mu)$ with small **min-entropy** values, this need not be the case.

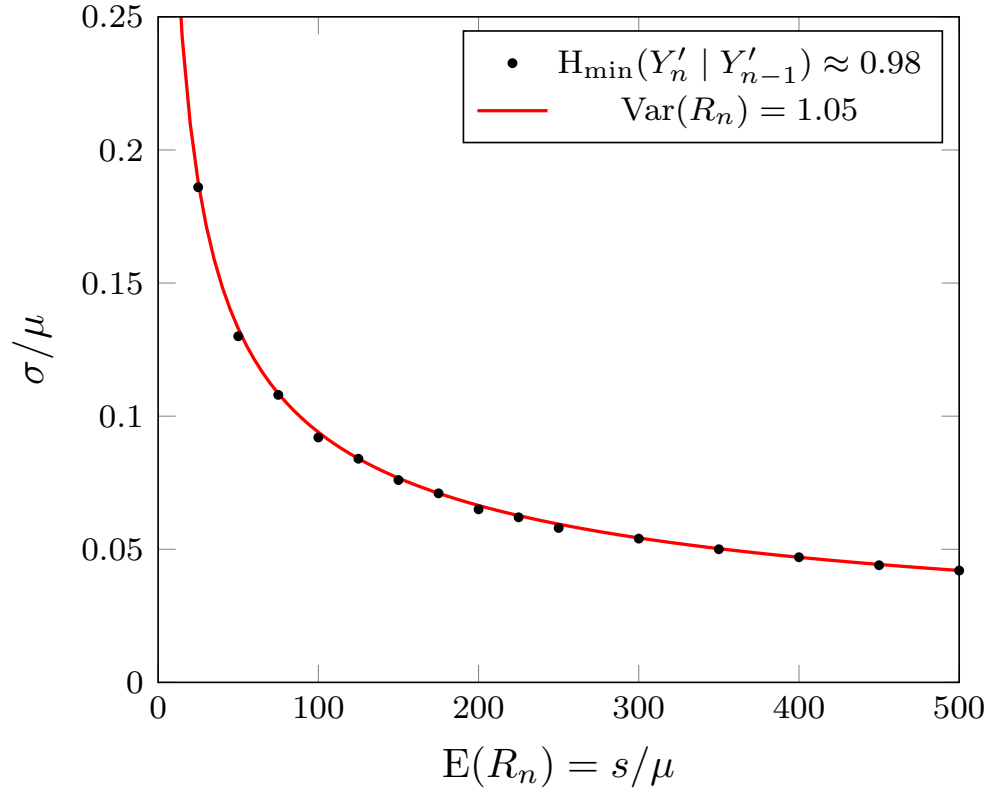


Figure 30: $T_j \sim N(\mu = 1, \sigma^2)$. The black dots belong to parameters with $H_{\min}(Y'_2 | Y'_1) = 0.98$.

- 1227 [online test, $T_j \sim N(\mu, \sigma^2)$] Fig. 31 illustrates how the variance of the R_j separates A_{real} and A_{bad} . The upper curve $\{\text{Var}(R_j) = 1.40\}$ corresponds to the conditional **min-entropy** $H_{\min}(Y'_2 | Y'_1) = 0.995$.
- 1228 [online test, $T_j \sim N(\mu, \sigma^2)$] For normally distributed **random variables** $T_1, T_2, \dots$, pars. 1226 and 1227 suggest an **online test** that only exploits the variance $\text{Var}(R_j)$. Here, analogously to par. 1225, subsets $A''_{\text{real}}, A''_{\text{bad}} \subseteq (0, \infty)$ for the variance $\text{Var}(R_j)$ need to be defined. This means that $\text{Var}(R_j) \in A''_{\text{real}}$ implies that $(s/\mu, \sigma/\mu) \in A_{\text{real}}$, whereas $(s/\mu, \sigma/\mu) \in A_{\text{bad}}$ implies that $\text{Var}(R_j) \in A''_{\text{bad}}$. In par. 1229, the sets $A''_{\text{real}} = (1.40, \infty)$ and $A''_{\text{bad}} = (0, 1.05)$ are used.
- 1229 [online test, $T_j \sim N(\mu, \sigma^2)$] For the sample of **raw random numbers** $r_1, \dots, r_m$, the **online test** calculates the one-dimensional empirical variance $\bar{s}^2$ of the **random variables** R_j with formula

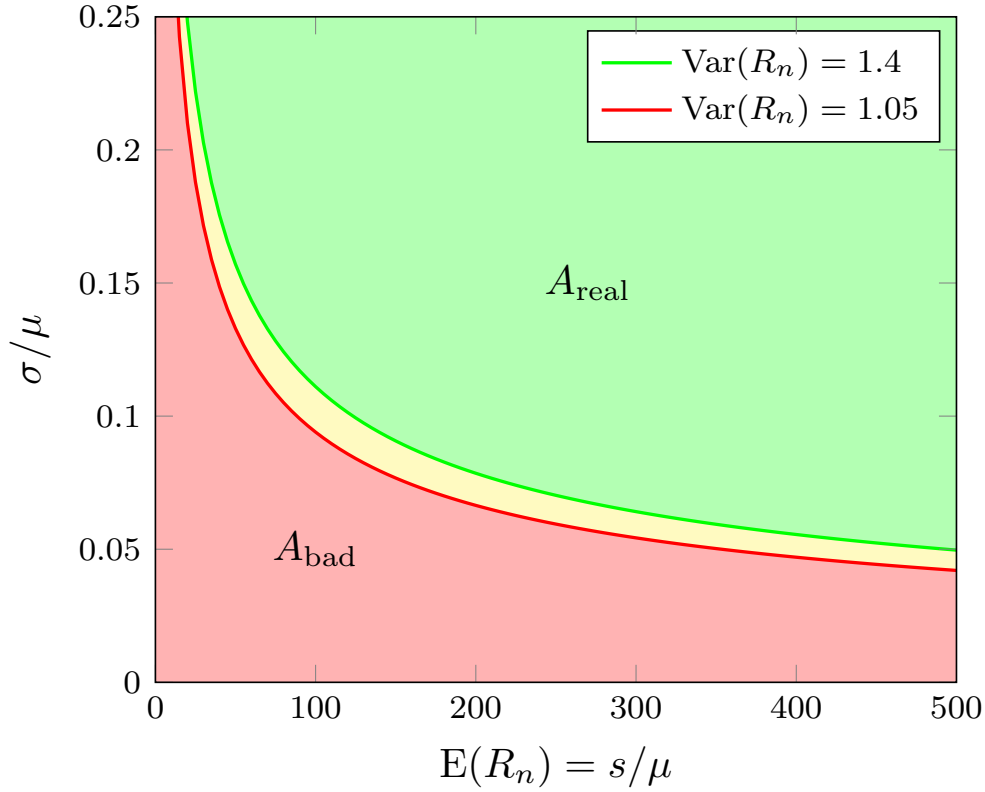


Figure 31: $T_j \sim N(\mu = 1, \sigma^2)$. The variance $\text{Var}(R_n)$ separates A_{real} from A_{bad} .

(4.31). The **online test** fails if $\bar{s}^2 < 1.20$. Tab. 8 collects the probabilities for **false positives** and false negatives, depending on the sample size m of the **online test**. **False positive** means that the **online test** fails although the true distribution is in A''_{real} . If the **online test** does not fail although the true distribution is in A''_{bad} , we speak of a false negative.

Table 8: Simulated probabilities for false positives and false negatives: $T_j \sim N(\mu, \sigma^2)$, $\mu = 1.0$, test sample size m , number of simulated test values $N = 10,000,000$.

m	Prob(false positive)	Prob(false negative)
1024	0.00034	0.00096
1536	0.00002	0.00007
2048	0.0000007	0.0000045

Note 1: Tab. 8 shows that the **online test** is very strong, especially for $N = 2048$. In this case, a complex **online test procedure** as described in Subsection 5.5.2 is not necessary unless the sample size $N = 2048$ is too large for the evaluated **physical noise source** design. (If necessary, this problem could be solved by buffering the **internal random number** some time before they are output.)

Note 2: For comparison, assume that $B_1, B_2, \dots, B_m$ are iid $B(1, p)$ -distributed with $H_{\min} \geq 0.995$ (as for $\{\text{Var}(R_j) \geq 1.40\}$). Then $|p - 0.5| \leq 0.0017$. For these binary-valued random variables, it requires a much larger sample size m to separate equally efficiently between $A_{\text{real}}^* = [0.4983, 0.5017]$ and $A_{\text{bad}}^* = [0, 0.4931] \cup [0.5069, 1]$.

- 1230 [online test, T_j is Gamma distributed] Finally, we consider Gamma-distributed random variables T_j (shape parameter α , rate parameter β). The variance $\text{Var}(R_j)$ follows from (5.153) with $n = 1$, where we again assume that the $o(1)$ -term is negligible for the selected parameters. For Gamma-distributed T_j , the third central moment does not vanish. Interestingly,

$$\frac{E((T_j - \mu)^3)}{3\mu^3} = \frac{\sigma^3 E\left(\left(\frac{T_j - \mu}{\sigma}\right)^3\right)}{3\mu^3} = \frac{1}{3} \left(\frac{\sigma}{\mu}\right)^3 \cdot \frac{2}{\sqrt{\alpha}} = \frac{2}{3} \left(\frac{\sigma}{\mu}\right)^4. \quad (5.161)$$

Fig. 32 illustrates the close connection between the min-entropy of the random variables Y'_j and $\text{Var}(R_j)$ if the T_j are Gamma distributed. The situation is rather similar to normally distributed random variables T_j ; cf. par. 1226, Fig. 30. Consequently, also for Gamma distributed T_j , the empirical variance can be used to design an appropriate online test.

Note: Care should be taken to ensure that the ratio s/μ is not too small. Otherwise, (even if σ/μ is sufficiently large) for $s/\mu \leq 2$, the results are sensitive to changes of the parameters.

- 1231 [online test] If the random variables are neither normally distributed nor Gamma distributed, the relation between the conditional min-entropy of the random variables Y'_j and $\text{Var}(R_j)$ needs to be analyzed as it was done above. Care should be taken to ensure that the empirical mean of the raw random numbers $r_1, r_2, \dots$ does not become too small, neither by design nor accidentally while the PTRNG is in operation. Unless the variance of the T_j increases correspondingly, the second scenario should be detected by an online test that computes the empirical variance of R_j . Otherwise, if it could be possible (under consideration of the physical noise source) that both the empirical variance and the empirical mean of the T_j can significantly increase at the same time, the online test should additionally monitor the empirical mean of the raw random numbers.

5.4.4 Sampling Events with iid Intermediate Time Intervals – Design B

- 1232 As in Subsection 5.4.3, this subsection focuses on the mathematical treatment of a generic stochastic model, which may fit different noise sources designs.

Note: The developer can refer to this subsection but, of course, has to give evidence that the stochastic model indeed fits the design under evaluation.

- 1233 In this subsection, we assume that a physical noise source latches a (perfect) square wave with constant period length s whenever a (design-specific) random event occurs. This event might be, for example, that a ring oscillator has terminated N periods since the last latching. The time intervals between two successive events are denoted by $t_1, t_2, \dots$. Thus, the square wave is latched at time instants $t_1, t_1 + t_2, \dots$, and the raw random numbers $r'_1, r'_2, \dots$ are given by

$$r'_j = \begin{cases} 0 & \text{if } w_0 + t_1 + \dots + t_j \in [ks, (k + 0.5)s) \text{ for some } k \in \mathbb{N}_0 \\ 1 & \text{if } w_0 + t_1 + \dots + t_j \in [(k + 0.5)s, (k + 1)s) \text{ for some } k \in \mathbb{N}_0. \end{cases} \quad (5.162)$$

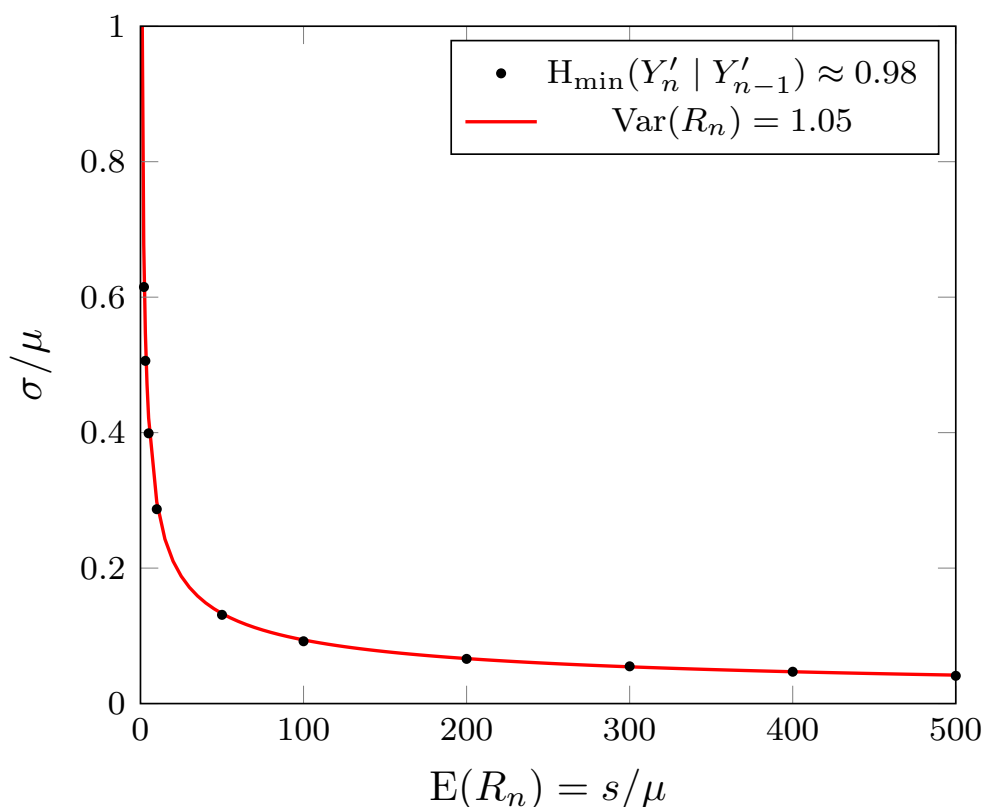


Figure 32: T_j is Gamma-distributed. The black dots belong to parameters with $H_{\min}(Y'_2 | Y'_1) = 0.98$.

Note: Compared to Subsection 5.4.3, the roles of the **noise source** and the regular signal are interchanged.

The goal is to determine the joint distribution of **random variables** $R'_1, \dots, R'_m$. This allows a determination of the joint **entropy** and conditional entropies for both **Shannon entropy** and **min-entropy**. Unlike in Subsection 5.4.3, the **raw random numbers** are not integer-valued but already binary-valued.

As in Subsection 5.4.3, we assume that sampling is ideal in the sense that the **raw random numbers** $r'_1, r'_2, \dots$ are given by (5.162). The latching event can occur around times ks or $(k + 0.5)s$ (for $k \in \mathbb{N}_0$) when the square wave changes its value (from 1 to 0 or from 0 to 1), which may cause deviations from (5.162). If such “latching errors” occur rarely, they may be neglected in the **stochastic model**.

[iid assumption] We assume that the time lengths $t_1, t_2, \dots$ can be viewed as realizations of **iid random variables** $T_1, T_2, \dots$. In particular, the **random variables** $X(t) := \sup\{k \mid W_0 + T_1 + \dots + T_k \leq t\}$ define a renewal process ($t \in [0, \infty)$); cf. par. 1163.

As in Subsection 5.4.3, the **random variable** W_0 describes the (random) time when the first event occurs after time $t = 0$. The **random variable** W_0 quantifies the phase of the random events relative to the square wave when the first considered time interval begins.

- 1238 [Assumption] The distribution of the **random variables** $T_1, T_2, \dots$ has density (to be mathematically precise: Lebesgue density) $g(\cdot)$. If $G_T(\cdot)$ denotes the cumulative distribution function of T_j , then W_{0j} has the density $g_W(\cdot) = \frac{1}{\mu}(1 - G_T(\cdot))$ (par. 1159).

Note 1: In particular, $\text{Prob}(W_j > 0) = 1$ for all $j \in \mathbb{N}$.

Note 2: The case that the distribution of T_j has a density constitutes the most relevant case for applications. We mention that similar results can be derived if the **random variables** $T_1, T_2, \dots$ do not have a density $g(\cdot)$ (although with greater mathematical effort). If the **random variables** T_j are discrete, the integrals below turn into sums.

- 1239 For each integer $\ell \geq 1$, the term $g^{*(\ell)}(\cdot)$ denotes the ℓ -fold convolution of the density $g(\cdot)$. In particular, $g^{*(1)}(\cdot) = g(\cdot)$.

Note: For each $t \in \mathbb{N}_0$ and $\ell \in \mathbb{N}$, the sum $T_{t+1} + \dots + T_{t+\ell}$ has density $g^{*(\ell)}(\cdot)$.

- 1240 We define the sets $A(0)$ and $A(1)$:

$$A(0) := \bigcup_{k=0}^{\infty} [ks, (k+0.5)s) \quad \text{and} \quad A(1) := \bigcup_{k=0}^{\infty} [(k+0.5)s, (k+1)s). \quad (5.163)$$

It is $R'_j = b$ iff $W_0 + T_1 + \dots + T_j \in A(b)$ for $b = 0, 1$.

- 1241 For $m \in \mathbb{N}$ and $b_1, \dots, b_m \in \{0, 1\}$, we have

$$\begin{aligned} & \text{Prob}(R'_1 = b_1, R'_2 = b_2, \dots, R'_m = b_m) = \\ & \text{Prob}(W_0 + T_1 \in A(b_1), W_0 + T_1 + T_2 \in A(b_2), \dots, W_0 + T_1 + \dots + T_m \in A(b_m)) = \\ & \int_{A(b_m)} \int_{A(b_{m-1})} \dots \int_{A(b_1)} \int_0^{\infty} g^{*(m)}(u_m - u_{m-1}) g^{*(m-1)}(u_{m-1} - u_{m-2}) \dots \\ & \quad g^{*(1)}(u_1 - u_0) g_W(u_0) du_0 du_1 \dots du_{m-1} du_m. \end{aligned} \quad (5.164)$$

Note 1: The term “ du_0 ” belongs to “ $\int_0^{\infty}$ ”.

Note 2: Since $g_W(u), g^{*(\ell)}(u) = 0$ for $u < 0$ ($\ell \geq 1$), the integrand does not contribute to the integral unless $0 \leq u_0 \leq u_1 \leq \dots \leq u_m$.

- 1242 If the **noise source** and the square wave signal are synchronized at time $t = 0$, then $W_0 \equiv 0$, and the inner integral ($\int_0^{\infty}$) in (5.164) drops out.

- 1243 [Numerical example] Table 9 provides several numerical examples. In all cases, the **random variables** T_j are assumed to be $N(\mu, \sigma^2)$ -distributed. The figures were not gained by the evaluation (5.164). Instead, the **random variables** T_j , and implicitly the **random variables** R'_j , were simulated (sample size N). The conditional **Shannon entropy** is computed with formula (4.101) (with $m-1$ in place of m), and the conditional **min-entropy** (last column of Table 9) is computed with formula (5.150) (cf. (4.110)).

$$H_{min}(R'_m \mid R'_1, \dots, R'_{m-1}) = \min\{H_{min}(R'_m \mid R'_1 = b_1, \dots, R'_{m-1} = b_{m-1}) \mid b_1, \dots, b_{m-1} \in \{0, 1\}\}. \quad (5.165)$$

Table 9: Simulation experiments (design type B): $T_j \sim N(\mu, \sigma^2)$, $\mu = 1.0$, sample size $N = 10,000,000$.

$\left(\frac{s}{\mu}, \frac{\sigma}{\mu}\right)$	m	$\frac{H(R'_1, \dots, R'_m)}{m}$	$H(R'_m \mid R'_1, \dots, R'_{m-1})$	$\frac{H_{min}(R'_1, \dots, R'_m)}{m}$	$H_{min}(R'_m \mid R'_1, \dots, R'_{m-1})$
(1971, 0.24)	4	0.9945	0.9913	0.9211	0.7776
(3942, 0.34)	4	0.9975	0.9967	0.9306	0.9012
(7885, 0.48)	4	0.9994	0.9999	0.9969	0.9948

It is much easier to express the joint probability $\text{Prob}(R'_1 = b_1, R'_2 = b_2, \dots, R'_m = b_m)$ by integrals than in Subsection 5.4.3, and concrete calculations require much less effort. On the negative side, it seems to be very difficult to develop an effective **online test** unless the device is able to measure the intermediate times $t_1, t_2, \dots$ (or, equivalently, the times $t_1, t_1 + t_2, \dots$). The reason is that the binary-valued **raw random numbers** $r'_1, r'_2, \dots$ contain much less information than the integer-valued **raw random numbers** $r_1, r_2, \dots$ in Subsection 5.4.3. 1244

5.4.5 A PTRNG Design Exploiting Radioactive Decay

In Subsection 5.4.5, we discuss the **stochastic model** of a **PTRNG** design that exploits a physical phenomenon, namely radioactive decay, detected and digitized by a Geiger counter. Mathematically, we follow [Neue04], Section 4.2. We mention that this design is also treated in ISO/IEC 20543 [ISO_20543], A.3.4, Example 3. 1245

[design] The **noise source** is a radioactive source that is assumed to decay spontaneously. There is a Geiger counter close to this radioactive source. The Geiger counter is connected to a computer. The device measures the intermediate times $t_1, t_2, \dots$ between consecutive impulses of the Geiger counter. From these intermediate times the **PTRNG** computes the **raw random numbers**. Furthermore, the half-life L of the radioactive material is substantially larger than the expected lifetime of the **PTRNG**. 1246

Note: In this subsection, we assume that the radioactive material decays to stable products. Our considerations apply to decay chains, too, if the number of decays of the generated (intermediate) radioactive products is negligible compared to the number of radioactive decays of the radioactive starting material. This is the case if the half-life of the generated radioactive products is very large (absolute and compared to that of the radioactive starting material). Of course, the considerations below can be adjusted to arbitrary decay chains, too, although in the general case, the mathematical treatment becomes more difficult. 1247

The measured intermediate times $t_1, t_2, \dots$ are interpreted as realizations of **random variables** $T_1, T_2, \dots$

1249 [ideal Geiger counter] To become familiar with this design and its special features at first, pars. 1250 to 1255 consider an ideal Geiger counter. This means that it detects all radioactive decays and measures the intermediate times exactly. In particular, the dead time of an ideal Geiger counter is 0.

1250 [ideal Geiger counter] If the Geiger counter is ideal, the random intermediate times are independent and exponentially distributed with parameter θ . The density $f(\cdot)$ and cumulative distribution function $F(\cdot)$ of the **random variables** T_j are given by

$$f(t) = \frac{1}{\theta} e^{-\theta t} \quad \text{and} \quad F(t) = 1 - e^{-\theta t} \quad \text{for } t > 0 \quad (5.166)$$

for some parameter $\theta > 0$. The parameter θ does not only depend on the radioactive material but also on its quantity. For a realistic lifetime of the **RNG**, we may assume that the parameter θ essentially remains constant; cf. par. 1247.

1251 [ideal Geiger counter] Under this assumption, a **random variable** N that counts the number of impulses within a time interval of length s is **POISSON** distributed with parameter $\lambda = \theta s$. In particular,

$$\text{Prob}(N = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad \text{for } k \in \mathbb{N}_0. \quad (5.167)$$

1252 [ideal Geiger counter] If T_j is exponentially distributed with parameter θ , then the **random variable** $U_j = e^{-\theta T_j}$ is **uniformly distributed** on the unit interval. (Note that $\text{Prob}(U_j \leq x) = x$ for each $x \in (0, 1)$.) Thus, the k most significant bits of $u_1, u_2, \dots$ (derived from the times $t_1, t_2, \dots$) may be used as k -bit **raw random numbers**.

1253 [ideal Geiger counter] The drawback of both methods described in par. 1252 is that one needs to know the (exact) parameter θ , resp. $\lambda = \theta s$. Additionally, for real-world (non-ideal) Geiger counters, the detection rate q affects θ and λ . These properties are in particularly unfavorable if many **PTRNGs** have to be evaluated and if it is not a realistic option to estimate θ (and the detection rate q) individually for each **PTRNG**. Instead, par. 1254 proposes an algorithm that gets by without knowledge of θ . Additionally, over time the parameter θ shrinks to some degree. The cost is that the output rate decreases to 50%.

1254 [ideal Geiger counter] Under the assumptions from par. 1250, the **random variables**

$$U_i = \frac{T_{2i}}{T_{2i-1} + T_{2i}} \quad \text{for } i \in \mathbb{N} \quad (5.168)$$

are **uniformly distributed** on the unit interval $[0, 1)$, regardless of θ . The k most significant bits R_i of the binary representation of U_i are **uniformly distributed** on $\{0, 1\}^k$. Thus, from $t_1, t_2, \dots$, one can compute the values $u_1, u_2, \dots$ and therefrom, the k -bit **raw random number** vectors $r_1, r_2, \dots$

1255

[ideal Geiger counter] Of course, a **stochastic model** as developed in pars. 1249 to 1254 will not be accepted by the evaluator because ideal Geiger counters do not exist in the real world. Below, we analyze a more realistic scenario. We follow [Neue04].

A “real-world” detection mechanism is not able to measure the intermediate times between decays exactly but only in multiples of a positive constant Δ (i.e., length of a clock cycle). If multiple impulses occur within one clock cycle, then only one impulse is counted. 1256

The random intermediate times $T_1, T_2, \dots$ are discrete and can assume values that are integer multiples of the dead time Δ . In particular, the **random variables** $T_1, T_2, \dots$ are **iid** geometrically distributed with parameter $p = 1 - e^{-\theta\Delta}$ ($= \text{Prob}(T_j \leq \Delta)$). 1257

These considerations are also appropriate for a Geiger counter that detects a decay with probability $q < 1$. If we assume $\Delta = 0$ for the moment, the **random variables** N of the detected decays per time unit would be Poisson distributed with parameter $\lambda = q\theta s$ instead of $\lambda = \theta s$. Thus, the **random variables** $T_1, T_2, \dots$ (modeling a Geiger counter with detection rate q and dead time Δ) are **iid** geometrically distributed with parameter $p = 1 - e^{-q\theta\Delta}$. 1258

Formula (5.168) requires a modification that considers these real-world assumptions. Analogously to (5.168), we set 1259

$$U'_i = \frac{T_{2i} - 0.5\Delta}{T_{2i-1} + T_{2i} - \Delta} \quad \text{for } i \in \mathbb{N}. \quad (5.169)$$

This leads to the following inequalities ([Neue04], Theorem 4.1)

$$\frac{1}{2} \tanh\left(\frac{p}{2}\right) \leq \|F_U - F_{U'}\|_\infty \leq 1 - e^{-\frac{p}{2}} \leq \frac{p}{2}. \quad (5.170)$$

Recall that $p = 1 - e^{-q\theta\Delta}$. The right-hand inequality is a well-known property of the exponential function. Here, $F_U(\cdot)$ and $F_{U'}(\cdot)$ denote the cumulative distribution functions of the uniform distribution U on $[0, 1)$ and of U' . Furthermore, $\|\cdot\|_\infty$ denotes the supremum norm in $\mathbb{R}$. This means that

$$\|F_U - F_{U'}\|_\infty = \sup_{x \in [0, 1]} \{|F_U(x) - F_{U'}(x)|\}. \quad (5.171)$$

For our purposes the upper bound in (5.170) is relevant. To avoid misunderstanding, we point out that [Neue04], Section 4.2, applies an alternate definition of the geometric distribution, cf. par. 513.

Formula (5.169) can be used to generate values $u'_1, u'_2, \dots \in [0, 1)$ from the intermediate times $t_1, t_2, \dots$ between consecutive impulses of the Geiger counter. From the sequence $u'_1, u'_2, \dots$, finally k -bit **raw random numbers** $r_1, r_2, \dots$ are computed. The **random variables** $R_1, R_2, \dots$ are no longer **uniformly distributed** on $\{0, 1\}^k$ as would be the case for an ideal Geiger counter. 1260

If the **random variables** T_j are geometrically distributed with parameter $p_\theta = 1 - e^{-q\theta\Delta}$, **raw random numbers** R_j have the distribution π_p . For the moment, let $\vec{b} = (b_1, \dots, b_k) \in \{0, 1\}^k$ and $s(\vec{b})$ be the binary representation $(0.b_1 \dots b_k)_2 = \sum_{j=1}^k b_j 2^{-j}$. Formula (5.170) quantifies the 1261

deviation of the **random variables** R_j from the uniform distribution on $\{0, 1\}^k$

$$\begin{aligned} \pi_p(\vec{b}) &= \text{Prob}\left(U' \in [s(\vec{b}), s(\vec{b}) + 2^{-k})\right) = \sum_{\ell=0} \sum_{m=0} \text{Prob}\left(T_{2i} = \ell, T_{2i-1} = m, U'_i \in [s(\vec{b}), s(\vec{b}) + 2^{-k})\right) = \\ &= \sum_{\ell=0} \sum_{m=0} (1-p)^2 p^\ell p^m 1_{\{s(\vec{b}) \leq \frac{\ell+0.5}{m+\ell+1} < s(\vec{b}) + 2^{-k}\}}. \end{aligned} \quad (5.172)$$

From (5.170) and (5.172) we conclude that

$$\left| \pi_p(\vec{b}) - 2^{-k} \right| \leq p \quad \text{for all } \vec{b} \in \{0, 1\}^k \quad (5.173)$$

which provides an estimate for the **min-entropy**.

Note: More accurate analysis of the right-hand term in (5.172) may yield larger **entropy** bounds.

- 1262 This **PTRNG** is based on well-understood physical laws that, in particular, describe the number of radioactive decay events per time unit. The chain of reasoning that connects random events to the **entropy** of the generated random numbers contains model assumptions.
- 1263 Within the evaluation process, the applicant has to give evidence that the design under evaluation indeed fulfills the model assumptions that were specified in pars. 1256 and 1257. Depending on the concrete design, it may turn out that refinements or corrections of the **stochastic model** are necessary.
- 1264 Note: To be PTG.2-compliant, an appropriate **online test** and an appropriate **total failure test** need to be implemented.
- 1265 Note: Alternatively, **raw random numbers** can be derived from the number of impulses of the Geiger counter within time intervals $I_1, I_2, \dots$, with $I_j = [(j-1)s, js)$; cf. Subsection 5.4.3.

5.4.6 A PLL-based Physical Noise Source

- 1266 Subsection 5.4.6 briefly discusses a PLL-based design of a **physical noise source**. The design is described, central features are explained, and the **stochastic model** is addressed. For details we refer the interested reader primarily to [FBB+23; FiDr02; BeFV10; FiBB19].
- 1267 [PLL] Fig. 33 shows a (particular structure of) PLL. As usual, the acronym “PLL” stands for “phase-locked loop”.
- 1268 [PLL] The PLL in Fig. 33 divides the frequency f_{in} of the input signal clk_{in} by a factor N and the output frequency of the VCO (voltage controlled oscillator) by a factor C . The PFD (phase-frequency detector) compares the phase and the frequency of the input clock signal clk_{in} with the output signal clk_{out} of the PLL. More precisely, the PFD ensures that the output frequency $f_{out(VCO)}$ of the VCO, divided by M , equals f_{in} , divided by N . Furthermore, in Fig. 33 the acronyms “CP” and “LF” stand for the charge pump and loop filter. In the following, we may assume that the phase difference between clk_{in} and clk_{out} remains constant. The numbers

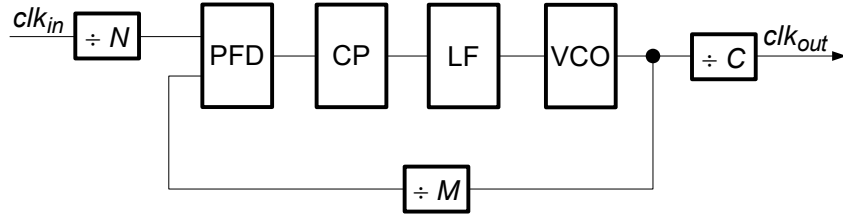


Figure 33: PLL (Phase-locked loop); source: [FiBB19], Fig. 2

N, M, C are integers. Altogether, this gives the relation

$$f_{out} = f_{in} \frac{M}{NC} = f_{in} \frac{K_M}{K_D}. \quad (5.174)$$

The terms $K_M, K_D \in \mathbb{N}$ denote the frequency multiplication and division factors of the PLL. We assume that $\gcd(K_M, K_D) = 1$ in the following, i.e., that K_M and K_D are relatively prime.

[PLL] The jitter of the input signal clk_{in} is intended to be kept as small as possible. This aim can be reached, for example, by using a low-jitter quartz oscillator. On the other hand, the PLL parameters (the bandwidth of the loop filter, for example) should be selected in a way such that the impact of the thermal noise on the output clock jitter is maximal. The VCO provides the main contribution to the jitter. 1269

[design] The PLL-based **physical noise source** discussed in this subsection uses a PLL as a source of randomness and a coherent sampling mechanism to convert the jitter of clk_1 into **raw random numbers**; see Fig. 34. In coherent sampling, both the sampled signal and the sampling signal are periodic signals with a known (fixed) frequency ratio. In the following, we assume that both signals are binary clock signals. The use of a PLL enables coherent sampling. 1270

[design] The lower part of Fig. 34 illustrates a design of a **physical noise source** that is based on one PLL. We note that PLL-based **physical noise sources** can use more than one PLL; cf. [FiBB19], Fig. 5, for example. In the following, we restrict our attention to the most elementary design with one PLL and refer the interested reader to the literature. The jittered clock signal clk_1 , i.e. the PLL output, is latched using a flip-flop (D-FF) on the rising edges of the reference clock signal clk_0 . The 1-bit counter outputs the parity of the number of sampled signals that equal 1 (XOR sum) within K_D cycles of the sampling signal clk_0 . This XOR sum provides a single binary-valued **raw random number** r_k . 1271

Note: We follow the notation in [FiBB19]. If the **physical noise source** uses only one PLL (as in Fig. 34), clk_0 (reference clock) and clk_1 (sampled clock signal) coincide with clk_{in} and clk_{out} (input and output clock of a PLL). In [FiBB19] the authors also consider designs that exploit more than one PLL, which motivates this notation.

[FPGA] PLLs are usually available on FPGAs, and PLLs are physically separated from the rest of the FPGA. This is a good feature for **physical noise sources**. Furthermore, PLLs are robust to environmental conditions. 1272

The PLL fixes the frequency relationship $f_1/f_0 = K_M/K_D$ and also ensures that the signals 1273

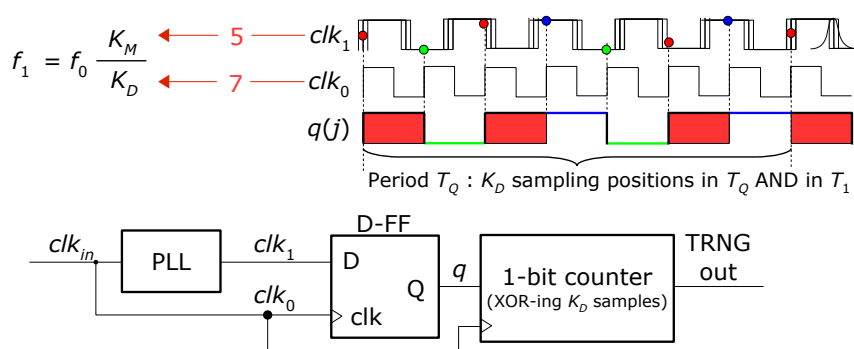


Figure 34: Sampling mechanism of a PLL-based **physical noise source** (previous version, schematic design); source: [FiBB19], Fig. 3

clk_{in} and clk_{out} are in phase. If the sampled clock signal clk_1 would be a regular signal with a fixed cycle length, i.e., if clk_1 (and clk_0) was jitter-free, the output sequence of the flip-flop would be K_D -periodic (in absolute time: K_D times the cycle length of clk_0). The sampled signal clk_1 is jittered so that the output sequence is only “almost” K_D -periodic, and the deviations induce the **entropy**. Also here, since the phase difference between clk_0 and clk_1 is (time-locally) constant, many output values of the flip-flop are always 0 or always 1 because the time period to the previous or to the next switch of clk_1 from 0 to 1 or from 1 to 0 is large compared to the jitter of clk_1 . Only the remaining output values of the flip-flop contribute to the **entropy** of the **raw random numbers**. The upper part of Fig. 34 illustrates this phenomenon by a toy example. The green circles and blue circles mark sampled values that are always 0 or always 1, respectively. Only the “random” red circles, which are close to the jittered clock edges, contribute to the **entropy** of the **raw random numbers**.

- 1274 Actually, both clk_0 and clk_1 are jittered. Since the PLLs (clock generators) are physically isolated (if more than one PLL is being used), it is supposed that their jitters are independent. Finally, the relative jitter between clk_0 and clk_1 is relevant. Consequently, from a logical point of view, the jitter of the reference clock signal clk_0 can be transferred to the sampled signal clk_1 . Thus, it is reasonable to consider a **stochastic model** where the reference clock signal is jitter-free.
- 1275 Within one “conversion period” (time interval needed to generate one raw random bit), the signal clk_1 is sampled K_D times on the rising edges of signal clk_0 . In particular, $\mu_1 = (K_D/K_M)\mu_0$, and a conversion period takes time $K_D\mu_0 = K_M\mu_1$. Here, μ_1 denotes the average cycle length of signal clk_1 and μ_0 the cycle length of clk_0 .
Note: To simplify reading, we refrain from an additional index k that labels the conversion period.
- 1276 [modified design] The design in Fig. 35 is a development of the design from Fig. 34. A significant improvement of the modified design in Fig. 35 is that in each conversion period the m -bit time-to-digital converter (TDC) calculates the integer sum of the output values x_i of the second flip-flop and not only its XOR sum. Since the outputted **raw random numbers** only depend on the least significant bit of the integer sum, this makes no difference to the design from Fig. 34. However, the integer sum provides more information for the evaluation of the **physical noise source**, and also the **online test** and the **total failure test** make use of this information, cf. par. 1286. A

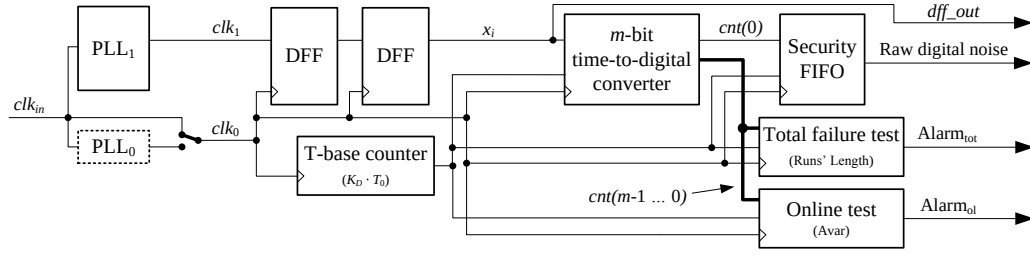


Figure 35: Sampling mechanism of a modified PLL-based **physical noise source** (schematic design) with **online test** and **total failure test**; source: [FBB+23], Fig. 3 (extract)

further improvement is that the output of the PLL₁ is sampled by two consecutive D flip-flops (DFFs) that are connected in series. The aim is to resolve from possible metastable events.

Note 1: The modified design principally allows the output of n PLL₁- clk_1 -signals that are delayed by regular phase shifts; cf. [FBB+23], Fig. 3. The intention is to increase the **entropy** rate. Below, we discuss the modified design from Fig. 35, which illustrates the case $n = 1$. It should be mentioned that the evaluation becomes more difficult for $n > 1$; cf. [FBB+23], Section 7.

Note 2: PLL₀ in Fig. 35 is optional. It can be omitted but most of the time the ranges of the coefficients K_M and K_D would be very limited and the **entropy** small. For these reasons, most of the time, PLL₀ is included.

Note 3: The aims and advantages of the modified design are summarized in [FBB+23], Subsection 2.2.

[**raw random numbers**] By $x_{i(j)}$ we mean the i^{th} output bit of the second D-flip-flop in the j^{th} conversion cycle, and $X_{i(j)}$ denotes the corresponding random variable. The modified design gives the designer freedom in the definition of **raw random numbers**. Principally, three options are possible:

$$r_j := \sum_{i=0}^{K_D-1} x_{i(j)} \pmod{2} \quad (5.175)$$

$$r'_j := \sum_{i=0}^{K_D-1} x_{i(j)} \quad (5.176)$$

$$\vec{r}''_j := (x_{0(j)}, \dots, x_{K_D-1(j)}) . \quad (5.177)$$

Clearly, $\vec{r}''_j$ contains more information than r'_j , and r'_j contains more information than r_j . So as not to make reading the paper [FBB+23] unnecessarily difficult like there we denote the r_j as **raw random numbers**. To prevent confusion, in this subsection we call $\vec{r}''_j$ and r'_j “pre-**raw random numbers**”.

Note: r'_j is the output of the integer counter (TDC) in conversion period j .

Due to the feedback mechanism of PLL₁ the random variables $X_{0(1)}, X_{1(1)}, \dots, X_{i(j)}, \dots$ are far from being **stationarily distributed**. On the other hand, we may assume that the pre-**raw random numbers** vectors $\vec{r}''_1, \vec{r}''_2, \dots \in \{0, 1\}^{K_D}$ can be interpreted as realizations of (time-locally)

stationarily distributed random vectors $\vec{R}_1'', \vec{R}_2'', \dots$; cf. [FBB+23], Assumption 1.

Note 1: If the random vectors $\vec{R}_j''$ are **stationarily distributed**, the random variables R_j' and finally also the random variables R_j are stationary, too, because R_j' is a function of $\vec{R}_j''$, as well as R_j is a function of R_j' ; cf. par. 587 a).

Note 2: Usually, the space $\{0, 1\}^{K_D}$ is very large, making rigorous statistical testing infeasible. Finally, it suffices to provide evidence that the random variables R_j , possibly also the random variables R_j' , are (time-locally) **stationarily distributed**. Reference [FBB+23] focuses on the analysis of the random variables R_j' and R_j ; cf. [FBB+23], Section 3.2.

Note 3: The fact that only some bits are contributing, favours practical analysis.

1279 Due to the feedback mechanism within a PLL, dependencies between the random vectors $\vec{R}_1'', \vec{R}_2'', \dots$ and thus between $R_1', R_2', \dots$ and between $R_1, R_2, \dots$ can occur. This aspect is considered in [FBB+23], Subsection 6.3.

1280 During each conversion period the signal clk_1 is latched K_D times. The sampled value x_i depends on the relative phase z_i within a cycle of clk_1 to clk_0 at the time of sampling. The value $\alpha \in [0, 1]$ denotes the duty cycle. (Usually, $\alpha \approx 0.5$.) If $z_i \in [0, \alpha\mu_1)$ (first half-period), the sampled value x_i is 1, and if $z_i \in [\alpha\mu_1, \mu_1)$ (second half-period), it equals 0. This observation suggests ordering the sampled values x_i with regard to the average relative phases at the sampling times. The average (expected) relative phase of the i^{th} sampled value equals $(\phi_0 + i \cdot \mu_0) \pmod{\mu_1}$. Here, ϕ_0 denotes the relative phase between clk_1 and clk_0 at the beginning of the conversion period. Regardless of the phase difference ϕ_0 , the average values of relative phases of K_D samples are uniformly distributed on $[0, \mu_1)$, where the distance between neighbored samples equals $\Delta := \mu_1/K_D$.

1281 Actually, the signal clk_1 is jittered, and thus the intervals between the neighbored relative phases are not perfectly identical but jittered. The phase of the clock signal is influenced by different sources of **entropy**. Thermal noise can be assumed to be independent from other sources of noise. Flicker noise is filtered out by the control loop of the PLL. Due to the properties of a PLL the jitter does not accumulate but remains bounded. In [BeFV10; FiBB19; FBB+23] the authors interpret the relative phase z_j of the j^{th} sampled value x_j as a realization of a **random variable** $Z_j \sim N(\mu_{(j)}, \sigma^2)$ with $\mu_{(j)} := \phi_0 + j \cdot \mu_0 \pmod{\mu_1}$.

Note: Due to the jitter of clk_1 , the **random variable** Z_j can assume values outside $[0, \mu_1)$.

1282 Since $\sigma \ll \mu_1$ it suffices to consider the nearest switch of a half-period of clk_1 . In [FiBB19], Eq. (5), respectively in [FBB+23], Proposition 2, the searched probability is calculated as

$$\text{Prob}(X_{(i(j))} = 1) = 1 - \frac{1}{\sqrt{2\pi}\sigma} \left(\int_{-\infty}^{\alpha\mu_1} e^{-\frac{(z-\mu_1)^2}{2\sigma^2}} dz - \int_0^{\mu_1/2} e^{-\frac{(z-\mu_1)^2}{2\sigma^2}} dz \right). \quad (5.178)$$

Note that $\text{Prob}(Z_{i(j)} < -\mu_1/2)$ and $\text{Prob}(Z_{i(j)} > 3\mu_1/2)$ are essentially 0 and need not be considered.

1283 As already pointed out in par. 1273, many (the majority) of the **random variables** $X_{(i(j))}$ assume the values 0 or 1 with probability ≈ 1 because $\sigma \ll \mu_1$. These sampled values hardly contribute to the **entropy** of the **random variables** R_j' and R_j , and thus, these $X_{(i(j))}$ are not considered in the following. The focus is on those **random variables** $X_{(i(j))}$ whose indices are in a subset $\mathcal{M} = \{i_1, \dots, i_s\} \subset \{0, \dots, K_D - 1\}$, for which $\text{Prob}(X_{(i(j))} = 1)$ differs significantly from 0 and 1. More precisely, [FBB+23] take those **random variables** $\text{Prob}(X_{(i(j))} = 1)$ with $0.02275 \leq$

$\text{Prob}(X_{i(j)} = 1) \leq 0.97725$ into account. These bits $x_{i(j)}$ and the corresponding random variables $X_{i(j)}$ are called *contributors* or *contributing bits*. Only these **random variables** $X_{i(j)}$ *contribute* significantly to the **entropy** of R'_j and R_j , justifying the naming. It is

$$R'_j = \sum_{i=0}^{K_D-1} X_{i(j)} \approx X_{i_1(j)} + \cdots + X_{i_s(j)} + \sum_{\{0 \leq i < K_D \mid \text{Prob}(X_{i(j)}=1) > 0.97725\}} 1. \quad (5.179)$$

The “ $\approx$ ” sign in (5.179) results from the fact that **random variables** $X_{i(j)}$ with $\text{Prob}(X_{i(j)} = 1) > 0.97725$ can assume 0 and the **random variables** $X_{i(j)}$ with $\text{Prob}(X_{i(j)} = 1) < 0.02275$ can assume 1 (although with small probabilities).

Note 1: The XOR sum of the non-contributing bits $x_{i(j)}$ can be 0 or 1, thereby affecting the value of the **raw random number** r_j . As already mentioned, these bits have little impact on the **entropy** of the **raw random numbers**. In particular, if the clk_1 (and clk_0) signal would be jitter-free, the generated **raw random numbers** would be constant 0 or 1, depending on the number of sampled values equal to one.

Note 2: When Δ decreases, more sample values will contribute to the **entropy** of the **raw random numbers** and the **entropy** will be increased.

Reference [FBB+23] investigates thoroughly, which **random variables** $X_{i(j)}$ are contributors. 1284
Ideally, contributing bits should be “far away” in the time domain, i.e., the differences of their indices should be “large”, so that they are at most weakly correlated. This is a criterion for the choice of suitable parameters K_D and K_M ; cf. [FBB+23], Subsection 3.3. Table 13 in [FBB+23] exemplary provides correlations between contributing bits. Table 13(a) belongs to a good configuration, Table 13(b) to a bad configuration.

For good configurations the contributing bits are “almost” independent, which simplifies the (ap- 1285
proximate) calculation, e.g., of the probability $\text{Prob}(X_{i_1(j)} + \cdots + X_{i_s(j)} \pmod{2})$ (independence assumption). Since the non-contributing bits are constant with large probability, [FBB+23] extends the independence assumption to $\text{Prob}(X_{0(j)} + \cdots + X_{K_D-1(j)} \pmod{2})$; cf. [FBB+23], Proposition 3.

The **online test** and the **total failure test** exploit the counter values and not only the **raw random 1286
numbers**. As already mentioned above, the counter values provide more information than their parities. In [FBB+23] the proposed **online test** computes the Allan variance on the basis of 4096 counter values. (The parameter 4096 can be adjusted.) The rejection boundary of the **online test** is determined by worst case analysis. The **total failure test** exploits runs of identical, subsequent counter values. The paper includes a latency analysis of the **total failure test**. For details the interested reader is referred to [FBB+23], Section 4.1 and Section 4.2.

For further details the interested reader is referred to [FBB+23]. 1287

5.5 Online Tests

In Section 5.5, different **online test schemes** are discussed; aims, advantages, and disadvantages 1288
are explained. The general considerations from Subsection 4.5.3 are supported by examples.

Unless otherwise stated, in this section we assume that binary-valued **raw random numbers** $r_1, r_2, \dots$ are tested. Furthermore, $c = \xi(r_1, \dots, r_m)$ denotes the test value, ξ is the applied statistical test, and m is the sample size of the statistical test ξ in bits.

- 1290 Consequently, we interpret the test value as a realization of the **random variable** $C = \xi(R_1, \dots, R_m)$. It is a relevant part of the evaluation to understand the distribution of C under the admissible parameters of the **stochastic model**.
- 1291 As explained in Subsection 4.5.3 (par. 828), the **online test** shall be selected with regard to the **stochastic model** of the **noise source**. In Section 5.5, we tacitly assume that the **online test**, or more precisely, the applied statistical test(s), is appropriate for the **stochastic model**. Instead, we focus on the suitability of the whole **online test procedure**, including its evaluation rules.

5.5.1 A Look at Single Statistical Tests

- 1292 In this subsection, we focus on single statistical (online) tests. A negative example is analyzed and desirable properties are discussed. The results motivate the design of more sophisticated test suites.
- 1293 [χ^2 goodness-of-fit test] If a χ^2 goodness-of-fit test on 4-bit words (a.k.a. poker test, often briefly denoted as χ^2 -test) is applied to the **raw random numbers**, the sequence $r_1, r_2, \dots, r_m$ is divided into non-overlapping 4-tuples $w_1, \dots, w_{m/4}$ where $w_j = (r_{4j-3}, r_{4j-2}, r_{4j-1}, r_{4j})$. For $i = 0, \dots, 15$ the term $fr(i) := |\{j \leq n \mid w_j = i\}|$ denotes the frequency of the 4-tuple i . We identify the 4-bit vector w_i with the binary representation of an integer. The test value is given by

$$c := \sum_{i=0}^{15} \frac{\left(fr(i) - \frac{m}{16}\right)^2}{\frac{m}{16}} . \quad (5.180)$$

Note: The χ^2 -test in (5.180) corresponds to a scenario where the null hypothesis says that the tested **raw random numbers** are generated by an ideal **RNG**.

- 1294 Negative Example ([Schi01], Example 2): The **online test** applies the χ^2 goodness-of-fit test from par. 1293 with sample size $m = 320$. The **online test** fails if the test value c exceeds 65.0. It is claimed that $\text{Prob}(C > 65.0) = 3.4 \cdot 10^{-8}$ for **ideal RNGs**, i.e., for **iid** $B(1, 0.5)$ -distributed **random variables** R_j .
- 1295 The example in par. 1294 shows several problems that can occur with **online tests**. We follow and extend the analysis and the conclusions in [Schi01].
- 1296 The distribution of the test variable C of the poker test (5.180) converges to the χ^2 -distribution with 15 degrees of freedom as the sample size $m \rightarrow \infty$. This is the justification for the significance level claim $3.4 \cdot 10^{-8}$ from par. 1294. However, $C > 65.0$ is a very rare event, at least if the tested **raw random numbers** are “almost” ideal. In general, at the tails of a distribution the rate of convergence can be low.

1297

[relative approximation error] Generally, when computing rejection probabilities from limit distributions, the relative approximation error

$$\frac{|p_{exact} - p_{approx}|}{|p_{approx}|} \quad (\text{relative error}) \quad (5.181)$$

is important. In (5.181), p_{exact} denotes the exact rejection probability, while p_{approx} is the approximate rejection probability given by the limit distribution (here: χ^2 -distribution with 15 degrees of freedom).

Note: We use p_{approx} instead of p_{exact} in the denominator because the designer of the PTRNG bases further considerations on p_{approx} .

[relative approximation error] In the example from par. 1294, the sample size $m = 320$ is rather small. In fact, for ideal RNGs the relative approximation error is 10.1. (Exploiting the symmetries allows the calculation of the exact rejection probability.) In this case, the developer would have underestimated the significance level of the online test (i.e., the number of (undesired) noise alarms under the null hypothesis) by a factor of more than 10. This may primarily affect the availability of the PTRNG, but for other statistical tests, the approximation error might swing in the opposite direction, leading to a significant overestimation of the significance level, which definitely would be a security issue. 1298

The relative approximation error should decrease if the sample size m increases; in the above example, e.g., for $m = 512$ or $m = 1024$, which are more typical sample sizes for poker tests. However, the considerations from pars. 1296 to 1298 point to a general problem when using limiting distributions. 1299

A further disadvantage of the online test from par. 1294 is that it is hardly feasible to estimate the true rejection probability if the distribution of the raw random numbers deviates from the output of an ideal RNG (due to a bias and dependencies). However, this is important for the assessment of the suitability of an online test. 1300

Note: A suitable online test shall reliably separate the sets of parameters A_{real} and A_{bad} .

If the developer cannot (at least approximately) determine the failure probabilities for the relevant parameters in A_{real} and A_{bad} , there is a lack of evidence whether Requirement PTG.2.4 (resp. PTG.3.7) is indeed fulfilled, i.e., whether the online test is effective. Then, the PTRNG cannot be certified as PTG.2- or PTG.3-compliant. 1301

Remark: For iid stochastic models, a monobit test can be applied. Then the Central Limit Theorem provides approximate rejection probabilities of the monobit test for iid $B(1, p)$ -distributed random variables, and (4.49) provides an upper bound for the approximation error. In principle, this would solve the problems addressed in pars. 1300 to 1301. 1302

However, the upper bound (4.49) converges in the order of $n^{-0.5}$, which means that the sample size of the monobit test had to be very large if extremely small rejection probabilities are concerned. Furthermore, the developer had to show that the proposed monobit test (with specified sample size m and evaluation rules) is sufficiently discriminating between A_{real} and A_{bad} . 1303

These problems motivate the search for more sophisticated online test schemes that apply more complex evaluation rules than just considering individual tests; cf. Subsection 5.5.2. 1304

- 1305 The key is to analyze the distribution of the test variable C for different parameters. The expectation $E(C)$, the variance $\text{Var}(C)$, and the standard deviation $\sigma_C := \sqrt{\text{Var}(C)}$ as well as the rejection probabilities $\text{Prob}(C \in E)$ (e.g., $\text{Prob}(C > x)$) can easily be estimated by *stochastic simulations*. The rejection probabilities $\text{Prob}(C_j \in E)$ can be small on A_{real} (let's say, $\in [10^{-4}, 10^{-2}]$) but should not be tiny (let's say, not $< 10^{-6}$) since otherwise, the required sample size for (trustworthy) stochastic simulations would “explode”.
- 1306 For stochastic simulations, it is not necessary to use a **DRNG** that is suitable for cryptographic applications. Instead, one can use a linear congruential generator or an **LFSR** since both types of **pseudorandom number generators** are very fast, and their statistical properties are suitable for this purpose (cf. [Schi09a], Subsection 2.4.3).
- 1307 From so-called standard random numbers $z_1, z_2, \dots \in [0, 1)$ (generated, e.g., by a linear congruential generator or an **LFSR**), one generates sequences of pseudorandom bits for different parameters. $B(1, p)$ -distributed random numbers, for example, can be obtained by $r_j := 1_{\{z_j \leq p\}}$. The simulated standard random numbers are assumed to be **uniformly distributed** on $[0, 1)$. The specified test (e.g., a χ^2 goodness-of-fit test on 4-bit words or a monobit test) is applied to the simulated **raw random numbers**. Finally, these empirical values yield estimates for $E(C)$, $\text{Var}(C)$, σ_C , and $\text{Prob}(C \in E)$.
- 1308 Table 10 provides exemplary results for the above-mentioned χ^2 goodness-of-fit test on 4-bit words. For simplicity, only **iid random variables** $R_1, R_2, \dots$ are considered. However, it is easy to simulate other distributions of the **raw random numbers** (e.g., for Markovian models) as well. It might be noted that for $p = 0.500$ for each sample size m , the χ^2 -approximation yields $E(C) = 15.0$ and $\text{Var}(C) = 5.477$. Table 10 and Table 11 collect simulation results for different parameter values and sample sizes,

Table 10: χ^2 goodness-of-fit test on 4-bit words: simulation results for **iid raw random numbers** ($R_j \sim B(1, p)$); $N_s = 10^6$ or 2^{20}

criteria	$p = 0.500$	$p = 0.503$	$p = 0.507$	$p = 0.510$	$p = 0.520$
$m = 512$					
$E(C)$	15.01	15.02	15.10	15.20	15.81
σ_C	5.46	5.46	5.49	5.53	5.76
$\text{Prob}(C > 34.0)$	0.0036	0.0035	0.0038	0.0041	0.0062
$m = 1024$					
$E(C)$	15.00	15.04	15.19	15.41	16.64
σ_C	5.46	5.48	5.53	5.62	6.05
$\text{Prob}(C > 34.0)$	0.0035	0.0036	0.0040	0.0045	0.0098
$m = 2^{20}$					
$E(C)$	15.01	52.79	229.43	434.9	1696.69
σ_C	5.47	13.4	29.24	41.4	82.53
$\text{Prob}(C > 150.0)$	0.0000	0.0000	0.9955	1.000	1.000

- 1309 The figures in Table 10 show that single χ^2 goodness-of-fit tests on 4-bit words reliably separate different parameters if the sample size m is extremely large. Of course, $m = 2^{20} = 1,048,576$ is

Table 11: χ^2 goodness-of-fit test on 4-bit words: simulation results for iid raw random numbers ($R_j \sim B(1, p)$); $N_s = 10^6$

criteria	$p = 0.570$	$p = 0.560$	$p = 0.540$	$p = 0.530$	$p = 0.520$
$m = 512$					
$E(C)$	25.23	22.47	18.20	16.84	15.81
σ_C	8.74	7.93	6.63	6.13	5.76
$\text{Prob}(C > 34.0)$	0.1489	0.0825	0.0215	0.0110	0.0062
$m = 1024$					
$E(C)$	35.60	30.00	21.60	18.69	16.64
σ_C	11.06	9.77	7.59	6.73	6.05
$\text{Prob}(C > 34.0)$	0.5156	0.3065	0.0645	0.0248	0.0098
$m = 2^{16}$					
$E(C)$	1337.71	979.17	438.51	252.17	120.12
σ_C	76.48	64.52	42.19	31.52	21.30
$\text{Prob}(C > 600)$	1.0000	1.0000	0.0002	0.0000	0.0000

by far too large for a single online test, but it comes into question for the test mode after a noise alarm has occurred or as an additional criterion for online test suites; cf. pars. 838, 1342, 1343, and 1323.

Table 11 underlines that if the parameters differ more significantly, much smaller sample sizes m suffice to distinguish these parameters. While Table 10 primarily concerns PTRNG designs where the raw random numbers have enough entropy, Table 11 applies to designs that need data-compressing algorithmic post-processing, e.g., XORing non-overlapping pairs of raw random number bits. In the second scenario, the raw random numbers may show significant weaknesses. This facilitates the efforts of designing efficient tests.

Assume that non-overlapping pairs of the iid raw random numbers are XORed. For $p = 0.44$ or $p = 0.47$, for example, the internal random numbers are $B(1, p')$ -distributed with $p' = 0.5072$ or $p' = 0.5018$, respectively.

5.5.2 A More Sophisticated Online Test Procedure

In this subsection, we provide an *example* of a generic approach that mitigates several problems mentioned in Subsection 5.5.1. The considerations are based on [Schi01] but also go beyond. Pre-versions are explained in [AIS31An_01], Example E.7, and [AIS2031An_11], Subsection 5.5.3. Note: The requirements of online tests have increased since then (mainly because of the increased entropy requirements).

The central idea is not just to apply independent, single statistical tests but to combine the information from several statistical tests by suitable evaluation rules.

Note: The online test schemes discussed in this subsection are exemplary. It is *not claimed* that it is optimal for each design. Other online test schemes are permitted, too, of course. The

suitability of an **online test scheme** depends on the concrete scenario; cf. par. 1354.

1315 First, the developer selects a so-called “basic test” that is tailored to the **stochastic model**. For the remainder of this subsection, this selection will not be discussed. We assume that the choice of the basic test is appropriate for the **PTRNG** design. As in Subsection 5.5.1, the letter m denotes the sample size of the basic test in bits.

1316 [start-up test] The basic test may also serve as start-up test (single application together with an appropriate rejection boundary, such that an ideal **RNG** would fail, let’s say, with probability $\approx 10^{-8}$ or 10^{-7}). The start-up test shall detect any total breakdown of the **noise source** and significant statistical weaknesses immediately when the **PTRNG** is started. The start-up test then fulfills functional Requirements PTG.2.3 and PTG.3.6.

1317 Let $C_1, C_2, \dots$ denote the **random variables** that correspond to the test values $c_1, c_2, \dots$ of the basic tests within an **online test** suite. Furthermore, $H_0, H_1, \dots$ are the so-called “history variables”. We set

$$H_0 := E(C_{1;\text{IRNG},t}) \quad (5.182)$$

where $E(C_{1;\text{IRNG},t})$ denotes the expectation of the test variable C_1 for an **ideal RNG**, rounded to a multiple of 2^{-t} . Moreover, we define the recursion

$$H_j := (1 - \beta) H_{j-1} + \beta C_j \quad \text{for } j = 1, 2, \dots \quad \text{with } \beta = 2^{-s} \quad (5.183)$$

where the H_j are rounded to t binary digits after the binary point. This allows the calculation of the “history values” $h_0, h_1, \dots$ with integer arithmetic.

1318 [**online test** suite] In Step j of the **online test** suite, a basic test is applied ($\rightarrow$ test value c_j), and three rules are evaluated unless a **noise alarm** has already occurred within this **online test** suite. The **online test** suite consists of N basic tests unless a **noise alarm** has occurred before. If no **noise alarm** has occurred during the N basic tests, a final test is applied. The evaluation rules (ER 1a), (ER 1b), (ER 2), and (ER 3) quantify these criteria.

(ER 1a) $c_j \in E_{1a} \Rightarrow$ **noise alarm**

(ER 1b) $c_{j-k+1}, \dots, c_{j-1}, c_j \in E_{1b} \Rightarrow$ **noise alarm**

(ER 2) $h_j := (1 - \beta) h_{j-1} + \beta c_j \in E_2 \Rightarrow$ **noise alarm**

(ER 3) at the end of the **online test** suite, if no **noise alarm** has occurred:

Apply the basic test to all **raw random numbers** that were tested within this **online test** suite (test value c_{total}).

$c_{total} \in E_3 \Rightarrow$ **noise alarm**

The parameter $k > 1$ is a small integer. A **noise alarm** terminates the current **online test** suite. If not interrupted by a **noise alarm**, an **online test** suite consists of N basic tests. If the **online test** suite has been terminated earlier due to a **noise alarm**, evaluation criteria (ER 3) is not applied. The evaluation rules (ER 1a), (ER 1b), (ER 2), and (ER 3) cover different aims. This topic will be taken up later. Possible consequences of a **noise alarm** are explained in pars. 838 to 840.

1319

Unless the **raw random numbers** generated by the **PTRNG** under consideration show long-term dependencies, we may assume that the random test variables $C_1, C_2, \dots$ are **iid**.

In par. 1318, the different evaluation criteria serve different aims; cf. par. 1326. 1320

Note: The probabilities $\text{Prob}(C_j \in E_1)$ can be estimated by stochastic simulations for relevant parameters; cf. pars. 1305 to 1311.

[**Time-local stationarity**] We assume that the **raw random numbers** can be viewed as **stationarily distributed** within an **online test** suite. 1321

The proposed **online test scheme** from par. 1318 is generic. To make the ideas more concrete, in the remainder of this subsection, we assume, as an example, that the basic test is a χ^2 goodness-of-fit test on 4-bit words. The sample size is m bits, or equivalently, $m/4$ four-bit words. 1322

Note 1: The following considerations can be transferred to any other basic test.

Note 2: (Reminder) Universally suitable **online tests** do not exist. The **online test** shall be tailored to the **physical noise source**. In Subsection 5.4.3, for example, the **online test** considers the expectation and the variance of integer-valued **raw random numbers**.

[**online test** suite: (special case: basic test = χ^2 goodness-of-fit test)] 16 local counters $fr(0)$ to $fr(15)$ count the number of 4-bit words within a basic test that equal $0, 1, \dots, 15$ (interpreted as the binary representation of integers). At the beginning of each **online test** suite, global counters are initialized: $tot_{fr}(0) = \dots = tot_{fr}(15) = 0$. 1323

In Step j of the **online test** suite, a basic test is applied ($\rightarrow$ test value c_j), the evaluation rules (ER 1a), (ER 1b), and (ER 2) are applied, and the global counters $tot_{fr}(0)$ to $tot_{fr}(15)$ are updated ($tot_{fr}(j) = tot_{fr}(j) + fr(j)$). A **noise alarm** terminates the **online test** suite immediately. Unless the **online test** suite has been terminated by a **noise alarm**, the final evaluation criterion (ER 3) is applied.

(ER 1a) $c_j > x_a \Rightarrow$ **noise alarm**

(ER 1b) $c_{j-k+1}, \dots, c_{j-1}, c_j > x_b \Rightarrow$ **noise alarm**

(ER 2) $h_j := (1 - \beta) h_{j-1} + \beta c_j \notin [u, v] \Rightarrow$ **noise alarm**

(ER 3) for $i = 0$ to 15 do $tot_{fr}(i) := tot_{fr}(i) + fr(j)(i)$
After all N basic tests have been evaluated:
 $c_{total} > x_{total} \Rightarrow$ **noise alarm**

[evaluation criteria] As already mentioned above, the four evaluation criteria (ER 1a), (ER 1b), (ER 2), and (ER 3) serve different purposes. The sample size of the basic tests and thus, their discriminatory power, is usually much smaller than that of typical evaluator tests. Evaluation rule (ER 3) is applied at most once per **online test** suite. Due to its large sample size, the discriminatory power of this final test is very large. The extra costs are limited to 16 additional integer counters $tot_{fr}(0)$ to $tot_{fr}(15)$ and the computation of one χ^2 -test value. 1324

Generally, one can expect that aging effects or changing environmental conditions (apart from attacks) change the **entropy** of the generated **raw random numbers** slowly. As will become clear 1325

below, such phenomena are reliably detected by (ER 3). Problems can arise shortly after start-up if the PTRNG in operation behaves very differently from typical specimen of the same series (and, of course, within targeted attacks although they are not in the scope of the online test). The evaluation criteria (ER 1a,b), (ER 2), and (ER 3) serve different goals.

1326 [aims of the evaluation criteria] The aim of the evaluation criteria (ER 1a) and (ER 1b) are to detect rapidly developing, significant weaknesses of the raw random numbers, which have to be detected very quickly. Evaluation rule (ER 2) mitigates the problem of the small sample size of a basic test to some degree, since the history values $h_1, h_2, \dots$ are sensitive to deviations with regard to the “true” expectation $E(C_j)$ (depending on the true parameters). The task of evaluation criterion (ER 2) is to detect smaller (but non-acceptable) weaknesses that are also rapidly developing. Finally, due to its large sample size, the aim of (ER 3) is the reliable (and sufficiently fast) detection of slowly developing small weaknesses, i.e., when the parameters (slowly) leave the set A_{good} .

1327 [total failure test] In principle, the online test schemes specified in pars. 1318 and 1323 can also include an evaluation rule that fulfills the requirements of a total failure test. This feature was discussed in [Schi01] (cf. [AIS31An_01], Example E.7, and [AIS2031An_11], Subsection 5.5.3) under the assumption that a total failure implies a constant sequence of raw random number bits. For example, a further evaluation rule could be added to par 1323:

$$(T) \quad c_j \geq 269.5 \Rightarrow \text{noise alarm (total failure)}$$

1328 [total failure test] The disadvantage of the approach sketched in par. 1327 is that the current basic test may not detect a total failure if it occurs too late within the test sample. If the last 220 bits of a test sample ($m = 512$) are constant 0 or constant 1, then $c_j \geq 269.5$, which triggers a noise alarm due to decision rule (T); cf. [Schi01]. This means that the detection of a total failure could be delayed by $219 + 512 = 731$ raw random number bits in the worst-case. Hence, the PTRNG design had to provide a large buffer for the internal random numbers. Specially designed total failure tests usually have much smaller delay times and thus are preferable in most cases.

1329 Compared to the online test suite discussed in [Schi01], the proposed procedures from pars. 1318 and 1323 lack a total failure test. Instead, the evaluation criteria (ER 1a) and (ER 3) have been added.

1330 Compared to online test schemes that apply independent statistical tests, the proposed online test scheme has several advantages. First of all, it is feasible to estimate the probabilities of noise alarms for any distribution of the raw random numbers. Secondly, there is a whole set of parameters $(m, N, k, t, \beta, E_{1a}, E_{1b}, E_2, E_3)$, resp. $(m, N, k, t, \beta, x_a, x_b, u, v, x_{total})$, which allow “fine-tuning”, i.e., the optimization of the online test scheme under consideration of the PTRNG design.

Note: The parameters x, u, v are specific for the selected χ^2 -test.

1331 Evaluation rule (ER 2) defines a random walk on $\{u, u + 2^{-t}, \dots, v\}$. Without rounding the history variables to a multiple of 2^{-t} , the expectations $E(H_j)$ would tend to $E(C_1)$ as $j \rightarrow \infty$.

If $E(C_1) \notin [u, v]$, the history variables should “drift out” of $[u, v]$ rather soon, causing a **noise alarm**. Even if $E(C_1) \in [u, v]$, a large distance of $E(C_1)$ from $E(C_{1;IRNG,t})$ should increase the probability that an absorbing barrier is reached earlier (provided, of course, that u and v have been properly selected).

By par. 1319 the evaluation rules (ER 1a,b) and (ER 2) in par. 1323 can be described by a homogeneous Markov chain on the finite state space 1332

$$\Omega = \{(2^{-t}\ell, i) \mid \ell \in \mathbb{N}, 2^{-t}\ell \in [u, v], 0 \leq i < k\} \cup \{\omega\} \quad (5.184)$$

with transition matrix P . Recall that $k > 1$ is a small integer. Furthermore, u and v are multiples of 2^{-t} , while ω is an absorbing state. The **online test scheme** reaches state (s, i) after step j if the history variable $h_j = s$ and if $c_{j-i} \leq x$ but $c_{j-i+1}, \dots, c_j > x$ (or if $j = i < k$). The absorbing state ω is reached if a **noise alarm** has occurred within the first j steps; see [Schi01] for details.

The state space Ω consists of $((v - u)2^t + 1)k + 1$ elements. The initial distribution ν_0 has total mass on the state $(E(C_{1;IRNG,t}), 0)$. Then $\nu_j(\omega) = \nu_0 P^j(\omega)$ equals the probability that a **noise alarm** has occurred within the first j steps. 1333

The probability $\text{Prob}(C_j > x)$ is estimated, and thus the transition matrix P is determined on the basis of stochastic simulations. For each relevant distribution, the **raw random numbers** are simulated and basic tests are performed. This provides the empirical cumulative distribution function of the **random variables** C_j . 1334

A small weight factor β on evaluation rule (ER 2) ensures that an “extreme” value of the history variable h_j is not caused by a single, very rare event but on several events that, taken individually, need not be rare. If a smaller $\beta = 2^{-s}$ is selected, the history values $h_0, h_1, \dots$ become more inert. Reasonable values seem to be $s = 4, 5, 6$. 1335

Note: The choice of the precision 2^{-t} also has impact on the probabilities for a **noise alarm**. 1336

[(ER 1b)] As explained above, the probability for a **noise alarm** within an **online test** suite can be calculated for any set of parameters. To support a targeted search for appropriate parameters, we consider the question about how many basic tests are needed on average until k successive failures occur (each with a probability $p \in (0, 1)$). For $j = 0, \dots, k$, let $e_k(j)$ denote the expected number of basic tests until k successive failures occur under the condition A_j that the j previous basic tests failed. If $j < k$, condition A_j implies condition A_0 in the next step with probability $1 - p$ and condition A_{j+1} with probability p . Condition A_k is the terminating condition. This leads to the following linear equations. 1337

$$e_k(0) = (1 - p)(e_k(0) + 1) + p(e_k(1) + 1) \iff pe_k(0) - pe_k(1) = 1 \quad (5.185)$$

$$e_k(j) = (1 - p)(e_k(0) + 1) + p(e_k(j + 1) + 1) \iff (p - 1)e_k(0) + e_k(j) - pe_k(j + 1) = 1 \quad (5.186)$$

for $j = 1, \dots, k - 1$

$$e_k(k) = 0 \quad (5.187)$$

The solution of the linear equations, in particular, yields $e_k(0)$, the value we are interested in.

[(ER 1b), Example] For $k = 4$, we obtain the linear equations

$$\begin{pmatrix} p & -p & 0 & 0 & 0 \\ p-1 & 1 & -p & 0 & 0 \\ p-1 & 0 & 1 & -p & 0 \\ p-1 & 0 & 0 & 1 & -p \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} e_4(0) \\ e_4(1) \\ e_4(2) \\ e_4(3) \\ e_4(4) \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \end{pmatrix}. \quad (5.188)$$

In particular, $e_4(0) = \frac{p^3+p^2+p+1}{p^4}$. Thus, $e_4(0) \approx \frac{1}{p^4}$ for small p .

1339 [(ER 1b), Example] For $k = 5$, we obtain the linear equations

$$\begin{pmatrix} p & -p & 0 & 0 & 0 & 0 \\ p-1 & 1 & -p & 0 & 0 & 0 \\ p-1 & 0 & 1 & -p & 0 & 0 \\ p-1 & 0 & 0 & 1 & -p & 0 \\ p-1 & 0 & 0 & 0 & 1 & -p \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} e_5(0) \\ e_5(1) \\ e_5(2) \\ e_5(3) \\ e_5(4) \\ e_5(5) \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \end{pmatrix}. \quad (5.189)$$

In particular, $e_5(0) = \frac{p^4+p^3+p^2+p+1}{p^5}$. Thus, $e_5(0) \approx \frac{1}{p^5}$ for small p .

1340 **PTRNGs** often generate much better **raw random numbers** than required by PTG.2.2 and PTG.3.5. As explained in Subsection 4.5.3, pars. 810 to 817, this eases the design of an effective and efficient **online test**. The probability for an (undesirable) **noise alarm** for the “very acceptable” parameters A_{real} should be small, while it should be large for inappropriate parameters A_{bad} . For the remaining “in-between” distributions $A_{good} \setminus A_{real}$, the probability for a **noise alarm** is not relevant.

1341 [Quality assessment] The suitability of an **online test** primarily depends on the **stochastic model** but also on the parameter sets A_{real} and A_{bad} . Other aspects are, for example, the test strategy and the output rate of the **PTRNG**, which affects the number of accidental **noise alarms**. For simplicity, we assume an **iid** model as an example for the remainder of this section. The general procedure would be the same, e.g., for a Markovian **stochastic model**.

1342 [consequences of a **noise alarm**] In Subsection 4.5.3, several options for the consequences of a **noise alarm** were addressed; cf. pars. 838 to 840. One of these options is to trigger a test mode (without outputting any random numbers, i.e., an “emergency test”) in order to check whether the **noise alarm** was justified or accidental. In the following, we focus on this possibility. The “emergency test” can be a single basic test but with much larger sample size.

1343 [Example: emergency test] A reasonable strategy is to apply the basic test to fresh Nm **raw random numbers** bits (the sample size of evaluation criterion (ER 3)). The decision rule can be the same as for (ER 3), but another decision rule can be selected as well. If the emergency test fails, the **noise alarm** is confirmed; otherwise, considered erroneous.

Natural requirement: If the **raw random numbers** belong to $\in A_{bad}$. The emergency test should detect this with overwhelming probability (cf. Tab. 10 and Tab. 11). On the other hand, if the true parameter(s) are in A_{real} , a failure of the emergency test should be very unlikely.

1344

[Scenario I, **iid stochastic model**] Assume that the designer is convinced that $\text{Prob}(R_j = 1) \in (0.497, 0.503)$ for all properly working specimens of this **PTRNG** design (e.g., for the **PTRNGs** on the chips of some product series). This is significantly better than demanded by Requirement PTG.2.2 (resp. by PTG.3.5), and this feature supports the design of an effective **online test**. In the notation of Subsection 4.5.3, this means that $A_{\text{real}(I)} = [0.497, 0.503]$ and $A_{\text{bad}(I)} = [0, 0.4931] \cup (0.5069, 1]$. For the parameters in $A_{\text{good}(1)} = [0.0, 1.1] \setminus A_{\text{bad}(I)}$, both the **Shannon entropy** and **min-entropy** of the corresponding distributions exceed the bounds that are specified in PTG.2.2 (0.9998 for **Shannon entropy** and 0.98 for **min-entropy**). Thus, a **post-processing** algorithm is possible but not necessary.

[Scenario II, **iid stochastic model**] As in par. 1344 we assume an **iid** model but here $A_{\text{real}(II)} = [0.470, 0.530]$ and $A_{\text{bad}(II)} = [0, 0.441] \cup (0.559, 1]$. Thus, a data-compressing **post-processing** algorithm is required. In the following, we assume that the **algorithmic post-processing** algorithm XORs non-overlapping pairs of consecutive **raw random number** bits. 1345

[Example: Scenario I, test parameters] $(m, N, k, \beta, t, x_a, x_b, u, v, x_{\text{total}}) = (1024, 1024, 5, 1/32, 5, 75.0, 34.0, 11.0, 20.5, 150.0)$. If a **noise alarm** has occurred, the **PTRNG** goes into the test mode (cf. par. 1343) and performs an emergency test. The tested **raw random numbers** are not output. As for the evaluation rule (ER 3), the emergency test applies the χ^2 -test to $2^{20} = 1,048,576$ bits, the sample size of an **online test** suite. The emergency test fails if the test value is > 150.0 . If the emergency test is passed, the **PTRNG** returns to the working mode (outputting random numbers). 1346

[Example: Scenario I, numerical values] Tab. 12 collects numerical results that illustrate the properties of the test parameters that were selected in par. 1346. Recall that 0.503 limits A_{real} , while (approximately) $p = 0.507$ defines the limit line between A_{good} and A_{bad} . The values in Tab. 12 are computed on the basis of the simulated cumulative distribution function of the test variable C_1 under the particular distributions of the **raw random numbers**, cf. par. 1305. Criterion (ER 1a) detects, with a probability of almost 1 (of > 0.9 , of > 0.5) if $p \in [0, 0.34] \cup$ 1347

Table 12: Scenario I: The first row provides the average number of basic tests until 5 consecutive basic test values exceed 34.0 (event A_5). The second row quantifies the probability that a **noise alarm** is triggered by evaluation criterion (ER 1a,b) or (ER 2), while the third row contains the probabilities that the (ER 3) causes a **noise alarm**. Both row 2 and row 3 refer to a single **online test** suite.

p=	0.500	0.503	0.507	0.525	0.530	0.535	0.560	0.570
$E(\# \text{ basic tests until } A_5 \text{ occurs})$	$1.9 \cdot 10^{12}$	$1.7 \cdot 10^{12}$	$9.8 \cdot 10^{11}$	$1.2 \cdot 10^9$	$1.1 \cdot 10^8$	$8.8 \cdot 10^6$	532	55
Prob(noise alarm by (ER 1a,b) or (ER 2))	0.00000	0.00000	0.00000	0.05856	0.85597	1.00000	1.00000	1.00000
Prob(noise alarm by (ER 3))	0.00000	0.00000	0.99552	1.00000	1.00000	1.00000	1.00000	1.00000

$[0.66, 1]$ (if $p \in [0, 0.36] \cup [0.64, 1]$, if $p \in [0, 0.38] \cup [0.62, 1]$), while the probability is essentially 0

if $p \in [0.43, 0.57]$. If $p \in [0.43, 0.57] \cap A_{bad}$ the other criteria apply.

- 1348 [Example: Scenario I, numerical values] Tab. 12 illustrates the different aims of the particular evaluation rules; cf. par. 1326. Evaluation criterion (ER 3) reliably separates A_{real} from A_{bad} . The absolute time that an **online test** suite requires depends on how many **raw random numbers** are generated per second. For typical **PTRNGs** it should not last longer than a few seconds.
- 1349 [Example: Scenario II] $(m, N, k, \beta, t, x_a, x_b, u, v, x_{total}) = (512, 128, 4, 1/32, 5, 75.0, 34.0, 11.0, 23.0, 600.0)$. If a **noise alarm** has occurred, the **PTRNG** goes into the test mode (cf. par. 1343) and performs an emergency test. The tested **raw random numbers** are not output. As for the evaluation rule (ER 3), the emergency test applies the χ^2 -test to $2^{16} = 65536$ bits (sample size of an **online test** suite). The emergency test fails if the test value is > 600.0 . If the emergency test is passed, the **PTRNG** returns to the working mode (outputting random numbers).
- 1350 [Example: Scenario II, numerical values] Tab. 13 collects numerical results that illustrate properties of the test parameters that were selected in par. 1349. Recall that 0.53 limits A_{real} while (approximately) $p = 0.56$ defines the limit line between A_{good} and A_{bad} . The values in Tab. 13 are computed on the basis of the simulated cumulative distribution function of the test variable C_1 under the particular distributions of the **raw random numbers**, cf. par. 1305.

Table 13: Scenario II (The **algorithmic post-processing** algorithm XORs non-overlapping pairs of **raw random numbers**): The first row provides the average number of basic tests until 4 consecutive basic test values exceed 34.0 (event A_4). The second row quantifies the probability that a **noise alarm** is triggered by evaluation criterion (ER 1) or (ER 2), while the third row contains the probabilities the (ER 3) causes a **noise alarm**. Both row 2 and row 3 refer to a single **online test** suite.

$p=$	0.520	0.530	0.560	0.570	0.580	0.590	0.60
$E(\# \text{ basic tests until } A_4 \text{ occurs})$	$6.9 \cdot 10^8$	$6.8 \cdot 10^7$	$2.3 \cdot 10^4$	2387	335	72	23
Prob(noise alarm by (ER 1) or (ER 2))	0.00000	0.000002	0.64744	0.99983	1.00000	1.00000	1.00000
Prob(noise alarm by (ER 3))	0.00000	0.00000	1.00000	1.00000	1.00000	1.00000	1.00000

- 1351 [Example: Scenario II] Criterion (ER 1a) detects, with a probability of almost 1 (of > 0.9 , of > 0.5), if $p \in [0, 0.34] \cup [0.66, 1]$ (if $p \in [0, 0.36] \cup [0.64, 1]$, if $p \in [0, 0.38] \cup [0.62, 1]$), while the probability is essentially 0 if $p \in [0.43, 0.57]$. If $p \in [0.43, 0.57] \cap A_{bad}$ the other criteria apply. Note: The discriminatory power of the evaluation rules (ER 1a), (ER 1b), and (ER 2) is rather high, even at the boundary to A_{bad} . It could be an option to omit evaluation criterion (ER 3) if the device is resource-constrained, possibly by increasing at the same time the number N of basic tests.
- 1352 [Example: Scenario II, numerical values] Tab. 13 illustrates the different aims of the particular evaluation rules; cf. par. 1326. It is obvious that in Scenario II, far fewer **online tests** suffice than

in Scenario I.

Note: The **online test schemes** specified in pars. 1318 and 1323 restart after N basic tests or after a **noise alarm** has occurred (provided that the **noise alarm** turned out to be erroneous). The limitation to at most N basic tests was introduced to simplify the computation of the probabilities for **noise alarms**. Of course, alternative designs where a test suite continues until a **noise alarm** occurs can also be appropriate. 1353

If the **online test** separates the sets A_{bad} and A_{real} better than the χ^2 -test in this subsection, simpler **online test** (or **online test procedure**) can be applied. An example would be the **online test** discussed in Subsection 5.4.3, see pars. 1225, for suitable parameters s/μ and σ/μ . 1354

[Example] The designer expects only a few requests for **internal random numbers** per day. To save time and energy needed to perform a continuous **online test**, the **PTRNG** design buffers **internal random numbers** that are ready for output. The buffer is refilled when the number of remaining **internal random numbers** falls below a specified lower bound. As long as the buffer does not require fresh **internal random numbers**, random numbers generated by the **PTRNG** are discarded and not tested. Then the **online tests** are applied again. 1355

[Example, ctd.] Such an approach is principally possible but the **online test** has to be adjusted to this situation. In particular, it does not suffice, e.g., to apply the **online test procedure** from par. 1323 and to continue the **online test** suite where it was interrupted after the buffer had been filled the last time. The reason is that between the subsequent basic tests, a large period of time may have elapsed. Thus, one aim of the **online test** suite, detecting slow drifts of the parameters, cannot be assured. However, it could be an option to apply an emergency test first (without outputting **internal random numbers**), and then to resume with continuously applied **online tests**. 1356

5.6 Linux /dev/random and /dev/urandom

The Linux operating system includes two **RNG** interfaces as part of the kernel: 1357

- the random number generator **/dev/random/**
- the random number generator **/dev/urandom/**

[Linux kernel versions as of 5.18.1] Figure 36 provides a schematic overview of **/dev/random/** and **/dev/urandom/**. Both **RNGs**, **/dev/random/** and **/dev/urandom/**, extract **entropy** from different **non-physical noise sources** that either depend on actions of the user or on internal system tasks. The bottom line of Figure 36 lists several possible **non-physical noise sources**. Every single event (e.g., keyboard and mouse actions; access to the hard disk; interrupt timestamps) is mapped to a **bit string**. The **bit strings** are mixed into a register called **input_pool** using the Blake2s hash function. For the interrupt **noise source**, there is an additional register per CPU called **fast_pool** (which uses SipHash) that accumulates several interrupt timestamps before its content is mixed into the **input_pool**. The **entropy** of the incoming **raw random numbers** is estimated using 1358

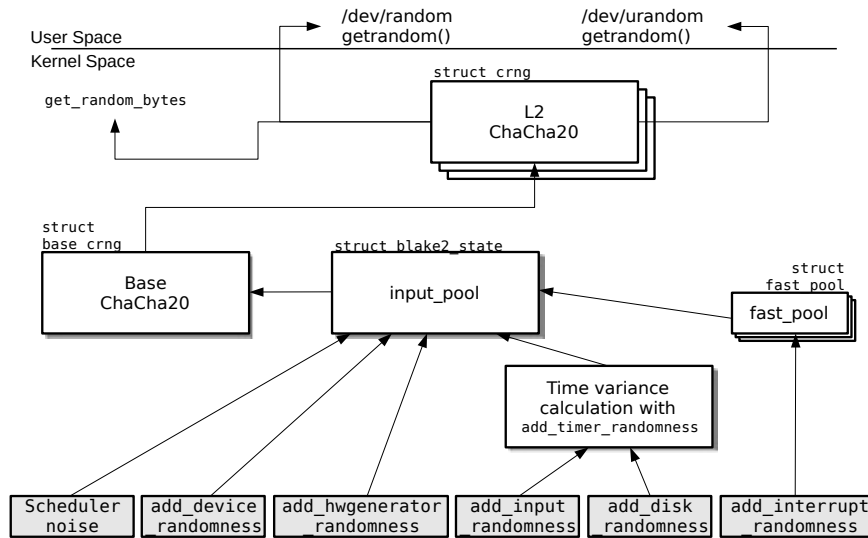


Figure 36: Functional design of the Linux NPTRNG (kernel version as of 5.18.1); source: [Linux_RNG_2024]

conservative heuristic rules. The “base ChaCha20 DRNG” is seeded from the `input_pool` as soon as a heuristic “entropy counter” has counted at least 256 bits of **min-entropy**. After the initial seeding of the “base ChaCha20 DRNG”, the **entropy** supposedly contained in the `input_pool` is no longer estimated and tracked. Upon an internal **request**, **seed material** is generated from the `input_pool` (without any **entropy** estimate). Both `/dev/random` and `/dev/urandom` use a DRNG based on the ChaCha20 stream cipher, which is seeded using the `input_pool`. Unlike for the Linux kernels 5.6 to 5.17, both `/dev/random` and `/dev/urandom` use several ChaCha20 DRNGs. There is one “base ChaCha20 DRNG” which seeds several “secondary ChaCha20 DRNGs” (one per CPU core).

- 1359 [previous Linux kernel versions] The next paragraphs briefly address previous Linux kernels.
- 1360 [Linux kernel versions 5.6 to 5.17] Figure 37 provides a schematic overview of both `/dev/random/` and `/dev/urandom/`. Figs. 36 and 37 indicate that there are many common features. Main differences are: The `input_pool` uses a linear-feedback shift register and SHA-1, and the `fast_pool` applies rotations and XOR operations. Furthermore, unlike for kernel version as of 5.18.1, there is only one DRNG, which is also based on the ChaCha20 cipher.
- 1361 [up to Linux kernel 5.5] Up to Linux kernel version 5.5, `/dev/random` used an additional register (first the so-called `blocking_pool` and later the **internal state** of the ChaCha20-based DRNG), which was seeded by the `input_pool` and had its own **entropy** counter. A blocking mechanism prevented the output of **internal random numbers** if the `blocking_pool` did not contain enough “unconsumed” **entropy**. In other words, `/dev/random` required that the `input_pool` and the `blocking_pool`, resp. the **internal state** of the ChaCha20-based DRNG, were updated continuously because they could not output more bits than indicated by the **entropy** counter. In contrast, `/dev/urandom` did not apply a blocking mechanism.

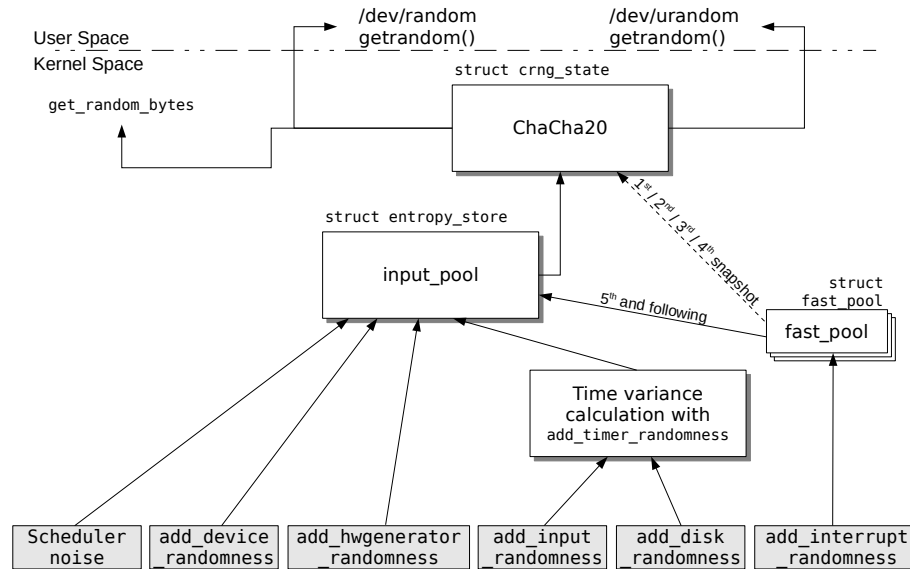


Figure 37: Functional design of the Linux **NPTRNG** (kernel version 5.6 to 5.17); source: [\[Linux_RNG_2022\]](#)

[blocking vs. non-blocking] As mentioned in par. 1361, up to Linux kernel 5.5 `/dev/random` applied a blocking mechanism, a necessary feature of an **NPTRNG** to be NTG.1-compliant. As of Linux kernel version 5.6 to kernel version 5.17, this strategy was changed. Currently, care is taken that `/dev/random` blocks until the ChaCha20-**DRNG** is seeded properly. From that moment on, no more blocking is applied. This increases the output rate of `/dev/random` but prevents NTG.1-compliance. Under suitable circumstances, compliance of the base ChaCha20 **DRNG** with functionality class DRG.3 is possible. In contrast, `/dev/urandom` does not apply any blocking mechanism, not even for the initial **seeding procedure**.

Since `/dev/random` and `/dev/urandom` have been used by many cryptographic applications, in 2012, the **BSI** has initiated a permanent study in which `/dev/random` and `/dev/urandom` are evaluated for each Linux kernel. The results are contained in reports [\[Linux_RNG_2016\]](#) (treating the Linux kernels 3.2.0 and 3.5 to 4.8), [\[Linux_RNG_2020\]](#) (treating the Linux kernels 4.9 to 5.5), [\[Linux_RNG_2022\]](#) (treating the Linux kernels 5.6 to 5.17), and [\[Linux_RNG_2024\]](#) (treating the Linux kernels as of 5.18.1). The last document is continuously supplemented by evaluation results for new kernels. All documents are available on the **BSI** website. Note: The first report is in German, the other documents are in English.

The documents [\[Linux_RNG_2016\]](#), [\[Linux_RNG_2020\]](#), [\[Linux_RNG_2022\]](#) as well as the newest document [\[Linux_RNG_2024\]](#) do not only contain the final results of the evaluations but also explain details of the evaluation methodology. The general methodology is also applicable to other **RNG** designs that use **non-physical noise sources**. The evaluation entails a detailed description and analysis of all components of `/dev/random`, including the **non-physical noise sources** and **entropy** gathering functions, the input, output, and state transition functions of the

entropy pools, the heuristics utilized by the entropy counter, and the ChaCha20-based DRNG. The analysis concludes that /dev/random significantly underestimates the collected entropy (i.e., that it actually collects more entropy than internally estimated). Theoretical considerations are supplemented by empirical entropy estimates. For the evaluation, a suite of entropy estimators is applied to the raw random numbers recorded from the non-physical noise sources of an instrumented Linux kernel during boot time and regular operation.

1365 The document [Linux_RNG_overview] provides a table that lists the compliance of /dev/random to functionality class NTG.1 or DRG.3 for the Linux kernel versions beginning with 3.5. Note that the class definitions refer to [AIS2031An_11]. An explanation of the differences can be found in the appendix. These results are only applicable to the RNG /dev/random if several requirements are met; cf. [Linux_RNG_overview], Notes, for details. In particular,

- The Linux system runs on a x86 platform.
- The CPU of the system has the RDTSC instruction.
- The clock frequency of the CPU is at least 1 GHz.
- The Linux system is not running in a virtual machine.
- The source files of the kernel that are relevant to /dev/random are unchanged as compared to the analyzed upstream version.

It is part of the evaluation to confirm that these requirements are satisfied.

1366 Table 14 summarizes the results from [Linux_RNG_overview].
Note: In the Linux kernel versions 5.18.1 to 6.7, there is no formal compliance with functionality

Table 14: Conformity of /dev/random to NTG.1 and DRG.3; see [Linux_RNG_overview]

Linux kernel	Conformity to functionality class ([AIS2031An_11])
3.5 – 3.14	NTG.1
3.15 – 3.16	—
3.17 – 3.19	NTG.1
4.1 – 4.20	NTG.1
5.1 – 5.5	NTG.1
5.6 – 5.17	DRG.3
5.18.1 – 6.7	—

class DRG.3 (as defined in [AIS2031An_11]) because several DRNGs occur, which are seeded by another DRNG. The first RNG (root DRNG), the base ChaCha20 DRNG, is compliant with functionality class DRG.3. This document contains a new functionality class DRT.1, which covers this feature.

1367 The report [RNG_virtual_env] considers random number generation in virtual environments.

Glossary

additional input Any data that are input to a **hybrid DRNG** by the state transition function (i.e., not by the **seeding procedure** or **reseeding procedures**, cf. par. 129). These data can be provided by an internal or external **noise source**; they can or may not contain **entropy** (e.g., predictable, low-**entropy**, high-**entropy**); they may be provided by a reliable source or be under the control of an **adversary**.

Note 1: In the context of functionality class DRG.4, the intuitive but imprecise wording “high-**entropy** additional input” is used if the **additional input** ensures **enhanced forward secrecy**.

Note 2: This, in particular, requires that the additional input has at least 240 bits of **min-entropy**, or, if the substitution condition applies, at least 250 bits of **Shannon entropy**, motivating the naming.

adversary A malicious entity whose goal is to determine, to guess, or to influence the output of an **RNG**. The term *attacker* is used synonymously.

AIS The term “AIS” stands for “Anwendungshinweise und Interpretationen im Schema” and denotes a class of documents containing notes on application and interpretation of the German **CC** scheme. In particular, the AIS 20 deals with **DRNGs** while the AIS 31 deals with **TRNGs**. This document is the mathematical-technical reference for the current versions of both the AIS 20 and the AIS 31, and is also often loosely referred to as the AIS 20, AIS 31, or AIS 20/31; cf. Subsection 1.1.

algorithmic post-processing A type of **post-processing** that is generally used for the purpose of increasing the **entropy** per data bit (**entropy extraction**). It is usually applied to the **raw random numbers**. The name is chosen to distinguish it from an analog transformation (e.g., amplification, band-pass filter).

Note 1: Viewed as a mathematical function, algorithmic post-processing algorithms usually have small domains and small ranges (in contrast to **cryptographic post-processing**). Algorithmic post-processing can be stateful (i.e., **with memory**) or stateless.

Note 2: Typical examples of algorithmic post-processing algorithms: XORing bits or binary vectors, modular addition, **LFSRs**.

attacker Synonym for **adversary**.

backtracking resistance Term from NIST SP 800-90[A,B,C]

Note: Backtracking resistance is similar to **enhanced backward secrecy**.

backward secrecy Assurance that knowledge about previous output values cannot be derived with practical computational effort from the knowledge of current or subsequent output values (cf. par. 68).

Note 1: “Deriving knowledge” means gaining significant advantage over blind guessing.

Note 2: **TRNGs** can ensure backward secrecy by **entropy**.

biased A value that is chosen from a sample space is said to be biased if one value is more likely to be chosen than another value. Contrast with **unbiased**.

bit string A finite sequence of ones and zeroes.

black box An idealized mechanism that accepts inputs and produces outputs. It is designed such that an observer cannot see inside the box or determine precisely what is happening inside that box. In contrast with a **glass box**.

compression rate Ratio between the average input bit length of the **cryptographic post-processing** algorithm and the bit length of the resulting **internal random numbers** per (short) time interval; ideally holds for each **internal random number** (cf. par. 303 and par. 388).

computational security Security against an **adversary** with bounded computing power. Quantified by the **security strength (of cryptographic mechanisms)**.

conceptual atomicity (**AIS 20**-specific term) The **requests** of a **DRNG** satisfy the condition of conceptual atomicity if the **DRNG** finishes every **request** by the application of the state transition function *before any of the requested bits are used* (cf. par. 109).

Note 1: In the context of CPU instructions, “atomic” refers to operations that are non-interruptible by other operations.

consuming application An application that uses random outputs from an **RNG**.

cryptographic State transition functions and output functions are considered cryptographic if they are composed of cryptographic primitives (e.g., block ciphers or hash functions). See par. 104.

Note: Incrementation by 1, simple XOR-additions, additions and multiplications in small moduli, **LFSRs**, and projections, for example, are not viewed as cryptographic.

cryptographic post-processing Stateful **post-processing** (i.e., **with memory**) for the purpose of gaining **DRNG** security properties (**computational security**). It is usually applied to **intermediate random numbers**, or to **internal random numbers** of a separate **TRNG**. It can also be applied to **raw random numbers**.

Note: By the definition given in this document, cryptographic post-processing is always stateful. Cryptographic constructions without memory (i.e., which do not grant **DRNG** security properties) are denoted as stateless **post-processing** with **cryptographic** functions.

das-random number Digitized-analog-signal random number. A **bit string** that results directly from the **digitization** of analog noise signals in a **PTRNG**. Das-random numbers constitute a special case of **raw random numbers**.

deterministic random bit generator A term from NIST SP 800-90. Equivalent to **DRNG**.

deterministic RNG An **RNG** that produces random numbers from a secret initial value called a seed or **seed material** by applying a deterministic algorithm.

Note 1: A deterministic **RNG**, at least initially, has access to a **randomness** source.

Note 2: The term is equivalent to **DRBG** (NIST SP 800-90).

Note 3: This document uses the abbreviation **DRNG**.

digitization The process of generating raw discrete digital values from non-deterministic events (e.g., analog **noise sources**) within a **noise source**.

Note 1: Raw discrete digital values are called **raw random numbers**.

Note 2: In addition to the actual conversion of analog data into digital values, the digitization mechanism can include elementary operations like skipping values (thinning out), dropping bits (e.g., casting 10-bit-values as bytes by cutting the two least significant bits), or counting.

digitization process See [digitization](#).

direct seed predecessor A [DRNG](#) from which a (non-root) [DRNG](#) in a [DRNG tree](#) receives initial [seed material](#).

DRNG tree ([AIS 20](#)-specific term) A set of [DRNGs](#) of which one [DRNG](#) is and remains distinguished (“*root DRNG*”). This includes the information about the direct seed predecessors. The set of [DRNGs](#) can be static or dynamic in the sense that [DRNGs](#) can be uninstantiated and new [DRNGs](#) can be instantiated. All [DRNGs](#) except the root [DRNG](#) receive [seed material](#) from another [DRNG](#) of this set. The root [DRNG](#) receives [seed material](#) from an initial randomness source. The initial randomness source generates [seed material](#) to which [entropy](#) can be assigned (cf. par. 165).

Note: Functionality class DRT.1 formulates requirements for DRNG trees.

effective internal state The security-critical part of the [internal state](#) of a [DRNG](#) that an [adversary](#) does not know and that he cannot determine or guess (with probability that is significantly greater than indicated by its size, assuming optimal encoding) even if he has seen many random numbers (cf. par. 91).

effective internal state space Set of all admissible [effective internal states](#) of a [DRNG](#).

enhanced backward secrecy Assurance that knowledge about previous output values cannot be derived with practical computational effort from the knowledge of the current [internal state](#) of an [RNG](#) (cf. par. 69).

Note 1: “Deriving knowledge” means gaining significant advantage over blind guessing.

Note 2: The notion of enhanced backward secrecy is trivial for memoryless [RNGs](#). Therefore, it is only a useful notion for [DRNGs](#) and [hybrid PTRNGs](#), the security of which rests at least in part on cryptographic properties of the state transition function and the output function of the [RNG](#).

Note 3: A term related to enhanced backward secrecy is [backtracking resistance](#) (from NIST SP 800-90[A,B,C]).

enhanced forward secrecy Assurance that knowledge about subsequent output values cannot be derived with practical computational effort from the knowledge of the current [internal state](#) of an [RNG](#).

Note 1: “Deriving knowledge” means gaining significant advantage over blind guessing (cf. par. 69).

Note 2: [Pure DRNGs](#) are unable to achieve enhanced forward secrecy. Unlike [forward secrecy](#) and [backward secrecy](#) as well as [enhanced backward secrecy](#), enhanced forward secrecy rests entirely on the capability of inserting as much [entropy](#) as is required to make the prediction of future outputs infeasible.

Note 3: A term related to enhanced forward secrecy is [prediction resistance](#) (from NIST SP 800-90[A,B,C]).

entropy A measure of disorder, randomness, or variability in a closed system (see par. 606).

Note 1: The [entropy](#) of a [random variable](#) X is a mathematical measure of the amount of information gained by an observation of X .

Note 2: The most common concepts are [Shannon entropy](#) and [min-entropy](#). In this document, [Shannon entropy](#) and [min-entropy](#) are used, depending on the context.

Note 3: [Min-entropy](#) is the measure used in NIST SP 800-90.

entropy extraction The process of increasing the [entropy](#) per data bit. Requires compression.

entropy source (Term from NIST SP 800-90B) The combination of a **noise source**, health tests, and optional conditioning component that produce **bit strings** containing **entropy**. A distinction is made between entropy sources having **physical noise sources** and those having **non-physical noise sources**.

Note 1: The terms “entropy source”, “health test”, and “conditioning” belong to NIST SP 800-90 [A,B,C].

Note 2: In the terminology of AIS 20/31, health tests comprise **start-up tests**, **online tests**, and **total failure tests**, while conditioning components correspond to **post-processing** algorithms. In the terminology of NIST SP 800-90 [A,B,C], health tests comprise continuous tests, on-demand tests, and start-up tests.

Note 3: A PTG.2-compliant **PTRNG** can be viewed as a (coarse) equivalent to a physical entropy source that generates random numbers whose **entropy** per bit is very close to 1.

Note 4: In the literature, the term entropy source is often used synonymously with **noise source**.

external random number **Internal random numbers** that have been output by an **RNG**, i.e., those **internal random number** bits that are actually delivered to a consuming application.

Note 1: (**DRNG**) Some bits of the last **internal random number** of a request might be cut off and not output.

Note 2: (**PTRNG**) If the **PTRNG** runs continuously, many **internal random numbers** might never be output (e.g., only be used for internal testing or periodically overwritten in a buffer).

false positive In the context of AIS 31, an **online test**, **total failure test**, or **start-up test** signaling an error even though the component was actually working correctly. Also, see Tab. 5.

forward secrecy Assurance that knowledge about subsequent output values cannot be derived with practical computational effort from the knowledge of current or previous output values (cf. par. 68).

Note 1: “Deriving knowledge” means gaining significant advantage over blind guessing.

Note 2: **TRNGs** can ensure forward secrecy by **entropy**.

fresh entropy A random **bit string** recently generated by a **noise source**. In particular, “fresh” means that the **bit string** has not been previously used to generate output or has otherwise been made externally available.

Note: The **noise source** should be compliant with PTG.2, PTG.3, or NTG.1.

glass box An idealized mechanism that accepts inputs and produces outputs. It is designed such that an observer can see inside the box and can determine exactly what is going on. In contrast with a **black box**.

granularity level (AIS 20-specific term) Auxiliary term to express for which segments of the output of a **DRNG** security properties such as **forward secrecy**, **backward secrecy**, and **enhanced backward secrecy** hold (cf. par. 107).

hybrid DRNG A **DRNG** accepting **additional input** during operation or being able to trigger **reseeding procedures**.

Note: The second condition requires that the **DRNG** has access to a **TRNG**.

hybrid PTRNG A **hybrid TRNG** with a **physical noise source**.

hybrid RNG An **RNG** that uses design elements from both **DRNGs** and **TRNGs**.

Note: This requires (stateful) **post-processing with memory**. See also **hybrid DRNG** and **hybrid TRNG**.

hybrid TRNG A **TRNG** with **cryptographic post-processing (with memory)**. Usually, the goal is to increase the computational complexity of the output sequence (**computational security**) and possibly also to increase the **entropy** per bit by data compression (**entropy extraction**).

Note: **Cryptographic post-processing** may be viewed as an additional security anchor for a situation where the **entropy** from the **noise source** per output bit is smaller than assumed.

ideal RNG A mathematical construct that generates independent and **uniformly distributed** random numbers.

information-theoretic security Security against an **adversary** with unlimited computing power. Requires **fresh entropy**.

initial randomness source The randomness source for the root **DRNG** of a **DRNG tree**; cf. **randomness**.

intermediate random number (PTG.3- and NTG.1-specific term) input data for **cryptographic post-processing**.

Example: Consider a PTG.3-compliant **RNG** that consists of a PTG.2-compliant **PTRNG** with DRG.3-compliant **cryptographic post-processing**. Here, the intermediate random numbers equal the **internal random numbers** generated by the PTG.2-compliant **PTRNG**.

internal random number Final stage of the random numbers of an **RNG** that are ready to be output (cf. par. 73). Compare to **external random numbers**.

internal state The collection of all secret and non-secret digitized information of an **RNG** as stored in memory at a given point in time.

Note: This also applies to **post-processing** algorithms for **TRNGs**.

internal state space Set of all admissible **internal states** of a **DRNG**.

Kerckhoffs's principle A security analysis is conducted under the basic assumption that the design and public keys of a cryptosystem are known to an **adversary**. Only secret keys and **seed material** are assumed to be unknown to an **adversary**.

known-answer test A test that uses a fixed input/output pair to test the correctness of a deterministic mechanism.

min-entropy A measure of **entropy** based on the minimal (worst-case) gain of information from an observation (see par. 610).

multi-target attack A scenario in which an **adversary** applies guesses or the results of a pre-computation to attack many instances of the same cryptosystem at once in hope that at least one instance succumbs to the attack.

noise alarm Consequence of an application of an **online test** that suggests (e.g., due to a failure of a statistical test) that the quality of the generated random numbers is not sufficiently good. A noise alarm can be a **false positive**.

noise source A source of unpredictable data that outputs raw discrete digital values. The **digitization** mechanism is considered to be part of the noise source (cf. par. 1116). A distinction is made between **physical noise sources** and **non-physical noise sources**.

Note: In AIS 31, raw discrete digital values are called **raw random numbers**.

non-physical noise source A **noise source** that typically exploits system data and/or user interaction to produce digitized random data.

Note 1: It is usually infeasible to determine a sufficiently precise characterization of the behavior of non-physical noise sources. Therefore, designers have to resort to heuristics to obtain a conservative **entropy** lower bound.

Note 2: Non-physical noise sources are used by **non-physical true RNGs** (NPTRNGs).

Note 3: Examples of system data: RAM data, system time of a PC, or the output of API functions. Examples of interaction: key strokes, mouse movement, etc.

non-physical true RNG A **true RNG** with a **non-physical noise source**.

one-way function A function with the property that it is easy to compute the output for a given input but it is computationally infeasible to find an input for a specific output that maps to this output.

online test A quality check of the random numbers (usually the **raw random numbers**) while a **PTRNG** is in operation; usually realized by a statistical test or by a test procedure that applies several statistical tests; often used synonymously for **online test procedure**.

online test procedure Consists of one or several statistical tests (online tests), evaluation rules, a calling scheme, and the specified consequences of a **noise alarm**.

online test scheme Synonym for **online test procedure**.

p-value The p-value quantifies the probability that the test values are at least as extreme as the particular value that has just been observed (tail probability) if the null hypothesis is true. If this p-value is smaller than a pre-defined bound, the null hypothesis is rejected (cf. par. 869).

personalization string An optional input value to a **DRNG** during instantiation to make one **RNG** instance behave differently from other instances.

Note: Can be a **secret parameter** or public parameter.

physical noise source A **noise source** that exploits physical phenomena (thermal noise, shot noise, jitter, metastability, radioactive decay, etc.) from dedicated hardware designs (using diodes, ring oscillators, etc.) or physical experiments to produce digitized random data.

Note 1: Dedicated hardware designs can use general-purpose components (like diodes, logic gates, etc.) if the designer is able to understand, describe, and quantify the characteristics of the design that are relevant for the generation of random numbers.

Note 2: Physical noise sources are used by **physical true RNGs** (PTRNGs).

physical true RNG A **TRNG** that uses a **physical noise source**.

Note 1: This document uses the abbreviated version “physical RNG” instead of “physical true RNG” because all physical RNGs are, by definition, **true RNGs**.

Note 2: This document uses the abbreviation “PTRNG” instead of “PRNG” to avoid confusion with **pseudorandom number generators**.

post-processing Generic term for any kind of transformation applied to random numbers at different stages in the generation of **internal random numbers** in a **TRNG** (e.g., applied to **raw random numbers**).

Note 1: Post-processing can have different goals: reducing bias or dependencies, **statistical inconspicuousness**, **entropy extraction**, **DRNG** fallback (**computational security**), etc.

Note 2: This document distinguishes between **algorithmic post-processing**, **cryptographic post-processing** (i.e., **with memory**), and (in general: stateless) post-processing with **cryptographic** functions.

Note 3: Examples of post-processing algorithms for functionality class PTG.2 can be found in par. 336. **Cryptographic post-processing** is a feature of functionality class PTG.3.

Note 4: Post-processing is related to the term conditioning function in NIST SP 800-90.

prediction resistance Term from NIST SP 800-90[A,B,C]

Note: Prediction resistance is similar to **enhanced forward secrecy**.

predictor An (r, s) -predictor makes a prediction about the next s random numbers on the basis of the previous r random numbers ($r \in \mathbb{N}_0 \cup \{\infty\}$, $s \in \mathbb{N}$); cf. pars. 872 ff..

pseudorandom number generator Another term for a **deterministic RNG**.

pure DRNG A **DRNG** that does not accept input except during the **seeding procedure** or (externally triggered) **reseeding procedure**.

Note 1: Identical **seed material** values result in identical **internal random numbers**.

Note 2: A **pure DRNG** is not able to trigger (by itself) a **reseeding procedure**.

pure PTRNG A **PTRNG** in which any post-processing is non-cryptographic or stateless cryptographic.

Note: A **total failure** of a pure PTRNG's **noise source** typically results in constant output or periodic patterns if no **post-processing** or stateless **post-processing** is implemented, or results in weak pseudorandom output if simple (non-cryptographic) **algorithmic post-processing** is implemented.

random number generator A group of components or an algorithm that outputs sequences of discrete values (usually represented as **bit strings** called **internal random numbers**).

random variable Mathematical construct that models probabilistic behavior (cf. par. 501 and par. 505).

Note: A real-valued random variable X is a function that assigns a value of $\mathbb{R}$ to each outcome in the sample space Ω , i.e., $X : \Omega \rightarrow \mathbb{R}$.

randomness An intuitive characterization of the unpredictability of a **bit string**. If the **bit string** is generated by a **true RNG**, its unpredictability is quantified by **entropy**. If the **bit string** is generated by a **DRNG**, its unpredictability is based on the unpredictability of the **seed material** of the generating **DRNG**, on the secrecy of its internal state, and on the difficulty for an **adversary** to break the cryptographic guarantees of the **DRNG** (in particular, **forward secrecy**, **backward secrecy**, and **enhanced backward secrecy**).

raw random number Raw random numbers are discrete values (usually bits, **bit strings**, or integers) that are derived at discrete points in time from a **noise source** of a **PTRNG** or **NPTRNG**. Raw random numbers have not been significantly post-processed.

Note: For certain **noise sources**, it may not be obvious which discrete values should be interpreted as the raw random numbers. For a meaningful analysis, it is recommended to choose the earliest possible stage.

request (AIS 20-specific term) Operation of a DRNG to generate a requested amount of random numbers that is completed by the update of the state transition function. This operation usually consists of concatenating shorter internal random numbers produced by a “core function” to a larger sequence (cf. par. 106).

Note: For better readability, we call the DRNG operation to generate the response also a “request”, identifying the operation with the response itself.

reseed To refresh the internal state of a DRNG with seed material. The seed material should contain sufficient entropy (sufficient randomness if generated by a DRNG) to allow recovery from a possible compromise.

Note: (verb), corresponds to reseeding procedure.

reseeding procedure Refreshing the internal state of an DRNG with sufficient entropy to allow recovery from a possible compromise.

Note: A reseeding procedure shall utilize the previous internal state. Even if an adversary knew the seed material, it shall not significantly be easier to determine the new internal state than the previous one. (Usually, it would be a little easier to determine the new internal state, e.g., because the seed material and the old internal state are hashed).

secret parameter An optional input value to the seeding procedure or reseeding procedure of a DRNG or to the initialization of the cryptographic post-processing algorithm of a PTRNG to achieve additional security against adversaries who are not in possession of this value.

security boundary A physical or conceptual perimeter that confines the secure domain that an adversary cannot observe or influence in a malicious way (according to the chosen threat model).

security strength (of cryptographic mechanisms) A cryptographic mechanism achieves a security strength of n bits if costs equivalent to 2^n calculations of the encryption function of an efficient block cipher (e.g., AES) are tied to each attack against the mechanism that breaks the security objective of the mechanism with a high probability of success.

seed To initialize the internal state of a DRNG with seed material. The seed material should contain sufficient entropy (sufficient randomness if generated by a DRNG) to meet the security requirements.

Note: (verb), corresponds to seeding procedure.

seed graph (AIS 20-specific term) A directed colored graph that contains information of the seeding and reseeding history of a DRNG tree. The nodes of the seed graph represent the DRNGs of the DRNG tree ever instantiated. The coloring of the nodes and edges allows to distinguish DRNGs that still exist from DRNGs that have already been uninstantiated and “seeding edges” from “reseeding edges”.

seed material A bit string containing entropy (containing randomness if generated by a DRNG) that is used for the seeding procedure or reseeding procedure a DRNG.

Note: This definition also applies to the cryptographic post-processing algorithm (with memory) of a TRNG.

seeding procedure Procedure for seeding (initialization) the internal state of a DRNG.

seedlife The period between (re-)seeding the internal state of a DRNG and subsequent reseeding with new seed material, or between (re-)seeding the internal state of a DRNG and uninstantiating the DRNG.

self test Synonym for **start-up test**.

Shannon entropy A measure of **entropy** based on the expected (average) gain of information from an observation (see par. 608).

start-up test A test that is applied when the **PTRNG** has been started. It is intended to detect severe statistical weaknesses and **total failures**.

stationarily distributed In general, this property of a sequence of **random variables** means that they form a **stationary** stochastic process. In the context of AIS 31, the term may also mean a relaxed condition called **time-local stationarity** if the **random variables** describe the behavior of a **physical noise source**.

stationary Depending on the context, the term stationary has two closely related, separate meanings in this document. For a stochastic process, it has the usual meaning of time-invariance (see par. 548). For a **physical noise source** (which can never satisfy this condition in a strict mathematical sense), we mean a relaxed condition that is more precisely denoted as **time-local stationarity** (see pars. 788 to 791).

statistical inconspicuousness The application of standard statistical tests does not distinguish the generated random numbers from ideal random numbers.

stochastic model A stochastic model provides a partial mathematical description (of the relevant properties) of a (physical) **noise source** using **random variables**. It allows the verification of a lower **entropy** bound for the output data (**internal random numbers** or **intermediate random numbers**) during the lifetime of the physical RNG, even if the quality of the digitized data goes down. The stochastic model is based on and justified by an understanding of the **noise source**.

Note 1: Ideally, a stochastic model consists of a family of probability distributions that contains the true distribution of the noise source output (raw random numbers) or of suitably defined auxiliary **random variables** during the lifetime of the physical RNG.

Note 2: It can suffice to model parts of the **entropy** contributions if it can be shown that the neglected effects do not decrease the **entropy**.

sufficiently soon AIS 31-specific term to quantify the ability of an **online test** to detect **entropy** defects of random numbers quickly enough. What is meant by “sufficient” in terms of tested bits or **internal random number** bits depends on the possible **entropy** defects as described in the **stochastic model**. Due to the probabilistic nature of statistical testing, it is impossible to prescribe explicit conditions that shall be detected instantly. As a general rule, severe **entropy** defects can and shall be detected very quickly (i.e., with high probability). Smaller deviations from the acceptable behavior are more difficult to detect but shall be detected after a reasonable amount of tested bits. Note that in contrast with an **online test**, a **total failure test** shall detect **total failures** of a **noise source** virtually immediately.

time-local stationarity (AIS 31-specific term that refers to the distribution of random numbers) A sequence of **random variables** $X_1, X_2, \dots$ is considered to be “time-local” **stationarily distributed** (often, loosely “**stationarily distributed**” if the context is clear) if this sequence may be viewed as **stationarily distributed** at least over “short” time-scales (in absolute time) that are, however, “large” compared to periods needed to generate samples for the online tests and the evaluator tests.

total failure The **noise source** is broken and delivers no or at most a small fraction of its expected **entropy**.

Note 1: Depending on the concrete design and **digitization**, a total failure of the **noise source** may result in constant or short-period sequences of **raw random numbers**.

Note 2: It is possible that the **raw random numbers** still contain **entropy** due to noise from other components (e.g., from an amplifier), but this scenario still constitutes a total failure.

total failure alarm Consequence of a failed **total failure** test.

total failure test A test that shall reliably detect **total failures** and prevent the output of low-**entropy** random numbers

Note: A total failure test is usually realized by physical measurements or by a statistical test. Due to the low **entropy**, a **total failure** can usually be detected very reliably, and the probability of a **false positive** is usually small.

true RNG A device or mechanism for which each output value depends on newly generated data from a **noise source**.

unbiased A **random variable** is said to be unbiased if all values of the finite sample space are chosen with the same probability. Contrast with **biased**.

Note: The terms unbiased and **uniformly distributed** are used synonymously.

uniformly distributed A **random variable** X with a finite range is considered to be uniformly distributed if X assumes each value with identical probability.

Note: The terms uniformly distributed and **unbiased** are used synonymously.

uninstantiation Uninstantiating an instance of a **DRNG** means that this instance no longer exists. In particular, the **internal state** and secret parameters are destroyed (i.e., securely deleted).

widely recognized cryptographic primitive A cryptographic primitive is considered widely recognized if it has undergone diversified scientific review from many researchers and if the cryptographic community has no serious doubts concerning its strength in relevant operational circumstances (cf. par. 98).

with memory Property of a **post-processing** algorithm. It means that the **post-processing** is stateful, i.e., has a state that retains information from previous invocations or steps.

Acronyms

AES Advanced encryption standard.

ANSSI Agence nationale de la sécurité des systèmes d'information.

BSI Bundesamt für Sicherheit in der Informationstechnik.

CLT Central limit theorem.

das Digitized analog noise signal.

DRBG *Deterministic random bit generator.*

DRNG *Deterministic RNG.*

ECC Elliptic-curve cryptography.

ECDSA Elliptic Curve Digital Signature Algorithm.

iid independent and identically distributed.

KAT Known-answer test.

LFSR Linear-feedback shift register.

NIST National Institute of Standards and Technology.

NPTRNG *Non-physical true RNG.*

OFB Output feedback.

PRNG *Pseudorandom number generator.*

PTRNG *Physical true RNG.*

RNG *Random number generator.*

RSA Rivest–Shamir–Adleman cryptosystem.

SHA Secure Hash Algorithm.

TRNG *True RNG.*

Abbreviations from Common Criteria

ADV Assurance Development.

AVA Assurance Vulnerability Analysis.

CC Common Criteria.

CEM Common Evaluation Methodology.

EAL Evaluation Assurance Level.

FCS Functional Class Cryptographic Support.

ITSEC Information Technology Security Evaluation Criteria.

ITSEM Information Technology Security Evaluation Manual.

PP Protection Profile.

SFR Security Functional Requirement.

ST Security Target.

TOE Target of Evaluation.

TSE TOE Security Functionality.

Symbols

$A \times B$ Cartesian product of the sets A and B .

$B(n, p)$ Binomial distribution with parameters n and p .

$H(X)$ **Shannon entropy** of the **random variable** X .

$H_2(X)$ collision **entropy** of the **random variable** X .

$H_{\min}(X)$ **Min-entropy** of the **random variable** X .

$N(0, 1)$ Standard normal (Gaussian) distribution with mean 0 and variance 1.

$N(\mu, \sigma^2)$ Normal (Gaussian) distribution with mean μ and variance σ^2 .

P_λ Poisson distribution with parameter λ .

$X \parallel Y$ Concatenation of two strings X and Y . The strings X and Y are either both **bit strings**, or both byte strings.

$\Phi(\cdot)$ Cumulative distribution of the standard normal (Gaussian) distribution with mean 0 and variance 1; $\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-0.5t^2} dt$.

$\text{Prob}(X = x)$ Probability that the **random variable** X assumes the value x .

$\text{Prob}(x)$ Short notation of $\text{Prob}(X = x)$ if it is clear which **random variable** is concerned.

$\gamma_{\alpha, \beta}(\cdot)$ density of the Gamma distribution with shape parameter α and rate parameter β .

$\lceil x \rceil$ Ceiling: the smallest integer greater than or equal to x , $\lceil x \rceil = \min \{n \in \mathbb{Z} \mid x \leq n\}$.

$\lfloor x \rfloor$ Floor: the largest integer less than or equal to x , $\lfloor x \rfloor = \max \{n \in \mathbb{Z} \mid n \leq x\}$.

$|X|$ For a finite set X the notation $|X|$ denotes its cardinality. If X is a string $|X|$ denotes its length.

$\log(\cdot)$ Natural logarithm, often denoted by $\ln(\cdot)$.

$\log_a(\cdot)$ Logarithm to base $a > 0$.

$\mathbb{N}$ Set of natural numbers, $= \{1, 2, \dots\}$.

$\mathbb{N}_0$ Set of natural numbers with zero, $= \{0, 1, 2, \dots\}$.

$\mathbb{R}$ Set of real numbers.

$\mathbb{Z}$ Set of integers.

$\mathbb{Z}_n$ $\{0, 1, \dots, n-1\}$.

$\oplus$ Addition in GF(2), $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.

$\pi_w(x)$ The projection of a vector $x = (x_1, x_2, \dots, x_n)$ onto the coordinates $w = \{i_1, i_2, \dots, i_{|w|}\} \subseteq \{1, \dots, n\}$. That is, $\pi_w(x) = (x_{i_1}, x_{i_2}, \dots, x_{i_{|w|}})$.

$\tau * \eta$ Convolution product of the measures τ and η .

$g * h(\cdot)$ Convolution of the densities $g(\cdot)$ and $h(\cdot)$.

$g \circ f$ Composition of mappings f and g .

$x(\bmod m)$ Smallest non-negative number that is congruent to x modulo m ; the tuple (x, m) can be integers or real numbers.

$x \equiv y \bmod m$ x is congruent y modulo m ; the triple (x, y, m) can be integers or real numbers.

$\mathcal{G}_p$ Geometric distribution with parameter p .

References

- [AIS20] BSI. *Funktionalitätsklassen und Evaluationsmethodologie für deterministische Zufallszahlengeneratoren*. Version 3.0 (15.05.2013). <https://www.bsi.bund.de/dok/6618284>.
- [AIS2031An_11] W. Killmann and W. Schindler. *A proposal for: Functionality classes for random number generators*. Version 2.0 (18.09.2011), Mathematical-technical reference of [AIS20] and [AIS31], <https://www.bsi.bund.de/dok/ais-20-31-appx-2011>.
- [AIS20An_99] W. Schindler. *AIS 20: Functionality classes and evaluation methodology for deterministic random number generators*. Version 2.0 (02.12.1999), Mathematical-technical reference of [AIS20], English translation. <https://www.bsi.bund.de/dok/ais-20-appx-1999>.
- [AIS31] BSI. *Funktionalitätsklassen und Evaluationsmethodologie für physikalische Zufallszahlengeneratoren*. Version 3 (15.05.2013). <https://www.bsi.bund.de/dok/6618252>.
- [AIS31An_01] W. Killmann and W. Schindler. *A proposal for: Functionality classes and evaluation methodology for true (physical) random number generators*. Version 3.1 (25.09.2001), English translation. <https://www.bsi.bund.de/dok/ais-31-appx-2001>.
- [ASPB+18] E.N. Allini, M. Skórski, O. Petura, F. Bernard, M. Laban, and V. Fischer. “Evaluation and Monitoring of Free Running Oscillators Serving as Source of Randomness”. In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018.3 (2018), pp. 214–242.
- [BSI_NIST_joint_2024] E. Barker, J. Kelsey, K. McKay, J. Mittmann, M. Peter, W. Schindler, and M.S. Turan. *Bridging the Gap between Standards on Random Number Generation — Comparison of SP 800-90 Series and AIS 20/31 (in progress)*. 2024.
- [BeFV10] F. Bernard, V. Fischer, and B. Valtchanov. “Mathematical model of physical RNGs based on coherent sampling”. In: *Tatra Mountains Mathematical Publications* 45.1 (2010), pp. 1–14. URL: <https://tatra.mat.savba.sk/Full/45/01be-f-v.pdf>.
- [BuLu08] M. Bucci and R. Luzzi. “Fully Digital Random Bit Generators for Cryptographic Applications, IEEE Transactions on Circuits and Systems I”. In: *Regular Papers* 5 (3 2008), pp. 861–875.
- [BuLu16] M. Bucci and R. Luzzi. “A Fully-Digital Chaos-Based Random Bit Generator”. In: *The New Codebreakers: Essays Dedicated to David Kahn on the Occasion of His 85th Birthday*. Ed. by P.Y.A. Ryan, D. Naccache, and J.-J. Quisquater. Vol. 9100. Lecture Notes in Computer Science. Springer, 2016, pp. 396–414.
- [CC2022R1] *Common Criteria for Information Technology Security Evaluation: Part 1 - Part 5*. CC:2022 Revision 1. CCMB-2022-11-001 – CCMB-2022-11-005. Nov. 2022.

- [CFPZ09] C. Chevalier, P.-A. Fouque, D. Pointcheval, and S. Zimmer. “Optimal Randomness Extraction from a Diffie-Hellman Element”. In: *Proceedings of the 28th Annual International Conference on Advances in Cryptology: the Theory and Applications of Cryptographic Techniques*. EUROCRYPT09. Cologne, Germany: Springer-Verlag, 2009, pp. 572–589.
- [Cox62] David Roxbee Cox. *Renewal theory*. London: Methuen, 1962.
- [DORR08] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam Smith. “Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data”. In: *SIAM Journal on Computing* 38.1 (Jan. 2008), 97–139. ISSN: 1095-7111. DOI: [10.1137/060651380](https://doi.org/10.1137/060651380). URL: <http://dx.doi.org/10.1137/060651380>.
- [DaRo87] W.B. Davenport Jr and W.L. Root. “An Introduction to the Theory of Random Signals and Noise (Institute of Electrical and Electronics Engineers, New York, 1987)”. In: *Author Affiliations Kazumasa Takada, Akira Himeno, Ken-ichi Yukimatsu NTT Communications Switching Laboratories, Nippon Telegraph and Telephone Corporation* (), pp. 9–11.
- [DiBi07] M. Dichtl. “Bad and Good Ways of Post-processing Biased Physical Random Numbers”. In: *Fast Software Encryption, 14th International Workshop, FSE 2007, Luxembourg, Luxembourg, March 26-28, 2007, Revised Selected Papers*. Ed. by A. Biryukov. Vol. 4593. Lecture Notes in Computer Science. Springer, 2007, pp. 137–152. DOI: [10.1007/978-3-540-74619-5_9](https://doi.org/10.1007/978-3-540-74619-5_9). URL: https://doi.org/10.1007/978-3-540-74619-5_9.
- [Diac88] P. Diaconis. “Group Representations in Probability and Statistics”. In: vol. 11. Lecture Notes — Monograph Series. Hayward, Calif. : Institute of Mathematical Statistics, 1988.
- [FBB+23] V. Fischer, F. Bernard, N. Bochard, Q. Dallison, and M. Skórski. “Enhancing Quality and Security of the PLL-TRNG”. In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2023.4 (2023), pp. 211–237. DOI: [10.46586/TCHES.V2023.I4.211-237](https://doi.org/10.46586/TCHES.V2023.I4.211-237). URL: <https://doi.org/10.46586/tches.v2023.i4.211-237>.
- [Fell65] W. Feller. *An Introduction to Probability Theory and Its Application (Vol. 2)*. Wiley, 1965.
- [FiBB19] V. Fischer, F. Bernard, and N. Bochard. “Modern random number generator design - Case study on a secured PLL-based TRNG”. In: *it - Information Technology* 61.1 (2019), pp. 3–13. DOI: [10.1515/itit-2018-0025](https://doi.org/10.1515/itit-2018-0025).
- [FiDr02] V. Fischer and M. Drutarovsky. “True Random Number Generator Embedded in Reconfigurable Hardware”. In: *Cryptographic Hardware and Embedded Systems - CHES 2002*. Vol. 2523. LNCS. Redwood Shores, CA, USA. Springer Verlag, 2002, pp. 415–430.
- [Flod89] P. Flajolet and A. Odlyzko. “Random mapping statistics”. In: *Advances in cryptology - EUROCRYPT’89*. Springer. 1990, pp. 329–354.

- [GaSt77] P. Gänsler and W. Stute. *Wahrscheinlichkeitstheorie*. Springer, 1977.
- [Geor15] H.-G. Georgii. *Stochastik — Einführung in die Wahrscheinlichkeitstheorie und Statistik*. 5th. De Gruyter, 2015.
- [Golo64] S.W. Golomb. “Random permutations”. In: *Bull. Amer. Math. Soc* 70 (1964).
- [HaLP34] G.H. Hardy, J.E. Littlewood, and G. Pólya. *Inequalities*. 1934.
- [HaTs15] M. Hayashi and T. Tsurumaru. “More Efficient Privacy Amplification with Less Random Seeds via Dual Universal Hash Function (version 5)”. In: (2015). Appeared in *IEEE Transactions on Information Theory*, Vol. 62, Issue 4, 2213 - 2232 (2016). URL: <https://arxiv.org/abs/1311.5322>.
- [HoRo48] W. Hoeffding and H. Robbins. “The central limit theorem for dependent random variables”. In: *Duke Math. J.* 15 (1948), pp. 773–780.
- [ISO_18031] *ISO / IEC 18031: Information technology – Security Techniques. Random Bit Generation. 2011 / Cor 1: 2014 / A1: 2017*.
- [ISO_20543] *ISO / IEC 20543: Information technology – Security Techniques. Test and Analysis Methods for Random Bit Generators within ISO / IEC 19790 and ISO / IEC 15408*. 2019.
- [IwSh13] M. Iwamoto and J. Shikata. “Revisiting Conditional Renyi Entropies and Generalizing Shannon’s Bounds in Information Theoretically Secure Encryption”. In: *Cryptology ePrint Archive* (2013). URL: <https://eprint.iacr.org/2013/440.pdf>.
- [Jitter-RNG] Stephan Müller. *CPU Time Jitter Based Non-Physical True Random Number Generator*. (July 2022). <https://www.chronox.de/jent/doc/CPU-Jitter-NPTRNG.pdf>.
- [Jones04] G.L. Jones. “On the Markov Chain Central Limit Theorem”. In: *Probability Surveys* 1 (2004), pp. 299–320.
- [KaTa75] S. Karlin and H.M. Taylor. *A First Course in Stochastic Processes*. 2nd edition. London: Academic Press, 1975.
- [Kels24] J. Kelsey. “Prediction, Backtracking, and Enhanced Backtracking Resistance in HMAC_DRBG (unpublished manuscript (2024))”. In: ().
- [KiSc08] W. Killmann and W. Schindler. “A Design for a Physical RNG with Robust Entropy Estimators.” In: *CHES 2008*. Ed. by E. Oswald and P. Rohatgi. Vol. 5154. Lecture Notes in Computer Science. Springer, 2008, pp. 146–163.
- [KoSC78] V.F. Kolchin, B.A. Sevast’yanov, and V.P. Chistyakov. *Random Allocations*. V.H. Winston, 1978.
- [Lach08] P. Lacharme. “Post-Processing Functions for a Biased Physical Random Number Generator”. In: *Fast Software Encryption, 15th International Workshop, FSE 2008, Lausanne, Switzerland, February 10-13, 2008, Revised Selected Papers*. Ed. by K. Nyberg. Vol. 5086. Lecture Notes in Computer Science. Springer, 2008, pp. 334–342. DOI: [10.1007/978-3-540-71039-4_21](https://doi.org/10.1007/978-3-540-71039-4_21). URL: https://doi.org/10.1007/978-3-540-71039-4_21.

-
- [Lall86] S.P. Lalley. “Renewal Theorem for a Class of Stationary Sequences”. In: *Probab. Th. Rel. Fields* 72 (1986), pp. 195–213.
- [Linux_RNG_2016] S. Müller and G. Krummeck and M. Romsey: *Dokumentation und Analyse des Linux-Pseudozufallszahlengenerators. Version 5.5 (includes the Linux kernels 3.2.0 and 3.5 to 4.8)*, report, produced by atsec information security GmbH, by order of Bundesamts für Sicherheit in der Informationstechnik (BSI), last updated: November 2016. <https://www.bsi.bund.de/LinuxRNG>.
- [Linux_RNG_2020] S. Müller: *Documentation and Analysis of the Linux Random Number Generator. Version 3.6 (includes the Linux kernels 4.9 to 5.5)*, report, produced by atsec information security GmbH, by order of Bundesamts für Sicherheit in der Informationstechnik (BSI), last updated: July 2020. www.bsi.bund.de/LinuxRNG.
- [Linux_RNG_2022] S. Müller: *Documentation and Analysis of the Linux Random Number Generator. Version 4.11 (includes the Linux kernels 5.6 to 5.17)*, report, produced by atsec information security GmbH, by order of Bundesamts für Sicherheit in der Informationstechnik (BSI), last updated: April 2022. www.bsi.bund.de/LinuxRNG.
- [Linux_RNG_2024] S. Müller: *Documentation and Analysis of the Linux Random Number Generator. Version 5.9 (includes the Linux kernels 5.18.1 to 6.7)*, report, produced by atsec information security GmbH, by order of Bundesamts für Sicherheit in der Informationstechnik (BSI), last updated: January 2024. www.bsi.bund.de/LinuxRNG.
- [Linux_RNG_overview] *Overview of Linux kernels with NTG.1- or DRG.3-compliant random number generator /dev/random*, Bundesamts für Sicherheit in der Informationstechnik (BSI), last updated: January 2024. www.bsi.bund.de/LinuxRNG.
- [Maur92] U. Maurer. “A universal statistical test for random bit generators”. In: *J. Cryptol.* 5.2 (1992), pp. 89–105.
- [Neue04] D. Neuenschwander. *Probabilistic and Statistical Methods in Cryptology, An Introduction by Selected Topics*. Vol. 3028. Lecture Notes in Computer Science. Springer, 2004.
- [Plia99] J. Pliam. “The Disparity between Work and Entropy in Cryptology”. In: (1998). Appeared in the Theory of Cryptography Library and has been included in the ePrint Archive. [pliam\spacefactor@m{ }ima.umn.edu](http://pliam.spacefactor@m{ }ima.umn.edu). 10500 received November 9th, 1998. Revised, February 1st, 1999. <https://eprint.iacr.org/1998/024>.
- [PuWi68] Purdom, P.W. and Williams, J.H. “Cycle Length in a Random Function”. In: *Transactions of the American Mathematical Society* 2.133 (547-551), pp. 88–100.

- [RNG_virtual_env] S. Müller and G. Krummeck and H. Kurth: *Analysis of random number generation in virtual environments. Report, produced by atsec information security GmbH, by order of Bundesamts für Sicherheit in der Informationstechnik (BSI), Version 1.0, October 21, 2016.*
<https://www.bsi.bund.de/dok/randomness-in-vms>.
- [SP800-22] A. Rukhin et al.: *A statistical test suite for random and pseudorandom number generators for cryptographic applications, NIST Special Publication 800-22 (with rev 1a, 2010 revisions dated May 15, 2001).*
- [SP800-90A] *Recommendation for Random Number Generation Using Deterministic Random Bit Generators, NIST Special Publication 800-90A, Revision 1 (June 2015).*
<https://doi.org/10.6028/NIST.SP.800-90Ar1>.
- [SP800-90B] *Recommendation for the Entropy Sources Used for Random Bit Generation, NIST Special Publication 800-90A (January 2018).*
<https://doi.org/10.6028/NIST.SP.800-90B>.
- [SP800-90C] *Recommendation for Random Bit Generator (RBG) Constructions, NIST Special Publication 800-90C (Fourth Draft) (July 2024).*
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-90C.4pd.pdf>.
- [Schi01] W. Schindler. “Efficient Online Tests for True Random Number Generators.” In: *CHES 2001*. Ed. by Ç.K. Koç, D. Naccache, and C. Paar. Vol. 2162. Lecture Notes in Computer Science. Springer, 2001, pp. 103–117.
- [Schi09a] W. Schindler. “Random Number Generators for Cryptographic Applications”. In: *Cryptographic Engineering*. Ed. by Ç.K. Koç. Springer, 2009, pp. 5–23.
- [Schi09b] W. Schindler. “Evaluation Criteria for Physical Random Number Generators”. In: *Cryptographic Engineering*. Ed. by Ç.K. Koç. Springer, 2009, pp. 25–54.
- [Shev11] I. Shevtsova. “On the absolute constants in the Berry Esseen type inequalities for identically distributed summands”. In: *arXiv:1111.6554* (2011).
- [Shou08] Victor Shoup. *A computational introduction to number theory and algebra, Version 2*. 2008. URL: https://www.shoup.net/ntb/ntbv2_0.pdf.
- [TR-02102] BSI, *TR-02102, Technical Guideline Cryptographic Mechanisms, (current version, courtesy translation),* <https://bsi.bund.de/TR-02102>.
- [WoSh18] J. Woodage and D. Shumow. “An Analysis of the NIST SP 800-90A Standard”. In: *IACR Cryptology ePrint Archive 2018* (2018). URL: <https://eprint.iacr.org/2018/349>.

6 Appendix

The appendix comprises two sections. In Section 6.1, the main differences of this document to [AIS2031An_11] are explained. Expanding on Subsection 4.4.2, Section 6.2 provides some results about the work factor.

6.1 Main Differences from [AIS2031An_11]

In Section 6.1, the main differences between this document and its predecessor, the mathematical-technical guideline of AIS 20 and AIS 31 from 2011 [AIS2031An_11] are pointed out.

6.1.1 DRNG: Main Differences from [AIS2031An_11]

The document [AIS2031An_11] defined a further functionality class DRG.1 that is weaker than DRG.2, because it only requires **forward secrecy** but not **backward secrecy**. Functionality class DRG.1 has been removed because it turned out that it was not relevant in certification practice. 1368

In this document, a new deterministic functionality class for **DRNG trees**, DRT.1, is introduced. Under suitable conditions, this functionality class allows seeding and reseeding of a **DRNG** by another **DRNG**. 1369

The preceding version [AIS2031An_11] distinguishes between different attack potentials; cf. Table 1, Table 2, Table 12, Table 13, and the corresponding paragraphs. In this document, the requirements do not depend on the attack potential. 1370

This document introduces the concept of **requests**, which did not exist in [AIS2031An_11]. There, only single **internal random numbers** could be output. 1371

Note: The introduction of **requests** in AIS 20 is part of the harmonization of AIS 20 and AIS 31 with the NIST documents NIST SP 800-90[A,B,C].

In [AIS2031An_11], compliance to functionality classes DRG.2 or DRG.3 required that the state transition function, the output function, or both shall be **cryptographic**, while DRG.4 demanded that both the state transition function and the output function shall be **cryptographic**, i.e., composed of cryptographic primitives (e.g., block ciphers or hash functions). In this document, functionality classes DRG.3 and DRG.4 require that the state transition function and the output function be **cryptographic**. 1372

The previous versions of AIS 20, [AIS20An_99; AIS2031An_11], contained a security requirement that within the life cycle of a **DRNG**, high-dimensional random bit vectors shall be mutually disjoint with very large probability; see the Requirements DRG.1.3, DRG.2.4, DRG.3.4, DRG.4.6 in [AIS2031An_11]. These requirements have been dropped. 1373

The mutual disjointness requirement in [AIS20An_99; AIS2031An_11] was motivated by the fact 1374

that in the past, the **internal states** of the **DRNG** implementations on resource-limited devices like smart cards were usually smaller than today. Furthermore, [AIS20An_99] also permitted non-cryptographic **DRNGs** (compliant with functionality classes K1 or K2), which could be used for non-sensitive applications. The main reason for this requirement was to prevent the output of too many random numbers within the life cycle of a **DRNG** relative to the size of the **internal state**. The requirements of the **seeding procedure**, the **reseeding procedure**, and the size of the **internal state** (or rather, the **effective internal state**) have significantly increased. For these reasons, the requirement of mutual disjointness has been dropped.

- 1375 The describing 6-tuple in [AIS2031An_11] (cf. par. 111 ff.) is replaced by a describing 9-tuple. This is mainly caused by the introduction of the concept of **requests**. Furthermore, describing 4-tuples for the **seeding procedure** and **reseeding procedure** have been introduced.
- 1376 In [AIS2031An_11], functionality classes DRG.2, DRG.3, and DRG.4 demanded that the evaluator applies statistical **black box** test suites to the **internal random numbers** (deprecated requirements in terminology of [AIS2031An_11]: DRG.2.5, DRG.3.5, DRG.4.7). This mainly had “historical reasons” because [AIS20An_99] contained two functionality classes (K1 and K2) which allowed non-cryptographic **DRNGs**.

6.1.2 PTRNG: Main Differences from [AIS2031An_11]

- 1377 The previous document [AIS2031An_11] defined an additional functionality class PTG.1, which only claimed statistical properties but not any **entropy** bound. Functionality class PTG.1 has been removed because it turned out that it was not relevant in certification practice.
- 1378 Compared to [AIS2031An_11] for PTG.2-compliant **PTRNGs**, the tolerated **entropy** defect has become significantly smaller. This change was motivated by the fact that the certified **PTRNGs** show significantly smaller **entropy** defects than allowed in [AIS2031An_11]; cf. also par. 319. Furthermore, it should be noted that in this document, the **entropy** claim refers to the **internal random numbers**, whereas in [AIS2031An_11], the **entropy** claim refers to the **raw random numbers**, and test suite A limits (e.g.) the bias of the **internal random numbers**. Moreover, this document also allows **min-entropy** claims in addition to or instead of **Shannon entropy** claims.
- 1379 Unlike in [AIS2031An_11], functionality class PTG.3 allows individual **entropy** claims (with small **entropy** defects) in both **Shannon entropy** and **min-entropy**.
- 1380 Among other things, in [AIS2031An_11], functionality class PTG.3 demanded that the evaluator applies statistical **black box** test suites to the **internal random numbers**. Just as in the case of functionality classes DRG.2, DRG.3, and DRG.4, this requirement has been dropped.
- 1381 The modifications of functionality class DRG.3 also affect functionality class PTG.3.

6.1.3 NPTRNG: Main Differences from [AIS2031An_11]

1382

Among other things, in [AIS2031An_11], functionality class NTG.1 demanded that the evaluator applies statistical **black box** test suites to the **internal random numbers**. Just as in the case of functionality classes DRG.2, DRG.3, and DRG.4, this requirement has been dropped.

In [AIS2031An_11], the **entropy** of the **internal random numbers** was claimed in **Shannon entropy**. In this document, **min-entropy** is used. 1383

The requirement of mutual disjointness of random vectors (Requirement NTG.1.4 in [AIS2031An_11]) has been dropped (as well as for functionality classes DRG.2, DRG.3, and DRG.4; cf. pars. 1373 to 1374). 1384

6.2 Further results on the work factor

Section 6.2 continues the considerations from Subsection 4.4.2, pars. 686 to 701, on the work factor. The definitions from Subsection 4.4.2 remain valid. In particular, we assume that the parameters n and m are large; cf. par. 695. 1385

[Case $\gamma \gg 1$] Differentiating the relative work factor defect (4.184) yields 1386

$$\begin{aligned} \frac{d}{dr} \frac{\Phi\left(\frac{\gamma-r+1.5}{\sqrt{\gamma}}\right) - \Phi\left(\frac{\gamma-r+0.5}{\sqrt{\gamma}}\right)}{\Phi\left(\frac{\gamma-r+1.5}{\sqrt{\gamma}}\right)} &= \frac{d}{dr} \left(1 - \frac{\Phi\left(\frac{\gamma-r+0.5}{\sqrt{\gamma}}\right)}{\Phi\left(\frac{\gamma-r+1.5}{\sqrt{\gamma}}\right)}\right) = \\ &= \frac{\Phi\left(\frac{\gamma-r+1.5}{\sqrt{\gamma}}\right) \frac{1}{\sqrt{\gamma}} \phi\left(\frac{\gamma-r+0.5}{\sqrt{\gamma}}\right) - \frac{1}{\sqrt{\gamma}} \phi\left(\frac{\gamma-r+1.5}{\sqrt{\gamma}}\right) \Phi\left(\frac{\gamma-r+0.5}{\sqrt{\gamma}}\right)}{\left(\Phi\left(\frac{\gamma-r+1.5}{\sqrt{\gamma}}\right)\right)^2}. \end{aligned} \quad (6.1)$$

Here, $\phi(\cdot)$ denotes the density of a standard normal distribution. Applying L'Hopital's rule to (4.184) shows that the relative work factor defect converges to 1 as $r \rightarrow \infty$, which matches with intuition. It should be considered that at the same time, the work factor $w_{e_r}(U)$ converges to 0 (and, of course, for fixed γ the model does no longer fit as $r \rightarrow \infty$). For the work factor defect, we obtain

$$w_{e_r}(U) - w_{e_r}(F(X)) \approx \phi\left(\frac{\gamma - r + 1}{\sqrt{\gamma}}\right). \quad (6.2)$$

Using the approximation (6.2) and applying the inequality (4.44) to the denominator of the right-hand side of (4.184) (with $x = (r - \gamma - 1.5)/\sqrt{\gamma}$) yields the approximation

$$\begin{aligned} \frac{w_{e_r}(U) - w_{e_r}(F(X))}{w_{e_r}(U)} &\approx \frac{\phi\left(\frac{\gamma-r+1}{\sqrt{\gamma}}\right)}{\frac{\sqrt{\gamma}}{r-\gamma-1.5} \phi\left(\frac{r-\gamma-1.5}{\sqrt{\gamma}}\right)} = \frac{r-\gamma-1.5}{\sqrt{\gamma}} e^{\frac{-0.5}{\gamma}((\gamma-r+1)^2 - (r-\gamma-1.5)^2)} = \\ &= \frac{r-\gamma-1.5}{\sqrt{\gamma}} e^{\frac{-0.5}{\gamma}(-(\gamma-r+1)-0.25)} = \frac{r-\gamma-1.5}{\sqrt{\gamma}} e^{\frac{-0.5}{\gamma}(r-\gamma-1.25)} \quad \text{for } r > \gamma + 1.5. \end{aligned} \quad (6.3)$$

[Case $\gamma \gg 1$] Equation (4.184) is the equivalent to (4.179) for arbitrary success probabilities α 1387

$$\frac{w_{\alpha}(U) - w_{\alpha}(F(X))}{w_{e_r}(U)} \approx \frac{\alpha - \Phi\left(\frac{\gamma-r_{\alpha}+0.5}{\sqrt{\gamma}}\right)}{\alpha}. \quad (6.4)$$

- 1388 [Case $\gamma = 1$, i.e., $n = m$] This corresponds to a **cryptographic post-processing** algorithm for which the input rate equals the output rate. In this case, the **random variables** Y and Y' may be viewed as a Poisson distributed P_τ with parameter $\tau = 1$. In particular, (4.179) simplifies to

$$\frac{w_{e_r}(U) - w_{e_r}(F(X))}{w_{e_r}(U)} \frac{P_1(r-1)}{\sum_{s=r-1}^{\infty} P_1(s)} = \frac{\frac{1}{(r-1)!}}{\sum_{s=r-1}^{\infty} \frac{1}{s!}}. \quad (6.5)$$

- 1389 Above, we computed the (average) work factor of a random mapping F , and we compared it to the work factor for the uniform distribution U on A_2 . An important question is how “typical” the (average) work factor is for a randomly selected mapping. The most extreme case clearly is if the selected mapping f maps all $a_1 \in A_1$ onto the same image. Then the **entropy** of $f(X)$ is 0, and its work factor is 1 for any success probability α . Of course, it is extremely unlikely that a randomly selected mapping is constant. In particular, dedicated cryptographic algorithms are definitely far away from these extremal cases. The question yet remains how typical the average work factor $f(X)$ of a randomly selected mapping $f \in \mathcal{F}_{A_1, A_2}$ is. Below, we develop a formula for the variance of the work factor of $F(X)$.

- 1390 With the same strategy as in (4.173), we compute the second moment of the work factor $w_{e_r}(F(X))$.

$$\begin{aligned} E_F(w_{e_r}(F(X))^2) &= E_F\left(\left|\bigcup_{s=r}^{b_1} V_{(F)s}\right|^2\right) = E_F\left(\sum_{s_1, s_2 \geq r} |V_{(F)s_1}| \cdot |V_{(F)s_2}|\right) \\ &= \sum_{s_1, s_2 \geq r} E_F\left(\sum_{z_1 \in A_2} 1_{\{s_1\}}(|F^{-1}(z_1)|) \cdot \sum_{z_2 \in A_2} 1_{\{s_2\}}(|F^{-1}(z_2)|)\right) = \\ &= \sum_{s_1, s_2 \geq r} E_F\left(\sum_{z_1, z_2 \in A_2} 1_{\{s_1, s_2\}}(|F^{-1}(z_1)|, |F^{-1}(z_2)|)\right) = \\ &= \sum_{s_1, s_2 \geq r} \sum_{z_1, z_2 \in A_2} \text{Prob}(|F^{-1}(z_1)| = s_1, |F^{-1}(z_2)| = s_2) = \\ &= \sum_{s_1, s_2 \geq r} \sum_{z_1, z_2 \in A_2} \text{Prob}(|F^{-1}(z_1)| = s_1) \cdot \text{Prob}(|F^{-1}(z_2)| = s_2 : |F^{-1}(z_1)| = s_1). \quad (6.6) \end{aligned}$$

Similarly as in (4.173), we obtain

$$\text{Prob}(|F^{-1}(z_1)| = s_1) = \binom{b_1}{s_1} p^{s_1} (1-p)^{b_1-s_1} = \text{Prob}(Y' = s_1). \quad (6.7)$$

The conditional probabilities require a case discrimination. It is

$$\begin{aligned} \text{Prob}(|F^{-1}(z_2)| = s_2 : |F^{-1}(z_1)| = s_1) &= \binom{b_1 - s_1}{s_2} p^{s_2} (1-p^*)^{b_1-s_1-s_2} = \\ \text{Prob}(Y'_{s_1} = s_2) \quad \text{with } p^* &= \frac{1}{b_2 - 1} \text{ and } Y'_{s_1} \sim B(b_1 - s_1, p^*) \text{ if } z_1 \neq z_2 \quad (6.8) \end{aligned}$$

and

$$\text{Prob}(|F^{-1}(z_2)| = s_2 : |F^{-1}(z_1)| = s_1) = 1_{\{s_1\}}(s_2) \quad \text{if } z_1 = z_2. \quad (6.9)$$

Putting the pieces together and substituting (6.7), (6.8), and (6.9) into (6.6) yields

$$\begin{aligned}
E_F(w_{e_r}(F(X))^2) &= \\
&\sum_{s_1, s_2 \geq r} \left(\sum_{z_1, z_2 \in A_2} \text{Prob}(Y' = s_1) \cdot \text{Prob}(Y'_{s_1} = s_2) + \text{Prob}(Y' = s_1) \sum_{z \in A_2} (1_{\{s_1\}}(s_2) - \text{Prob}(Y'_{s_1} = s_2)) \right) = \\
&\sum_{s_1, s_2 \geq r} \text{Prob}(Y' = s_1) ((b_2^2 - b_2) \text{Prob}(Y'_{s_1} = s_2) + b_2 \cdot 1_{\{s_1\}}(s_2)) = \\
&\sum_{s_1=r}^{b_1} \text{Prob}(Y' = s_1) (b_2^2 (1 - b_2^{-1}) \text{Prob}(Y'_{s_1} \geq r) + b_2 1_{\{\leq b_2/2\}}(s_1)) . \tag{6.10}
\end{aligned}$$

If $s_1 > b_2/2$, then in (6.9) $s_2 < s_1$, which verifies the indicator function $1_{\{\leq b_2/2\}}(s_1)$.

We first note that $E(w_{e_r}(F(X))) = w_{e_r}(F(X))$ and

1391

$$(w_{e_r}(F(X)))^2 = b_2^2 \text{Prob}(Y' \geq r)^2 = b_2^2 \sum_{s_1 \geq r} \text{Prob}(Y' = s_1) \text{Prob}(Y' \geq r) . \tag{6.11}$$

Hence,

$$\begin{aligned}
\text{Var}_F(w_{e_r}(F(X))) &= E_F(w_{e_r}(F(X))^2) - (w_{e_r}(F(X)))^2 = \\
&\sum_{s_1=r}^{b_1} \text{Prob}(Y' = s_1) (b_2^2 [(1 - b_2^{-1}) \text{Prob}(Y'_{s_1} \geq r) - \text{Prob}(Y' \geq r)] + b_2 1_{\{\leq b_2/2\}}(s_1)) . \tag{6.12}
\end{aligned}$$

We try to simplify (6.12). Recall that $b_1 = 2^n$ and $b_2 = 2^m$ are large. Since $E(Y') = b_1/b_2 \ll b_1$, the probability $\text{Prob}(Y' > b_2/2)$ is essentially zero. Omitting the indicator function $1_{\{\leq b_2/2\}}(s_1)$ thus does not significantly change the value of (6.12). Similarly, $1 - b_2^{-1} \approx 1$. For $c \geq 1$, the Chernoff inequality implies $\text{Prob}(Y' \geq (1+c)E(Y')) \leq e^{-(c/3)(b_1/b_2)}$. This means that for cryptographic purposes (estimating work factors for reasonable success probabilities), this tail probability can be made sufficiently small for $c \in O(1)$. For $s_1 \leq (1+O(1))E(Y')$, we have $\text{Prob}(Y' \geq r) \approx \text{Prob}(Y'_{s_1} \geq r)$. Altogether, this justifies replacing the bracket $[\cdot]$ in (6.12) by 0. Putting the pieces together, (6.12) simplifies to

1392

$$\text{Var}_F(w_{e_r}(F(X))) \approx b_2 \text{Prob}(Y' \geq r) = w_{e_r}(F(X)) . \tag{6.13}$$

The standard deviation of the work factor $w_{e_r}(F(X))$ is $\approx \sqrt{w_{e_r}(F(X))}$. For cryptographically meaningful success probabilities, the standard deviation is small compared to the work factor. Hence, we may assume that a randomly selected mapping in $\mathcal{F}_{A_1, A_2}$ is “typical” with regard to the work factors.

It is $\text{Prob}_X(|F^{-1}(X)| = r) = \text{Prob}_X(|F^{-1}(X)| \geq r) - \text{Prob}_X(|F^{-1}(X)| \geq r+1)$. By (4.172) and (4.173),

1393

$$\text{Prob}_X(|F^{-1}(X)| = r) = \text{Prob}(Y = r-1) \quad \text{and similarly} \tag{6.14}$$

$$E_F(|\{y \in A_2 \mid |F^{-1}(y)| = r\}|) = E_F\left(\left|\bigcup_{s=r}^r V_{(F)s}\right|\right) = b_2 \text{Prob}(Y' = r) \tag{6.15}$$

where the random variables Y and Y' are binomially $B(b_1 - 1, \frac{1}{b_2})$ -distributed and $B(b_1, \frac{1}{b_2})$ -distributed, respectively.